

Java Dokumentation

Java Dokumentation

ArrayListAufgaben

[Artikel Warenkorb Wohngebiet](#)

Builder_Pattern

[Adresse Wohnung Wohnverwaltung](#)

Dienstag

[Kosten Program](#)

Dienstag05082025

[Primitive_Variable Schleifen Start](#)

Dienstag2

[Program](#)

Dienstags_Abend

[Program](#)

Do

[Starter](#)

Donnerstag

[Analyse Flugzeug GewinnMatrix Luftfahrzeug Program Pruefziffer Zifferbestimmung](#)

Donnerstag07082025

[FitData Trainer](#)

Freitag

[Fahrradladen Fahrradtyp](#)

Freitag05092025

[Mathedinger](#)

HaushaltskassenApp

[Eintrag FestWert IO SparzielManager Uebersicht Verwaltung](#)

HaushaltskassenAppGUI

[Eintrag](#) [EintragDialog](#) [FestWert](#) [FestWertDialog](#) [Uebersicht](#) [Verwaltung](#)

HaushaltskassenApp_Mit_Gui_Versuch2

[Eintrag](#) [FestWert](#) [IO](#) [Uebersicht](#) [VerwaltungGUI](#)

HaushaltskassenApp_zwei

[BudgetManager](#) [DataModel](#) [EintraegeTabController](#) [Eintrag](#) [FestWert](#)
[FesteWerteTabController](#) [GUIDialogHelper](#) [IO](#) [IO](#) [Schlusspruch](#) [Schlusspruch](#) [Sonstiges](#)
[SparzielManager](#) [SparzielTabController](#) [Statistik](#) [Statistik](#) [StatistikTabController](#) [Uebersicht](#)
[UebersichtTabController](#) [Verwaltung](#)

Immutable

[Bestellung](#)

InOut

[IO](#)

Logistikuebung

[DroneDelivery](#) [ExpressParcel](#) [Freight](#) [Insurable](#) [Polymorphie](#) [Sendungen](#) [Shipment](#)
[StandardParcel](#) [Trackable](#)

Logistikuebung_2

[ExpressParcel](#) [Freight](#) [Insurable](#) [Polymorphie](#) [Sendungen](#) [Shipment](#) [ShipmentCsvReader](#)
[ShipmentCsvReader2](#) [ShipmentReportWriter](#) [StandardParcel](#) [Trackable](#)
[UnknownShipmentTypeException](#)

Mittwoch

[BeispielExceptionHandler](#) [Diagnostik](#) [Start](#)

Mitwoch

[Billing](#) [Start](#)

Montag11082025

[Artikel](#) [Flugzeug](#) [Kasse](#) [Luftfahrzeug](#)

NorthwindApp

[TestNWApp](#)

NorthwindApp.GUI

[EmployeeWindow](#)

NorthwindApp.nw_db_tool

[NorthwindDB](#)

ObserverPattern

[AbstractDevice](#) [AirConditioning Device](#) [DeviceFailureException](#) [FloorHeating](#) [Heater](#)
[Humidifier](#) [LogAnalyzer](#) [Logger](#) [Main](#) [Statistics](#) [TemperatureSensor](#)

Pong

[Ball](#) [Pong](#) [Racket](#)

Pythagoras_Zeugs

[Polyeder](#)

SingeltonPattern

[AppConfig](#) [Monat](#) [Singelton](#) [SingeltonPattern](#)

Uebung_03_03_2026

[Main](#) [MitarbeiterDesMonats](#) [Person](#)

Uebung_2_03_03_2026

[Angestellte](#) [Arbeiter](#) [Gast](#) [Person](#) [Verwaltung](#)

_01_26

[Person](#) [Test](#) [Zusammenfassung](#)

_01_26_uebung

[Fahrzeug](#) [Fuhrpark](#) [Person](#) [Test](#)

_01_27

[ArraysInJava](#) [Arrayuebung](#) [Benutzer](#) [Fahrzeug](#) [Lager](#) [Prog](#) [TestLager](#) [Uebung](#)
[VergleichbarkeitVonString](#) [mehrdimensionaleArrays](#)

_01_28

[IO](#) [MinMax](#) [Teilnehmerverwaltung](#) [Temperaturdfferenzen](#) [Test](#) [UebungArrayVerlaengern](#)
[Warenkorb](#)

_01_28_selbst

[Artikel](#)

_01_29

[Artikel](#) [OnlineShop](#) [Quiz](#) [Quiz_2](#) [Referenz_Und_Value_Semantik](#) [StringConstantPool](#)
[Warenkorb](#)

_01_29_selbst

Artikel Quiz Quiz2 Referenz_und_Value_Semantik StringConstantPool Warenkorb

_01_30

Garten Kreis Punkt Quadrat Task Test Vererbung

_01_30_selbst

Form Kreis Punkt Quadrat Rechteck Test Vererbung

_02_03_2026

App Device Ladeger Ladestation MaintenanceContract NichtLadend NormalLadend Router
SchnellLadend Server Utilities Workstation Zustand

_03_03

Ladegeraet NichtLadend NormalLadend Person SchnellLadend TeeEtickett TestEtickett
TestFactoryDesign Uhrzeit Zustand

_03_03_2026

DT Ladestation NormalLadend SchnellLadend

_03_05.ap2_2021

Artikel ExpressLieferung OnlineShopApp StandardLieferung Versand Warenkorb

_03_06_logistik

App ExpressParcel Freight Insurable Polymorphie Shipment StandardParcel Trackable

_03_09

Ausnahmbehandlung IllegalAmountException Sparkonto

_03_11

Lesen Service

_03_16

MieteComparator SortierenUndSuchen Wohnung WrapperKlassen

_03_20

Filter FilterLambda IntegerTester Test Wohnung WohnungTester

_03_25

Adresse AdresseMitBuilder Main Wohnung

_03_26

AppConfig EconomyPricing ExpressPricing Monat PricingStrategie PriorityPricing
StandardPricing VersandKostenRechner

_03_27

Main Praefix PriorityPricing Shipment TrackingCodeGenerator

_03_27_2026

EconomyPricing ExpressPricing Main PricingStrategie PriorityPricing Serialization
Shipment StandardPricing

_04_03_2026

Auto Interfaces Mehrfachvererbung Uebung

_04_03_2026_Uebung

ApplPay CreditCard Konto PayPal PaymentMethod Shop

_05_03_2026

Artikel Movable Tier Versand Warenkorb

_09_03_2026

Ausnahmebehandlung DroneDelivery ExpressParcel Insurable
InvalidShipmentDataException Polymorphie Sendungen Shipment StandardParcel
Trackable

_10_03_2026

FileIO Gemischt_quadratische_Gleichungen Uebung

_10_13

Mensch Test vs

_10_13_lab

Mensch Test vs

_10_14

BMIberechnenAuswerten Klassenname Person

_10_14_lab

BMIberechnenundAuswerten Person

_10_15

IO Punkt Test

_10_15_lab

Kreis Test

_10_16

Kreis Test Uebung

_10_16_lab

Datum Kreis Test Uebung

_10_17

Datum Helper Test

_10_17_1

Datum Helper Test

_10_17_lab

Datum Datum2 Helper Test Uebung

_10_18

Datum

_10_20

Datenstrukturen MehrDimensionaleArrays Uebung Zusammenfassung

_10_20_lab

Datenstrukturen MehrDimensionaleArrays Uebung Zusammenfassung

_10_21_Arbeitsblatt

Auto Hausverwaltung Mitarbeiter Mitarbeiterverwaltung TestAuto Wohngebiet

_10_21_lab

CallByReference Uebung Uebung_2 Zusammenfassung

_10_22.arbeitsblatt

Aufgabe_3 Auto Wohngebiet

_10_22_arbeitsblatt

Aufgabe_1 Aufgabe_3 Wohngebiet

_10_23

Artikel Datenstruktur Warenkorb Wohngebiet

_10_23_lab

Artikel Datenstruktur Warenkorb Wohngebiet

_10_24

Warenkorb

_10_24_lab

Warenkorb Wohngebiet

_11_03_2026

App Kreis Lesen Punkt Service

_16_03_2026

MieteComparator SortierenUndSuchen WrapperKlassen Zimmer

_17_03_2026

Tescht

_18_03_26

AnonymeInnereKlassen TeschtLambda

_19_03_2026

Filter RentOutOfLimitException Test

_20_03_2026

DoubleTester FilterLambda FilterTescht Filter_selbst IntegerTester

PredicateStringExample RentOutOfLimitException StringTester Test WohnungTester

_23_02_2026

Pseudocode

_23_03_2026

AVGFilter DruckSensor Filter MaxFilter SampleProvider TemperaturSensor TeschtLambda

_24_03_2026

Immutable Punkt Student StudentImmutable

_25_03_2026

Auto CarData

_25_03_2026_arbeitsblatt

Buchung Verwendung

_26_02_26

Dreieck Form Garten GleichseitigesDreieck Kreis Punkt Quadrat Rechteck Task Test
Vererbung

_26_03_2026

EconomyPricing ExpressPricing IPricingStrategy Logistikdienstleister PriorityPricing
Sendung StandardPricing Strategy VersandkostenRechner

_26_03_StrategyPattern

EconomyPricing ExpressPricing Main PricingStrategy PriorityPricing StandardPricing
VersandkostenRechner

_27_02_2026

Device MaintenanceContract Router Server Test Workstation

_27_03_2026

ShipmentKlassen Testklasse TrackingCodeGenerator

_28_04_2026

Superklasse

app

Sendungen Sendungen Test Test

bd_test

TestConnect

comparator

CustomSort CustomSort DistanceComparatorAsc DistanceComparatorAsc
DistanceComparatorDesc DistanceComparatorDesc ShippingCostComparatorAsc
ShippingCostComparatorAsc ShippingCostComparatorDesc ShippingCostComparatorDesc
SortByDistance SortByDistance SortByShippingCost SortByShippingCost SortByWeight
SortByWeight WeightComparatorAsc WeightComparatorAsc WeightComparatorDesc
WeightComparatorDesc

controller

Controller Datenbank

dienstag

Kreditkarte Theke

exceptions

InvalidShipmentDataException InvalidShipmentDataException
UnknownShipmentTypeException UnknownShipmentTypeException

gui

Eintrag FestWert HauptfensterController HaushaltsApp IO JFrame_first Schlussspruch
SparzielManager Statistik window_builder

interfaces

Insurable Insurable PricingStrategy PricingStrategy Trackable Trackable

io

ShipmentCsvReader2 ShipmentCsvReader2 ShipmentReportWriter ShipmentReportWriter

main

Main

mi

teschtle

model

ExpressParcel ExpressParcel Freight Freight Kunde Shipment Shipment StandardParcel
StandardParcel

montag04082025

Konto Konto Kontoverwaltung Kontoverwaltung

simple

SecondGui ThirdGUI

test

ModelTest ViewTest

util

ShipmentCostCalculator ShipmentFactory ShipmentFactory ShipmentFilter ShipmentFilter
SortSelection SortSelection Statistic Statistic shipmentOverviewCreator
shipmentOverviewCreator

view

View

Java Dokumentation

Package: ArrayListAufgaben

Klasse: Artikel

```
package ArrayListAufgaben;
```

```
import _10_15.IO;
```

```
public class Artikel {
    private final String name;
    private double preis;
    private int menge;

    public Artikel(String name, double preis, int menge) {
        this.name = name;
        setPreis(preis);
        setMenge(menge);
    }

    @Override
    public String toString() {
        return "Artikel [name=" + name + ", preis=" + preis + ",
menge=" + menge + "]\n";
    }

    public double getPreis() {
        return preis;
    }

    public void setPreis(double preis) {
        if (preis >= 0) {
            this.preis = preis;
        } else {
            preis = IO.readDouble("Preis von Artikel: ");
            while (preis < 0) {
                preis = IO.readDouble("Preis von Artikel: ");
            }
            this.preis = preis;
        }
    }

    public int getMenge() {
        return menge;
    }

    public void setMenge(int menge) {
        if (menge > 0) {
```

```

        this.menge = menge;
    } else {
        menge = IO.readInt("Menge des Artikels: ");
        while (menge <= 0) {
            menge = IO.readInt("Menge des Artikels: ");
        }
        this.menge = menge;
    }
}

public String getName() {
    return name;
}

public double getGesamtPreis() {
    return preis * menge;
}

public void zeigeInfo() {
    System.out.println(this);
}

public static void main(String[] args) {
    Artikel a = new Artikel("College Block", 1.99, 17);
    a.zeigeInfo();
    System.out.println(a.getGesamtPreis());
}
}package _01_28;

import java.util.Objects;

public class Artikel {
    private final String name;
    private double preis;

    public Artikel(String name, double preis) {
        this.name = name;
        this.preis = preis;
    }

    public double getPreis() {
        return preis;
    }

    @Override
    public String toString() {
        return "" + name + ", " + preis;
    }
}

```

```

    public void setPreis(double preis) {
        this.preis = preis;
    }

    public String getName() {
        return name;
    }

    public String format() {
        return "%15s %8.2f€ ".formatted(this.name,this.preis);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, preis);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Artikel other = (Artikel) obj;
        return Objects.equals(name, other.name)
            && Double.doubleToLongBits(preis) ==
Double.doubleToLongBits(other.preis);
    }

```

```

}

```

Klasse: Warenkorb

```

package ArrayListAufgaben;

import _10_15.IO;
import java.util.ArrayList;

public class Warenkorb {

```

```

private ArrayList<Artikel> artikelList;

public Warenkorb(int kapazitaet) {
    if (kapazitaet > 0) {
        artikelList = new ArrayList<>(kapazitaet);
    } else {
        kapazitaet = IO.readInt("Kapazität vom Warenkorb: ");
        while (kapazitaet <= 0) {
            kapazitaet = IO.readInt("Kapazität vom Warenkorb: ");
        }
        artikelList = new ArrayList<>(kapazitaet);
    }
}

public void artikelHinzufuegen(Artikel a) {
    artikelList.add(a);
    System.out.println("Artikel '" + a.getName() + "'
hinzugefügt.");
}

public double gesamtwertBerechnen() {
    double gesamtwert = 0.0;
    for (Artikel artikel : artikelList) {
        if (artikel != null) {
            gesamtwert += artikel.getGesamtPreis();
        }
    }
    return gesamtwert;
}

public void anzeigen() {
    int i = 1;
    for (Artikel artikel : artikelList) {
        if (artikel != null) {
            System.out.println((i++) + ". " + artikel);
        }
    }
}

public static void main(String[] args) {
    Warenkorb korb = new Warenkorb(10);

    Artikel laptop = new Artikel("Gaming Laptop", 1299.99, 1);
    Artikel maus = new Artikel("Gaming Maus", 59.99, 2);
    Artikel tastatur = new Artikel("Mechanische Tastatur", 149.99,
1);
    Artikel monitor = new Artikel("27'' Monitor", 299.99, 2);

    korb.artikelHinzufuegen(laptop);

```

```

        korb.artikelHinzufuegen(maus);
        korb.artikelHinzufuegen(tastatur);
        korb.artikelHinzufuegen(monitor);

        korb.anzeigen();
        System.out.println("Gesamtwert: " + korb.gesamtwertBerechnen()
+ "€");
    }
}package _01_28_selbst;

import java.util.ArrayList;

public class Warenkorb {

    // Artikel [] items2;
    ArrayList<Artikel> items;

    public Warenkorb(int kapazitaet) {
//        this.items2= new Artikel[kapazitaet];
        this.items= new ArrayList<Artikel>();
    }

    public void artikelHinzufuegen(String name, double preis,int
stueckzahl) {
        for (int i =0; i<stueckzahl;i++){
            items.add(new Artikel(name,preis));
        }
    }

    public double berechneGesamtwert() {
        double gesamtwert=0.0;
        for(Artikel artikel:items) {
            gesamtwert +=artikel.getPreis();
        }
        return gesamtwert;
    }

    public static void main(String[] args) {

//        Artikel a1 = new Artikel("Banane",1.23);
//        Artikel a2 = new Artikel("Mango",2.49);
//
//        System.out.println(a1);
//        System.out.println(a2);
//
//        int[] anzahl= {0,0};
//        double zuZahlenderBetrag=0.0;
//
//

```

```

//      a1.setPreis(1.19);
//      System.out.println(a1.getPreis());
//      System.out.println(a1.getName());
//
//      anzahl[0]=12;
//      anzahl[1]=13;
//
//      zuZahlenderBetrag +=ermittlePreis(a1,anzahl[0]);
//      zuZahlenderBetrag +=ermittlePreis(a2,anzahl[1]);
//
//      System.out.println("Gesamtpreis für "+anzahl[0]+"
"+a1.getName() +": "+ermittlePreis(a1,anzahl[0])+"€");
//      System.out.println("Gesamtpreis für "+anzahl[1]+"
"+a2.getName() +": "+ermittlePreis(a2,anzahl[1])+"€");
//      System.out.println("Zu zahlender Betrag: "+
zuZahlenderBetrag+"€");

        Warenkorb korb =new Warenkorb(30);

        korb.artikelHinzufuegen("Collage Block",2.0, 16);
        korb.artikelHinzufuegen("Kugelschreiber", 3.0, 16);
//      korb.artikelHinzufuegen(null, 0, 0);

        System.out.println("Gesamtpreis: "+korb.berechneGesamtwert()
+"€");

    }

    public static double ermittlePreis(Artikel a,int menge) {
        double gesamtpreis=0.0;
        gesamtpreis=a.getPreis()*menge;

        return gesamtpreis;

    }

}

```

Klasse: Wohngebiet

```

package ArrayListAufgaben;

import java.util.ArrayList;

```

```

import _10_15.IO;
import _10_22.arbeitsblatt.Haus;

public class Wohngebiet {
    private ArrayList<Haus> haeuser; // Array zu ArrayList geändert

    public Wohngebiet(int n) {
        if (n >= 0) {
            haeuser = new ArrayList<>(n); // Initiale Kapazität
        } else {
            System.out.println("Anzahl darf nicht negativ sein");
            n = IO.readInt("Anzahl Häuser auf dem Gebiet: ");
            while (n < 0) {
                n = IO.readInt("Anzahl Häuser auf dem Gebiet: ");
            }
            haeuser = new ArrayList<>(n);
        }
    }

    /**
     * @param adresse
     * @param zimmer
     * @param flaeche
     * @return true, wenn das Haus hinzugefügt wurde, sonst false
     */
    public boolean baueEinHaus(String adresse, int zimmer, double
flaeche) {
        haeuser.add(new Haus(adresse, zimmer, flaeche)); // Direkt
hinzufügen
        return true;
    }

    public double berechneGesamtFlaeche() {
        double gesamtFlaeche = 0.0;
        for (Haus haus : haeuser) {
            if (haus != null) { // Null-Check bleibt der Konsistenz
halber
                gesamtFlaeche += haus.getFlaeche();
            }
        }
        return gesamtFlaeche;
    }

    public int ermittleAnzahlHaeuser() {
        return haeuser.size(); // ArrayList size() nutzen
    }

    public int ermittleAnzahlHaeuser_v2() {
        int anzahlHaeuser = 0;
        for (Haus haus : haeuser) {

```



```

        if (haus != null) {
            anzahlHaeuser++;
        }
    }
    return anzahlHaeuser;
}

public int ermittleAnzahlHaeuser_v3() {
    int anzahlHaeuser = 0;
    int i = 0;
    while (i < haeuser.size()) {
        if (haeuser.get(i) != null) {
            anzahlHaeuser++;
        }
        i++;
    }
    return anzahlHaeuser;
}

public int ermittleAnzahlHaeuser_v4() {
    int anzahlHaeuser = 0;
    for (Haus haus : haeuser) {
        if (haus != null) {
            anzahlHaeuser++;
        }
    }
    return anzahlHaeuser;
}

public boolean existiertEinHaus() {
    return !haeuser.isEmpty(); // ArrayList isEmpty() nutzen
}

public boolean existiertEinHaus_v2() {
    for (Haus haus : haeuser) {
        if (haus != null) {
            return true;
        }
    }
    return false;
}

public boolean existierPlatzFuerNeuesHaus() {
    return true; // ArrayList hat keine feste Größe
}

public boolean existierPlatzFuerNeuesHaus_v2() {
    return true; // ArrayList hat keine feste Größe
}

```

```

        public void insert(Haus h, int index) {
            if (index < 0 || index > haeuser.size()) { // Größer als size
erlaubt, da ArrayList dynamisch wächst
                System.out.println("Ungültiger Index: " + index);
                return;
            }
            if (index < haeuser.size() && haeuser.get(index) != null) {
                System.out.println("Wohngebiet an der Stelle " + index + "
ist belegt.");
            } else {
                haeuser.add(index, h); // An spezifischem Index einfügen
                System.out.println("Haus wurde erfolgreich eingetragen");
            }
        }

        public void printHaeuser() {
            System.out.println("=====");
            int nummer = 1;
            for (Haus haus : haeuser) {
                if (haus != null) {
                    System.out.printf("%d. %s%n", nummer++, haus);
                }
            }
        }

        public static void main(String[] args) {
            Wohngebiet w = new Wohngebiet(10);
            System.out.println(w.baueEinHaus("Wundt Str. 18, 10778
Berlin", 3, 95)); // true
            System.out.println(w.baueEinHaus("Ott Str. 28, 10758 Berlin",
4, 105)); // true
            System.out.println(w.baueEinHaus("Marten Str. 10, 10338
Berlin", 2, 75)); // true
            System.out.println(w.baueEinHaus("Thorben Str. 48, 10149
Berlin", 5, 130)); // true

            w.printHaeuser();

            System.out.println(w.berechneGesamtFlaeche());

            Wohngebiet w2 = new Wohngebiet(0);
            System.out.println(w2.ermittleAnzahlHaeuser_v2()); // 0
        }
    }

```

Package: Builder_Pattern

Klasse: Adresse

```
package Builder_Pattern;
```

```
public class Adresse {
```

```
    private final String name;  
    private final String vorname;  
    private final String strasse;  
    private final String hausnummer;  
    private final String plz;  
    private final String stadt;  
    private final String land;
```

```
    public Adresse(Builder b) {  
        this.name = b.name;  
        this.vorname = b.vorname;  
        this.strasse = b.strasse;  
        this.hausnummer = b.hausnummer;  
        this.plz = b.plz;  
        this.stadt = b.stadt;  
        this.land = b.land;  
    }
```

```
    @Override  
    public String toString() {  
        return "Adresse:\n" + vorname + "\n" + name + "\n" + strasse +  
" " + hausnummer  
        + "\n" + plz + " " + stadt + "\n" + land;  
    }
```

```
    public static class Builder {
```

```
        private String name;  
        private String vorname;  
        private String strasse;  
        private String hausnummer;  
        private String plz;  
        private String stadt;  
        private String land;
```

```

        public Builder name(String n) {this.name=n;return this;}
        public Builder vorname(String v) {this.vorname=v;return this;}
        public Builder strasse(String s) { this.strasse = s; return
this; }
        public Builder hausnummer(String h) {this.hausnummer=h;return
this;}
        public Builder plz(String p) { this.plz = p; return this; }
        public Builder stadt(String s) { this.stadt = s; return
this; }
        public Builder land (String l) {this.land=l;return this;}

        public Adresse build() { return new Adresse(this); }
    }

}

```

Klasse: Wohnung

```
package Builder_Pattern;
```

```

public final class Wohnung {
    private final String strasse;
    private final String plz;
    private final String stadt;
    private final double miete;
    private final int zimmer;

    private Wohnung(Builder b) {
        this.strasse = b.strasse;
        this.plz = b.plz;
        this.stadt = b.stadt;
        this.miete = b.miete;
        this.zimmer = b.zimmer;
    }

    @Override
    public String toString() {
        return "Wohnung [strasse=" + strasse + ", plz=" + plz + ",
stadt=" + stadt + ", miete=" + miete + ", zimmer="
            + zimmer + "]";
    }

    public static class Builder {
        private String strasse;

```

```

        private String plz;
        private String stadt;
        private double miete;
        private int zimmer;

        public Builder strasse(String s) {
            this.strasse = s;
            return this;
        }

        public Builder plz(String p) {
            this.plz = p;
            return this;
        }

        public Builder stadt(String s) {
            this.stadt = s;
            return this;
        }

        public Builder miete(double m) {
            this.miete = m;
            return this;
        }

        public Builder zimmer(int z) {
            this.zimmer = z;
            return this;
        }

        public Wohnung build() {
            return new Wohnung(this);
        }
    }
}

```

Klasse: Wohnverwaltung

```
package Builder_Pattern;
```

```

public class Wohnverwaltung {

    public static void main(String[] args) {

        Wohnung w = new Wohnung.Builder()
            .strasse("Hauptstrasse 1")
            .plz("12345")
            .stadt("Berlin")
            .miete(750)

```

```

        .zimmer(3)
        .build();
System.out.println(w);

Adresse a= new Adresse.Builder()
        .name("Schmitti")
        .vorname("Nick")
        .strasse("Eckstr")
        .hausnummer("10a")
        .plz("123456")
        .stadt("Niemandsstadt")
        .land("Pfffland")
        .build();
System.out.println(a);
    }
}

```

Package: Dienstag

Klasse: Kosten

```

package Dienstag;

public class Kosten {

    double berechneKosten(int onlinezeit,int anzahlSeiten) {
        double onlineK= (onlinezeit/15 + 1)*0.5;
        if (onlinezeit%15==0) {
            onlineK=onlinezeit/30.0;
        }
        double printK=0;
        double satz=0.0;
        if (anzahlSeiten<20) {
            satz=0.12;
        }
        else if (anzahlSeiten<50) {
            satz=0.1;
        }
        else if (anzahlSeiten<=100) {
            satz=0.08;
        }
        else {
            satz=0.05;
        }
        printK=anzahlSeiten*satz;

        return onlineK+printK;
    }
}

```

```

    }

}

Klasse: Program
package Dienstag;

public class Program {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Kosten k1= new Kosten();
        // k1.berrechneKosten(31, 17);
        System.out.println("Die Kosten betragen:
"+k1.berrechneKosten(30, 101)+" €");
        System.out.println("Die Kosten betragen:
"+k1.berrechneKosten(15, 100)+" €");
        System.out.println("Die Kosten betragen:
"+k1.berrechneKosten(30, 1)+" €");
        System.out.println("Die Kosten betragen:
"+k1.berrechneKosten(30, 50)+" €");

    }

}

```

Package: Dienstag05082025

Klasse: Primitive_Variable

```

package Dienstag05082025;

public class Primitive_Variable {

    void zeigeVariable() {

        //ganze Zahlen
        byte b1=15;
        /*b1=234;
        b1=-210;
        Wertebereich wurde über bzw unterschritten
        */
        short s1= 30000;
        //s1=40000;

        int i1 = 2000000000;
        //i1=3000000000;

        long l1 = 200000L;
    }
}

```

```

        //grosses oder kleines "L"
        l1= 2000l;

        System.out.println("byte: "+ b1 + "\nshort: "+ s1+ "\nint:
"+i1+ "\nlong: "+l1 );

        //Gleitkommazahlen

        float f1 = 0.123456789f;//f hinten anstellen
        System.out.println("float: "+f1);
        //Datentyp float :Literal mit f oder F ....wird auf die achten
Stelle gerundet

        double d1 = 0.12345678901234567;
        System.out.println("double: " + d1 );

        // Einzelzeichen (Buchstaben, Ziffer, Sonderzeichen)
        char c1='G';
        c1='0';
        c1='$';
        System.out.println("char: "+c1);

        // Wahrheitswert

        boolean bol=true;
        //bol=false;

        System.out.println("boolean: "+bol);
    }

    void rechnenmitVariable_Operatoren() {
        int wert =-15;
        wert=wert+5;
        int erg = wert % 4;
        System.out.println("Ergebnis: "+erg);

        //Integer-Division

        int erg2 = 5/2;

        //Modulo-Operator zeigt an was der Rest ist
        int erg4 = 5%2;
        System.out.println("Ergebnis2: "+erg2 +" Rest: "+erg4);

        //implizite Konvertierung
        //wird vom Compiler ausgeführt

```



```

        float erg3 = 5.0f/2;
        float erg5 =5/2;
        System.out.println("Ergebnis3: "+erg3 +" statt: "+erg5);
    }

    void zeigeVergleichsOperatoren() {
        //jeder Vergleichsoperator erzeugt einen logischen Wert(true
oder false)

        boolean erg6= 5>4;
        System.out.println("Ergebnis erg6: "+erg6);

        boolean erg7= 6!=7;
        System.out.println("Ergebnis erg7: "+erg7);

        boolean erg8= 6==7;
        System.out.println("Ergebnis erg8: "+erg8);

        int erg9=0;
        if (5>4) {
            erg9=5+4;
        }
        else {
            erg9=5-4;
        }
        System.out.println("Ergebnis erg9: "+erg9);

        int erg10=0;
        if (5<4) {
            erg10=5+4;
        }
        else {
            erg10=5-4;
        }
        System.out.println("Ergebnis erg10: "+erg10);
    }

    void zeigeCompoundOperatoren() {

        //5 Operatoren
        int wert =7;
        wert+=4;
        System.out.println("Wert: "+wert);

        double d1 =-5.67;

```

```

    d1 *=3.5;
    System.out.println("d1: " +d1);

    int ergebnis =12;
    ergebnis %=6;
    System.out.println("Ergebnis: "+ergebnis);

}

void zeigePreundPost() {

    //Inkrement ++ Pre
    int zahl =6;
    ++zahl;
    System.out.println("Zahl pre: "+zahl);

    //Inkrement ++ Post
    zahl = 15;
    zahl++;
    System.out.println("Zahl post: "+zahl);

    //entscheidende
    zahl=6;
    int ergebnis3 = zahl++;
    System.out.println("Ergebnis3: "+ergebnis3);
    zahl=6;
    int ergebnis4=++zahl;
    System.out.println("Ergebnis4: "+ergebnis4);
    zahl=6;
    int ergebnis2 = zahl++ + (++zahl);
    System.out.println("Ergebnis2: "+ergebnis2);

    //Dekrement -- Pre
    int wert=15;
    --wert;
    System.out.println("Wert pre: "+wert);

    //Dekrement -- Post
    int wert2 = 21;
    wert2--;
    System.out.println("Wert2 post: "+wert2);

    //entscheidende
    int zahl2=15;
    int ergebnis5 =zahl2--;
    System.out.println("Ergebnis5: "+ergebnis5);

    zahl2=15;

```

```

        int ergebnis6=--zahl2;
        System.out.println("Ergebnis6: "+ergebnis6);

        zahl2=15;
        int ergebnis7=zahl2-- - (--zahl2);
        System.out.println("Ergebnis7: "+ergebnis7);
    }

```

```

}

```

Klasse: Schleifen

```

package Dienstag05082025;

```

```

import java.util.Scanner;

```

```

public class Schleifen {

```

```

    void zeigeWhile() {
        //kopfgesteuerte Schleife

```

```

        int k=0;
        while (k<=10) {
            System.out.println(k);
            k=k+1;

```

```

            if (k==5) {
                break;//bricht die schleife ab sobald k 5 ist
            }
        }

```

```

    }

```

```

    void zeigeZaehlschleife() {
        for(int i =0;i<10;i++) {
            if (i==5) {
                continue;    //die 5 wird übersprungen
            }
            System.out.println(i);

```

```

        }
    }

```

```

    void zeigeDoWhile() {

```

```

        int i=0;
        do {
            System.out.println(i);
            i=i+1;
        }
        while (i<=10);
    }

    void zeigeFussgesteuerteSchleife() {
        Scanner input =new Scanner(System.in);
        String text ="";
        do {
            System.out.println("Bitte geben Sie einen Text ein: ");
            text=input.nextLine();
            System.out.println("Textlänge: "+text.length());

            if (text.length()==0 || text.equals("q") ) {
                break;
            }
            else {
                System.out.println("Eingegebener Text: "+text);
            }

        }
        while(true);    //Semikolon nicht vergessen!
        input.close();
    }

    void zeigeFelder() {
/*        //eindimensionale Felder
        double [] noten= {2.5,1.7,1.5,2.0,1.5};

        //Ausgabe der Noten
        //Feldindex 0-3

        for(int i=0;i<noten.length;i++) {
            System.out.println(noten[i]);
        }
        int[] feld=new int[5];
        feld[0]=20;
        feld[4]=10;

        for (int i =0; i<feld.length;i++) {
            System.out.println(feld[i]);
        }

        //anlegen eines zweidimensionalen Feldes

```

```

//wie bei einer Matrix erst zeilen dann spalten
//feld für 10 teilnehmer jeder teilnehmer 5 ergebnisse */

double feld2D[][]= new double [10][5];

//teilnehmer 1 mit 3 ergebnissen
feld2D[0][0]= 1.7;
feld2D[0][1]= 2.0;
feld2D[0][2]= 1.5;
//teilnehmer 3 mit den ersten 3 ergebnissen
feld2D[2][0]= 1.0;
feld2D[2][1]= 2.0;
feld2D[2][2]= 2.5;

//Ausgabe der Notenaufstellung
System.out.println("Notenaufstellung");

for (int i =0;i<feld2D.length;i++) {    //i=zeilenindex
    System.out.println("Teilnehmer "+(i+1)+":");
    //System.out.println("\n");
    for (int j=0 ;j<feld2D[i].length;j++) { //j=Spaltenindex
        System.out.print(feld2D[i][j]+" ", );
    }
    System.out.println("\n");
}

}

void zeigeKonditionalOperator() {
    //wenn im if und else die gleiche zuweisung hat
    int a=3, b=5, max=0;
    //Konditionaloperator

    max=(a>b) ? a:b;
    System.out.println("der größte Wert ist: "+max);

    /*
    gleichbedeutend mit:
    if(a>b){
        max=a;
    }
    else{
        max=b;
    }
    */
}

void zeigeSwitchCase(int selektor) {

```

```

        switch (selektor) {
        case 0: System.out.println("rot");
            break;
        case 1: System.out.println("grün");
            break;
        case 2: System.out.println("blau");
            break;
        default: System.out.println("Stellung nicht belegt");
        }
    }
}

```

```

}

```

Klasse: Start

```

package Dienstag05082025;

```

```

public class Start {

    public static void main(String[] args) {

        Primitive_Variable pv = new Primitive_Variable();
        //pv.zeigeVariable();

        // pv.rechenmitVariable_Operatoren();

        // pv.zeigeVergleichsOperatoren();

        Schleifen sc= new Schleifen();

        // sc.zeigeWhile();
        // sc.zeigeZaehlschleife();
        // sc.zeigeDoWhile();
        // sc.zeigeFussgesteuerteSchleife();
        // sc.zeigeFelder();
        // pv.zeigeCompoundOperatoren();
        pv.zeigePreundPost();
        // sc.zeigeKonditionalOperator();
        /* sc.zeigeSwitchCase(0);
        sc.zeigeSwitchCase(1);
        sc.zeigeSwitchCase(2);
        sc.zeigeSwitchCase(5);*/

    }
}

```

```
}
```

Package: Dienstag2

Klasse: Program

```
package Dienstag2;
```

```
public class Program {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
        Umrechnung u1= new Umrechnung();  
        u1.showTime(5000);  
        u1.showTime(3600);  
        u1.showTime(100);  
        u1.showTime(86400);  
        u1.showTime(3857);  
  
        u1.rechneSekunden(3600);  
        u1.rechneSekunden(3840);  
        u1.rechneSekunden(86400);  
    }
```

```
}
```

Package: Dienstags_Abend

Klasse: Program

```
package Dienstags_Abend;
```

```
public class Program {
```

```
    public static void main(String[] args) {  
        int a=13;  
        int b=91;  
        int c=82;  
  
        int ergebnis1=0;  
        int ergebnis2=0;  
        int ergebnis3 = 0;  
        int ergebnis4=0;  
        int ergebnis5=0;  
  
        Rechnung re = new Rechnung();  
  
        // re.berechneAbstand(-187,-56 );  
        re.printeErgebnis(re.berechneAbstand(-187,-56 ));
```

```

        re.setName("Wurstwarenlieferung");
        re.setDatum("5.August 2025");
        re.setRechnungsnummer(123456);

        System.out.printf("Die Rechnung mit der Nummer: %s war am: %s
und heisst: %s",re.getRechnungsnummer(),re.getDatum(),re.getName());
        int limit=5;

        for (int i=0;i<limit+1;i+=2) {
            print(i);
        }
        System.out.print(limit+1);
//-----

        ergebnis1=a+b;    //104
        // int ergebnis1=a+b; das ist die Schnellversion    (unbrauchbar
in Verzweigungen)

        ergebnis2=b-c;    //9

        ergebnis3= a*b;    //1183

        ergebnis4=b/a;    //7

        ergebnis5= c%a;    //4

        System.out.println(ergebnis1);
        System.out.println(ergebnis2);
        System.out.println(ergebnis3);
        System.out.println(ergebnis4);
        System.out.println(ergebnis5);

        //Ausgabe
        System.out.println(ergebnis1+",\n"+ergebnis2);
        System.out.print(ergebnis1+",\t"+ergebnis2);
        System.out.printf("Ergebnis Addition: %s \nErgebnis Modulo: %s
\tErgebnis Multiplikation: %s",ergebnis1,ergebnis5,ergebnis3);
        System.out.printf("Der Abstand der Zahlen %s und %s ist:
%s",c,ergebnis3,re.berechneAbstand(c,ergebnis3));

    }

    static void print(int temp) {
        System.out.printf("\n%s ist kleiner als  ",temp);
    }

```



```
}
```

Package: Do

Klasse: Starter

```
package Do;
```

```
public class Starter {
```

```
    public static void main(String[] args) {  
        /*double a=17;  
        double b=5;  
        double ergebnis=a/b;  
        System.out.println("Ergebnis: "+a+"/"+b+" = "+ergebnis);  
        */  
        boolean x=false;  
        boolean y=false;  
  
        System.out.println(!x^!y); //XOR
```

```
    }
```

```
}
```

Package: Donnerstag

Klasse: Analyse

```
package Donnerstag;
```

```
public class Analyse {
```

```
    public static void main(String[] args) {  
        GewinnMatrix matrix = new GewinnMatrix();  
        matrix.ausgabe();  
        double[] minMax = matrix.findeMinMax();  
        System.out.printf("\n\tMinimum: %8.2f €\n\tMaximum: %8.2f €\n", minMax[0], minMax[1]);  
        double[] durchschnitt = matrix.durchschnittAbteilungen();  
        System.out.printf(" Durchschnitt:\n");  
        for (int i = 0; i < durchschnitt.length; i++) {  
            System.out.printf("%15s: %8.2f €\n",  
matrix.abteilungen[i], durchschnitt[i]);  
        }  
    }
```

```
}
```

Klasse: Flugzeug

```
package Donnerstag;
```

```
public class Flugzeug extends Luftfahrzeug {
    private double spannweite;
    private int motorenAnzahl;

    public Flugzeug() {
        this.spannweite=0.0;
        this.motorenAnzahl=0;
    }

    public Flugzeug(String bezeichnung,double gewicht,int
    baujahr,double spannweite,int motorenAnzahl) {
        /* this.bezeichnung=super.bezeichnung;
        this.gewicht=super.gewicht;
        this.baujahr=super.baujahr;*/
        // super(bezeichnung);
        this.spannweite=spannweite;
        this.motorenAnzahl=motorenAnzahl;
    }

    public double getSpannweite() {
        return spannweite;
    }

    public void setSpannweite(double spannweite) {
        this.spannweite = spannweite;
    }

    public int getMotorenAnzahl() {
        return motorenAnzahl;
    }

    public void setMotorenAnzahl(int motorenAnzahl) {
        this.motorenAnzahl = motorenAnzahl;
    }

    @Override
    public String getDate() {
        String daten;
        daten= super.getDate()+"\nSpannweite: "+this.spannweite+"m\
nAnzahl der Motoren: "+this.motorenAnzahl;
        return daten;
    }
}
```

```
    }  
}
```

Klasse: GewinnMatrix

```
package Donnerstag;
```

```
public class GewinnMatrix {  
    double[][] gewinne = new double[6][12];  
    String[] abteilungen = {"Vertrieb", "Produktion", "IT", "HR",  
"Marketing", "Finanzen"};  
    String[] monate = {"Jan", "Feb", "Mrz", "Apr", "Mai", "Jun",  
"Jul", "Aug", "Sep", "Okt", "Nov", "Dez"};  
  
    // Konstruktor: Initialisiert fiktive Werte  
    public GewinnMatrix() {  
        for (int i = 0; i < 6; i++) {  
            for (int j = 0; j < 12; j++) {  
                gewinne[i][j] = Math.round((Math.random() * 20000 -  
10000) * 100.0) / 100.0; // Werte von -10.000 bis +10.000  
            }  
        }  
    }  
  
    // Maximum und Minimum finden  
    public double[] findeMinMax() {  
        double min = Double.MAX_VALUE;  
        double max = Double.MIN_VALUE;  
        for (double[] abt : gewinne) {  
            for (double wert : abt) {  
                if (wert < min) min = wert;  
                if (wert > max) max = wert;  
            }  
        }  
        return new double[]{min, max};  
    }  
  
    // Durchschnitt pro Abteilung  
    public double[] durchschnittAbteilungen() {  
        double[] durchschnitt = new double[6];  
        for (int i = 0; i < 6; i++) {  
            double summe = 0;  
            for (int j = 0; j < 12; j++) {  
                summe += gewinne[i][j];  
            }  
            durchschnitt[i] = summe / 12;  
        }  
        return durchschnitt;  
    }  
  
    // Ausgabe
```

```

public void ausgabe() {
    System.out.printf("%15s", "");
    for (String monat : monate) System.out.printf("%10s", monat);
    System.out.println();
    for (int i = 0; i < 6; i++) {
        System.out.printf("%15s", abteilungen[i]);
        for (int j = 0; j < 12; j++) {
            System.out.printf("%10.2f", gewinne[i][j]);
        }
        System.out.println();
    }
}
}
}

```

Klasse: Luftfahrzeug

```
package Donnerstag;
```

```

public class Luftfahrzeug {

    protected String bezeichnung;
    protected double gewicht;
    protected int baujahr;

    Luftfahrzeug () {
        this.bezeichnung="unbekannt";
        this.gewicht=0.0;
        this.baujahr=0;
    }

    Luftfahrzeug(String bezeichnung,double gewicht,int baujahr){
        this.bezeichnung=bezeichnung;
        this.gewicht=gewicht;
        this.baujahr=baujahr;
    }

    public String getBezeichnung() {
        return bezeichnung;
    }
    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }
    public double getGewicht() {
        return gewicht;
    }
    public void setGewicht(double gewicht) {
        this.gewicht = gewicht;
    }
}

```

```

    public int getBaujahr() {
        return baujahr;
    }
    public void setBaujahr(int baujahr) {
        this.baujahr = baujahr;
    }

    public String getDate() {
        return "Name: "+bezeichnung+"\nGewicht:
"+Double.toString(gewicht)+"kg\nBaujahr: "+Integer.toString(baujahr) ;
    }

}

```

Klasse: Program

```
package Donnerstag;
```

```

public class Program {

    public static void main(String[] args) {
        Luftfahrzeug lf1 =new Luftfahrzeug();
        Luftfahrzeug lf2 =new Luftfahrzeug("Boing707",98102,2001);

        Flugzeug f1= new Flugzeug();
        // Flugzeug f2= new Flugzeug("A505",45333,2012,56.7,8);

        f1.setBezeichnung("A390");
        f1.setGewicht(90000);
        f1.setBaujahr(2001);
        f1.setSpannweite(45.6);
        f1.setMotorenAnzahl(4);

        lf1.setBezeichnung("A311");
        lf1.setGewicht(71840);
        lf1.setBaujahr(1995);

        System.out.println(lf1.getDate());
        System.out.println(lf2.getDate());
        System.out.println("-----");
        // System.out.println(f1.getDate()+"\nSpannweite:

```

```

"+f1.getSpannweite()+" m\nAnzahl der Motoren:
"+f1.getMotorenAnzahl());
    // System.out.println(f2.getDaten()+"\nSpannweite:
"+f2.getSpannweite()+" m\nAnzahl der Motoren:
"+f2.getMotorenAnzahl());
    System.out.println(f1.getDaten());
    // System.out.println(f2.getDaten());
}

}

```

Klasse: Pruefziffer

```
package Donnerstag;
```

```

public class Pruefziffer {

    int berrechnenPruefziffer(String teilcode) {
        char[] ziffern = new char[12];
        int ziffer=0;
        int gewichteteSumme=0;
        int temp=0;

        for (int index=0;index<12;index++) {
            ziffern[index]=teilcode.charAt(index);
        }

        for (int j=0;j<ziffern.length;j++) {
            if (j%2==0) {
                gewichteteSumme=gewichteteSumme +
Character.getNumericValue(ziffern[j]);
            }
            else {
                gewichteteSumme=gewichteteSumme +
Character.getNumericValue(ziffern[j])*3;
            }
        }
        // System.out.println(gewichteteSumme);
        temp=gewichteteSumme%10;
        if (temp<6) {
            ziffer=temp;
        }
        else {
            ziffer=10-temp;
        }

        return ziffer;
    }
}

```

```
}
```

Klasse: Zifferbestimmung

```
package Donnerstag;
```

```
public class Zifferbestimmung {  
  
    public static void main(String[] args) {  
        Pruefziffer pz = new Pruefziffer();  
        System.out.println("Die 13. Ziffer(Prüfziffer) ist:  
"+pz.berrechenPruefziffer("401613810060"));  
        System.out.println("Die 13. Ziffer(Prüfziffer) ist:  
"+pz.berrechenPruefziffer("401613810063"));  
    }  
  
}
```

Package: Donnerstag07082025

Klasse: FitData

```
package Donnerstag07082025;
```

```
public class FitData {  
  
    private float gewicht;  
    private float groesse;  
    private String name;  
  
    //Standardkonstruktor, wenn nicht ergänzt vom Compiler erzeugt  
    public FitData() {  
  
    }  
  
    //parametrisierter Konstruktor als Ersatz für die Set-Methoden  
    public FitData(float gewicht, float groesse, String name) {  
        this.gewicht=gewicht;  
        this.name=name;  
        this.groesse=groesse;  
    }  
  
    public float getGewicht() {  
        return gewicht;  
    }  
    public void setGewicht(float gewicht) {  
        this.gewicht = gewicht;  
    }  
    public float getGroesse() {
```

```

        return groesse;
    }
    public void setGroesse(float groesse) {
        this.groesse = groesse;
    }
    public String getName() {
        return this.name;
    }
    public void setName(String name) {
        this.name = name;
    }

    public float getBMI() {
        float bmi=0.0f;
        bmi=gewicht/(groesse*groesse);
        return bmi;
    }

    public void bmiAuswert() {
        if (getBMI()<19.0) {
            System.out.println("Sie sollten mehr essen! Falls Sie
Esstörungen haben, kontaktieren Sie einen Arzt!");
        }
        else if (getBMI()>25.0) {
            System.out.println("Essen Sie gesünder!");
        }
        else {
            System.out.println("Ihr BMI ist in Ordnung!");
        }
    }

}
}

```

Klasse: Trainer

```
package Donnerstag07082025;
```

```

public class Trainer {

    public static void main(String[] args) {
        FitData fd=new FitData();
        //parametrisierter Konstruktor
        FitData fd2 = new FitData(88.0f,1.66f,"Luigi");

        fd.setGewicht(88.0f);
    }
}

```



```

        fd.setGroesse(1.89f);
        fd.setName("Guschti");

        System.out.println(fd.getName());
        fd.bmiAuswert();

        System.out.println(fd2.getName());
        fd2.bmiAuswert();
    }
}

```

Package: Freitag

Klasse: Fahrradladen

```

package Freitag;

public class Fahrradladen {

    public static void main(String[] args) {
        Fahrradtyp f1= new Fahrradtyp("City-Bike",50.0,true,2100.0);

        System.out.println(f1.gibDaten());
    }
}

```

Klasse: Fahrradtyp

```

package Freitag;

public class Fahrradtyp {

    private String bezeichnung;
    private double rahmenhöhe;
    private boolean elektrischerAntrieb;
    private double einkaufspreis;

    Fahrradtyp(String bezeichnung,double rahmenhöhe,boolean
elektrischerAntrieb,double einkaufspreis ){
        this.bezeichnung=bezeichnung;
        this.rahmenhöhe=rahmenhöhe;
        this.elektrischerAntrieb=elektrischerAntrieb;
        this.einkaufspreis=einkaufspreis;
    }
}

```

```

    public String getBezeichnung() {
        return bezeichnung;
    }
    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }
    public double getRahmenhöhe() {
        return rahmenhöhe;
    }
    public void setRahmenhöhe(double rahmenhöhe) {
        this.rahmenhöhe = rahmenhöhe;
    }
    public boolean isElekrischerAntrieb() {
        return elektrischerAntrieb;
    }
    public void setElekrischerAntrieb(boolean elektrischerAntrieb) {
        this.elekrischerAntrieb = elektrischerAntrieb;
    }
    public double getEinkaufspreis() {
        return einkaufspreis;
    }
    public void setEinkaufspreis(double einkaufspreis) {
        this.einkaufspreis = einkaufspreis;
    }

    public String gibDaten() {
        String daten="";
        daten= "Bezeichnung: "+bezeichnung+
            "\nRahmenhöhe: "+rahmenhöhe +
            "\nelekrischer Antrieb: "+elekrischerAntrieb+
            "\nEinkaufspreis: "+einkaufspreis+ " €";

        return daten;
    }
}

```

Package: Freitag05092025

Klasse: Mathedinger

```
package Freitag05092025;
```

```
import java.util.*;
```

```
public class Mathedinger {
```

```

public static void main(String[] args) {
    // TODO Auto-generated method stub

    /* double flaeche=0.0;
       double umfang =0.0;
       double radius=0.0;

Scanner sc =new Scanner(System.in)  ;

    System.out.print("Geben Sie den Radius ein: ");
    radius=sc.nextDouble();
    sc.close();

    umfang=Math.PI *2*radius;
    flaeche=Math.PI *radius*radius;

    System.out.println("Die Fläche des Kreises beträgt:
"+flaeche);
    System.out.println("Der Umfang des Kreises beträgt: "+umfang);

    */

    /*Scanner    sc =new Scanner(System.in)  ;
    double zinssatz=0.0;
    double verdopplungszeit=0;

    System.out.print("Geben Sie den Zinssatz in Prozent an: ");
    zinssatz=sc.nextDouble();
    sc.close();

    verdopplungszeit=Math.log(2.0)/Math.log(zinssatz/100.0+1.0);
    System.out.println("Nach "+verdopplungszeit+" Jahren hat sich
ihr Kapital verdoppelt");
    */

Scanner sc =new Scanner(System.in)  ;

    /*
    int zahl1=0;
    int zahl2=0;
    int ergebnis=0;
    int ergebnis2=0;
    int kgv=0;

    System.out.print("Geben sie die erste Zahl ein: ");
    zahl1 = sc.nextInt();
    System.out.print("Geben sie die zweite Zahl ein: ");
    zahl2 = sc.nextInt();

```

```

        ergebnis = berechneGcd(zahl1,zahl2);
        System.out.println("Der größte gemeinsame Teiler ist:
"+ergebnis);
        ergebnis2 = berechneGcdUeberDifferenz(zahl1,zahl2);
        System.out.println("Der größte gemeinsame Teiler ist:
"+ergebnis2+" über die Differenz ermittelt");
        kgv=zahl1*zahl2/ergebnis;
        System.out.println("Das kleinste gemeinsame Vielfache ist:
"+kgv);
    */

```

```

    /* long zahl=0;
    System.out.print("Geben Sie die Zahl ein: ");
    zahl= sc.nextLong();
    int count=1;
    long max=0;
    while (zahl!=1) {

        if (zahl%2==0) {
            zahl=zahl/2;
        }else {
            zahl=3*zahl+1;
        }
        System.out.println(zahl);
        count++;
        max= (zahl > max) ? zahl : max;
    }
    System.out.println("Anzahl der Durchläufe: "+count);
    System.out.println("Die größte Zahl beim Durchlauf ist:
"+max);
    */

```

```

    int zaehler =0;
    int nenner =1;
    int zaehlergekuerzt=0;
    int nennergekuerzt =1;
    int max=0;
    int stellen=0;

    System.out.print("Geben Sie den Zähler ein: ");
    zaehler = sc.nextInt();
    System.out.print("Geben Sie den Nenner ein: ");
    nenner= sc. nextInt();

    zaehlergekuerzt=zaehler/berrechneGcd(zaehler,nenner);
    nennergekuerzt=nenner/berrechneGcd(zaehler,nenner);
    max=(zaehlergekuerzt>nennergekuerzt)?
    zaehlergekuerzt:nennergekuerzt;
    stellen =(int)(Math.log(max)/Math.log(10));//ln ist

```

```

Math.log!!!!
    System.out.println("Der gekürzte Bruch ist:\n");
    //System.out.println(stellen);
    //System.out.println(max);
    System.out.println(zaehlergekuerzt);

    for(int i =1;i<=(stellen);i++) {
        System.out.print("-");
    }
    System.out.print("-\n");
    System.out.println(nennergekuerzt);
    if (nenner==nennergekuerzt) {
        System.out.println("\nHinweis: Man konnte nicht kürzen!");
    } else {
        System.out.println("\nHinweis: gekürzt wurde mit:
"+berrechneGcd(zaehler,nenner));
    }

}

private static int berrechneGcd(int a, int b) {

    int temp=0;
    temp=a%b;

    while (temp!=0) {
        a=b;
        b=temp;
        temp=a%b;
    }
    return b;
}

private static int berrechneGcdUeberDifferenz(int a, int b) {

    while(b!=0) {
        if (a>b) {
            a -=b;
        }
        else {
            b -=a;
        }
    }
    return a;
}

```

```
}
```

Package: HaushaltskassenApp

Klasse: Eintrag

```
package HaushaltskassenApp;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Objects;

public class Eintrag {

    public String bezeichnung;
    public double betrag;
    public boolean istEinnahme;
    public LocalDate datum;
    public String kategorie;

    public Eintrag(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie) {
        super();
        this.bezeichnung = bezeichnung;
        this.betrag = betrag;
        this.istEinnahme = istEinnahme;
        this.datum = datum;
        this.kategorie = kategorie;
    }

    public String getBezeichnung() {
        return bezeichnung;
    }

    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }

    public double getBetrag() {
        return betrag;
    }

    public void setBetrag(double betrag) {
        this.betrag = betrag;
    }
}
```

```

    public boolean istEinnahme() {
        return istEinnahme;
    }

    public void setIstEinnahme(boolean istEinnahme) {
        this.istEinnahme = istEinnahme;
    }

    public LocalDate getDatum() {
        return datum;
    }

    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }

    public String getKategorie() {
        return kategorie;
    }

    public void setKategorie(String kategorie) {
        this.kategorie = kategorie;
    }

    @Override
    public int hashCode() {
        return Objects.hash(betrag, bezeichnung, datum, istEinnahme,
kategorie);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Eintrag other = (Eintrag) obj;
        return Double.doubleToLongBits(betrag) ==
Double.doubleToLongBits(other.betrag)
&& Objects.equals(bezeichnung, other.bezeichnung) &&
Objects.equals(datum, other.datum)
&& istEinnahme == other.istEinnahme &&
Objects.equals(kategorie, other.kategorie);
    }

    @Override
    public String toString() {

```

```

        return "Eintrag [bezeichnung=" + bezeichnung + ", betrag=" +
betrag + ", istEinnahme=" + istEinnahme
        + ", datum=" + datum + ", kategorie=" + kategorie +
        "]" +
    }
}

```

```

    public static void loescheEintrag(ArrayList<Eintrag> ae,int index)
{
    if (index >= 0 && index < ae.size()) {
        ae.remove(index);
    } else {
        System.out.println("Ungültiger Index!");
    }
}

    public static Eintrag aendereEintrag() {
        return new Eintrag(
            IO.readString("Bezeichnung: "),
            IO.readDouble("Betrag: "),
            IO.readBoolean("Einnahme?(j/n): "),
            IO.readDate("Datum: "),
            IO.readString("Kategorie: ")
        );
    }

    public static void fuegeEintragHinzu(ArrayList<Eintrag>
ae ,Eintrag e) {
        ae.add(e);
    }
}

```

Klasse: FestWert

```

package HaushaltskassenApp;

import java.time.LocalDate;

public class FestWert extends Eintrag{
    public int periode;

    public FestWert(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie,
        int periode) {
        super(bezeichnung, betrag, istEinnahme, datum, kategorie);
        this.periode = periode;
    }

    public int getPeriode() {
        return periode;
    }
}

```



```

    }

    public void setPeriode(int periode) {
        this.periode = periode;
    }

    @Override
    public String toString() {
        return "FestWert [periode=" + periode + ", bezeichnung=" +
bezeichnung + ", betrag=" + betrag + ", istEinnahme="
        + istEinnahme + ", datum=" + datum + ", kategorie=" +
kategorie + "]\n";
    }

}

```

Klasse: IO

```

package HaushaltskassenApp;

import java.time.LocalDate;
import java.util.Scanner;

public class IO {

    public static int readInt(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextInt();
    }

    public static double readDouble(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextDouble();
    }

    public static String readString(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextLine();
    }

    public static boolean readBoolean(String prompt) {
        Scanner scan = new Scanner(System.in);
        while (true) {

```

```

        System.out.print(prompt);
        String input = scan.nextLine().trim().toLowerCase();
        if (input.equals("j") || input.equals("ja") ||
input.equals("y") || input.equals("yes")) {
            return true;
        } else if (input.equals("n") || input.equals("nein") ||
input.equals("no")) {
            return false;
        } else {
            System.out.println("Ungültige Eingabe! Bitte 'j',
'ja', 'n' oder 'nein' eingeben.");
        }
    }
}

public static LocalDate readDate(String prompt) {
    System.out.println(prompt);
    try {
        return LocalDate.of(
            IO.readInt("Jahr: "),
            IO.readInt("Monat: "),
            IO.readInt("Tag: ")
        );
    } catch (Exception e) {
        System.out.println("Ungültiges Datum! Heutiges Datum wird
verwendet!");
        return LocalDate.now();
    }
}
}

```

Klasse: SparzielManager

```
package HaushaltskassenApp;
```

```

import java.io.*;
import java.time.*;
import java.util.*;
import java.util.Locale; // <-- WICHTIG!

public class SparzielManager {
    private static final List<Sparziel> ziele = new ArrayList<>();
    private static final String DATEI = "sparziel.txt";

    // --- SPARRATE AUS FESTEN WERTEN LESEN ---
    public static double getSparrateAusFesteWerte(ArrayList<FestWert>
festeWerte) {
        return festeWerte.stream()
            .filter(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie))

```

```

        .mapToDouble(f -> f.betrag)
        .sum();
    }

    public static class Sparziel {
        String name;
        double zielbetrag;
        LocalDate zielDatum;
        double aktuell;
        int priorit et;

        public Sparziel(String name, double zielbetrag, LocalDate
zielDatum, double aktuell, int priorit et) {
            this.name = name;
            this.zielbetrag = zielbetrag;
            this.zielDatum = zielDatum;
            this.aktuell = aktuell;
            this.priorit et = priorit et;
        }

        public double fehlend() { return Math.max(0, zielbetrag -
aktuell); }
        public double prozent() { return zielbetrag > 0 ? (aktuell /
zielbetrag) * 100 : 0; }

        @Override
        public String toString() {
            int balken = (int) (prozent() / 10);
            return String.format("P%d: %-15s | %6.0f / %6.0f € | [%s
%s] %3.0f%% | %6.0f € fehlen",
                priorit et, name, aktuell, zielbetrag,
                "█".repeat(balken), " ".repeat(10 - balken),
                prozent(), fehlend());
        }
    }

    // --- LADEN ---
    public static void ladeSparziele() {
        ziele.clear();
        File file = new File(DATEI);
        if (!file.exists()) return;

        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) continue;
                String[] p = line.split("#", 5);
                if (p.length == 5) {

```

```

        ziele.add(new Sparziel(
            p[0].trim(),
            Double.parseDouble(p[1].trim().replace(',', '.')),
            LocalDate.parse(p[2].trim()),
            Double.parseDouble(p[3].trim().replace(',', '.')),
            Integer.parseInt(p[4].trim())
        ));
    }
}
System.out.println("Sparziele geladen: " + ziele.size());
} catch (Exception e) {
    System.err.println("Fehler beim Laden: " +
e.getMessage());
}

// --- SPEICHERN ---
public static void speichereSparziele() {
    try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
    {
        pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
        ziele.stream()
            .sorted(Comparator.comparingInt(s -> s.prioritaet))
            .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d%n",
                sz.name, sz.zielbetrag, sz.zielDatum, sz.aktuell,
                sz.prioritaet));
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern: " +
e.getMessage());
    }
}

// --- NEU ---
public static void neuesSparziel() {
    String name = IO.readString("Name: ");
    double betrag = IO.readDouble("Zielbetrag: ");
    LocalDate datum = IO.readDate("Datum: ");
    int prio = IO.readInt("Priorität (1 = höchste): ");
    ziele.add(new Sparziel(name, betrag, datum, 0.0, prio));
    speichereSparziele();
}

// --- MONATLICHE ZUFÜHRUNG (EINZIGE METHODE) ---
public static void monatlicheZufuehrung(ArrayList<FestWert>
festeWerte) {
    double sparrate = getSparrateAusFesteWerte(festeWerte);
    if (sparrate <= 0 || ziele.isEmpty()) {
        System.out.println("Keine Sparrate oder Sparziele

```

```

    vorhanden.");
        return;
    }

    List<Sparziel> sortiert = new ArrayList<>(ziele);
    sortiert.sort(Comparator.comparingInt(s -> s.prioritaet));

    double rest = sparrate;
    for (Sparziel sz : sortiert) {
        if (rest <= 0) break;
        double fehlend = sz.fehlend();
        if (fehlend > 0) {
            double zuweisung = Math.min(rest, fehlend);
            sz.aktuell += zuweisung;
            rest -= zuweisung;
        }
    }
    speichereSparziele();
    System.out.printf("Monatliche Sparrate: %.2f € verteilt.\n",
sparrate);
}

// --- SPARRATE ÄNDERN (EINZIGE METHODE) ---
public static void setzeSparrate(ArrayList<FestWert> festeWerte) {
    double aktuell = getSparrateAusFesteWerte(festeWerte);
    double neu = IO.readDouble("Neue monatliche Sparrate (aktuell:
" + String.format(Locale.US, "%.2f", aktuell) + " €): ");
    if (neu < 0) return;

    // Alte entfernen
    festeWerte.removeIf(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie));

    // Neuen hinzufügen
    if (neu > 0) {
        LocalDate datum = LocalDate.now().withDayOfMonth(1);
        festeWerte.add(new FestWert("Sparplan", neu, false, datum,
"Sparen", 1));
    }

    speichereFesteWerte(festeWerte); // <-- SOFORT SPEICHERN!
    System.out.println("Sparrate aktualisiert und in
festeWerte.txt gespeichert.");
}

// --- FESTEWERTE SPEICHERN (deine Logik) ---
public static void speichereFesteWerte(ArrayList<FestWert>
festeWerte) {
    try (FileWriter writer = new FileWriter("festeWerte.txt")) {

```

```

        for (FestWert f : festeWerte) {
            writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%d%n",
                f.bezeichnung, f.betrag, f.istEinnahme, f.datum,
                f.kategorie, f.periode));
        }
        System.out.println("Feste Werte in festeWerte.txt
        gespeichert.");
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern: " +
        e.getMessage());
    }
}

```

```

public static void aendereSparziel() {
    zeigeSparziele();
    if (ziele.isEmpty()) return;
    int idx = IO.readInt("Nummer (0 = Abbruch): ") - 1;
    if (idx < 0 || idx >= ziele.size()) return;

    Sparziel sz = ziele.get(idx);
    System.out.println("Aktuell: " + sz);

    // Name
    String n = IO.readString("Name (" + sz.name + ", Enter =
    beibehalten): ");
    if (!n.isEmpty()) sz.name = n;

    // Zielbetrag
    double b = IO.readDouble("Zielbetrag (" + sz.zielbetrag + ", 0
    = beibehalten): ");
    if (b > 0) sz.zielbetrag = b;

    // Datum – NUR ÄNDERN, WENN ETWAS EINGETIPPT WURDE
    LocalDate neuesDatum = IO.readDate("Datum (" + sz.zielDatum +
    "): ");
    if (neuesDatum != null) {
        sz.zielDatum = neuesDatum;
    }

    // Priorität
    int p = IO.readInt("Priorität (" + sz.prioritaet + ", 0 =
    beibehalten): ");
    if (p > 0) sz.prioritaet = p;

    speichereSparziele();
    System.out.println("Sparziel aktualisiert.");
}

```

```

// --- ANZEIGEN (KORREKT NUMMERIERT) ---
public static void zeigeSparziele() {
    if (ziele.isEmpty()) {
        System.out.println("Keine Sparziele.");
        return;
    }
    System.out.println("=== SPARZIELE (Prio 1 = höchste) ===");
    List<Sparziel> sortiert = ziele.stream()
        .sorted(Comparator.comparingInt(s -> s.prioritaet))
        .toList();
    for (int i = 0; i < sortiert.size(); i++) {
        System.out.println((i + 1) + ": " + sortiert.get(i));
    }
}

// --- LÖSCHEN ---
public static void loescheSparziel() {
    zeigeSparziele();
    if (ziele.isEmpty()) return;
    int idx = IO.readInt("Löschen (0 = Abbruch): ") - 1;
    if (idx >= 0 && idx < ziele.size()) {
        ziele.remove(idx);
        speichereSparziele();
    }
}
}

```

Klasse: Uebersicht

```

package HaushaltskassenApp;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;

public class Uebersicht {

    public static void zeigeAusgaben(ArrayList<Eintrag> eintraege) {
        System.out.println("Ausgaben:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie\n| Bezeichnung | Betrag |");

        System.out.println("-----");
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s| %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);
            }
        }
    }
}

```

```

        summe += e.betrag;
    }
    count++;
}

System.out.println("=====
=====");
    System.out.printf("|Gesamt Ausgaben: %54.2f€ |\n", summe);

System.out.println("=====
=====");

    }
    public static void zeigefesteAusgaben(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Ausgaben:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum      |          Kategorie
|          Bezeichnung      | Periode(Monate) | Betrag |");

System.out.println("-----
-----");

        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                int countE=0;
                while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                    countE++;
                }
                for (int i =0;i<countE;i++) {

                    System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                    summe += f.betrag;
                }
                monatsbeitrag +=f.betrag/f.periode;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt feste Ausgaben: %77.2f€ |\n",
summe);
        System.out.printf("|Gesamt feste Ausgaben/Monat: %71.2f€ |\n",
monatsbeitrag);

```



```

System.out.println("=====
=====");

}

    public static void zeigeEinnahmen(ArrayList<Eintrag> eintraege) {
        System.out.println("Einnahmen:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|          Bezeichnung      |          Betrag |");

System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %25s | %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);
                summe += e.betrag;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt Einnahmen: %64.2f€ |\n", summe);

System.out.println("=====
=====");
    }

    public static void zeigefesteEinnahmen(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Einnahmen:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|          Bezeichnung | Periode(Monate) |          Betrag |");

System.out.println("-----
-----");
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                int countE=0;
                while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                    countE++;
                }
            }
        }
    }

```

```

        for (int i =0;i<countE;i++) {

            System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
            summe += f.betrag;
        }
        monatsbeitrag +=f.betrag/f.periode;
    }
    count++;
}

System.out.println("=====
=====");
    System.out.printf("|Gesamt feste Einnahmen: %66.2f€ |\n",
summe);
    System.out.printf("|Gesamt feste Einnahmen/Monat: %60.2f€ |\n",
monatsbeitrag);

System.out.println("=====
=====");

}

```

```

    public static void zeigeBilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        zeigeAusgaben(eintraege);
        zeigefesteAusgaben(festeWerte);
        zeigeEinnahmen(eintraege);
        zeigefesteEinnahmen(festeWerte);
        double bilanzwert=0.0;
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                bilanzwert +=f.betrag;
            }
            else {
                bilanzwert -= f.betrag;
            }
        }
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                bilanzwert +=e.betrag;
            }
            else {
                bilanzwert -= e.betrag;
            }
        }

        System.out.println("=====
=====");
    }

```

```

        System.out.printf("| Bilanzwert: %30.2f€ |\n",bilanzwert);
System.out.println("=====");
    }

    public static void zeigeBalkendiagrammAusgaben(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte) {
    // Sammle Ausgaben pro Kategorie
    var katSum = new java.util.HashMap<String, Double>();

    for (Eintrag e : eintraege) {
        if (!e.istEinnahme) {
            katSum.merge(e.kategorie, e.betrag, Double::sum);
        }
    }
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            katSum.merge(f.kategorie, f.betrag, Double::sum);
        }
    }

    if (katSum.isEmpty()) {
        System.out.println("Keine Ausgaben vorhanden.");
        return;
    }

    double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
    int balkenLaenge = 20; // Länge des Balkens

    System.out.println("\nAusgaben nach Kategorie:");
    katSum.entrySet().stream()
        .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
        .forEach(e -> {
            int anzahl = (int) (e.getValue() / maxBetrag *
balkenLaenge);
            String balken = "█".repeat(anzahl) + "
".repeat(balkenLaenge - anzahl);
            System.out.printf("%s %s %8.2f €\n", balken,
String.format("%-12s", e.getKey()), e.getValue());
        });
    }

    public static final String RESET = "\u001B[0m";
    public static final String GRUEN = "\u001B[32m";

```

```

public static final String GELB = "\u001B[33m";
public static final String ROT = "\u001B[31m";

public static void zeigeMonatsBalken(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    LocalDate jetzt = LocalDate.now();
    int jahr = jetzt.getYear();
    int monat = jetzt.getMonthValue();

    var katSum = new java.util.HashMap<String, Double>();

    // Normale Einträge
    for (Eintrag e : eintraege) {
        if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
            katSum.merge(e.kategorie, e.betrag, Double::sum);
        }
    }

    // Feste Werte (wiederkehrend)
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            LocalDate naechste = f.datum;
            while (naechste.isBefore(jetzt.plusMonths(1))) {
                if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                    katSum.merge(f.kategorie, f.betrag,
Double::sum);
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }
    }

    if (katSum.isEmpty()) {
        System.out.println("Keine Ausgaben im aktuellen Monat.");
        return;
    }

    double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
    int balkenLaenge = 25;

    System.out.printf("\n=== %s %d ===\n", jetzt.getMonth(),
jahr);

```

```

        System.out.println("Budget-Warnung: " + ROT + "Überschritten"
+ RESET + ", " + GELB + "Kritisch" + RESET + ", " + GRUEN + "OK" +
RESET);

```

```

        katSum.entrySet().stream()
            .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
            .forEach(e -> {
                double ausgegeben = e.getValue();
                double budget = BudgetManager.getBudget(e.getKey()); // <-
0.0 wenn nicht vorhanden
                int anzahl = (int) (ausgegeben / maxBetrag *
balkenLaenge);
                String balken;
                String farbe;

                if (budget > 0 && ausgegeben > budget) {
                    balken = ROT + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                    farbe = ROT;
                } else if (budget > 0 && ausgegeben > budget * 0.9) {
                    balken = GELB + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                    farbe = GELB;
                } else {
                    balken = GRUEN + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                    farbe = (budget == 0) ? RESET : GRUEN; // Grau, wenn
kein Budget
                }

                String budgetStr = (budget == 0)
                    ? String.format("%8.2f €", ausgegeben)
                    : String.format("%8.2f / %.2f €", ausgegeben, budget);

                System.out.printf("%s %s%s %s\n",
                    balken + " ".repeat(balkenLaenge - Math.min(anzahl,
balkenLaenge)),
                    farbe,
                    String.format("%-15s", e.getKey()),
                    budgetStr + RESET);
            });
    }

```

```

public static void zeigeGefiltert(ArrayList<Eintrag>

```

```

    eintraege, ArrayList<FestWert> festeWerte, String kategorie) {
        System.out.println("Einträge in Kategorie: " + kategorie);
        System.out.println("| Datum      | Betrag   | Kategorie  
| Bezeichnung | Art      | Ausgabe/Einnahme|");

        System.out.println("-----  
-----");

        String art="";
        boolean gefunden = false;
        for (Eintrag e : eintraege) {
            if (e.kategorie.equals(kategorie)) {
                art= e.istEinnahme ? "Einnahme":"Ausgabe";
                System.out.printf("| %10s | %8.2f | %17s | %15s |  
Eintrag    |%16s|\n",
                    e.datum, e.betrag, e.kategorie, e.bezeichnung,
                    art);
                gefunden = true;
            }
        }
        for (FestWert f: festeWerte) {
            if (f.kategorie.equals(kategorie)) {
                art= f.istEinnahme ? "Einnahme":"Ausgabe";
                System.out.printf("| %10s | %8.2f | %17s | %15s |  
fester Wert|%16s|\n",
                    f.datum, f.betrag, f.kategorie, f.bezeichnung,
                    art);
                gefunden = true;
            }
        }

        if (!gefunden) {
            System.out.println("=====");
            System.out.println("      Keine Einträge in dieser  
Kategorie!");
            System.out.println("=====");
        }
    }

    public static void zeigeSortiertNachBetrag(ArrayList<Eintrag>
    eintraege, ArrayList<FestWert> festeWerte, boolean absteigend) {
        ArrayList<Eintrag> kopieEintraege = new
        ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
        ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = absteigend ?
            Comparator.comparingDouble((Eintrag e) ->

```

```

e.betrag).reversed() : Comparator.comparingDouble((Eintrag e) ->
e.betrag);

        Comparator<FestWert> compF = absteigend ?
            Comparator.comparingDouble((FestWert f) ->
f.betrag).reversed() : Comparator.comparingDouble((FestWert f) ->
f.betrag);

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }

    public static void zeigeSortiertNachDatum(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean a) {

        ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = Comparator.comparing(e -> e.datum,

            Comparator.nullsLast(Comparator.naturalOrder()));
        Comparator<FestWert> compF = Comparator.comparing(f ->
f.datum,
            Comparator.nullsLast(Comparator.naturalOrder()));

        if (a) {
            compE = compE.reversed();
            compF = compF.reversed();
        }

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }

    public static void zeigeBevorstehendeAusgaben(ArrayList<FestWert>
festeWerte) {

        int tage= IO.readInt("Geben Sie die Tage des Intervalls an:
");

```

```

        LocalDate now=LocalDate.now(); // LocalDate.of(2025,12,1);

System.out.println("=====
=====");
        System.out.println("
Bevorstehende Ausgaben");

System.out.println("=====
=====");
        System.out.println("| Fälligkeitsdatum |      Kategorie
|      Bezeichnung      | Periode(Monate) |      Betrag |");

System.out.println("-----
-----");
        double summe=0.0;

        for (FestWert f :festeWerte)

            if (!f.istEinnahme) {
                int countE=0;
                int countA=0;

                while (!
now.plusDays(tage).isBefore(f.datum.plusMonths(countE*f.periode))) {

                    countE++;
                }
                while
(now.isAfter(f.datum.plusMonths(countA*f.periode))) {
//!now.isBefore(f.datum.plusMonths(countA*f.periode))---heute nicht
dabei
                    countA++;
                }
                for (int i =countA;i<countE;i++) {
                    System.out.printf("|%17s | %20s | %25s| %17d|
%8.2f€ |\n",f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                    summe +=f.betrag;
                }

            }

System.out.println("=====
=====");
        if(summe !=0.0) {
            System.out.printf("Die zu erwartende aufzubringende Summe in
den nächsten %"+4+"d Tagen ist: %27.2f€ |\n",tage,summe);
        }
        else {
            System.out.printf("In den nächsten %d Tagen erwartet Dich

```



```

keine Ausgabe fester Werte!\n",tage);
    }

    System.out.println("=====
=====");

    }

}

```

Klasse: Verwaltung

```

package HaushaltskassenApp;

import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Locale;

public class Verwaltung {

    public static void main(String[] args) {

        Scanner sc =new Scanner(System.in);
        BudgetManager.ladeBudgets();
        SparzielManager.ladeSparziele();
        int auswahl;
        int festeWertAuswahl;

        ArrayList<Eintrag> eintraege = new ArrayList<>();
        ArrayList<FestWert> festeWerte = new ArrayList<>();
        String
Fehlertext="=====\\n
Fehlerhafte Eingabe!!!!\\
n=====\\n";
        try (Scanner fileScanner = new Scanner(new
FileReader("eintraege.txt"))) {
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                String[] parts = line.split("#");
                if (parts.length == 5) {
                    Eintrag e = new Eintrag(
                        parts[0].trim(),

```

```

        Double.parseDouble(parts[1].trim()),
        Boolean.parseBoolean(parts[2].trim()),
        LocalDate.parse(parts[3].trim()),
        parts[4].trim()
    );
    eintraege.add(e);
}
}
System.out.println("Einträge aus eintraege.txt geladen.");
} catch (FileNotFoundException e) {
    System.out.println("Keine gespeicherten Einträge gefunden,
    starte mit leerer Liste.");
} catch (Exception e) {
    System.err.println("Fehler beim Laden: " +
    e.getMessage());
}
    festeWerte = new ArrayList<>();
    try (Scanner fileScanner = new Scanner(new
    FileReader("festeWerte.txt"))) {
        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            String[] parts = line.split("#");
            if (parts.length == 6) {
                FestWert f = new FestWert(
                    parts[0].trim(),
                    Double.parseDouble(parts[1].trim()),
                    Boolean.parseBoolean(parts[2].trim()),
                    LocalDate.parse(parts[3].trim()),
                    parts[4].trim(),
                    Integer.parseInt(parts[5].trim())
                );
                festeWerte.add(f);
            }
        }
        System.out.println("Feste Werte aus festeWerte.txt
        geladen.");
    } catch (FileNotFoundException e) {
        System.out.println("Keine festen Werte gefunden, starte
        mit leerer Liste.");
    } catch (Exception e) {
        System.err.println("Fehler beim Laden: " +
        e.getMessage());
    }
}

```

```

    Eintrag eTescht=new
    Eintrag("Kühlschrank",588.95,false,LocalDate.of(2025,10,27),"Einmal-
    Anschaffung");
    Eintrag eTescht2=new

```

```

Eintrag("Flohmarkt",345.11,true,LocalDate.of(2025,10,28),"Krimskrams")
;
    Eintrag eTesch3=new
Eintrag("Flohmarkt",12.50,false,LocalDate.of(2025,10,28),"Krimskrams")
;
//      eintraege.add(eTesch);
//      eintraege.add(eTesch2);
//      eintraege.add(eTesch3);

    FestWert fWTesch=new
FestWert("GEZ",58.95,false,LocalDate.of(2025, 9,
1),"Rundfunkgebühren",3);
    FestWert fWTesch2=new
FestWert("Pacht",1500,true,LocalDate.of(2025, 1,
1),"Mieteinnahmen",12);
//      festeWerte.add(fWTesch);
//      festeWerte.add(fWTesch2);

    // System.out.println(eintraege);
    while (true) {

System.out.println("=====");
        System.out.println("                HAUSHALTSKASSEN-APP");

System.out.println("=====");
        System.out.println("Was wünschen Sie?");

System.out.println("=====");
        System.out.printf(" 1: Eintrag hinzufügen\n 2: Eintrag
löschen\n 3: Eintrag ändern\n 4: Feste Werte\n 5: Übersicht der
Ausgaben\n 6: Übersicht der Einnahmen\n 7: Gegenüberstellung der
Einnahmen und Ausgaben\n 8: Gefilterte Einträge\n 9: Sortierte
Ausgabe\n10: Bevorstehende Ausgaben fester Werte in den von Ihnen
eingegebenen Tagen\n11: Balkendiagramm Ausgaben\n12: Budget-Manager\
n13: Sonstiges\n14: Beenden\n");
        auswahl=sc.nextInt();

System.out.println("=====");

        if (auswahl==1) {
            Eintrag e = new Eintrag(
                IO.readString("Bezeichnung: "),
                IO.readDouble("Betrag: "),
                IO.readBoolean("Einnahme?(j/n): "),
                IO.readDate("Datum:"),
                IO.readString("Kategorie: ")
            );
            Eintrag.fuegeEintragHinzu(eintraege, e);
            // System.out.println(eintraege);

```

```

    }
    else if(auswahl==2) {
        int index = IO.readInt("Geben Sie den Index an: ");
        Eintrag.loescheEintrag(eintraege, index);
    }
    else if(auswahl==3) {
        int index = IO.readInt("Geben Sie den Index an: ");
        if (index >= 0 && index < eintraege.size()) {
            eintraege.set(index, Eintrag.aendereEintrag());
        } else {
            System.out.println(Fehlertext);
        }
    }

    }
    else if (auswahl==4) {
        boolean istAuswahl=true;
        while (istAuswahl) {

System.out.println("=====");
            System.out.println("Was wollen Sie mit den
festen Werten machen?");

System.out.println("=====");
            System.out.printf("1: Festen Wert hinzufügen\
n2: Festen Wert löschen\n3: Festen Wert ändern\n4: Zurück zum
Hauptmenu\n");

            festeWertAuswahl=sc.nextInt();
            switch (festeWertAuswahl) {
            case 1:
                festeWerte.add(new FestWert(
                    IO.readString("Bezeichnung: "),
                    IO.readDouble("Betrag: "),
                    IO.readBoolean("Einnahme?(j/n): "),
                    IO.readDate("Datum: "),
                    IO.readString("Kategorie: "),
                    IO.readInt("Periode (Monate): ")
                ));
                break;
            case 2:
                int index = IO.readInt("Index: ");
                if (index >= 0 && index <
festeWerte.size()) {
                    festeWerte.remove(index);
                } else {
                    System.out.println(Fehlertext);
                }
                break;
            case 3:
                index = IO.readInt("Index: ");
                if (index >= 0 && index <

```

```

festeWerte.size()) {
    festeWerte.set(index, new FestWert(
        IO.readString("Bezeichnung: "),
        IO.readDouble("Betrag: "),
        IO.readBoolean("Einnahme?(j/n):
"),
        IO.readDate("Datum: "),
        IO.readString("Kategorie: "),
        IO.readInt("Periode (Monate): ")
    ));
} else {
    System.out.println(Fehlertext);
}
break;
case 4:
    istAuswahl=false;
    break;
default:
    System.out.println(Fehlertext);
}

}
}
else if (auswahl == 5) {
    Uebersicht.zeigeAusgaben(eintraege);
    Uebersicht.zeigefesteAusgaben(festeWerte);
} else if (auswahl == 6) {
    Uebersicht.zeigeEinnahmen(eintraege);
    Uebersicht.zeigefesteEinnahmen(festeWerte);
} else if (auswahl == 7) {
    Uebersicht.zeigeBilanz(eintraege, festeWerte);
}
else if (auswahl == 8) {

System.out.println("=====");
    System.out.println("                Gefilterte Ausgabe");

System.out.println("=====");
    String filter=IO.readString("Geben Sie die Kategotie
an: ");
    Uebersicht.zeigeGefiltert(eintraege,festeWerte,
filter);
} else if (auswahl == 9) {

System.out.println("=====");
    System.out.println("                Sortierte Ausgabe");

System.out.println("=====");

```

```

        int sortAuswahl=I0.readInt("Sortieren nach Betrag:
(1)\nSortieren nach Datum: (2)\n");
        switch (sortAuswahl) {
            case 1: {
                boolean isAbsteigend=I0.readBoolean("Nach
Betrag absteigend? (j/n): ");
                Uebersicht.zeigeSortiertNachBetrag(eintraege,
festeWerte,isAbsteigend);
                break;
            }
            case 2: {
                boolean isAbsteigend=I0.readBoolean("Nach Datum
absteigend? (j/n): ");
                Uebersicht.zeigeSortiertNachDatum(eintraege,
festeWerte, isAbsteigend);
                break;
            }
            default:
                System.out.println(Fehlertext);
        }
    }else if (auswahl==10) {
        Uebersicht.zeigeBevorstehendeAusgaben(festeWerte);
    }
    else if (auswahl == 11) {
        Uebersicht.zeigeBalkendiagrammAusgaben(eintraege,
festeWerte);
        Uebersicht.zeigeMonatsBalken(eintraege, festeWerte);
    }

    else if (auswahl==12) {
        budgetMenue();
    }
    else if (auswahl==13) {
        Sonstiges.zeigeMenü(eintraege, festeWerte);
    }

    else if (auswahl==14) {

System.out.println("=====");
        System.out.println("Vielen Dank! Bis zum nächsten
Mal!");

System.out.println("=====");
        //        SparzielManager.speichereFesteWerte(festeWerte);
        break;
    }

    else {
        //
System.out.println("=====");

```

```

//          System.out.println("          Fehlerhafte
Eingabe!!!!");
//
System.out.println("=====");
        System.out.printf(Fehlertext);
    }
}
try (FileWriter writer = new FileWriter("eintraege.txt")) {
    for (Eintrag e : eintraege) {
        writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s%n",
                                e.bezeichnung,
                                e.betrag,
                                e.istEinnahme,
                                e.datum,
                                e.kategorie
                                ));
    }
    System.out.println("Einträge in eintraege.txt
gespeichert.");
} catch (IOException e) {
    System.err.println("Fehler beim Speichern: " +
e.getMessage());
}

    try (FileWriter writer = new FileWriter("festeWerte.txt")) {
        for (FestWert f : festeWerte) {
            writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s#%d%n",
                                f.bezeichnung,
                                f.betrag,
                                f.istEinnahme,
                                f.datum,
                                f.kategorie,
                                f.periode
                                ));
        }
        System.out.println("Feste Werte in festeWerte.txt
gespeichert.");
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern: " +
e.getMessage());
    }

    sc.close();

}

```

```

        private static void budgetMenu() {
            Scanner sc = new Scanner(System.in);
            while (true) {

System.out.println("=====");
                System.out.println("                BUDGET-VERWALTUNG");

System.out.println("=====");
                System.out.println("1: Budget eingeben/ändern");
                System.out.println("2: Budget löschen");
                System.out.println("3: Alle Budgets anzeigen");
                System.out.println("4: Zurück zum Hauptmenü");
                System.out.print("Wahl: ");
                int wahl = sc.nextInt();
                sc.nextLine(); // Zeilenumbruch

                if (wahl == 1) {
                    String kat = IO.readString("Kategorie: ");
                    double betrag = IO.readDouble("Budget (0 = löschen):");
");
                    BudgetManager.setzeBudget(kat, betrag);
                }
                else if (wahl == 2) {
                    String kat = IO.readString("Kategorie zum Löschen: ");
                    BudgetManager.loescheBudget(kat);
                }
                else if (wahl == 3) {
                    BudgetManager.zeigeAlleBudgets();
                }
                else if (wahl == 4) {
                    break;
                }
                else {
                    System.out.println("Ungültige Eingabe!");
                }
            }

        }

    }
}

```

Package: HaushaltskassenAppGUI

Klasse: Eintrag

```
package HaushaltskassenAppGUI;
```

```
import java.time.LocalDate;
```



```
import java.util.ArrayList;
import java.util.Objects;

public class Eintrag {

    public String bezeichnung;
    public double betrag;
    public boolean istEinnahme;
    public LocalDate datum;
    public String kategorie;

    public Eintrag(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie) {
        super();
        this.bezeichnung = bezeichnung;
        this.betrag = betrag;
        this.istEinnahme = istEinnahme;
        this.datum = datum;
        this.kategorie = kategorie;
    }

    public String getBezeichnung() {
        return bezeichnung;
    }

    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }

    public double getBetrag() {
        return betrag;
    }

    public void setBetrag(double betrag) {
        this.betrag = betrag;
    }

    public boolean isEinnahme() {
        return istEinnahme;
    }

    public void setIstEinnahme(boolean istEinnahme) {
        this.istEinnahme = istEinnahme;
    }

    public LocalDate getDatum() {
        return datum;
    }
}
```

```

    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }

    public String getKategorie() {
        return kategorie;
    }

    public void setKategorie(String kategorie) {
        this.kategorie = kategorie;
    }

    @Override
    public int hashCode() {
        return Objects.hash(betrag, bezeichnung, datum, istEinnahme,
kategorie);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Eintrag other = (Eintrag) obj;
        return Double.doubleToLongBits(betrag) ==
Double.doubleToLongBits(other.betrag)
            && Objects.equals(bezeichnung, other.bezeichnung) &&
Objects.equals(datum, other.datum)
            && istEinnahme == other.istEinnahme &&
Objects.equals(kategorie, other.kategorie);
    }

    @Override
    public String toString() {
        return "Eintrag [bezeichnung=" + bezeichnung + ", betrag=" +
betrag + ", istEinnahme=" + istEinnahme
            + ", datum=" + datum + ", kategorie=" + kategorie +
"]";
    }

    public static void loescheEintrag(ArrayList<Eintrag> ae,int index)
{
    if (index >= 0 && index < ae.size()) {
        ae.remove(index);
    } else {

```

```

        System.out.println("Ungültiger Index!");
    }
}

// public static Eintrag aendereEintrag() {
//     return new Eintrag(
//         IO.readString("Bezeichnung: "),
//         IO.readDouble("Betrag: "),
//         IO.readBoolean("Einnahme?(j/n): "),
//         IO.readDate("Datum: "),
//         IO.readString("Kategorie: ")
//     );
// }
// }
// public static void fuegeEintragHinzu(ArrayList<Eintrag>
ae ,Eintrag e) {
    ae.add(e);
}

}

```

Klasse: EintragDialog

```
package HaushaltskassenAppGUI;
```

```
import javax.swing.*;
import java.awt.*;
import java.time.LocalDate;
```

```

public class EintragDialog extends JDialog {
    private JTextField txtBezeichnung, txtBetrag, txtKategorie;
    private JCheckBox chkEinnahme;
    private JSpinner spJahr, spMonat, spTag;
    private JButton btnSave;
    private Eintrag eintrag;
    private boolean saved = false;

    public EintragDialog(JFrame parent, Eintrag e) {
        super(parent, e == null ? "Eintrag hinzufügen" : "Eintrag
ändern", true);
        this.eintrag = e;
        initUI();
        setSize(500, 300);
        setLocationRelativeTo(parent);
    }

    private void initUI() {
        JPanel panel = new JPanel(new GridLayout(6, 2, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(20, 20, 20,
20));

        panel.add(new JLabel("Bezeichnung:"));
    }
}

```

```

        txtBezeichnung = new JTextField();
        panel.add(txtBezeichnung);

        panel.add(new JLabel("Betrag:"));
        txtBetrag = new JTextField();
        panel.add(txtBetrag);

        panel.add(new JLabel("Einnahme:"));
        chkEinnahme = new JCheckBox();
        panel.add(chkEinnahme);

        panel.add(new JLabel("Datum (J/M/T):"));
        JPanel datePanel = new JPanel();
        spJahr = new JSpinner(new SpinnerNumberModel(2025, 2000, 2030,
1));
        spMonat = new JSpinner(new SpinnerNumberModel(1, 1, 12, 1));
        spTag = new JSpinner(new SpinnerNumberModel(1, 1, 31, 1));
        datePanel.add(spJahr); datePanel.add(spMonat);
datePanel.add(spTag);
        panel.add(datePanel);

        panel.add(new JLabel("Kategorie:"));
        txtKategorie = new JTextField();
        panel.add(txtKategorie);

        btnSave = new JButton("Speichern");
        panel.add(btnSave);

        add(panel);

        btnSave.addActionListener(e -> save());

        if (eintrag != null) loadData();
    }

    private void loadData() {
        txtBezeichnung.setText(eintrag.bezeichnung);
        txtBetrag.setText(String.valueOf(eintrag.betrag));
        chkEinnahme.setSelected(eintrag.istEinnahme);
        spJahr.setValue(eintrag.datum.getYear());
        spMonat.setValue(eintrag.datum.getMonthValue());
        spTag.setValue(eintrag.datum.getDayOfMonth());
        txtKategorie.setText(eintrag.kategorie);
    }

    private void save() {
        try {
            String bez = txtBezeichnung.getText();
            double betrag = Double.parseDouble(txtBetrag.getText());

```

```

        boolean einnahme = chkEinnahme.isSelected();
        LocalDate datum = LocalDate.of(
            (int)spJahr.getValue(),
            (int)spMonat.getValue(),
            (int)spTag.getValue()
        );
        String kat = txtKategorie.getText();

        if (eintrag == null) {
            eintrag = new Eintrag(bez, betrag, einnahme, datum,
kat);
        } else {
            eintrag.bezeichnung = bez;
            eintrag.betrag = betrag;
            eintrag.istEinnahme = einnahme;
            eintrag.datum = datum;
            eintrag.kategorie = kat;
        }
        saved = true;
        dispose();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ungültige Eingabe!");
    }
}

    public Eintrag getEintrag() { return eintrag; }
    public boolean isSaved() { return saved; }
}package NorthwindApp.nw_db_tool;

import java.time.LocalDate;

public class Employee {

    private int empid;
    private String firstname;
    private String lastname;
    private LocalDate birthdate;
    public Employee(int empid, String firstname, String lastname,
LocalDate birthdate) {
        this.empid = empid;
        this.firstname = firstname;
        this.lastname = lastname;
        this.birthdate = birthdate;
    }
    public int getEmpid() {
        return empid;
    }
    public void setEmpid(int empid) {
        this.empid = empid;
    }
}

```

```

    public String getFirstname() {
        return firstname;
    }
    public void setFirstname(String firstname) {
        this.firstname = firstname;
    }
    public String getLastname() {
        return lastname;
    }
    public void setLastname(String lastname) {
        this.lastname = lastname;
    }
    public LocalDate getBirthdate() {
        return birthdate;
    }
    public void setBirthdate(LocalDate birthdate) {
        this.birthdate = birthdate;
    }

    @Override
    public String toString() {
        // TODO Auto-generated method stub
        return firstname + " " + lastname;
    }
}

```

Klasse: FestWert

```

package HaushaltskassenAppGUI;

import java.time.LocalDate;

public class FestWert extends Eintrag{
    public int periode;

    public FestWert(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie,
        int periode) {
        super(bezeichnung, betrag, istEinnahme, datum, kategorie);
        this.periode = periode;
    }

    public int getPeriode() {
        return periode;
    }

    public void setPeriode(int periode) {
        this.periode = periode;
    }
}

```

```

        @Override
        public String toString() {
            return "FestWert [periode=" + periode + ", bezeichnung=" +
bezeichnung + ", betrag=" + betrag + ", istEinnahme="
            + istEinnahme + ", datum=" + datum + ", kategorie=" +
kategorie + "]";
        }

```

```

    }

```

Klasse: FestWertDialog

```

package HaushaltskassenAppGUI;

```

```

import javax.swing.*;
import java.awt.*;
import java.time.LocalDate;

```

```

public class FestWertDialog extends JDialog {
    private JTextField txtBezeichnung, txtBetrag, txtKategorie;
    private JCheckBox chkEinnahme;
    private JSpinner spJahr, spMonat, spTag, spPeriode;
    private JButton btnSave;
    private FestWert festWert;
    private boolean saved = false;

    public FestWertDialog(JFrame parent, FestWert f) {
        super(parent, f == null ? "Festen Wert hinzufügen" : "Festen
Wert bearbeiten", true);
        this.festWert = f;
        initUI();
        pack();
        setLocationRelativeTo(parent);
    }

    private void initUI() {
        setLayout(new BorderLayout());
        JPanel panel = new JPanel(new GridLayout(14, 1, 10, 10));
        panel.setBorder(BorderFactory.createEmptyBorder(15, 15, 15,
15));
        panel.add(new JLabel("Bezeichnung:"));
        txtBezeichnung = new JTextField(20);
        panel.add(txtBezeichnung);
        panel.add(new JLabel("Betrag:"));
        txtBetrag = new JTextField();
        panel.add(txtBetrag);

```

```

        panel.add(new JLabel("Einnahme:"));
        chkEinnahme = new JCheckBox();
        panel.add(chkEinnahme);
        panel.add(new JLabel("Jahr:"));
        spJahr = new JSpinner(new
SpinnerNumberModel(LocalDate.now().getYear(), 2000, 2030, 1));
        panel.add(spJahr);
        panel.add(new JLabel("Monat:"));
        spMonat = new JSpinner(new SpinnerNumberModel(1, 1, 12, 1));
        panel.add(spMonat);
        panel.add(new JLabel("Tag:"));
        spTag = new JSpinner(new SpinnerNumberModel(1, 1, 31, 1));
        panel.add(spTag);
        panel.add(new JLabel("Kategorie:"));
        txtKategorie = new JTextField();
        panel.add(txtKategorie);
        panel.add(new JLabel("Periode (Monate:)"));
        spPeriode = new JSpinner(new SpinnerNumberModel(1, 1, 120,
1));
        panel.add(spPeriode);
        btnSave = new JButton("Speichern");
        btnSave.addActionListener(e -> save());
        JPanel btnPanel = new JPanel();
        btnPanel.add(btnSave);
        add(panel, BorderLayout.CENTER);
        add(btnPanel, BorderLayout.SOUTH);
        if (festWert != null)
            loadData();
    }

    private void loadData() {
        txtBezeichnung.setText(festWert.bezeichnung);
        txtBetrag.setText(String.valueOf(festWert.betrag));
        chkEinnahme.setSelected(festWert.istEinnahme);
        spJahr.setValue(festWert.datum.getYear());
        spMonat.setValue(festWert.datum.getMonthValue());
        spTag.setValue(festWert.datum.getDayOfMonth());
        txtKategorie.setText(festWert.kategorie);
        spPeriode.setValue(festWert.periode);
    }

    private void save() {
        try {
            String bez = txtBezeichnung.getText().trim();
            double betrag =
Double.parseDouble(txtBetrag.getText().trim());
            boolean einnahme = chkEinnahme.isSelected();
            LocalDate datum = LocalDate.of((int) spJahr.getValue(),
(int) spMonat.getValue(), (int) spTag.getValue());
            String kat = txtKategorie.getText().trim();

```



```

        int periode = (int) spPeriode.getValue();
        if (bez.isEmpty() || kat.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Bitte alle Felder
ausfüllen!");
            return;
        }
        if (festWert == null) {
            festWert = new FestWert(bez, betrag, einnahme, datum,
kat, periode);
        } else {
            festWert.bezeichnung = bez;
            festWert.betrag = betrag;
            festWert.istEinnahme = einnahme;
            festWert.datum = datum;
            festWert.kategorie = kat;
            festWert.periode = periode;
        }
        saved = true;
        dispose();
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Ungültige Eingabe: "
+ ex.getMessage());
    }
}

    public FestWert getFestWert() {
        return festWert;
    }

    public boolean isSaved() {
        return saved;
    }
}package _11_03_2026;

import java.io.BufferedReader;
import java.io.BufferedWriter;

/**
 * IO Input output
 *
 *
 * Dateibasiert:
 *     -File(Datei oder Verzeichnis)
 *     -Lesen aus einer Datei
 *     -Schreiben in eine Datei
 *
 * Text-basierte Dateien
 *     -Plain text
 *     -CSV
 *

```

```

*/

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.ArrayList;
import java.util.List;

public class FileIO {

    public static void main(String[] args) {

        /**
         * Schreiben:
         *      -1. Datei erzeugen bzw existierende Datei zum
Schreiben öffnen
         *      -2. Etwas in die Datei schreiben
         *      -3. Datei schließen
         *
         *C:\Users\Student\OneDrive - GFN GmbH (EDU)\Desktop\Ordner 1\
LF  ZQ19A
         */
        //Datei erzeugen

        //File f = new File("C:\\Users\\Student\\OneDrive - GFN GmbH
(EDU)\\Desktop\\Ordner 1\\LF  ZQ19A\\temperaturen.txt");
        File f = new File("names.txt"); //Objekt auf dem HEAP, es ist
noch keine Datei erzeugt worden
        //C:\Users\Student\eclipse-workspace\LF_ZQ19A
        //Datei erzeugen auf dem Dateisystem

        try {
            boolean success= f.createNewFile();
            if (success)
                System.out.println("Datei wurde erzeugt");
            else
                System.out.println("Datei existiert bereits");
            System.out.println("success = "+ success);

            //Schreiben mit BufferedWriter
            BufferedWriter br= new BufferedWriter(new
FileWriter(f,true)); //das true für das anhängen egal ob append oder
write

```

```

n");
    br.write("=====Mittels BufferedWriter=====\\n");
    br.write("Bob");
    br.newLine();//neue zeile bzw zeilenumbruch
    br.write("Alice");
    br.newLine();
    br.write("Chris");
    br.newLine();
    br.flush();//jetzt wird es in die datei geschrieben
    br.append("das kommt neu dazu");
    br.newLine();
    br.flush();
    br.append("das kommt nochmal dazu");
    br.newLine();
    br.close();//hier ist das flush() nicht mehr nötig

```

```

//Schreiben mittels PrintWriter(beste)
PrintWriter pw = new PrintWriter(new FileWriter(f,true));

pw.write("=====Mittels PrintWriter=====\\n");
pw.println("=".repeat(30));
pw.println("Bob");
pw.println("Alice");
pw.println("Chris");
//pw.flush();
pw.close();

```

```

//Schreiben mittels FileWriter
FileWriter fw=new FileWriter(f, true);//append statt
überschreiben

fw.write("=====Mittels FileWriter=====\\n");
fw.write("Bob \\n");
fw.write("Alice\\r\\n");
fw.write("Chris\\r\\n");
fw.write("Bob \\t");
fw.write("Alice\\t");
fw.write("Chris\\n");
fw.close();

```

```

} catch (IOException e) {
    System.out.println("Datei konnte nicht erzeugt werden

```

```

"+e.getMessage());
    }

    //Lesen
    List<String> ls = new ArrayList<String>();
    try (BufferedReader br =
Files.newBufferedReader(Paths.get("names.txt")
)) {
        String s;
        while ((s=br.readLine()) != null)
            ls.add(s);
    } catch (IOException e) {
        System.out.println("Datei konnte nicht gelesen werden
"+e.getMessage());
    }

    for(String s : ls)
        System.out.println(s);

    System.out.println("Programm terminiert normal");

}

}

```

Klasse: Uebersicht

```

package HaushaltskassenAppGUI;

import java.time.LocalDate;
import java.util.ArrayList;

public class Uebersicht {

    public static String getUebersichtText(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte) {
        StringBuilder sb = new StringBuilder();
        sb.append(" HAUSHALTSKASSEN-APP - UEBERBLICK\n");
        sb.append(" Generiert am: ").append(LocalDate.now()).append("\
n");
        sb.append("=".repeat(70)).append("\n\n");

        // --- AUSGABEN ---
        printAusgaben(sb, eintraege);

        // --- FESTE AUSGABEN ---
        printFesteAusgaben(sb, festeWerte);
    }
}

```

```

// --- EINNAHMEN ---
printEinnahmen(sb, eintraege);

// --- FESTE EINNAHMEN ---
printFesteEinnahmen(sb, festeWerte);

// --- BILANZ ---
double summeAusgaben = eintraege.stream().filter(e -> !
e.istEinnahme).mapToDouble(e -> e.betrag).sum();
double summeFeste = festeWerte.stream().filter(f -> !
f.istEinnahme).mapToDouble(f -> f.betrag).sum();
double summeEinnahmen = eintraege.stream().filter(e ->
e.istEinnahme).mapToDouble(e -> e.betrag).sum();
double summeFesteE = festeWerte.stream().filter(f ->
f.istEinnahme).mapToDouble(f -> f.betrag).sum();

double bilanz = summeEinnahmen + summeFesteE - summeAusgaben -
summeFeste;

sb.append("=".repeat(70)).append("\n");
sb.append(String.format(" GESAMTBILANZ: %48.2f€\n", bilanz));
sb.append(bilanz >= 0 ? " POSITIV\n" : " NEGATIV\n");
sb.append("=".repeat(70)).append("\n");

return sb.toString();
}

//
=====
// HILFSMETHODEN – AUTOMATISCHE TABELLEN
//
=====

private static void printAusgaben(StringBuilder sb,
ArrayList<Eintrag> eintraege) {
    String[] headers = {"Datum", "Kategorie", "Bezeichnung",
"Betrag"};
    int[] widths = new int[headers.length];
    for (int i = 0; i < headers.length; i++) widths[i] =
headers[i].length();

    double summe = 0;
    for (Eintrag e : eintraege) {
        if (e.istEinnahme) continue;
        widths[0] = Math.max(widths[0], 12);
        widths[1] = Math.max(widths[1], e.kategorie.length());
        widths[2] = Math.max(widths[2], e.bezeichnung.length());
        widths[3] = Math.max(widths[3], String.format("%.2f€",
e.betrag).length());

```

```

        summe += e.betrag;
    }
    for (int i = 0; i < headers.length; i++) {
        widths[i] = Math.max(widths[i], headers[i].length());
    }

    String headerFmt = "%-" + widths[0] + "s %-" + widths[1] + "s
    %-" + widths[2] + "s %" + widths[3] + "s\n";
    String rowFmt = "%-" + widths[0] + "s %-" + widths[1] + "s
    %-" + widths[2] + "s %" + widths[3] + ".2f€\n";
    int totalWidth = widths[0] + widths[1] + widths[2] + widths[3]
    + 3;

    sb.append(" AUSGABEN\n");
    sb.append("-".repeat(totalWidth)).append("\n");
    sb.append(String.format(headerFmt, (Object[]) headers));
    sb.append("-".repeat(totalWidth)).append("\n");

    for (Eintrag e : eintraege) {
        if (!e.istEinnahme) {
            sb.append(String.format(rowFmt, e.datum, e.kategorie,
            e.bezeichnung, e.betrag));
        }
    }
    sb.append("-".repeat(totalWidth)).append("\n");
    sb.append(String.format("%" + (widths[0] + widths[1] +
    widths[2] + 3) + "s %" + widths[3] + ".2f€\n\n",
    "Gesamt Ausgaben:", summe));
}

private static void printFesteAusgaben(StringBuilder sb,
ArrayList<FestWert> festeWerte) {
    String[] headers = {"Datum", "Betrag", "Kategorie",
    "Bezeichnung", "Periode"};
    int[] widths = new int[headers.length];
    for (int i = 0; i < headers.length; i++) widths[i] =
    headers[i].length();

    double summe = 0, monatlich = 0;
    for (FestWert f : festeWerte) {
        if (f.istEinnahme) continue;
        widths[0] = Math.max(widths[0], 12);
        widths[1] = Math.max(widths[1], String.format("%.2f€",
    f.betrag).length());
        widths[2] = Math.max(widths[2], f.kategorie.length());
        widths[3] = Math.max(widths[3], f.bezeichnung.length());
        widths[4] = Math.max(widths[4], String.format("%d Monate",
    f.periode).length());
        summe += f.betrag;
        monatlich += f.betrag / f.periode;
    }
}

```

```

    }
    for (int i = 0; i < headers.length; i++) {
        widths[i] = Math.max(widths[i], headers[i].length());
    }

    String headerFmt = "%-" + widths[0] + "s %" + widths[1] + "s
    %-" + widths[2] + "s %-" + widths[3] + "s %" + widths[4] + "s\n";
    String rowFmt = "%-" + widths[0] + "s %" + widths[1] +
    ".2f€ %-" + widths[2] + "s %-" + widths[3] + "s %d Monate\n";
    int totalWidth = widths[0] + widths[1] + widths[2] + widths[3]
    + widths[4] + 4;

    sb.append(" FESTE AUSGABEN\n");
    sb.append("-".repeat(totalWidth)).append("\n");
    sb.append(String.format(headerFmt, (Object[]) headers));
    sb.append("-".repeat(totalWidth)).append("\n");

    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            sb.append(String.format(rowFmt, f.datum, f.betrag,
            f.kategorie, f.bezeichnung, f.periode));
        }
    }
    sb.append("-".repeat(totalWidth)).append("\n");
    sb.append(String.format("%" + (widths[0] + widths[1] +
    widths[2] + widths[3] + 4) + "s %" + widths[1] + ".2f€\n",
    "Gesamt:", summe));
    sb.append(String.format("%" + (widths[0] + widths[1] +
    widths[2] + widths[3] + 4) + "s %" + widths[1] + ".2f€\n\n",
    "Monatlich:", monatlich));
}

private static void printEinnahmen(StringBuilder sb,
ArrayList<Eintrag> eintraege) {
    String[] headers = {"Datum", "Betrag", "Kategorie",
    "Bezeichnung"};
    int[] widths = new int[headers.length];
    for (int i = 0; i < headers.length; i++) widths[i] =
    headers[i].length();

    double summe = 0;
    for (Eintrag e : eintraege) {
        if (!e.istEinnahme) continue;
        widths[0] = Math.max(widths[0], 12);
        widths[1] = Math.max(widths[1], String.format("%.2f€",
    e.betrag).length());
        widths[2] = Math.max(widths[2], e.kategorie.length());
        widths[3] = Math.max(widths[3], e.bezeichnung.length());
        summe += e.betrag;
    }
}

```

```

        for (int i = 0; i < headers.length; i++) {
            widths[i] = Math.max(widths[i], headers[i].length());
        }

        String headerFmt = "%-" + widths[0] + "s %" + widths[1] + "s
        %-" + widths[2] + "s %-" + widths[3] + "s\n";
        String rowFmt    = "%-" + widths[0] + "s %" + widths[1] +
        ".2f€ %-" + widths[2] + "s %-" + widths[3] + "s\n";
        int totalWidth = widths[0] + widths[1] + widths[2] + widths[3]
        + 3;

        sb.append(" EINNAHMEN\n");
        sb.append("-".repeat(totalWidth)).append("\n");
        sb.append(String.format(headerFmt, (Object[]) headers));
        sb.append("-".repeat(totalWidth)).append("\n");

        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                sb.append(String.format(rowFmt, e.datum, e.betrag,
                e.kategorie, e.bezeichnung));
            }
        }
        sb.append("-".repeat(totalWidth)).append("\n");
        sb.append(String.format("%" + (widths[0] + widths[1] +
        widths[2] + 3) + "s %" + widths[1] + ".2f€\n\n",
        "Gesamt Einnahmen:", summe));
    }

    private static void printFesteEinnahmen(StringBuilder sb,
    ArrayList<FestWert> festeWerte) {
        String[] headers = {"Datum", "Betrag", "Kategorie",
        "Bezeichnung", "Periode"};
        int[] widths = new int[headers.length];
        for (int i = 0; i < headers.length; i++) widths[i] =
        headers[i].length();

        double summe = 0, monatlich = 0;
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) continue;
            widths[0] = Math.max(widths[0], 12);
            widths[1] = Math.max(widths[1], String.format("%.2f€",
            f.betrag).length());
            widths[2] = Math.max(widths[2], f.kategorie.length());
            widths[3] = Math.max(widths[3], f.bezeichnung.length());
            widths[4] = Math.max(widths[4], String.format("%d Monate",
            f.periode).length());
            summe += f.betrag;
            monatlich += f.betrag / f.periode;
        }
        for (int i = 0; i < headers.length; i++) {

```



```

        widths[i] = Math.max(widths[i], headers[i].length());
    }

    String headerFmt = "%-" + widths[0] + "s %" + widths[1] + "s
    %-" + widths[2] + "s %-" + widths[3] + "s %" + widths[4] + "s\n";
    String rowFmt    = "%-" + widths[0] + "s %" + widths[1] +
    ".2f€ %-" + widths[2] + "s %-" + widths[3] + "s %d Monate\n";
    int totalWidth = widths[0] + widths[1] + widths[2] + widths[3]
    + widths[4] + 4;

    sb.append(" FESTE EINNAHMEN\n");
    sb.append("-".repeat(totalWidth)).append("\n");
    sb.append(String.format(headerFmt, (Object[]) headers));
    sb.append("-".repeat(totalWidth)).append("\n");

    for (FestWert f : festeWerte) {
        if (f.istEinnahme) {
            sb.append(String.format(rowFmt, f.datum, f.betrag,
            f.kategorie, f.bezeichnung, f.periode));
        }
    }
    sb.append("-".repeat(totalWidth)).append("\n");
    sb.append(String.format("%" + (widths[0] + widths[1] +
    widths[2] + widths[3] + 4) + "s %" + widths[1] + ".2f€\n",
    "Gesamt:", summe));
    sb.append(String.format("%" + (widths[0] + widths[1] +
    widths[2] + widths[3] + 4) + "s %" + widths[1] + ".2f€\n\n",
    "Monatlich:", monatlich));
}
}package gui;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;

public class Uebersicht {

    public static void zeigeAusgaben(ArrayList<Eintrag> eintraege) {
        System.out.println("Ausgaben:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|   Bezeichnung |   Betrag   |");
        System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s| %8.2f€ |\n
n",count, e.datum,  e.kategorie, e.bezeichnung, e.betrag);
            }
        }
    }
}

```

```

        summe += e.betrag;
    }
    count++;
}

System.out.println("=====
=====");
    System.out.printf("|Gesamt Ausgaben: %54.2f€ |\n", summe);

System.out.println("=====
=====");

    }
    public static void zeigefesteAusgaben(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Ausgaben:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum      |          Kategorie
|          Bezeichnung      | Periode(Monate) | Betrag |");

System.out.println("-----
-----");

        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                int countE=0;
                while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                    countE++;
                }
                for (int i =0;i<countE;i++) {

                    System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                    summe += f.betrag;
                }
                monatsbeitrag +=f.betrag/f.periode;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt feste Ausgaben: %77.2f€ |\n",
summe);
        System.out.printf("|Gesamt feste Ausgaben/Monat: %71.2f€ |\n",
monatsbeitrag);

```

```

System.out.println("=====
=====");

}

    public static void zeigeEinnahmen(ArrayList<Eintrag> eintraege) {
        System.out.println("Einnahmen:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|          Bezeichnung      |          Betrag |");

System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %25s | %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);
                summe += e.betrag;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt Einnahmen: %64.2f€ |\n", summe);

System.out.println("=====
=====");
    }

    public static void zeigefesteEinnahmen(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Einnahmen:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|          Bezeichnung | Periode(Monate) |          Betrag |");

System.out.println("-----
-----");
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                int countE=0;
                while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                    countE++;
                }
            }
        }
    }

```

```

        for (int i =0;i<countE;i++) {

            System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
            summe += f.betrag;
        }
        monatsbeitrag +=f.betrag/f.periode;
    }
    count++;
}

System.out.println("=====
=====");
    System.out.printf("|Gesamt feste Einnahmen: %66.2f€ |\n",
summe);
    System.out.printf("|Gesamt feste Einnahmen/Monat: %60.2f€ |\n",
monatsbeitrag);

System.out.println("=====
=====");

}

```

```

    public static void zeigeBilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        zeigeAusgaben(eintraege);
        zeigefesteAusgaben(festeWerte);
        zeigeEinnahmen(eintraege);
        zeigefesteEinnahmen(festeWerte);
        double bilanzwert=0.0;
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                bilanzwert +=f.betrag;
            }
            else {
                bilanzwert -= f.betrag;
            }
        }
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                bilanzwert +=e.betrag;
            }
            else {
                bilanzwert -= e.betrag;
            }
        }

        System.out.println("=====
=====");
    }

```

```

        System.out.printf("| Bilanzwert: %30.2f€ |\n",bilanzwert);
System.out.println("=====");
    }

    public static void zeigeBalkendiagrammAusgaben(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte) {
        // Sammle Ausgaben pro Kategorie
        var katSum = new java.util.HashMap<String, Double>();

        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                katSum.merge(e.kategorie, e.betrag, Double::sum);
            }
        }
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                katSum.merge(f.kategorie, f.betrag, Double::sum);
            }
        }

        if (katSum.isEmpty()) {
            System.out.println("Keine Ausgaben vorhanden.");
            return;
        }

        double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
        int balkenLaenge = 20; // Länge des Balkens

        System.out.println("\nAusgaben nach Kategorie:");
        katSum.entrySet().stream()
            .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
            .forEach(e -> {
                int anzahl = (int) (e.getValue() / maxBetrag *
balkenLaenge);
                String balken = "█".repeat(anzahl) + "
".repeat(balkenLaenge - anzahl);
                System.out.printf("%s %s %8.2f €\n", balken,
String.format("%-12s", e.getKey()), e.getValue());
            });
    }

    public static final String RESET = "\u001B[0m";
    public static final String GRUEN = "\u001B[32m";

```

```

public static final String GELB = "\u001B[33m";
public static final String ROT = "\u001B[31m";

public static void zeigeMonatsBalken(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    LocalDate jetzt = LocalDate.now();
    int jahr = jetzt.getYear();
    int monat = jetzt.getMonthValue();

    var katSum = new java.util.HashMap<String, Double>();

    // Normale Einträge
    for (Eintrag e : eintraege) {
        if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
            katSum.merge(e.kategorie, e.betrag, Double::sum);
        }
    }

    // Feste Werte (wiederkehrend)
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            LocalDate naechste = f.datum;
            while (naechste.isBefore(jetzt.plusMonths(1))) {
                if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                    katSum.merge(f.kategorie, f.betrag,
Double::sum);
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }
    }

    if (katSum.isEmpty()) {
        System.out.println("Keine Ausgaben im aktuellen Monat.");
        return;
    }

    double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
    int balkenLaenge = 25;

    System.out.printf("\n=== %s %d ===\n", jetzt.getMonth(),
jahr);

```

```

        System.out.println("Budget-Warnung: " + ROT + "Überschritten"
+ RESET + ", " + GELB + "Kritisch" + RESET + ", " + GRUEN + "OK" +
RESET);

```

```

        katSum.entrySet().stream()
            .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
            .forEach(e -> {
                double ausgegeben = e.getValue();
                double budget = BudgetManager.getBudget(e.getKey()); // <-
0.0 wenn nicht vorhanden
                int anzahl = (int) (ausgegeben / maxBetrag *
balkenLaenge);
                String balken;
                String farbe;

                if (budget > 0 && ausgegeben > budget) {
                    balken = ROT + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                    farbe = ROT;
                } else if (budget > 0 && ausgegeben > budget * 0.9) {
                    balken = GELB + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                    farbe = GELB;
                } else {
                    balken = GRUEN + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                    farbe = (budget == 0) ? RESET : GRUEN; // Grau, wenn
kein Budget
                }

                String budgetStr = (budget == 0)
                    ? String.format("%8.2f €", ausgegeben)
                    : String.format("%8.2f / %.2f €", ausgegeben, budget);

                System.out.printf("%s %s%s %s\n",
                    balken + " ".repeat(balkenLaenge - Math.min(anzahl,
balkenLaenge)),
                    farbe,
                    String.format("%-15s", e.getKey()),
                    budgetStr + RESET);
            });
    }

```

```

public static void zeigeGefiltert(ArrayList<Eintrag>

```

```

    eintraege, ArrayList<FestWert> festeWerte, String kategorie) {
        System.out.println("Einträge in Kategorie: " + kategorie);
        System.out.println("| Datum      | Betrag   | Kategorie  
| Bezeichnung | Art      | Ausgabe/Einnahme|");

        System.out.println("-----  
-----");

        String art="";
        boolean gefunden = false;
        for (Eintrag e : eintraege) {
            if (e.kategorie.equals(kategorie)) {
                art= e.istEinnahme ? "Einnahme":"Ausgabe";
                System.out.printf("| %10s | %8.2f | %17s | %15s |  
Eintrag    |%16s|\n",
                    e.datum, e.betrag, e.kategorie, e.bezeichnung,
                    art);
                gefunden = true;
            }
        }
        for (FestWert f: festeWerte) {
            if (f.kategorie.equals(kategorie)) {
                art= f.istEinnahme ? "Einnahme":"Ausgabe";
                System.out.printf("| %10s | %8.2f | %17s | %15s |  
fester Wert|%16s|\n",
                    f.datum, f.betrag, f.kategorie, f.bezeichnung,
                    art);
                gefunden = true;
            }
        }

        if (!gefunden) {
            System.out.println("=====");
            System.out.println("      Keine Einträge in dieser  
Kategorie!");
            System.out.println("=====");
        }
    }

    public static void zeigeSortiertNachBetrag(ArrayList<Eintrag>
    eintraege, ArrayList<FestWert> festeWerte, boolean absteigend) {
        ArrayList<Eintrag> kopieEintraege = new
        ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
        ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = absteigend ?
            Comparator.comparingDouble((Eintrag e) ->

```



```

e.betrag).reversed() : Comparator.comparingDouble((Eintrag e) ->
e.betrag);

        Comparator<FestWert> compF = absteigend ?
            Comparator.comparingDouble((FestWert f) ->
f.betrag).reversed() : Comparator.comparingDouble((FestWert f) ->
f.betrag);

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }

    public static void zeigeSortiertNachDatum(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean a) {

        ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = Comparator.comparing(e -> e.datum,

            Comparator.nullsLast(Comparator.naturalOrder()));
        Comparator<FestWert> compF = Comparator.comparing(f ->
f.datum,
            Comparator.nullsLast(Comparator.naturalOrder()));

        if (a) {
            compE = compE.reversed();
            compF = compF.reversed();
        }

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }

    public static void zeigeBevorstehendeAusgaben(ArrayList<FestWert>
festeWerte) {

        int tage= IO.readInt("Geben Sie die Tage des Intervalls an:
");

```

```

        LocalDate now=LocalDate.now(); // LocalDate.of(2025,12,1);

System.out.println("=====
=====");
        System.out.println("
Bevorstehende Ausgaben");

System.out.println("=====
=====");
        System.out.println("| Fälligkeitsdatum |      Kategorie
|      Bezeichnung      | Periode(Monate) |      Betrag |");

System.out.println("-----
-----");
        double summe=0.0;

        for (FestWert f :festeWerte)

            if (!f.istEinnahme) {
                int countE=0;
                int countA=0;

                while (!
now.plusDays(tage).isBefore(f.datum.plusMonths(countE*f.periode))) {

                    countE++;
                }
                while
(now.isAfter(f.datum.plusMonths(countA*f.periode))) {
//!now.isBefore(f.datum.plusMonths(countA*f.periode))---heute nicht
dabei
                    countA++;
                }
                for (int i =countA;i<countE;i++) {
                    System.out.printf("|%17s | %20s | %25s| %17d|
%8.2f€ |\n",f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                    summe +=f.betrag;
                }

            }

System.out.println("=====
=====");
        if(summe !=0.0) {
            System.out.printf("Die zu erwartende aufzubringende Summe in
den nächsten %"+4+"d Tagen ist: %27.2f€ |\n",tage,summe);
        }
        else {
            System.out.printf("In den nächsten %d Tagen erwartet Dich

```

```

keine Ausgabe fester Werte!\n",tage);
    }

    System.out.println("=====
=====");

    }

}

```

Klasse: Verwaltung

```

package HaushaltskassenAppGUI;

import javax.swing.SwingUtilities;

public class Verwaltung {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
HaushaltsAppGUI().setVisible(true));
    }
}package HaushaltskassenApp_zwei;

import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Scanner;
import java.util.Locale;

public class Verwaltung {

    public static void main(String[] args) {
        // Prüfe ob GUI gestartet werden soll
        if (args.length > 0 && args[0].equals("--gui")) {
            // Starte JavaFX GUI
            HaushaltskassenApp.main(args);
            return;
        }

        // Sonst: Starte Konsolen-Version
        Scanner sc =new Scanner(System.in);
        BudgetManager.ladeBudgets();
        SparzielManager.ladeSparziele();
        int auswahl;
        int festeWertAuswahl;

        ArrayList<Eintrag> eintraege = new ArrayList<>();
    }
}

```

```

        ArrayList<FestWert> festeWerte = new ArrayList<>();
        String
        Fehlertext="=====\n
        Fehlerhafte Eingabe!!!!\n
        n=====\\n";
        try (Scanner fileScanner = new Scanner(new
        FileReader("eintraege.txt"))) {
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                String[] parts = line.split("#");
                if (parts.length == 5) {
                    Eintrag e = new Eintrag(
                        parts[0].trim(),
                        Double.parseDouble(parts[1].trim()),
                        Boolean.parseBoolean(parts[2].trim()),
                        LocalDate.parse(parts[3].trim()),
                        parts[4].trim()
                    );
                    eintraege.add(e);
                }
            }
            System.out.println("Einträge aus eintraege.txt geladen.");
        } catch (FileNotFoundException e) {
            System.out.println("Keine gespeicherten Einträge gefunden,
            starte mit leerer Liste.");
        } catch (Exception e) {
            System.err.println("Fehler beim Laden: " +
            e.getMessage());
        }
        festeWerte = new ArrayList<>();
        try (Scanner fileScanner = new Scanner(new
        FileReader("festeWerte.txt"))) {
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                String[] parts = line.split("#");
                if (parts.length == 6) {
                    FestWert f = new FestWert(
                        parts[0].trim(),
                        Double.parseDouble(parts[1].trim()),
                        Boolean.parseBoolean(parts[2].trim()),
                        LocalDate.parse(parts[3].trim()),
                        parts[4].trim(),
                        Integer.parseInt(parts[5].trim())
                    );
                    festeWerte.add(f);
                }
            }
            System.out.println("Feste Werte aus festeWerte.txt
            geladen.");
        } catch (FileNotFoundException e) {

```

```

        System.out.println("Keine festen Werte gefunden, starte
mit leerer Liste.");
    } catch (Exception e) {
        System.err.println("Fehler beim Laden: " +
e.getMessage());
    }

    SparzielManager.automatischeZufuehrungBeimStart(festeWerte);
    Eintrag eTesch1=new
Eintrag("Kühlschrank",588.95,false,LocalDate.of(2025,10,27),"Einmal-
Anschaffung");
    Eintrag eTesch2=new
Eintrag("Flohmarkt",345.11,true,LocalDate.of(2025,10,28),"Krimskrams")
;
    Eintrag eTesch3=new
Eintrag("Flohmarkt",12.50,false,LocalDate.of(2025,10,28),"Krimskrams")
;
//    eintraege.add(eTesch1);
//    eintraege.add(eTesch2);
//    eintraege.add(eTesch3);

    FestWert fWTesch1=new
FestWert("GEZ",58.95,false,LocalDate.of(2025, 9,
1),"Rundfunkgebühren",3);
    FestWert fWTesch2=new
FestWert("Pacht",1500,true,LocalDate.of(2025, 1,
1),"Mieteinnahmen",12);
//    festeWerte.add(fWTesch1);
//    festeWerte.add(fWTesch2);

    // System.out.println(eintraege);
    while (true) {

System.out.println("=====");
        System.out.println("                HAUSHALTSKASSEN-APP");

System.out.println("=====");
        System.out.println("Was wünschen Sie?");

System.out.println("=====");
        System.out.printf(" 1: Eintrag hinzufügen\n 2: Eintrag
löschen\n 3: Eintrag ändern\n 4: Feste Werte\n 5: Übersicht der
Ausgaben\n 6: Übersicht der Einnahmen\n 7: Gegenüberstellung der
Einnahmen und Ausgaben\n 8: Gefilterte Einträge\n 9: Sortierte
Ausgabe\n10: Bevorstehende Ausgaben fester Werte in den von Ihnen
eingegebenen Tagen\n11: Balkendiagramm Ausgaben\n12: Budget-Manager\
n13: Statistik\n14: Sonstiges\n15: Beenden\n");
        auswahl=sc.nextInt();

```

```
System.out.println("=====");
```

```
    if (auswahl==1) {
        Eintrag e = new Eintrag(
            IO.readString("Bezeichnung: "),
            IO.readDouble("Betrag: "),
            IO.readBoolean("Einnahme?(j/n): "),
            IO.readDate("Datum:"),
            IO.readString("Kategorie: ")
        );
        Eintrag.fuegeEintragHinzu(eintraege, e);
        // System.out.println(eintraege);
    }
    else if (auswahl==2) {
        int index = IO.readInt("Geben Sie den Index an: ");
        Eintrag.loescheEintrag(eintraege, index);
    }
    else if (auswahl==3) {
        int index = IO.readInt("Geben Sie den Index an: ");
        if (index >= 0 && index < eintraege.size()) {
            eintraege.set(index, Eintrag.aendereEintrag());
        } else {
            System.out.println(Fehlertext);
        }
    }

    }
    else if (auswahl==4) {
        boolean istAuswahl=true;
        while (istAuswahl) {
```

```
System.out.println("=====");
        System.out.println("Was wollen Sie mit den
festen Werten machen?");
```

```
System.out.println("=====");
        System.out.printf("1: Festen Wert hinzufügen\
n2: Festen Wert löschen\n3: Festen Wert ändern\n4: Zurück zum
Hauptmenu\n");
```

```
        festeWertAuswahl=sc.nextInt();
        switch (festeWertAuswahl) {
            case 1:
                festeWerte.add(new FestWert(
                    IO.readString("Bezeichnung: "),
                    IO.readDouble("Betrag: "),
                    IO.readBoolean("Einnahme?(j/n): "),
                    IO.readDate("Datum: "),
                    IO.readString("Kategorie: "),
                    IO.readInt("Periode (Monate): ")
                ));
```

```

        break;
    case 2:
        int index = IO.readInt("Index: ");
        if (index >= 0 && index <
festeWerte.size()) {
            festeWerte.remove(index);
        } else {
            System.out.println(Fehlertext);
        }
        break;
    case 3:
        index = IO.readInt("Index: ");
        if (index >= 0 && index <
festeWerte.size()) {
            festeWerte.set(index, new FestWert(
                IO.readString("Bezeichnung: "),
                IO.readDouble("Betrag: "),
                IO.readBoolean("Einnahme?(j/n):
"),
                IO.readDate("Datum: "),
                IO.readString("Kategorie: "),
                IO.readInt("Periode (Monate): ")
            ));
        } else {
            System.out.println(Fehlertext);
        }
        break;
    case 4:
        istAuswahl=false;
        break;
    default:
        System.out.println(Fehlertext);
}

    }
}
else if (auswahl == 5) {
    Uebersicht.zeigeAusgaben(eintraege);
    Uebersicht.zeigefesteAusgaben(festeWerte);
} else if (auswahl == 6) {
    Uebersicht.zeigeEinnahmen(eintraege);
    Uebersicht.zeigefesteEinnahmen(festeWerte);
} else if (auswahl == 7) {
    Uebersicht.zeigeBilanz(eintraege, festeWerte);
}
else if (auswahl == 8) {

System.out.println("=====");

```

```

        System.out.println("                Gefilterete Ausgabe");
System.out.println("=====");
        String filter=I0.readString("Geben Sie die Kategorie
an: ");
        Uebersicht.zeigeGefiltert(eintraege,festeWerte,
filter);
    }else if (auswahl == 9) {
System.out.println("=====");
        System.out.println("                Sortierte Ausgabe");
System.out.println("=====");
        int sortAuswahl=I0.readInt("Sortieren nach Betrag:
(1)\nSortieren nach Datum: (2)\n");
        switch (sortAuswahl) {
            case 1: {
                boolean isAbsteigend=I0.readBoolean("Nach
Betrag absteigend? (j/n): ");
                Uebersicht.zeigeSortiertNachBetrag(eintraege,
festeWerte,isAbsteigend);
                break;
            }
            case 2: {
                boolean isAbsteigend=I0.readBoolean("Nach Datum
absteigend? (j/n): ");
                Uebersicht.zeigeSortiertNachDatum(eintraege,
festeWerte, isAbsteigend);
                break;
            }
            default:
                System.out.println(Fehlertext);
        }
    }else if (auswahl==10) {
        Uebersicht.zeigeBevorstehendeAusgaben(festeWerte);
    }
    else if (auswahl == 11) {
        Uebersicht.zeigeBalkendiagrammAusgaben(eintraege,
festeWerte);
        Uebersicht.zeigeMonatsBalken(eintraege, festeWerte);
    }

    else if (auswahl==12) {
        budgetMenue();
    }
    else if (auswahl==13) {
        boolean istStatistik=true;
        System.out.printf("1: Statistik mit allen Kategorien\
n2: Satistik mit einer von Ihnen gewählten Auswahl an Kategorien\n3:
Statistik mit allen Kategorien im letzten Monat\n4: Statistik mit

```


einer von Ihnen gewählten Auswahl an Kategorien im letzten Monat\n5:
Statistik mit allen Kategorien im vorletzten Monat\n6: Statistik mit
einer von Ihnen gewählten Auswahl an Kategorien im vorletzten Monat\
n7: Vergleich der Mittelwerte der letzten 2 Monate\n8: zurück\n");

```
        while (istStatistik) {
            int statauswahl=IO.readInt("Wahl: ");
            switch(statauswahl) {
                case 1:{
                    Statistik.printMittelwert(eintraege,0);

Statistik.berechneStandardabweichung(eintraege,0);
                    break;
                }
                case 2:{

Statistik.berechneMitKategorie(eintraege,0);
                    break;
                }
                case 3:{
                    Statistik.printMittelwert(eintraege,1);

Statistik.berechneStandardabweichung(eintraege,1);
                    break;
                }
                case 4:{

Statistik.berechneMitKategorie(eintraege,1);
                    break;
                }
                case 5:{
                    Statistik.printMittelwert(eintraege,2);

Statistik.berechneStandardabweichung(eintraege,2);
                    break;
                }
                case 6:{

Statistik.berechneMitKategorie(eintraege,2);
                    break;
                }
                case 7:{
                    berechneVergleich(eintraege);
                    break;
                }
                case 8:{
                    istStatistik=false;
                    break;
                }
                default:
                    System.out.println(Fehlertext);
            }
        }
    }
}
```

```

        }
    }
    else if (auswahl==14) {
        Sonstiges.zeigeMenue(eintraege, festeWerte);
    }

    else if (auswahl==15) {

System.out.println("=====");
        System.out.println("Vielen Dank! Bis zum nächsten
Mal!");

System.out.println("=====");
//        SparzielManager.speichereFesteWerte(festeWerte);
        Schlussspruch.schreibeSchluss();
        break;
    }

    else {
//
System.out.println("=====");
//        System.out.println("Fehlerhafte
Eingabe!!!!");
//
System.out.println("=====");
        System.out.printf(Fehlertext);
    }
}
try (FileWriter writer = new FileWriter("eintraege.txt")) {
    for (Eintrag e : eintraege) {
        writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s%n",
            e.bezeichnung,
            e.betrag,
            e.istEinnahme,
            e.datum,
            e.kategorie
        ));
    }
    System.out.println("Einträge in eintraege.txt
gespeichert.");
} catch (IOException e) {
    System.err.println("Fehler beim Speichern: " +
e.getMessage());
}

try (FileWriter writer = new FileWriter("festeWerte.txt")) {
    for (FestWert f : festeWerte) {

```

```

        writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%s#%d%n",
            f.bezeichnung,
            f.betrag,
            f.istEinnahme,
            f.datum,
            f.kategorie,
            f.periode
        ));
    }
    System.out.println("Feste Werte in festeWerte.txt
gespeichert.");
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern: " +
e.getMessage());
    }

    sc.close();

}

private static void berechneVergleich(ArrayList<Eintrag>
eintraege) {
    double mit1=Math.round(Statistik.berechneMittelwert(eintraege,
1)*100)/100.0;
    double mit2=Math.round(Statistik.berechneMittelwert(eintraege,
2)*100)/100.0;
    String vergleich= "Die Ausgaben blieben zufälligerweise
gleich!";
    double diff=0.0;
    if (mit1 != mit2) {
        diff=mit1>mit2 ? mit1-mit2 : mit2-mit1;
        vergleich="Die Ausgaben wurden also %.2f€"+(mit1>mit2 ? "
mehr!": " weniger!");
    }

    System.out.println("=====");
    System.out.println("Vergleich der letzten Mittelwerte:");
    System.out.printf ("vorletzter Monat: %28.2f€\n",mit2);
    System.out.printf ("letzter Monat: %31.2f€\n",mit1);
    System.out.printf(vergleich+"\n",diff);

    System.out.println("=====");
}

private static void budgetMenue() {
    Scanner sc = new Scanner(System.in);
    while (true) {

```

```

System.out.println("=====");
System.out.println("          BUDGET-VERWALTUNG");

System.out.println("=====");
System.out.println("1: Budget eingeben/ändern");
System.out.println("2: Budget löschen");
System.out.println("3: Alle Budgets anzeigen");
System.out.println("4: Zurück zum Hauptmenü");
System.out.print("Wahl: ");
int wahl = sc.nextInt();
sc.nextLine(); // Zeilenumbruch

if (wahl == 1) {
    String kat = IO.readString("Kategorie: ");
    double betrag = IO.readDouble("Budget (0 = löschen): ");
    BudgetManager.setzeBudget(kat, betrag);
}
else if (wahl == 2) {
    String kat = IO.readString("Kategorie zum Löschen: ");
    BudgetManager.loescheBudget(kat);
}
else if (wahl == 3) {
    BudgetManager.zeigeAlleBudgets();
}
else if (wahl == 4) {
    break;
}
else {
    System.out.println("Ungültige Eingabe!");
}
}

}

```

Package: HaushaltskassenApp_Mit_Gui_Versuch2

Klasse: Eintrag

```
package HaushaltskassenApp_Mit_Gui_Versuch2;
```

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Objects;

```

```
public class Eintrag {

    public String bezeichnung;
    public double betrag;
    public boolean istEinnahme;
    public LocalDate datum;
    public String kategorie;

    public Eintrag(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie) {
        super();
        this.bezeichnung = bezeichnung;
        this.betrag = betrag;
        this.istEinnahme = istEinnahme;
        this.datum = datum;
        this.kategorie = kategorie;
    }

    public String getBezeichnung() {
        return bezeichnung;
    }

    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }

    public double getBetrag() {
        return betrag;
    }

    public void setBetrag(double betrag) {
        this.betrag = betrag;
    }

    public boolean isEinnahme() {
        return istEinnahme;
    }

    public void setIstEinnahme(boolean istEinnahme) {
        this.istEinnahme = istEinnahme;
    }

    public LocalDate getDatum() {
        return datum;
    }

    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }
}
```

```

    }

    public String getKategorie() {
        return kategorie;
    }

    public void setKategorie(String kategorie) {
        this.kategorie = kategorie;
    }

    @Override
    public int hashCode() {
        return Objects.hash(betrag, bezeichnung, datum, istEinnahme,
kategorie);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Eintrag other = (Eintrag) obj;
        return Double.doubleToLongBits(betrag) ==
Double.doubleToLongBits(other.betrag)
            && Objects.equals(bezeichnung, other.bezeichnung) &&
Objects.equals(datum, other.datum)
            && istEinnahme == other.istEinnahme &&
Objects.equals(kategorie, other.kategorie);
    }

    @Override
    public String toString() {
        return "Eintrag [bezeichnung=" + bezeichnung + ", betrag=" +
betrag + ", istEinnahme=" + istEinnahme
            + ", datum=" + datum + ", kategorie=" + kategorie +
"]";
    }

    public static void loescheEintrag(ArrayList<Eintrag> ae,int index)
{
    if (index >= 0 && index < ae.size()) {
        ae.remove(index);
    } else {
        System.out.println("Ungültiger Index!");
    }
}

```

```

    }

    public static Eintrag aendereEintrag() {
        return new Eintrag(
            IO.readString("Bezeichnung: "),
            IO.readDouble("Betrag: "),
            IO.readBoolean("Einnahme?(j/n): "),
            IO.readDate("Datum: "),
            IO.readString("Kategorie: ")
        );
    }

    public static void fuegeEintragHinzu(ArrayList<Eintrag>
ae ,Eintrag e) {
        ae.add(e);
    }

}package HaushaltskassenApp_zwei;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Objects;

public class Eintrag {

    public String bezeichnung;
    public double betrag;
    public boolean istEinnahme;
    public LocalDate datum;
    public String kategorie;

    public Eintrag(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie) {
        super();
        this.bezeichnung = bezeichnung;
        this.betrag = betrag;
        this.istEinnahme = istEinnahme;
        this.datum = datum;
        this.kategorie = kategorie;
    }

    public String getBezeichnung() {
        return bezeichnung;
    }

    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }

    public double getBetrag() {
        return betrag;
    }

```

```

    }

    public void setBetrag(double betrag) {
        this.betrag = betrag;
    }

    public boolean istEinnahme() {
        return istEinnahme;
    }

    public void setIstEinnahme(boolean istEinnahme) {
        this.istEinnahme = istEinnahme;
    }

    public LocalDate getDatum() {
        return datum;
    }

    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }

    public String getKategorie() {
        return kategorie;
    }

    public void setKategorie(String kategorie) {
        this.kategorie = kategorie;
    }

    @Override
    public int hashCode() {
        return Objects.hash(betrag, bezeichnung, datum, istEinnahme,
kategorie);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Eintrag other = (Eintrag) obj;
        return Double.doubleToLongBits(betrag) ==
Double.doubleToLongBits(other.betrag)
            && Objects.equals(bezeichnung, other.bezeichnung) &&
Objects.equals(datum, other.datum)
    }

```



```

        && istEinnahme == other.istEinnahme &&
Objects.equals(kategorie, other.kategorie);
    }

    @Override
    public String toString() {
        return "Eintrag [bezeichnung=" + bezeichnung + ", betrag=" +
betrag + ", istEinnahme=" + istEinnahme
        + ", datum=" + datum + ", kategorie=" + kategorie +
        + "]";
    }

    public static void loescheEintrag(ArrayList<Eintrag> ae,int index)
{
    if (index >= 0 && index < ae.size()) {
        ae.remove(index);
    } else {
        System.out.println("Ungültiger Index!");
    }
}

    public static Eintrag aendereEintrag() {
        return new Eintrag(
            IO.readString("Bezeichnung: "),
            IO.readDouble("Betrag: "),
            IO.readBoolean("Einnahme?(j/n): "),
            IO.readDate("Datum: "),
            IO.readString("Kategorie: ")
        );
    }
    public static void fuegeEintragHinzu(ArrayList<Eintrag>
ae ,Eintrag e) {
        ae.add(e);
    }
}

```

Klasse: FestWert

```
package HaushaltskassenApp_Mit_Gui_Versuch2;
```

```
import java.time.LocalDate;
```

```
public class FestWert extends Eintrag{
    public int periode;
```

```
    public FestWert(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie,
```

```

        int periode) {
            super(bezeichnung, betrag, istEinnahme, datum, kategorie);
            this.periode = periode;
        }

        public int getPeriode() {
            return periode;
        }

        public void setPeriode(int periode) {
            this.periode = periode;
        }

        @Override
        public String toString() {
            return "FestWert [periode=" + periode + ", bezeichnung=" +
bezeichnung + ", betrag=" + betrag + ", istEinnahme="
            + istEinnahme + ", datum=" + datum + ", kategorie=" +
kategorie + "]";
        }
    }
}package HaushaltskassenApp_zwei;

import java.time.LocalDate;

public class FestWert extends Eintrag{
    public int periode;

    public FestWert(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie,
        int periode) {
        super(bezeichnung, betrag, istEinnahme, datum, kategorie);
        this.periode = periode;
    }

    public int getPeriode() {
        return periode;
    }

    public void setPeriode(int periode) {
        this.periode = periode;
    }

    @Override
    public String toString() {
        return "FestWert [periode=" + periode + ", bezeichnung=" +
bezeichnung + ", betrag=" + betrag + ", istEinnahme="
        + istEinnahme + ", datum=" + datum + ", kategorie=" +
kategorie + "]";
    }
}

```

```
}
```

Klasse: IO

```
package HaushaltskassenApp_Mit_Gui_Versuch2;
```

```
import java.time.LocalDate;  
import java.util.Scanner;
```

```
public class IO {
```

```
    public static int readInt(String prompt) {  
        System.out.print(prompt);  
        Scanner scan = new Scanner(System.in);  
        return scan.nextInt();  
    }  
    public static double readDouble(String prompt) {  
        System.out.print(prompt);  
        Scanner scan = new Scanner(System.in);  
        return scan.nextDouble();  
    }  
    public static String readString(String prompt) {  
        System.out.print(prompt);  
        Scanner scan = new Scanner(System.in);  
        return scan.nextLine();  
    }  
    public static boolean readBoolean(String prompt) {  
        Scanner scan = new Scanner(System.in);  
        while (true) {  
            System.out.print(prompt);  
            String input = scan.nextLine().trim().toLowerCase();  
            if (input.equals("j") || input.equals("ja") ||  
input.equals("y") || input.equals("yes")) {  
                return true;  
            } else if (input.equals("n") || input.equals("nein") ||  
input.equals("no")) {  
                return false;  
            } else {  
                System.out.println("Ungültige Eingabe! Bitte 'j',  
'ja', 'n' oder 'nein' eingeben.");  
            }  
        }  
    }  
}
```

```

    }
}
public static LocalDate readDate(String prompt) {
    System.out.println(prompt);
    try {
        return LocalDate.of(
            IO.readInt("Jahr: "),
            IO.readInt("Monat: "),
            IO.readInt("Tag: ")
        );
    } catch (Exception e) {
        System.out.println("Ungültiges Datum!");
        return LocalDate.now();
    }
}
}

```

Klasse: Uebersicht

```
package HaushaltskassenApp_Mit_Gui_Versuch2;
```

```
import java.util.ArrayList;
import java.util.Comparator;
```

```
public class Uebersicht {

    public static void zeigeAusgaben(ArrayList<Eintrag> eintraege) {
        System.out.println("Ausgaben:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
| Bezeichnung | Betrag |");

System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s| %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);
                summe += e.betrag;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt Ausgaben: %54.2f€ |\n", summe);

System.out.println("=====
=====");
    }
}

```

```

=====");

    }
    public static void zeigeFesteAusgaben(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Ausgaben:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|   Bezeichnung |   Periode(Monate) |   Betrag   |");
        System.out.println("-----
-----");
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s| %17d|
%8.2f€ |\n",count,f.datum, f.kategorie, f.bezeichnung, f.periode,
f.betrag);
                summe += f.betrag;
                monatsbeitrag +=f.betrag/f.periode;
            }
            count++;
        }

        System.out.println("=====
=====");
        System.out.printf("|Gesamt feste Ausgaben: %7.2f€ |\n",
summe);
        System.out.printf("|Gesamt feste Ausgaben/Monat: %61.2f€ |\n",
monatsbeitrag);

        System.out.println("=====
=====");

    }

    public static void zeigeEinnahmen(ArrayList<Eintrag> eintraege) {
        System.out.println("Einnahmen:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|   Bezeichnung |   Betrag   |");
        System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s | %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);

```

```

        summe += e.betrag;
    }
    count++;
}

System.out.println("=====
=====");
    System.out.printf("|Gesamt Einnahmen: %54.2f€ |\n", summe);

System.out.println("=====
=====");
}

    public static void zeigefesteEinnahmen(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Einnahmen:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum |          Kategorie
| Bezeichnung | Periode(Monate) | Betrag |");

System.out.println("-----
-----");
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s| %17d|
%8.2f€ |\n",count, f.datum, f.kategorie, f.bezeichnung, f.periode,
f.betrag);
                summe += f.betrag;
                monatsbeitrag +=f.betrag/f.periode;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt feste Einnahmen: %66.2f€ |\n",
summe);
        System.out.printf("|Gesamt feste Einnahmen/Monat: %60.2f€ |\n",
monatsbeitrag);

System.out.println("=====
=====");

    }

    public static void zeigeBilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {

```

```

        zeigeAusgaben(eintraege);
        zeigefesteAusgaben(festeWerte);
        zeigeEinnahmen(eintraege);
        zeigefesteEinnahmen(festeWerte);
        double bilanzwert=0.0;
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                bilanzwert +=f.betrag;
            }
            else {
                bilanzwert -= f.betrag;
            }
        }
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                bilanzwert +=e.betrag;
            }
            else {
                bilanzwert -= e.betrag;
            }
        }

        System.out.println("=====");
        System.out.printf("| Bilanzwert: %30.2f€ |\n",bilanzwert);

        System.out.println("=====");
    }

    public static void zeigeGefiltert(ArrayList<Eintrag>
        eintraege,ArrayList<FestWert> festeWerte, String kategorie) {
        System.out.println("Einträge in Kategorie: " + kategorie);
        System.out.println("| Datum      | Betrag   | Kategorie
| Bezeichnung | Art      |");
        System.out.println("-----
-----");

        boolean gefunden = false;
        for (Eintrag e : eintraege) {
            if (e.kategorie.equals(kategorie)) {
                System.out.printf("| %10s | %8.2f | %17s | %15s |
Eintrag      |\n",
                    e.datum, e.betrag, e.kategorie, e.bezeichnung);
                gefunden = true;
            }
        }
        for (FestWert f: festeWerte) {
            if (f.kategorie.equals(kategorie)) {
                System.out.printf("| %10s | %8.2f | %17s | %15s |

```

```

fester Wert|\n",
            f.datum, f.betrag, f.kategorie, f.bezeichnung);
        gefunden = true;
    }
}

if (!gefunden) {

System.out.println("=====");
        System.out.println("        Keine Einträge in dieser
Kategorie!");

System.out.println("=====");
    }
}

// public static void zeigeSortiertNachBetrag(ArrayList<Eintrag>
eintraege,ArrayList<FestWert> festeWerte,boolean a) {
//     ArrayList<Eintrag> kopieEintraege=new ArrayList<>(eintraege);
//     ArrayList<FestWert> kopieFesteWerte=new
ArrayList<>(festeWerte);
//     if (!a) {
//         kopieEintraege.sort(Comparator.comparingDouble(e ->
e.betrag));
//         kopieFesteWerte.sort(Comparator.comparingDouble(f ->
f.betrag));
//     }
//     else {
//         kopieEintraege.sort(Comparator.comparingDouble(e ->
e.betrag).reversed());
//         kopieFesteWerte.sort(Comparator.comparingDouble(f ->
f.betrag).reversed());
//     }
//     zeigeBilanz(kopieEintraege,kopieFesteWerte);
// }

public static void zeigeSortiertNachBetrag(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean absteigend) {
    ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);
    ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

    Comparator<Eintrag> compE = absteigend ?
        Comparator.comparingDouble((Eintrag e) ->
e.betrag).reversed() : Comparator.comparingDouble((Eintrag e) ->
e.betrag);

    Comparator<FestWert> compF = absteigend ?
        Comparator.comparingDouble((FestWert f) ->

```



```

f.betrag).reversed() : Comparator.comparingDouble((FestWert f) ->
f.betrag);

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }

    public static void zeigeSortiertNachDatum(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean a) {

        ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = Comparator.comparing(e -> e.datum,

            Comparator.nullsLast(Comparator.naturalOrder()));
        Comparator<FestWert> compF = Comparator.comparing(f ->
f.datum,
            Comparator.nullsLast(Comparator.naturalOrder()));

        if (a) {
            compE = compE.reversed();
            compF = compF.reversed();
        }

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }
}

package HaushaltskassenApp_zwei;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;

public class Uebersicht {

    public static void zeigeAusgaben(ArrayList<Eintrag> eintraege) {
        System.out.println("Ausgaben:");
    }
}

```

```

        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|   Bezeichnung   |   Betrag   |");
System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s| %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);
                summe += e.betrag;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt Ausgaben: %54.2f€ |\n", summe);

System.out.println("=====
=====");

    }
    public static void zeigefesteAusgaben(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Ausgaben:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|   Bezeichnung   |   Periode(Monate) |   Betrag   |");
System.out.println("-----
-----");

        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                int countE=0;
                while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                    countE++;
                }
                for (int i =0;i<countE;i++) {

                    System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                    summe += f.betrag;
                }
            }
        }
    }

```

```

        monatsbeitrag +=f.betrag/f.periode;
    }
    count++;
}

System.out.println("=====
=====");
    System.out.printf("|Gesamt feste Ausgaben: %77.2f€ |\n",
summe);
    System.out.printf("|Gesamt feste Ausgaben/Monat: %71.2f€ |\n",
monatsbeitrag);

System.out.println("=====
=====");

}

    public static void zeigeEinnahmen(ArrayList<Eintrag> eintraege) {
        System.out.println("Einnahmen:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|          Bezeichnung      |          Betrag |");

System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %25s | %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);
                summe += e.betrag;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt Einnahmen: %64.2f€ |\n", summe);

System.out.println("=====
=====");
    }

    public static void zeigefesteEinnahmen(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Einnahmen:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie

```

```

|   Bezeichnung | Periode(Monate) |   Betrag |");
System.out.println("-----
-----");
    for (FestWert f : festeWerte) {
        if (f.istEinnahme) {
            int countE=0;
            while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                countE++;
            }
            for (int i =0;i<countE;i++) {

                System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                summe += f.betrag;
            }
            monatsbeitrag +=f.betrag/f.periode;
        }
        count++;
    }

System.out.println("=====
=====");
    System.out.printf("|Gesamt feste Einnahmen: %66.2f€ |\n",
summe);
    System.out.printf("|Gesamt feste Einnahmen/Monat: %60.2f€ |\n",
monatsbeitrag);

System.out.println("=====
=====");

}

```

```

    public static void zeigeBilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        zeigeAusgaben(eintraege);
        zeigefesteAusgaben(festeWerte);
        zeigeEinnahmen(eintraege);
        zeigefesteEinnahmen(festeWerte);
        double bilanzwert=0.0;
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                bilanzwert +=f.betrag;
            }
            else {
                bilanzwert -= f.betrag;
            }
        }
    }

```

```

    }
    for (Eintrag e : eintraege) {
        if (e.istEinnahme) {
            bilanzwert += e.betrag;
        }
        else {
            bilanzwert -= e.betrag;
        }
    }

    System.out.println("=====");
    System.out.printf("| Bilanzwert: %30.2f€ |\n", bilanzwert);

    System.out.println("=====");
}

public static void zeigeBalkendiagrammAusgaben(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte) {
    // Sammle Ausgaben pro Kategorie
    var katSum = new java.util.HashMap<String, Double>();

    for (Eintrag e : eintraege) {
        if (!e.istEinnahme) {
            katSum.merge(e.kategorie, e.betrag, Double::sum);
        }
    }
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            katSum.merge(f.kategorie, f.betrag, Double::sum);
        }
    }

    if (katSum.isEmpty()) {
        System.out.println("Keine Ausgaben vorhanden.");
        return;
    }

    double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
    int balkenLaenge = 20; // Länge des Balkens

    System.out.println("\nAusgaben nach Kategorie:");
    katSum.entrySet().stream()
        .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
        .forEach(e -> {
            int anzahl = (int) (e.getValue() / maxBetrag *
balkenLaenge);

```

```

        String balken = "█".repeat(anzahl) + "
".repeat(balkenLaenge - anzahl);
        System.out.printf("%s %s %8.2f €\n", balken,
String.format("%-12s", e.getKey()), e.getValue());
    });
}

```

```

public static final String RESET = "\u001B[0m";
public static final String GRUEN = "\u001B[32m";
public static final String GELB = "\u001B[33m";
public static final String ROT = "\u001B[31m";

```

```

    public static void zeigeMonatsBalken(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        LocalDate jetzt = LocalDate.now();
        int jahr = jetzt.getYear();
        int monat = jetzt.getMonthValue();

        var katSum = new java.util.HashMap<String, Double>();

        // Normale Einträge
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
                katSum.merge(e.kategorie, e.betrag, Double::sum);
            }
        }

        // Feste Werte (wiederkehrend)
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                LocalDate naechste = f.datum;
                while (naechste.isBefore(jetzt.plusMonths(1))) {
                    if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                        katSum.merge(f.kategorie, f.betrag,
Double::sum);
                        break;
                    }
                    naechste = naechste.plusMonths(f.periode);
                }
            }
        }

        if (katSum.isEmpty()) {

```

```

        System.out.println("Keine Ausgaben im aktuellen Monat.");
        return;
    }

    double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
    int balkenLaenge = 25;

    System.out.printf("\n=== %s %d ===\n", jetzt.getMonth(),
jahr);
    System.out.println("Budget-Warnung: " + ROT + "Überschritten"
+ RESET + ", " + GELB + "Kritisch" + RESET + ", " + GRUEN + "OK" +
RESET);

    katSum.entrySet().stream()
        .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
        .forEach(e -> {
            double ausgegeben = e.getValue();
            double budget = BudgetManager.getBudget(e.getKey()); // <-
0.0 wenn nicht vorhanden
            int anzahl = (int) (ausgegeben / maxBetrag *
balkenLaenge);
            String balken;
            String farbe;

            if (budget > 0 && ausgegeben > budget) {
                balken = ROT + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                farbe = ROT;
            } else if (budget > 0 && ausgegeben > budget * 0.9) {
                balken = GELB + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                farbe = GELB;
            } else {
                balken = GRUEN + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
                farbe = (budget == 0) ? RESET : GRUEN; // Grau, wenn
kein Budget
            }

            String budgetStr = (budget == 0)
                ? String.format("%8.2f €", ausgegeben)
                : String.format("%8.2f / %.2f €", ausgegeben, budget);

            System.out.printf("%s %s%s %s\n",
                balken + " ".repeat(balkenLaenge - Math.min(anzahl,

```

```

balkenLaenge)),
    farbe,
    String.format("%-15s", e.getKey()),
    budgetStr + RESET);
    });
}

```

```

    public static void zeigeGefiltert(ArrayList<Eintrag>
    eintraege, ArrayList<FestWert> festeWerte, String kategorie) {
        System.out.println("Einträge in Kategorie: " + kategorie);
        System.out.println("| Datum      | Betrag   | Kategorie
| Bezeichnung | Art      |Ausgabe/Einnahme|");

System.out.println("-----
-----");

        String art="";
        boolean gefunden = false;
        for (Eintrag e : eintraege) {
            if (e.kategorie.equals(kategorie)) {
                art= e.istEinnahme ? "Einnahme":"Ausgabe";
                System.out.printf("| %10s | %8.2f | %17s | %15s |
Eintrag      |%16s|\n",
                    e.datum, e.betrag, e.kategorie, e.bezeichnung,
art);
                gefunden = true;
            }
        }
        for (FestWert f: festeWerte) {
            if (f.kategorie.equals(kategorie)) {
                art= f.istEinnahme ? "Einnahme":"Ausgabe";
                System.out.printf("| %10s | %8.2f | %17s | %15s |
fester Wert |%16s|\n",
                    f.datum, f.betrag, f.kategorie, f.bezeichnung,
art);
                gefunden = true;
            }
        }

        if (!gefunden) {

System.out.println("=====");
        System.out.println("      Keine Einträge in dieser
Kategorie!");

System.out.println("=====");
        }
    }

```



```
}
```

```
    public static void zeigeSortiertNachBetrag(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean absteigend) {
        ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = absteigend ?
            Comparator.comparingDouble((Eintrag e) ->
e.betrag).reversed() : Comparator.comparingDouble((Eintrag e) ->
e.betrag);

        Comparator<FestWert> compF = absteigend ?
            Comparator.comparingDouble((FestWert f) ->
f.betrag).reversed() : Comparator.comparingDouble((FestWert f) ->
f.betrag);

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }
```

```
    public static void zeigeSortiertNachDatum(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean a) {

        ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = Comparator.comparing(e -> e.datum,
            Comparator.nullsLast(Comparator.naturalOrder()));
        Comparator<FestWert> compF = Comparator.comparing(f ->
f.datum,
            Comparator.nullsLast(Comparator.naturalOrder()));

        if (a) {
            compE = compE.reversed();
            compF = compF.reversed();
        }
```

```

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }

    public static void zeigeBevorstehendeAusgaben(ArrayList<FestWert>
festeWerte) {

        int tage= IO.readInt("Geben Sie die Tage des Intervalls an:
");
        LocalDate now=LocalDate.now(); // LocalDate.of(2025,12,1);

        System.out.println("=====
=====");
        System.out.println("
Bevorstehende Ausgaben");

        System.out.println("=====
=====");
        System.out.println("| Fälligkeitsdatum |           Kategorie
| Bezeichnung          | Periode(Monate) | Betrag |");

        System.out.println("-----
-----");
        double summe=0.0;

        for (FestWert f :festeWerte)

            if (!f.istEinnahme) {
                int countE=0;
                int countA=0;

                while (!
now.plusDays(tage).isBefore(f.datum.plusMonths(countE*f.periode))) {

                    countE++;
                }
                while
(now.isAfter(f.datum.plusMonths(countA*f.periode))) {
//!now.isBefore(f.datum.plusMonths(countA*f.periode))---heute nicht
dabei

                    countA++;
                }
                for (int i =countA;i<countE;i++) {
                    System.out.printf("|%17s | %20s | %25s| %17d|
%8.2f€ |\n",f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                    summe +=f.betrag;

```

```

        }

    }

    System.out.println("=====
=====");
        if(summe !=0.0) {
            System.out.printf("Die zu erwartende aufzubringende Summe in
den nächsten %"+4+"d Tagen ist: %27.2f€ |\n",tage,summe);
        }
        else {
            System.out.printf("In den nächsten %d Tagen erwartet Dich
keine Ausgabe fester Werte!\n",tage);
        }

    System.out.println("=====
=====");

}

}

```

```

//package HaushaltKassenAppGUI;
//
//import java.time.LocalDate;
//import java.util.ArrayList;
//import java.util.HashMap;
//import java.util.Map;
//
//public class Uebersicht {
//    public static String getUebersichtText(ArrayList<Eintrag>
//    eintraege, ArrayList<FestWert> festeWerte) {
//        StringBuilder sb = new StringBuilder();
//        sb.append(" HAUSHALTSKASSEN-APP - UEBERBLICK\n");
//        sb.append(" Generiert am: ").append(LocalDate.now()).append("\n");
//        sb.append("=".repeat(70)).append("\n\n");
//        //// Ausgaben
//        sb.append(" AUSGABEN\n");
//        sb.append("-".repeat(110)).append("\n");
//        sb.append(String.format("Datum           Kategorie
//Bezeichnung                               Betrag\n"));
//        sb.append("-".repeat(110)).append("\n");
//        double summeAusgaben = 0;
//        for (Eintrag e : eintraege) {
//            if (!e.istEinnahme) {
//                sb.append(String.format("%-15s  %-30s %30s %15.2f€\n",
//e.datum, e.kategorie, e.bezeichnung, e.betrag));
//                summeAusgaben += e.betrag;
//            }
//        }
//    }
//}

```

```

//      }
//      }
//      sb.append("-".repeat(70)).append("\n");
//      sb.append(String.format("Gesamt Ausgaben: %50.2f€\n\n",
summeAusgaben));
///// Feste Ausgaben
//      sb.append(" FESTE AUSGABEN\n");
//      sb.append("-".repeat(70)).append("\n");
//      sb.append(
//          String.format("%-12s %-8s %-20s %-15s %s\n", "Datum",
"Betrag", "Kategorie", "Bezeichnung", "Periode"));
//      sb.append("-".repeat(70)).append("\n");
//      double summeFeste = 0, monatsFeste = 0;
//      for (FestWert f : festeWerte) {
//          if (!f.istEinnahme) {
//              sb.append(String.format("%-12s %8.2f€ %-20s %-15s %d
Monate\n", f.datum, f.betrag, f.kategorie,
//                  f.bezeichnung, f.periode));
//              summeFeste += f.betrag;
//              monatsFeste += f.betrag / f.periode;
//          }
//      }
//      sb.append("-".repeat(70)).append("\n");
//      sb.append(String.format("Gesamt: %58.2f€\n", summeFeste));
//      sb.append(String.format("Monatlich: %55.2f€\n\n",
monatsFeste));
///// Einnahmen
//      sb.append(" EINNAHMEN\n");
//      sb.append("-".repeat(70)).append("\n");
//      sb.append(String.format("%-12s %-8s %-20s %s\n", "Datum",
"Betrag", "Kategorie", "Bezeichnung"));
//      sb.append("-".repeat(70)).append("\n");
//      double summeEinnahmen = 0;
//      for (Eintrag e : eintraege) {
//          if (e.istEinnahme) {
//              sb.append(String.format("%-12s %8.2f€ %-20s %s\n",
e.datum, e.betrag, e.kategorie, e.bezeichnung));
//              summeEinnahmen += e.betrag;
//          }
//      }
//      sb.append("-".repeat(70)).append("\n");
//      sb.append(String.format("Gesamt Einnahmen: %49.2f€\n\n",
summeEinnahmen));
///// Feste Einnahmen
//      sb.append(" FESTE EINNAHMEN\n");
//      sb.append("-".repeat(70)).append("\n");
//      sb.append(
//          String.format("%-12s %-9s %-20s %-15s %s\n", "Datum",
"Betrag", "Kategorie", "Bezeichnung", "Periode"));
//      sb.append("-".repeat(70)).append("\n");

```

```

//      double summeFesteE = 0, monatsFesteE = 0;
//      for (FestWert f : festeWerte) {
//          if (f.istEinnahme) {
//              sb.append(String.format("%-12s %8.2f€ %-20s %-15s %d
Monate\n", f.datum, f.betrag, f.kategorie,
//                  f.bezeichnung, f.periode));
//              summeFesteE += f.betrag;
//              monatsFesteE += f.betrag / f.periode;
//          }
//      }
//      sb.append("-".repeat(70)).append("\n");
//      sb.append(String.format("Gesamt: %58.2f€\n", summeFesteE));
//      sb.append(String.format("Monatlich: %55.2f€\n\n",
monatsFesteE));
///// Bilanz
//      double bilanz = summeEinnahmen + summeFesteE - summeAusgaben -
summeFeste;
//      sb.append("=".repeat(70)).append("\n");
//      sb.append(String.format(" GESAMTBILANZ: %48.2f€\n", bilanz));
//      sb.append(bilanz >= 0 ? " POSITIV\n" : " NEGATIV\n");
//      sb.append("=".repeat(70)).append("\n");
//      return sb.toString();
//  }
//}

```

Klasse: VerwaltungGUI

```
package HaushaltskassenApp_Mit_Gui_Versuch2;
```

```

import javax.swing.*;
import javax.swing.table.*;
import java.awt.*;
import java.io.*;
import java.time.LocalDate;
import java.util.*;
import java.util.List;

public class VerwaltungGUI extends JFrame {
    private final ArrayList<Eintrag> eintraege = new ArrayList<>();
    private final ArrayList<FestWert> festeWerte = new ArrayList<>();
    private JTable table;
    private DefaultTableModel tableModel;
    private String currentFilter = null;
    private boolean sortByBetrag = false;
    private boolean absteigend = false;

    private enum AnzeigeModus { ALLE, AUSGABEN, EINNAHMEN, BILANZ }
    private AnzeigeModus aktuellerModus = AnzeigeModus.ALLE;

    public VerwaltungGUI() {

```

```

setTitle("Haushaltskassen-App");
setSize(1500, 900);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLayout(new BorderLayout());

// === MENÜLEISTE ===
JMenuBar menuBar = new JMenuBar();

JMenu eintragMenu = new JMenu("Einträge");
JMenuItem addEintrag = new JMenuItem("Eintrag hinzufügen");
JMenuItem deleteEintrag = new JMenuItem("Eintrag löschen");
JMenuItem editEintrag = new JMenuItem("Eintrag ändern");
eintragMenu.add(addEintrag);
eintragMenu.add(deleteEintrag);
eintragMenu.add(editEintrag);

JMenu festWertMenu = new JMenu("Feste Werte");
JMenuItem addFestWert = new JMenuItem("Festen Wert
hinzufügen");
JMenuItem deleteFestWert = new JMenuItem("Festen Wert
löschen");
JMenuItem editFestWert = new JMenuItem("Festen Wert ändern");
festWertMenu.add(addFestWert);
festWertMenu.add(deleteFestWert);
festWertMenu.add(editFestWert);

JMenu uebersichtMenu = new JMenu("Übersichten");
JMenuItem showAusgaben = new JMenuItem("Ausgaben anzeigen");
JMenuItem showEinnahmen = new JMenuItem("Einnahmen anzeigen");
JMenuItem showBilanz = new JMenuItem("Bilanz anzeigen");
JMenuItem showAlle = new JMenuItem("Alle anzeigen");
uebersichtMenu.add(showAusgaben);
uebersichtMenu.add(showEinnahmen);
uebersichtMenu.add(showBilanz);
uebersichtMenu.addSeparator();
uebersichtMenu.add(showAlle);

JMenu filterSortMenu = new JMenu("Filter & Sortieren");
JMenuItem applyFilter = new JMenuItem("Filter anwenden");
JMenuItem sortBetragAsc = new JMenuItem("Sortieren nach Betrag
↑");
JMenuItem sortBetragDesc = new JMenuItem("Sortieren nach
Betrag ↓");
JMenuItem sortDatumAsc = new JMenuItem("Sortieren nach Datum
↑");
JMenuItem sortDatumDesc = new JMenuItem("Sortieren nach Datum
↓");
filterSortMenu.add(applyFilter);
filterSortMenu.add(sortBetragAsc);
filterSortMenu.add(sortBetragDesc);

```

```

filterSortMenu.add(sortDatumAsc);
filterSortMenu.add(sortDatumDesc);

JMenu fileMenu = new JMenu("Datei");
JMenuItem exit = new JMenuItem("Beenden");
fileMenu.add(exit);

menuBar.add(eintragMenu);
menuBar.add(festWertMenu);
menuBar.add(uebersichtMenu);
menuBar.add(filterSortMenu);
menuBar.add(fileMenu);
setJMenuBar(menuBar);

// === TABELLE ===
tableModel = new DefaultTableModel(new Object[]{"Index",
"Datum", "Kategorie", "Bezeichnung", "Betrag", "Einnahme", "Periode",
"Typ"}, 0);
table = new JTable(tableModel) {
    @Override
    public Component prepareRenderer(TableCellRenderer
renderer, int row, int column) {
        Component c = super.prepareRenderer(renderer, row,
column);
        if (c instanceof JComponent jc &&
table.getColumnCount() == 8) {
            try {
                String typ = (String) getValueAt(row, 7);
                boolean einnahme = (Boolean) getValueAt(row,
5);

                if ("Eintrag".equals(typ)) {
                    jc.setBackground(einnahme ? new Color(144,
238, 144) : new Color(255, 182, 193));
                } else if ("FestWert".equals(typ)) {
                    jc.setBackground(einnahme ? new Color(24,
220, 142) : new Color(200, 120, 120));
                } else {
                    jc.setBackground(Color.WHITE);
                }
            } catch (Exception e) {
                jc.setBackground(Color.WHITE);
            }
            jc.setOpaque(true);
        }
        return c;
    }
};
table.setRowHeight(30);
table.setFont(new Font("SansSerif", Font.PLAIN, 14));

```

```

add(new JScrollPane(table), BorderLayout.CENTER);

// === ACTION LISTENER ===
addEintrag.addActionListener(e -> addEintragDialog());
deleteEintrag.addActionListener(e -> deleteEintragDialog());
editEintrag.addActionListener(e -> editEintragDialog());

addFestWert.addActionListener(e -> addFestWertDialog());
deleteFestWert.addActionListener(e -> deleteFestWertDialog());
editFestWert.addActionListener(e -> editFestWertDialog());

showAusgaben.addActionListener(e -> { aktuellerModus =
AnzeigeModus.AUSGABEN; currentFilter = null; refreshTable(); });
showEinnahmen.addActionListener(e -> { aktuellerModus =
AnzeigeModus.EINNAHMEN; currentFilter = null; refreshTable(); });
showBilanz.addActionListener(e -> { aktuellerModus =
AnzeigeModus.BILANZ; currentFilter = null; refreshTable(); });
showAlle.addActionListener(e -> { aktuellerModus =
AnzeigeModus.ALLE; currentFilter = null; refreshTable(); });

applyFilter.addActionListener(e -> applyFilterDialog());

sortBetragAsc.addActionListener(e -> { sortByBetrag = true;
absteigend = false; refreshTable(); });
sortBetragDesc.addActionListener(e -> { sortByBetrag = true;
absteigend = true; refreshTable(); });
sortDatumAsc.addActionListener(e -> { sortByBetrag = false;
absteigend = false; refreshTable(); });
sortDatumDesc.addActionListener(e -> { sortByBetrag = false;
absteigend = true; refreshTable(); });

exit.addActionListener(e -> {
    saveData();
    System.exit(0);
});

loadData();
refreshTable();
}

// === DIALOGUE ===
private void addEintragDialog() {
    JTextField bez = new JTextField(20);
    JTextField betrag = new JTextField(10);
    JComboBox<String> einnahmeBox = new JComboBox<>(new String[]
{"Ausgabe", "Einnahme"});
    JTextField datum = new JTextField(LocalDate.now().toString(),
10);
    JTextField kat = new JTextField(15);

```



```

        JPanel panel = new JPanel(new GridLayout(0, 2));
        panel.add(new JLabel("Bezeichnung:")); panel.add(bez);
        panel.add(new JLabel("Betrag:")); panel.add(betrag);
        panel.add(new JLabel("Typ:")); panel.add(einnahmeBox);
        panel.add(new JLabel("Datum (YYYY-MM-DD):"));
panel.add(datum);
        panel.add(new JLabel("Kategorie:")); panel.add(kat);

        if (JOptionPane.showConfirmDialog(this, panel, "Neuer
Eintrag", JOptionPane.OK_CANCEL_OPTION) == JOptionPane.OK_OPTION) {
            try {
                eintraege.add(new Eintrag(bez.getText(),
Double.parseDouble(betrag.getText()),
                    einnahmeBox.getSelectedIndex() == 1,
LocalDate.parse(datum.getText()), kat.getText()));
                refreshTable();
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Ungültige
Eingabe!");
            }
        }
    }

    private void deleteEintragDialog() {
        String idx = JOptionPane.showInputDialog(this, "Index des zu
löschesdes Eintrags:");
        if (idx != null) {
            try {
                int i = Integer.parseInt(idx);
                if (i >= 0 && i < eintraege.size()) {
                    eintraege.remove(i);
                    refreshTable();
                } else {
                    JOptionPane.showMessageDialog(this, "Ungültiger
Index!");
                }
            } catch (Exception ignored) {}
        }
    }

    private void editEintragDialog() {
        String idx = JOptionPane.showInputDialog(this, "Index des zu
ändernden Eintrags:");
        if (idx != null) {
            try {
                int i = Integer.parseInt(idx);
                if (i >= 0 && i < eintraege.size()) {
                    Eintrag e = eintraege.get(i);
                    JTextField bez = new JTextField(e.bezeichnung,

```

```

20);
        JTextField betrag = new
JTextField(String.valueOf(e.betrag), 10);
        JComboBox<String> einnahmeBox = new
JComboBox<>(new String[]{"Ausgabe", "Einnahme"});
        einnahmeBox.setSelectedIndex(e.istEinnahme ? 1 :
0);
        JTextField datum = new
JTextField(e.datum.toString(), 10);
        JTextField kat = new JTextField(e.kategorie, 15);

        JPanel panel = new JPanel(new GridLayout(0, 2));
        panel.add(new JLabel("Bezeichnung:"));
panel.add(bez);
        panel.add(new JLabel("Betrag:"));
panel.add(betrag);
        panel.add(new JLabel("Typ:"));
panel.add(einnahmeBox);
        panel.add(new JLabel("Datum:")); panel.add(datum);
        panel.add(new JLabel("Kategorie:"));
panel.add(kat);

        if (JOptionPane.showConfirmDialog(this, panel,
"Eintrag ändern", JOptionPane.OK_CANCEL_OPTION) ==
JOptionPane.OK_OPTION) {
            eintraege.set(i, new Eintrag(bez.getText(),
Double.parseDouble(betrag.getText()),
                einnahmeBox.getSelectedIndex() == 1,
LocalDate.parse(datum.getText()), kat.getText()));
            refreshTable();
        }
        } else {
            JOptionPane.showMessageDialog(this, "Ungültiger
Index!");
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Fehler bei
Änderung!");
    }
}

private void addFestWertDialog() {
    JTextField bez = new JTextField(20);
    JTextField betrag = new JTextField(10);
    JComboBox<String> einnahmeBox = new JComboBox<>(new String[]
{"Ausgabe", "Einnahme"});
    JTextField datum = new JTextField(LocalDate.now().toString(),
10);
    JTextField kat = new JTextField(15);

```

```

        JTextField periode = new JTextField("12", 5);

        JPanel panel = new JPanel(new GridLayout(0, 2));
        panel.add(new JLabel("Bezeichnung:")); panel.add(bez);
        panel.add(new JLabel("Betrag:")); panel.add(betrag);
        panel.add(new JLabel("Typ:")); panel.add(einnahmeBox);
        panel.add(new JLabel("Datum:")); panel.add(datum);
        panel.add(new JLabel("Kategorie:")); panel.add(kat);
        panel.add(new JLabel("Periode (Monate:)"));
panel.add(periode);

        if (JOptionPane.showConfirmDialog(this, panel, "Neuer fester
Wert", JOptionPane.OK_CANCEL_OPTION) == JOptionPane.OK_OPTION) {
            try {
                festeWerte.add(new FestWert(bez.getText(),
Double.parseDouble(betrag.getText()),
                    einnahmeBox.getSelectedIndex() == 1,
LocalDate.parse(datum.getText()), kat.getText(),
                        Integer.parseInt(periode.getText())));
                refreshTable();
            } catch (Exception ex) {
                JOptionPane.showMessageDialog(this, "Ungültige
Eingabe!");
            }
        }

        private void deleteFestWertDialog() {
            String idx = JOptionPane.showInputDialog(this, "Index des zu
löschen des Festen Wertes:");
            if (idx != null) {
                try {
                    int i = Integer.parseInt(idx);
                    if (i >= 0 && i < festeWerte.size()) {
                        festeWerte.remove(i);
                        refreshTable();
                    } else {
                        JOptionPane.showMessageDialog(this, "Ungültiger
Index!");
                    }
                } catch (Exception ignored) {}
            }
        }

        private void editFestWertDialog() {
            String idx = JOptionPane.showInputDialog(this, "Index des zu
ändernden Festen Wertes:");
            if (idx != null) {
                try {
                    int i = Integer.parseInt(idx);

```

```

        if (i >= 0 && i < festeWerte.size()) {
            FestWert f = festeWerte.get(i);
            JTextField bez = new JTextField(f.bezeichnung,
20);
            JTextField betrag = new
JTextField(String.valueOf(f.betrag), 10);
            JComboBox<String> einnahmeBox = new
JComboBox<>(new String[]{"Ausgabe", "Einnahme"});
            einnahmeBox.setSelectedIndex(f.istEinnahme ? 1 :
0);
            JTextField datum = new
JTextField(f.datum.toString(), 10);
            JTextField kat = new JTextField(f.kategorie, 15);
            JTextField periode = new
JTextField(String.valueOf(f.periode), 5);

            JPanel panel = new JPanel(new GridLayout(0, 2));
            panel.add(new JLabel("Bezeichnung:"));
            panel.add(bez);
            panel.add(new JLabel("Betrag:"));
            panel.add(betrag);
            panel.add(new JLabel("Typ:"));
            panel.add(einnahmeBox);
            panel.add(new JLabel("Datum:")); panel.add(datum);
            panel.add(new JLabel("Kategorie:"));
            panel.add(kat);
            panel.add(new JLabel("Periode:"));
            panel.add(periode);

            if (JOptionPane.showConfirmDialog(this, panel,
"Fester Wert ändern", JOptionPane.OK_CANCEL_OPTION) ==
JOptionPane.OK_OPTION) {
                festeWerte.set(i, new FestWert(bez.getText(),
Double.parseDouble(betrag.getText()),
                einnahmeBox.getSelectedIndex() == 1,
                LocalDate.parse(datum.getText()),
                kat.getText(),
                Integer.parseInt(periode.getText())));
                refreshTable();
            }
        } else {
            JOptionPane.showMessageDialog(this, "Ungültiger
Index!");
        }
    } catch (Exception ex) {
        JOptionPane.showMessageDialog(this, "Fehler bei
Änderung!");
    }
}
}

```

```

        private void applyFilterDialog() {
            String filter = JOptionPane.showInputDialog(this, "Kategorie
            filtern (leer = alle):");
            currentFilter = (filter == null || filter.trim().isEmpty()) ?
            null : filter.trim();
            refreshTable();
        }

        // === TABELLE AKTUALISIEREN ===
        private void refreshTable() {
            // Beim Verlassen des Bilanzmodus: Spalten zurücksetzen
            if (aktuellerModus != AnzeigeModus.BILANZ) {
                tableModel.setColumnCount(0);
                tableModel.setColumnIdentifiers(new Object[]{
                    "Index", "Datum", "Kategorie", "Bezeichnung",
                    "Betrag", "Einnahme", "Periode", "Typ"
                });
            }

            tableModel.setRowCount(0);

            if (aktuellerModus == AnzeigeModus.BILANZ) {
                zeigeBilanzModus();
                return;
            }

            List<Object[]> rows = new ArrayList<>();
            boolean zeigeAusgaben = aktuellerModus == AnzeigeModus.ALLE ||
            aktuellerModus == AnzeigeModus.AUSGABEN;
            boolean zeigeEinnahmen = aktuellerModus == AnzeigeModus.ALLE
            || aktuellerModus == AnzeigeModus.EINNAHMEN;

            // Einträge
            for (int i = 0; i < eintraege.size(); i++) {
                Eintrag e = eintraege.get(i);
                if ((e.istEinnahme && zeigeEinnahmen) || (!e.istEinnahme
                && zeigeAusgaben)) {
                    if (currentFilter == null ||
                    e.kategorie.equals(currentFilter)) {
                        rows.add(new Object[]{i, e.datum, e.kategorie,
                        e.bezeichnung, e.betrag, e.istEinnahme, null, "Eintrag"});
                    }
                }
            }

            // Feste Werte
            for (int i = 0; i < festeWerte.size(); i++) {
                FestWert f = festeWerte.get(i);
                if ((f.istEinnahme && zeigeEinnahmen) || (!f.istEinnahme
                && zeigeAusgaben)) {

```

```

        if (currentFilter == null ||
f.kategorie.equals(currentFilter)) {
            rows.add(new Object[]{i, f.datum, f.kategorie,
f.bezeichnung, f.betrag, f.istEinnahme, f.periode, "FestWert"});
        }
    }

    // Sortierung
    Comparator<Object[]> comp = sortByBetrag
        ? Comparator.comparingDouble(o -> (Double) o[4])
        : Comparator.comparing(o -> (LocalDate) o[1]);
    if (absteigend) comp = comp.reversed();
    rows.sort(comp);

    for (Object[] row : rows) {
        tableModel.addRow(row);
    }

    // Spaltenbreiten (Normalmodus)
    table.getColumnModel().getColumn(0).setPreferredWidth(50);
    table.getColumnModel().getColumn(1).setPreferredWidth(110);
    table.getColumnModel().getColumn(2).setPreferredWidth(150);
    table.getColumnModel().getColumn(3).setPreferredWidth(220);
    table.getColumnModel().getColumn(4).setPreferredWidth(100);
    table.getColumnModel().getColumn(5).setPreferredWidth(80);
    table.getColumnModel().getColumn(6).setPreferredWidth(80);
    table.getColumnModel().getColumn(7).setPreferredWidth(80);
}

private void zeigeBilanzModus() {
    tableModel.setColumnCount(0);
    tableModel.addColumn("Einnahmen");
    tableModel.addColumn("Betrag €");
    tableModel.addColumn("Ausgaben");
    tableModel.addColumn("Betrag €");

    List<Object[]> einnahmen = new ArrayList<>();
    List<Object[]> ausgaben = new ArrayList<>();
    double summeEinnahmen = 0, summeAusgaben = 0;

    for (Eintrag e : eintraege) {
        if (e.istEinnahme && (currentFilter == null ||
e.kategorie.equals(currentFilter))) {
            einnahmen.add(new Object[]{e.bezeichnung + " (" +
e.datum + ")", e.betrag});
            summeEinnahmen += e.betrag;
        } else if (!e.istEinnahme && (currentFilter == null ||
e.kategorie.equals(currentFilter))) {
            ausgaben.add(new Object[]{e.bezeichnung + " (" +

```

```

e.datum + ")", e.betrag));
        summeAusgaben += e.betrag;
    }
}
for (FestWert f : festeWerte) {
    String label = f.bezeichnung + " (fester Wert, " +
f.periode + " Monate)";
    if (f.istEinnahme && (currentFilter == null ||
f.kategorie.equals(currentFilter))) {
        einnahmen.add(new Object[]{label, f.betrag});
        summeEinnahmen += f.betrag;
    } else if (!f.istEinnahme && (currentFilter == null ||
f.kategorie.equals(currentFilter))) {
        ausgaben.add(new Object[]{label, f.betrag});
        summeAusgaben += f.betrag;
    }
}

einnahmen.sort(Comparator.comparingDouble(o -> -(Double)
o[1]));
ausgaben.sort(Comparator.comparingDouble(o -> -(Double)
o[1]));

int max = Math.max(einnahmen.size(), ausgaben.size());
for (int i = 0; i < max; i++) {
    Object[] row = new Object[4];
    if (i < einnahmen.size()) { row[0] = einnahmen.get(i)[0];
row[1] = einnahmen.get(i)[1]; }
    if (i < ausgaben.size()) { row[2] = ausgaben.get(i)[0];
row[3] = ausgaben.get(i)[1]; }
    tableModel.addRow(row);
}

tableModel.addRow(new Object[]{"", "", "", ""});
tableModel.addRow(new Object[]{"Gesamt Einnahmen:",
summeEinnahmen, "Gesamt Ausgaben:", summeAusgaben});

final double bilanz = summeEinnahmen - summeAusgaben;
final int bilanzRow = tableModel.getRowCount();
tableModel.addRow(new Object[]{"", "", "Bilanz:",
String.format("%.2f €", bilanz)});

// Spaltenbreiten
table.getColumnModel().getColumn(0).setPreferredWidth(280);
table.getColumnModel().getColumn(1).setPreferredWidth(100);
table.getColumnModel().getColumn(2).setPreferredWidth(280);
table.getColumnModel().getColumn(3).setPreferredWidth(100);

// Renderer für Bilanz-Zeile
DefaultTableCellRenderer bilanzRenderer = new

```

```

DefaultTableCellRenderer() {
    @Override
    public Component getTableCellRendererComponent(JTable
table, Object value,
                                                    boolean
isSelected, boolean hasFocus, int row, int column) {
        Component c =
super.getTableCellRendererComponent(table, value, isSelected,
hasFocus, row, column);
        if (row == bilanzRow) {
            if (column == 3) {
                c.setForeground(bilanz >= 0 ? new Color(0,
120, 0) : Color.RED);
                c.setFont(c.getFont().deriveFont(Font.BOLD));
            } else if (column == 2) {
                c.setFont(c.getFont().deriveFont(Font.BOLD));
                c.setForeground(Color.BLACK);
            }
        } else {
            c.setForeground(table.getForeground());
            c.setFont(table.getFont());
        }
        return c;
    }
};

```

```

table.getColumnModel().getColumn(3).setCellRenderer(bilanzRenderer);
table.getColumnModel().getColumn(2).setCellRenderer(new
DefaultTableCellRenderer() {
    @Override
    public Component getTableCellRendererComponent(JTable
table, Object value,
                                                    boolean
isSelected, boolean hasFocus, int row, int column) {
        Component c =
super.getTableCellRendererComponent(table, value, isSelected,
hasFocus, row, column);
        if (row == bilanzRow) {
            c.setFont(c.getFont().deriveFont(Font.BOLD));
        }
        return c;
    }
});
}

```

```

// === DATEIEN ===
private void loadData() {
    try (Scanner sc = new Scanner(new File("eintraege.txt"))) {
        while (sc.hasNextLine()) {

```



```

        String[] p = sc.nextLine().split("#");
        if (p.length == 5) {
            eintraege.add(new Eintrag(p[0].trim(),
Double.parseDouble(p[1].trim()),
                Boolean.parseBoolean(p[2].trim()),
LocalDate.parse(p[3].trim()), p[4].trim()));
        }
    } catch (Exception ignored) {}

    try (Scanner sc = new Scanner(new File("festeWerte.txt"))) {
        while (sc.hasNextLine()) {
            String[] p = sc.nextLine().split("#");
            if (p.length == 6) {
                festeWerte.add(new FestWert(p[0].trim(),
Double.parseDouble(p[1].trim()),
                Boolean.parseBoolean(p[2].trim()),
LocalDate.parse(p[3].trim()), p[4].trim(),
                Integer.parseInt(p[5].trim())));
            }
        }
    } catch (Exception ignored) {}
}

private void saveData() {
    try (FileWriter w = new FileWriter("eintraege.txt")) {
        for (Eintrag e : eintraege) {
            w.write(String.format(Locale.US, "%s#%.2f#%b#%s#%s\n",
                e.bezeichnung, e.betrag, e.istEinnahme,
e.datum, e.kategorie));
        }
    } catch (IOException ignored) {}

    try (FileWriter w = new FileWriter("festeWerte.txt")) {
        for (FestWert f : festeWerte) {
            w.write(String.format(Locale.US, "%s#%.2f#%b#%s#%s#%d
%n",
                f.bezeichnung, f.betrag, f.istEinnahme,
f.datum, f.kategorie, f.periode));
        }
    } catch (IOException ignored) {}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new
VerwaltungGUI().setVisible(true));
}
}

```

Package: HaushaltskassenApp_zwei

Klasse: BudgetManager

```
package HaushaltskassenApp_zwei;
```

```
import java.io.*;
import java.util.*;
```

```
public class BudgetManager {
    private static Map<String, Double> budgets = new HashMap<>();
    private static final String DATEI = "budgets.txt";

    public static void ladeBudgets() {
        budgets.clear();
        File file = new File(DATEI);
        if (!file.exists()) {
            System.out.println("budgets.txt nicht gefunden – neue
Datei wird erstellt.");
            return;
        }

        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            int zeile = 0;
            while ((line = br.readLine()) != null) {
                zeile++;
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) continue;

                String[] parts = line.split("#", 2);
                if (parts.length < 2) {
                    System.err.println("Fehler in budgets.txt Zeile "
+ zeile + ": Zu wenig #");
                    continue;
                }

                String kategorie = parts[0].trim();
                String betragStr = parts[1].trim().split("#")
[0].trim();

                if (kategorie.isEmpty() || betragStr.isEmpty()) {
                    System.err.println("Fehler in Zeile " + zeile + ":
Leere Kategorie/Betrag");
                    continue;
                }
            }
        }
    }
}
```

```

        // KOMMA DURCH PUNKT ERSETZEN!
        betragStr = betragStr.replace(',', '.', '.');

        try {
            double betrag = Double.parseDouble(betragStr);
            if (betrag > 0) {
                budgets.put(kategorie, betrag);
            } else {
                System.err.println("Warnung: Betrag ≤ 0 in
Zeile " + zeile + " → ignoriert");
            }
        } catch (NumberFormatException e) {
            System.err.println("Fehler in Zeile " + zeile + ":
Ungültiger Betrag: " + betragStr);
        }
    }
    System.out.println("Budgets geladen: " + budgets.size() +
" Kategorien.");
} catch (Exception e) {
    System.err.println("Kritischer Fehler beim Laden von
budgets.txt: " + e.getMessage());
    e.printStackTrace();
}
}

```

```

public static void speichereBudgets() {
    try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
    {
        pw.println("# Kategorie#Budget");
        budgets.entrySet().stream()
            .sorted(Map.Entry.comparingByKey())
            .forEach(e -> pw.println(e.getKey() + "#" +
String.format(Locale.US, "%.2f", e.getValue())));
        System.out.println("Budgets gespeichert: " +
budgets.size() + " Einträge.");
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern: " +
e.getMessage());
    }
}

```

// --- Budget setzen ---

```

public static void setzeBudget(String kategorie, double betrag) {
    if (betrag > 0) {
        budgets.put(kategorie.trim(), betrag);
    } else {
        budgets.remove(kategorie.trim());
    }
}

```

```

        }
        speichereBudgets();
    }

    // --- Budget löschen ---
    public static void loescheBudget(String kategorie) {
        budgets.remove(kategorie.trim());
        speichereBudgets();
    }

    // --- Alle Budgets anzeigen ---
    public static void zeigeAlleBudgets() {
        if (budgets.isEmpty()) {
            System.out.println("Keine Budgets gesetzt.");
            return;
        }
        System.out.println("=== BUDGETS ===");
        System.out.println("| Kategorie          | Budget          |");
        System.out.println("-----");
        budgets.entrySet().stream()
            .sorted(Map.Entry.comparingByKey())
            .forEach(e -> System.out.printf("| %-17s | %8.2f € |\n",
e.getKey(), e.getValue()));
        System.out.println("-----");
    }

    // --- Sichere Abfragen ---
    public static double getBudget(String kategorie) {
        return budgets.getOrDefault(kategorie.trim(), 0.0);
    }

    public static boolean istUeberschritten(String kategorie, double
ausgegeben) {
        double budget = getBudget(kategorie);
        return budget > 0 && ausgegeben > budget;
    }

    public static String format(String kategorie, double ausgegeben) {
        double budget = getBudget(kategorie);
        return budget == 0
            ? String.format("%8.2f €", ausgegeben)
            : String.format("%8.2f / %.2f €", ausgegeben, budget);
    }
}
package gui;

import java.io.*;
import java.util.*;

public class BudgetManager {

```

```

private static Map<String, Double> budgets = new HashMap<>();
private static final String DATEI = "budgets.txt";

public static void ladeBudgets() {
    budgets.clear();
    File file = new File(DATEI);
    if (!file.exists()) {
        System.out.println("budgets.txt nicht gefunden – neue
Datei wird erstellt.");
        return;
    }

    try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
        String line;
        int zeile = 0;
        while ((line = br.readLine()) != null) {
            zeile++;
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#")) continue;

            String[] parts = line.split("#", 2);
            if (parts.length < 2) {
                System.err.println("Fehler in budgets.txt Zeile "
+ zeile + ": Zu wenig #");
                continue;
            }

            String kategorie = parts[0].trim();
            String betragStr = parts[1].trim().split("#")
[0].trim();

            if (kategorie.isEmpty() || betragStr.isEmpty()) {
                System.err.println("Fehler in Zeile " + zeile + ":
Leere Kategorie/Betrag");
                continue;
            }

            // KOMMA DURCH PUNKT ERSETZEN!
            betragStr = betragStr.replace(',', '.');

            try {
                double betrag = Double.parseDouble(betragStr);
                if (betrag > 0) {
                    budgets.put(kategorie, betrag);
                } else {
                    System.err.println("Warnung: Betrag ≤ 0 in

```

```

Zeile " + zeile + " → ignoriert");
    }
    } catch (NumberFormatException e) {
        System.err.println("Fehler in Zeile " + zeile + ":
Ungültiger Betrag: " + betragStr);
    }
}
System.out.println("Budgets geladen: " + budgets.size() +
" Kategorien.");
} catch (Exception e) {
    System.err.println("Kritischer Fehler beim Laden von
budgets.txt: " + e.getMessage());
    e.printStackTrace();
}
}
}

```

```

public static void speichereBudgets() {
    try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
    {
        pw.println("# Kategorie#Budget");
        budgets.entrySet().stream()
            .sorted(Map.Entry.comparingByKey())
            .forEach(e -> pw.println(e.getKey() + "#" +
String.format(Locale.US, "%.2f", e.getValue())));
        System.out.println("Budgets gespeichert: " +
budgets.size() + " Einträge.");
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern: " +
e.getMessage());
    }
}

```

```

// --- Budget setzen ---
public static void setzeBudget(String kategorie, double betrag) {
    if (betrag > 0) {
        budgets.put(kategorie.trim(), betrag);
    } else {
        budgets.remove(kategorie.trim());
    }
    speichereBudgets();
}

```

```

// --- Budget löschen ---
public static void loescheBudget(String kategorie) {
    budgets.remove(kategorie.trim());
    speichereBudgets();
}

```

```

// --- Alle Budgets anzeigen ---
public static void zeigeAlleBudgets() {
    if (budgets.isEmpty()) {
        System.out.println("Keine Budgets gesetzt.");
        return;
    }
    System.out.println("=== BUDGETS ===");
    System.out.println("| Kategorie          | Budget          |");
    System.out.println("-----");
    budgets.entrySet().stream()
        .sorted(Map.Entry.comparingByKey())
        .forEach(e -> System.out.printf("| %-17s | %8.2f € |\n",
e.getKey(), e.getValue()));
    System.out.println("-----");
}

// --- Sichere Abfragen ---
public static double getBudget(String kategorie) {
    return budgets.getDefault(kategorie.trim(), 0.0);
}

public static boolean istUeberschritten(String kategorie, double
ausgegeben) {
    double budget = getBudget(kategorie);
    return budget > 0 && ausgegeben > budget;
}

public static String format(String kategorie, double ausgegeben) {
    double budget = getBudget(kategorie);
    return budget == 0
        ? String.format("%8.2f €", ausgegeben)
        : String.format("%8.2f / %.2f €", ausgegeben, budget);
}
}package HaushaltskassenApp_zwei;

import java.io.*;
import java.util.*;

public class BudgetManager {
    private static Map<String, Double> budgets = new HashMap<>();
    private static final String DATEI = "budgets.txt";

    public static void ladeBudgets() {
        budgets.clear();
        File file = new File(DATEI);
        if (!file.exists()) {

```

```

        System.out.println("budgets.txt nicht gefunden – neue
Datei wird erstellt.");
        return;
    }

    try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
        String line;
        int zeile = 0;
        while ((line = br.readLine()) != null) {
            zeile++;
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#")) continue;

            String[] parts = line.split("#", 2);
            if (parts.length < 2) {
                System.err.println("Fehler in budgets.txt Zeile "
+ zeile + ": Zu wenig #");
                continue;
            }

            String kategorie = parts[0].trim();
            String betragStr = parts[1].trim().split("#")
[0].trim();

            if (kategorie.isEmpty() || betragStr.isEmpty()) {
                System.err.println("Fehler in Zeile " + zeile + ":
Leere Kategorie/Betrag");
                continue;
            }

            // KOMMA DURCH PUNKT ERSETZEN!
            betragStr = betragStr.replace(',', '.');

            try {
                double betrag = Double.parseDouble(betragStr);
                if (betrag > 0) {
                    budgets.put(kategorie, betrag);
                } else {
                    System.err.println("Warnung: Betrag ≤ 0 in
Zeile " + zeile + " → ignoriert");
                }
            } catch (NumberFormatException e) {
                System.err.println("Fehler in Zeile " + zeile + ":
Ungültiger Betrag: " + betragStr);
            }
        }
        System.out.println("Budgets geladen: " + budgets.size() +
" Kategorien.");
    } catch (Exception e) {

```



```

        System.err.println("Kritischer Fehler beim Laden von
budgets.txt: " + e.getMessage());
        e.printStackTrace();
    }
}

```

```

    public static void speichereBudgets() {
        try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
        {
            pw.println("# Kategorie#Budget");
            budgets.entrySet().stream()
                .sorted(Map.Entry.comparingByKey())
                .forEach(e -> pw.println(e.getKey() + "#" +
String.format(Locale.US, "%.2f", e.getValue())));
            System.out.println("Budgets gespeichert: " +
budgets.size() + " Einträge.");
        } catch (IOException e) {
            System.err.println("Fehler beim Speichern: " +
e.getMessage());
        }
    }
}

```

```

// --- Budget setzen ---
public static void setzeBudget(String kategorie, double betrag) {
    if (betrag > 0) {
        budgets.put(kategorie.trim(), betrag);
    } else {
        budgets.remove(kategorie.trim());
    }
    speichereBudgets();
}

```

```

// --- Budget löschen ---
public static void loescheBudget(String kategorie) {
    budgets.remove(kategorie.trim());
    speichereBudgets();
}

```

```

// --- Alle Budgets anzeigen ---
public static void zeigeAlleBudgets() {
    if (budgets.isEmpty()) {
        System.out.println("Keine Budgets gesetzt.");
        return;
    }
    System.out.println("=== BUDGETS ===");
    System.out.println("| Kategorie          | Budget          |");
    System.out.println("-----");
}

```

```

        budgets.entrySet().stream()
            .sorted(Map.Entry.comparingByKey())
            .forEach(e -> System.out.printf("| %-17s | %8.2f € |\n",
e.getKey(), e.getValue()));
        System.out.println("-----");
    }

    // --- Sichere Abfragen ---
    public static double getBudget(String kategorie) {
        return budgets.getDefault(kategorie.trim(), 0.0);
    }

    public static boolean istUeberschritten(String kategorie, double
ausgegeben) {
        double budget = getBudget(kategorie);
        return budget > 0 && ausgegeben > budget;
    }

    public static String format(String kategorie, double ausgegeben) {
        double budget = getBudget(kategorie);
        return budget == 0
            ? String.format("%8.2f €", ausgegeben)
            : String.format("%8.2f / %.2f €", ausgegeben, budget);
    }

    // --- Alle Budgets als Map zurückgeben (für GUI) ---
    public static Map<String, Double> getAllBudgets() {
        return new HashMap<>(budgets); // Kopie zurückgeben
    }
}package HaushaltskassenApp;

import java.io.*;
import java.util.*;

public class BudgetManager {
    private static Map<String, Double> budgets = new HashMap<>();
    private static final String DATEI = "budgets.txt";

    public static void ladeBudgets() {
        budgets.clear();
        File file = new File(DATEI);
        if (!file.exists()) {
            System.out.println("budgets.txt nicht gefunden – neue
Datei wird erstellt.");
            return;
        }
    }

```

```

        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            int zeile = 0;
            while ((line = br.readLine()) != null) {
                zeile++;
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) continue;

                String[] parts = line.split("#", 2);
                if (parts.length < 2) {
                    System.err.println("Fehler in budgets.txt Zeile "
+ zeile + ": Zu wenig #");
                    continue;
                }

                String kategorie = parts[0].trim();
                String betragStr = parts[1].trim().split("#")
[0].trim();

                if (kategorie.isEmpty() || betragStr.isEmpty()) {
                    System.err.println("Fehler in Zeile " + zeile + ":
Leere Kategorie/Betrag");
                    continue;
                }

                // KOMMA DURCH PUNKT ERSETZEN!
                betragStr = betragStr.replace(',', '.', '');

                try {
                    double betrag = Double.parseDouble(betragStr);
                    if (betrag > 0) {
                        budgets.put(kategorie, betrag);
                    } else {
                        System.err.println("Warnung: Betrag ≤ 0 in
Zeile " + zeile + " → ignoriert");
                    }
                } catch (NumberFormatException e) {
                    System.err.println("Fehler in Zeile " + zeile + ":
Ungültiger Betrag: " + betragStr);
                }

                System.out.println("Budgets geladen: " + budgets.size() +
" Kategorien.");
            } catch (Exception e) {
                System.err.println("Kritischer Fehler beim Laden von
budgets.txt: " + e.getMessage());
                e.printStackTrace();
            }
        }
    }
}

```

```

    public static void speichereBudgets() {
        try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
        {
            pw.println("# Kategorie#Budget");
            budgets.entrySet().stream()
                .sorted(Map.Entry.comparingByKey())
                .forEach(e -> pw.println(e.getKey() + "#" +
String.format(Locale.US, "%.2f", e.getValue())));
            System.out.println("Budgets gespeichert: " +
budgets.size() + " Einträge.");
        } catch (IOException e) {
            System.err.println("Fehler beim Speichern: " +
e.getMessage());
        }
    }

    // --- Budget setzen ---
    public static void setzeBudget(String kategorie, double betrag) {
        if (betrag > 0) {
            budgets.put(kategorie.trim(), betrag);
        } else {
            budgets.remove(kategorie.trim());
        }
        speichereBudgets();
    }

    // --- Budget löschen ---
    public static void loescheBudget(String kategorie) {
        budgets.remove(kategorie.trim());
        speichereBudgets();
    }

    // --- Alle Budgets anzeigen ---
    public static void zeigeAlleBudgets() {
        if (budgets.isEmpty()) {
            System.out.println("Keine Budgets gesetzt.");
            return;
        }
        System.out.println("=== BUDGETS ===");
        System.out.println("| Kategorie          | Budget          |");
        System.out.println("-----");
        budgets.entrySet().stream()
            .sorted(Map.Entry.comparingByKey())
            .forEach(e -> System.out.printf("| %-17s | %8.2f € |\n",
e.getKey(), e.getValue()));
        System.out.println("-----");
    }

```

```

    }

    // --- Sichere Abfragen ---
    public static double getBudget(String kategorie) {
        return budgets.getDefault(kategorie.trim(), 0.0);
    }

    public static boolean istUeberschritten(String kategorie, double
ausgegeben) {
        double budget = getBudget(kategorie);
        return budget > 0 && ausgegeben > budget;
    }

    public static String format(String kategorie, double ausgegeben) {
        double budget = getBudget(kategorie);
        return budget == 0
            ? String.format("%8.2f €", ausgegeben)
            : String.format("%8.2f / %.2f €", ausgegeben, budget);
    }
}package HaushaltskassenApp_zwei;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import java.util.Map;

/**
 * Controller für den Budget-Tab
 *
 * Verwaltet Budgets für verschiedene Kategorien
 */
public class BudgetTabController {

    private DataModel dataModel;
    private TableView<Map.Entry<String, Double>> tableView;
    private VBox content;

    public BudgetTabController(DataModel dataModel) {
        this.dataModel = dataModel;
        createUI();
    }

    private void createUI() {
        content = new VBox(10);
        content.setPadding(new Insets(10));

        Label title = new Label("Budget-Verwaltung");

```

```

        title.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        // Toolbar
        HBox toolbar = new HBox(10);
        toolbar.setPadding(new Insets(5));

        Button addBtn = new Button("Budget setzen/ändern");
        addBtn.setOnAction(e -> setBudget());

        Button deleteBtn = new Button("Budget löschen");
        deleteBtn.setOnAction(e -> deleteBudget());

        Button refreshBtn = new Button("Aktualisieren");
        refreshBtn.setOnAction(e -> refresh());

        toolbar.getChildren().addAll(addBtn, deleteBtn, refreshBtn);

        // Tabelle
        tableView = new TableView<>();
        createTableColumns();

        content.getChildren().addAll(title, toolbar, tableView);
        VBox.setVgrow(tableView, Priority.ALWAYS);

        refresh();
    }

    private void createTableColumns() {
        TableColumn<Map.Entry<String, Double>, String> kategorieCol =
new TableColumn<>("Kategorie");
        kategorieCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleStringProperty(param.getValue().getKey()))
;
        kategorieCol.setPrefWidth(200);

        TableColumn<Map.Entry<String, Double>, Double> budgetCol = new
TableColumn<>("Budget");
        budgetCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleDoubleProperty(param.getValue().getValue()
).asObject());
        budgetCol.setCellFactory(column -> new
TableCell<Map.Entry<String, Double>, Double>() {
            @Override
            protected void updateItem(Double item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                } else {

```

```

        setText(String.format("%.2f €", item));
    }
});
budgetCol.setPrefWidth(150);

tableView.getColumns().addAll(kategorieCol, budgetCol);
}

private void setBudget() {
    String kategorie = GUIDialogHelper.showTextInputDialog(
        "Budget setzen",
        "Bitte geben Sie die Kategorie ein:",
        "Kategorie:"
    );

    if (kategorie == null || kategorie.trim().isEmpty()) {
        return;
    }

    Double betrag = GUIDialogHelper.showDoubleInputDialog(
        "Budget setzen",
        "Bitte geben Sie das Budget ein:",
        "Budget (0 = löschen):"
    );

    if (betrag == null) {
        return;
    }

    BudgetManager.setzeBudget(kategorie.trim(), betrag);
    refresh();
    GUIDialogHelper.showInfo("Budget gesetzt!");
}

private void deleteBudget() {
    String kategorie = GUIDialogHelper.showTextInputDialog(
        "Budget löschen",
        "Bitte geben Sie die Kategorie ein:",
        "Kategorie:"
    );

    if (kategorie == null || kategorie.trim().isEmpty()) {
        return;
    }

    if (GUIDialogHelper.showConfirmation("Budget löschen",
        "Möchten Sie das Budget für '" + kategorie + "'
wirklich löschen?")) {
        BudgetManager.loescheBudget(kategorie.trim());
    }
}

```

```

        refresh();
        GUIDialogHelper.showInfo("Budget gelöscht!");
    }
}

public void refresh() {
    // Budgets aus BudgetManager laden
    Map<String, Double> budgetsMap =
BudgetManager.getAllBudgets();
    ObservableList<Map.Entry<String, Double>> budgets =
FXCollections.observableArrayList(budgetsMap.entrySet());
    tableView.setItems(budgets);
}

public VBox getContent() {
    return content;
}
}

```

Klasse: DataModel

```

package HaushaltskassenApp_zwei;

import java.io.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Locale;
import java.util.Scanner;
import java.util.function.Consumer;

/**
 * Model-Klasse für die Haushaltskassen-App (MVC-Pattern)
 *
 * Diese Klasse verwaltet alle Daten (Einträge, Feste Werte) und
stellt
 * Methoden zum Laden, Speichern und Manipulieren bereit.
 *
 * WICHTIG: Alle Datenoperationen sollten über diese Klasse laufen,
 * um die Trennung zwischen GUI und Backend zu gewährleisten.
 */
public class DataModel {

    private ArrayList<Eintrag> eintraege;
    private ArrayList<FestWert> festeWerte;
    private Consumer<String> statusCallback; // Für Status-Updates an
die GUI

    public DataModel() {
        this.eintraege = new ArrayList<>();
        this.festeWerte = new ArrayList<>();
    }
}

```



```

    /**
     * Setzt einen Callback für Status-Updates (z.B. für Statuszeile
in der GUI)
     */
    public void setStatusCallback(Consumer<String> callback) {
        this.statusCallback = callback;
    }

    private void updateStatus(String message) {
        if (statusCallback != null) {
            statusCallback.accept(message);
        }
    }

    /**
     * Lädt alle Daten beim Programmstart
     */
    public void ladeAlleDaten() {
        ladeEintraege();
        ladeFesteWerte();
        BudgetManager.ladeBudgets();
        SparzielManager.ladeSparziele();
        SparzielManager.automatischeZufuehrungBeimStart(festeWerte);
        updateStatus("Daten erfolgreich geladen");
    }

    /**
     * Lädt Einträge aus eintraege.txt
     */
    private void ladeEintraege() {
        eintraege.clear();
        try (Scanner fileScanner = new Scanner(new
FileReader("eintraege.txt"))) {
            while (fileScanner.hasNextLine()) {
                String line = fileScanner.nextLine();
                String[] parts = line.split("#");
                if (parts.length == 5) {
                    Eintrag e = new Eintrag(
                        parts[0].trim(),
                        Double.parseDouble(parts[1].trim()),
                        Boolean.parseBoolean(parts[2].trim()),
                        LocalDate.parse(parts[3].trim()),
                        parts[4].trim()
                    );
                    eintraege.add(e);
                }
            }
            updateStatus("Einträge geladen: " + eintraege.size());
        } catch (FileNotFoundException e) {

```

```

        updateStatus("Keine gespeicherten Einträge gefunden");
    } catch (Exception e) {
        updateStatus("Fehler beim Laden der Einträge: " +
e.getMessage());
    }
}

/**
 * Lädt Feste Werte aus festeWerte.txt
 */
private void ladeFesteWerte() {
    festeWerte.clear();
    try (Scanner fileScanner = new Scanner(new
FileReader("festeWerte.txt"))) {
        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            String[] parts = line.split("#");
            if (parts.length == 6) {
                FestWert f = new FestWert(
                    parts[0].trim(),
                    Double.parseDouble(parts[1].trim()),
                    Boolean.parseBoolean(parts[2].trim()),
                    LocalDate.parse(parts[3].trim()),
                    parts[4].trim(),
                    Integer.parseInt(parts[5].trim())
                );
                festeWerte.add(f);
            }
        }
        updateStatus("Feste Werte geladen: " + festeWerte.size());
    } catch (FileNotFoundException e) {
        updateStatus("Keine festen Werte gefunden");
    } catch (Exception e) {
        updateStatus("Fehler beim Laden der festen Werte: " +
e.getMessage());
    }
}

/**
 * Speichert alle Daten
 */
public void speichereAlleDaten() {
    speichereEintraege();
    speichereFesteWerte();
    BudgetManager.speichereBudgets();
    SparzielManager.speichereSparziele();
    updateStatus("Alle Daten gespeichert");
}

/**

```

```

    * Speichert Einträge in eintraege.txt
    */
    private void speichereEintraege() {
        try (FileWriter writer = new FileWriter("eintraege.txt")) {
            for (Eintrag e : eintraege) {
                writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%n",
                    e.bezeichnung,
                    e.betrag,
                    e.istEinnahme,
                    e.datum,
                    e.kategorie
                ));
            }
        } catch (IOException e) {
            updateStatus("Fehler beim Speichern der Einträge: " +
e.getMessage());
        }
    }

    /**
    * Speichert Feste Werte in festeWerte.txt
    */
    private void speichereFesteWerte() {
        try (FileWriter writer = new FileWriter("festeWerte.txt")) {
            for (FestWert f : festeWerte) {
                writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%s#%d%n",
                    f.bezeichnung,
                    f.betrag,
                    f.istEinnahme,
                    f.datum,
                    f.kategorie,
                    f.periode
                ));
            }
        } catch (IOException e) {
            updateStatus("Fehler beim Speichern der festen Werte: " +
e.getMessage());
        }
    }

    // Getter-Methoden
    public ArrayList<Eintrag> getEintraege() {
        return eintraege;
    }

    public ArrayList<FestWert> getFesteWerte() {
        return festeWerte;
    }

```

```

// Eintrag-Operationen
public void fuegeEintragHinzu(Eintrag e) {
    Eintrag.fuegeEintragHinzu(eintraege, e);
    speichereEintraege();
    updateStatus("Eintrag hinzugefügt");
}

public void loescheEintrag(int index) {
    if (index >= 0 && index < eintraege.size()) {
        Eintrag.loescheEintrag(eintraege, index);
        speichereEintraege();
        updateStatus("Eintrag gelöscht");
    } else {
        updateStatus("Ungültiger Index!");
    }
}

public void aendereEintrag(int index, Eintrag neuerEintrag) {
    if (index >= 0 && index < eintraege.size()) {
        eintraege.set(index, neuerEintrag);
        speichereEintraege();
        updateStatus("Eintrag geändert");
    } else {
        updateStatus("Ungültiger Index!");
    }
}

// FestWert-Operationen
public void fuegeFestWertHinzu(FestWert f) {
    festeWerte.add(f);
    speichereFesteWerte();
    updateStatus("Fester Wert hinzugefügt");
}

public void loescheFestWert(int index) {
    if (index >= 0 && index < festeWerte.size()) {
        festeWerte.remove(index);
        speichereFesteWerte();
        updateStatus("Fester Wert gelöscht");
    } else {
        updateStatus("Ungültiger Index!");
    }
}

public void aendereFestWert(int index, FestWert neuerFestWert) {
    if (index >= 0 && index < festeWerte.size()) {
        festeWerte.set(index, neuerFestWert);
        speichereFesteWerte();
        updateStatus("Fester Wert geändert");
    }
}

```

```

        } else {
            updateStatus("Ungültiger Index!");
        }
    }
}

```

Klasse: EintraegeTabController

```
package HaushaltskassenApp_zwei;
```

```

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

```

```

/**
 * Controller für den Einträge-Tab
 *
 * Diese Klasse verwaltet die Anzeige und Bearbeitung von Einträgen.
 *
 * HINWEIS: Um neue Funktionen hinzuzufügen:
 * 1. Erstellen Sie eine neue Methode in dieser Klasse
 * 2. Verbinden Sie sie mit einem Button oder Menüpunkt
 * 3. Nutzen Sie dataModel für Datenoperationen
 * 4. Aktualisieren Sie die Tabelle mit refreshTable()
 */

```

```

public class EintraegeTabController {

    private DataModel dataModel;
    private TableView<Eintrag> tableView;
    private ObservableList<Eintrag> eintraegeList;
    private VBox content;

    public EintraegeTabController(DataModel dataModel) {
        this.dataModel = dataModel;
        createUI();
    }

    private void createUI() {
        content = new VBox(10);
        content.setPadding(new Insets(10));

        // Toolbar
        HBox toolbar = new HBox(10);
        toolbar.setPadding(new Insets(5));

        Button addBtn = new Button("Hinzufügen");
    }
}

```

```

addBtn.setOnAction(e -> addEintrag());

Button editBtn = new Button("Bearbeiten");
editBtn.setOnAction(e -> editEintrag());

Button deleteBtn = new Button("Löschen");
deleteBtn.setOnAction(e -> deleteEintrag());

Button refreshBtn = new Button("Aktualisieren");
refreshBtn.setOnAction(e -> refresh());

toolbar.getChildren().addAll(addBtn, editBtn, deleteBtn,
refreshBtn);

// Tabelle
tableView = new TableView<>();
createTableColumns();

// Filter
HBox filterBox = new HBox(10);
filterBox.setPadding(new Insets(5));

Label filterLabel = new Label("Filter:");
ComboBox<String> filterCombo = new ComboBox<>();
filterCombo.setPromptText("Alle");
filterCombo.getItems().add("Alle");
filterCombo.getItems().add("Nur Einnahmen");
filterCombo.getItems().add("Nur Ausgaben");
filterCombo.setOnAction(e ->
applyFilter(filterCombo.getValue()));

TextField searchField = new TextField();
searchField.setPromptText("Suche nach Bezeichnung oder
Kategorie...");
searchField.textProperty().addListener((obs, oldVal, newVal) -
> {
    filterTable(newVal, filterCombo.getValue());
});

filterBox.getChildren().addAll(filterLabel, filterCombo,
searchField);
HBox.setHgrow(searchField, Priority.ALWAYS);

content.getChildren().addAll(toolbar, filterBox, tableView);
VBox.setVgrow(tableView, Priority.ALWAYS);

refresh();
}

private void createTableColumns() {

```

```

        TableColumn<Eintrag, String> bezeichnungCol = new
TableColumn<>("Bezeichnung");
        bezeichnungCol.setCellValueFactory(new
PropertyValueFactory<>("bezeichnung"));
        bezeichnungCol.setPrefWidth(200);

        TableColumn<Eintrag, Double> betragCol = new
TableColumn<>("Betrag");
        betragCol.setCellValueFactory(new
PropertyValueFactory<>("betrag"));
        betragCol.setCellFactory(column -> new TableCell<Eintrag,
Double>() {
            @Override
            protected void updateItem(Double item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                } else {
                    setText(String.format("%.2f €", item));
                }
            }
        });
        betragCol.setPrefWidth(100);

        TableColumn<Eintrag, Boolean> typCol = new
TableColumn<>("Typ");
        typCol.setCellValueFactory(new
PropertyValueFactory<>("istEinnahme"));
        typCol.setCellFactory(column -> new TableCell<Eintrag,
Boolean>() {
            @Override
            protected void updateItem(Boolean item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                    setStyle("");
                } else {
                    setText(item ? "Einnahme" : "Ausgabe");
                    setStyle(item ? "-fx-text-fill: green;" : "-fx-
text-fill: red;");
                }
            }
        });
        typCol.setPrefWidth(100);

        TableColumn<Eintrag, LocalDate> datumCol = new
TableColumn<>("Datum");
        datumCol.setCellValueFactory(new
PropertyValueFactory<>("datum"));
        datumCol.setCellFactory(column -> new TableCell<Eintrag,

```

```

LocalDate>() {
    @Override
    protected void updateItem(LocalDate item, boolean empty) {
        super.updateItem(item, empty);
        if (empty || item == null) {
            setText(null);
        } else {
            setText(item.format(DateTimeFormatter.ofPattern("dd.MM.yyyy")));
        }
    }
});
datumCol.setPrefWidth(120);

TableColumn<Eintrag, String> kategorieCol = new
TableColumn<>("Kategorie");
kategorieCol.setCellValueFactory(new
PropertyValueFactory<>("kategorie"));
kategorieCol.setPrefWidth(150);

tableView.getColumns().addAll(bezeichnungCol, betragCol,
typCol, datumCol, kategorieCol);
}

private void addEintrag() {
    Eintrag neuerEintrag =
GUIDialogHelper.showEintragDialog(null);
    if (neuerEintrag != null) {
        dataModel.fuegeEintragHinzu(neuerEintrag);
        refresh();
    }
}

private void editEintrag() {
    Eintrag selected =
tableView.getSelectionModel().getSelectedItem();
    if (selected == null) {
        GUIDialogHelper.showError("Bitte wählen Sie einen Eintrag
aus!");
        return;
    }

    int index = tableView.getSelectionModel().getSelectedIndex();
    Eintrag geaenderterEintrag =
GUIDialogHelper.showEintragDialog(selected);
    if (geaenderterEintrag != null) {
        dataModel.aendereEintrag(index, geaenderterEintrag);
        refresh();
    }
}
}

```



```

        private void deleteEintrag() {
            Eintrag selected =
tableView.getSelectionModel().getSelectedItem();
            if (selected == null) {
                GUIDialogHelper.showError("Bitte wählen Sie einen Eintrag
aus!");
                return;
            }

            if (GUIDialogHelper.showConfirmation("Löschen",
                "Möchten Sie diesen Eintrag wirklich löschen?")) {
                int index =
tableView.getSelectionModel().getSelectedIndex();
                dataModel.loescheEintrag(index);
                refresh();
            }
        }

        private void applyFilter(String filter) {
            if (filter == null || filter.equals("Alle")) {
                tableView.setItems(eintraegeList);
            } else if (filter.equals("Nur Einnahmen")) {
                ObservableList<Eintrag> filtered =
FXCollections.observableArrayList();
                for (Eintrag e : eintraegeList) {
                    if (e.istEinnahme) {
                        filtered.add(e);
                    }
                }
                tableView.setItems(filtered);
            } else if (filter.equals("Nur Ausgaben")) {
                ObservableList<Eintrag> filtered =
FXCollections.observableArrayList();
                for (Eintrag e : eintraegeList) {
                    if (!e.istEinnahme) {
                        filtered.add(e);
                    }
                }
                tableView.setItems(filtered);
            }
        }

        private void filterTable(String searchText, String filter) {
            ObservableList<Eintrag> filtered =
FXCollections.observableArrayList();
            String searchLower = searchText == null ? "" :
searchText.toLowerCase();

            for (Eintrag e : dataModel.getEintraege()) {

```

```

        // Filter nach Typ
        if (filter != null && !filter.equals("Alle")) {
            if (filter.equals("Nur Einnahmen") && !e.istEinnahme)
continue;
            if (filter.equals("Nur Ausgaben") && e.istEinnahme)
continue;
        }

        // Suche
        if (searchLower.isEmpty() ||
            e.bezeichnung.toLowerCase().contains(searchLower) ||
            e.kategorie.toLowerCase().contains(searchLower)) {
            filtered.add(e);
        }
    }

    tableView.setItems(filtered);
}

public void refresh() {
    eintraegeList =
FXCollections.observableArrayList(dataModel.getEintraege());
    tableView.setItems(eintraegeList);
}

public VBox getContent() {
    return content;
}
}

```

Klasse: Eintrag

```

package HaushaltskassenApp_zwei;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Objects;

public class Eintrag {

    public String bezeichnung;
    public double betrag;
    public boolean istEinnahme;
    public LocalDate datum;
    public String kategorie;

    public Eintrag(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie) {
        super();
        this.bezeichnung = bezeichnung;
        this.betrag = betrag;
    }
}

```

```

        this.istEinnahme = istEinnahme;
        this.datum = datum;
        this.kategorie = kategorie;
    }

    public String getBezeichnung() {
        return bezeichnung;
    }

    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }

    public double getBetrag() {
        return betrag;
    }

    public void setBetrag(double betrag) {
        this.betrag = betrag;
    }

    public boolean isEinnahme() {
        return istEinnahme;
    }

    public void setIstEinnahme(boolean istEinnahme) {
        this.istEinnahme = istEinnahme;
    }

    public LocalDate getDatum() {
        return datum;
    }

    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }

    public String getKategorie() {
        return kategorie;
    }

    public void setKategorie(String kategorie) {
        this.kategorie = kategorie;
    }

    @Override
    public int hashCode() {
        return Objects.hash(betrag, bezeichnung, datum, istEinnahme,
            kategorie);
    }

```

```

    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Eintrag other = (Eintrag) obj;
        return Double.doubleToLongBits(betrag) ==
Double.doubleToLongBits(other.betrag)
            && Objects.equals(bezeichnung, other.bezeichnung) &&
Objects.equals(datum, other.datum)
            && istEinnahme == other.istEinnahme &&
Objects.equals(kategorie, other.kategorie);
    }

    @Override
    public String toString() {
        return "Eintrag [bezeichnung=" + bezeichnung + ", betrag=" +
betrag + ", istEinnahme=" + istEinnahme
            + ", datum=" + datum + ", kategorie=" + kategorie +
        "]\n";
    }

    public static void loescheEintrag(ArrayList<Eintrag> ae,int index)
{
    if (index >= 0 && index < ae.size()) {
        ae.remove(index);
    } else {
        System.out.println("Ungültiger Index!");
    }
}

    public static Eintrag aendereEintrag() {
        return new Eintrag(
            IO.readString("Bezeichnung: "),
            IO.readDouble("Betrag: "),
            IO.readBoolean("Einnahme?(j/n): "),
            IO.readDate("Datum: "),
            IO.readString("Kategorie: ")
        );
    }
    public static void fuegeEintragHinzu(ArrayList<Eintrag>
ae ,Eintrag e) {
        ae.add(e);
    }

```

```

    }

}

Klasse: FestWert
package HaushaltskassenApp_zwei;

import java.time.LocalDate;

public class FestWert extends Eintrag{
    public int periode;

    public FestWert(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie,
        int periode) {
        super(bezeichnung, betrag, istEinnahme, datum, kategorie);
        this.periode = periode;
    }

    public int getPeriode() {
        return periode;
    }

    public void setPeriode(int periode) {
        this.periode = periode;
    }

    @Override
    public String toString() {
        return "FestWert [periode=" + periode + ", bezeichnung=" +
bezeichnung + ", betrag=" + betrag + ", istEinnahme="
        + istEinnahme + ", datum=" + datum + ", kategorie=" +
kategorie + "]";
    }

}

```

```

Klasse: FesteWerteTabController
package HaushaltskassenApp_zwei;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.control.*;

```

```

import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;

/**
 * Controller für den Feste Werte-Tab
 *
 * Verwaltet wiederkehrende Einträge (z.B. Miete, GEZ, etc.)
 */
public class FesteWerteTabController {

    private DataModel dataModel;
    private TableView<FestWert> tableView;
    private ObservableList<FestWert> festeWerteList;
    private VBox content;

    public FesteWerteTabController(DataModel dataModel) {
        this.dataModel = dataModel;
        createUI();
    }

    private void createUI() {
        content = new VBox(10);
        content.setPadding(new Insets(10));

        // Toolbar
        HBox toolbar = new HBox(10);
        toolbar.setPadding(new Insets(5));

        Button addBtn = new Button("Hinzufügen");
        addBtn.setOnAction(e -> addFestWert());

        Button editBtn = new Button("Bearbeiten");
        editBtn.setOnAction(e -> editFestWert());

        Button deleteBtn = new Button("Löschen");
        deleteBtn.setOnAction(e -> deleteFestWert());

        Button refreshBtn = new Button("Aktualisieren");
        refreshBtn.setOnAction(e -> refresh());

        toolbar.getChildren().addAll(addBtn, editBtn, deleteBtn,
refreshBtn);

        // Tabelle
        tableView = new TableView<>();
        createTableColumns();

        content.getChildren().addAll(toolbar, tableView);
    }
}

```

```

        VBox.setVgrow(tableView, Priority.ALWAYS);

        refresh();
    }

    private void createTableColumns() {
        TableColumn<FestWert, String> bezeichnungCol = new
        TableColumn<>("Bezeichnung");
        bezeichnungCol.setCellValueFactory(new
        PropertyValueFactory<>("bezeichnung"));
        bezeichnungCol.setPrefWidth(200);

        TableColumn<FestWert, Double> betragCol = new
        TableColumn<>("Betrag");
        betragCol.setCellValueFactory(new
        PropertyValueFactory<>("betrag"));
        betragCol.setCellFactory(column -> new TableCell<FestWert,
        Double>() {
            @Override
            protected void updateItem(Double item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                } else {
                    setText(String.format("%.2f €", item));
                }
            }
        });
        betragCol.setPrefWidth(100);

        TableColumn<FestWert, Boolean> typCol = new
        TableColumn<>("Typ");
        typCol.setCellValueFactory(new
        PropertyValueFactory<>("istEinnahme"));
        typCol.setCellFactory(column -> new TableCell<FestWert,
        Boolean>() {
            @Override
            protected void updateItem(Boolean item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                    setStyle("");
                } else {
                    setText(item ? "Einnahme" : "Ausgabe");
                    setStyle(item ? "-fx-text-fill: green;" : "-fx-
text-fill: red;");
                }
            }
        });
        typCol.setPrefWidth(100);
    }

```

```

        TableColumn<FestWert, LocalDate> datumCol = new
TableColumn<>("Startdatum");
        datumCol.setCellValueFactory(new
PropertyValueFactory<>("datum"));
        datumCol.setCellFactory(column -> new TableCell<FestWert,
LocalDate>() {
            @Override
            protected void updateItem(LocalDate item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                } else {
                    setText(item.format(DateTimeFormatter.ofPattern("dd.MM.yyyy")));
                }
            }
        });
        datumCol.setPrefWidth(120);

        TableColumn<FestWert, String> kategorieCol = new
TableColumn<>("Kategorie");
        kategorieCol.setCellValueFactory(new
PropertyValueFactory<>("kategorie"));
        kategorieCol.setPrefWidth(150);

        TableColumn<FestWert, Integer> periodeCol = new
TableColumn<>("Periode (Monate)");
        periodeCol.setCellValueFactory(new
PropertyValueFactory<>("periode"));
        periodeCol.setPrefWidth(120);

        tableView.getColumns().addAll(bezeichnungCol, betragCol,
typCol, datumCol, kategorieCol, periodeCol);
    }

    private void addFestWert() {
        FestWert neuerFestWert =
GUIDialogHelper.showFestWertDialog(null);
        if (neuerFestWert != null) {
            dataModel.fuegeFestWertHinzu(neuerFestWert);
            refresh();
        }
    }

    private void editFestWert() {
        FestWert selected =
tableView.getSelectionModel().getSelectedItem();
        if (selected == null) {
            GUIDialogHelper.showError("Bitte wählen Sie einen festen

```



```

Wert aus!");
        return;
    }

    int index = tableView.getSelectionModel().getSelectedIndex();
    FestWert geaenderterFestWert =
GUIDialogHelper.showFestWertDialog(selected);
    if (geaenderterFestWert != null) {
        dataModel.aendereFestWert(index, geaenderterFestWert);
        refresh();
    }
}

private void deleteFestWert() {
    FestWert selected =
tableView.getSelectionModel().getSelectedItem();
    if (selected == null) {
        GUIDialogHelper.showError("Bitte wählen Sie einen festen
Wert aus!");
        return;
    }

    if (GUIDialogHelper.showConfirmation("Löschen",
        "Möchten Sie diesen festen Wert wirklich löschen?")) {
        int index =
tableView.getSelectionModel().getSelectedIndex();
        dataModel.loescheFestWert(index);
        refresh();
    }
}

public void refresh() {
    festeWerteList =
FXCollections.observableArrayList(dataModel.getFesteWerte());
    tableView.setItems(festeWerteList);
}

public VBox getContent() {
    return content;
}
}

```

Klasse: GUIDialogHelper

```
package HaushaltskassenApp_zwei;
```

```

import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.scene.layout.Priority;
import javafx.util.StringConverter;
import java.time.LocalDate;

```

```

import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

/**
 * Hilfsklasse für Dialoge zur Benutzereingabe
 *
 * Diese Klasse stellt Methoden bereit, um Eingabedialoge für die
verschiedenen
 * Datentypen zu erstellen (String, Double, Boolean, LocalDate, etc.)
 *
 * WICHTIG: Um neue Eingabefelder hinzuzufügen, erweitern Sie diese
Klasse
 * mit entsprechenden Dialog-Methoden.
 */
public class GUIDialogHelper {

    private static final DateTimeFormatter DATE_FORMATTER =
DateTimeFormatter.ofPattern("dd.MM.yyyy");

    /**
     * Zeigt einen Dialog zur Eingabe eines Eintrags
     */
    public static Eintrag showEintragDialog(Eintrag existingEintrag) {
        Dialog<Eintrag> dialog = new Dialog<>();
        dialog.setTitle(existingEintrag == null ? "Neuer Eintrag" :
"Eintrag bearbeiten");
        dialog.setHeaderText(existingEintrag == null ? "Bitte geben
Sie die Daten ein:" : "Bitte ändern Sie die Daten:");

        ButtonType okButtonType = new ButtonType("OK",
ButtonBar.ButtonData.OK_DONE);
        dialog.getDialogPane().getButtonTypes().addAll(okButtonType,
ButtonType.CANCEL);

        GridPane grid = new GridPane();
        grid.setHgap(10);
        grid.setVgap(10);
        grid.setPadding(new javafx.geometry.Insets(20, 150, 10, 10));

        TextField bezeichnungField = new TextField();
        bezeichnungField.setPromptText("Bezeichnung");
        TextField betragField = new TextField();
        betragField.setPromptText("Betrag");
        ComboBox<String> typCombo = new ComboBox<>();
        typCombo.getItems().addAll("Ausgabe", "Einnahme");
        typCombo.setValue("Ausgabe");
        DatePicker datumPicker = new DatePicker();
        datumPicker.setValue(LocalDate.now());
        TextField kategorieField = new TextField();
        kategorieField.setPromptText("Kategorie");
    }
}

```

```

        // Vorbelegung bei Bearbeitung
        if (existingEintrag != null) {
            bezeichnungField.setText(existingEintrag.bezeichnung);

betragField.setText(String.valueOf(existingEintrag.betrag));
        typCombo.setValue(existingEintrag.istEinnahme ? "Einnahme"
: "Ausgabe");
        datumPicker.setValue(existingEintrag.datum);
        kategorieField.setText(existingEintrag.kategorie);
    }

    grid.add(new Label("Bezeichnung:"), 0, 0);
    grid.add(bezeichnungField, 1, 0);
    grid.add(new Label("Betrag:"), 0, 1);
    grid.add(betragField, 1, 1);
    grid.add(new Label("Typ:"), 0, 2);
    grid.add(typCombo, 1, 2);
    grid.add(new Label("Datum:"), 0, 3);
    grid.add(datumPicker, 1, 3);
    grid.add(new Label("Kategorie:"), 0, 4);
    grid.add(kategorieField, 1, 4);

    GridPane.setHgrow(bezeichnungField, Priority.ALWAYS);
    GridPane.setHgrow(betragField, Priority.ALWAYS);
    GridPane.setHgrow(kategorieField, Priority.ALWAYS);

    dialog.getDialogPane().setContent(grid);

    // Validierung
    dialog.setResultConverter(dialogButton -> {
        if (dialogButton == okButtonType) {
            try {
                String bezeichnung =
betragField.getText().trim();
                double betrag =
Double.parseDouble(betragField.getText().replace(',', ' '));
                boolean istEinnahme =
typCombo.getValue().equals("Einnahme");
                LocalDate datum = datumPicker.getValue();
                String kategorie =
kategorieField.getText().trim();

                if (bezeichnung.isEmpty() || kategorie.isEmpty())
{
                    showError("Bitte füllen Sie alle Felder
aus!");
                    return null;
                }
            }
        }
    });

```

```

        if (betrag <= 0) {
            showError("Der Betrag muss größer als 0
sein!");
            return null;
        }

        return new Eintrag(bezeichnung, betrag,
istEinnahme, datum, kategorie);
    } catch (NumberFormatException e) {
        showError("Ungültiger Betrag!");
        return null;
    }
}
return null;
});

return dialog.showAndWait().orElse(null);
}

/**
 * Zeigt einen Dialog zur Eingabe eines Festen Werts
 */
public static FestWert showFestWertDialog(FestWert
existingFestWert) {
    Dialog<FestWert> dialog = new Dialog<>();
    dialog.setTitle(existingFestWert == null ? "Neuer Fester Wert"
: "Fester Wert bearbeiten");
    dialog.setHeaderText(existingFestWert == null ? "Bitte geben
Sie die Daten ein:" : "Bitte ändern Sie die Daten:");

    ButtonType okButtonType = new ButtonType("OK",
ButtonBar.ButtonData.OK_DONE);
    dialog.getDialogPane().getButtonTypes().addAll(okButtonType,
ButtonType.CANCEL);

    GridPane grid = new GridPane();
    grid.setHgap(10);
    grid.setVgap(10);
    grid.setPadding(new javafx.geometry.Insets(20, 150, 10, 10));

    TextField bezeichnungField = new TextField();
    bezeichnungField.setPromptText("Bezeichnung");
    TextField betragField = new TextField();
    betragField.setPromptText("Betrag");
    ComboBox<String> typCombo = new ComboBox<>();
    typCombo.getItems().addAll("Ausgabe", "Einnahme");
    typCombo.setValue("Ausgabe");
    DatePicker datumPicker = new DatePicker();
    datumPicker.setValue(LocalDate.now());
    TextField kategorieField = new TextField();

```

```

    kategorieField.setPromptText("Kategorie");
    TextField periodeField = new TextField();
    periodeField.setPromptText("Periode in Monaten");

    // Vorbelegung bei Bearbeitung
    if (existingFestWert != null) {
        bezeichnungField.setText(existingFestWert.bezeichnung);

    betragField.setText(String.valueOf(existingFestWert.betrag));
        typCombo.setValue(existingFestWert.istEinnahme ?
"Einnahme" : "Ausgabe");
        datumPicker.setValue(existingFestWert.datum);
        kategorieField.setText(existingFestWert.kategorie);

    periodeField.setText(String.valueOf(existingFestWert.periode));
    }

    grid.add(new Label("Bezeichnung:"), 0, 0);
    grid.add(bezeichnungField, 1, 0);
    grid.add(new Label("Betrag:"), 0, 1);
    grid.add(betragField, 1, 1);
    grid.add(new Label("Typ:"), 0, 2);
    grid.add(typCombo, 1, 2);
    grid.add(new Label("Datum:"), 0, 3);
    grid.add(datumPicker, 1, 3);
    grid.add(new Label("Kategorie:"), 0, 4);
    grid.add(kategorieField, 1, 4);
    grid.add(new Label("Periode (Monate):"), 0, 5);
    grid.add(periodeField, 1, 5);

    GridPane.setHgrow(bezeichnungField, Priority.ALWAYS);
    GridPane.setHgrow(betragField, Priority.ALWAYS);
    GridPane.setHgrow(kategorieField, Priority.ALWAYS);
    GridPane.setHgrow(periodeField, Priority.ALWAYS);

    dialog.getDialogPane().setContent(grid);

    // Validierung
    dialog.setResultConverter(dialogButton -> {
        if (dialogButton == okButtonType) {
            try {
                String bezeichnung =
bezeichnungField.getText().trim();
                double betrag =
Double.parseDouble(betragField.getText().replace(',', ' '));
                boolean istEinnahme =
typCombo.getValue().equals("Einnahme");
                LocalDate datum = datumPicker.getValue();
                String kategorie =
kategorieField.getText().trim();

```

```

        int periode =
Integer.parseInt(periodeField.getText());

        if (bezeichnung.isEmpty() || kategorie.isEmpty())
{
            showError("Bitte füllen Sie alle Felder
aus!");
            return null;
        }

        if (betrag <= 0) {
            showError("Der Betrag muss größer als 0
sein!");
            return null;
        }

        if (periode <= 0) {
            showError("Die Periode muss größer als 0
sein!");
            return null;
        }

        return new FestWert(bezeichnung, betrag,
istEinnahme, datum, kategorie, periode);
    } catch (NumberFormatException e) {
        showError("Ungültige Eingabe!");
        return null;
    }
}
return null;
});

return dialog.showAndWait().orElse(null);
}

/**
 * Zeigt einen einfachen Text-Eingabedialog
 */
public static String showTextInputDialog(String title, String
header, String prompt) {
    TextInputDialog dialog = new TextInputDialog();
    dialog.setTitle(title);
    dialog.setHeaderText(header);
    dialog.setContentText(prompt);
    return dialog.showAndWait().orElse(null);
}

/**
 * Zeigt einen Double-Eingabedialog
 */

```

```

    public static Double showDoubleInputDialog(String title, String
header, String prompt) {
        TextInputDialog dialog = new TextInputDialog();
        dialog.setTitle(title);
        dialog.setHeaderText(header);
        dialog.setContentText(prompt);

        String result = dialog.showAndWait().orElse(null);
        if (result == null) return null;

        try {
            return Double.parseDouble(result.replace(',', ' '));
        } catch (NumberFormatException e) {
            showError("Ungültiger Betrag!");
            return null;
        }
    }

    /**
     * Zeigt einen Integer-Eingabedialog
     */
    public static Integer showIntInputDialog(String title, String
header, String prompt) {
        TextInputDialog dialog = new TextInputDialog();
        dialog.setTitle(title);
        dialog.setHeaderText(header);
        dialog.setContentText(prompt);

        String result = dialog.showAndWait().orElse(null);
        if (result == null) return null;

        try {
            return Integer.parseInt(result);
        } catch (NumberFormatException e) {
            showError("Ungültige Zahl!");
            return null;
        }
    }

    /**
     * Zeigt einen Datum-Eingabedialog
     */
    public static LocalDate showDateInputDialog(String title, String
header) {
        Dialog<LocalDate> dialog = new Dialog<>();
        dialog.setTitle(title);
        dialog.setHeaderText(header);

        ButtonType okButtonType = new ButtonType("OK",
        ButtonBar.ButtonData.OK_DONE);

```

```

        dialog.getDialogPane().getButtonTypes().addAll(okButtonType,
ButtonType.CANCEL);

        DatePicker datePicker = new DatePicker();
        datePicker.setValue(LocalDate.now());

        GridPane grid = new GridPane();
        grid.setPadding(new javafx.geometry.Insets(20));
        grid.add(new Label("Datum:"), 0, 0);
        grid.add(datePicker, 1, 0);

        dialog.getDialogPane().setContent(grid);

        dialog.setResultConverter(dialogButton -> {
            if (dialogButton == okButtonType) {
                return datePicker.getValue();
            }
            return null;
        });

        return dialog.showAndWait().orElse(null);
    }

    /**
     * Zeigt eine Fehlermeldung
     */
    public static void showError(String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setTitle("Fehler");
        alert.setHeaderText(null);
        alert.setContentText(message);
        alert.showAndWait();
    }

    /**
     * Zeigt eine Informationsmeldung
     */
    public static void showInfo(String message) {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("Information");
        alert.setHeaderText(null);
        alert.setContentText(message);
        alert.showAndWait();
    }

    /**
     * Zeigt eine Bestätigungsdialog
     */
    public static boolean showConfirmation(String title, String
message) {

```



```

        Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
        alert.setTitle(title);
        alert.setHeaderText(null);
        alert.setContentText(message);
        return alert.showAndWait().orElse(ButtonType.CANCEL) ==
ButtonType.OK;
    }
}

```

Klasse: IO

```
package HaushaltskassenApp_zwei;
```

```
import java.time.LocalDate;
import java.util.Scanner;
```

```

public class IO {

    public static int readInt(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextInt();
    }
    public static double readDouble(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextDouble();
    }
    public static String readString(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextLine();
    }
    public static boolean readBoolean(String prompt) {
        Scanner scan = new Scanner(System.in);
        while (true) {
            System.out.print(prompt);
            String input = scan.nextLine().trim().toLowerCase();
            if (input.equals("j") || input.equals("ja") ||
input.equals("y") || input.equals("yes")) {
                return true;
            } else if (input.equals("n") || input.equals("nein") ||
input.equals("no")) {
                return false;
            } else {
                System.out.println("Ungültige Eingabe! Bitte 'j',
'ja', 'n' oder 'nein' eingeben.");
            }
        }
    }
}

```

```

    }
}
public static LocalDate readDate(String prompt) {
    System.out.println(prompt);
    try {
        return LocalDate.of(
            IO.readInt("Jahr: "),
            IO.readInt("Monat: "),
            IO.readInt("Tag: ")
        );
    } catch (Exception e) {
        System.out.println("Ungültiges Datum! Heutiges Datum wird
verwendet!");
        return LocalDate.now();
    }
}

}
//package HaushaltskassenAppGUI;
//
//import java.time.LocalDate;
//import java.util.Scanner;
//
//
//
//public class IO {
//
//
//
//
//    public static int readInt(String prompt) {
//        System.out.print(prompt);
//        Scanner scan = new Scanner(System.in);
//        return scan.nextInt();
//    }
//    public static double readDouble(String prompt) {
//        System.out.print(prompt);
//        Scanner scan = new Scanner(System.in);
//        return scan.nextDouble();
//    }
//    public static String readString(String prompt) {
//        System.out.print(prompt);
//        Scanner scan = new Scanner(System.in);
//        return scan.nextLine();
//    }
//    public static boolean readBoolean(String prompt) {
//        Scanner scan = new Scanner(System.in);
//        while (true) {
//            System.out.print(prompt);
//            String input = scan.nextLine().trim().toLowerCase();
//            if (input.equals("j") || input.equals("ja") ||
input.equals("y") || input.equals("yes")) {

```

```

//          return true;
//      } else if (input.equals("n") || input.equals("nein") ||
input.equals("no")) {
//          return false;
//      } else {
//          System.out.println("Ungültige Eingabe! Bitte 'j',
'ja', 'n' oder 'nein' eingeben.");
//      }
//  }
// }
// public static LocalDate readDate(String prompt) {
//     System.out.println(prompt);
//     try {
//         return LocalDate.of(
//             IO.readInt("Jahr: "),
//             IO.readInt("Monat: "),
//             IO.readInt("Tag: ")
//         );
//     } catch (Exception e) {
//         System.out.println("Ungültiges Datum!");
//         return LocalDate.now();
//     }
// }
// }
// }
// }

```

Klasse: IO

```
package HaushaltskassenApp_zwei;
```

```
import java.time.LocalDate;
import java.util.Scanner;
```

```

public class IO {

    public static int readInt(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextInt();
    }
    public static double readDouble(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextDouble();
    }
    public static String readString(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
    }
}

```


"Geld ist wie ein sechster Sinn – ohne ihn kann man die anderen fünf nicht richtig nutzen.",
 "Geld spricht. Meistens sagt es: Leb wohl!",
 "Man sagt, Geld allein macht nicht glücklich. Es muss schon einem gehören.",
 "Geld ist wie ein Bumerang – es kommt nur selten zurück.",
 "Geld allein macht nicht glücklich – es gehören auch noch ein bisschen Glück und Fantasie dazu.",
 "Geld ist besser als Armut – wenn auch nur aus finanziellen Gründen." ,
 "Mein Kontostand ist wie eine Zwiebel: Wenn ich ihn ansehe, muss ich weinen.",
 "Geld ist wie Dünger. Es nützt nur, wenn man es verteilt." ,
 "Ich habe keine Bank, ich habe kein Geld – ich habe Kinder!",
 "Sparen ist gut, besonders wenn die Eltern es tun." ,
 "Früher war mehr Lametta – und mehr Geld im Portemonnaie.",
 "Geld schläft nicht, es liegt nur manchmal auf anderen Konten.",
 "Wer den Cent nicht ehrt, hat schon den Euro verloren.",
 "Am Ende des Geldes ist noch so viel Monat übrig.",
 "Geld allein macht nicht glücklich. Es gehören auch noch Aktien, Gold und Grundstücke dazu.",
 "Reich ist nicht, wer viel hat, sondern wer wenig braucht.",
 "Ich besitze nichts, aber alles, was ich brauche, habe ich immer bei mir.",
 "Die Götter haben alles Überflüssige teuer gemacht und das Notwendige billig.",
 "Wer zufrieden ist, ist reich.",
 "Je weniger Bedürfnisse, desto weniger Sorgen.",
 "Nicht der Mangel, sondern die Gier macht arm.",
 "Weniger ist oft mehr."

```

    };
    static double zufallszahl=Math.random();
    static int zufallszahlGanz=(int) (zufallszahl*sprueche.length);
    static String schlusstext=sprueche[zufallszahlGanz];

    public static void schreibeSchluss() {
        System.out.println("=".repeat(schlusstext.length()));
        try {

```

```

        System.out.println(schlusstext);
    } catch (Exception e) {

        e.printStackTrace();
    }
    System.out.println("=".repeat(schlusstext.length()));
}

}

```

Klasse: Schlussspruch

```
package HaushaltskassenApp_zwei;
```

```
public class Schlussspruch {
```

```

    static String [] sprueche= {"Rechnen ist schön, wenn am Ende mehr
    rauskommt als reinging.",
    "Wer das Geld hat, hat die Taschen voll –
    wer keins hat, die Hände.",
    "Geld ist nichts. Aber viel Geld, das ist
    etwas anderes.",
    "Geld verdirbt den Charakter – vor allem,
    wenn man keines hat.",
    "Ich spare, egal was es kostet.",
    "Geld ist wie ein sechster Sinn – ohne ihn
    kann man die anderen fünf nicht richtig nutzen.",
    "Geld spricht. Meistens sagt es: Leb
    wohl!",
    "Man sagt, Geld allein macht nicht
    glücklich. Es muss schon einem gehören.",
    "Geld ist wie ein Bumerang – es kommt nur
    selten zurück.",
    "Geld allein macht nicht glücklich – es
    gehören auch noch ein bisschen Glück und Fantasie dazu.",
    "Geld ist besser als Armut – wenn auch nur
    aus finanziellen Gründen.",
    "Mein Kontostand ist wie eine Zwiebel:
    Wenn ich ihn ansehe, muss ich weinen.",
    "Geld ist wie Dünger. Es nützt nur, wenn
    man es verteilt." ,
    "Ich habe keine Bank, ich habe kein Geld –
    ich habe Kinder!",
    "Sparen ist gut, besonders wenn die Eltern
    es tun." ,
    "Früher war mehr Lametta – und mehr Geld
    im Portemonnaie.",
    "Geld schläft nicht, es liegt nur manchmal
    auf anderen Konten.",
    "Wer den Cent nicht ehrt, hat schon den

```

```

Euro verloren.",
                                "Am Ende des Geldes ist noch so viel Monat
übrig.",
                                "Geld allein macht nicht glücklich. Es
gehören auch noch Aktien, Gold und Grundstücke dazu.",
                                "Reich ist nicht, wer viel hat, sondern
wer wenig braucht.",
                                "Ich besitze nichts, aber alles, was ich
brauche, habe ich immer bei mir.",
                                "Die Götter haben alles Überflüssige teuer
gemacht und das Notwendige billig.",
                                "Wer zufrieden ist, ist reich.",
                                "Je weniger Bedürfnisse, desto weniger
Sorgen.",
                                "Nicht der Mangel, sondern die Gier macht
arm.",
                                "Weniger ist oft mehr."
};
static double zufallszahl=Math.random();
static int zufallszahlGanz=(int) (zufallszahl*sprueche.length);
static String schlusstext=sprueche[zufallszahlGanz];

public static void schreibeSchluss() {
    System.out.println("=".repeat(schlusstext.length()));
    try {
        System.out.println(schlusstext);
    } catch (Exception e) {

        e.printStackTrace();
    }
    System.out.println("=".repeat(schlusstext.length()));
}

}

```

Klasse: Sonstiges

```

package HaushaltskassenApp_zwei;

import java.io.*;
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.time.temporal.ChronoUnit;

public class Sonstiges {

    private static final DateTimeFormatter df =
DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss");
    private static double sparziel = 5000.0;

```

```

        private static LocalDate zielDatum = LocalDate.of(2025, 12, 31);

        public static void zeigeMenue(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
            Scanner sc = new Scanner(System.in);
            while (true) {

System.out.println("=====");
                System.out.println("
                                SONSTIGES");

System.out.println("=====");
                System.out.println("1: Monatsvergleich (vs. Vormonat)");
                System.out.println("2: Sparziel + Fortschritt");
                System.out.println("3: Backup erstellen");
                System.out.println("4: Jahresdiagramm (ASCII)");
                System.out.println("5: Tages-Bilanz");
                System.out.println("6: Teuerster Tag");
                System.out.println("7: Wiederherstellen (Setzt ein Backup
voraus)");
                System.out.println("8: Zurück");
                System.out.print("Wahl: ");
                int wahl = IO.readInt("");

                switch (wahl) {
                    case 1 -> monatsvergleich(eintraege, festeWerte);
                    case 2 -> sparzielMenue(eintraege, festeWerte);
                    case 3 -> backupErstellen();
                    case 4 -> jahresdiagramm(eintraege, festeWerte);
                    case 5 -> tagesbilanz(eintraege, festeWerte);
                    case 6 -> teuersterTag(eintraege, festeWerte);
                    case 7 -> wiederherstellen();
                    case 8 -> { return; }
                    default -> System.out.println("Ungültig!");
                }
            }
        }

        // 1. Monatsvergleich
        private static void monatsvergleich(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
            LocalDate jetzt = LocalDate.now();
            int jahr = jetzt.getYear();
            int monat = jetzt.getMonthValue();

            double aktuell = berechneAusgaben(eintraege, festeWerte, jahr,
monat);
            double vormonat = berechneAusgaben(eintraege, festeWerte,
monat - 1 == 0 ? jahr - 1 : jahr, monat - 1 == 0 ? 12 : monat - 1);

            System.out.printf("=== %s %d vs. %s %d ===\n",

```



```

        jetzt.getMonth(), jahr,
        jetzt.minusMonths(1).getMonth(),
jetzt.minusMonths(1).getYear());
    System.out.printf("Ausgaben: %.2f € (" , aktuell);
    if (vormonat == 0) {
        System.out.println("kein Vormonat");
    } else {
        double diff = aktuell - vormonat;
        String trend = diff > 0 ? "MEHR" : "WENIGER";
        System.out.printf("%.2f € %s als Vormonat)\n",
Math.abs(diff), trend);
    }
}

// 2. Sparziel
private static void sparzielAnzeigen(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    double saldo = berechneBilanz(eintraege, festeWerte);

    // RICHTIG: ChronoUnit.DAYS.between
    long tageBisZiel = ChronoUnit.DAYS.between(LocalDate.now(),
zielDatum);
    if (tageBisZiel < 0) tageBisZiel = 0;

    double fehlend = sparziel - saldo;
    double proTag = tageBisZiel > 0 ? fehlend / tageBisZiel : 0;

    System.out.printf("Sparziel: %.2f € bis %s\n", sparziel,
zielDatum);
    System.out.printf("Aktuell: %.2f € (%.2f € fehlen)\n", saldo,
fehlend);
    if (tageBisZiel == 0) {
        System.out.println("Ziel erreicht oder abgelaufen!");
    } else {
        System.out.printf("Pro Tag: %.2f € (noch %d Tage)\n",
proTag, tageBisZiel);
    }
}

// 3. Backup
private static void backupErstellen() {
    String ts = LocalDateTime.now().format(df);
    String backupDir = "backup";
    new File(backupDir).mkdirs();

    kopiere("eintraege.txt", backupDir + "/eintraege_" + ts +
".txt");
    kopiere("festeWerte.txt", backupDir + "/festeWerte_" + ts +
".txt");
    kopiere("budgets.txt", backupDir + "/budgets_" + ts + ".txt");
}

```

```

        System.out.println("Backup erstellt: " + backupDir + "/..._" +
ts + ".txt");
    }

```

```

private static void kopiere(String von, String nach) {
    try (FileInputStream fis = new FileInputStream(von);
        FileOutputStream fos = new FileOutputStream(nach)) {
        fis.transferTo(fos);
    } catch (IOException e) {
        System.out.println("Backup fehlgeschlagen: " + von);
    }
}

```

```

// 4. Jahresdiagramm
private static void jahresdiagramm(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    int jahr = LocalDate.now().getYear();
    double[] monate = new double[12];

    for (int m = 1; m <= 12; m++) {
        monate[m-1] = berechneAusgaben(eintraege, festeWerte,
jahr, m);
    }

    double max = Arrays.stream(monate).max().orElse(1);
    System.out.println("Ausgaben pro Monat " + jahr + ":");
    for (int m = 0; m < 12; m++) {
        int balken = (int) (monate[m] / max * 40);
        System.out.printf("%s %6.0f € %s\n",
            LocalDate.of(jahr, m+1,
1).getMonth().toString().substring(0,3),
            monate[m], "█".repeat(balken));
    }
}

```

```

// 5. Tages-Bilanz
private static void tagesbilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    LocalDate tag = IO.readDate("Datum für Tages-Bilanz:");
    double einnahmen = 0, ausgaben = 0;

    for (Eintrag e : eintraege) {
        if (e.datum.equals(tag)) {
            if (e.istEinnahme) einnahmen += e.betrag;
            else ausgaben += e.betrag;
        }
    }
    // Feste Werte prüfen
    for (FestWert f : festeWerte) {
        LocalDate naechste = f.datum;

```

```

        while (naechste.isBefore(tag.plusMonths(1))) {
            if (naechste.equals(tag)) {
                if (f.istEinnahme) einnahmen += f.betrag;
                else ausgaben += f.betrag;
                break;
            }
            naechste = naechste.plusMonths(f.periode);
        }
    }

    System.out.printf("Tages-Bilanz %s: +%.2f € | -%.2f € | Saldo:
%.2f €\n",
        tag, einnahmen, ausgaben, einnahmen - ausgaben);
}

// 6. Teuerster Tag
private static void teuersterTag(ArrayList<Eintrag> eintraege,
    ArrayList<FestWert> festeWerte) {
    Map<LocalDate, Double> tagesausgaben = new HashMap<>();

    for (Eintrag e : eintraege) {
        if (!e.istEinnahme) {
            tagesausgaben.merge(e.datum, e.betrag, Double::sum);
        }
    }
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            LocalDate naechste = f.datum;
            while
(naechste.isBefore(LocalDate.now().plusYears(1))) {
                tagesausgaben.merge(naechste, f.betrag,
Double::sum);
                naechste = naechste.plusMonths(f.periode);
            }
        }
    }

    if (tagesausgaben.isEmpty()) {
        System.out.println("Keine Ausgaben.");
        return;
    }

    LocalDate teuerster = null;
    double max = 0;
    for (Map.Entry<LocalDate, Double> e :
tagesausgaben.entrySet()) {
        if (e.getValue() > max) {
            max = e.getValue();
            teuerster = e.getKey();
        }
    }
}

```

```

    }

    System.out.printf("Teuerster Tag: %s mit %.2f €\n", teuerster,
max);
}

// Hilfsmethode: Ausgaben berechnen
private static double berechneAusgaben(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, int jahr, int monat) {
    double summe = 0;
    for (Eintrag e : eintraege) {
        if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
            summe += e.betrag;
        }
    }
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            LocalDate naechste = f.datum;
            while
(naechste.isBefore(LocalDate.now().plusMonths(1))) {
                if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                    summe += f.betrag;
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }
    }
    return summe;
}

private static double berechneBilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    double saldo = 0;
    for (Eintrag e : eintraege) {
        saldo += e.istEinnahme ? e.betrag : -e.betrag;
    }
    for (FestWert f : festeWerte) {
        saldo += f.istEinnahme ? f.betrag : -f.betrag;
    }
    return saldo;
}

private static void sparzielMenue(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    while (true) {

```

```

System.out.println("=====");
        System.out.println("                SPARZIEL-MANAGER");

        double sparrate = (festeWerte != null)
            ?
SparzielManager.getSparrateAusFesteWerte(festeWerte)
            : 0.0;
        System.out.println("Monatliche Sparrate: " +
String.format(Locale.US, "%.2f", sparrate) + " €");

System.out.println("=====");
        SparzielManager.zeigeSparziele();
        System.out.println("1: Neues Sparziel");
        System.out.println("2: Sparziel ändern");
        System.out.println("3: Sparziel löschen");
        System.out.println("4: Sparrate ändern"); // <-- NUR
NOCH DAS!
        System.out.println("5: Zurück");
        int wahl = IO.readInt("");

        switch (wahl) {
            case 1 -> SparzielManager.neuesSparziel();
            case 2 -> SparzielManager.aendereSparziel();
            case 3 -> SparzielManager.loescheSparziel();
            case 4 ->
SparzielManager.setzeSparrate(festeWerte);
            case 5 -> { return; }
            default -> System.out.println("Ungültig!");
        }
    }
}

    public static void wiederherstellen() {

System.out.println("=====");
        System.out.println("Anleitung zur Wiederherstellung Ihrer
Daten:");

System.out.println("=====");
        System.out.println("Ersetzen Sie die aktuellen Dateien\
neintraege.txt\nfesteWerte.txt\nbudges.txt\nsparziel.txt\ndurch die
vom Backup erzeugten Dateien und benennen Sie diese in \
neintraege.txt\nfesteWerte.txt\nbudges.txt\nsparziel.txt\nnum.Speichern
Sie und laden Sie das Programm neu.");
    }

}package gui;

```

```

import java.io.*;
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.time.temporal.ChronoUnit;

public class Sonstiges {

    private static final DateTimeFormatter df =
DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss");
    private static double sparziel = 5000.0;
    private static LocalDate zielDatum = LocalDate.of(2025, 12, 31);

    public static void zeigeMenue(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        Scanner sc = new Scanner(System.in);
        while (true) {

System.out.println("=====");
                System.out.println("
                                SONSTIGES");

System.out.println("=====");
                System.out.println("1: Monatsvergleich (vs. Vormonat)");
                System.out.println("2: Sparziel + Fortschritt");
                System.out.println("3: Backup erstellen");
                System.out.println("4: Jahresdiagramm (ASCII)");
                System.out.println("5: Tages-Bilanz");
                System.out.println("6: Teuerster Tag");
                System.out.println("7: Zurück");
                System.out.print("Wahl: ");
                int wahl = IO.readInt("");

                switch (wahl) {
                    case 1 -> monatsvergleich(eintraege, festeWerte);
                    case 2 -> sparzielMenue(eintraege, festeWerte);
                    case 3 -> backupErstellen();
                    case 4 -> jahresdiagramm(eintraege, festeWerte);
                    case 5 -> tagesbilanz(eintraege, festeWerte);
                    case 6 -> teuersterTag(eintraege, festeWerte);
                    case 7 -> { return; }
                    default -> System.out.println("Ungültig!");
                }
            }
        }

    // 1. Monatsvergleich
    private static void monatsvergleich(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        LocalDate jetzt = LocalDate.now();
        int jahr = jetzt.getYear();

```

```

        int monat = jetzt.getMonthValue();

        double aktuell = berechneAusgaben(eintraege, festeWerte, jahr,
monat);
        double vormonat = berechneAusgaben(eintraege, festeWerte,
monat - 1 == 0 ? jahr - 1 : jahr, monat - 1 == 0 ? 12 : monat - 1);

        System.out.printf("=== %s %d vs. %s %d ===\n",
            jetzt.getMonth(), jahr,
            jetzt.minusMonths(1).getMonth(),
jetzt.minusMonths(1).getYear());
        System.out.printf("Ausgaben: %.2f € (" , aktuell);
        if (vormonat == 0) {
            System.out.println("kein Vormonat");
        } else {
            double diff = aktuell - vormonat;
            String trend = diff > 0 ? "MEHR" : "WENIGER";
            System.out.printf("%.2f € %s als Vormonat)\n",
Math.abs(diff), trend);
        }
    }

    // 2. Sparziel
    private static void sparzielAnzeigen(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        double saldo = berechneBilanz(eintraege, festeWerte);

        // RICHTIG: ChronoUnit.DAYS.between
        long tageBisZiel = ChronoUnit.DAYS.between(LocalDate.now(),
zielDatum);
        if (tageBisZiel < 0) tageBisZiel = 0;

        double fehlend = sparziel - saldo;
        double proTag = tageBisZiel > 0 ? fehlend / tageBisZiel : 0;

        System.out.printf("Sparziel: %.2f € bis %s\n", sparziel,
zielDatum);
        System.out.printf("Aktuell: %.2f € (%.2f € fehlen)\n", saldo,
fehlend);
        if (tageBisZiel == 0) {
            System.out.println("Ziel erreicht oder abgelaufen!");
        } else {
            System.out.printf("Pro Tag: %.2f € (noch %d Tage)\n",
proTag, tageBisZiel);
        }
    }

    // 3. Backup
    private static void backupErstellen() {
        String ts = LocalDateTime.now().format(df);

```

```

        String backupDir = "backup";
        new File(backupDir).mkdirs();

        kopiere("eintraege.txt", backupDir + "/eintraege_" + ts +
".txt");
        kopiere("festeWerte.txt", backupDir + "/festeWerte_" + ts +
".txt");
        kopiere("budgets.txt", backupDir + "/budgets_" + ts + ".txt");
        System.out.println("Backup erstellt: " + backupDir + "/..._" +
ts + ".txt");
    }

    private static void kopiere(String von, String nach) {
        try (FileInputStream fis = new FileInputStream(von);
            FileOutputStream fos = new FileOutputStream(nach)) {
            fis.transferTo(fos);
        } catch (IOException e) {
            System.out.println("Backup fehlgeschlagen: " + von);
        }
    }

    // 4. Jahresdiagramm
    private static void jahresdiagramm(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        int jahr = LocalDate.now().getYear();
        double[] monate = new double[12];

        for (int m = 1; m <= 12; m++) {
            monate[m-1] = berechneAusgaben(eintraege, festeWerte,
jahr, m);
        }

        double max = Arrays.stream(monate).max().orElse(1);
        System.out.println("Ausgaben pro Monat " + jahr + ":");
        for (int m = 0; m < 12; m++) {
            int balken = (int) (monate[m] / max * 40);
            System.out.printf("%s %6.0f € %s\n",
                LocalDate.of(jahr, m+1,
1).getMonth().toString().substring(0,3),
                monate[m], "█".repeat(balken));
        }
    }

    // 5. Tages-Bilanz
    private static void tagesbilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        LocalDate tag = IO.readDate("Datum für Tages-Bilanz:");
        double einnahmen = 0, ausgaben = 0;

```



```

        for (Eintrag e : eintraege) {
            if (e.datum.equals(tag)) {
                if (e.istEinnahme) einnahmen += e.betrag;
                else ausgaben += e.betrag;
            }
        }
        // Feste Werte prüfen
        for (FestWert f : festeWerte) {
            LocalDate naechste = f.datum;
            while (naechste.isBefore(tag.plusMonths(1))) {
                if (naechste.equals(tag)) {
                    if (f.istEinnahme) einnahmen += f.betrag;
                    else ausgaben += f.betrag;
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }

        System.out.printf("Tages-Bilanz %s: +%.2f € | -%.2f € | Saldo: %.2f €\n",
            tag, einnahmen, ausgaben, einnahmen - ausgaben);
    }

    // 6. Teuerster Tag
    private static void teuersterTag(ArrayList<Eintrag> eintraege,
        ArrayList<FestWert> festeWerte) {
        Map<LocalDate, Double> tagesausgaben = new HashMap<>();

        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                tagesausgaben.merge(e.datum, e.betrag, Double::sum);
            }
        }
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                LocalDate naechste = f.datum;
                while
(naechste.isBefore(LocalDate.now().plusYears(1))) {
                    tagesausgaben.merge(naechste, f.betrag,
Double::sum);
                    naechste = naechste.plusMonths(f.periode);
                }
            }
        }

        if (tagesausgaben.isEmpty()) {
            System.out.println("Keine Ausgaben.");
            return;
        }
    }

```

```

        LocalDate teuerster = null;
        double max = 0;
        for (Map.Entry<LocalDate, Double> e :
tagesausgaben.entrySet()) {
            if (e.getValue() > max) {
                max = e.getValue();
                teuerster = e.getKey();
            }
        }

        System.out.printf("Teuerster Tag: %s mit %.2f €\n", teuerster,
max);
    }

    // Hilfsmethode: Ausgaben berechnen
    private static double berechneAusgaben(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, int jahr, int monat) {
        double summe = 0;
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
                summe += e.betrag;
            }
        }
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                LocalDate naechste = f.datum;
                while
(naechste.isBefore(LocalDate.now().plusMonths(1))) {
                    if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                        summe += f.betrag;
                        break;
                    }
                    naechste = naechste.plusMonths(f.periode);
                }
            }
        }
        return summe;
    }

    private static double berechneBilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        double saldo = 0;
        for (Eintrag e : eintraege) {
            saldo += e.istEinnahme ? e.betrag : -e.betrag;
        }
        for (FestWert f : festeWerte) {
            saldo += f.istEinnahme ? f.betrag : -f.betrag;
        }
    }

```

```

        }
        return saldo;
    }

    private static void sparzielMenue(ArrayList<Eintrag> eintraege,
    ArrayList<FestWert> festeWerte) {
        while (true) {

System.out.println("=====");
            System.out.println("                SPARZIEL-MANAGER");

            double sparrate = (festeWerte != null)
                ?
SparzielManager.getSparrateAusFesteWerte(festeWerte)
                : 0.0;
            System.out.println("Monatliche Sparrate: " +
String.format(Locale.US, "%.2f", sparrate) + " €");

System.out.println("=====");
                SparzielManager.zeigeSparziele();
                System.out.println("1: Neues Sparziel");
                System.out.println("2: Sparziel ändern");
                System.out.println("3: Sparziel löschen");
                System.out.println("4: Sparrate ändern"); // <-- NUR
NOCH DAS!

                System.out.println("5: Zurück");
                int wahl = IO.readInt("");

                switch (wahl) {
                    case 1 -> SparzielManager.neuesSparziel();
                    case 2 -> SparzielManager.aendereSparziel();
                    case 3 -> SparzielManager.loescheSparziel();
                    case 4 ->
SparzielManager.setzeSparrate(festeWerte);
                    case 5 -> { return; }
                    default -> System.out.println("Ungültig!");
                }
            }
        }
    }

}package HaushaltskassenApp_zwei;

import java.io.*;
import java.time.*;
import java.time.format.DateTimeFormatter;

```

```

import java.util.*;
import java.time.temporal.ChronoUnit;

public class Sonstiges {

    private static final DateTimeFormatter df =
DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss");
    private static double sparziel = 5000.0;
    private static LocalDate zielDatum = LocalDate.of(2025, 12, 31);

    public static void zeigeMenue(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        Scanner sc = new Scanner(System.in);
        while (true) {

System.out.println("=====");
            System.out.println("                SONSTIGES");

System.out.println("=====");
            System.out.println("1: Monatsvergleich (vs. Vormonat)");
            System.out.println("2: Sparziel + Fortschritt");
            System.out.println("3: Backup erstellen");
            System.out.println("4: Jahresdiagramm (ASCII)");
            System.out.println("5: Tages-Bilanz");
            System.out.println("6: Teuerster Tag");
            System.out.println("7: Wiederherstellen (Setzt ein Backup
voraus)");
            System.out.println("8: Zurück");
            System.out.print("Wahl: ");
            int wahl = IO.readInt("");

            switch (wahl) {
                case 1 -> monatsvergleich(eintraege, festeWerte);
                case 2 -> sparzielMenue(eintraege, festeWerte);
                case 3 -> backupErstellen();
                case 4 -> jahresdiagramm(eintraege, festeWerte);
                case 5 -> tagesbilanz(eintraege, festeWerte);
                case 6 -> teuersterTag(eintraege, festeWerte);
                case 7 -> wiederherstellen();
                case 8 -> { return; }
                default -> System.out.println("Ungültig!");
            }
        }
    }

    // 1. Monatsvergleich
    private static void monatsvergleich(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        LocalDate jetzt = LocalDate.now();
        int jahr = jetzt.getYear();

```

```

        int monat = jetzt.getMonthValue();

        double aktuell = berechneAusgaben(eintraege, festeWerte, jahr,
monat);
        double vormonat = berechneAusgaben(eintraege, festeWerte,
monat - 1 == 0 ? jahr - 1 : jahr, monat - 1 == 0 ? 12 : monat - 1);

        System.out.printf("=== %s %d vs. %s %d ===\n",
            jetzt.getMonth(), jahr,
            jetzt.minusMonths(1).getMonth(),
jetzt.minusMonths(1).getYear());
        System.out.printf("Ausgaben: %.2f € (" , aktuell);
        if (vormonat == 0) {
            System.out.println("kein Vormonat");
        } else {
            double diff = aktuell - vormonat;
            String trend = diff > 0 ? "MEHR" : "WENIGER";
            System.out.printf("%.2f € %s als Vormonat)\n",
Math.abs(diff), trend);
        }
    }

    // 2. Sparziel
    private static void sparzielAnzeigen(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        double saldo = berechneBilanz(eintraege, festeWerte);

        // RICHTIG: ChronoUnit.DAYS.between
        long tageBisZiel = ChronoUnit.DAYS.between(LocalDate.now(),
zielDatum);
        if (tageBisZiel < 0) tageBisZiel = 0;

        double fehlend = sparziel - saldo;
        double proTag = tageBisZiel > 0 ? fehlend / tageBisZiel : 0;

        System.out.printf("Sparziel: %.2f € bis %s\n", sparziel,
zielDatum);
        System.out.printf("Aktuell: %.2f € (%.2f € fehlen)\n", saldo,
fehlend);
        if (tageBisZiel == 0) {
            System.out.println("Ziel erreicht oder abgelaufen!");
        } else {
            System.out.printf("Pro Tag: %.2f € (noch %d Tage)\n",
proTag, tageBisZiel);
        }
    }

    // 3. Backup
    public static void backupErstellen() {
        String ts = LocalDateTime.now().format(df);

```

```

        String backupDir = "backup";
        new File(backupDir).mkdirs();

        kopiere("eintraege.txt", backupDir + "/eintraege_" + ts +
".txt");
        kopiere("festeWerte.txt", backupDir + "/festeWerte_" + ts +
".txt");
        kopiere("budgets.txt", backupDir + "/budgets_" + ts + ".txt");
        System.out.println("Backup erstellt: " + backupDir + "/..._" +
ts + ".txt");
    }

    private static void kopiere(String von, String nach) {
        try (FileInputStream fis = new FileInputStream(von);
            FileOutputStream fos = new FileOutputStream(nach)) {
            fis.transferTo(fos);
        } catch (IOException e) {
            System.out.println("Backup fehlgeschlagen: " + von);
        }
    }

    // 4. Jahresdiagramm
    private static void jahresdiagramm(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        int jahr = LocalDate.now().getYear();
        double[] monate = new double[12];

        for (int m = 1; m <= 12; m++) {
            monate[m-1] = berechneAusgaben(eintraege, festeWerte,
jahr, m);
        }

        double max = Arrays.stream(monate).max().orElse(1);
        System.out.println("Ausgaben pro Monat " + jahr + ":");
        for (int m = 0; m < 12; m++) {
            int balken = (int) (monate[m] / max * 40);
            System.out.printf("%s %6.0f € %s\n",
                LocalDate.of(jahr, m+1,
1).getMonth().toString().substring(0,3),
                monate[m], "█".repeat(balken));
        }
    }

    // 5. Tages-Bilanz
    private static void tagesbilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        LocalDate tag = IO.readDate("Datum für Tages-Bilanz:");
        double einnahmen = 0, ausgaben = 0;

```

```

        for (Eintrag e : eintraege) {
            if (e.datum.equals(tag)) {
                if (e.istEinnahme) einnahmen += e.betrag;
                else ausgaben += e.betrag;
            }
        }
        // Feste Werte prüfen
        for (FestWert f : festeWerte) {
            LocalDate naechste = f.datum;
            while (naechste.isBefore(tag.plusMonths(1))) {
                if (naechste.equals(tag)) {
                    if (f.istEinnahme) einnahmen += f.betrag;
                    else ausgaben += f.betrag;
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }

        System.out.printf("Tages-Bilanz %s: +%.2f € | -%.2f € | Saldo: %.2f €\n",
            tag, einnahmen, ausgaben, einnahmen - ausgaben);
    }

    // 6. Teuerster Tag
    private static void teuersterTag(ArrayList<Eintrag> eintraege,
        ArrayList<FestWert> festeWerte) {
        Map<LocalDate, Double> tagesausgaben = new HashMap<>();

        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                tagesausgaben.merge(e.datum, e.betrag, Double::sum);
            }
        }
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                LocalDate naechste = f.datum;
                while
(naechste.isBefore(LocalDate.now().plusYears(1))) {
                    tagesausgaben.merge(naechste, f.betrag,
Double::sum);
                    naechste = naechste.plusMonths(f.periode);
                }
            }
        }

        if (tagesausgaben.isEmpty()) {
            System.out.println("Keine Ausgaben.");
            return;
        }
    }

```

```

        LocalDate teuerster = null;
        double max = 0;
        for (Map.Entry<LocalDate, Double> e :
tagesausgaben.entrySet()) {
            if (e.getValue() > max) {
                max = e.getValue();
                teuerster = e.getKey();
            }
        }

        System.out.printf("Teuerster Tag: %s mit %.2f €\n", teuerster,
max);
    }

```

```

// Hilfsmethode: Ausgaben berechnen
private static double berechneAusgaben(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, int jahr, int monat) {
    double summe = 0;
    for (Eintrag e : eintraege) {
        if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
            summe += e.betrag;
        }
    }
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            LocalDate naechste = f.datum;
            while
(naechste.isBefore(LocalDate.now().plusMonths(1))) {
                if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                    summe += f.betrag;
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }
    }
    return summe;
}

```

```

private static double berechneBilanz(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    double saldo = 0;
    for (Eintrag e : eintraege) {
        saldo += e.istEinnahme ? e.betrag : -e.betrag;
    }
    for (FestWert f : festeWerte) {
        saldo += f.istEinnahme ? f.betrag : -f.betrag;
    }
}

```



```

        }
        return saldo;
    }

    private static void sparzielMenue(ArrayList<Eintrag> eintraege,
    ArrayList<FestWert> festeWerte) {
        while (true) {

System.out.println("=====");
            System.out.println("                SPARZIEL-MANAGER");

                double sparrate = (festeWerte != null)
                    ?
SparzielManager.getSparrateAusFesteWerte(festeWerte)
                    : 0.0;
                System.out.println("Monatliche Sparrate: " +
String.format(Locale.US, "%.2f", sparrate) + " €");

System.out.println("=====");
                SparzielManager.zeigeSparziele();
                System.out.println("1: Neues Sparziel");
                System.out.println("2: Sparziel ändern");
                System.out.println("3: Sparziel löschen");
                System.out.println("4: Sparrate ändern"); // <-- NUR
NOCH DAS!

                System.out.println("5: Zurück");
                int wahl = IO.readInt("");

                switch (wahl) {
                    case 1 -> SparzielManager.neuesSparziel();
                    case 2 -> SparzielManager.aendereSparziel();
                    case 3 -> SparzielManager.loescheSparziel();
                    case 4 ->
SparzielManager.setzeSparrate(festeWerte);
                    case 5 -> { return; }
                    default -> System.out.println("Ungültig!");
                }
            }
        }

        public static void wiederherstellen() {

System.out.println("=====");
            System.out.println("Anleitung zur Wiederherstellung Ihrer
Daten:");

System.out.println("=====");
            System.out.println("Ersetzen Sie die aktuellen Dateien\

```

```

neintraege.txt\nfesteWerte.txt\nbudes.txt\nsparziel.txt\ndurch die
vom Backup erzeugten Dateien und benennen Sie diese in \
neintraege.txt\nfesteWerte.txt\nbudes.txt\nsparziel.txt\num.Speichern
Sie und laden Sie das Programm neu.");
    }

```

```

}package HaushaltskassenApp;

```

```

import java.io.*;
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;
import java.time.temporal.ChronoUnit;

```

```

public class Sonstiges {

```

```

    private static final DateTimeFormatter df =
DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss");
    private static double sparziel = 5000.0;
    private static LocalDate zielDatum = LocalDate.of(2025, 12, 31);

```

```

    public static void zeigeMenü(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        Scanner sc = new Scanner(System.in);
        while (true) {

```

```

System.out.println("=====");
        System.out.println("                SONSTIGES");

```

```

System.out.println("=====");
        System.out.println("1: Monatsvergleich (vs. Vormonat)");
        System.out.println("2: Sparziel + Fortschritt");
        System.out.println("3: Backup erstellen");
        System.out.println("4: Jahresdiagramm (ASCII)");
        System.out.println("5: Tages-Bilanz");
        System.out.println("6: Teuerster Tag");
        System.out.println("7: Zurück");
        System.out.print("Wahl: ");
        int wahl = IO.readInt("");

```

```

        switch (wahl) {
            case 1 -> monatsvergleich(eintraege, festeWerte);
            case 2 -> sparzielMenu(eintraege, festeWerte);
            case 3 -> backupErstellen();
            case 4 -> jahresdiagramm(eintraege, festeWerte);
            case 5 -> tagesbilanz(eintraege, festeWerte);
            case 6 -> teuersterTag(eintraege, festeWerte);
            case 7 -> { return; }
            default -> System.out.println("Ungültig!");
        }

```

```

    }
}

// 1. Monatsvergleich
private static void monatsvergleich(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    LocalDate jetzt = LocalDate.now();
    int jahr = jetzt.getYear();
    int monat = jetzt.getMonthValue();

    double aktuell = berechneAusgaben(eintraege, festeWerte, jahr,
monat);
    double vormonat = berechneAusgaben(eintraege, festeWerte,
monat - 1 == 0 ? jahr - 1 : jahr, monat - 1 == 0 ? 12 : monat - 1);

    System.out.printf("=== %s %d vs. %s %d ===\n",
        jetzt.getMonth(), jahr,
        jetzt.minusMonths(1).getMonth(),
jetzt.minusMonths(1).getYear());
    System.out.printf("Ausgaben: %.2f € (" , aktuell);
    if (vormonat == 0) {
        System.out.println("kein Vormonat");
    } else {
        double diff = aktuell - vormonat;
        String trend = diff > 0 ? "MEHR" : "WENIGER";
        System.out.printf("%.2f € %s als Vormonat)\n",
Math.abs(diff), trend);
    }
}

// 2. Sparziel
private static void sparzielAnzeigen(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
    double saldo = berechneBilanz(eintraege, festeWerte);

    // RICHTIG: ChronoUnit.DAYS.between
    long tageBisZiel = ChronoUnit.DAYS.between(LocalDate.now(),
zielDatum);
    if (tageBisZiel < 0) tageBisZiel = 0;

    double fehlend = sparziel - saldo;
    double proTag = tageBisZiel > 0 ? fehlend / tageBisZiel : 0;

    System.out.printf("Sparziel: %.2f € bis %s\n", sparziel,
zielDatum);
    System.out.printf("Aktuell: %.2f € (%.2f € fehlen)\n", saldo,
fehlend);
    if (tageBisZiel == 0) {
        System.out.println("Ziel erreicht oder abgelaufen!");
    }
}

```

```

        } else {
            System.out.printf("Pro Tag:  %.2f € (noch %d Tage)\n",
proTag, tageBisZiel);
        }
    }

    // 3. Backup
    private static void backupErstellen() {
        String ts = LocalDateTime.now().format(df);
        String backupDir = "backup";
        new File(backupDir).mkdirs();

        kopiere("eintraege.txt", backupDir + "/eintraege_" + ts +
".txt");
        kopiere("festeWerte.txt", backupDir + "/festeWerte_" + ts +
".txt");
        kopiere("budgets.txt", backupDir + "/budgets_" + ts + ".txt");
        System.out.println("Backup erstellt: " + backupDir + "/..._" +
ts + ".txt");
    }

    private static void kopiere(String von, String nach) {
        try (FileInputStream fis = new FileInputStream(von);
            FileOutputStream fos = new FileOutputStream(nach)) {
            fis.transferTo(fos);
        } catch (IOException e) {
            System.out.println("Backup fehlgeschlagen: " + von);
        }
    }

    // 4. Jahresdiagramm
    private static void jahresdiagramm(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        int jahr = LocalDate.now().getYear();
        double[] monate = new double[12];

        for (int m = 1; m <= 12; m++) {
            monate[m-1] = berechneAusgaben(eintraege, festeWerte,
jahr, m);
        }

        double max = Arrays.stream(monate).max().orElse(1);
        System.out.println("Ausgaben pro Monat " + jahr + ":");
        for (int m = 0; m < 12; m++) {
            int balken = (int) (monate[m] / max * 40);
            System.out.printf("%s %6.0f € %s\n",
                LocalDate.of(jahr, m+1,
1).getMonth().toString().substring(0,3),
                monate[m], "█".repeat(balken));
        }
    }

```

```

    }

    // 5. Tages-Bilanz
    private static void tagesbilanz(ArrayList<Eintrag> eintraege,
    ArrayList<FestWert> festeWerte) {
        LocalDate tag = IO.readDate("Datum für Tages-Bilanz:");
        double einnahmen = 0, ausgaben = 0;

        for (Eintrag e : eintraege) {
            if (e.datum.equals(tag)) {
                if (e.istEinnahme) einnahmen += e.betrag;
                else ausgaben += e.betrag;
            }
        }
        // Feste Werte prüfen
        for (FestWert f : festeWerte) {
            LocalDate naechste = f.datum;
            while (naechste.isBefore(tag.plusMonths(1))) {
                if (naechste.equals(tag)) {
                    if (f.istEinnahme) einnahmen += f.betrag;
                    else ausgaben += f.betrag;
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }

        System.out.printf("Tages-Bilanz %s: +%.2f € | -%.2f € | Saldo:
        %.2f €\n",
            tag, einnahmen, ausgaben, einnahmen - ausgaben);
    }

    // 6. Teuerster Tag
    private static void teuersterTag(ArrayList<Eintrag> eintraege,
    ArrayList<FestWert> festeWerte) {
        Map<LocalDate, Double> tagesausgaben = new HashMap<>();

        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                tagesausgaben.merge(e.datum, e.betrag, Double::sum);
            }
        }
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                LocalDate naechste = f.datum;
                while
                (naechste.isBefore(LocalDate.now().plusYears(1))) {
                    tagesausgaben.merge(naechste, f.betrag,
                Double::sum);
                    naechste = naechste.plusMonths(f.periode);
                }
            }
        }
    }

```

```

        }
    }
}

if (tagesausgaben.isEmpty()) {
    System.out.println("Keine Ausgaben.");
    return;
}

LocalDate teuerster = null;
double max = 0;
for (Map.Entry<LocalDate, Double> e :
tagesausgaben.entrySet()) {
    if (e.getValue() > max) {
        max = e.getValue();
        teuerster = e.getKey();
    }
}

System.out.printf("Teuerster Tag: %s mit %.2f €\n", teuerster,
max);
}

// Hilfsmethode: Ausgaben berechnen
private static double berechneAusgaben(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, int jahr, int monat) {
    double summe = 0;
    for (Eintrag e : eintraege) {
        if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
            summe += e.betrag;
        }
    }
    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            LocalDate naechste = f.datum;
            while
(naechste.isBefore(LocalDate.now().plusMonths(1))) {
                if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                    summe += f.betrag;
                    break;
                }
                naechste = naechste.plusMonths(f.periode);
            }
        }
    }
    return summe;
}
}

```

```

        private static double berechneBilanz(ArrayList<Eintrag> eintraege,
        ArrayList<FestWert> festeWerte) {
            double saldo = 0;
            for (Eintrag e : eintraege) {
                saldo += e.istEinnahme ? e.betrag : -e.betrag;
            }
            for (FestWert f : festeWerte) {
                saldo += f.istEinnahme ? f.betrag : -f.betrag;
            }
            return saldo;
        }
    }

```

```

        private static void sparzielMenue(ArrayList<Eintrag> eintraege,
        ArrayList<FestWert> festeWerte) {
            while (true) {

                System.out.println("=====");
                System.out.println("                SPARZIEL-MANAGER");

                double sparrate = (festeWerte != null)
                    ?
                SparzielManager.getSparrateAusFesteWerte(festeWerte)
                    : 0.0;
                System.out.println("Monatliche Sparrate: " +
                String.format(Locale.US, "%.2f", sparrate) + " €");

                System.out.println("=====");
                SparzielManager.zeigeSparziele();
                System.out.println("1: Neues Sparziel");
                System.out.println("2: Sparziel ändern");
                System.out.println("3: Sparziel löschen");
                System.out.println("4: Sparrate ändern"); // <-- NUR
                NOCH DAS!
                System.out.println("5: Zurück");
                int wahl = IO.readInt("");

                switch (wahl) {
                    case 1 -> SparzielManager.neuesSparziel();
                    case 2 -> SparzielManager.aendereSparziel();
                    case 3 -> SparzielManager.loescheSparziel();
                    case 4 ->
                SparzielManager.setzeSparrate(festeWerte);
                    case 5 -> { return; }
                    default -> System.out.println("Ungültig!");
                }
            }
        }
    }

```

```

}package HaushaltskassenApp_zwei;

import javafx.geometry.Insets;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import java.time.LocalDate;

/**
 * Controller für den Sonstiges-Tab
 *
 * Enthält zusätzliche Funktionen wie Backup, Monatsvergleich, etc.
 */
public class SonstigesTabController {

    private DataModel dataModel;
    private VBox content;

    public SonstigesTabController(DataModel dataModel) {
        this.dataModel = dataModel;
        createUI();
    }

    private void createUI() {
        content = new VBox(10);
        content.setPadding(new Insets(10));

        Label title = new Label("Sonstiges");
        title.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        VBox buttonBox = new VBox(10);
        buttonBox.setPadding(new Insets(10));

        Button monatsvergleichBtn = new Button("Monatsvergleich (vs. Vormonat)");
        monatsvergleichBtn.setOnAction(e -> showMonatsvergleich());

        Button sparzielBtn = new Button("Sparziel + Fortschritt");
        sparzielBtn.setOnAction(e -> showSparziel());

        Button backupBtn = new Button("Backup erstellen");
        backupBtn.setOnAction(e -> createBackup());

        Button jahresdiagrammBtn = new Button("Jahresdiagramm");
        jahresdiagrammBtn.setOnAction(e -> showJahresdiagramm());

        Button tagesbilanzBtn = new Button("Tages-Bilanz");
        tagesbilanzBtn.setOnAction(e -> showTagesbilanz());
    }

```



```

        Button teuersterTagBtn = new Button("Teuerster Tag");
        teuersterTagBtn.setOnAction(e -> showTeuersterTag());

        Button wiederherstellenBtn = new Button("Wiederherstellen
(Anleitung)");
        wiederherstellenBtn.setOnAction(e -> showWiederherstellen());

        buttonBox.getChildren().addAll(monatsvergleichBtn,
sparzielBtn, backupBtn,
                jahresdiagrammBtn, tagesbilanzBtn, teuersterTagBtn,
wiederherstellenBtn);

        content.getChildren().addAll(title, buttonBox);
    }

    private void showMonatsvergleich() {
        LocalDate jetzt = LocalDate.now();
        int jahr = jetzt.getYear();
        int monat = jetzt.getMonthValue();

        double aktuell = berechneAusgaben(jahr, monat);
        int vormonatJahr = monat == 1 ? jahr - 1 : jahr;
        int vormonatMonat = monat == 1 ? 12 : monat - 1;
        double vormonat = berechneAusgaben(vormonatJahr,
vormonatMonat);

        String message = String.format(
            "Monatsvergleich:\n\n" +
            "%s %d: %.2f €\n" +
            "%s %d: %.2f €\n\n",
            jetzt.getMonth(), jahr, aktuell,
            jetzt.minusMonths(1).getMonth(),
jetzt.minusMonths(1).getYear(), vormonat
        );

        if (vormonat > 0) {
            double diff = aktuell - vormonat;
            message += String.format("Differenz: %.2f € (%s)",
                Math.abs(diff), diff > 0 ? "mehr" : "weniger");
        }

        GUIDialogHelper.showInfo(message);
    }

    private void showSparziel() {
        // Sparziel-Informationen anzeigen
        double sparrate =
SparzielManager.getSparrateAusFesteWerte(dataModel.getFesteWerte());
        String message = String.format(

```

```

        "Sparziel-Informationen:\n\n" +
        "Monatliche Sparrate: %.2f €\n\n" +
        "Bitte nutzen Sie den Sparziel-Tab für detaillierte
Informationen.",
        sparrate
    );
    GUIDialogHelper.showInfo(message);
}

private void createBackup() {
    try {
        Sonstiges.backupErstellen();
        GUIDialogHelper.showInfo("Backup erfolgreich erstellt!");
    } catch (Exception ex) {
        GUIDialogHelper.showError("Fehler beim Erstellen des
Backups: " + ex.getMessage());
    }
}

private void showJahresdiagramm() {
    // Jahresdiagramm in einem neuen Fenster anzeigen
    GUIDialogHelper.showInfo("Jahresdiagramm wird angezeigt.
(Vollständige Implementierung folgt)");
}

private void showTagesbilanz() {
    LocalDate datum = GUIDialogHelper.showDateInputDialog("Tages-
Bilanz", "Bitte wählen Sie ein Datum:");
    if (datum == null) return;

    double einnahmen = 0;
    double ausgaben = 0;

    for (Eintrag e : dataModel.getEintraege()) {
        if (e.datum.equals(datum)) {
            if (e.istEinnahme) {
                einnahmen += e.betrag;
            } else {
                ausgaben += e.betrag;
            }
        }
    }

    for (FestWert f : dataModel.getFesteWerte()) {
        LocalDate naechste = f.datum;
        while (naechste.isBefore(datum.plusMonths(1))) {
            if (naechste.equals(datum)) {
                if (f.istEinnahme) {
                    einnahmen += f.betrag;
                } else {

```

```

        ausgaben += f.betrag;
    }
    break;
}
naechste = naechste.plusMonths(f.periode);
}
}

double saldo = einnahmen - ausgaben;

String message = String.format(
    "Tages-Bilanz für %s:\n\n" +
    "Einnahmen: +%.2f €\n" +
    "Ausgaben: -%.2f €\n" +
    "Saldo: %.2f €",

datum.format(java.time.format.DateTimeFormatter.ofPattern("dd.MM.yyyy"
)),
    einnahmen, ausgaben, saldo
);

GUIDialogHelper.showInfo(message);
}

private void showTeuersterTag() {
    java.util.Map<LocalDate, Double> tagesausgaben = new
java.util.HashMap<>();

    for (Eintrag e : dataModel.getEintraege()) {
        if (!e.istEinnahme) {
            tagesausgaben.merge(e.datum, e.betrag, Double::sum);
        }
    }

    for (FestWert f : dataModel.getFesteWerte()) {
        if (!f.istEinnahme) {
            LocalDate naechste = f.datum;
            while
(naechste.isBefore(LocalDate.now().plusYears(1))) {
                tagesausgaben.merge(naechste, f.betrag,
Double::sum);
                naechste = naechste.plusMonths(f.periode);
            }
        }
    }

    if (tagesausgaben.isEmpty()) {
        GUIDialogHelper.showInfo("Keine Ausgaben gefunden.");
        return;
    }
}

```

```

        LocalDate teuerster = null;
        double max = 0;
        for (java.util.Map.Entry<LocalDate, Double> e :
tagesausgaben.entrySet()) {
            if (e.getValue() > max) {
                max = e.getValue();
                teuerster = e.getKey();
            }
        }

        String message = String.format(
            "Teuerster Tag:\n\n" +
            "Datum: %s\n" +
            "Ausgaben: %.2f €",

teuerster.format(java.time.format.DateTimeFormatter.ofPattern("dd.MM.y
yyy))),
            max
        );

        GUIDialogHelper.showInfo(message);
    }

    private void showWiederherstellen() {
        String message =
            "Anleitung zur Wiederherstellung:\n\n" +
            "1. Ersetzen Sie die aktuellen Dateien:\n" +
            "    - eintraege.txt\n" +
            "    - festeWerte.txt\n" +
            "    - budgets.txt\n" +
            "    - sparziel.txt\n\n" +
            "2. Durch die vom Backup erzeugten Dateien\n\n" +
            "3. Benennen Sie diese in die Originalnamen um\n\n" +
            "4. Speichern Sie und laden Sie das Programm neu";

        GUIDialogHelper.showInfo(message);
    }

    private double berechneAusgaben(int jahr, int monat) {
        double summe = 0;
        for (Eintrag e : dataModel.getEintraege()) {
            if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
                summe += e.betrag;
            }
        }
        for (FestWert f : dataModel.getFesteWerte()) {
            if (!f.istEinnahme) {
                LocalDate naechste = f.datum;

```

```

        while
(naechste.isBefore(LocalDate.now().plusMonths(1))) {
            if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                summe += f.betrag;
                break;
            }
            naechste = naechste.plusMonths(f.periode);
        }
    }
    return summe;
}

public void refresh() {
    // Sonstiges-Tab aktualisieren falls nötig
}

public VBox getContent() {
    return content;
}
}

```

Klasse: SparzielManager

```

package HaushaltskassenApp_zwei;
//
//
//import java.io.*;
//import java.time.*;
//import java.util.*;
//import java.util.Locale;
//
//public class SparzielManager {
//    private static final List<Sparziel> ziele = new ArrayList<>();
//    private static final String DATEI = "sparziel.txt";
//    private static LocalDate letzteZufuehrung = null;
//    private static final String LETZTE_ZUFUEHRUNG_KEY =
"letzteZufuehrung";
//
//    // --- SPARRATE AUS FESTEN WERTEN LESEN ---
////    public static double
getSparrateAusFesteWerte(ArrayList<FestWert> festeWerte) {
////        return festeWerte.stream()
////            .filter(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie))
////            .mapToDouble(f -> f.betrag)
////            .sum();
////    }
//    public static double
getSparrateAusFesteWerte(ArrayList<FestWert> festeWerte) {

```

```

//      return festeWerte.stream()
//          .filter(f -> !f.istEinnahme &&
//              "Sparen".equals(f.kategorie) &&
//              "Sparplan".equals(f.bezeichnung) &&
//              f.periode == 1)
//          .mapToDouble(f -> f.betrag)
//          .findFirst()
//          .orElse(0.0);
//  }
//
//  public static class Sparziel {
//      String name;
//      double zielbetrag;
//      LocalDate zielDatum;
//      double aktuell;
//      int prioritaaet;
//
//      public Sparziel(String name, double zielbetrag, LocalDate
zielDatum, double aktuell, int prioritaaet) {
//          this.name = name;
//          this.zielbetrag = zielbetrag;
//          this.zielDatum = zielDatum;
//          this.aktuell = aktuell;
//          this.prioritaaet = prioritaaet;
//      }
//
//      public double fehlend() { return Math.max(0, zielbetrag -
aktuell); }
//      public double prozent() { return zielbetrag > 0 ? (aktuell /
zielbetrag) * 100 : 0; }
//
//      @Override
//      public String toString() {
//          int balken = (int) (prozent() / 10);
//          return String.format("P%d: %-15s | %6.0f / %6.0f € | [%s
%s] %3.0f%% | %6.0f € fehlen",
//              prioritaaet, name, aktuell, zielbetrag,
//              "█".repeat(balken), " ".repeat(10 - balken),
//              prozent(), fehlend());
//      }
//  }
//
//  --- LADEN ---
//  public static void ladeSparziele() {
//      ziele.clear();
//      File file = new File(DATEI);
//      if (!file.exists()) return;
//
//      try (BufferedReader br = new BufferedReader(new
//          FileReader(file))) {

```

```

////      String line;
////      while ((line = br.readLine()) != null) {
////          line = line.trim();
////          if (line.isEmpty() || line.startsWith("#"))
continue;
////          String[] p = line.split("#", 5);
////          if (p.length == 5) {
////              ziele.add(new Sparziel(
////                  p[0].trim(),
////                  Double.parseDouble(p[1].trim().replace(',', ' ')),
////                  LocalDate.parse(p[2].trim()),
////                  Double.parseDouble(p[3].trim().replace(',', ' ')),
////                  Integer.parseInt(p[4].trim())
////              ));
////          }
////      }
////      System.out.println("Sparziele geladen: " +
ziele.size());
////      } catch (Exception e) {
////          System.err.println("Fehler beim Laden: " +
e.getMessage());
////      }
////  }
//      public static void ladeSparziele() {
//          ziele.clear();
//          File file = new File(DATEI);
//          if (!file.exists()) return; try (BufferedReader br = new
BufferedReader(new FileReader(file))) {
//              String line;
//              while ((line = br.readLine()) != null) {
//                  line = line.trim();
//                  if (line.isEmpty() || line.startsWith("#")) continue;
//                  if (line.startsWith(LETZTE_ZUFUEHRUNG_KEY + "#")) {
//                      String datumStr =
line.substring(LETZTE_ZUFUEHRUNG_KEY.length() + 1).trim();
//                      if (!datumStr.isEmpty()) {
//                          letzteZufuehrung = LocalDate.parse(datumStr);
//                      }
//                      continue;
//                  }
//              }
//              String[] p = line.split("#", 5);
//              if (p.length == 5) {
//                  ziele.add(new Sparziel(
//                      p[0].trim(),
//                      Double.parseDouble(p[1].trim().replace(',', ' ')),
//                      LocalDate.parse(p[2].trim()),
//                      Double.parseDouble(p[3].trim().replace(',', ' ')),
//                      Integer.parseInt(p[4].trim())

```

```

//                ));
//            }
//        }
//        System.out.println("Sparziele geladen: " + ziele.size());
//    } catch (Exception e) {
//        System.err.println("Fehler beim Laden: " +
e.getMessage());
//    }
//    }
//
//
//
//    // --- SPEICHERN ---
////    public static void speichereSparziele() {
////        try (PrintWriter pw = new PrintWriter(new
FileWriter(DATEI))) {
////            pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
////            ziele.stream()
////                .sorted(Comparator.comparingInt(s ->
s.prioritaet))
////                .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d%n",
////                    sz.name, sz.zielbetrag, sz.zielDatum,
sz.aktuell, sz.prioritaet));
////        } catch (IOException e) {
////            System.err.println("Fehler beim Speichern: " +
e.getMessage());
////        }
////    }
//
//    public static void speichereSparziele() {
//        try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
//        {
//            pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
//            ziele.stream()
//                .sorted(Comparator.comparingInt(s -> s.prioritaet))
//                .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d%n",
//                    sz.name, sz.zielbetrag, sz.zielDatum, sz.aktuell,
sz.prioritaet));
//            // LETZTE ZUFÜHRUNG SPEICHERN
//            if (letzteZufuehrung != null) {
//                pw.println(LETZTE_ZUFUEHRUNG_KEY + "#" +
letzteZufuehrung);
//            }
//        } catch (IOException e) {
//            System.err.println("Fehler beim Speichern: " +
e.getMessage());
//        }
//    }
//
//    // --- NEU ---

```



```

//      public static void neuesSparziel() {
//          String name = IO.readString("Name: ");
//          double betrag = IO.readDouble("Zielbetrag: ");
//          LocalDate datum = IO.readDate("Datum: ");
//          int prio = IO.readInt("Priorität (1 = höchste): ");
//          ziele.add(new Sparziel(name, betrag, datum, 0.0, prio));
//          speichereSparziele();
//      }
//
//      // --- MONATLICHE ZUFÜHRUNG (EINZIGE METHODE) ---
//      public static void monatlicheZufuehrung(ArrayList<FestWert>
festeWerte) {
//          double sparrate = getSparrateAusFesteWerte(festeWerte);
//          if (sparrate <= 0 || ziele.isEmpty()) {
//              System.out.println("Keine Sparrate oder Sparziele
vorhanden.");
//              return;
//          }
//
//          List<Sparziel> sortiert = new ArrayList<>(ziele);
//          sortiert.sort(Comparator.comparingInt(s -> s.prioritaet));
//
//          double rest = sparrate;
//          for (Sparziel sz : sortiert) {
//              if (rest <= 0) break;
//              double fehlend = sz.fehlend();
//              if (fehlend > 0) {
//                  double zuweisung = Math.min(rest, fehlend);
//                  sz.aktuell += zuweisung;
//                  rest -= zuweisung;
//              }
//          }
//          speichereSparziele();
//          System.out.printf("Monatliche Sparrate: %.2f € verteilt.\n",
sparrate);
//      }
//
//      public static void
automatischeZufuehrungBeimStart(ArrayList<FestWert> festeWerte){
//          LocalDate heute = LocalDate.now();
//          LocalDate monatsBeginn = heute.withDayOfMonth(1);
//          if (letzteZufuehrung == null || !
letzteZufuehrung.equals(monatsBeginn)) {
//              monatlicheZufuehrung(festeWerte);
//              letzteZufuehrung = monatsBeginn;
//              speichereSparziele(); // WICHTIG: speichern!
//          }
//      }
//
//      // --- SPARRATE ÄNDERN (EINZIGE METHODE) ---

```

```

//      public static void setzeSparrate(ArrayList<FestWert> festeWerte)
{
//          double aktuell = getSparrateAusFesteWerte(festeWerte);
//          double neu = IO.readDouble("Neue monatliche Sparrate
(aktuell: " + String.format(Locale.US, "%.2f", aktuell) + " €): ");
//          if (neu < 0) return;
//
//////          // Alte entfernen
//////          festeWerte.removeIf(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie));
//////
//////          // Neuen hinzufügen
//////          if (neu > 0) {
//////              LocalDate datum = LocalDate.now().withDayOfMonth(1);
//////              festeWerte.add(new FestWert("Sparplan", neu, false,
datum, "Sparen", 1));
//////          }
//          // Alten Sparplan entfernen
//          festeWerte.removeIf(f -> !f.istEinnahme &&
//                                  "Sparen".equals(f.kategorie) &&
//                                  "Sparplan".equals(f.bezeichnung));
//
//          // Neuen hinzufügen
//          if (neu > 0) {
//              LocalDate datum = LocalDate.now().withDayOfMonth(1);
//              festeWerte.add(new FestWert("Sparplan", neu, false,
datum, "Sparen", 1));
//          }
//          speichereFesteWerte(festeWerte); // <-- SOFORT SPEICHERN!
//          System.out.println("Sparrate aktualisiert und in
festeWerte.txt gespeichert.");
//      }
//
//      // --- FESTEWERTE SPEICHERN (deine Logik) ---
//      public static void speichereFesteWerte(ArrayList<FestWert>
festeWerte) {
//          try (FileWriter writer = new FileWriter("festeWerte.txt")) {
//              for (FestWert f : festeWerte) {
//                  writer.write(String.format(Locale.US, "%s#%.2f#%s#
%s#%s#%d%n",
//                                  f.bezeichnung, f.betrag, f.istEinnahme, f.datum,
f.kategorie, f.periode));
//              }
//              System.out.println("Feste Werte in festeWerte.txt
gespeichert.");
//          } catch (IOException e) {
//              System.err.println("Fehler beim Speichern: " +
e.getMessage());
//          }

```

```

//    }
//
//
//    public static void aendereSparziel() {
//        zeigeSparziele();
//        if (ziele.isEmpty()) return;
//        int idx = IO.readInt("Nummer (0 = Abbruch): ") - 1;
//        if (idx < 0 || idx >= ziele.size()) return;
//
//        Sparziel sz = ziele.get(idx);
//        System.out.println("Aktuell: " + sz);
//
//        // Name
//        String n = IO.readString("Name (" + sz.name + ", Enter =
beibehalten): ");
//        if (!n.isEmpty()) sz.name = n;
//
//        // Zielbetrag
//        double b = IO.readDouble("Zielbetrag (" + sz.zielbetrag + ",
0 = beibehalten): ");
//        if (b > 0) sz.zielbetrag = b;
//
//        // Datum – NUR ÄNDERN, WENN ETWAS EINGETIPPT WURDE
//        LocalDate neuesDatum = IO.readDate("Datum (" + sz.zielDatum
+ "): ");
//        if (neuesDatum != null) {
//            sz.zielDatum = neuesDatum;
//        }
//
//        // Priorität
//        int p = IO.readInt("Priorität (" + sz.prioritaet + ", 0 =
beibehalten): ");
//        if (p > 0) sz.prioritaet = p;
//
//        speichereSparziele();
//        System.out.println("Sparziel aktualisiert.");
//    }
//
//    --- ANZEIGEN (KORREKT NUMMERIERT) ---
//    public static void zeigeSparziele() {
//        if (ziele.isEmpty()) {
//            System.out.println("Keine Sparziele.");
//            return;
//        }
//        System.out.println("=== SPARZIELE (Prio 1 = höchste) ===");
//        List<Sparziel> sortiert = ziele.stream()
//            .sorted(Comparator.comparingInt(s -> s.prioritaet))
//            .toList();
//        for (int i = 0; i < sortiert.size(); i++) {
//            System.out.println((i + 1) + ": " + sortiert.get(i));

```

```

//      }
//    }
//
//    // --- LÖSCHEN ---
//    public static void loescheSparziel() {
//        zeigeSparziele();
//        if (ziele.isEmpty()) return;
//        int idx = IO.readInt("Löschen (0 = Abbruch): ") - 1;
//        if (idx >= 0 && idx < ziele.size()) {
//            ziele.remove(idx);
//            speichereSparziele();
//        }
//    }
//}
//

```

```

import java.io.*;
import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;

public class SparzielManager {
    private static final List<Sparziel> ziele = new ArrayList<>();
    private static final String DATEI = "sparziel.txt";
    private static LocalDate letzteZufuehrung = null;
    private static final String LETZTE_ZUFUEHRUNG_KEY =
"letzteZufuehrung";

    //
=====
==
    // 1. SPARRATE AUS FESTEN WERTEN LESEN (NUR EIN EINTRAG!)
    //
=====
==
    public static double getSparrateAusFesteWerte(ArrayList<FestWert>
festeWerte) {
        return festeWerte.stream()
            .filter(f -> !f.istEinnahme
                && "Sparen".equals(f.kategorie)
                && "Sparplan".equals(f.bezeichnung)
                && f.periode == 1)
            .mapToDouble(f -> f.betrag)
            .findFirst()
            .orElse(0.0);
    }
}

```

```

//
=====
==
// 2. SPARZIEL KLASSE (mit Fortschrittsbalken)
//
=====
==
    public static class Sparziel {
        String name;
        double zielbetrag;
        LocalDate zielDatum;
        double aktuell;
        int prioritael;

        public Sparziel(String name, double zielbetrag, LocalDate
zielDatum, double aktuell, int prioritael) {
            this.name = name;
            this.zielbetrag = zielbetrag;
            this.zielDatum = zielDatum;
            this.aktuell = aktuell;
            this.prioritael = prioritael;
        }

        public double fehlend() { return Math.max(0, zielbetrag -
aktuell); }
        public double prozent() { return zielbetrag > 0 ? (aktuell /
zielbetrag) * 100 : 0; }

        @Override
        public String toString() {
            int balken = (int) (prozent() / 10);
            return String.format("P%d: %-15s | %6.0f / %6.0f € | [%s
%s] %3.0f%% | %6.0f € fehlen",
                prioritael, name, aktuell, zielbetrag,
                "█".repeat(balken), " ".repeat(10 - balken),
prozent(), fehlend());
        }
    }

//
=====
==
// 3. LADEN AUS DATEI
//
=====
==
    public static void ladeSparziele() {
        ziele.clear();
        File file = new File(DATEI);
    }
}

```

```

        if (!file.exists()) return;

        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) continue;

                if (line.startsWith(LETZTE_ZUFUEHRUNG_KEY + "#")) {
                    String datumStr =
line.substring(LETZTE_ZUFUEHRUNG_KEY.length() + 1).trim();
                    if (!datumStr.isEmpty()) {
                        letzteZufuehrung = LocalDate.parse(datumStr);
                    }
                    continue;
                }

                String[] p = line.split("#", 5);
                if (p.length == 5) {
                    ziele.add(new Sparziel(
                        p[0].trim(),
                        Double.parseDouble(p[1].trim().replace(',', '
')),
                        LocalDate.parse(p[2].trim()),
                        Double.parseDouble(p[3].trim().replace(',', '
')),
                        Integer.parseInt(p[4].trim())
                    ));
                }
            }
            System.out.println("Sparziele geladen: " + ziele.size());
        } catch (Exception e) {
            System.err.println("Fehler beim Laden der Sparziele: " +
e.getMessage());
        }

        //
=====
// 4. SPEICHERN IN DATEI
//
=====
public static void speichereSparziele() {
    try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
    {
        pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
        ziele.stream()

```

```

        .sorted(Comparator.comparingInt(s -> s.prioritaet))
        .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d%n",
            sz.name, sz.zielbetrag, sz.zielDatum, sz.aktuell,
            sz.prioritaet));

        if (letzteZufuehrung != null) {
            pw.println(LETZTE_ZUFUEHRUNG_KEY + "#" +
letzteZufuehrung);
        }
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern der Sparziele: "
+ e.getMessage());
    }
}

//
=====
==
// 5. NEUES SPARZIEL
//
=====
==
    public static void neuesSparziel() {
        String name = IO.readString("Name: ");
        double betrag = IO.readDouble("Zielbetrag: ");
        LocalDate datum = IO.readDate("Ziel-Datum (JJJJ-MM-TT): ");
        int prio = IO.readInt("Priorität (1 = höchste): ");
        ziele.add(new Sparziel(name, betrag, datum, 0.0, prio));
        speichereSparziele();
        System.out.println("Sparziel hinzugefügt.");
    }

//
=====
==
// 6. MONATLICHE ZUFÜHRUNG (PRIORISIERT!)
//
=====
==
    public static void monatlicheZufuehrung(ArrayList<FestWert>
festeWerte) {
        double sparrate = getSparrateAusFesteWerte(festeWerte);
        if (sparrate <= 0 || ziele.isEmpty()) {
            System.out.println("Keine Sparrate oder keine
Sparziele.");
            return;
        }

        List<Sparziel> sortiert = new ArrayList<>(ziele);
        sortiert.sort(Comparator.comparingInt(s -> s.prioritaet));

```

```

        double rest = sparrate;
        for (Sparziel sz : sortiert) {
            if (rest <= 0) break;
            double fehlend = sz.fehlend();
            if (fehlend > 0) {
                double zuweisung = Math.min(rest, fehlend);
                sz.aktuell += zuweisung;
                rest -= zuweisung;
            }
        }

        speichereSparziele();
        System.out.printf("Monatliche Sparrate: %.2f € verteilt.\n",
sparrate);
    }

    //
=====
==
    // 7. AUTOMATISCHE ZUFÜHRUNG BEIM PROGRAMMSTART
    //
=====
==
    public static void
    automatischeZufuehrungBeimStart(ArrayList<FestWert> festeWerte) {
        LocalDate heute = LocalDate.now();
        LocalDate monatsBeginn = heute.withDayOfMonth(1);

        if (letzteZufuehrung == null || !
    letzteZufuehrung.equals(monatsBeginn)) {
            monatlicheZufuehrung(festeWerte);
            letzteZufuehrung = monatsBeginn;
            speichereSparziele(); // WICHTIG: speichert neue
    letzteZufuehrung
        }
    }

    //
=====
==
    // 8. SPARRATE ÄNDERN (ersetzt alten "Sparplan")
    //
=====
==
    public static void setzeSparrate(ArrayList<FestWert> festeWerte) {
        double aktuell = getSparrateAusFesteWerte(festeWerte);
        double neu = IO.readDouble("Neue monatliche Sparrate (aktuell:
" + String.format(Locale.US, "%.2f", aktuell) + " €): ");
        if (neu < 0) return;

```



```

// Alten Sparplan entfernen
festeWerte.removeIf(f -> !f.istEinnahme
                    && "Sparen".equals(f.kategorie)
                    && "Sparplan".equals(f.bezeichnung));

// Neuen hinzufügen
if (neu > 0) {
    LocalDate datum = LocalDate.now().withDayOfMonth(1);
    festeWerte.add(new FestWert("Sparplan", neu, false, datum,
"Sparen", 1));
}

// Sofort speichern!
speichereFesteWerte(festeWerte);
System.out.println("Sparrate aktualisiert: " +
String.format(Locale.US, "%.2f", neu) + " €/Monat");
}

//
=====
==
// 9. FESTEWERTE SPEICHERN (wird von setzeSparrate() genutzt)
//
=====
==
    public static void speichereFesteWerte(ArrayList<FestWert>
festeWerte) {
        try (FileWriter writer = new FileWriter("festeWerte.txt")) {
            for (FestWert f : festeWerte) {
                writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s#%d%n",
                                f.bezeichnung, f.betrag, f.istEinnahme, f.datum,
f.kategorie, f.periode));
            }
            System.out.println("Feste Werte gespeichert (inkl.
Sparrate).");
        } catch (IOException e) {
            System.err.println("Fehler beim Speichern der festen
Werte: " + e.getMessage());
        }
    }

//
=====
==
// 10. SPARZIEL ÄNDERN
//
=====
==

```

```

public static void aendereSparziel() {
    zeigeSparziele();
    if (ziele.isEmpty()) return;

    int idx = IO.readInt("Nummer ändern (0 = Abbruch): ") - 1;
    if (idx < 0 || idx >= ziele.size()) return;

    Sparziel sz = ziele.get(idx);
    System.out.println("Aktuell: " + sz);

    String n = IO.readString("Name (" + sz.name + ", Enter =
beibehalten): ");
    if (!n.isEmpty()) sz.name = n;

    double b = IO.readDouble("Zielbetrag (" + sz.zielbetrag + ", 0
= beibehalten): ");
    if (b > 0) sz.zielbetrag = b;

    LocalDate d = IO.readDate("Ziel-Datum (" + sz.zielDatum + "):
");
    if (d != null) sz.zielDatum = d;

    int p = IO.readInt("Priorität (" + sz.prioritaet + ", 0 =
beibehalten): ");
    if (p > 0) sz.prioritaet = p;

    speichereSparziele();
    System.out.println("Sparziel aktualisiert.");
}

//
=====
// 11. SPARZIELE ANZEIGEN
//
=====
public static void zeigeSparziele() {
    if (ziele.isEmpty()) {
        System.out.println("Keine Sparziele vorhanden.");
        return;
    }
    System.out.println("=== SPARZIELE (Prio 1 = höchste) ===");
    List<Sparziel> sortiert = ziele.stream()
        .sorted(Comparator.comparingInt(s -> s.prioritaet))
        .collect(Collectors.toList());
    for (int i = 0; i < sortiert.size(); i++) {
        System.out.println((i + 1) + ": " + sortiert.get(i));
    }
}

```

```

    }

    //
=====
==
    // 12. SPARZIEL LÖSCHEN
    //
=====
==
    public static void loescheSparziel() {
        zeigeSparziele();
        if (ziele.isEmpty()) return;

        int idx = IO.readInt("Löschen (0 = Abbruch): ") - 1;
        if (idx >= 0 && idx < ziele.size()) {
            ziele.remove(idx);
            speichereSparziele();
            System.out.println("Sparziel gelöscht.");
        }
    }

    //
=====
==
    // 13. ALLE SPARZIELE FÜR GUI ZURÜCKGEBEN
    //
=====
==
    public static List<Sparziel> getAllSparziele() {
        return new ArrayList<>(ziele); // Kopie zurückgeben
    }

    //
=====
==
    // 14. SPARZIEL NACH INDEX LÖSCHEN (FÜR GUI)
    //
=====
==
    public static void loescheSparzielByIndex(int index) {
        if (index >= 0 && index < ziele.size()) {
            ziele.remove(index);
            speichereSparziele();
        }
    }

    //
=====
==
    // 15. GUI-FREUNDLICHE METHODEN (ohne Konsoleneingabe)

```

```

//
=====
==

/**
 * Fügt ein neues Sparziel hinzu (für GUI)
 */
public static void neuesSparzielGUI(String name, double
zielbetrag, LocalDate zielDatum, int prioritaet) {
    if (name == null || name.trim().isEmpty()) return;
    if (zielbetrag <= 0) return;
    if (zielDatum == null) return;
    if (prioritaet <= 0) return;

    ziele.add(new Sparziel(name.trim(), zielbetrag, zielDatum,
0.0, prioritaet));
    speichereSparziele();
}

/**
 * Ändert ein Sparziel (für GUI)
 */
public static void aendereSparzielGUI(int index, String name,
Double zielbetrag, LocalDate zielDatum, Integer prioritaet) {
    if (index < 0 || index >= ziele.size()) return;

    Sparziel sz = ziele.get(index);
    if (name != null && !name.trim().isEmpty()) {
        sz.name = name.trim();
    }
    if (zielbetrag != null && zielbetrag > 0) {
        sz.zielbetrag = zielbetrag;
    }
    if (zielDatum != null) {
        sz.zielDatum = zielDatum;
    }
    if (prioritaet != null && prioritaet > 0) {
        sz.prioritaet = prioritaet;
    }

    speichereSparziele();
}

/**
 * Setzt die Sparrate (für GUI)
 */
public static void setzeSparrateGUI(ArrayList<FestWert>
festeWerte, double neueSparrate) {
    if (neueSparrate < 0) return;

```

```

        // Alten Sparplan entfernen
        festeWerte.removeIf(f -> !f.istEinnahme
                                && "Sparen".equals(f.kategorie)
                                && "Sparplan".equals(f.bezeichnung));

        // Neuen hinzufügen
        if (neueSparrate > 0) {
            LocalDate datum = LocalDate.now().withDayOfMonth(1);
            festeWerte.add(new FestWert("Sparplan", neueSparrate,
false, datum, "Sparen", 1));
        }

        // Sofort speichern!
        speichereFesteWerte(festeWerte);
    }

    /**
     * Gibt ein Sparziel nach Index zurück (für GUI)
     */
    public static Sparziel getSparzielByIndex(int index) {
        if (index >= 0 && index < ziele.size()) {
            return ziele.get(index);
        }
        return null;
    }
}

```

Klasse: SparzielTabController

```

package HaushaltskassenApp_zwei;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import java.time.LocalDate;
import java.util.List;

/**
 * Controller für den Sparziel-Tab
 *
 * Verwaltet Sparziele und Sparrate
 */
public class SparzielTabController {

    private DataModel dataModel;
    private TableView<SparzielManager.Sparziel> tableView;
    private VBox content;

```

```

public SparzielTabController(DataModel dataModel) {
    this.dataModel = dataModel;
    createUI();
}

private void createUI() {
    content = new VBox(10);
    content.setPadding(new Insets(10));

    Label title = new Label("Sparziel-Verwaltung");
    title.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

    // Sparrate anzeigen
    Label sparrateLabel = new Label();
    updateSparrateLabel(sparrateLabel);

    // Toolbar
    HBox toolbar = new HBox(10);
    toolbar.setPadding(new Insets(5));

    Button addBtn = new Button("Neues Sparziel");
    addBtn.setOnAction(e -> {
        addSparziel();
        refresh();
    });

    Button editBtn = new Button("Sparziel ändern");
    editBtn.setOnAction(e -> {
        editSparziel();
        refresh();
    });

    Button deleteBtn = new Button("Sparziel löschen");
    deleteBtn.setOnAction(e -> {
        deleteSparziel();
        refresh();
    });

    Button sparrateBtn = new Button("Sparrate ändern");
    sparrateBtn.setOnAction(e -> {
        changeSparrate();
        updateSparrateLabel(sparrateLabel);
        refresh();
    });

    Button refreshBtn = new Button("Aktualisieren");
    refreshBtn.setOnAction(e -> {
        refresh();
        updateSparrateLabel(sparrateLabel);
    });
}

```

```

        toolbar.getChildren().addAll(addBtn, editBtn, deleteBtn,
sparrateBtn, refreshBtn);

        // Tabelle
        tableView = new TableView<>();
        createTableColumns();

        content.getChildren().addAll(title, sparrateLabel, toolbar,
tableView);
        VBox.setVgrow(tableView, Priority.ALWAYS);

        refresh();
    }

    private void updateSparrateLabel(Label label) {
        double sparrate =
SparzielManager.getSparrateAusFesteWerte(dataModel.getFesteWerte());
        label.setText(String.format("Monatliche Sparrate: %.2f €",
sparrate));
        label.setStyle("-fx-font-size: 14px; -fx-font-weight: bold;");
    }

    private void createTableColumns() {
        TableColumn<SparzielManager.Sparziel, String> nameCol = new
TableColumn<>("Name");
        nameCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleStringProperty(param.getValue().name));
        nameCol.setPrefWidth(200);

        TableColumn<SparzielManager.Sparziel, Double> zielCol = new
TableColumn<>("Zielbetrag");
        zielCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleDoubleProperty(param.getValue().zielbetrag
).asObject());
        zielCol.setCellFactory(column -> new
TableCell<SparzielManager.Sparziel, Double>() {
            @Override
            protected void updateItem(Double item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                } else {
                    setText(String.format("%.2f €", item));
                }
            }
        });
        zielCol.setPrefWidth(120);
    }

```

```

        TableColumn<SparzielManager.Sparziel, Double> aktuellCol = new
TableColumn<>("Aktuell");
        aktuellCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleDoubleProperty(param.getValue().aktuell).a
sObject());
        aktuellCol.setCellFactory(column -> new
TableCell<SparzielManager.Sparziel, Double>() {
            @Override
            protected void updateItem(Double item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                } else {
                    setText(String.format("%.2f €", item));
                }
            }
        });
        aktuellCol.setPrefWidth(120);

        TableColumn<SparzielManager.Sparziel, Double> prozentCol = new
TableColumn<>("Fortschritt");
        prozentCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleDoubleProperty(param.getValue().prozent())
.asObject());
        prozentCol.setCellFactory(column -> new
TableCell<SparzielManager.Sparziel, Double>() {
            @Override
            protected void updateItem(Double item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                    setStyle("");
                } else {
                    setText(String.format("%.1f%%", item));
                    // Fortschrittsbalken-Simulation durch Textfarbe
                    if (item >= 100) {
                        setStyle("-fx-text-fill: green;");
                    } else if (item >= 50) {
                        setStyle("-fx-text-fill: orange;");
                    } else {
                        setStyle("-fx-text-fill: red;");
                    }
                }
            }
        });
        prozentCol.setPrefWidth(100);

```



```

        TableColumn<SparzielManager.Sparziel, LocalDate> datumCol =
new TableColumn<>("Zieldatum");
        datumCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleObjectProperty<>(param.getValue().zielDatum));
        datumCol.setCellFactory(column -> new
TableCell<SparzielManager.Sparziel, LocalDate>() {
            @Override
            protected void updateItem(LocalDate item, boolean empty) {
                super.updateItem(item, empty);
                if (empty || item == null) {
                    setText(null);
                } else {
                    setText(item.format(java.time.format.DateTimeFormatter.ofPattern("dd.MM.yyyy"))));
                }
            }
        });
        datumCol.setPrefWidth(120);

        TableColumn<SparzielManager.Sparziel, Integer> prioCol = new
TableColumn<>("Priorität");
        prioCol.setCellValueFactory(param ->
            new
javafx.beans.property.SimpleIntegerProperty(param.getValue().prioritaet).asObject());
        prioCol.setPrefWidth(80);

        tableView.getColumns().addAll(nameCol, zielCol, aktuellCol,
prozentCol, datumCol, prioCol);
    }

    private void addSparziel() {
        String name = GUIDialogHelper.showTextInputDialog("Neues
Sparziel", "Name:", "Name:");
        if (name == null || name.trim().isEmpty()) return;

        Double zielbetrag =
GUIDialogHelper.showDoubleInputDialog("Neues Sparziel", "Zielbetrag:",
"Zielbetrag:");
        if (zielbetrag == null || zielbetrag <= 0) return;

        LocalDate zielDatum =
GUIDialogHelper.showDateInputDialog("Neues Sparziel", "Zieldatum:");
        if (zielDatum == null) return;

        Integer prioritaet = GUIDialogHelper.showIntInputDialog("Neues
Sparziel", "Priorität:", "Priorität (1 = höchste):");
    }

```

```

        if (prioritaet == null || prioritaet <= 0) return;

        // Sparziel über SparzielManager hinzufügen (GUI-Version)
        SparzielManager.neuesSparzielGUI(name, zielbetrag, zielDatum,
prioritaet);
        refresh();
        GUIDialogHelper.showInfo("Sparziel hinzugefügt!");
    }

    private void editSparziel() {
        SparzielManager.Sparziel selected =
tableView.getSelectionModel().getSelectedItem();
        if (selected == null) {
            GUIDialogHelper.showError("Bitte wählen Sie ein Sparziel
aus!");
            return;
        }

        int index = tableView.getSelectionModel().getSelectedIndex();

        // Dialog für Bearbeitung
        String name = GUIDialogHelper.showTextInputDialog("Sparziel
bearbeiten",
            "Name (Enter = beibehalten):", "Name:");

        Double zielbetrag =
GUIDialogHelper.showDoubleInputDialog("Sparziel bearbeiten",
            "Zielbetrag (0 = beibehalten):", "Zielbetrag:");

        LocalDate zielDatum =
GUIDialogHelper.showDateInputDialog("Sparziel bearbeiten",
            "Zieldatum (Abbrechen = beibehalten):");

        Integer prioritaet =
GUIDialogHelper.showIntInputDialog("Sparziel bearbeiten",
            "Priorität (0 = beibehalten):", "Priorität:");

        // Sparziel über SparzielManager bearbeiten (GUI-Version)
        SparzielManager.aendereSparzielGUI(index, name,
            (zielbetrag != null && zielbetrag > 0) ? zielbetrag :
null,
            zielDatum,
            (prioritaet != null && prioritaet > 0) ? prioritaet :
null);

        refresh();
        GUIDialogHelper.showInfo("Sparziel aktualisiert!");
    }

    private void deleteSparziel() {

```

```

        SparzielManager.Sparziel selected =
tableView.getSelectionModel().getSelectedItem();
        if (selected == null) {
            GUIDialogHelper.showError("Bitte wählen Sie ein Sparziel
aus!");
            return;
        }

        if (GUIDialogHelper.showConfirmation("Löschen",
            "Möchten Sie dieses Sparziel wirklich löschen?")) {
            int index =
tableView.getSelectionModel().getSelectedIndex();
            SparzielManager.loescheSparzielByIndex(index);
            refresh();
            GUIDialogHelper.showInfo("Sparziel gelöscht!");
        }
    }

    private void changeSparrate() {
        double aktuell =
SparzielManager.getSparrateAusFesteWerte(dataModel.getFesteWerte());
        Double neu = GUIDialogHelper.showDoubleInputDialog(
            "Sparrate ändern",
            "Neue monatliche Sparrate:",
            String.format("Neue Sparrate (aktuell: %.2f €):", aktuell)
        );

        if (neu != null && neu >= 0) {
            // Sparrate über SparzielManager setzen (GUI-Version)

SparzielManager.setzeSparrateGUI(dataModel.getFesteWerte(), neu);
            dataModel.speichereAlleDaten();
            GUIDialogHelper.showInfo("Sparrate aktualisiert!");
        }
    }

    public void refresh() {
        // Sparziele aus SparzielManager laden
        List<SparzielManager.Sparziel> zieleList =
SparzielManager.getAllSparziele();
        ObservableList<SparzielManager.Sparziel> ziele =
FXCollections.observableArrayList(zieleList);
        tableView.setItems(ziele);
    }

    public VBox getContent() {
        return content;
    }
}

```

Klasse: Statistik

```
package HaushaltskassenApp_zwei;
```

```
/**
 * TODO: boolean array mit verschiedenen bedingungen und methoden
 vereinheitlichen!
 */
```

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
```

```
public class Statistik {
```

```
// public static boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme
&& e.datum.plusMonths(1).isAfter(LocalDate.now())};
```

```
// public static double berechneMittelwert(ArrayList<Eintrag>
eintraege ) {
```

```
//     double mittelwert=0.0;
//     double summe=0.0;
//     int count=0;
//     for (Eintrag e :eintraege) {
//         if (!e.istEinnahme) {
//             summe+=e.getBetrag();
//             count++;
//         }
//     }
```

```
//     mittelwert=summe/count;
```

```
//     return mittelwert;
```

```
// }
```

```
// public static void printMittelwert(ArrayList<Eintrag> eintraege) {
//     double a=berechneMittelwert(eintraege);
```

```
//     System.out.println("=====");
```

```
//     System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);
```

```
//     System.out.println("=====");
```

```
// }
```

```
// public static void berechneStandardabweichung(ArrayList<Eintrag>
```

```

    eintraege ) {
//      double standardabweichung=0.0;
//      double temp=0.0;
//      double mittelwert=berechneMittelwert(eintraege);
//      int count=0;
//      for (Eintrag e:eintraege) {
//          if (!e.istEinnahme) {
//              temp+=Math.pow(e.getBetrag()-mittelwert,2 );
//              count++;
//          }
//      }
//      if (count>1) {
//          standardabweichung=Math.sqrt(temp/(count-1));
//
//      System.out.println("=====");
//      System.out.printf("Standardabweichung der Ausgaben: %13.2f€\n",standardabweichung);
//
//      System.out.println("=====");
//
//      }
//      else {
//          System.out.println("Eine Standardabweichung macht keinen
//      Sinn bei der Menge an Daten!");
//      }
//  }
//
//
//  public static void  berechneMitKategorie(ArrayList<Eintrag>
//  eintraege){
//
//
//
//      Set<String> kategorienSet = new HashSet<>();
//      for (Eintrag e : eintraege) {
//          if (!e.istEinnahme) {
//              kategorienSet.add(e.getKategorie());
//          }
//      }
//
//
//      List<String> kategorienListe = new ArrayList<>(kategorienSet);
//      Collections.sort(kategorienListe);
//
//      System.out.println("Wähle die gewünschten Kategorien aus:");
//      for (int i = 0; i < kategorienListe.size(); i++) {
//          System.out.println((i + 1) + ": " +
//      kategorienListe.get(i));
//      }
//      System.out.println("0: Auswahl beenden");
//
//      Set<String> ausgewaehlteKategorien = new HashSet<>();

```

```

//      while (true) {
//          int auswahl = IO.readInt("Nummer eingeben: ");
//          if (auswahl == 0) break;
//          if (auswahl > 0 && auswahl <= kategorienListe.size()) {
//              ausgewaehlteKategorien.add(kategorienListe.get(auswahl -
1));
//          } else {
//              System.out.println("Ungültige Auswahl!");
//          }
//      }
//      System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);
//
//
//
//
//      ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
//      for (Eintrag e: einträge) {
//          if(!e.istEinnahme) {
//              if (ausgewaehlteKategorien.contains(e.kategorie)) {
//                  auswahlbeträge.add(e.betrag);
//              }
//          }
//      }
//
//      double mit=0.0;
//      double sum=0.0;
//      double var=0.0;
//      double stdabweichung=0.0;
//      for (double a:auswahlbeträge) {
//          sum+=a;
//      }
//      mit=sum/auswahlbeträge.size();
//      if (auswahlbeträge.size()>1) {
//          for (double a:auswahlbeträge) {
//              var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
//          }
//      }
//      else {
//          var=0.0;
//      }
//      stdabweichung=Math.sqrt(var);
//
System.out.println("=====");
;
//      System.out.println("In den von Ihnen ausgewählten
Kategorien sind:");
//      System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
//      System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);

```

```

n",stdabweichung);
//
System.out.println("=====");
;
//
//
//
// }
// public static double berechneMittelwertMonat(ArrayList<Eintrag>
eintraege ) {
//     double mittelwert=0.0;
//     double summe=0.0;
//     int count=0;
//     for (Eintrag e :eintraege) {
//         if (!e.istEinnahme) {
// schonmal was von && gehört???
//             if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//                 summe+=e.getBetrag();
//                 count++;
//             }
//         }
//     }
//     mittelwert=summe/count;
//     return mittelwert;
// }
// public static void printMittelwertMonat(ArrayList<Eintrag>
eintraege) {
//     double a=berechneMittelwert(eintraege);
//
System.out.println("=====");
//     System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);
//
System.out.println("=====");
// }
//
// public static void
berechneStandardabweichungMonat(ArrayList<Eintrag> eintraege ) {
//     double standardabweichung=0.0;
//     double temp=0.0;
//     double mittelwert=berechneMittelwert(eintraege);
//     int count=0;
//     for (Eintrag e:eintraege) {
//         if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
// schonmal was von && gehört???
//             if (!e.istEinnahme) {
//                 temp+=Math.pow(e.getBetrag()-mittelwert,2 );

```

```

//          count++;
//      }
//  }
//  }
//  if (count>1) {
//      standardabweichung=Math.sqrt(temp/(count-1));
//
System.out.println("=====");
//      System.out.printf("Standardabweichung der Ausgaben: %13.2f€\n",standardabweichung);
//
System.out.println("=====");
//
//  }
//  else {
//      System.out.println("Eine Standardabweichung macht keinen
Sinn bei der Menge an Daten!");
//  }
// }
//
//
// public static void berechneMitKategorieMonat(ArrayList<Eintrag>
eintraege){
//
//
//      Set<String> kategorienSet = new HashSet<>();
//      for (Eintrag e : eintraege) {
//          if (!e.istEinnahme) {
//schonmal was von && gehört???
//              if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//                  kategorienSet.add(e.getKategorie());
//              }
//          }
//      }
//
//      List<String> kategorienListe = new ArrayList<>(kategorienSet);
//      Collections.sort(kategorienListe);
//
//      System.out.println("Wähle die gewünschten Kategorien aus:");
//      for (int i = 0; i < kategorienListe.size(); i++) {
//          System.out.println((i + 1) + ": " +
kategorienListe.get(i));
//      }
//      System.out.println("0: Auswahl beenden");
//
//      Set<String> ausgewaehlteKategorien = new HashSet<>();
//      while (true) {
//          int auswahl = IO.readInt("Nummer eingeben: ");
//          if (auswahl == 0) break;
//          if (auswahl > 0 && auswahl <= kategorienListe.size()) {

```



```

//          ausgewaehlteKategorien.add(kategorienListe.get(auswahl -
1));
//          } else {
//          System.out.println("Ungültige Auswahl!");
//          }
//      }
//      System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);
//
//
//
//
//      ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
//      for (Eintrag e:einträge) {
//          if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//              if(!e.istEinnahme) {
//                  if (ausgewaehlteKategorien.contains(e.kategorie))
//                  {
//                      auswahlbeträge.add(e.betrag);
//                  }
//              }
//          }
//      }
//
//      double mit=0.0;
//      double sum=0.0;
//      double var=0.0;
//      double stdabweichung=0.0;
//      for (double a:auswahlbeträge) {
//          sum+=a;
//      }
//      mit=sum/auswahlbeträge.size();
//      if (auswahlbeträge.size()>1) {
//          for (double a:auswahlbeträge) {
//              var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
//          }
//      }
//      else {
//          var=0.0;
//      }
//      stdabweichung=Math.sqrt(var);
//
System.out.println("=====");
;
//      System.out.println("In den von Ihnen ausgewählten
Kategorien sind:");
//      System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
//      System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);

```

```
//
System.out.println("=====");
;
//
//
//
// }
//}
//
```

```
//public static boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme
&& e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
```

```
public static double berechneMittelwert(ArrayList<Eintrag>
eintraege ,int b) {
    double mittelwert=0.0;
    double summe=0.0;
    int count=0;
    for (Eintrag e :eintraege) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if (bedingung[b]) {
            summe+=e.getBetrag();
            count++;
        }
    }
    mittelwert=summe/count;

    return mittelwert;
}
```

```
public static void printMittelwert(ArrayList<Eintrag> eintraege, int
b) {
    double a=berechneMittelwert(eintraege,b);

    System.out.println("=====");
    System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);

    System.out.println("=====");
}
```

```
public static void berechneStandardabweichung(ArrayList<Eintrag>
eintraege, int b ) {
    double standardabweichung=0.0;
```

```

        double temp=0.0;
        double mittelwert=berechneMittelwert(eintraege,b);
        int count=0;
        for (Eintrag e:eintraege) {
            boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
            if (bedingung[b]) {
                temp+=Math.pow(e.getBetrag()-mittelwert,2 );
                count++;
            }
        }
        if (count>1) {
            standardabweichung=Math.sqrt(temp/(count-1));

System.out.println("=====");
            System.out.printf("Standardabweichung der Ausgaben: %13.2f€\n",standardabweichung);

System.out.println("=====");

        }
        else {
            System.out.println("Eine Standardabweichung macht keinen Sinn
bei der Menge an Daten!");
        }
    }

public static void berechneMitKategorie(ArrayList<Eintrag> eintraege,
int b){

    Set<String> kategorienSet = new HashSet<>();
    for (Eintrag e : eintraege) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if (bedingung[b]) {
            kategorienSet.add(e.getKategorie());
        }
    }

    List<String> kategorienListe = new ArrayList<>(kategorienSet);
    Collections.sort(kategorienListe);

    System.out.println("Wähle die gewünschten Kategorien aus:");
    for (int i = 0; i < kategorienListe.size(); i++) {

```

```

        System.out.println((i + 1) + ": " + kategorienListe.get(i));
    }
    System.out.println("0: Auswahl beenden");

    Set<String> ausgewaehlteKategorien = new HashSet<>();
    while (true) {
        int auswahl = IO.readInt("Nummer eingeben: ");
        if (auswahl == 0) break;
        if (auswahl > 0 && auswahl <= kategorienListe.size()) {
            ausgewaehlteKategorien.add(kategorienListe.get(auswahl - 1));
        } else {
            System.out.println("Ungültige Auswahl!");
        }
    }
    System.out.println("Gewählte Kategorien: " +
        ausgewaehlteKategorien);

```

```

    ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
    for (Eintrag e:einträge) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if(bedingung[b]) {
            if (ausgewaehlteKategorien.contains(e.kategorie)) {
                auswahlbeträge.add(e.betrag);
            }
        }
    }

    double mit=0.0;
    double sum=0.0;
    double var=0.0;
    double stdabweichung=0.0;
    for (double a:auswahlbeträge) {
        sum+=a;
    }
    mit=sum/auswahlbeträge.size();
    if (auswahlbeträge.size()>1) {
        for (double a:auswahlbeträge) {
            var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
        }
    }
    else {
        var=0.0;
    }
    stdabweichung=Math.sqrt(var);

```

```

System.out.println("=====")
;
    System.out.println("In den von Ihnen ausgewählten Kategorien
sind:");
    System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
    System.out.printf("Standardabweichung der Ausgaben: %14.2f€\
n",stdabweichung);

System.out.println("=====")
;

    }

```

```

}

```

Klasse: Statistik

```

package HaushaltskassenApp_zwei;

```

```

/**
 * TODO: boolean array mit verschiedenen bedingungen und methoden
vereinheitlichen!
 */

```

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

```

```

public class Statistik {

```

```

// public static boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme
&& e.datum.plusMonths(1).isAfter(LocalDate.now())};

```

```

// public static double berechneMittelwert(ArrayList<Eintrag>
eintraege ) {
//     double mittelwert=0.0;
//     double summe=0.0;
//     int count=0;

```

```

//      for (Eintrag e :eintraege) {
//          if (!e.istEinnahme) {
//              summe+=e.getBetrag();
//              count++;
//          }
//      }
//      mittelwert=summe/count;
//
//      return mittelwert;
//  }
//
//  public static void printMittelwert(ArrayList<Eintrag> eintraege) {
//      double a=berechneMittelwert(eintraege);
//
//      System.out.println("=====");
//      System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);
//
//      System.out.println("=====");
//  }
//
//  public static void berechneStandardabweichung(ArrayList<Eintrag>
eintraege ) {
//      double standardabweichung=0.0;
//      double temp=0.0;
//      double mittelwert=berechneMittelwert(eintraege);
//      int count=0;
//      for (Eintrag e:eintraege) {
//          if (!e.istEinnahme) {
//              temp+=Math.pow(e.getBetrag()-mittelwert,2 );
//              count++;
//          }
//      }
//      if (count>1) {
//          standardabweichung=Math.sqrt(temp/(count-1));
//
//      System.out.println("=====");
//      System.out.printf("Standardabweichung der Ausgaben: %13.2f€\n",standardabweichung);
//
//      System.out.println("=====");
//
//      }
//      else {
//          System.out.println("Eine Standardabweichung macht keinen
Sinn bei der Menge an Daten!");
//      }
//  }
//
//

```

```

// public static void berechneMitKategorie(ArrayList<Eintrag>
eintraege){
//
//
//      Set<String> kategorienSet = new HashSet<>();
//      for (Eintrag e : eintraege) {
//          if (!e.istEinnahme) {
//              kategorienSet.add(e.getKategorie());
//          }
//      }
//
//      List<String> kategorienListe = new ArrayList<>(kategorienSet);
//      Collections.sort(kategorienListe);
//
//      System.out.println("Wähle die gewünschten Kategorien aus:");
//      for (int i = 0; i < kategorienListe.size(); i++) {
//          System.out.println((i + 1) + ": " +
kategorienListe.get(i));
//      }
//      System.out.println("0: Auswahl beenden");
//
//      Set<String> ausgewaehlteKategorien = new HashSet<>();
//      while (true) {
//          int auswahl = IO.readInt("Nummer eingeben: ");
//          if (auswahl == 0) break;
//          if (auswahl > 0 && auswahl <= kategorienListe.size()) {
//              ausgewaehlteKategorien.add(kategorienListe.get(auswahl -
1));
//          } else {
//              System.out.println("Ungültige Auswahl!");
//          }
//      }
//      System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);
//
//
//
//
//      ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
//      for (Eintrag e:eintraege) {
//          if(!e.istEinnahme) {
//              if (ausgewaehlteKategorien.contains(e.kategorie)) {
//                  auswahlbeträge.add(e.betrag);
//              }
//          }
//      }
//
//      double mit=0.0;
//      double sum=0.0;
//      double var=0.0;

```

```

//      double stdabweichung=0.0;
//      for (double a:auswahlbeträge) {
//          sum+=a;
//      }
//      mit=sum/auswahlbeträge.size();
//      if (auswahlbeträge.size()>1) {
//          for (double a:auswahlbeträge) {
//              var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
//          }
//      }
//      else {
//          var=0.0;
//      }
//      stdabweichung=Math.sqrt(var);
//
System.out.println("=====");
;
//      System.out.println("In den von Ihnen ausgewählten
Kategorien sind:");
//      System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
//      System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);
//
System.out.println("=====");
;
//
//
//
//  }
//  public static double berechneMittelwertMonat(ArrayList<Eintrag>
einträge ) {
//      double mittelwert=0.0;
//      double summe=0.0;
//      int count=0;
//      for (Eintrag e :einträge) {
//          if (!e.istEinnahme) {
//              //schonmal was von && gehört???
//              if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//                  summe+=e.getBetrag();
//                  count++;
//              }
//          }
//      }
//      mittelwert=summe/count;
//
//      return mittelwert;
//
//  }

```



```

//
// public static void printMittelwertMonat(ArrayList<Eintrag>
eintraege) {
//     double a=berechneMittelwert(eintraege);
//
// System.out.println("=====");
//     System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);
//
// System.out.println("=====");
// }
//
// public static void
berechneStandardabweichungMonat(ArrayList<Eintrag> eintraege ) {
//     double standardabweichung=0.0;
//     double temp=0.0;
//     double mittelwert=berechneMittelwert(eintraege);
//     int count=0;
//     for (Eintrag e:eintraege) {
//         if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
// schonmal was von && gehört???
//             if (!e.istEinnahme) {
//                 temp+=Math.pow(e.getBetrag()-mittelwert,2 );
//                 count++;
//             }
//         }
//     }
//     if (count>1) {
//         standardabweichung=Math.sqrt(temp/(count-1));
//
// System.out.println("=====");
//         System.out.printf("Standardabweichung der Ausgaben: %13.2f€\n",standardabweichung);
//
// System.out.println("=====");
//
//     }
//     else {
//         System.out.println("Eine Standardabweichung macht keinen
Sinn bei der Menge an Daten!");
//     }
// }
//
// public static void berechneMitKategorieMonat(ArrayList<Eintrag>
eintraege){
//
//
//     Set<String> kategorienSet = new HashSet<>();
//     for (Eintrag e : eintraege) {
//         if (!e.istEinnahme) {

```

```

//schonmal was von && gehört???
//          if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//              kategorienSet.add(e.getKategorie());
//          }
//      }
//  }
//
//      List<String> kategorienListe = new ArrayList<>(kategorienSet);
//      Collections.sort(kategorienListe);
//
//      System.out.println("Wähle die gewünschten Kategorien aus:");
//      for (int i = 0; i < kategorienListe.size(); i++) {
//          System.out.println((i + 1) + ": " +
kategorienListe.get(i));
//      }
//      System.out.println("0: Auswahl beenden");
//
//      Set<String> ausgewaehlteKategorien = new HashSet<>();
//      while (true) {
//          int auswahl = IO.readInt("Nummer eingeben: ");
//          if (auswahl == 0) break;
//          if (auswahl > 0 && auswahl <= kategorienListe.size()) {
//              ausgewaehlteKategorien.add(kategorienListe.get(auswahl -
1));
//          } else {
//              System.out.println("Ungültige Auswahl!");
//          }
//      }
//      System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);
//
//
//
//
//      ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
//      for (Eintrag e: einträge) {
//          if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//              if(!e.istEinnahme) {
//                  if (ausgewaehlteKategorien.contains(e.kategorie))
//                  {
//                      auswahlbeträge.add(e.betrag);
//                  }
//              }
//          }
//      }
//
//      double mit=0.0;
//      double sum=0.0;
//      double var=0.0;
//      double stdabweichung=0.0;

```

```

//      for (double a:auswahlbeträge) {
//          sum+=a;
//      }
//      mit=sum/auswahlbeträge.size();
//      if (auswahlbeträge.size()>1) {
//          for (double a:auswahlbeträge) {
//              var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
//          }
//      }
//      else {
//          var=0.0;
//      }
//      stdabweichung=Math.sqrt(var);
//
System.out.println("=====");
;
//      System.out.println("In den von Ihnen ausgewählten
Kategorien sind:");
//      System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
//      System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);
//
System.out.println("=====");
;
//
//
//
//  }
//}
//

```

```

//public static boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme
&& e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};

```

```

public static double berechneMittelwert(ArrayList<Eintrag>
einträge ,int b) {
    double mittelwert=0.0;
    double summe=0.0;
    int count=0;
    for (Eintrag e :einträge) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if (bedingung[b]) {
            summe+=e.getBetrag();

```

```

        count++;
    }
}
mittelwert=summe/count;

return mittelwert;

}

public static void printMittelwert(ArrayList<Eintrag> eintraege, int
b) {
    double a=berechneMittelwert(eintraege,b);

    System.out.println("=====");
    System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);

    System.out.println("=====");
}

public static void berechneStandardabweichung(ArrayList<Eintrag>
eintraege, int b ) {
    double standardabweichung=0.0;
    double temp=0.0;
    double mittelwert=berechneMittelwert(eintraege,b);
    int count=0;
    for (Eintrag e:eintraege) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if (bedingung[b]) {
            temp+=Math.pow(e.getBetrag()-mittelwert,2 );
            count++;
        }
    }
    if (count>1) {
        standardabweichung=Math.sqrt(temp/(count-1));

        System.out.println("=====");
        System.out.printf("Standardabweichung der Ausgaben: %13.2f€\
n",standardabweichung);

        System.out.println("=====");

    }
    else {
        System.out.println("Eine Standardabweichung macht keinen Sinn
bei der Menge an Daten!");
    }
}

```

```
public static void berechneMitKategorie(ArrayList<Eintrag> eintraege,
int b){
```

```
    Set<String> kategorienSet = new HashSet<>();
    for (Eintrag e : eintraege) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if (bedingung[b]) {
            kategorienSet.add(e.getKategorie());
        }
    }

    List<String> kategorienListe = new ArrayList<>(kategorienSet);
    Collections.sort(kategorienListe);

    System.out.println("Wähle die gewünschten Kategorien aus:");
    for (int i = 0; i < kategorienListe.size(); i++) {
        System.out.println((i + 1) + ": " + kategorienListe.get(i));
    }
    System.out.println("0: Auswahl beenden");
```

```
    Set<String> ausgewaehlteKategorien = new HashSet<>();
    while (true) {
        int auswahl = IO.readInt("Nummer eingeben: ");
        if (auswahl == 0) break;
        if (auswahl > 0 && auswahl <= kategorienListe.size()) {
            ausgewaehlteKategorien.add(kategorienListe.get(auswahl - 1));
        } else {
            System.out.println("Ungültige Auswahl!");
        }
    }
    System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);
```

```
    ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
    for (Eintrag e: eintraege) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if(bedingung[b]) {
            if (ausgewaehlteKategorien.contains(e.kategorie)) {
```

```

        auswahlbeträge.add(e.betrag);
    }
}

double mit=0.0;
double sum=0.0;
double var=0.0;
double stdabweichung=0.0;
for (double a:auswahlbeträge) {
    sum+=a;
}
mit=sum/auswahlbeträge.size();
if (auswahlbeträge.size()>1) {
    for (double a:auswahlbeträge) {
        var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
    }
}
else {
    var=0.0;
}
stdabweichung=Math.sqrt(var);

System.out.println("=====");
;
    System.out.println("In den von Ihnen ausgewählten Kategorien
sind:");
    System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
    System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);

System.out.println("=====");
;

}

```

```

}

```

Klasse: StatistikTabController

```

package HaushaltskassenApp_zwei;

```

```

import javafx.geometry.Insets;
import javafx.scene.control.*;
import javafx.scene.layout.*;

```

```

import java.util.ArrayList;
import java.util.HashSet;
import java.util.Set;

/**
 * Controller für den Statistik-Tab
 *
 * Zeigt statistische Auswertungen der Ausgaben
 */
public class StatistikTabController {

    private DataModel dataModel;
    private VBox content;

    public StatistikTabController(DataModel dataModel) {
        this.dataModel = dataModel;
        createUI();
    }

    private void createUI() {
        content = new VBox(10);
        content.setPadding(new Insets(10));

        Label title = new Label("Statistik");
        title.setStyle("-fx-font-size: 18px; -fx-font-weight: bold;");

        VBox buttonBox = new VBox(10);
        buttonBox.setPadding(new Insets(10));

        Button allStatsBtn = new Button("Statistik mit allen
Kategorien");
        allStatsBtn.setOnAction(e -> showAllStats(0));

        Button selectedStatsBtn = new Button("Statistik mit
ausgewählten Kategorien");
        selectedStatsBtn.setOnAction(e -> showSelectedStats(0));

        Button lastMonthAllBtn = new Button("Statistik - Letzter Monat
(alle)");
        lastMonthAllBtn.setOnAction(e -> showAllStats(1));

        Button lastMonthSelectedBtn = new Button("Statistik - Letzter
Monat (ausgewählt)");
        lastMonthSelectedBtn.setOnAction(e -> showSelectedStats(1));

        Button secondLastMonthAllBtn = new Button("Statistik -
Vorletzter Monat (alle)");
        secondLastMonthAllBtn.setOnAction(e -> showAllStats(2));

        Button secondLastMonthSelectedBtn = new Button("Statistik -

```

```

Vorletzter Monat (ausgewählt)");
    secondLastMonthSelectedBtn.setOnAction(e ->
showSelectedStats(2));

    Button compareBtn = new Button("Vergleich der letzten 2
Monate");
    compareBtn.setOnAction(e -> showComparison());

    buttonBox.getChildren().addAll(allStatsBtn, selectedStatsBtn,
lastMonthAllBtn,
    lastMonthSelectedBtn, secondLastMonthAllBtn,
secondLastMonthSelectedBtn, compareBtn);

    content.getChildren().addAll(title, buttonBox);
}

private void showAllStats(int periode) {
    double mittelwert =
Statistik.berechneMittelwert(dataModel.getEintraege(), periode);
    double standardabweichung =
berechneStandardabweichung(dataModel.getEintraege(), periode);

    String periodeText = periode == 0 ? "Alle" : periode == 1 ?
"Letzter Monat" : "Vorletzter Monat";

    String message = String.format(
        "Statistik (%s):\n\n" +
        "Mittelwert: %.2f €\n" +
        "Standardabweichung: %.2f €",
        periodeText, mittelwert, standardabweichung
    );

    GUIDialogHelper.showInfo(message);
}

private void showSelectedStats(int periode) {
    // Kategorien sammeln
    Set<String> kategorien = new HashSet<>();
    for (Eintrag e : dataModel.getEintraege()) {
        if (!e.istEinnahme) {
            kategorien.add(e.kategorie);
        }
    }

    if (kategorien.isEmpty()) {
        GUIDialogHelper.showError("Keine Kategorien gefunden!");
        return;
    }

    // Kategorien-Auswahl-Dialog

```



```

ChoiceDialog<String> dialog = new ChoiceDialog<>();
dialog.setTitle("Kategorie auswählen");
dialog.setHeaderText("Bitte wählen Sie eine Kategorie:");
dialog.getItems().addAll(kategorien);

String selected = dialog.showAndWait().orElse(null);
if (selected == null) return;

// Statistik für ausgewählte Kategorie berechnen
ArrayList<Eintrag> gefiltert = new ArrayList<>();
for (Eintrag e : dataModel.getEintraege()) {
    if (!e.istEinnahme && e.kategorie.equals(selected)) {
        gefiltert.add(e);
    }
}

if (gefiltert.isEmpty()) {
    GUIDialogHelper.showError("Keine Daten für diese Kategorie
gefunden!");
    return;
}

double mittelwert = Statistik.berechneMittelwert(gefiltert,
periode);
double standardabweichung =
berechneStandardabweichung(gefiltert, periode);

String periodeText = periode == 0 ? "Alle" : periode == 1 ?
"Letzter Monat" : "Vorletzter Monat";

String message = String.format(
    "Statistik für Kategorie '%s' (%s):\n\n" +
    "Mittelwert: %.2f €\n" +
    "Standardabweichung: %.2f €",
    selected, periodeText, mittelwert, standardabweichung
);

GUIDialogHelper.showInfo(message);
}

private void showComparison() {
    double mit1 =
Math.round(Statistik.berechneMittelwert(dataModel.getEintraege(), 1) *
100) / 100.0;
    double mit2 =
Math.round(Statistik.berechneMittelwert(dataModel.getEintraege(), 2) *
100) / 100.0;

    String vergleich = "Die Ausgaben blieben gleich!";
    double diff = 0.0;

```

```

        if (mit1 != mit2) {
            diff = mit1 > mit2 ? mit1 - mit2 : mit2 - mit1;
            vergleich = String.format("Die Ausgaben wurden also %.2f €
%s!",
                                    diff, mit1 > mit2 ? "mehr" : "weniger");
        }

        String message = String.format(
            "Vergleich der letzten Mittelwerte:\n\n" +
            "Vorletzter Monat: %.2f €\n" +
            "Letzter Monat: %.2f €\n\n" +
            "%s",
            mit2, mit1, vergleich
        );

        GUIDialogHelper.showInfo(message);
    }

    private double berechneStandardabweichung(ArrayList<Eintrag>
    eintraege, int periode) {
        double mittelwert = Statistik.berechneMittelwert(eintraege,
        periode);
        double temp = 0.0;
        int count = 0;

        for (Eintrag e : eintraege) {
            boolean[] bedingung = {
                !e.istEinnahme,
                !e.istEinnahme &&
e.datum.plusMonths(1).isAfter(java.time.LocalDate.now()),
                !e.istEinnahme &&
e.datum.plusMonths(2).isAfter(java.time.LocalDate.now())
                && !
e.datum.plusMonths(1).isAfter(java.time.LocalDate.now())
            };

            if (bedingung[periode]) {
                temp += Math.pow(e.betrag - mittelwert, 2);
                count++;
            }
        }

        if (count > 1) {
            return Math.sqrt(temp / (count - 1));
        }
        return 0.0;
    }

    public void refresh() {
        // Statistik-Daten aktualisieren falls nötig
    }

```

```

    }

    public VBox getContent() {
        return content;
    }
}

```

Klasse: Uebersicht

```
package HaushaltskassenApp_zwei;
```

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Comparator;
```

```
public class Uebersicht {

    public static void zeigeAusgaben(ArrayList<Eintrag> eintraege) {
        System.out.println("Ausgaben:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|   Bezeichnung |   Betrag   |");

System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %15s| %8.2f€ |\n",count, e.datum,  e.kategorie, e.bezeichnung, e.betrag);
                summe += e.betrag;
            }
            count++;
        }

System.out.println("=====
=====");
        System.out.printf("|Gesamt Ausgaben: %54.2f€ |\n", summe);

System.out.println("=====
=====");

    }

    public static void zeigefesteAusgaben(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Ausgaben:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|   Bezeichnung   |   Periode(Monate) |   Betrag   |");

```

```

System.out.println("-----
-----");

    for (FestWert f : festeWerte) {
        if (!f.istEinnahme) {
            int countE=0;
            while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                countE++;
            }
            for (int i =0;i<countE;i++) {

                System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                summe += f.betrag;
            }
            monatsbeitrag +=f.betrag/f.periode;
        }
        count++;
    }

System.out.println("=====
=====");
    System.out.printf("|Gesamt feste Ausgaben: %77.2f€ |\n",
summe);
    System.out.printf("|Gesamt feste Ausgaben/Monat: %71.2f€ |\n",
monatsbeitrag);

System.out.println("=====
=====");

}

    public static void zeigeEinnahmen(ArrayList<Eintrag> eintraege) {
        System.out.println("Einnahmen:");
        double summe = 0.0;
        int count=0;
        System.out.println("|Index|          Datum   |          Kategorie
|          Bezeichnung          |          Betrag |");

System.out.println("-----
-----");
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                System.out.printf("|%5d|%14s | %20s | %25s | %8.2f€ |\n",count, e.datum, e.kategorie, e.bezeichnung, e.betrag);
                summe += e.betrag;
            }
        }
    }

```

```

        count++;
    }

    System.out.println("=====
=====");
    System.out.printf("|Gesamt Einnahmen: %64.2f€ |\n", summe);

    System.out.println("=====
=====");
}

    public static void zeigefesteEinnahmen(ArrayList<FestWert>
festeWerte) {
        System.out.println("Feste Einnahmen:");
        double summe = 0.0;
        double monatsbeitrag=0.0;
        int count=0;
        System.out.println("|Index|          Datum |          Kategorie
| Bezeichnung | Periode(Monate) | Betrag |");
        System.out.println("-----
-----");
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                int countE=0;
                while (!
LocalDate.now().isBefore(f.datum.plusMonths(countE*f.periode))) {
                    countE++;
                }
                for (int i =0;i<countE;i++) {

                    System.out.printf("|%5d|%14s | %20s | %25s| %17d|
%8.2f€ |\n",count,f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
                    summe += f.betrag;
                }
                monatsbeitrag +=f.betrag/f.periode;
            }
            count++;
        }

        System.out.println("=====
=====");
        System.out.printf("|Gesamt feste Einnahmen: %66.2f€ |\n",
summe);
        System.out.printf("|Gesamt feste Einnahmen/Monat: %60.2f€ |\n",
monatsbeitrag);

        System.out.println("=====
=====");
    }

```

```
}
```

```
    public static void zeigeBilanz(ArrayList<Eintrag> eintraege,
    ArrayList<FestWert> festeWerte) {
        zeigeAusgaben(eintraege);
        zeigefesteAusgaben(festeWerte);
        zeigeEinnahmen(eintraege);
        zeigefesteEinnahmen(festeWerte);
        double bilanzwert=0.0;
        for (FestWert f : festeWerte) {
            if (f.istEinnahme) {
                bilanzwert +=f.betrag;
            }
            else {
                bilanzwert -= f.betrag;
            }
        }
        for (Eintrag e : eintraege) {
            if (e.istEinnahme) {
                bilanzwert +=e.betrag;
            }
            else {
                bilanzwert -= e.betrag;
            }
        }

        System.out.println("=====");
        System.out.printf("| Bilanzwert: %30.2f€ |\n",bilanzwert);

        System.out.println("=====");
    }
}
```

```
    public static void zeigeBalkendiagrammAusgaben(ArrayList<Eintrag>
    eintraege, ArrayList<FestWert> festeWerte) {
        // Sammle Ausgaben pro Kategorie
        var katSum = new java.util.HashMap<String, Double>();

        for (Eintrag e : eintraege) {
            if (!e.istEinnahme) {
                katSum.merge(e.kategorie, e.betrag, Double::sum);
            }
        }
        for (FestWert f : festeWerte) {
            if (!f.istEinnahme) {
                katSum.merge(f.kategorie, f.betrag, Double::sum);
            }
        }
    }
}
```

```

        if (katSum.isEmpty()) {
            System.out.println("Keine Ausgaben vorhanden.");
            return;
        }

        double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
        int balkenLaenge = 20; // Länge des Balkens

        System.out.println("\nAusgaben nach Kategorie:");
        katSum.entrySet().stream()
            .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
            .forEach(e -> {
                int anzahl = (int) (e.getValue() / maxBetrag *
balkenLaenge);
                String balken = "█".repeat(anzahl) + "
".repeat(balkenLaenge - anzahl);
                System.out.printf("%s %s %8.2f €\n", balken,
String.format("%-12s", e.getKey()), e.getValue());
            });
    }

```

```

public static final String RESET = "\u001B[0m";
public static final String GRUEN = "\u001B[32m";
public static final String GELB = "\u001B[33m";
public static final String ROT = "\u001B[31m";

```

```

    public static void zeigeMonatsBalken(ArrayList<Eintrag> eintraege,
ArrayList<FestWert> festeWerte) {
        LocalDate jetzt = LocalDate.now();
        int jahr = jetzt.getYear();
        int monat = jetzt.getMonthValue();

        var katSum = new java.util.HashMap<String, Double>();

        // Normale Einträge
        for (Eintrag e : eintraege) {
            if (!e.istEinnahme && e.datum.getYear() == jahr &&
e.datum.getMonthValue() == monat) {
                katSum.merge(e.kategorie, e.betrag, Double::sum);
            }
        }
    }

```

```

// Feste Werte (wiederkehrend)
for (FestWert f : festeWerte) {
    if (!f.istEinnahme) {
        LocalDate naechste = f.datum;
        while (naechste.isBefore(jetzt.plusMonths(1))) {
            if (naechste.getYear() == jahr &&
naechste.getMonthValue() == monat) {
                katSum.merge(f.kategorie, f.betrag,
Double::sum);
                break;
            }
            naechste = naechste.plusMonths(f.periode);
        }
    }
}

if (katSum.isEmpty()) {
    System.out.println("Keine Ausgaben im aktuellen Monat.");
    return;
}

double maxBetrag =
katSum.values().stream().mapToDouble(Double::doubleValue).max().orElse
(1.0);
int balkenLaenge = 25;

System.out.printf("\n=== %s %d ===\n", jetzt.getMonth(),
jahr);
System.out.println("Budget-Warnung: " + ROT + "Überschritten"
+ RESET + ", " + GELB + "Kritisch" + RESET + ", " + GRUEN + "OK" +
RESET);

katSum.entrySet().stream()
    .sorted(java.util.Map.Entry.<String,
Double>comparingByValue().reversed())
    .forEach(e -> {
        double ausgegeben = e.getValue();
        double budget = BudgetManager.getBudget(e.getKey()); // <-
0.0 wenn nicht vorhanden
        int anzahl = (int) (ausgegeben / maxBetrag *
balkenLaenge);
        String balken;
        String farbe;

        if (budget > 0 && ausgegeben > budget) {
            balken = ROT + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;

```



```

        farbe = ROT;
    } else if (budget > 0 && ausgegeben > budget * 0.9) {
        balken = GELB + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
        farbe = GELB;
    } else {
        balken = GRUEN + "█".repeat(Math.min(anzahl,
balkenLaenge)) + RESET;
        farbe = (budget == 0) ? RESET : GRUEN; // Grau, wenn
kein Budget
    }

    String budgetStr = (budget == 0)
        ? String.format("%8.2f €", ausgegeben)
        : String.format("%8.2f / %.2f €", ausgegeben, budget);

    System.out.printf("%s %s%s %s\n",
        balken + " ".repeat(balkenLaenge - Math.min(anzahl,
balkenLaenge)),
        farbe,
        String.format("%-15s", e.getKey()),
        budgetStr + RESET);
    });
}

```

```

    public static void zeigeGefiltert(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, String kategorie) {
        System.out.println("Einträge in Kategorie: " + kategorie);
        System.out.println("| Datum          | Betrag    | Kategorie
| Bezeichnung      | Art        |Ausgabe/Einnahme|");

        System.out.println("-----
-----");

        String art="";
        boolean gefunden = false;
        for (Eintrag e : eintraege) {
            if (e.kategorie.equals(kategorie)) {
                art= e.istEinnahme ? "Einnahme":"Ausgabe";
                System.out.printf("| %10s | %8.2f | %17s | %15s |
Eintrag    |%16s|\n",
                    e.datum, e.betrag, e.kategorie, e.bezeichnung,
art);
                gefunden = true;
            }
        }
        for (FestWert f: festeWerte) {

```

```

        if (f.kategorie.equals(kategorie)) {
            art= f.istEinnahme ? "Einnahme":"Ausgabe";
            System.out.printf("| %10s | %8.2f | %17s | %15s |
fester Wert|%16s|\n",
                f.datum, f.betrag, f.kategorie, f.bezeichnung,
            art);
            gefunden = true;
        }
    }

    if (!gefunden) {
        System.out.println("=====");
        System.out.println("      Keine Einträge in dieser
Kategorie!");
        System.out.println("=====");
    }
}

```

```

    public static void zeigeSortiertNachBetrag(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean absteigend) {
        ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);
        ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = absteigend ?
            Comparator.comparingDouble((Eintrag e) ->
e.betrag).reversed() : Comparator.comparingDouble((Eintrag e) ->
e.betrag);

        Comparator<FestWert> compF = absteigend ?
            Comparator.comparingDouble((FestWert f) ->
f.betrag).reversed() : Comparator.comparingDouble((FestWert f) ->
f.betrag);

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }
}

```

```

    public static void zeigeSortiertNachDatum(ArrayList<Eintrag>
eintraege, ArrayList<FestWert> festeWerte, boolean a) {

        ArrayList<Eintrag> kopieEintraege = new
ArrayList<>(eintraege);

```

```

        ArrayList<FestWert> kopieFesteWerte = new
ArrayList<>(festeWerte);

        Comparator<Eintrag> compE = Comparator.comparing(e -> e.datum,

            Comparator.nullsLast(Comparator.naturalOrder()));
        Comparator<FestWert> compF = Comparator.comparing(f ->
f.datum,
            Comparator.nullsLast(Comparator.naturalOrder()));

        if (a) {
            compE = compE.reversed();
            compF = compF.reversed();
        }

        kopieEintraege.sort(compE);
        kopieFesteWerte.sort(compF);

        zeigeBilanz(kopieEintraege, kopieFesteWerte);
    }

    public static void zeigeBevorstehendeAusgaben(ArrayList<FestWert>
festeWerte) {

        int tage= IO.readInt("Geben Sie die Tage des Intervalls an:
");
        LocalDate now=LocalDate.now(); // LocalDate.of(2025,12,1);

        System.out.println("=====
=====");
        System.out.println("
Bevorstehende Ausgaben");

        System.out.println("=====
=====");
        System.out.println("| Fälligkeitsdatum |      Kategorie
|
Bezeichnung          | Periode(Monate) |      Betrag |");
        System.out.println("- - - - -
- - - - -");
        double summe=0.0;

        for (FestWert f :festeWerte)

            if (!f.istEinnahme) {
                int countE=0;

```

```

        int countA=0;

        while (!
now.plusDays(tage).isBefore(f.datum.plusMonths(countE*f.periode))) {

            countE++;
        }
        while
(now.isAfter(f.datum.plusMonths(countA*f.periode))) {
//!now.isBefore(f.datum.plusMonths(countA*f.periode))---heute nicht
dabei
            countA++;
        }
        for (int i =countA;i<countE;i++) {
            System.out.printf("|%17s | %20s | %25s| %17d|
%8.2f€ |\n",f.datum.plusMonths(i*f.periode), f.kategorie,
f.bezeichnung, f.periode, f.betrag);
            summe +=f.betrag;
        }

    }

System.out.println("=====
=====");
    if(summe !=0.0) {
        System.out.printf("Die zu erwartende aufzubringende Summe in
den nächsten %"+4+"d Tagen ist: %27.2f€ |\n",tage,summe);
    }
    else {
        System.out.printf("In den nächsten %d Tagen erwartet Dich
keine Ausgabe fester Werte!\n",tage);
    }

System.out.println("=====
=====");

    }

}

```

Klasse: UebersichtTabController

```

package HaushaltskassenApp_zwei;

import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.geometry.Insets;
import javafx.scene.chart.BarChart;
import javafx.scene.chart.CategoryAxis;
import javafx.scene.chart.NumberAxis;
import javafx.scene.chart.XYChart;

```

```

import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.scene.layout.*;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.HashMap;
import java.util.Map;

/**
 * Controller für den Übersicht-Tab
 *
 * Zeigt verschiedene Übersichten, Bilanzen und Diagramme
 */
public class UebersichtTabController {

    private DataModel dataModel;
    private VBox content;
    private TabPane innerTabPane;

    public UebersichtTabController(DataModel dataModel) {
        this.dataModel = dataModel;
        createUI();
    }

    private void createUI() {
        content = new VBox(10);
        content.setPadding(new Insets(10));

        innerTabPane = new TabPane();

        // Tab: Bilanz
        Tab bilanzTab = new Tab("Bilanz");
        bilanzTab.setContent(createBilanzView());

        // Tab: Ausgaben
        Tab ausgabenTab = new Tab("Ausgaben");
        ausgabenTab.setContent(createAusgabenView());

        // Tab: Einnahmen
        Tab einnahmenTab = new Tab("Einnahmen");
        einnahmenTab.setContent(createEinnahmenView());

        // Tab: Gefiltert
        Tab gefiltertTab = new Tab("Gefiltert");
        gefiltertTab.setContent(createGefiltertView());

        // Tab: Sortiert
        Tab sortiertTab = new Tab("Sortiert");
        sortiertTab.setContent(createSortiertView());
    }

```

```

// Tab: Bevorstehende Ausgaben
Tab bevorstehendTab = new Tab("Bevorstehende Ausgaben");
bevorstehendTab.setContent(createBevorstehendView());

// Tab: Diagramme
Tab diagrammTab = new Tab("Diagramme");
diagrammTab.setContent(createDiagrammView());

innerTabPane.getTabs().addAll(bilanzTab, ausgabenTab,
einnahmenTab,
    gefiltertTab, sortiertTab, bevorstehendTab,
diagrammTab);

content.getChildren().add(innerTabPane);
VBox.setVgrow(innerTabPane, Priority.ALWAYS);
}

private VBox createBilanzView() {
    VBox view = new VBox(10);
    view.setPadding(new Insets(10));

    Button refreshBtn = new Button("Aktualisieren");
    refreshBtn.setOnAction(e -> refreshBilanz(view));

    view.getChildren().add(refreshBtn);
    refreshBilanz(view);

    return view;
}

private void refreshBilanz(VBox container) {
    // Alte Inhalte entfernen (außer Button)
    if (container.getChildren().size() > 1) {
        container.getChildren().remove(1,
container.getChildren().size());
    }

    // Bilanz berechnen
    double einnahmen = 0;
    double ausgaben = 0;

    for (Eintrag e : dataModel.getEintraege()) {
        if (e.istEinnahme) {
            einnahmen += e.betrag;
        } else {
            ausgaben += e.betrag;
        }
    }

    for (FestWert f : dataModel.getFesteWerte()) {

```

```

        if (f.istEinnahme) {
            einnahmen += f.betrag;
        } else {
            ausgaben += f.betrag;
        }
    }

    double bilanz = einnahmen - ausgaben;

    // Anzeige
    Label einnahmenLabel = new Label(String.format("Einnahmen:
%.2f €", einnahmen));
    einnahmenLabel.setStyle("-fx-font-size: 14px; -fx-text-fill:
green;");
    Label ausgabenLabel = new Label(String.format("Ausgaben: %.2f
€", ausgaben));
    ausgabenLabel.setStyle("-fx-font-size: 14px; -fx-text-fill:
red;");
    Label bilanzLabel = new Label(String.format("Bilanz: %.2f €",
bilanz));
    bilanzLabel.setStyle(String.format("-fx-font-size: 16px; -fx-
font-weight: bold; -fx-text-fill: %s;",
        bilanz >= 0 ? "green" : "red"));

    VBox bilanzBox = new VBox(5);
    bilanzBox.setPadding(new Insets(10));
    bilanzBox.getChildren().addAll(einnahmenLabel, ausgabenLabel,
bilanzLabel);

    container.getChildren().add(bilanzBox);
}

private VBox createAusgabenView() {
    VBox view = new VBox(10);
    view.setPadding(new Insets(10));

    TableView<Eintrag> table = new TableView<>();
    ObservableList<Eintrag> ausgaben =
FXCollections.observableArrayList();

    for (Eintrag e : dataModel.getEintraege()) {
        if (!e.istEinnahme) {
            ausgaben.add(e);
        }
    }

    table.setItems(ausgaben);
    createEintragTableColumns(table);

    double summe = ausgaben.stream().mapToDouble(e ->

```

```

e.betrag).sum();
    Label summeLabel = new Label(String.format("Gesamt: %.2f €",
summe));
    summeLabel.setStyle("-fx-font-size: 14px; -fx-font-weight:
bold;");

    view.getChildren().addAll(summeLabel, table);
    VBox.setVgrow(table, Priority.ALWAYS);

    return view;
}

private VBox createEinnahmenView() {
    VBox view = new VBox(10);
    view.setPadding(new Insets(10));

    TableView<Eintrag> table = new TableView<>();
    ObservableList<Eintrag> einnahmen =
FXCollections.observableArrayList();

    for (Eintrag e : dataModel.getEintraege()) {
        if (e.istEinnahme) {
            einnahmen.add(e);
        }
    }

    table.setItems(einnahmen);
    createEintragTableColumns(table);

    double summe = einnahmen.stream().mapToDouble(e ->
e.betrag).sum();
    Label summeLabel = new Label(String.format("Gesamt: %.2f €",
summe));
    summeLabel.setStyle("-fx-font-size: 14px; -fx-font-weight:
bold;");

    view.getChildren().addAll(summeLabel, table);
    VBox.setVgrow(table, Priority.ALWAYS);

    return view;
}

private VBox createGefiltertView() {
    VBox view = new VBox(10);
    view.setPadding(new Insets(10));

    HBox filterBox = new HBox(10);
    TextField kategorieField = new TextField();
    kategorieField.setPromptText("Kategorie eingeben");
    Button filterBtn = new Button("Filtern");

```



```

        TableView<Eintrag> table = new TableView<>();
        createEintragTableColumns(table);

        filterBtn.setOnAction(e -> {
            String kategorie = kategorieField.getText().trim();
            if (kategorie.isEmpty()) {
                GUIDialogHelper.showError("Bitte geben Sie eine
Kategorie ein!");
            }
            return;
        })

        ObservableList<Eintrag> gefiltert =
FXCollections.observableArrayList();
        for (Eintrag e : dataModel.getEintraege()) {
            if (e.kategorie.equals(kategorie)) {
                gefiltert.add(e);
            }
        }
        for (FestWert f : dataModel.getFesteWerte()) {
            if (f.kategorie.equals(kategorie)) {
                // FestWert als Eintrag anzeigen (vereinfacht)
                gefiltert.add(new Eintrag(f.bezeichnung, f.betrag,
f.istEinnahme, f.datum, f.kategorie));
            }
        }

        table.setItems(gefiltert);
    });

    filterBox.getChildren().addAll(new Label("Kategorie:"),
kategorieField, filterBtn);
    HBox.setHgrow(kategorieField, Priority.ALWAYS);

    view.getChildren().addAll(filterBox, table);
    VBox.setVgrow(table, Priority.ALWAYS);

    return view;
}

private VBox createSortiertView() {
    VBox view = new VBox(10);
    view.setPadding(new Insets(10));

    HBox sortBox = new HBox(10);
    ComboBox<String> sortCombo = new ComboBox<>();
    sortCombo.getItems().addAll("Nach Betrag (aufsteigend)", "Nach
Betrag (absteigend)",
        "Nach Datum (aufsteigend)", "Nach Datum
(absteigend)");

```

```

        sortCombo.setValue("Nach Betrag (absteigend)");
        Button sortBtn = new Button("Sortieren");

        TableView<Eintrag> table = new TableView<>();
        createEintragTableColumns(table);

        sortBtn.setOnAction(e -> {
            ObservableList<Eintrag> sortiert =
FXCollections.observableArrayList(dataModel.getEintraege());
            String sortierung = sortCombo.getValue();

            if (sortierung.contains("Betrag")) {
                sortiert.sort((e1, e2) -> Double.compare(e1.betrag,
e2.betrag));
                if (sortierung.contains("absteigend")) {
                    FXCollections.reverse(sortiert);
                }
            } else if (sortierung.contains("Datum")) {
                sortiert.sort((e1, e2) ->
e1.datum.compareTo(e2.datum));
                if (sortierung.contains("absteigend")) {
                    FXCollections.reverse(sortiert);
                }
            }

            table.setItems(sortiert);
        });

        sortBox.getChildren().addAll(new Label("Sortierung:"),
sortCombo, sortBtn);

        view.getChildren().addAll(sortBox, table);
        VBox.setVgrow(table, Priority.ALWAYS);

        return view;
    }

    private VBox createBevorstehendView() {
        VBox view = new VBox(10);
        view.setPadding(new Insets(10));

        HBox inputBox = new HBox(10);
        TextField tageField = new TextField();
        tageField.setPromptText("Anzahl Tage");
        Button anzeigenBtn = new Button("Anzeigen");

        TableView<FestWert> table = new TableView<>();
        // Spalten für FestWert (vereinfacht)

        anzeigenBtn.setOnAction(e -> {

```

```

        try {
            int tage = Integer.parseInt(tageField.getText());
            LocalDate now = LocalDate.now();
            ObservableList<FestWert> bevorstehend =
FXCollections.observableArrayList();

            for (FestWert f : dataModel.getFesteWerte()) {
                if (!f.istEinnahme) {
                    LocalDate naechste = f.datum;
                    while (naechste.isBefore(now.plusDays(tage)))
{
                    if (naechste.isAfter(now) ||
naechste.equals(now)) {
                        bevorstehend.add(f);
                        break;
                    }
                    naechste = naechste.plusMonths(f.periode);
                }
            }

            table.setItems(bevorstehend);
        } catch (NumberFormatException ex) {
            GUIDialogHelper.showError("Ungültige Eingabe!");
        }
    });

    inputBox.getChildren().addAll(new Label("Tage:"), tageField,
anzeigenBtn);

    view.getChildren().addAll(inputBox, table);
    VBox.setVgrow(table, Priority.ALWAYS);

    return view;
}

private VBox createDiagrammView() {
    VBox view = new VBox(10);
    view.setPadding(new Insets(10));

    Button refreshBtn = new Button("Diagramm aktualisieren");
    refreshBtn.setOnAction(e -> refreshDiagramm(view));

    view.getChildren().add(refreshBtn);
    refreshDiagramm(view);

    return view;
}

private void refreshDiagramm(VBox container) {

```

```

        if (container.getChildren().size() > 1) {
            container.getChildren().remove(1,
container.getChildren().size());
        }

        // Ausgaben nach Kategorie sammeln
        Map<String, Double> katSum = new HashMap<>();
        for (Eintrag e : dataModel.getEintraege()) {
            if (!e.istEinnahme) {
                katSum.merge(e.kategorie, e.betrag, Double::sum);
            }
        }
        for (FestWert f : dataModel.getFesteWerte()) {
            if (!f.istEinnahme) {
                katSum.merge(f.kategorie, f.betrag, Double::sum);
            }
        }

        // Balkendiagramm erstellen
        CategoryAxis xAxis = new CategoryAxis();
        NumberAxis yAxis = new NumberAxis();
        BarChart<String, Number> barChart = new BarChart<>(xAxis,
yAxis);
        barChart.setTitle("Ausgaben nach Kategorie");

        XYChart.Series<String, Number> series = new
XYChart.Series<>();
        for (Map.Entry<String, Double> entry : katSum.entrySet()) {
            series.getData().add(new XYChart.Data<>(entry.getKey(),
entry.getValue()));
        }

        barChart.getData().add(series);
        barChart.setPrefHeight(400);

        container.getChildren().add(barChart);
    }

    private void createEintragTableColumns(Table<Eintrag> table) {
        TableColumn<Eintrag, String> bezeichnungCol = new
TableColumn<>("Bezeichnung");
        bezeichnungCol.setCellValueFactory(new
PropertyValueFactory<>("bezeichnung"));

        TableColumn<Eintrag, Double> betragCol = new
TableColumn<>("Betrag");
        betragCol.setCellValueFactory(new
PropertyValueFactory<>("betrag"));
        betragCol.setCellFactory(column -> new TableCell<Eintrag,
Double>() {

```

```

        @Override
        protected void updateItem(Double item, boolean empty) {
            super.updateItem(item, empty);
            if (empty || item == null) {
                setText(null);
            } else {
                setText(String.format("%.2f €", item));
            }
        }
    });

    TableColumn<Eintrag, LocalDate> datumCol = new
    TableColumn<>("Datum");
    datumCol.setCellValueFactory(new
    PropertyValueFactory<>("datum"));
    datumCol.setCellFactory(column -> new TableCell<Eintrag,
    LocalDate>() {
        @Override
        protected void updateItem(LocalDate item, boolean empty) {
            super.updateItem(item, empty);
            if (empty || item == null) {
                setText(null);
            } else {
                setText(item.format(DateTimeFormatter.ofPattern("dd.MM.yyyy")));
            }
        }
    });

    TableColumn<Eintrag, String> kategorieCol = new
    TableColumn<>("Kategorie");
    kategorieCol.setCellValueFactory(new
    PropertyValueFactory<>("kategorie"));

    table.getColumns().addAll(bezeichnungCol, betragCol, datumCol,
    kategorieCol);
}

public void refresh() {
    // Alle Untertabs aktualisieren
    if (innerTabPane != null &&
    innerTabPane.getSelectionModel().getSelectedItem() != null) {
        Tab selected =
    innerTabPane.getSelectionModel().getSelectedItem();
        if (selected.getText().equals("Bilanz")) {
            refreshBilanz((VBox) selected.getContent());
        } else if (selected.getText().equals("Diagramme")) {
            refreshDiagramm((VBox) selected.getContent());
        }
    }
}

```

Klasse: Verwaltung

```
try (Scanner fileScanner = new Scanner(new
FileReader("eintraege.txt"))) {
    while (fileScanner.hasNextLine()) {
        String line = fileScanner.nextLine();
        String[] parts = line.split("#");
        if (parts.length == 5) {
            Eintrag e = new Eintrag(
                parts[0].trim(),
                Double.parseDouble(parts[1].trim()),
                Boolean.parseBoolean(parts[2].trim()),
                LocalDate.parse(parts[3].trim()),
```

```

        parts[4].trim()
    );
    eintraege.add(e);
}
}
System.out.println("Einträge aus eintraege.txt geladen.");
} catch (FileNotFoundException e) {
    System.out.println("Keine gespeicherten Einträge gefunden,
    starte mit leerer Liste.");
} catch (Exception e) {
    System.err.println("Fehler beim Laden: " +
    e.getMessage());
}
    festeWerte = new ArrayList<>();
    try (Scanner fileScanner = new Scanner(new
    FileReader("festeWerte.txt"))) {
        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            String[] parts = line.split("#");
            if (parts.length == 6) {
                FestWert f = new FestWert(
                    parts[0].trim(),
                    Double.parseDouble(parts[1].trim()),
                    Boolean.parseBoolean(parts[2].trim()),
                    LocalDate.parse(parts[3].trim()),
                    parts[4].trim(),
                    Integer.parseInt(parts[5].trim())
                );
                festeWerte.add(f);
            }
        }
        System.out.println("Feste Werte aus festeWerte.txt
        geladen.");
    } catch (FileNotFoundException e) {
        System.out.println("Keine festen Werte gefunden, starte
        mit leerer Liste.");
    } catch (Exception e) {
        System.err.println("Fehler beim Laden: " +
        e.getMessage());
    }
}

```

```

    SparzielManager.automatischeZufuehrungBeimStart(festeWerte);
    Eintrag eTesch1=new
    Eintrag("Kühlschrank",588.95,false,LocalDate.of(2025,10,27),"Einmal-
    Anschaffung");
    Eintrag eTesch2=new
    Eintrag("Flohmarkt",345.11,true,LocalDate.of(2025,10,28),"Krimskrams")
    ;
    Eintrag eTesch3=new

```

```

Eintrag("Flohmarkt",12.50,false,LocalDate.of(2025,10,28),"Krimskrams")
;
//      eintraege.add(eTescht);
//      eintraege.add(eTescht2);
//      eintraege.add(eTescht3);

        FestWert fWTescht=new
FestWert("GEZ",58.95,false,LocalDate.of(2025, 9,
1),"Rundfunkgebühren",3);
        FestWert fWTescht2=new
FestWert("Pacht",1500,true,LocalDate.of(2025, 1,
1),"Mieteinnahmen",12);
//      festeWerte.add(fWTescht);
//      festeWerte.add(fWTescht2);

        // System.out.println(eintraege);
        while (true) {

System.out.println("=====");
        System.out.println("                HAUSHALTSKASSEN-APP");

System.out.println("=====");
        System.out.println("Was wünschen Sie?");

System.out.println("=====");
        System.out.printf(" 1: Eintrag hinzufügen\n 2: Eintrag
löschen\n 3: Eintrag ändern\n 4: Feste Werte\n 5: Übersicht der
Ausgaben\n 6: Übersicht der Einnahmen\n 7: Gegenüberstellung der
Einnahmen und Ausgaben\n 8: Gefilterte Einträge\n 9: Sortierte
Ausgabe\n10: Bevorstehende Ausgaben fester Werte in den von Ihnen
eingegebenen Tagen\n11: Balkendiagramm Ausgaben\n12: Budget-Manager\
n13: Statistik\n14: Sonstiges\n15: Beenden\n");
        auswahl=sc.nextInt();

System.out.println("=====");

        if (auswahl==1) {
            Eintrag e = new Eintrag(
                IO.readString("Bezeichnung: "),
                IO.readDouble("Betrag: "),
                IO.readBoolean("Einnahme?(j/n): "),
                IO.readDate("Datum:"),
                IO.readString("Kategorie: ")
            );
            Eintrag.fuegeEintragHinzu(eintraege, e);
            // System.out.println(eintraege);
        }
        else if(auswahl==2) {
            int index = IO.readInt("Geben Sie den Index an: ");

```



```

        IO.readDouble("Betrag: "),
        IO.readBoolean("Einnahme?(j/n):

"),

        IO.readDate("Datum: "),
        IO.readString("Kategorie: "),
        IO.readInt("Periode (Monate): ")
    ));
    } else {
        System.out.println(Fehlertext);
    }
    break;
case 4:
    istAuswahl=false;
    break;
default:
    System.out.println(Fehlertext);
}

    }
}
else if (auswahl == 5) {
    Uebersicht.zeigeAusgaben(eintraege);
    Uebersicht.zeigefesteAusgaben(festeWerte);
} else if (auswahl == 6) {
    Uebersicht.zeigeEinnahmen(eintraege);
    Uebersicht.zeigefesteEinnahmen(festeWerte);
} else if (auswahl == 7) {
    Uebersicht.zeigeBilanz(eintraege, festeWerte);
}
else if (auswahl == 8) {

System.out.println("=====");
        System.out.println("                Gefilterete Ausgabe");

System.out.println("=====");
        String filter=IO.readString("Geben Sie die Kategotie
an: ");
        Uebersicht.zeigeGefiltert(eintraege,festeWerte,
filter);
    }else if (auswahl == 9) {

System.out.println("=====");
        System.out.println("                Sortierte Ausgabe");

System.out.println("=====");
        int sortAuswahl=IO.readInt("Sortieren nach Betrag:
(1)\nSortieren nach Datum: (2)\n");
        switch (sortAuswahl) {

```

```

        case 1: {
            boolean isAbsteigend=IO.readBoolean("Nach
Betrag absteigend? (j/n): ");
            Uebersicht.zeigeSortiertNachBetrag(eintraege,
festeWerte,isAbsteigend);
            break;
        }
        case 2: {
            boolean isAbsteigend=IO.readBoolean("Nach Datum
absteigend? (j/n): ");
            Uebersicht.zeigeSortiertNachDatum(eintraege,
festeWerte, isAbsteigend);
            break;
        }
        default:
            System.out.println(Fehlertext);
    }
    }else if (auswahl==10) {
        Uebersicht.zeigeBevorstehendeAusgaben(festeWerte);
    }
    else if (auswahl == 11) {
        Uebersicht.zeigeBalkendiagrammAusgaben(eintraege,
festeWerte);
        Uebersicht.zeigeMonatsBalken(eintraege, festeWerte);
    }

    else if (auswahl==12) {
        budgetMenue();
    }
    else if (auswahl==13) {
        boolean istStatistik=true;
        System.out.printf("1: Statistik mit allen Kategorien\
n2: Statistik mit einer von Ihnen gewählten Auswahl an Kategorien\n3:
Statistik mit allen Kategorien im letzten Monat\n4: Statistik mit
einer von Ihnen gewählten Auswahl an Kategorien im letzten Monat\n5:
Statistik mit allen Kategorien im vorletzten Monat\n6: Statistik mit
einer von Ihnen gewählten Auswahl an Kategorien im vorletzten Monat\
n7: Vergleich der Mittelwerte der letzten 2 Monate\n8: zurück\n");
        while (istStatistik) {
            int statauswahl=IO.readInt("Wahl: ");
            switch(statauswahl) {
                case 1:{
                    Statistik.printMittelwert(eintraege,0);
                    Statistik.berechneStandardabweichung(eintraege,0);
                    break;
                }
                case 2:{
                    Statistik.berechneMitKategorie(eintraege,0);

```

```

        break;
    }
    case 3:{
        Statistik.printMittelwert(eintraege,1);
Statistik.berechneStandardabweichung(eintraege,1);
        break;
    }
    case 4:{
Statistik.berechneMitKategorie(eintraege,1);
        break;
    }
    case 5:{
        Statistik.printMittelwert(eintraege,2);
Statistik.berechneStandardabweichung(eintraege,2);
        break;
    }
    case 6:{
Statistik.berechneMitKategorie(eintraege,2);
        break;
    }
    case 7:{
        berechneVergleich(eintraege);
        break;
    }
    case 8:{
        istStatistik=false;
        break;
    }
    default:
        System.out.println(Fehlertext);
    }
}
else if (auswahl==14) {
    Sonstiges.zeigeMenue(eintraege, festeWerte);
}

else if (auswahl==15) {

System.out.println("=====");
        System.out.println("Vielen Dank! Bis zum nächsten
Mal!");

System.out.println("=====");
//        SparzielManager.speichereFesteWerte(festeWerte);
        Schlussspruch.schreibeSchluss();

```

```

        break;
    }

    else {
//
System.out.println("=====");
//          System.out.println("          Fehlerhafte
Eingabe!!!!");
//
System.out.println("=====");
        System.out.printf(Fehlertext);
    }
}
try (FileWriter writer = new FileWriter("eintraege.txt")) {
    for (Eintrag e : eintraege) {
        writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s%n",
                                e.bezeichnung,
                                e.betrag,
                                e.istEinnahme,
                                e.datum,
                                e.kategorie
                                ));
    }
    System.out.println("Einträge in eintraege.txt
gespeichert.");
} catch (IOException e) {
    System.err.println("Fehler beim Speichern: " +
e.getMessage());
}

    try (FileWriter writer = new FileWriter("festeWerte.txt")) {
        for (FestWert f : festeWerte) {
            writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s#%d%n",
                                f.bezeichnung,
                                f.betrag,
                                f.istEinnahme,
                                f.datum,
                                f.kategorie,
                                f.periode
                                ));
        }
        System.out.println("Feste Werte in festeWerte.txt
gespeichert.");
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern: " +
e.getMessage());
    }
}

```

```

        sc.close();

    }

    private static void berechneVergleich(ArrayList<Eintrag>
    eintraege) {
        double mit1=Math.round(Statistik.berechneMittelwert(eintraege,
    1)*100)/100.0;
        double mit2=Math.round(Statistik.berechneMittelwert(eintraege,
    2)*100)/100.0;
        String vergleich= "Die Ausgaben blieben zufälligerweise
    gleich!";
        double diff=0.0;
        if (mit1 != mit2) {
            diff=mit1>mit2 ? mit1-mit2 : mit2-mit1;
            vergleich="Die Ausgaben wurden also %.2f€"+(mit1>mit2 ? "
    mehr!": " weniger!");
        }

    System.out.println("=====");
        System.out.println("Vergleich der letzten Mittelwerte:");
        System.out.printf ("vorletzter Monat: %28.2f€\n",mit2);
        System.out.printf ("letzter Monat: %31.2f€\n",mit1);
        System.out.printf(vergleich+"\n",diff);

    System.out.println("=====");
    }

    private static void budgetMenue() {
        Scanner sc = new Scanner(System.in);
        while (true) {

    System.out.println("=====");
        System.out.println("
            BUDGET-VERWALTUNG");

    System.out.println("=====");
        System.out.println("1: Budget eingeben/ändern");
        System.out.println("2: Budget löschen");
        System.out.println("3: Alle Budgets anzeigen");
        System.out.println("4: Zurück zum Hauptmenü");
        System.out.print("Wahl: ");
        int wahl = sc.nextInt();
        sc.nextLine(); // Zeilenumbruch

        if (wahl == 1) {
            String kat = IO.readString("Kategorie: ");
            double betrag = IO.readDouble("Budget (0 = löschen):

```

```

");
        BudgetManager.setzeBudget(kat, betrag);
    }
    else if (wahl == 2) {
        String kat = IO.readString("Kategorie zum Löschen: ");
        BudgetManager.loescheBudget(kat);
    }
    else if (wahl == 3) {
        BudgetManager.zeigeAlleBudgets();
    }
    else if (wahl == 4) {
        break;
    }
    else {
        System.out.println("Ungültige Eingabe!");
    }
}

}

```

Package: Immutable

Klasse: Bestellung

```

package Immutable;

import java.util.ArrayList;
import java.util.List;

public final class Bestellung {
    private final String kunde;
    private final List<String> artikel;

    public Bestellung(String kunde, List<String> artikel) {
        this.kunde = kunde;
        this.artikel = new ArrayList<String>(artikel);
    }

    public List<String> getArtikel() {
        return new ArrayList<String>(artikel); //Defensive Copy
    }

    public String getKunde() {
        return kunde;
    }
}

```

```

        public Bestellung withKunde(String kunde) {
            return new Bestellung(kunde, this.artikel);
        }

        public Bestellung withArtikel(List<String> artikel) {
            return new Bestellung(this.kunde, artikel);
        }
    }

}package model;

import java.time.LocalDate;

public class Bestellung {

    private int id;
    private LocalDate datum;
    private double umsatz;
    private Kunde kunde;
    public Bestellung(int id, Kunde kunde, double umsatz) {
        this.id = id;
        this.umsatz = umsatz;
        this.kunde = kunde;
    }
    public int getId() {
        return id;
    }

    public LocalDate getDatum() {
        return datum;
    }
    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }
    public double getUmsatz() {
        return umsatz;
    }
    public void setUmsatz(double umsatz) {
        this.umsatz = umsatz;
    }
    public Kunde getKunde() {
        return kunde;
    }
    public void setKunde(Kunde kunde) {
        this.kunde = kunde;
    }
    @Override
    public String toString() {
        return "Bestellung [id=" + id + ", datum=" + datum + ",
umsatz=" + umsatz + " ]";
    }
}

```



```
}
```

Package: InOut

Klasse: IO

```
package InOut;
```

```
import java.time.LocalDate;  
import java.util.Scanner;
```

```
public class IO {
```

```
    public static int readInt(String prompt) {  
        System.out.print(prompt);  
        try (Scanner scan = new Scanner(System.in)) {  
            return scan.nextInt();  
        }  
    }  
    public static double readDouble(String prompt) {  
        System.out.print(prompt);  
        try (Scanner scan = new Scanner(System.in)) {  
            return scan.nextDouble();  
        }  
    }  
    public static String readString(String prompt) {  
        System.out.print(prompt);  
        try (Scanner scan = new Scanner(System.in)) {  
            return scan.nextLine();  
        }  
    }  
    public static boolean readBoolean(String prompt) {  
        try (Scanner scan = new Scanner(System.in)) {  
            while (true) {  
                System.out.print(prompt);  
                String input = scan.nextLine().trim().toLowerCase();  
                if (input.equals("j") || input.equals("ja") ||  
input.equals("y") || input.equals("yes")) {  
                    return true;  
                } else if (input.equals("n") || input.equals("nein")  
|| input.equals("no")) {  
                    return false;  
                } else {  
                    System.out.println("Ungültige Eingabe! Bitte 'j',  
'ja', 'n' oder 'nein' eingeben.");  
                }  
            }  
        }  
    }  
}
```

```

        }
    }
}
public static LocalDate readDate(String prompt) {
    System.out.println(prompt);
    try {
        return LocalDate.of(
            IO.readInt("Jahr: "),
            IO.readInt("Monat: "),
            IO.readInt("Tag: ")
        );
    } catch (Exception e) {
        System.out.println("Ungültiges Datum! Heutiges Datum wird verwendet!");
        return LocalDate.now();
    }
}
}

```

Package: Logistikuebung

Klasse: DroneDelivery

```
package Logistikuebung;
```

```

public class DroneDelivery extends Shipment{

    public DroneDelivery(double weightKg, double distanceKm) {
        super(weightKg, distanceKm);
    }

    @Override
    public double shippingCost() {

        return 444;
    }

}

```

Klasse: ExpressParcel

```
package Logistikuebung;
```

```
import InOut.IO;
```

```

public class ExpressParcel extends Shipment implements Trackable,
Insurable {

```

```

        private final double warenwert;
        public static final double GRUNDPREIS=9.99;

        public ExpressParcel(double weightKg, double distanceKm, double
warenwert) {
            super(weightKg, distanceKm);
            while (warenwert<0) {
                warenwert=IO.readDouble("Geben Sie für den Warenwert eine
nichtnegative Zahl ein: ");
            }
            this.warenwert = warenwert;
        }

        @Override
        public String toString() {
            return String.format("ExpressParcel:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
                + "Trackingcode: %26s\nShipping-Kosten: %22.2f€\n"
+super.toString(),warenwert,calculateInsurance(),getTrackingCode(),shi
ppingCost());
        }

        @Override
        public double calculateInsurance() {
            return Math.max(warenwert*0.01,2.5);
        }

        @Override
        public String getTrackingCode() {
            return hashCode()+ "";
        }

        @Override
        public double shippingCost() {
            return GRUNDPREIS+1.1*getWeightKg()+0.02*getDistanceKm();
        }
    }

```

Klasse: Freight

```
package Logistikuebung;
```

```
import InOut.IO;
```

```

public class Freight extends Shipment implements Insurable {
    private double warenwert;
    public static final double GRUNDPREIS=49;
    public Freight(double weightKg, double dinstanceKm, double
warenwert) {
        super(weightKg, dinstanceKm);
    }
}

```

```

        while (warenwert<0) {
            warenwert=IO.readDouble("Geben Sie für den Warenwert eine
nichtnegative Zahl ein: ");
        }
        this.warenwert = warenwert;
    }

```

```

    @Override
    public String toString() {
        return String.format("ExpressParcel:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
            + "Spipping-Kosten: %22.2f€\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost());
    }

```

```

    @Override
    public double calculateInsurance() {
        return Math.max(warenwert*0.005, 10);
    }

```

```

    @Override
    public double shippingCost() {
        return GRUNDPREIS+0.35*getWeightKg()+0.015*getDistanceKm();
    }

```

```

}

```

Klasse: Insurable

```

package Logistikuebung;

```

```

public interface Insurable {

    double calculateInsurance();

}

```

Klasse: Polymorphie

```

package Logistikuebung;

```

```

/**
 *
 * Shipment s = new StandardParcel(100, 850);
 * Deklarationstyp: Shipment
 *
 * Laufzeittyp: StandardParcel

```

```

*/

public class Polymorphie {

    public static void main(String[] args) {
        Shipment s = new StandardParcel(100, 850);

        s = new ExpressParcel(150, 950, 1250);
//        String tc=((StandardParcel) s).getTrackingCode();

        if (s instanceof Trackable track) {
            String tc= track.getTrackingCode();
            System.out.println("tc = "+tc);
        }
//        String tc = null;
//        if(s instanceof Trackable) {
//            tc=((StandardParcel) s).getTrackingCode();
//        }
//        System.out.println(tc);
        if (s instanceof Trackable) {
            System.out.println("Trackingcode: " + ((Trackable)
s).getTrackingCode());
        }
    }
}

```

Klasse: Sendungen

```

package Logistikuebung;

public class Sendungen {

    public static void main(String[] args) {
        Shipment[] shipments = { new StandardParcel(15, 250),
                                new ExpressParcel(75, 500, 750),
                                new Freight(4500, 2300, 8000),
                                new Freight(400, 50, 180),
                                new ExpressParcel(-1400, -150, -180)
                                };

        System.out.println("=".repeat(40));
        double gesamtkosten = 0.0;
        for (Shipment s : shipments) {
            System.out.println(s);

            if (s instanceof Insurable insurable) {
                gesamtkosten += insurable.calculateInsurance();
                System.out.println(String.format("Versicherung:
%25.2f" , insurable.calculateInsurance()) + "€");
            } else {

```

```

        System.out.println("keine Versicherung verfügbar");
    }

    if (s instanceof Trackable track) {
        System.out.println(String.format("Tackingcode:
%27s",track.getTrackingCode()));
    }else {
        System.out.println("kein Trackingcode verfügbar");
    }

    System.out.println(String.format("Versandkosten: %24.2f" ,
s.shippingCost()) + "€");
    gesamtkosten += s.shippingCost();
    System.out.println("=".repeat(40));
}

    System.out.println(String.format("Gesamtkosten: %25.2f" ,
gesamtkosten) + "€");
}
/**
 * einfacher ist es, die Versicherungskosten jeweils direkt zu den
shippingkosten dazuzurechnen,
 * da man dann einfach über die shippingkosten in shipments summiert
 */
}

```

Klasse: Shipment

```

package Logistikuebung;

import InOut.IO;

public abstract class Shipment {
    private final double weightKg;
    private final double distanceKm;

    public double getWeightKg() {
        return weightKg;
    }

    public double getDistanceKm() {
        return distanceKm;
    }

    public Shipment(double weightKg, double distanceKm) {
        while (weightKg<=0) {
            weightKg=IO.readDouble("Geben Sie für das Gewicht eine

```

```

        positive Zahl ein: ");
    }
    this.weightKg=weightKg;

    while(distanceKm<=0) {
        distanceKm=IO.readDouble("Geben Sie für die Strecke
eine positive Zahl ein: ");
    }
    this.distanceKm = distanceKm;

}

@Override
public String toString() {
    return String.format("Gewicht: %29.1fkg\nEntfernung: %26.1fkm\n",weightKg,distanceKm);
}

public abstract double shippingCost();

}

```

Klasse: StandardParcel

```

package Logistikuebung;

public class StandardParcel extends Shipment implements Trackable{
    public static final double GRUNDPREIS=4.99;
    public StandardParcel(double weightKg, double distanceKm) {
        super(weightKg, distanceKm);
    }

    @Override
    public String toString() {
        return String.format("StandardParcel:\nShipping-Kosten:
%22.2f€\nTrackingcode:%27s\n" +super.toString(), shippingCost(),
getTrackingCode());
    }

    @Override
    public double shippingCost() {
        return GRUNDPREIS+0.6*getWeightKg()
+Math.min(0.01*getDistanceKm(), 10);
    }
}

```

```

        @Override
        public String getTrackingCode() {
            return hashCode()+ "";
        }
    }
}

```

Klasse: Trackable

```

package Logistikuebung;

public interface Trackable {

    String getTrackingCode();

}

```

Package: Logistikuebung_2

Klasse: ExpressParcel

```

package Logistikuebung_2;

public class ExpressParcel extends Shipment implements Trackable,
    Insurable {

    private final double warenwert;
    public static final double GRUNDPREIS=9.99;

    public ExpressParcel(double weightKg, double distanceKm, double
    warenwert) throws InvalidShipmentDataException {
        super(weightKg, distanceKm);
        if (warenwert>=0) {
            this.warenwert = warenwert;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
    sein");
        }
    }

    @Override
    public String toString() {
        return String.format("ExpressParcel:\nWarenwert: %28.2f€\n
    nVersicherungskosten: %18.2f€\n"
            + "Shipping-Kosten: %22.2f€\nTrackingcode: %26s\n"
    +super.toString(),warenwert,calculateInsurance(),shippingCost(),getTra
    ckingCode());
    }

    @Override

```



```

    public double calculateInsurance() {
        return Math.max(warenwert*0.01,2.5);
    }

    @Override
    public String getTrackingCode() {
        return hashCode()+ "";
    }

    @Override
    public double shippingCost() {
        return GRUNDPREIS+1.1*getWeightKg()+0.02*getDistanceKm();
    }
}

```

Klasse: Freight

```
package Logistikuebung_2;
```

```
import InOut.IO;
```

```

public class Freight extends Shipment implements Insurable {
    private double warenwert;
    public static final double GRUNDPREIS=49;
    public Freight(double weightKg, double dinstanceKm, double
warenwert) throws InvalidShipmentDataException {
        super(weightKg, dinstanceKm);
        if (warenwert>=0) {
            this.warenwert = warenwert;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
sein");
        }
    }

    @Override
    public String toString() {
        return String.format("Freight:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
            + "Shipping-Kosten: %22.2f€\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost());
    }

    @Override
    public double calculateInsurance() {
        return Math.max(warenwert*0.005, 10);
    }
}

```

```

        @Override
        public double shippingCost() {
            return GRUNDPREIS+0.35*getWeightKg()+0.015*getDistanceKm();
        }
    }
}

```

Klasse: Insurable

```

package Logistikuebung_2;

public interface Insurable {

    double calculateInsurance();

}

```

Klasse: Polymorphie

```

package Logistikuebung_2;

/**
 *
 * Shipment s = new StandardParcel(100, 850);
 * Deklarationstyp: Shipment
 *
 * Laufzeittyp: StandardParcel
 */

public class Polymorphie {

    public static void main(String[] args) throws
InvalidShipmentDataException {
        Shipment s = new StandardParcel(100, 850);

        s= new ExpressParcel(150, 950, 1250);
//        String tc=((StandardParcel) s).getTrackingCode();

        if (s instanceof Trackable track) {
            String tc= track.getTrackingCode();
            System.out.println("tc = "+tc);
        }
//        String tc = null;
//        if(s instanceof Trackable) {
//            tc=((StandardParcel) s).getTrackingCode();
//        }
//        System.out.println(tc);
    }
}

```

```

        if (s instanceof Trackable) {
            System.out.println("Trackingcode: " + ((Trackable)
s).getTrackingCode());
        }
    }
}

```

Klasse: Sendungen

```
package Logistikuebung_2;
```

```
public class Sendungen {
```

```

    public static void main(String[] args)throws
UnknownShipmentTypeException, InvalidShipmentDataException {
        Shipment[] shipments = { new StandardParcel(15, 250),
                                new ExpressParcel(75, 500, 750),
                                new Freight(4500, 2300, 8000),
                                new Freight(400, 50, 180),
                                new ExpressParcel(1400, 150, 180)
                                };

        System.out.println("=".repeat(40));
        double gesamtkosten = 0.0;
        for (Shipment s : shipments) {
            System.out.println(s);

            if (s instanceof Insurable insurable) {
                gesamtkosten += insurable.calculateInsurance();
                System.out.println(String.format("Versicherung:
%25.2f" , insurable.calculateInsurance()) + "€");
            } else {
                System.out.println("keine Versicherung verfügbar");
            }

            if (s instanceof Trackable track) {
                System.out.println(String.format("Tackingcode:
%27s",track.getTrackingCode()));
            }else {
                System.out.println("kein Trackingcode verfügbar");
            }

            System.out.println(String.format("Versandkosten: %24.2f" ,
s.shippingCost()) + "€");
            gesamtkosten += s.shippingCost();
            System.out.println("=".repeat(40));
        }

        System.out.println(String.format("Gesamtkosten: %25.2f" ,
gesamtkosten) + "€");
    }
}

```

```
ShipmentCsvReader2.readShipmentsFromCSV("warensendungen.csv");
```

```
    }  
    /**  
    * einfacher ist es, die Versicherungskosten jeweils direkt zu den  
    shippingkosten dazuzurechnen,  
    * da man dann einfach über die shippingkosten in shipments summiert  
    */  
}
```

Klasse: Shipment

```
package Logistikuebung_2;
```

```
//import InOut.IO;
```

```
public abstract class Shipment {  
    private final double weightKg;  
    private final double distanceKm;  
  
    public double getWeightKg() {  
        return weightKg;  
    }  
  
    public double getDistanceKm() {  
        return distanceKm;  
    }  
}
```

```
    public Shipment(double weightKg, double distanceKm) throws  
InvalidShipmentDataException{  
        if (weightKg >0) {  
            this.weightKg=weightKg;  
        }  
        else {  
            throw new InvalidShipmentDataException("Wert muss positiv  
sein");  
        }  
  
        if(distanceKm>0) {  
            this.distanceKm = distanceKm;  
        }  
        else {  
            throw new InvalidShipmentDataException("Wert muss  
positiv sein");  
        }  
}
```

```

    }

    @Override
    public String toString() {
        return String.format("Gewicht: %29.1fkg\nEntfernung: %26.1fkm\n", weightKg, distanceKm);
    }

    public abstract double shippingCost();

}

```

Klasse: ShipmentCsvReader

```

package Logistikuebung_2;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;

public class ShipmentCsvReader {

    File f = new File("wareneingangsliste.txt");

    public static ArrayList<Shipment> readShipmentsFromTXT(String
pfad) throws UnknownShipmentTypeException,
InvalidShipmentDataException{
        ArrayList<Shipment> result = new ArrayList<Shipment>();

        try {
            BufferedReader br = new BufferedReader(new
FileReader(pfad));

            String line;
            int counter=0;
            String shipmenttyp = null;
            double weight = 0;
            double distance = 0;
            Double warenwert = null;

            while ((line =br.readLine())!= null ) {
                if(line.startsWith("Gesamtkosten"))
                    break;

```

```

        if(line.startsWith("=") && shipmenttyp != null) {
            switch(shipmenttyp) {
                case "StandardParcel":{
                    result.add(new StandardParcel(weight,
distance));
                }
                break;
                case "ExpressParcel":{
                    result.add(new ExpressParcel(weight, distance,
warenwert));
                }
                break;
                case "Freight":{
                    result.add(new Freight(weight, distance,
warenwert));
                }
                break;
                default:
                    throw new
UnknownShipmentTypeException("Shipment unbekannt");
            }
            weight = 0;
            distance = 0;
            warenwert = null;
        }
        if(line.isEmpty() || line.isBlank() ||
line.startsWith("="))
            continue;

        if(line.endsWith(":")) {
            shipmenttyp = line.replace(":", "").trim();
        }
        String[] values = line.replace('€', '
').replace("kg", " ").replace("km", " ").replace("keine Versicherung
verfügbar", " ").replace("kein Trackingcode verfügbar", "
").trim().replace(',', '.').split(":");
        //         replace('=' , ' ').
        //         for (String value :values) {

//
        (values[1].contains("="))
            counter++;

        if(values.length < 2)
            continue;

```

```

        if (values[0].equals("Gewicht"))
            weight =
Double.parseDouble(values[1].trim());
        if (values[0].equals("Warenwert"))
            warenwert =
Double.parseDouble(values[1].trim());
        if (values[0].equals("Entfernung"))
            distance =
Double.parseDouble(values[1].trim());

        System.out.println(shipmenttyp);
        System.out.println(weight);
        System.out.println(distance);
        if(warenwert!=null)
            System.out.println(warenwert);

    }
//        System.out.println(value);
//        System.out.println("Counter: "+counter);

```

```

//        for (int i =0;i< values.length-1;i++) {
//            double weight;
//            double distance;
//            double warenwert;
//            values[i].trim();
//            if (values[i].contains("="))
//                counter++;
//            if (values[i].equals("Gewicht"))
//                weight = Double.parseDouble(values[i+1]);
//            if (values[i].equals("Warenwert"))
//                warenwert =
Double.parseDouble(values[i+1]);
//            if (values[i].equals("Entfernung"))
//                distance = Double.parseDouble(values[i+1]);

//            System.out.println(values[i]);
//            System.out.println(counter);
//        }
//        System.out.println();
//        result.add(new Shipment(x,y));

```

```

        br.close();

        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println(e.getMessage());
        }
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        }
        for (Shipment s:result)
            System.out.println(s);

        return result;
    }
}

```

Klasse: ShipmentCsvReader2

```
package Logistikuebung_2;
```

```

import java.io.BufferedReader;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;

```

```
public class ShipmentCsvReader2 {
```

```

    public static ArrayList<Shipment> readShipmentsFromCSV(String
pfad) throws UnknownShipmentTypeException,
InvalidShipmentDataException{
        File f = new File(pfad);

```

```
        ArrayList<Shipment> result = new ArrayList<Shipment>();
```

```

        try {
            BufferedReader br = new BufferedReader(new
FileReader(pfad));

```

```

            String line;
            String shipmenttyp = null;
            double weight = 0;
            double distance = 0;

```



```

        Double warenwert = null;

        while ((line =br.readLine())!= null ) {
            if(line.startsWith("type"))
                continue;

            if( shipmenttyp != null) {
                switch(shipmenttyp.toLowerCase()) {
                    case "standard":{
                        result.add(new StandardParcel(weight,
distance));
                    }
                    break;
                    case "express":{
                        result.add(new ExpressParcel(weight, distance,
warenwert));
                    }
                    break;
                    case "freight":{
                        result.add(new Freight(weight, distance,
warenwert));
                    }
                    break;
                    default:
                        throw new
UnknownShipmentTypeException("Shipment unbekannt");
                }
                weight = 0;
                distance = 0;
                warenwert = null;
            }
            if(line.isEmpty()|| line.isBlank())
                continue;

            String[] values = line.trim().split(",");
            //      replace('=',' ' )..replace('€', ' ').replace("kg","
").replace("km"," ").replace("keine Versicherung verfügbar","
").replace("kein Trackingcode verfügbar", " ")
            //      for (String value :values) {

                if(values.length == 4) {

                    shipmenttyp=values[0].trim();
                    weight =
Double.parseDouble(values[1].trim());
                    distance =
Double.parseDouble(values[2].trim());

```

```

        warenwert =
Double.parseDouble(values[3].trim());
    }
    else if(values.length == 3) {
        shipmenttyp=values[0].trim();
        weight =
Double.parseDouble(values[1].trim());
        distance =
Double.parseDouble(values[2].trim());
    }
    else {
        throw new
InvalidShipmentDataException("Shipment unbekannt");
    }

    System.out.println(shipmenttyp);
    System.out.println(weight);
    System.out.println(distance);
    if(warenwert!=null)
        System.out.println(warenwert);
}

br.close();

} catch (FileNotFoundException e) {
    System.out.println(e.getMessage());
// } catch (NumberFormatException e) {
//     System.out.println(e.getMessage());
// }
} catch (IOException e) {
    System.out.println(e.getMessage());
}
for (Shipment s:result)
    System.out.println(s);

return result;
}
}

```

Klasse: ShipmentReportWriter

```
package Logistikuebung_2;
```

```
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.time.LocalDateTime;
```

```

import java.time.format.DateTimeFormatter;
import java.util.List;

public class ShipmentReportWriter {

    public void writeReport(List<Shipment> shipments, String fileName)
    {

        LocalDateTime timestamp = LocalDateTime.now();
        File f = new File(fileName);
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-
MM-yyyy HH:mm:ss");
        String timestampToString=timestamp.format(formatter);
        try {
            boolean success= f.createNewFile();
            if (success)
                System.out.println("Datei wurde erzeugt");
            else
                System.out.println("Datei existiert bereits");
            BufferedWriter br= new BufferedWriter(new
FileWriter(f,true));
            br.write("=".repeat(50));
            br.newLine();
            br.write(timestampToString);
            br.newLine();
            br.write("");

        }catch (Exception e) {
            // TODO: handle exception
        }

    }

}

```

Klasse: StandardParcel

```

package Logistikuebung_2;

public class StandardParcel extends Shipment implements Trackable{
    public static final double GRUNDPREIS=4.99;
    public StandardParcel(double weightKg, double distanceKm) throws
InvalidShipmentDataException {
        super(weightKg, distanceKm);
    }
}

```

```

    @Override
    public String toString() {
        return String.format("StandardParcel:\nShipping-Kosten:
%22.2f€\nTrackingcode:%27s\n" +super.toString(), shippingCost(),
getTrackingCode());
    }

```

```

    @Override
    public double shippingCost() {
        return GRUNDPREIS+0.6*getWeightKg()
+Math.min(0.01*getDistanceKm(), 10);
    }

```

```

    @Override
    public String getTrackingCode() {
        return hashCode()+ "";
    }

```

```

}

```

Klasse: Trackable

```

package Logistikuebung_2;

```

```

public interface Trackable {

    String getTrackingCode();

}

```

Klasse: UnknownShipmentTypeException

```

package Logistikuebung_2;

```

```

public class UnknownShipmentTypeException extends Exception {

    /**
     *
     */
    private static final long serialVersionUID = 1L;

    public UnknownShipmentTypeException(String message) {
        super(message);
    }

}

```

Package: Mittwoch

Klasse: BeispielExceptionHandling

```
package Mittwoch;

import java.util.*;

public class BeispielExceptionHandling {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);
        try {
            System.out.print("Geben Sie bitte eine ganze Zahl ein: ");

            int zahl =input.nextInt();

            System.out.println("Sie haben eine "+zahl+ " eingegeben");
        }
        catch (Exception ex){
            System.out.println("Es ist ein Fehler aufgetreten "+ex);
        }
        finally {
            System.out.println("Das Programm wird beendet");
            input.close();
        }
    }
}
```

Klasse: Diagnostik

```
package Mittwoch;

public class Diagnostik {

    String [] komprimiereBilddaten(String[] unkomprimiert) {
        int i=0;
        String[] komprimiert =null;
        for ( int index=0;index< unkomprimiert.length;index++)

            return komprimiert;
    }
}
```

```

    }

}

/*int count=1;
if (unkomprimiert[i+1]==unkomprimiert[i]) {

    count++;

    if (count>4) {komprimiereCode()}

    else {count=1;}
}*/package ObserverPattern;

import java.util.Set;

public class DisplayDevice implements Device {
    private String name = "Smart-Display";
    private boolean on = true;

    @Override
    public void update(float temperature) {
        System.out.println("\n=== " + name + " ===");
        System.out.println("Aktuelle Temperatur: " + temperature +
"°C");
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public boolean isOn() {
        return on;
    }

    @Override
    public void resetToDefaults() {
        System.out.println(name + " zurückgesetzt");
    }

    public void showDeviceStatus(Set<Device> devices) {
        System.out.println("\n--- Geräte Status ---");
        devices.forEach(d -> System.out.println(d.getName() + ": " +
(d.isOn() ? "AN" : "AUS")));
    }
}

```

```
    }  
}
```

Klasse: Start

```
package Mittwoch;
```

```
public class Start {
```

```
    public static void main(String[] args) {  
        String[] unkomprimiert =new String[22];
```

```
        unkomprimiert[0]=unkomprimiert[1]=unkomprimiert[2]=unkomprimiert[3]="Z  
        ";
```

```
        for (int i=4;i<14;i++) {  
            unkomprimiert[i]="7";  
        }
```

```
        unkomprimiert[14]="M";  
        for (int j=15;j<20;j++) {  
            unkomprimiert[j]="P";  
        }
```

```
        unkomprimiert[20]=unkomprimiert[21]="H";
```

```
        for (int m=0; m< unkomprimiert.length;m++) {  
            System.out.print(unkomprimiert[m]);  
        }
```

```
        Diagnostik dg1 = new Diagnostik();
```

```
    }
```

```
}
```

Package: Mittwoch

Klasse: Billing

```
package Mittwoch;
```

```
import java.util.Scanner;
```

```
public class Billing {
```

```
    void showTime(int sec) {  
        int day=0;  
        int hour=0;  
        int temp=0;  
        int minute=0;  
        int second =0;  
        int temp2=0;  
        int temp3=0;  
        int week=0;
```

```

        if (sec>=604800) {
            week=sec/604800;
        }

        temp3=sec%604800;
        if(temp3>=86400) {
            day=temp3/86400;
        }
        temp2=sec%86400;
        hour= temp2 / 3600;
        temp= temp2%3600;
        minute= temp/60;
        second=sec%60;

        System.out.println(sec+" Sekunden sind :");
        if(week>0) {
            System.out.println(week+" Wochen");
        }

        if(day>0) {
            System.out.println(day+" Tage ");
        }
        System.out.println(hour+" Stunden \n"+minute+" Minuten \n"+
second +" Sekunden");
    }
    int liesZeitein() {
        Scanner input =new Scanner(System.in);
        System.out.print("Geben Sie die Zeit in Sekunden an: ");
        int zeit=input.nextInt();
        input.close();
        return zeit;
    }

}

```

Klasse: Start

```
package Mitwoch;
```

```
public class Start {
```

```

    public static void main(String[] args) {
        Billing b1=new Billing();
        b1.showTime(b1.liesZeitein());
    }
}

```



```
    }  
}
```

Package: Montag11082025

Klasse: Artikel

```
package Montag11082025;
```

```
public class Artikel {  
  
    private String bezeichnung;  
    private double preis;  
  
    //statische Variable  
    private static double mwstsatz = 0.19;  
  
    public Artikel(String bezeichnung,double preis) {  
        this.bezeichnung=bezeichnung;  
        this.preis=preis;  
    }  
  
    public String getBezeichnung() {  
        return bezeichnung;  
    }  
  
    public void setBezeichnung(String bezeichnung) {  
        this.bezeichnung = bezeichnung;  
    }  
  
    public double getPreis() {  
        return preis;  
    }  
  
    public void setPreis(double preis) {  
        this.preis = preis;  
    }  
  
    public static double getMwstsatz() {  
        return mwstsatz;  
    }  
  
    public static void setMwstsatz(double mwstsatz) {  
        Artikel.mwstsatz = mwstsatz;  
    }  
}
```

```
}
```

Klasse: Flugzeug

```
package Montag11082025;
```

```
public class Flugzeug extends Luftfahrzeug {

    private double spannweite;
    private int motorenAnzahl;

    public Flugzeug(String bezeichnung,double gewicht,
                    int baujahr,double spannweite, int motorenAnzahl)
    {
        this.bezeichnung=bezeichnung;
        this.gewicht=gewicht;
        this.baujahr=baujahr;
        this.spannweite=spannweite;
        this.motorenAnzahl=motorenAnzahl;
    }

    public double getSpannweite() {
        return spannweite;
    }

    public void setSpannweite(double spannweite) {
        this.spannweite = spannweite;
    }

    public int getMotorenAnzahl() {
        return motorenAnzahl;
    }

    public void setMotorenAnzahl(int motorenAnzahl) {
        this.motorenAnzahl = motorenAnzahl;
    }

    public String getDaten() {
        String daten ="Bezeichnung: "+ bezeichnung +"\nGewicht:
"+gewicht+"\nBaujahr: " +baujahr+
        "\nSpannweite: " +spannweite+ "m\nAnzahl der Motoren:
"+motorenAnzahl;

        return daten;
    }

}
```

Klasse: Kasse

```
package Montag11082025;
```

```
public class Kasse {
```

```
    public static void main(String[] args) {  
        System.out.println("MWStSatz: "+Artikel.getMwstsatz());
```

```
        Artikel a1=new Artikel("Cola",0.99);  
        double brutto =a1.getPreis()*(1+Artikel.getMwstsatz());
```

```
        System.out.println(brutto+" €");
```

```
    }
```

```
}
```

Klasse: Luftfahrzeug

```
package Montag11082025;
```

```
public class Luftfahrzeug {
```

```
    //Attribute gelockerter Zugriffsschutz  
    protected String bezeichnung;  
    protected double gewicht;  
    protected int baujahr;
```

```
    public String getBezeichnung() {  
        return bezeichnung;
```

```
    }
```

```
    public void setBezeichnung(String bezeichnung) {  
        this.bezeichnung = bezeichnung;
```

```
    }
```

```
    public double getGewicht() {  
        return gewicht;
```

```
    }
```

```
    public void setGewicht(double gewicht) {  
        this.gewicht = gewicht;
```

```
    }
```

```
    public int getBaujahr() {  
        return baujahr;
```

```
    }
```

```
    public void setBaujahr(int baujahr) {  
        this.baujahr = baujahr;
```

```
    }
```

```
}
```

Package: NorthwindApp

Klasse: TestNWApp

```
package NorthwindApp;
import java.util.ArrayList;
import java.time.LocalDate;
import NorthwindApp.nw_db_tool.Employee;
import NorthwindApp.nw_db_tool.NorthwindDB;

public class TestNWApp {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        testNWDBTool();
    }

    public static void testNWDBTool() {
        ArrayList<Employee> emplist =NorthwindDB.getEmployees();
        for(Employee employee : emplist){
            /*      System.out.print(employee.toString());
                if (employee.getBirthdate() != null) {
                    System.out.println(" geboren am:
"+employee.getBirthdate());
                }
                else {
                    System.out.println(" ");
                } */
            if (employee.getBirthdate() != null) {
                System.out.printf("ID: %3s, Vorname: %-12s Nachname: %-12s
geboren am: %s\
n",employee.getEmpid(),employee.getFirstname(),employee.getLastname(),
employee.getBirthdate());
            }
            else {
                System.out.printf("ID: %3s, Vorname: %-12s Nachname:
%-12s\
n",employee.getEmpid(),employee.getFirstname(),employee.getLastname())
;
            }
        }
    }
}
```

Package: NorthwindApp.GUI

Klasse: EmployeeWindow

```
package NorthwindApp.GUI;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.time.LocalDate;
import java.util.ArrayList;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JMenu;
import javax.swing.JMenuBar;
import javax.swing.JMenuItem;
import javax.swing.JOptionPane;
import javax.swing.JTextField;
import NorthwindApp.nw_db_tool.Employee;
import NorthwindApp.nw_db_tool.NorthwindDB;

public class EmployeeWindow extends JFrame {
    private JComboBox<Employee> cboEmployees;
    private JTextField txtFirstName;
    private JTextField txtLastName;
    private JTextField txtBirthDate;
    private JButton btnUpdate;
    private int EmployeeID;

    public EmployeeWindow() {
        super();
        initWindow();
        initControls();
        this.setVisible(true);
    }

    public void initWindow() {
        setSize(360, 300);
        setLocation(400, 400);
        setTitle("Mitarbeiter");
        setResizable(false);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(null);
        setJMenuBar(createMenuBar());
    }

    private JMenuBar createMenuBar() {
        JMenuBar menuBar = new JMenuBar();
        JMenu fileMenu = new JMenu("File");
```

```

JMenuItem newEmployeeItem = new JMenuItem("New Employee");
newEmployeeItem.addActionListener(e -> prepareNewEmployee());
fileMenu.add(newEmployeeItem);

JMenuItem exitItem = new JMenuItem("Exit");
exitItem.addActionListener(e -> System.exit(0));
fileMenu.add(exitItem);

menuBar.add(fileMenu);
return menuBar;
}

private void prepareNewEmployee() {
    clearForm();
    cboEmployees.setSelectedIndex(-1);
    EmployeeID = 0;
}

public void initControls() {
    JLabel lblEmpList = new JLabel("Mitarbeiter");
    lblEmpList.setBounds(10, 10, 80, 25);
    add(lblEmpList);
    cboEmployees = new JComboBox<>();
    cboEmployees.setBounds(100, 10, 220, 25);
    cboEmployees.addActionListener(e -> fillForm());
    add(cboEmployees);

    JLabel lblFirstName = new JLabel("Vorname");
    lblFirstName.setBounds(10, 40, 80, 25);
    add(lblFirstName);
    txtFirstName = new JTextField();
    txtFirstName.setBounds(100, 40, 220, 25);
    add(txtFirstName);

    JLabel lblLastName = new JLabel("Nachname");
    lblLastName.setBounds(10, 70, 80, 25);
    add(lblLastName);
    txtLastName = new JTextField();
    txtLastName.setBounds(100, 70, 220, 25);
    add(txtLastName);

    JLabel lblBirthDate = new JLabel("Geburtsdatum");
    lblBirthDate.setBounds(10, 100, 80, 25);
    add(lblBirthDate);
    txtBirthDate = new JTextField();
    txtBirthDate.setBounds(100, 100, 220, 25);
    add(txtBirthDate);
}

```

```

        btnUpdate = new JButton("Save");
        btnUpdate.setBounds(120, 130, 100, 25);
        btnUpdate.addActionListener(e -> updateData());
        add(btnUpdate);

        JButton btnNew = new JButton("New");
        btnNew.setBounds(220, 130, 100, 25);
        btnNew.addActionListener(e -> prepareNewEmployee());
        add(btnNew);

        fillcboEmployees();
    }

    public void fillcboEmployees() {
        ActionListener[] listeners =
cboEmployees.getActionListeners();
        for (ActionListener listener : listeners) {
            cboEmployees.removeActionListener(listener);
        }

        cboEmployees.removeAllItems();
        try {
            ArrayList<Employee> emplist = NorthwindDB.getEmployees();
            if (emplist == null || emplist.isEmpty()) {
                JOptionPane.showMessageDialog(this, "No employees
found in the database.");
            } else {
                for (Employee employee : emplist) {
                    cboEmployees.addItem(employee);
                }
                cboEmployees.setSelectedIndex(0);
            }
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Error retrieving
employees: " + e.getMessage());
        }

        for (ActionListener listener : listeners) {
            cboEmployees.addActionListener(listener);
        }
    }

    public void fillForm() {
        Employee emp = (Employee) cboEmployees.getSelectedItem();
        if (emp != null) {
            txtFirstName.setText(emp.getFirstname());
            txtLastName.setText(emp.getLastname());
            txtBirthDate.setText(emp.getBirthdate() != null ?
emp.getBirthdate().toString() : "");
            EmployeeID = emp.getEmpid();
        }
    }

```

```

        } else {
            txtFirstName.setText("");
            txtLastName.setText("");
            txtBirthDate.setText("");
            EmployeeID = 0;
        }
    }

    public void updateData() {
        String firstname = txtFirstName.getText();
        String lastname = txtLastName.getText();
        String bdate = txtBirthDate.getText();
        if (firstname.isEmpty() || lastname.isEmpty() ||
bdate.isEmpty()) {
            JOptionPane.showMessageDialog(this, "Please fill in First
Name, Last Name, and Birth Date.");
            return;
        }
        try {
            LocalDate birthdate = LocalDate.parse(bdate);
            boolean success;
            if (EmployeeID == 0) {
                success = NorthwindDB.insertEmployee(firstname,
lastname, birthdate);
                if (!success) {
                    JOptionPane.showMessageDialog(this, "Insert
failed.");
                    return;
                }
            } else {
                success = NorthwindDB.updateEmployee(EmployeeID,
firstname, lastname, birthdate);
                if (!success) {
                    JOptionPane.showMessageDialog(this, "Update
failed.");
                    return;
                }
            }
            fillcboEmployees();
            clearForm();
            if (cboEmployees.getItemCount() > 0) {
                cboEmployees.setSelectedIndex(0);
                fillForm();
            }
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Invalid date
format.");
        }
    }
}

```



```

        public void clearForm() {
            txtFirstName.setText("");
            txtLastName.setText("");
            txtBirthDate.setText("");
            EmployeeID = 0;
        }

        public static void main(String[] args) {
            new EmployeeWindow();
        }
    }package _25_03_2026;

    public class Engine {
        public String type;

        public Engine() {
        }
    }package _03_03;

    public abstract class EtickettenFabrik {

        public TeeEtickett erstelleEtickett(String typ) {

            TeeEtickett t = fabrikMethode(typ);

            t.berechneVerfallsDatum();

            return t;
        }

        protected abstract TeeEtickett fabrikMethode(String typ);
    }

    class EtickettenFabrikKoeln extends EtickettenFabrik{

        @Override
        protected TeeEtickett fabrikMethode(String typ) {
            switch (typ) {
                case "Bronchial":
                    return new KoellnerBronchilaTeeEtickett();
                case "Magen":
                    return new KoellnerMagenTeeEtickett();

                //...

                default:
                    throw new IllegalArgumentException("Unebekannte Teesort: "
+ typ);
            }
        }
    }

```

```

    }

}

class EtickettenFabrikMuenchen extends EtickettenFabrik{

    @Override
    protected TeeEtickett fabrikMethode(String typ) {
        switch (typ) {
            case "Bronchial":

                return new MuenchnerBronchialTeeEtickett();
            case "Magen":
                return new MuenchnerMagenTeeEtickett();

            default:
                throw new IllegalArgumentException("Unbekannte Teesorte: "
+ typ);
        }
    }

}

}package NorthwindApp.GUI;

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JTextField;

public class ExampleGUI extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    private JTextField textField;
    private JTextField textField_1;
    private JTextField textField_2;

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    ExampleGUI frame = new ExampleGUI();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }
}

```

```

        });
    }

    /**
     * Create the frame.
     */
    public ExampleGUI() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 300);
        contentPane = new JPanel();
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);

        textField_2 = new JTextField();
        contentPane.add(textField_2);
        textField_2.setColumns(10);

        textField = new JTextField();
        contentPane.add(textField);
        textField.setColumns(10);

        textField_1 = new JTextField();
        contentPane.add(textField_1);
        textField_1.setColumns(10);

        JPanel panel = new JPanel();
        contentPane.add(panel);
    }
}

```

Package: NorthwindApp.nw_db_tool

Klasse: NorthwindDB

```

package NorthwindApp.nw_db_tool;

import java.sql.*;
import java.time.LocalDate;
import java.util.ArrayList;

public class NorthwindDB {
    private static String url =
"jdbc:mysql://localhost:3306/Northwind";
    private static String user = "root";
    private static String password = "";

    public static Connection getConnection() {

```

```

        Connection con = null;
        try {
            con = DriverManager.getConnection(url, user, password);
        } catch (SQLException e) {
            System.out.println("Connection failed: " +
e.getMessage());
            System.exit(-999);
        }
        return con;
    }
}

```

```

    public static ResultSet getResult(String sql) {
        Connection con = getConnection();
        Statement stmt = null;
        ResultSet result = null;
        try {
            stmt =
con.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,
ResultSet.CONCUR_READ_ONLY);
            result = stmt.executeQuery(sql);
            return result;
        } catch (SQLException e) {
            System.out.println("get rowset failed: " +
e.getMessage());
            try {
                if (result != null) result.close();
                if (stmt != null) stmt.close();
                if (con != null) con.close();
            } catch (SQLException ex) {
                ex.printStackTrace();
            }
            System.exit(-900);
        }
        return null;
    }
}

```

```

    public static ArrayList<Employee> getEmployees() {
        ArrayList<Employee> emps = new ArrayList<>();
        String sql = "Select EmployeeID, FirstName, LastName,
BirthDate from Employees";
        Connection con = null;
        Statement stmt = null;
        ResultSet result = null;
        try {
            con = getConnection();
            stmt = con.createStatement();
            result = stmt.executeQuery(sql);
            while (result.next()) {
                int id = result.getInt("EmployeeID");
                String fname = result.getString("FirstName");

```

```

        String lname = result.getString("LastName");
        Date bdate = result.getDate("BirthDate", null);
        LocalDate birthdate = bdate != null ?
bdate.toLocalDate() : null;
        Employee emp = new Employee(id, fname, lname,
birthdate);
        emps.add(emp);
    }
} catch (SQLException e) {
    e.printStackTrace();
} finally {
    try {
        if (result != null) result.close();
        if (stmt != null) stmt.close();
        if (con != null) con.close();
    } catch (SQLException e) {
        e.printStackTrace();
    }
}
return emps;
}

```

```

    public static boolean updateEmployee(int ID, String fname, String
lname, LocalDate bdate) {
        Connection con = getConnection();
        PreparedStatement stmt = null;
        String sql = "Update Employees set FirstName = ?, LastName
= ?, BirthDate = ? where EmployeeID = ?;";
        try {
            stmt = con.prepareStatement(sql);
            stmt.setString(1, fname);
            stmt.setString(2, lname);
            Date date = Date.valueOf(bdate);
            stmt.setDate(3, date);
            stmt.setInt(4, ID);
            int updated = stmt.executeUpdate();
            return updated > 0;
        } catch (SQLException e) {
            e.printStackTrace();
            return false;
        } finally {
            try {
                if (stmt != null) stmt.close();
                if (con != null) con.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

        public static boolean insertEmployee(String fname, String lname,
        LocalDate bdate) {
            Connection con = getConnection();
            PreparedStatement stmt = null;
            String sql = "INSERT INTO Employees (FirstName, LastName,
            BirthDate, Notes, Title, HireDate, Address, City, PostalCode, Country,
            HomePhone) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
            try {
                stmt = con.prepareStatement(sql);
                stmt.setString(1, fname);
                stmt.setString(2, lname);
                Date date = Date.valueOf(bdate);
                stmt.setDate(3, date);
                stmt.setString(4, "");
                stmt.setString(5, "Employee");
                stmt.setDate(6, Date.valueOf(LocalDate.now()));
                stmt.setString(7, "Unknown");
                stmt.setString(8, "Unknown");
                stmt.setString(9, "00000");
                stmt.setString(10, "Unknown");
                stmt.setString(11, "000-000-0000");
                int inserted = stmt.executeUpdate();
                return inserted > 0;
            } catch (SQLException e) {
                e.printStackTrace();
                return false;
            } finally {
                try {
                    if (stmt != null) stmt.close();
                    if (con != null) con.close();
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}package _01_29_selbst;

```

```

public class OnlineShop {

    static Warenkorb wk = new Warenkorb();
    /**
     * definiere folgende Methoden
     *
     * 1) ein gegebenen Artikel in den Warenkorb aufnehmen
     * 2) einen gegebenen Artikel aus dem Warenkorb entfernen
     * 3) Anzahl der Artikel im Warenkorb ermitteln
     * 4) Überprüfen ob der Warenkorb leer ist
     * 5) Bestellübersicht erstellen und zurückgeben
     *
     * Hinweis:
     */
}

```

```

* angenommen im warenkorb sind _:
* 1.Apfel,1.50
* 2.Bananen,2.50
* 3.Orange,3.50
*
* Bestellübersicht:
*   Apfel,1.50
*   Bananen,2.50
*   Orange,3.50
*
*   Summe: 7.50
*
*
*
* @param args
*/

public static void aufnehmenWarenkorb(Artikel a) {
    wk.items.add(a);
}

public static void entfernenWarenkorb(Artikel a) {
    if (wk.items.contains(a)) {
        int j=wk.items.indexOf(a);
        wk.items.remove(j);
    }
    else
        System.out.println("Der Artikel "+ a +" ist nicht im
Warenkorb!");
}

public static int ermittleAnzahl() {
    return wk.items.size();
}

public static boolean pruefeObLeer() {
    return wk.items.size()==0;//oder .isEmpty
}

public static String erstelleBestelluebersicht() {
    String formatiert="";
    double summe=0.0;
    for (Artikel a: wk.items) {
        formatiert += "%15s %8.2f€\n".formatted(a.getName(),a.getPreis());
        summe +=a.getPreis();
    }
    formatiert += "- ".repeat(30)+"\n"           Summe %8.2f€\n
".formatted(summe);

```

```

        return formatiert;
    }

    public static void main(String[] args) {
        //Test

        Artikel birn= new Artikel("Birnen",4.5);
        System.out.println(OnlineShop.pruefeObLeer());
        OnlineShop.aufnehmenWarenkorb(new Artikel("Apfel",1.5));
        OnlineShop.aufnehmenWarenkorb(new Artikel("Banane",2.5));
        OnlineShop.aufnehmenWarenkorb(new Artikel("Orange",3.5));
        OnlineShop.aufnehmenWarenkorb(new Artikel("Birnen",4.5));

        System.out.println(OnlineShop.erstelleBestelluebersicht());
        System.out.println("=".repeat(30));

        OnlineShop.entfernenWarenkorb((birn));

        System.out.println(OnlineShop.erstelleBestelluebersicht());
        System.out.println("=".repeat(30));

        System.out.println(OnlineShop.ermittleAnzahl());
        OnlineShop.entfernenWarenkorb((birn));

    }
}

```

Package: ObserverPattern

Klasse: AbstractDevice

```

package ObserverPattern;

public abstract class AbstractDevice implements Device {
    protected boolean on = false;
    protected String name;

    public AbstractDevice(String name) {
        this.name = name;
    }

    @Override
    public String getName() {
        return name;
    }
}

```



```

    }

    @Override
    public boolean isOn() {
        return on;
    }

    protected void log(String message) {
        Logger.getInstance().log(message);
    }

    @Override
    public abstract void update(float temperature) throws
DeviceFailureException;

    @Override
    public abstract void resetToDefaults();
}

```

Klasse: AirConditioning

```

package ObserverPattern;

public class AirConditioning extends AbstractDevice {
    private final float DEFAULT_MAX_TEMP = 25;
    private float maxTemp;

    public AirConditioning() {
        super("Klimaanlage");
        maxTemp = DEFAULT_MAX_TEMP;
    }

    public void setMaxTemp(float t) {
        if (t < 20 || t > 30)
            throw new IllegalArgumentException("Ungültige Max-
Temperatur");
        maxTemp = t;
    }

    @Override
    public void update(float temperature) throws
DeviceFailureException {
        if (Math.random() < 0.05)
            throw new DeviceFailureException(name + " ausgefallen!");
        if (temperature > maxTemp && !on) {
            on = true;
            log(name + " eingeschaltet bei " + temperature + "°C");
            System.out.println(name + ": AN");
        } else if (temperature <= maxTemp && on) {
            on = false;
        }
    }
}

```

```

        log(name + " ausgeschaltet bei " + temperature + "°C");
        System.out.println(name + ": AUS");
    }
}

@Override
public void resetToDefaults() {
    maxTemp = DEFAULT_MAX_TEMP;
    log(name + " zurückgesetzt auf Werkseinstellung");
}
}

```

Klasse: Device

```

package ObserverPattern;

public interface Device {
    void update(float temperature) throws DeviceFailureException;

    String getName();

    boolean isOn();

    void resetToDefaults();
}

```

Klasse: DeviceFailureException

```

package ObserverPattern;

public class DeviceFailureException extends Exception {
    public DeviceFailureException(String message) {
        super(message);
    }
}

```

Klasse: FloorHeating

```

package ObserverPattern;

public class FloorHeating extends AbstractDevice {
    private final float DEFAULT_TARGET_TEMP = 21;
    private float targetTemp;

    public FloorHeating() {
        super("Bodenheizung");
        this.targetTemp = DEFAULT_TARGET_TEMP;
    }

    public void setTargetTemp(float t) {
        if (t < 15 || t > 30)
            throw new IllegalArgumentException("Ungültige

```

```

Temperatur");
    this.targetTemp = t;
}

@Override
public void update(float temperature) throws
DeviceFailureException {
    if (Math.random() < 0.05)
        throw new DeviceFailureException(name + " ausgefallen!");
    if (temperature < targetTemp && !on) {
        on = true;
        log(name + " eingeschaltet bei " + temperature + "°C");
        System.out.println(name + ": AN");
    } else if (temperature >= targetTemp && on) {
        on = false;
        log(name + " ausgeschaltet bei " + temperature + "°C");
        System.out.println(name + ": AUS");
    }
}

@Override
public void resetToDefaults() {
    targetTemp = DEFAULT_TARGET_TEMP;
    log(name + " zurückgesetzt auf Werkseinstellung");
}
}

```

Klasse: Heater

```

package ObserverPattern;

public class Heater extends AbstractDevice {
    private final float DEFAULT_TARGET_TEMP = 22;
    private float targetTemp;

    public Heater() {
        super("Heizung");
        this.targetTemp = DEFAULT_TARGET_TEMP;
    }

    public void setTargetTemp(float t) {
        if (t < 15 || t > 30)
            throw new IllegalArgumentException("Ungültige
Temperatur");
        this.targetTemp = t;
    }

    @Override
    public void update(float temperature) throws
DeviceFailureException {
        if (Math.random() < 0.05)

```

```

        throw new DeviceFailureException(name + " ausgefallen!");
    if (temperature < targetTemp && !on) {
        on = true;
        log(name + " eingeschaltet bei " + temperature + "°C");
        System.out.println(name + ": AN");
    } else if (temperature >= targetTemp && on) {
        on = false;
        log(name + " ausgeschaltet bei " + temperature + "°C");
        System.out.println(name + ": AUS");
    }
}

@Override
public void resetToDefaults() {
    targetTemp = DEFAULT_TARGET_TEMP;
    log(name + " zurückgesetzt auf Werkseinstellung");
}
}

```

Klasse: Humidifier

```
package ObserverPattern;
```

```

public class Humidifier implements Device {
    private final float DEFAULT_TARGET_HUMIDITY = 40;
    private float targetHumidity;
    private boolean on = false;
    private String name = "Luftbefeuchter";

    public Humidifier() {
        targetHumidity = DEFAULT_TARGET_HUMIDITY;
    }

    public void setTargetHumidity(float t) {
        targetHumidity = t;
    }

    @Override
    public void update(float temperature) throws
DeviceFailureException {
        if (Math.random() < 0.05)
            throw new DeviceFailureException(name + " ausgefallen!");
        float humidity = 50 - (temperature - 20);
        if (humidity < targetHumidity && !on) {
            on = true;
            log(name + " eingeschaltet bei " + humidity + "%");
            System.out.println(name + ": AN");
        } else if (humidity >= targetHumidity && on) {
            on = false;
            log(name + " ausgeschaltet bei " + humidity + "%");
            System.out.println(name + ": AUS");
        }
    }
}

```

```

    }
}

@Override
public String getName() {
    return name;
}

@Override
public boolean isOn() {
    return on;
}

@Override
public void resetToDefaults() {
    targetHumidity = DEFAULT_TARGET_HUMIDITY;
    log(name + " zurückgesetzt auf Werkseinstellung");
}
private void log(String message) {
    System.out.println("LOG: " + message);
}
}

```

Klasse: LogAnalyzer

```

package ObserverPattern;

import java.io.*;
import java.util.*;
import java.util.stream.*;

public class LogAnalyzer {
    static class DeviceStats {
        long switches = 0, failures = 0;
        List<Long> onTimestamps = new ArrayList<>();
        long totalOnTime = 0;
    }

    public static void analyzeLogs(String filePath) {
        try (BufferedReader br = new BufferedReader(new
FileReader(filePath))) {
            List<String> lines =
br.lines().skip(1).collect(Collectors.toList());
            Map<String, DeviceStats> statsMap = new HashMap<>();
            List<Double> temps = new ArrayList<>();

            for (String line : lines) {
                String[] parts = line.split(",", 2);
                long ts = Long.parseLong(parts[0]);
                String msg = parts[1];
                if (msg.contains("°C"))

```

```

        for (String m : msg.split(" "))
            if (m.contains("°C"))

temps.add(Double.parseDouble(m.replace("°C", "")));
        String deviceName = msg.split(" ")[0];
        statsMap.putIfAbsent(deviceName, new DeviceStats());
        DeviceStats ds = statsMap.get(deviceName);
        if (msg.contains("eingeschaltet")) {
            ds.switches++;
            ds.onTimestamps.add(ts);
        } else if (msg.contains("aus") && !
ds.onTimestamps.isEmpty()) {
            ds.totalOnTime += ts -
ds.onTimestamps.remove(ds.onTimestamps.size() - 1);
        } else if (msg.contains("FEHLER"))
            ds.failures++;
    }

    statsMap.forEach((d, ds) -> {
        System.out.println("\nGerät: " + d);
        System.out.println("    Ein-/Ausschaltungen: " +
ds.switches);
        System.out.println("    Ausfälle: " + ds.failures);
        System.out.println("    Gesamtlaufzeit: " +
ds.totalOnTime / 1000 + " Sekunden");
    });

    DoubleSummaryStatistics tempStats =
temps.stream().mapToDouble(Double::doubleValue).summaryStatistics();
    System.out.println("\nTemperatur Statistik:");
    System.out.println("    Durchschnitt: " +
tempStats.getAverage() + "°C");
    System.out.println("    Minimal: " + tempStats.getMin() +
"°C");
    System.out.println("    Maximal: " + tempStats.getMax() +
"°C");

    } catch (IOException e) {
        System.out.println("Fehler beim Lesen der Logdatei: " +
e.getMessage());
    }
}
}

```

Klasse: Logger

```

package ObserverPattern;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

```

```

public class Logger {
    private static Logger instance;
    private BufferedWriter writer;

    private Logger() {
        try {
            java.io.File file = new java.io.File("log.csv");
            boolean needsHeader = !file.exists() || file.length() ==
0;

            writer = new BufferedWriter(new FileWriter("log.csv",
true));

            if (needsHeader) {
                writer.write("Timestamp,Message\n");
                writer.flush();
                System.out.println("[Logger] Neue Logdatei erstellt
mit Header.");
            }
        } catch (IOException e) {
            System.err.println("Fehler beim Initialisieren des
Loggers: " + e.getMessage());
            e.printStackTrace();
        }
    }

    public static Logger getInstance() {
        if (instance == null)
            instance = new Logger();
        return instance;
    }

    public void log(String message) {
        String line = System.currentTimeMillis() + "," + message;
        System.out.println("[LOG] " + message);
        try {
            writer.write(line);
            writer.newLine();
            writer.flush();
        } catch (IOException e) {
            System.out.println("Fehler beim Schreiben in log.csv: " +
e.getMessage());
        }
    }

    public void close() {
        try {
            if (writer != null)

```

```

        writer.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

Klasse: Main

```
package ObserverPattern;
```

```

public class Main {
    public static void main(String[] args) {
        TemperatureSensor sensor = new TemperatureSensor();
        Statistics stats = new Statistics();

        Heater heater = new Heater();
        FloorHeating floor = new FloorHeating();
        AirConditioning ac = new AirConditioning();
        Humidifier humidifier = new Humidifier();
        DisplayDevice display = new DisplayDevice();

        sensor.addDevice(heater);
        sensor.addDevice(floor);
        sensor.addDevice(ac);
        sensor.addDevice(humidifier);
        sensor.addDevice(display);

        // Simulation initial
        float[] temps = { 20, 23, 26 };
        for (float t : temps) {
            System.out.println("\nTemperatur auf " + t + "°C
setzen:");
            sensor.setTemperature(t);
            stats.recordTemperature(t);
            display.showDeviceStatus(sensor.getDevices());
        }

        // Mieterwechsel
        System.out.println("\n=== Mieterwechsel ===");
        heater.setTargetTemp(20);
        floor.setTargetTemp(19);
        ac.setMaxTemp(24);
        humidifier.setTargetHumidity(35);

        float[] tempsNew = { 18, 21, 23 };
        for (float t : tempsNew) {
            System.out.println("\nTemperatur auf " + t + "°C
setzen:");
            sensor.setTemperature(t);
            stats.recordTemperature(t);

```



```

        display.showDeviceStatus(sensor.getDevices());
    }

    // Werkseinstellungen
    System.out.println("\n=== Werkseinstellungen zurücksetzen
===");
    heater.resetToDefaults();
    floor.resetToDefaults();
    ac.resetToDefaults();
    humidifier.resetToDefaults();

    float[] finalTemps = { 19, 22 };
    for (float t : finalTemps) {
        System.out.println("\nTemperatur auf " + t + "°C
setzen:");
        sensor.setTemperature(t);
        stats.recordTemperature(t);
        display.showDeviceStatus(sensor.getDevices());
    }

    System.out.println("\nDurchschnittstemperatur insgesamt: " +
stats.averageTemperature() + "°C");

    Logger.getInstance().close();

//      System.out.println("\n=== Auswertung der Logdatei ===");
//      LogAnalyzer.analyzeLogs("log.csv");
    }
}

```

Klasse: Statistics

```

package ObserverPattern;

import java.util.*;
import java.util.stream.*;

public class Statistics {
    private List<Float> temperatureHistory = new ArrayList<>();

    public void recordTemperature(float temp) {
        temperatureHistory.add(temp);
    }

    public double averageTemperature() {
        return
temperatureHistory.stream().mapToDouble(Float::doubleValue).average().
orElse(0);
    }
}

```

```

        public void printOnDevices(Set<Device> devices) {
            devices.stream().filter(Device::isOn).forEach(d ->
System.out.println(d.getName() + " ist AN"));
        }
    }
}

```

Klasse: TemperatureSensor

```
package ObserverPattern;
```

```
import java.util.HashSet;
```

```
import java.util.Set;
```

```

public class TemperatureSensor {
    private Set<Device> devices = new HashSet<>();
    private float temperature;

    public void addDevice(Device d) {
        devices.add(d);
    }

    public void removeDevice(Device d) {
        devices.remove(d);
    }

    public void setTemperature(float t) {
        temperature = t;
        notifyDevices();
    }

    private void notifyDevices() {
        for (Device d : devices) {
            try {
                d.update(temperature);
            } catch (DeviceFailureException e) {
                Logger.getInstance().log("FEHLER: " + e.getMessage());
                System.out.println("Warnung: " + e.getMessage());
            }
        }
    }

    public Set<Device> getDevices() {
        return devices;
    }
}

```

Package: Pong

Klasse: Ball

```
package Pong;

import java.awt.Graphics;
import java.awt.Rectangle;

import javax.swing.JOptionPane;

public class Ball {
    private static final int WIDTH = 30, HEIGHT = 30;
    private Pong game;
    private int x, y, xa = 2, ya = 2;

    public Ball(Pong game) {
        this.game = game;
        x = game.getWidth() / 2;
        y = game.getHeight() / 2;
    }

    public void update() {
        x += xa;
        y += ya;
        if (x < 0) {
            game.getPanel().increaseScore(1);
            x = game.getWidth() / 2;
            xa = -xa;
        }
        else if (x > game.getWidth() - WIDTH - 7) {
            game.getPanel().increaseScore(2);
            x = game.getWidth() / 2;
            xa = -xa;
        }
        else if (y < 0 || y > game.getHeight() - HEIGHT - 29)
            ya = -ya;
        if (game.getPanel().getScore(1) == 10)
            JOptionPane.showMessageDialog(null, "Player 1 wins",
"Pong", JOptionPane.PLAIN_MESSAGE);
        else if (game.getPanel().getScore(2) == 10)
            JOptionPane.showMessageDialog(null, "Player 2 wins",
"Pong", JOptionPane.PLAIN_MESSAGE);
        checkCollision();
    }

    public void checkCollision() {
        if
(game.getPanel().getPlayer(1).getBounds().intersects(getBounds()) ||
game.getPanel().getPlayer(2).getBounds().intersects(getBounds()))
```

```

        xa = -xa;
    }

    public Rectangle getBounds() {
        return new Rectangle(x, y, WIDTH, HEIGHT);
    }

    public void paint(Graphics g) {
        g.fillRect(x, y, WIDTH, HEIGHT);
    }
}package Immutable;

public final class Bankkonto {
    private final String inhaber;
    private final double kontostand;

    public String getInhaber() {
        return inhaber;
    }

    public double getKontostand() {
        return kontostand;
    }

    public Bankkonto(String inhaber, double kontostand) {
        this.inhaber = inhaber;
        this.kontostand = kontostand;
    }

    public Bankkonto einzahlen(double betrag) {
        return new Bankkonto(this.inhaber, this.kontostand + betrag);
    }
}package _04_03_2026_Uebung;

public class BankTransfer implements PaymentMethod{

    @Override
    public Konto fuereZahlungDurch(Konto k,double betrag) {
        betrag=Math.abs(betrag);
        if(k.getKontostand()+k.getKreditrahmen()>= betrag +0.5) {
            k.setKontostand(k.getKontostand()-betrag-0.5);
            System.out.println("Überweisung in Höhe von ***
"+String.format("%.2f€",betrag) + " *** erfolgreich ausgeführt. 50
Cent Bearbeitungsgebühr.");
        }
        else
            System.out.println("Kreditrahmen ausgeschöpft.Zahlung
nicht möglich.");
        return k;
    }
}

```

```

    }
    @Override
    public Konto erhalteZahlung(Konto k, double betrag) {
        betrag=Math.abs(betrag);
        k.setKontostand(k.getKontostand()+betrag-0.5);
        System.out.println("Überweisung in Höhe von ***
"+String.format("%.2f€",betrag )+ " *** erfolgreich entgegen genommen.
50 Cent Bearbeitungsgebühr.");
        return null;
    }
}

```

```

}

```

Klasse: Pong

```

package Pong;

import java.awt.Color;

import javax.swing.JFrame;

public class Pong extends JFrame {
    private final static int WIDTH = 700, HEIGHT = 450;
    private PongPanel panel;

    public Pong() {
        setSize(WIDTH, HEIGHT);
        setTitle("Pong");
        setBackground(Color.WHITE);
        setResizable(false);
        setVisible(true);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        panel = new PongPanel(this);
        add(panel);
    }

    public PongPanel getPanel() {
        return panel;
    }

    public static void main(String[] args) {
        new Pong();
    }
}package Pong;

import java.awt.Color;
import java.awt.Graphics;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.awt.event.KeyEvent;

```

```

import java.awt.event.KeyListener;

import javax.swing.JPanel;
import javax.swing.Timer;

public class PongPanel extends JPanel implements ActionListener,
KeyListener {
    private Pong game;
    private Ball ball;
    private Racket player1, player2;
    private int score1, score2;

    public PongPanel(Pong game) {
        setBackground(Color.WHITE);
        this.game = game;
        ball = new Ball(game);
        player1 = new Racket(game, KeyEvent.VK_UP, KeyEvent.VK_DOWN,
game.getWidth() - 36);
        player2 = new Racket(game, KeyEvent.VK_W, KeyEvent.VK_S, 20);
        Timer timer = new Timer(5, this);
        timer.start();
        addKeyListener(this);
        setFocusable(true);
    }

    public Racket getPlayer(int playerNo) {
        if (playerNo == 1)
            return player1;
        else
            return player2;
    }

    public void increaseScore(int playerNo) {
        if (playerNo == 1)
            score1++;
        else
            score2++;
    }

    public int getScore(int playerNo) {
        if (playerNo == 1)
            return score1;
        else
            return score2;
    }

    private void update() {
        ball.update();
        player1.update();
    }

```

```

        player2.update();
    }

    public void actionPerformed(ActionEvent e) {
        update();
        repaint();
    }

    public void keyPressed(KeyEvent e) {
        player1.pressed(e.getKeyCode());
        player2.pressed(e.getKeyCode());
    }

    public void keyReleased(KeyEvent e) {
        player1.released(e.getKeyCode());
        player2.released(e.getKeyCode());
    }

    public void keyTyped(KeyEvent e) {
        ;
    }

    @Override
    public void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.drawString(game.getPanel().getScore(1) + " : " +
game.getPanel().getScore(2), game.getWidth() / 2, 10);
        ball.paint(g);
        player1.paint(g);
        player2.paint(g);
    }
}package _03_27_2026;

public enum Praefix {

    STD,EXP
}

```

Klasse: Racket

```
package Pong;
```

```
import java.awt.Graphics;
import java.awt.Rectangle;
```

```
public class Racket {
    private static final int WIDTH = 10, HEIGHT = 60;
    private Pong game;
    private int up, down;
    private int x;

```

```

private int y, ya;

public Racket(Pong game, int up, int down, int x) {
    this.game = game;
    this.x = x;
    y = game.getHeight() / 2;
    this.up = up;
    this.down = down;
}

public void update() {
    if (y > 0 && y < game.getHeight() - HEIGHT - 29)
        y += ya;
    else if (y == 0)
        y++;
    else if (y == game.getHeight() - HEIGHT - 29)
        y--;
}

public void pressed(int keyCode) {
    if (keyCode == up)
        ya = -1;
    else if (keyCode == down)
        ya = 1;
}

public void released(int keyCode) {
    if (keyCode == up || keyCode == down)
        ya = 0;
}

public Rectangle getBounds() {
    return new Rectangle(x, y, WIDTH, HEIGHT);
}

public void paint(Graphics g) {
    g.fillRect(x, y, WIDTH, HEIGHT);
}
}package Dienstags_Abend;

public class Rechnung {

    String name;
    int rechnungsnummer;
    String datum;

    //Methoden
    static int berechneAbstand(int x,int y) {

```



```

        int abstand=0;

        if(x>=y) {
            abstand=x-y;
        }
        else {
            abstand=y-x;
        }
        return abstand;
    }

    static void printeErgebnis(int temp) {
        System.out.println("Das Ergebnis ist: "+temp);
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getRechnungsnummer() {
        return rechnungsnummer;
    }

    public void setRechnungsnummer(int rechnungsnummer) {
        this.rechnungsnummer = rechnungsnummer;
    }

    public String getDatum() {
        return datum;
    }

    public void setDatum(String datum) {
        this.datum = datum;
    }

}

```

Package: Pythagoras_Zeugs

Klasse: Polyeder

```
package Pythagoras_Zeugs;
```

```

import java.util.*;
import java.io.FileWriter;
import java.io.IOException;

public class Polyeder {

    public static void main(String[] args) {
        int lauf = getLaufIndex();
        ArrayList<int[]> triples = generateTriples(lauf);
        triples.sort((arr1, arr2) -> {
            if (arr1[3] != arr2[3]) return
Integer.compare(arr1[3], arr2[3]);
            if (arr1[0] != arr2[0]) return
Integer.compare(arr1[0], arr2[0]);
            if (arr1[1] != arr2[1]) return
Integer.compare(arr1[1], arr2[1]);
            return Integer.compare(arr1[2], arr2[2]);
        });
        // printArray(removeDuplicateArrays(triples));

        printArray(removeDuplicateAndTeilerfremdArrays(triples));
    }

    private static int getLaufIndex() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Geben Sie den größtmöglichen Laufindex
an: ");
        int lauf = sc.nextInt();
        sc.close();
        return lauf;
    }

    private static ArrayList<int[]> generateTriples(int lauf) {
        ArrayList<int[]> triples = new ArrayList<>();
        for (int s = 1; s <= lauf; s++) {
            for (int t = 0; t <= lauf; t++) {
                for (int u = 1; u <= lauf; u++) {
                    for (int v = 0; v <= lauf; v++) {
                        addTripleIfValid(triples, s, t, u, v);
                    }
                }
            }
        }
        return triples;
    }

    private static void addTripleIfValid(List<int[]> triples, int
s, int t, int u, int v) {
        int a = 2 * (s * v + t * u);
    }
}

```



```

        writer.close();
        fileCounter++;
        writer = new FileWriter("ausgabe_" +
fileCounter + ".txt");
        System.out.println("Neue Datei
erstellt: ausgabe_" + fileCounter + ".txt");
    }
}
}
}
} catch (IOException e) {
    System.out.println("Fehler beim Schreiben in die Datei: "
+ e.getMessage());
} finally {
    try {
        if (writer != null) writer.close();
    } catch (IOException e) {
        System.out.println("Fehler beim Schließen der Datei: "
+ e.getMessage());
    }
}

}

private static String printTetraeder(int[] first, int[]
second) {
    int result1 = calculateResult1(first, second);
    int result3 = calculateResult3(first, second);

    double k1 = first[0], k2 = first[1], k3 = first[2];
    double l1 = second[0], l2 = second[1], l3 = second[2];
    double m = first[3];

    StringBuilder output = new StringBuilder();

    if (result1 == 0) {
        output.append(print0GSVariante1(k1, k2, k3, l1, l2, l3,
m));
        output.append(printTetraeder1(k1, k2, k3, l1, l2, l3, m));
        output.append(print0ktaeder1(k1, k2, k3, l1, l2, l3, m));
    } else if (result3 == 0) {
        output.append(print0GSVariante3(k1, k2, k3, l1, l2, l3,
m));
        output.append(printTetraeder3(k1, k2, k3, l1, l2, l3, m));
        output.append(print0ktaeder3(k1, k2, k3, l1, l2, l3, m));
    }

    return output.toString();
}
}

```

```

        private static int calculateResult1(int[] first, int[] second)
        {
            return -first[2] * second[0] - first[0] * second[1] +
first[1] * second[2];
        }

        private static int calculateResult3(int[] first, int[] second)
        {
            return first[1] * second[0] - first[2] * second[1] +
first[0] * second[2];
        }

        private static String printOGSVariante1(double k1, double k2,
double k3, double l1, double l2, double l3, double m) {
            return String.format(
                "OGS Variante 1:\n  - d = %s, \t a = %s,\tb = %s,\
tc = %s\n " +
                "\t\t\t f = %s,\tg = -%s,\th = %s\n " +
                "\t\t\t u = %s,\tv = %s,\tw = %s
        bilden OGS!\n\n",
                m, k1, k2, k3, l2, l3, l1,
                (k2 * l1 + k3 * l3) / m, (k3 * l2 - k1 * l1) / m,
                (-k1 * l3 - k2 * l2) / m
            );
        }

        private static String printTetraeder1(double k1, double k2,
double k3, double l1, double l2, double l3, double m) {
            return String.format(
                "Tetraeder 1&2:\n  - A(0;0;0), B(%s;%s;%s), C(%s;%s;
%s), D(%s;%s;%s)\n" +
                "  - E(%s;%s;%s), F(%s;%s;%s), G(%s;%s;%s), H(%s;%s;
%s)\n",
                k1 + l2, k2 - l3, k3 + l1,
                k1 + (k2 * l1 + k3 * l3) / m, k2 + (k3 * l2 - k1 * l1)
/ m, k3 + (-k1 * l3 - k2 * l2) / m,
                l2 + (k2 * l1 + k3 * l3) / m, -l3 + (k3 * l2 - k1 *
l1) / m, l1 + (-k1 * l3 - k2 * l2) / m,
                k1, k2, k3, l2, -l3, l1,
                (k2 * l1 + k3 * l3) / m, (k3 * l2 - k1 * l1) / m, (-k1
* l3 - k2 * l2) / m,
                k1 + l2 + (k2 * l1 + k3 * l3) / m, k2 - l3 + (k3 * l2
- k1 * l1) / m, k3 + l1 + (-k1 * l3 - k2 * l2) / m
            );
        }

        private static String printOktaeder1(double k1, double k2,
double k3, double l1, double l2, double l3, double m) {
            return String.format(
                "Oktaeder 1:\n  - A(%s;%s;%s), B(%s;%s;%s), C(%s;
%s;%s)\n" +

```

```

        "    D(%s;%s;%s), E(%s;%s;%s), F(%s;%s;%s)\n",
        (k1 + l2) / 2, (k2 - l3) / 2, (k3 + l1) / 2,
        (k1 + (k2 * l1 + k3 * l3) / m) / 2, (k2 + (k3 * l2
- k1 * l1) / m) / 2, (k3 + (-k1 * l3 - k2 * l2) / m) / 2,
        (l2 + (k2 * l1 + k3 * l3) / m) / 2, (-l3 + (k3 *
l2 - k1 * l1) / m) / 2, (l1 + (-k1 * l3 - k2 * l2) / m) / 2,
        k1 + (l2 + (k2 * l1 + k3 * l3) / m) / 2, k2 + (-l3
+ (k3 * l2 - k1 * l1) / m) / 2, k3 + (l1 + (-k1 * l3 - k2 * l2) / m) /
2,
        l2 + (k1 + (k2 * l1 + k3 * l3) / m) / 2, -l3 + (k2
+ (k3 * l2 - k1 * l1) / m) / 2, l1 + (k3 + (-k1 * l3 - k2 * l2) / m) /
2,
        (k2 * l1 + k3 * l3) / m + (k1 + l2) / 2, (k3 * l2
- k1 * l1) / m + (k2 - l3) / 2, (-k1 * l3 - k2 * l2) / m + (k3 + l1) /
2
    );
}

```

```

private static String printOGSVariante3(double k1, double k2,
double k3, double l1, double l2, double l3, double m) {
    return String.format(
        "OGS Variante 2:\n    - d = %s, \t a = %s,\tb = %s,\n
tc = %s\n" +
        "\t\t\t f = %s,\tg = %s,\th = -%s\n" +
        "\t\t\t u = %s,\tv = %s,\tw = %s
        bilden OGS!\n\n",
        m, k1, k2, k3, l3, l1, l2,
        (-k2 * l2 - k3 * l1) / m, (k3 * l3 + k1 * l2) / m,
        (k1 * l1 - k2 * l3) / m
    );
}

```

```

private static String printTetraeder3(double k1, double k2,
double k3, double l1, double l2, double l3, double m) {
    return String.format(
        "Tetraeder 3&4:\n    - A(0;0;0), B(%s;%s;%s), C(%s;%s;
%s), D(%s;%s;%s)\n" +
        "    - E(%s;%s;%s), F(%s;%s;%s), G(%s;%s;%s), H(%s;%s;
%s)\n",
        k1 + l3, k2 + l1, k3 - l2,
        k1 + (-k2 * l2 - k3 * l1) / m, k2 + (k3 * l3 + k1 *
l2) / m, k3 + (k1 * l1 - k2 * l3) / m,
        l3 + (-k2 * l2 - k3 * l1) / m, l1 + (k3 * l3 + k1 *
l2) / m, -l2 + (k1 * l1 - k2 * l3) / m,
        k1, k2, k3, l3, l1, -l2,
        (-k2 * l2 - k3 * l1) / m, (k3 * l3 + k1 * l2) / m, (k1
* l1 - k2 * l3) / m,
        k1 + l3 + (-k2 * l2 - k3 * l1) / m, k2 + l1 + (k3 * l3
+ k1 * l2) / m, k3 - l2 + (k1 * l1 - k2 * l3) / m
    );
}

```

```

        private static String printOktaeder3(double k1, double k2,
double k3, double l1, double l2, double l3, double m) {
        return String.format(
            "Oktaeder 2:\n - A(%s;%s;%s), B(%s;%s;%s), C(%s;
%s;%s)\n" +
            "    D(%s;%s;%s), E(%s;%s;%s), F(%s;%s;%s)\n",
            (k1 + l3) / 2, (k2 + l1) / 2, (k3 - l2) / 2,
            (k1 + (-k2 * l2 - k3 * l1) / m) / 2, (k2 + (k3 *
l3 + k1 * l2) / m) / 2, (k3 + (k1 * l1 - k2 * l3) / m) / 2,
            (l3 + (-k2 * l2 - k3 * l1) / m) / 2, (l1 + (k3 *
l3 + k1 * l2) / m) / 2, (-l2 + (k1 * l1 - k2 * l3) / m) / 2,
            k1 + (l3 + (-k2 * l2 - k3 * l1) / m) / 2, k2 + (l1
+ (k3 * l3 + k1 * l2) / m) / 2, k3 + (-l2 + (k1 * l1 - k2 * l3) / m) /
2,
            l3 + (k1 + (-k2 * l2 - k3 * l1) / m) / 2, l1 + (k2
+ (k3 * l3 + k1 * l2) / m) / 2, -l2 + (k3 + (k1 * l1 - k2 * l3) / m) /
2,
            (-k2 * l2 - k3 * l1) / m + (k1 + l3) / 2, (k3 * l3
+ k1 * l2) / m + (k2 + l1) / 2, (k1 * l1 - k2 * l3) / m + (k3 - l2) /
2
        );
    }

```

```

/* public static List<int[]> removeDuplicateArrays(List<int[]>
list) {
    Set<String> seen = new HashSet<>();
    List<int[]> uniqueList = new ArrayList<>();

    for (int[] array : list) {
        String arrayString = Arrays.toString(array);
        if (seen.add(arrayString)) {
            uniqueList.add(array);
        }
    }
    return uniqueList;
}
*/

```

```

    public static ArrayList<int[]>
removeDuplicateAndTeilerfremdArrays(ArrayList<int[]> list) {
        HashSet<String> seen = new HashSet<>();
        ArrayList<int[]> uniqueList = new ArrayList<>();

        for (int[] array : list) {
            String arrayString = Arrays.toString(array);
            if (!seen.contains(arrayString) &&
sindTeilerfremd(array)) {
                uniqueList.add(array);
                seen.add(arrayString);
            }
        }
    }

```

```

    }
    return uniqueList;
}

public static boolean sindTeilerfremd(int[] array) {
    int gcd = array[0];
    for (int i = 1; i < array.length; i++) {
        gcd = berechneGCD(gcd, array[i]);
        if (gcd == 1) {
            return true; // Werte sind teilerfremd
        }
    }
    return false;
}

public static int berechneGCD(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
}

```

Package: SingletonPattern

Klasse: AppConfig

```

package SingletonPattern;

public class AppConfig {

    private String name;
    private String version;

    private static AppConfig instance = new AppConfig("App-XX","1.0");
    //wichtig statisch und privat

    private AppConfig(String name, String version) {
        this.name = name;
        this.version = version;
    }

    public static AppConfig getInstance() {
        return instance;
    }

    @Override

```



```

        public String toString() {
            return "AppConfig [name=" + name + ", version=" + version +
"]";
        }
        public String getName() {
            return name;
        }
        public void setName(String name) {
            this.name = name;
        }
        public String getVersion() {
            return version;
        }
        public void setVersion(String version) {
            this.version = version;
        }
    }
}

```

Klasse: Monat

```
package SingletonPattern;
```

```
import java.util.Arrays;
```

```
public enum Monat {
```

```

    JANUAR(31), FEBRUAR(28), MAERZ(31), APRIL(30), MAI(31), JUNI(30), JULI(31), AUGUST(31),
    SEPTEMBER(30), OKTOBER(31), NOVEMBER(30), DEZEMBER(31);

```

```
    private int tage;
```

```

    public int getTage(boolean isSchaltjahr) {
        if(ordinal()==1 && isSchaltjahr)
            return 29;
        return tage;
    }

```

```
    private Monat(int tage) {
```

```

        this.tage=tage;
    }

```

```
}
```

```
class EnumTest{
```

```

    public static void main(String[] args) {
        Monat m1= Monat.JANUAR;
    }
}

```

```

        System.out.println("m1="+m1.getTage(false));
        Monat m2= Monat.FEBRUAR;
        System.out.println("m2="+m2.getTage(false));
        Monat m3= Monat.MAERZ;
        System.out.println("m3="+m3.getTage(false));
        Monat m4= Monat.APRIL;
        System.out.println("m4="+m4.getTage(false));
        Monat m5= Monat.MAI;
        System.out.println("m5="+m5.getTage(false));
        Monat m6= Monat.JUNI;
        System.out.println("m6="+m6.getTage(false));
        Monat m7= Monat.JULI;
        System.out.println("m7="+m7.getTage(false));
        Monat m8= Monat.AUGUST;
        System.out.println("m8="+m8.getTage(false));
        Monat m9= Monat.SEPTEMBER;
        System.out.println("m9="+m9.getTage(false));
        Monat m10= Monat.OKTOBER;
        System.out.println("m10="+m10.getTage(false));
        Monat m11= Monat.NOVEMBER;
        System.out.println("m11="+m11.getTage(false));
        Monat m12= Monat.DEZEMBER;
        System.out.println("m12="+m12.getTage(false));

        //System.out.println(Monat.values().toString());geht nicht
        System.out.println(Arrays.toString(Monat.values()));
        System.out.println(m1.ordinal());//wie ein index

        for (Monat mt : Monat.values())
            System.out.println(mt+ " hat "+ mt.getTage(true) + "
Tage");

    }

}

```

Klasse: Singelton

```

package SingeltonPattern;

public class Singelton {

    public static void main(String[] args) {
        AppConfig a1 = AppConfig.getInstance();
        AppConfig a2 = AppConfig.getInstance();

        System.out.println(a1);
        System.out.println(a2);
        System.out.println(a1==a2);
    }
}

```

```

        System.out.println(a1.hashCode()+" "+ a2.hashCode());
    }

}

```

Klasse: SingeltonPattern

```

package SingeltonPattern;

import java.io.Console;

/**
 * nur ein einziges Objekt der Klasse
 */
public class SingeltonPattern {

    public static void main(String[] args) {
        Singleton s = Singleton.getInstance();
        // Console c =System.console();

        Coin c = Coin.ZWEIEURO;
        System.out.println(c.getWert());
    }

}

enum Coin{
    ZWEIEURO(200),EINEURO(100),FUENFZIGCENT(50);
    private int wert;

    public int getWert() {
        return wert;
    }

    private Coin(int wert) {
        this.wert=wert;
    }

}

class Singleton{
    private static Singleton instance= new Singleton();
    //..

    private Singleton() {
        //...
    }
}

```

```

    }
    public static Singleton getInstance() {
        return instance;
    }
}

package _03_26;

import java.io.Console;

/**
 * Nur ein einziges Objekt pro Klasse
 */

public class SingetonPattern {
    public static void main(String[] args) {
        Singleton s = Singleton.getInstance();
        //Console c = System.console();

        Coin c = Coin.ZWEIEURO;
        System.out.println(c.getWert());

    }
}

enum Coin{
    ZWEIEURO(200), EINEURO(100), FUENFZIGZENT(50);
    private int wert;

    public int getWert() {
        return wert;
    }

    private Coin(int wert) {
        this.wert = wert;
    }
}

class Singleton{
    private static Singleton instance = new Singleton() ;

    //...

    private Singleton() {
        //....
    }

    public static Singleton getInstance() {
        return instance ;
    }
}

package _03_26;

```

```

public class Singleton {
public static void main(String[] args) {
    AppConfig a1 = AppConfig.getInstance();
    AppConfig a2 = AppConfig.getInstance();

    System.out.println(a1);
    System.out.println(a2);
    System.out.println(a1 == a2);
    System.out.println(a1.hashCode() + " " + a2.hashCode());

}
}

```

Package: Uebung_03_03_2026

Klasse: Main

```

package Uebung_03_03_2026;

import java.time.LocalDate;

public class Main {

    public static void main(String[] args) {

        Person wondmu = new Person("Alemu", "Wodemu", 44,
LocalDate.of(2025, 1, 3));

        Person selim = new Person("Palim", "Selim", 34,
LocalDate.of(2026, 1, 2));
        Person selim2 = new Person("Palim", "Selim2", 14,
LocalDate.of(2026, 1, 4));

        System.out.println(wondmu);
        System.out.println(selim);
        System.out.println(selim2);
        wondmu.gehtAufsKlo();
        selim.gehtAufsKlo();
        Person.benenneMotto();
//        System.out.println(LocalDate.now());
        Person aushilfe =new Person("Gushti", "Gushtine",21,
LocalDate.now());
        System.out.println(aushilfe);
        MitarbeiterDesMonats mdm =
MitarbeiterDesMonats.bestimmeMitarbeiterdesMonats(aushilfe);
        System.out.println(mdm);
        mdm =
MitarbeiterDesMonats.bestimmeMitarbeiterdesMonats(selim);

```

```

        System.out.println(mdm);
    }
}

```

Klasse: MitarbeiterDesMonats

```
package Uebung_03_03_2026;
```

```

public class MitarbeiterDesMonats {

    public static MitarbeiterDesMonats mitarbeiter;
    public int zusatzgehalt;
    private Person p;

    private MitarbeiterDesMonats( Person p, int zusatzgehalt) {

        this.setP(p);
        this.zusatzgehalt = zusatzgehalt;
    }
    public int getZusatzgehalt() {
        return zusatzgehalt;
    }


    public void setZusatzgehalt(int zusatzgehalt) {
        this.zusatzgehalt = zusatzgehalt;
    }

    @Override
    public String toString() {
        return "MitarbeiterDesMonats [zusatzgehalt=" + zusatzgehalt +
"€, p=" + p + "]";
    }
    public static MitarbeiterDesMonats
bestimmeMitarbeiterdesMonats(Person p) {
        if (mitarbeiter==null) {
            mitarbeiter=new MitarbeiterDesMonats(p, 100);
        }
        else {
            System.out.println(p+" darf das nicht sein! Mitarbeiter
des Monats ist schon vergeben!");
        }
        return mitarbeiter;
    }
    public Person getP() {
        return p;
    }
    public void setP(Person p) {
        this.p = p;
    }
}

```

```
}
```

```
}
```

Klasse: Person

```
package Uebung_03_03_2026;
```

```
import java.time.LocalDate;
```

```
public class Person {
```

```
    private String name;
```

```
    private String vorname;
```

```
    private int alter;
```

```
    private LocalDate datum;
```

```
    private int id;
```

```
    public static int mitarbeiteranzahl = 0;
```

```
    public static String motto="Mitarbeiter bei der GFN!Kotz!";
```

```
    public Person(String name, String vorname, int alter, LocalDate  
datum) {
```

```
        this.name = name;
```

```
        this.vorname = vorname;
```

```
        this.alter = alter;
```

```
        this.datum = datum;
```

```
        mitarbeiteranzahl++;
```

```
        this.id = mitarbeiteranzahl;
```

```
    }
```

```
    public String getName() {
```

```
        return name;
```

```
    }
```

```
    public void setName(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    public String getVorname() {
```

```
        return vorname;
```

```
    }
```

```
    public void setVorname(String vorname) {
```

```
        this.vorname = vorname;
```

```
    }
```

```

    public int getAlter() {
        return alter;
    }

    public void setAlter(int alter) {
        this.alter = alter;
    }

    public LocalDate getDatum() {
        return datum;
    }

    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }

    public static int getMitarbeiteranzahl() {
        return mitarbeiteranzahl;
    }

    public static void setMitarbeiteranzahl(int mitarbeiteranzahl) {
        Person.mitarbeiteranzahl = mitarbeiteranzahl;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        // if.....
        this.id = id;
    }

    @Override
    public String toString() {
        return "Person [name=" + name + ", vorname=" + vorname + ",
alter=" + alter + ", datum=" + datum
        + ", mitarbeiteranzahl= " + mitarbeiteranzahl + ",ID="
+ id + " ]";
    }

    public void gehtAufsKlo(){
        System.out.println(vorname+" "+name + " geht schon wieder aufs
Klo!");
    }

    public static void benenneMotto(){
        System.out.println(motto);
    }

```



```
    }  
}
```

Package: Uebung_2_03_03_2026

Klasse: Angestellte

```
package Uebung_2_03_03_2026;
```

```
public class Angestellte extends Person{  
    private double gehalt;  
    private String firmenwagen;  
  
    public Angestellte(String name, String vorname, int alter, double  
gehalt, String firmenwagen) {  
        super(name, vorname, alter);  
        this.gehalt = gehalt;  
        this.firmenwagen = firmenwagen;  
    }  
    public double getGehalt() {  
        return gehalt;  
    }  
    public void setGehalt(double gehalt) {  
        this.gehalt = gehalt;  
    }  
    public String getFirmenwagen() {  
        return firmenwagen;  
    }  
    public void setFirmenwagen(String firmenwagen) {  
        this.firmenwagen = firmenwagen;  
    }  
    @Override  
    public String toString() {  
        return "Angestellte [name="+name+", vorname="+vorname+",  
alter="+alter+", gehalt=" + gehalt + ", firmenwagen=" + firmenwagen +  
"]";  
    }  
    @Override  
    public String arbeitetUnter() {  
  
        return "CEO-Ungemütlich";  
    }  
    @Override//write=schreiben  
    public String hatWlanZugang() {  
  
        return "Glasfaser lwl 2000, Passwort=1234";  
    }  
}
```

```
}
```

Klasse: Arbeiter

```
package Uebung_2_03_03_2026;
```

```
public class Arbeiter extends Person{
    private double lohn;
    private int urlaubstage;
    public Arbeiter(String name, String vorname, int alter, double
    lohn, int urlaubstage) {
        super(name, vorname, alter);
        this.lohn = lohn;
        this.urlaubstage = urlaubstage;
    }
    public double getLohn() {
        return lohn;
    }
    public void setLohn(double lohn) {
        this.lohn = lohn;
    }
    public int getUrlaubstage() {
        return urlaubstage;
    }
    public void setUrlaubstage(int urlaubstage) {
        this.urlaubstage = urlaubstage;
    }
    @Override
    public String toString() {
        return "Arbeiter [name="+name+", vorname="+vorname+",
alter="+alter+", lohn=" + lohn + ", urlaubstage=" + urlaubstage + "];"
    }
    @Override
    public String arbeitetUnter() {

        return "Der Cheffe";
    }
    @Override
    public String hatWlanZugang() {

        return "Base T1000, Passwort= Pa$$w0rd";
    }
}
```

Klasse: Gast

```
package Uebung_2_03_03_2026;

public class Gast extends Person{
    private int aufenthalstdauer;
    private boolean hatPrivatAuto;
    public Gast(String name, String vorname, int alter, int
aufenthalstdauer, boolean hatPrivatAuto) {
        super(name, vorname, alter);
        this.aufenthalstdauer = aufenthalstdauer;
        this.hatPrivatAuto = hatPrivatAuto;
    }
    public Gast(String name, int alter, int aufenthalstdauer, boolean
hatPrivatAuto) {
        super(name, alter);
        this.aufenthalstdauer = aufenthalstdauer;
        this.hatPrivatAuto = hatPrivatAuto;
    }
    public Gast(String name, int aufenthalstdauer, boolean
hatPrivatAuto) {
        super(name);
        this.aufenthalstdauer = aufenthalstdauer;
        this.hatPrivatAuto = hatPrivatAuto;
    }

    public int getAufenthalstdauer() {
        return aufenthalstdauer;
    }
    public void setAufenthalstdauer(int aufenthalstdauer) {
        this.aufenthalstdauer = aufenthalstdauer;
    }
    public boolean isHatPrivatAuto() {
        return hatPrivatAuto;
    }
    public void setHatPrivatAuto(boolean hatPrivatAuto) {
        this.hatPrivatAuto = hatPrivatAuto;
    }

    @Override
    public String toString() {
        String entscheidung= hatPrivatAuto ? "ja" : "nein";
        // return "Gast ["+vorname+" "+name+", alter="+alter+",
aufenthalstdauer=" + aufenthalstdauer + "h, hatPrivatAuto=" +
hatPrivatAuto + "]";
        return String.format("Gast:\n-----\nVorname: %s\nNachname:
%s\nAlter: %d\nAufenthaltsdauer: %dh\nhat ein privates Auto: %s\
n", vorname, name, alter, aufenthalstdauer, entscheidung);
    }
    @Override
    public String arbeitetUnter() {
```

```

        return "der muss gar nicht arbeiten, du Depp!";
    }
    @Override
    public String hatWlanZugang() {
        return "Gastzugang micro, Passwort=GastGFN";
    }
}

```

Klasse: Person

```

package Uebung_2_03_03_2026;

public abstract class Person {
    protected String name;
    protected String vorname;
    protected int alter;

    public Person(String name, String vorname, int alter) {
        this.name = name;
        this.vorname = vorname;
        this.alter = alter;
    }
    /**
     * Überladen
     * @param name
     * @param alter
     */
    public Person(String name,int alter) {
        this.name=name;
        this.vorname="hat kein Vorname";//Defaultwert
        setAlter(alter);
    }

    /**
     * noch mehr Überladen:
     */
    public Person(String name) {
        this.name=name;
        this.vorname="hat kein Vorname";//Defaultwert
        this.alter=3000;//Defaultwert
    }

    public String getName() {
        return name;
    }
}

```

```

    public void setName(String name) {
        this.name = name;
    }

    public String getVorname() {
        return vorname;
    }

    public void setVorname(String vorname) {
        this.vorname = vorname;
    }

    public int getAlter() {
        return alter;
    }

    public void setAlter(int alter) {
        if (alter >= 0)
            this.alter = alter;
        else
            throw new IllegalArgumentException("Alter unmöglich!");
    }

    @Override
    public String toString() {
        return "Person [name=" + name + ", vorname=" + vorname + ",
alter=" + alter + "]";
    }

    public abstract String arbeitetUnter();

    public abstract String hatWlanZugang();
}

```

Klasse: Verwaltung

```
package Uebung_2_03_03_2026;
```

```

public class Verwaltung {

    public static void main(String[] args) {
        Person[] personen = new Person[6];
    }
}

```

```

        personen[0]=new Arbeiter("Alemu", "Wondmu", 44, 1234.56, 1);
        personen[1]=new Angestellte("Dias","Vivian", 29, 1320.46,
"Fiat 500");
        personen[2]=new Gast("nebenan", "Der Typ von", 88, 12, false);
        personen[3]=new Gast("Schneider", 88, 12, false);
        personen[4]=new Gast("Goehte", 300, 12, true);
        personen[5]=new Gast("Maradonna", 100, true);

        for (Person pers : personen) {
            System.out.println("=".repeat(75));
            System.out.println(pers);
            System.out.println("arbeitet unter:
"+pers.arbeitetUnter());
            System.out.println("Wlanzugang: "+pers.hatWlanZugang());
            System.out.println("=".repeat(75));
        }

//      int[] zahlenarray= {1,2,3,4,5,6,7,8,9};
//
//      for(int zahl : zahlenarray){
//          System.out.println(zahl);
//      }

    }

}

```

Package: [_01_26](#)

Klasse: [Person](#)

```

package _01_26;

import java.util.Objects;
/**
 * Wenn Attribute explizit nicht initialisiert worden sind,
 * dann bekommen diese Attribute "default values"
 * Ganze Zahlen: 0
 * Fließkommazahlen: 0.0
 * Wahrheitswert: false
 * String: null
 *
 */
public class Person {

    private String name;
    private int alter;

```

```

/**
 * Die Klasse Person hat neben Name und Alter Gewicht in kg und
Grösse in cm
 * TODO Deklariere diese 2 Attribute entsprechend
 * Definiere Getters und Setters
 * Konstruktor soll angepasst werden
 * Achte auf sinnvolle Wertüberprüfung
 * Definiere zwei zusätzliche Methoden:
 * -Die methode berechnet den BMI und gibt den Wert zurück
 * -Die Methode erhöht das Gewicht der Person um einen gegebenen
Betrag, wenn der Betrag > 0 ist
 *
 * Teste die Klasse
 */

public Person(){
}

public Person(String name, int alter){
    this.name=name;
//    if (alter >=0) {
//        this.alter=alter;
//    }
    setAlter(alter);
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public int getAlter() {
    return alter;
}
public void setAlter(int alter) {
    if (alter >=0) {
        this.alter = alter;
    }
}

}
@Override
public int hashCode() {
    return Objects.hash(alter, name);
}
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;

```

```

        if (getClass() != obj.getClass())
            return false;
        Person other = (Person) obj;
        return alter == other.alter && Objects.equals(name,
other.name);
    }

```

```

}

```

Klasse: Test

```

package _01_26;
/**
 * Wiederholung:
 *   Arrays
 *   OOP
 *   keywords:static, final
 */
public class Test {
    public static void main(String[] args) {
        System.out.println("Hallo Welt");

        Person p = new Person("Patrick", 24);
        p.setAlter(27);
        System.out.println("Name: "+p.getName());
        System.out.println("Alter: "+p.getAlter());
        Person p2 = new Person("Luigi",33);
        System.out.println(p2.getName()+" ist "+ p2.getAlter() + "
Jahre alt.");
        Person p3= new Person("Alice",-21);
        System.out.println(p3.getName());
        System.out.println(p3.getAlter());
    }
}

```

Klasse: Zusammenfassung

```

package _01_26;

/**
 * Klassen definieren
 *
 * Attribute
 *   -private, public
 *   -static
 *   -final
 *
 * Konstruktoren
 * Methoden
 * Setters() und Getters()

```



```

* toString()
* -
* -Validation
*
* Objekte erzeugen
*
*/

```

```

public class Zusammenfassung {

}

```

Package: [_01_26_uebung](#)

Klasse: Fahrzeug

```

package _01_26_uebung;

import java.time.LocalDate;

public class Fahrzeug {

    private String marke;
    private final int baujahr;
    private double kilometerstand;
    private String standort;
    public static int autoanzahl=0;
    private final int id;

    public String toString() {
        return "Fahrzeug [Marke=" + marke + ", Baujahr=" + baujahr +
", Kilometerstand=" + kilometerstand
        + "km, Standort=" + standort + ", Fahrzeugs-ID: "+
id+"]";
    }

    public Fahrzeug(String marke, int baujahr, double kilometerstand,
String standort) {

        this.marke = marke;
        if (baujahr>=1885 && baujahr <= LocalDate.now().getYear()) {
            this.baujahr = baujahr;
        }
        else
            this.baujahr=-1;
        setKilometerstand(kilometerstand);
        this.standort = standort;
        autoanzahl++;
        this.id=autoanzahl;
    }
}

```

```

    }

    public String getMarke() {
        return marke;
    }

    public void setMarke(String marke) {
        this.marke = marke;
    }

    public double getKilometerstand() {

        return kilometerstand;
    }

    public void setKilometerstand(double kilometerstand) {
        if (kilometerstand>=0)
            this.kilometerstand = kilometerstand;
        else
            this.kilometerstand=0;
    }

    public String getStandort() {
        return standort;
    }

    public void setStandort(String standort) {
        this.standort = standort;
    }

    public int getBaujahr() {
        return baujahr;
    }

    public void fahren(double strecke) {
        if (strecke >0) {
            kilometerstand +=strecke;
            System.out.println("Update  nach der Fahrt: "+this);

        }else
            System.out.println("Die gefahrene Strecke muss positiv
sein!");
    }

}

```

Klasse: Fuhrpark

```
package _01_26_uebung;

public class Fuhrpark {

    public static void main(String[] args) {

        Fahrzeug f =new Fahrzeug("BMW",1977,23554.2,"Freiburg");
        System.out.println(f);
        f.fahren(12.5);
        System.out.println(f);
        f.fahren(-445.11);

        Fahrzeug f2= new Fahrzeug("Mercedes",2024,112.4,"Nürnberg");
        System.out.println(f2);
        f2.fahren(788.02);

        System.out.println("Die Anzahl der Autos im Fuhrpark sind:
        "+Fahrzeug.autoanzahl);

        final Fahrzeug fahr= new Fahrzeug("VW-
        Käfer",1974,250000.3,"Schrottplatz");
        fahr.fahren(35.7);
        // fahr=new Fahrzeug("Mercedes",2024,112.4,"Nürnberg"); geht
        nicht wegen final!

        Fahrzeug f3=new Fahrzeug("Porsche
        911",1998,23501.2,"Stuttgart");
        System.out.println(f3);
        f3.fahren(204.7);
    }

}
```

Klasse: Person

```
package _01_26_uebung;

import java.util.Objects;

public class Person {
    /**
     * @param
     *
     * gewicht in kg
     * groesse in cm
     * alter in Jahren
     *
     * die Klasse Personen modelliert Personen mit folgenden
     */
}
```

Einschränkungen:

```
* mindestalter=18
* maximalalter=122
* mindestgrösse=60cm
* maximalgrösse=226cm
* mindestgewicht=5,5kg
* maximalgewicht=360kg
*
*/
```

```
private final String geburtsort;
private String name;
private double gewicht;
private double groesse;
private int alter;
```

```
public static final int MIN_ALTER=18;
public static final int MAX_ALTER=122;
```

```
public static final double MIN_GROESSE = 60;
public static final double MAX_GROESSE = 226;
```

```
public static final double MIN_GEWICHT = 5.5;
public static final double MAX_GEWICHT = 360.0;
```

```
// double groesse_meter; abgeleitete grössen sind wegen folgefehler
schlecht
```

```
public Person(String name, int alter, double gewicht, double
groesse,String geburtsort) {
    this.name = name;
    setAlter(alter);
    setGewicht(gewicht);
    setGroesse(groesse);
    this.geburtsort=geburtsort;
    // groesse_meter=groesse/100.0;
}
```

```
public String getGeburtsort() {
    return geburtsort;
}
```

```
public String getName() {
    return name;
}
```

```
public void setName(String name) {
    this.name = name;
}
```

```
public int getAlter() {
```

```

        return alter;
    }
    public void setAlter(int alter) {
        if ( alter>=MIN_ALTER && alter <=MAX_ALTER) {
            this.alter = alter;
        }
        else
            this.alter=MIN_ALTER;
    }
    public double getGewicht() {
        return gewicht;
    }
    public void setGewicht(double gewicht) {
        if (gewicht>=MIN_GEWICHT && gewicht <=MAX_GEWICHT) {
            this.gewicht = gewicht;
        }
        else {
            this.gewicht=MIN_GEWICHT;
        }
    }

    }
    public double getGroesse() {
        return groesse;
    }
    public void setGroesse(double groesse) {
        if (groesse>=MIN_GROESSE && groesse <= MAX_GROESSE) {
            this.groesse = groesse;
        }
        else
            this.groesse=MIN_GROESSE;
    }
    }

    public double berechne_BMI() {

        double bmi = gewicht/groesse/groesse*10000;
        return ((int)(bmi*100))/100.0;
    }

    public double erhoehe_gewicht(double erhoehung) {
        if(erhoehung > 0 && (erhoehung+gewicht) <=MAX_GEWICHT) {
            gewicht += erhoehung;
        }
        else if (erhoehung+gewicht>360) {
            System.out.println(name+" darf nicht soviel
zunehmen!");
            gewicht = MAX_GEWICHT;
        }
        else {
            System.out.println("Nur positive Werte für eine

```

```

    Erhöhung eingeben!");
    }
    return gewicht;
}

@Override
public int hashCode() {
    return Objects.hash(alter, gewicht, groesse, name);
}

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Person other = (Person) obj;
    return alter == other.alter &&
Double.doubleToLongBits(gewicht) ==
Double.doubleToLongBits(other.gewicht)
        && Double.doubleToLongBits(groesse) ==
Double.doubleToLongBits(other.groesse)
        && Objects.equals(name, other.name);
}

    public String toString() {
        return "Person [Name=" + name + ", Gewicht=" + gewicht + "kg,
Groesse=" + groesse + "cm, Alter=" + alter + " Jahre"+ " Geburtsort:
"+geburtsort+ "]" ;
    }

}

```

Klasse: Test

```

package _01_26_uebung;

public class Test {
    public static void main(String[] args) {

        Person p =new Person("Luigi", 44, 88.3, 173.7,"Bonn");
        System.out.println("Name: "+p.getName());
        System.out.println("Alter: "+p.getAlter());
        System.out.println("Gewicht in kg: "+p.getGewicht());
        System.out.println("Grösse in cm: "+p.getGroesse());
        System.out.println("Der BMI ist: "+p.berechne_BMI());
        p.erhoehe_gewicht(2.8);
    }
}

```

```

        System.out.println("Neues Gewicht von "+p.getName()+":
"+p.getGewicht()+"kg");
        System.out.println(p);

        Person p2 = new Person("Alice", -21, 55.0, 165.0,"Bern");
        System.out.println(p2);

        Person p3= new Person("Gewichti",49, 359.4 ,177.2,"Freiburg");
        System.out.println(p3);
        p3.erhoehe_gewicht(2.7);
        p3.erhoehe_gewicht(-55.1);
        p3.erhoehe_gewicht(0.2);
        System.out.println(p3);

        Person p4= new Person("Falschi",17,5.1,228,"Berlin");
        System.out.println(p4);

        Person p5= new Person("Zunehmi",19,100,226,"Duisburg");
        System.out.println(p5);
        p5.erhoehe_gewicht(261.2);
        System.out.println(p5);

//        final int i =3;
//        i++; geht nicht da final

        final Person pers= new Person ("Chris",
21 ,78.5,155.7,"Leipzig");
        System.out.println(pers);
        pers.setAlter(22);
        pers.setGewicht(80.2);
        System.out.println(pers);

    }
}

```

Package: _01_27

Klasse: ArraysInJava

package _01_27;

import java.util.Iterator;

public class ArraysInJava {

public static void main(String[] args) {

/**

* Array der Grösse 8 erzeugt

```

        * Die Elemente sind vom Typ: String
        * Alle 8 Elemente sind auf default value gesetzt: null
        * Zugriff auf die Elemente mittels Indizierung
        */
String[] muenzen = new String[8];

//      System.out.println(muenzen[0]);
//      System.out.println(muenzen[1]);
//      System.out.println(muenzen[2]);

String[] coins = {"1 Cent","2 Cent","5 Cent","10 Cent","20
Cent","50 Cent","1 Euro","2 Euro"};

    for (int i=0;i<coins.length;i++) {
        muenzen[i]=coins[i];
        System.out.printf("%7s\n",coins[i]);

    }
    System.out.println("-".repeat(20));

//      for (int i=0;i<coins.length;i++) {
//          muenzen[i]=coins[i];
//          System.out.printf("%7s\n",muenzen[i]);
//      }
//Besser mit der for-each:

        for(String muenze : muenzen){
            System.out.printf("%7s\n",muenze);
        }
        print(muenzen);
        print(coins);
    }

static void print(String[] array) {
    for(String element : array) {
        System.out.print(element + " ");
    }
    System.out.println("-".repeat(58));
}

}

```

Klasse: Arrayuebung

```
package _01_27;
```

```
import java.util.Arrays;
import java.util.Scanner;
```

```
public class Arrayuebung {
```



```

public static void main(String[] args) {
    int[] verkaufszahlen= {22,34,10,9,12,55,0};
    double durchschnitt=0.0;
    double summe=0;
    for(int i:verkaufszahlen) {
        summe +=i;
    }
    durchschnitt=summe/verkaufszahlen.length;
    System.out.printf("Durchschnitt: %.2f\n",durchschnitt);
    int max=verkaufszahlen[0];
    for(int i:verkaufszahlen) {
        max= i > max ? i: max;
    }
    System.out.println("Maximum: "+max);

    int[][] tescht= new int[15][7];
    for (int i=0; i<tescht.length;i++) {
        for(int j=0;j<tescht[0].length;j++) {
            tescht[i][j]=7*i+j+1;
        }
    }
    double summe3=0;
    double durchschnitt3=0.0;
    int max3= tescht[0][0];

    for (int i=0; i<tescht.length;i++) {
        for(int j=0;j<tescht[0].length;j++) {
            max3 = tescht[i][j] > max3 ? tescht[i][j]:max3;
            summe3 += tescht[i][j];
        }
    }
    durchschnitt3=summe3/(tescht.length*tescht[0].length);
    System.out.printf("Durchschnitt2: %.2f\n",durchschnitt3);
    System.out.println("Maximum2: "+max3);
    for (int[] bla : tescht) {
        System.out.println(Arrays.toString(bla));
    }

    int[][] wochenuebersicht = {
        {1,2,3,4,5,6,7},
        {8,9,10,11,12,13,14},
        {15,16,17,18,19,20,21},
        {22,23,24,25,26,27,28}
    };

    double summe2=0;

```

```

        double durchschnitt2=0.0;
        int max2= wochenuebersicht[0][0];
        for (int i=0; i<wochenuebersicht.length;i++) {
            for(int j=0;j<wochenuebersicht[0].length;j++) {
                max2 = wochenuebersicht[i][j] > max2 ?
wochenuebersicht[i][j]:max2;
                summe2 += wochenuebersicht[i][j];
            }
        }

        durchschnitt2=summe2/(wochenuebersicht.length*wochenuebersicht[0].length);

        System.out.printf("Durchschnitt2: %.2f\n",durchschnitt2);
        System.out.println("Maximum2: "+max2);
        for (int[] bla : tescht) {
            System.out.println(Arrays.toString(bla));
        }

        Scanner sc =new Scanner(System.in);
        System.out.println("Geben Sie einen Namen ein: ");
        String name= sc.next();
        if(name.length() <3) {
            System.out.println("Bitte Namen mit mindestens 3
Buchsteaben eingeben!");
            name=sc.next();
        }
        String name2="Bob";
        if (name.equals(name2))
            System.out.println("Die Namen sind gleich!");
        System.out.println(name.toUpperCase());
        System.out.println(name.length());
        // String[] zeichen2= name.getChars(0, name.length(), [],
name.length());
        // char[] zeichen = new char[name.length()];
        // name.getChars(0, name.length(), zeichen, 0);
        // System.out.println(zeichen.length);
        //
        sc.close();
        System.out.println(Arrays.deepToString(tescht));
    }
}

```

Klasse: Benutzer

```

package _01_27;

public class Benutzer {

    private String name;

```

```

private static int benutzeranzahl;
private final int id;

private static final int MAX_BENUTZER=7;

public Benutzer(String name) {
    this.name = name;
    benutzeranzahl++;
    this.id=benutzeranzahl;
}
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
public static int getBenutzeranzahl() {
    return benutzeranzahl;
}
public static void setBenutzeranzahl(int benutzeranzahl) {
    Benutzer.benutzeranzahl = benutzeranzahl;
}
public int getId() {
    return id;
}
public static int getMaxBenutzer() {
    return MAX_BENUTZER;
}
public String toString() {
    return "Benutzer [Name=" + name + ", id=" + id + "]";
}

public static void main(String[] args) {

    Benutzer b1= new Benutzer("Gushti");
    Benutzer b2= new Benutzer("Luigi");
    Benutzer b3= new Benutzer("Gerd");
    Benutzer b4= new Benutzer("Gertrude");
    Benutzer b5= new Benutzer("Geraldine");

    System.out.println(b1);
    System.out.println(b2);
    System.out.println(b3);
    System.out.println(b4);
    System.out.println(b5);
    System.out.println("Anzahl der Benutzer: "+
Benutzer.benutzeranzahl);
}

```

```
    }  
}
```

Klasse: Fahrzeug

```
package _01_27;
```

```
import java.time.LocalDate;
```

```
public class Fahrzeug {
```

```
    private final String marke;  
    private final int baujahr;  
    private double kilometerstand;  
    private String standort;  
    private static int autoanzahl=0;  
    public static int getAutoanzahl() {  
        return autoanzahl;  
    }  
}
```

```
    public static void setAutoanzahl(int autoanzahl) {  
        Fahrzeug.autoanzahl = autoanzahl;  
    }  
}
```

```
    public int getId() {  
        return id;  
    }  
}
```

```
    private final int id;
```

```
    public String toString() {  
        return "Fahrzeug [Marke=" + marke + ", Baujahr=" + baujahr +  
", Kilometerstand=" + kilometerstand  
        + "km, Standort=" + standort + ", Fahrzeugs-ID: "+  
id+"]";  
    }  
}
```

```
    public Fahrzeug(String marke, int baujahr, double kilometerstand,  
String standort) {
```

```
        this.marke = marke;  
        if (baujahr>=1885 && baujahr <= LocalDate.now().getYear()) {  
            this.baujahr = baujahr;  
        }  
        else  
            this.baujahr=-1;  
        if (kilometerstand>=0)  
            this.kilometerstand = kilometerstand;
```

```
        this.standort = standort;  
        autoanzahl++;  
    }  
}
```

```

        this.id=autoanzahl;
    }

    public String getMarke() {
        return marke;
    }

// public void setMarke(String marke) {
//     this.marke = marke;
// }

    public double getKilometerstand() {

        return kilometerstand;
    }

// public void setKilometerstand(double kilometerstand) {
//     if (kilometerstand>=0)
//         this.kilometerstand = kilometerstand;
//     else
//         this.kilometerstand=0;
// }

    public String getStandort() {
        return standort;
    }

    public void setStandort(String standort) {
        this.standort = standort;
    }

    public int getBaujahr() {
        return baujahr;
    }

    public void fahren(double strecke) {
        if (strecke >0) {
            kilometerstand +=strecke;
            System.out.println("Update  nach der Fahrt: "+this);

        }else
            System.out.println("Die gefahrene Strecke muss positiv
sein!");
    }

```

```
}
```

Klasse: Lager

```
package _01_27;
```

```
import java.util.Arrays;
```

```
public class Lager {
```

```
    static String[] produkte = {"Milch", "Bier", "Wasser"};
```

```
    static int[][] verkaufszahlen= {  
                                {85,75,65,35,49,89,76},  
                                {45,19,39,45,64,12,0},  
                                {47,48,18,32,78,12,6}  
                                };
```

```
// static int[] verkaufszahlen_wasser ={85,75,65,35,49,89,76};
```

```
    static double berechneDurchschnitt(int[] vekproProdukt) {
```

```
        double sum =0.0;
```

```
        for( int verkaufszahl1: vekproProdukt){
```

```
            sum +=verkaufszahl1;
```

```
        }
```

```
        return sum/vekproProdukt.length;
```

```
    }
```

```
    static int ermittleMax(int[] vekproProdukt) {
```

```
        int max=vekproProdukt[0];
```

```
        for( int verkaufszahl1: vekproProdukt){
```

```
            if(verkaufszahl1>max) {
```

```
                max=verkaufszahl1;
```

```
            }
```

```
        }
```

```
        return max;
```

```
    }
```

```
    static void stats() {
```

```
        for(int i=0;i<produkte.length;i++){
```

```
            System.out.println("===== ");
```

```
            System.out.println("===== " + produkte[i]+"  
=====");
```

```
            System.out.println("Verkaufszahlen  "+  
Arrays.toString(verkaufszahlen[i]));
```

```
            System.out.println("Durchschnitt: " +  
berechneDurchschnitt(verkaufszahlen[i]));
```

```
            System.out.println("Höchstwert: "+  
ermittleMax(verkaufszahlen[i]));
```

```
        }
```

```
}
```

```
}
```

Klasse: Prog

```
package _01_27;
```

```
import _01_26.Person ;
```

```
public class Prog {
```

```
    public static void main(String[] args) {
```

```
        Person p = new Person("Wondmu",45);
```

```
        p.setName("Wondmu Alemu");
```

```
        System.out.println(p.getName() +" ist am letzten Montag  
"+p.getAlter()+" Jahre alt geworden.");
```

```
    }
```

```
}
```

Klasse: TestLager

```
package _01_27;
```

```
public class TestLager {
```

```
    public static void main(String[] args) {
```

```
        Lager.stats();
```

```
    }
```

```
}
```

Klasse: Uebung

```
package _01_27;
```

```
import java.util.Arrays;
```

```
public class Uebung {
```

```
    public static void main(String[] args) {
```

```
        int[] numbers = {1,3,5,7,9};
```

```
        int x=45;
```

```
        Fahrzeug f=new Fahrzeug("VW", 1975,10000,"Wolfsburg");
```

```
        System.out.println("numbers = "+numbers);//hashcode
```

```

        System.out.println("numbers = "+Arrays.toString(numbers));
        System.out.println("x=" +x);
        System.out.println("f="+f);//ohne toString() Methode kommt
hier ein hashcode
        System.out.println(numbers.toString());//hash-code

```

```

        /**
         *
         */
        double [] punkte = {1.3,2.7,2.3,1.7,3.3,3.0};
        System.out.println("punkte=" + Arrays.toString(punkte));

        //Punkte sortieren (aufsteigend)

        Arrays.sort(punkte);
        System.out.println("punkte=" + Arrays.toString(punkte));
    }
}

```

Klasse: VergleichbarkeitVonString

```

package _01_27;

public class VergleichbarkeitVonString {

    public static void main(String[] args) {
        String s1="Bob";
        String s2="Bob";
        System.out.println(s1==s2);//true
        System.out.println(s1.equals(s2));//true

        String s3 = new String("Alice");
        String s4 = new String("Alice");

        System.out.println("=".repeat(20));

        System.out.println(s3==s4);//false
        System.out.println(s3.equals(s4));//true
    }

}

```

Klasse: mehrdimensionaleArrays

```

package _01_27;

import java.util.Arrays;

```



```

public class mehrdimensionaleArrays {
public static void main(String[] args) {

    int[] numbers= {1,3,5,7,9};

    int[]numbers2= new int[5];
    for (int i=0; i<numbers2.length; i+=2 ) {
        numbers2[i]=i+1;
    }

    //Zweidimensionales Array

    int[][] table= {
        {1,3,5,7,9},
        {0,2,4,6,8}
    };
    System.out.println(table.length);//2
    System.out.println(table[0].length);//5
    System.out.println(table[1].length);//5

    int[][] table2=new int [2][5];
    for (int i =0;i<table2.length;i++) {
        for (int j=0;j<table2[i].length;j++) {
            table2[i][j]=2*j+1-i%2;
        }
    }
    System.out.println(Arrays.toString(numbers2));
    for(int[] is:table2) {
        System.out.println(Arrays.toString(is));
    }

}

}
}

```

Package: 01_28

Klasse: IO

```
package _01_28;
```

```
import java.time.LocalDate;
import java.util.Scanner;
```

```
public class IO {
```

```

    public static int readInt(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextInt();
    }
    public static double readDouble(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextDouble();
    }
    public static String readString(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextLine();
    }
    public static boolean readBoolean(String prompt) {
        Scanner scan = new Scanner(System.in);
        while (true) {
            System.out.print(prompt);
            String input = scan.nextLine().trim().toLowerCase();
            if (input.equals("j") || input.equals("ja") ||
input.equals("y") || input.equals("yes")) {
                return true;
            } else if (input.equals("n") || input.equals("nein") ||
input.equals("no")) {
                return false;
            } else {
                System.out.println("Ungültige Eingabe! Bitte 'j',
'ja', 'n' oder 'nein' eingeben.");
            }
        }
    }
    public static LocalDate readDate(String prompt) {
        System.out.println(prompt);
        try {
            return LocalDate.of(
                IO.readInt("Jahr: "),
                IO.readInt("Monat: "),
                IO.readInt("Tag: ")
            );
        } catch (Exception e) {
            System.out.println("Ungültiges Datum! Heutiges Datum wird
verwendet!");
            return LocalDate.now();
        }
    }
}

```

Klasse: MinMax

```
package _01_28;

import java.util.Scanner;

public class MinMax {
    public static void main(String[] args) {
        int[] zahlen = {5,2,10,9,12,3};
        int min =zahlen [0];
        int max =zahlen [0];

        bestimmeStatistik(zahlen);
        Scanner sc =new Scanner(System.in);
        sc.useDelimiter("[^0-9-]+");
        int[] zahlen2 = new int[5];
        System.out.print("Geben sie die "+zahlen2.length+ " Zahlen mit
einem Komma getrennt ein: ");

        for (int i=0;i<zahlen2.length;i++) {
            zahlen2[i]=sc.nextInt();
        }
        sc.close();
        bestimmeStatistik(zahlen2);
    }

    public static void bestimmeStatistik(int[] zahlen) {
        int min =zahlen [0];
        int max =zahlen [0];
        System.out.print("Der Such-Array beträgt: ");
        for (int zahl :zahlen) {
            System.out.print(zahl+" ");
            min=min<zahl? min:zahl;
            max=max>zahl? max:zahl;
        }
        System.out.println();
        System.out.println();
        System.out.println("Der Max-Wert beträgt: "+max);
        System.out.println("Der Min-Wert beträgt: "+min);
    }
}

//Version Holger
//package _01_28;
//
//import java.util.Scanner;
//
//public class MinMax {
//    // public static int[] zahlen = { 5, 2, 10, 9, 12, 3 };
//    //
//    // public static void getMinMax(int[] zahlen) {
```

```

// int min = zahlen[0];
// int max = zahlen[0];
// for (int i = 0; i < zahlen.length; i++) {
//     if (zahlen[i] < min) {
//         min = zahlen[i];
//     }
//     if (zahlen[i] > max) {
//         max = zahlen[i];
//     }
// }
// System.out.printf("Der Such-Array beträgt:");
// for (int zahl : zahlen) {
//     System.out.printf(" %d", zahl);
// }
// int len;
// if ((" " + max).length() > (" " + min).length()) {
//     len = (" " + max).length();
// } else {
//     len = (" " + min).length();
// }
// System.out.printf("\n\nDer Max-Wert beträgt: %" + len + "d\nDer
Min-Wert beträgt: %" + len
//     + "d\n-----\n", max, min);
// }
//
// public static int[] giveNumbers() {
//     Scanner sc = new Scanner(System.in);
//     sc.useDelimiter("[^0-9-]+");
//     int[] numbers = new int[5];
//     System.out.printf("Bitte geben Sie die Zahlen ein:\n");
//     int i = 0;
//     while (i < numbers.length && sc.hasNextInt()) {
//         numbers[i] = sc.nextInt();
//         System.out.println(numbers[i]);
//         i++;
//     }
//
//     sc.close();
//     return numbers;
// }
//
// public static void main(String[] args) {
//     getMinMax(zahlen);
//     getMinMax(giveNumbers());
// }
//}
//

```

Klasse: Teilnehmerverwaltung

```
package _01_28;

import java.util.Arrays;

public class Teilnehmerverwaltung {

    private Teilnehmer[] teilnehmerliste;

    public Teilnehmerverwaltung(int n) {
        teilnehmerliste=new Teilnehmer[n];
    }

    public void add(Teilnehmer t) {
        for (int i=0;i<teilnehmerliste.length;i++) {
            if(teilnehmerliste[i]==null) {
                teilnehmerliste[i]=t;
                return ;
            }
        }

        teilnehmerliste=Arrays.copyOf(teilnehmerliste,teilnehmerliste.length+1
);
        teilnehmerliste[teilnehmerliste.length-1]=t;
    }

    public static void main(String[] args) {
        Teilnehmerverwaltung tv =new Teilnehmerverwaltung(5);
        //tl -> [null,null,null,null,null]
        tv.add(new Teilnehmer("Benjamin"));
        tv.add(new Teilnehmer("Holger"));
        tv.add(new Teilnehmer("Olga"));
        tv.add(new Teilnehmer("Tibor"));
        tv.add(new Teilnehmer("Sebastian"));

        tv.add(new Teilnehmer("Patrick"));

        System.out.println(Arrays.toString(tv.teilnehmerliste));
    }
}

class Teilnehmer{
    private String name;

    public Teilnehmer(String name) {
```

```

        this.name = name;
    }

    @Override
    public String toString() {
        return "Teilnehmer [name=" + name + "]";
    }
}

```

Klasse: Temperaturdifferenzen

```

package _01_28;

import java.util.Arrays;

public class Temperaturdifferenzen {

    public static void main(String[] args) {
        int[] temperaturen= {12,15,14,17,22,19,16};
        System.out.println("Temperaturen der Woche:");
        for (int temperatur :temperaturen) {
            System.out.print(temperatur+" ");
        }
        System.out.println();
        // System.out.println("Temperaturen der Woche:\n"+
        Arrays.toString(temperaturen));
        System.out.println("Größte Temperaturdifferenz:
        "+berechneGroessterUnterschied(temperaturen));
    }

    public static int berechneGroessterUnterschied(int[] temperaturen)
    {
        int unterschied=0;
        for (int i=0;i<temperaturen.length-1;i++) {
            int diff=Math.abs(temperaturen[i]-temperaturen[i+1]);
            unterschied=unterschied > diff ? unterschied : diff;
        }
        return unterschied;
    }
}

```

Klasse: Test

```

package _01_28;

import java.util.Arrays;

/**

```

```

* TODO:
*     - Ermögliche, dass Inhalt des Warenkorbs nummeriert und
tabelarische auf der Konsole ausgegeben wird.
*     - Ermittle das teuerste Artikel im Warenkorb.
*/
public class Test {
public static void main(String[] args) {
    Warenkorb korb = new Warenkorb();

    korb.artikelHinzufuegen("College Block", 2.0, 2); // 32
    korb.artikelHinzufuegen("Kugelschreiber", 3.0, 3); // 48
    korb.artikelHinzufuegen("Ipad",329.0,2);
    korb.artikelHinzufuegen("Monitor",145.0,1);
    korb.artikelHinzufuegen("Schreibtisch",243.0,1);
// double max=0.0;
// String nameMaxPreis="";
// int i=1;
// for(Artikel a : korb.items) {
//     System.out.printf("%4d %15s %8.2f€\n",i,a.getName(),a.getPreis());//d=decimal,s=string,f=float,\n=neue
Zeile(parameter mit Komma trennen)
//     i++;
//     max = a.getPreis() > max ? a.getPreis():max;
//     if(max==a.getPreis()) {
//         nameMaxPreis=a.getName();
//     }
// }
// System.out.printf("Der höchste Preis ist %.2f € bei dem Artikel:
%s \n",max,nameMaxPreis);
//
//
    System.out.printf("Der Gesamtpreis ist: %.2f € \n",korb.berechneGesamtwert()); // 80

    korb.anzeigen();
    System.out.println("Der teuerste Artikel ist:
"+korb.ermittleTeuersteArtikel()+"€");

System.out.println(korb.istImWarenkorb(IO.readString("Artikelsuche:\nGeben Sie den Artikelname ein: ")));
    System.out.println(Arrays.asList(new Artikel("Katzenfutter",
3.29)));
    System.out.println(korb.istImWarenkorb(new Artikel("College
Block",2.0)));
}
}

```

Klasse: UebungArrayVerlaengern

package _01_28;

```

import java.util.Arrays;

public class UebungArrayVerlaengern {

    public static void main(String[] args) {
        int[] numbers = {1,3,5};

        // numbers[2]=7;
        // numbers[3]=9; geht nicht!

        int [] data= new int[numbers.length+1];

        for (int i=0; i< numbers.length;i++) {
            data[i]=numbers[i];
        }
        data[data.length -1]=7;

        System.out.println(Arrays.toString(numbers));
        System.out.println(Arrays.toString(data));

        int[] kopie=Arrays.copyOf(numbers, numbers.length+1);
        kopie[kopie.length-1]=7;//letzte Stelle
        System.out.println(Arrays.toString(kopie));

        //verlängern und zum gleichen Objekt zuordnen
        kopie=Arrays.copyOf(kopie, kopie.length+1);
        kopie[kopie.length-1]=7;
        System.out.println(Arrays.toString(kopie));

    }

}

```

Klasse: Warenkorb

```

package _01_28;

import java.util.ArrayList;

public class Warenkorb {
    //Artikel[] items;

    ArrayList<Artikel> items;

    public Warenkorb() {
        this.items = new ArrayList<Artikel>();
    }
}

```



```

    public void artikelHinzufuegen(String name, double preis, int
stueckZahl) {
        for(int i = 0 ; i < stueckZahl; i++)
            items.add(new Artikel(name, preis));
    }

    public double berechneGesamtwert() {
        double gesamtwert = 0.0;
        for (Artikel artikel : items)
            gesamtwert += artikel.getPreis();
        return gesamtwert;
    }

    public void anzeigen() {
        int i=1;
        for (Artikel item: items) {
            System.out.printf("%4d %s\n",i++,item.format());
        }
    }

    public Artikel ermittleTeuersteArtikel() {
        Artikel a =items.get(0);//mit get kommt dann der Index
        for(Artikel item:items) {
            if (item.getPreis()>a.getPreis())
                a=item;
        }
        return a;
    }
    /**
     * definiere eine Methode die überprüft ob ein gegebener Artikel
    schon im Warenkorb ist
    */

    public String istImWarenkorb(String pruefname) {
        String ergebnis="";
        boolean istDrin=false;
        for(Artikel item:items) {
            if
(item.getName().toLowerCase().equals(pruefname.toLowerCase())) {
                istDrin=true;
                break;
            }
        }
        if( istDrin)
            ergebnis= pruefname +" ist bereits im Warenkorb";
        else
            ergebnis= pruefname +" ist noch nicht im Warenkorb";
    }

```

```

        return ergebnis;
    }

    public boolean istImWarenkorb(Artikel a) {
//        for(Artikel item:items) {
//            if(item.equals(a))
//                return true;
//        }
//        return false;
        return items.contains(a);
    }

}

```

Package: `_01_28_selbst`

Klasse: Artikel

```

package _01_28_selbst;

public class Artikel {

    private final String name;
    private double preis;

    public Artikel(String name, double preis) {

        this.name = name;
        setPreis(preis);
    }

    public String toString() {
        return "Artikel [Name=" + name + ", Preis=" + preis + "€]";
    }

    public double getPreis() {
        return preis;
    }

    public void setPreis(double preis) {
        if (preis>0)
            this.preis = preis;
    }

    public String getName() {
        return name;
    }

}

```

```
}
```

Package: _01_29

Klasse: Artikel

```
package _01_29;
```

```
import java.util.Objects;
```

```
public class Artikel {  
    private final String name;  
    private double preis;
```

```
  
    public Artikel(String name, double preis) {  
        this.name = name;  
        this.preis = preis;  
    }
```

```
  
    public double getPreis() {  
        return preis;  
    }
```

```
  
    public void setPreis(double preis) {  
        this.preis = preis;  
    }
```

```
  
    public String getName() {  
        return name;  
    }
```

```
  
    public String format() {  
        return "%15s %8.2f €".formatted(this.name, this.preis);  
    }
```

```
  
    @Override  
    public String toString() {  
        return name + ", " + preis;  
    }
```

```

@Override
public int hashCode() {
    return Objects.hash(name, preis);
}

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Artikel other = (Artikel) obj;
    return Objects.equals(name, other.name)
        && Double.doubleToLongBits(preis) ==
Double.doubleToLongBits(other.preis);
}

}

```

Klasse: OnlineShop

```

package _01_29;

public class OnlineShop {
    static Warenkorb wk = new Warenkorb();

    static void artikelHinzufuegen(Artikel a) {

        wk.getItems().add(a);
    }
    /**
     *
     * @param a
     * @return true, wenn a im Warenkorb vorkommt und entfernt werden
könnte, sonst false
     *
     * Alle Elemente mit e.equals(a) == true, werden entfernt
     */
    static boolean artikelEntfernen(Artikel a) {

        boolean success = wk.getItems().remove(a);

        while(wk.getItems().remove(a))
            wk.getItems().remove(a);
        return success;
    }
}

```

```

}
static int getAnzahlVonArtikel() {
    return wk.getItems().size();
}

static boolean istWarenkorbLeer() {
    return wk.getItems().isEmpty();
}

static StringBuilder erzeugeBestellUebersicht() {
    //String result = "";
    StringBuilder result = new StringBuilder();
    for (Artikel item : wk.getItems()) {
        //result += String.format("%15s %8.2f €\n",
item.getName(), item.getPreis());
        result.append(String.format("%15s %8.2f €\n",
item.getName(), item.getPreis()));
    }

    result.append("\n===== \nSumme
" +String.format("%8.2f", wk.berechneGesamtwert())+" 0€");
    return result;
}
/**
 *
 * Definieren Methoden:
 *      1 - Ein gegebenes Artikel ins Warenkorb aufnehmen
 *      2 - Ein gegebenes Artikel aus dem Warenkorb entfernen
 *      3 - Anzahl von Artikel im Warenkorb ermitteln
 *      4 - Überprüfen ob Warenkorb leer ist
 *      5 - Bestellungsübersicht erzeugen und zurückgeben:
 *
 * Hinweis:
 *      Angenommen, Warenkorb besteht aus folgenden
Artikeln:
 *
 *      1. Apfel, Preis 1.50
 *      2. Bananne, Preis 2.50
 *      3. Orange, Preis 3.50
 *
 * Bestellungsübersicht:
 *      Apfel, Preis 1.50
 *      Bananne, Preis 2.50
 *      Orange, Preis 3.50
 *
 *      Summe:    7.50 €
 *
 * Formatierung ist gewünscht...

```

```

        *
        */
public static void main(String[] args) {
    artikelHinzufuegen(new Artikel("Apfel", 1.5));
    artikelHinzufuegen(new Artikel("Bananne", 2.50));
    artikelHinzufuegen(new Artikel("Orange", 3.50));

    System.out.println(erzeugeBestellUebersicht());
}
}

```

Klasse: Quiz

```

package _01_29;

import java.util.Arrays;

public class Quiz {

    public static void main(String[] args) {
        int[] ia = new int[] {1,3,5};
        int[] data = ia; // data verweist auf dasselbe Objekt wie ia
        int [] numbers = new int[] {1,3,5};

        int i = 3;
        int j = i; // Value Assignment (Wert-Zuweisung): Inhalt der
        Variable i wird kopiert und an j zugewiesen

        i++; // i -> 4, j -> 3
        ia[0] = 0;

        numbers [0] = -1;

        /**
         * 1. Wie viele Objekte gibt es an dieser Stelle? 2
         *
         * 2. Was ist Inhalt der Variablen an dieser Stelle?
         */

        System.out.println(Arrays.toString(ia));
        System.out.println(Arrays.toString(data));
        System.out.println(Arrays.toString(numbers));
        System.out.println(i);
        System.out.println(j);

    }

}

```

Klasse: Quiz_2

```
package _01_29;

import _01_27.Fahrzeug;

public class Quiz_2 {
    public static void main(String[] args) {
        String s1 = new String("Alice");
        Fahrzeug f1 = new Fahrzeug("VW", 1975, 100, "Wolfsburg");

        System.out.println("s1 = " + s1); // Alice
        System.out.println("f1 = " + f1);

        //...

        String s2 = s1.toUpperCase();

        f1.fahren(500);
        f1.setStandort("München");

        System.out.println("s1 = " + s1); // Alice
        System.out.println("s2 = " + s2); // ALICE
        System.out.println("f1 = " + f1); // VW 1975 600 München

        StringBuffer sbuf = new StringBuffer("Alice");

        sbuf.append(" Marley");

        System.out.println(sbuf);

    }
}
```

Klasse: Referenz_Und_Value_Semantik

```
package _01_29;

import java.util.ArrayList;
import java.util.Arrays;

public class Referenz_Und_Value_Semantik {

    static void inc(int x) {
        x++;
    }

    static void set(int[] data, int index, int e) {
        data[index] = e;
    }
}
```

```

    }

    static void append(ArrayList<String> list, String e) {
        list.add(e);
    }

    public static void main(String[] args) {
        int i = 3;
        int[] numbers = {1,3,5};

        ArrayList<String> names = new
ArrayList<String>(Arrays.asList("Bob", "Alice"));

        System.out.println("names = " + names);
        System.out.println("numbers = " + Arrays.toString(numbers));
        System.out.println("i = " + i);

        System.out.println("=====");
        inc(i);
        set(numbers, 0, 21);
        append(names, "Chris");
        // Was ist die Ausgabe?
        System.out.println("names = " + names);
        System.out.println("numbers = " + Arrays.toString(numbers));
        System.out.println("i = " + i);

    }

}

```

Klasse: StringConstantPool

```

package _01_29;

public class StringConstantPool {

    public static void main(String[] args) {

        // Normale Heap-Objekte (Konstruktor wurde aufgerufen)
        String s1 = new String("Alice");
        String s2 = new String("Alice");
        String s3 = new String("Alice");

        // Literal sind im String Constant Pool
        String s4 = "Alice";
        String s5 = "Alice";
        String s6 = "Alice";
    }
}

```



```

// s4, s5 und s6 sind identisch

System.out.println((s1 == s2) + " " + (s1 == s3));

System.out.println((s4 == s5) + " " + (s4 == s6));

//

String name = "Wondmu";
name.concat(" ").concat("Alemu").concat("
").concat("Kidie").concat(" ").concat("Checkol");
System.out.println("name = " + name);

StringBuilder myName = new StringBuilder("Wondmu");
myName.append(" ").append("Alemu").append("
").append("Kidie").append(" ").append("Checkol");
System.out.println("His name = " + myName);

}

}

```

Klasse: Warenkorb

```

package _01_29;

import java.util.ArrayList;

public class Warenkorb {

    private final ArrayList<Artikel> items;

    public ArrayList<Artikel>.getItems() {
        return items;
    }

    public Warenkorb() {
        this.items = new ArrayList<Artikel>();
    }

    public void artikelHinzufuegen(String name, double preis, int
stueckZahl) {
        for(int i = 0 ; i < stueckZahl; i++)
            items.add(new Artikel(name, preis));
    }
}

```

```

public double berechneGesamtwert() {
    double gesamtwert = 0.0;

    for (Artikel artikel : items)
        gesamtwert += artikel.getPreis();

    return gesamtwert;
}

public void anzeigen() {
    int i = 1;
    for (Artikel item : items) {
        System.out.printf("%4d %s €\n", i++, item.format());
    }
}

public Artikel ermittleTeuerstesArtikel() {
    Artikel a = items.get(0);

    for (Artikel item : items) {
        if (item.getPreis() > a.getPreis())
            a = item;
    }
    return a;
}

/**
 * Definiere eine Methode, die überprüft ob ein gegebenes Artikel
im Warenkorb beinhaltet ist
 */
public boolean istArtikelImWarenkorb(String artikelName) {

    for (Artikel item : items) {
        if(item.getName().equals(artikelName))
            return true;
    }

    return false;
}

public boolean istArtikelImWarenkorb(Artikel a) {

```

```

        return items.contains(a);
    }
}

```

Package: _01_29_selbst

Klasse: Artikel

```

package _01_29_selbst;

import java.util.Objects;

public class Artikel {
    private final String name;
    private double preis;

    public Artikel(String name, double preis) {
        this.name = name;
        this.preis = preis;
    }

    public double getPreis() {
        return preis;
    }

    @Override
    public String toString() {
        return "" + name + ", " + preis;
    }

    public void setPreis(double preis) {
        this.preis = preis;
    }

    public String getName() {
        return name;
    }

    public String format() {
        return "%15s %8.2f€ ".formatted(this.name, this.preis);
    }

    @Override
    public int hashCode() {
        return Objects.hash(name, preis);
    }

    @Override

```

```

    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Artikel other = (Artikel) obj;
        return Objects.equals(name, other.name)
            && Double.doubleToLongBits(preis) ==
Double.doubleToLongBits(other.preis);
    }

```

```

}

```

Klasse: Quiz

```

package _01_29_selbst;

import java.util.Arrays;

public class Quiz {

    public static void main(String[] args) {
        int[] ia = {1,3,5};
        int[] data=ia; //data verweist auf das selbe objekt wie ia
        int[] numbers= new int[] {1,3,5}; //neues objekt erzeugt

        int i =3;//Stack-Speicher
        int j =i;//Value Assignment(Wertzuzuweisung) der inhalt der
Variable i wird kopiert und j zugewiesen

        i++;//verändert nur i nicht j
        ia[0]=0;//verändert sowohl ia als auch data

        numbers[0]= -1;//verändert nur numbers

        System.out.println(i);//i=4
        System.out.println(j);//j=3
        System.out.println(ia);//[I@5f184fc6
        System.out.println(data);//[I@5f184fc6
        System.out.println(numbers);//[I@3feba861
        System.out.println(Arrays.toString(numbers));//[-1, 3, 5]
    }
}

```

```

        System.out.println(Arrays.toString(ia));//[0, 3, 5]
        System.out.println(Arrays.toString(data));//[0, 3, 5]

    }
}

Klasse: Quiz2
package _01_29_selbst;

import _01_27.Fahrzeug;

public class Quiz2 {

    public static void main(String[] args) {
        String s1 =new String("Alice");
        Fahrzeug f1 =new Fahrzeug("VW",1975,100,"Wolfsburg");

        System.out.println("s1 = "+s1);
        System.out.println("f1 = "+ f1);

        String s2 = s1.toUpperCase();

        s1.toUpperCase();
        f1.fahren(500);
        f1.setStandort("München");

        System.out.println("s1 = "+s1);//Alice nicht ALICE! Strings
sind immutable
        System.out.println("s2 = "+s2);//ALICE
        System.out.println("f1 = "+ f1);

        StringBuffer sbuf= new StringBuffer("Alice");//StringBuffer
ist mutable
        sbuf.append(" Marley");
        System.out.println(sbuf);

    }

}

```

Klasse: Referenz_und_Value_Semantik

```

package _01_29_selbst;

import java.util.ArrayList;
import java.util.Arrays;

```

```

public class Referenz_und_Value_Semantik {

    static void inc(int x){
        x++;
    }

    static void set (int[] data, int index, int e) {
        data[index]=e;
    }

    static void append(ArrayList<String>list, String e) {
        list.add(e);
    }

    public static void main(String[] args) {
        int i=3;
        int[] numbers= {1,3,5};

        ArrayList<String> names =new
ArrayList<String>(Arrays.asList("Bob","Alice"));
        System.out.println("names = "+names);
        System.out.println("numbers= "+Arrays.toString(numbers));
        System.out.println("i= "+i);

        System.out.println("=".repeat(30));
        inc(i);
        set(numbers,0,21);
        append(names,"Chris");

        //Was ist die Ausgabe?
        System.out.println("names = "+names);//
["Bob","Alice","Chris"]
        System.out.println("numbers=
"+Arrays.toString(numbers));//[21,3,5]
        System.out.println("i= "+i);// i=3

    }

}

```

Klasse: StringConstantPool

```
package _01_29_selbst;
```

```
public class StringConstantPool {
```

```
    public static void main(String[] args) {
```

```
        //Literal sind im String Constant POOL
        String s1="Alice";
        String s2 ="Alice";
        String s3 = "Alice";
```

```
        //hier neues Objekt(Heap-Objekte) Konstruktor wurde aufgerufen
        String s4 = new String("Alice");
```

```
        System.out.println(s1.hashCode()+" "+ s2.hashCode() +" "+
s3.hashCode()+" "+s4.hashCode());
        System.out.println(s1==s2);
        System.out.println(s1==s4);
```

```
        s2=s2.toLowerCase();
        System.out.println(s1);
        System.out.println(s2);
        System.out.println(s1==s2);
        System.out.println(s1==s3);
```

```
        String name = "Wondmu";
        name.concat(" ").concat("Alemu").concat("
").concat("Kidie").concat(" ").concat("Checkol");
        System.out.println("name = " + name);
```

```
        StringBuilder myName = new StringBuilder("Wondmu");
        myName.append(" ").append("Alemu").append("
").append("Kidie").append(" ").append("Checkol");
        System.out.println("myName = " + myName);
```

```
    }
```

```
}
```

Klasse: Warenkorb

```
package _01_29_selbst;
```

```
import java.util.ArrayList;
```

```
public class Warenkorb {
```

```

//Artikel[] items;

ArrayList<Artikel> items;

public Warenkorb() {
    this.items = new ArrayList<Artikel>();
}

public void artikelHinzufuegen(String name, double preis, int
stueckZahl) {
    for(int i = 0 ; i < stueckZahl; i++)
        items.add(new Artikel(name, preis));
}

public double berechneGesamtwert() {
    double gesamtwert = 0.0;
    for (Artikel artikel : items)
        gesamtwert += artikel.getPreis();
    return gesamtwert;
}

public void anzeigen() {
    int i=1;
    for (Artikel item: items) {
        System.out.printf("%4d %s\n",i++,item.format());
    }
}

public Artikel ermittleTeuersteArtikel() {
    Artikel a =items.get(0);//mit get kommt dann der Index
    for(Artikel item:items) {
        if (item.getPreis()>a.getPreis())
            a=item;
    }
    return a;
}
/**
 * definiere eine Methode die überprüft ob ein gegebener Artikel
schon im Warenkorb ist
 */

public String istImWarenkorb(String pruefname) {
    String ergebnis="";
    boolean istDrin=false;
    for(Artikel item:items) {
        if
(item.getName().toLowerCase().equals(pruefname.toLowerCase())) {
            istDrin=true;

```



```

                break;
            }
        }
        if( istDrin)
            ergebnis= pruefname +" ist bereits im Warenkorb";
        else
            ergebnis= pruefname +" ist noch nicht im Warenkorb";
        return ergebnis;
    }

    public boolean istImWarenkorb(Artikel a) {
//        for(Artikel item:items) {
//            if(item.equals(a))
//                return true;
//        }
//        return false;
        return items.contains(a);
    }

}

```

Package: _01_30

Klasse: Garten

```

package _01_30;

import java.util.ArrayList;

public class Garten {

    static ArrayList<Form> figures = new ArrayList<>();

    public static double berechneGesamtflaeche() {
        double total=0.0;
        for(Form f : figures) {
            total += f.berechneFlaeche();
        }
        return total;
    }

    public static void main(String[] args) {
        figures.add(new Rechteck(new Punkt(0,0), 3, 4));
        figures.add(new Kreis(new Punkt(1,2), 5));
        figures.add(new Quadrat(new Punkt(6,9), 8));

        System.out.println(figures);
    }
}

```

```

        System.out.println(berechneGesamtflaeche());
    }
}

Klasse: Kreis
package _01_30;

public class Kreis extends Form{

    private double radius;

    public Kreis(Punkt mittelpunkt, double radius) {
        super(mittelpunkt);
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    @Override
    public String toString() {
        return "Kreis [mittelpunkt=" + mittelpunkt + ", radius=" +
radius + "]";
    }

    public double berechneFlaeche() {
        return Math.round(Math.PI*radius*radius*100)/100.0;
    }

    public double berechneUmfang() {
        return Math.round(Math.PI*radius*200)/100.0;
    }

}package Immutable;

import java.util.ArrayList;
import java.util.List;

public final class Kunde {

    private final NameT name;
    private final List<String> bestellungen;

    public Kunde(NameT name, List<String> bestellungen) {
        this.name = new NameT(name);
    }
}

```

```

        this.bestellungen = new ArrayList<String>(bestellungen);
    }

    public List<String> getBestellungen() {
        return new ArrayList<String>(bestellungen);
    }

    public NameT getName() {
        return name;
    }

    public Kunde withName(NameT name) {
        return new Kunde(name, this.bestellungen);
    }

    public Kunde withBestellung(List<String> bestellungen) {
        return new Kunde(this.name, bestellungen);
    }
}

class NameT{

    private String vorname;
    private String nachname;
    public NameT(String vorname, String nachname) {
        this.vorname = vorname;
        this.nachname = nachname;
    }

    //Kopierkonstruktor
    public NameT(NameT other) {
        this.vorname=other.vorname;
        this.nachname= other.nachname;
    }
    public String getVorname() {
        return vorname;
    }
    public void setVorname(String vorname) {
        this.vorname = vorname;
    }
    public String getNachname() {
        return nachname;
    }
    public void setNachname(String nachname) {
        this.nachname = nachname;
    }
}

```

```
}
```

Klasse: Punkt

```
package _01_30;
```

```
public class Punkt{
    private double x;
    private double y;

    public Punkt(double x, double y) {

        this.x = x;
        this.y = y;
    }

    public double getX() {
        return x;
    }
    public void setX(double x) {
        this.x = x;
    }
    public double getY() {
        return y;
    }
    public void setY(double y) {
        this.y = y;
    }
    @Override
    public String toString() {
        return "Punkt [x=" + x + ", y=" + y + "]";
    }
}
```

```
}package _24_03_2026;
```

```
public final class PunktImmutable { //final Klassen können nicht
vererben
```

```
    private final double x;
    private final double y;
    public PunktImmutable(double x, double y) {
        this.x = x;
        this.y = y;
    }
    public double getX() {
        return x;
    }
    public double getY() {
```

```

        return y;
    }
    @Override
    public String toString() {
        return "PunktImmutable (" + x + "|" + y + ")";
    }

    public PunktImmutable withX(double xneu) {
        return new PunktImmutable(xneu,this.y);
    }
    public PunktImmutable withY(double yneu) {
        return new PunktImmutable(this.x,yneu);
    }
}

}

Klasse: Quadrat
package _01_30;
/**
 * Ein Quadrat ist ein Rechteck mit Länge = Breite
 *
 * TODO:
 *      - Definieren entsprechend die Klasse Quadrat
 */
public class Quadrat extends Rechteck{

    public Quadrat(Punkt mittelpunkt, double a) {
        super(mittelpunkt,a,a);
    }

    @Override
    public void setLaenge(double laenge) {
        super.setLaenge(laenge);
        super.setBreite(laenge);
    }
    @Override
    public void setBreite(double laenge) {
        super.setLaenge(laenge);
        super.setBreite(laenge);
    }
    public String toString() {
        return "Quadrat [Mittelpunkt=" + mittelpunkt + ",
Seitenlaenge=" + laenge + "]";
    }
}

}

```

Klasse: Task

```
package _01_30;
/**
 * es soll möglich sein
 *
 * Berechnung von:
 *     flächeninhalt und umfang für
 *     kreis, rechteck, quadrat
 */
```

```
public class Task {

}
```

Klasse: Test

```
package _01_30;

public class Test {
    public static void main(String[] args) {
        Punkt p  = new Punkt(0.33, 0.68);
        Punkt p2 = new Punkt(0.25, 0.5);
        Punkt p3 = new Punkt(0.2, 0.3);

        // Form f = new Form(p);
        Rechteck r = new Rechteck(p2, 3.0, 4.0);
        Kreis k = new Kreis(p3, 1.0);

        // System.out.println(f.getMittelpunkt());
        // f.setMittelpunkt(p3);

        //
        System.out.println(k.getMittelpunkt());
        k.setMittelpunkt(p);

        System.out.println(k);

        Quadrat q = new Quadrat(new Punkt(1.25, 0.75), 2.5);
        q.setBreite(4.5);
        System.out.println(q);
        System.out.println(k.berechneFlaeche());
        System.out.println(k.berechneUmfang());
        System.out.println(r.berechneFlaeche());
        System.out.println(r.berechneUmfang());
        System.out.println(q.berechneFlaeche());
        System.out.println(q.berechneUmfang());
    }
}
```

Klasse: Vererbung

```
package _01_30;
/**
 * Motivation:
 *             - Redundanz vermeiden
 *             - Klassen werden schmaler => Lesbarkeit
 *             - Abbildung der realen Welt
 *             - Beziehung zwischen Klassen (IS-A-Relationship)
 */
public class Vererbung {

}
```

Package: _01_30_selbst

Klasse: Form

```
package _01_30_selbst;

public class Form {

    protected Punkt mittelpunkt;//damit die Kinder Erben können, nicht
    private wählen

    public Form(Punkt mittelpunkt) {
        this.mittelpunkt = mittelpunkt;
    }

    public Punkt getMittelpunkt() {
        return mittelpunkt;
    }

    public void setMittelpunkt(Punkt mittelpunkt) {
        this.mittelpunkt = mittelpunkt;
    }

}
```

Klasse: Kreis

```
package _01_30_selbst;

public class Kreis extends Form{

    private double radius;
    public Kreis(Punkt mittelpunkt, double radius) {
        super(mittelpunkt);
    }

}
```

```

        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }
    public void setRadius(double radius) {
        this.radius = radius;
    }
    @Override
    public String toString() {
        return "Kreis [mittelpunkt=" + mittelpunkt + ", radius=" +
radius + "]";
    }

```

```

}package _03_11;

```

```

public class Kreis {

    private Punkt mp;
    private double radius;
    public Kreis(Punkt mp, double radius) {
        this.mp = mp;
        this.radius = radius;
    }

    // Konstruktoren

    // Validierung

    // Fehlerklasse definieren

    // nützliche Methoden

    public String toCSV() {
        return mp.toCSV() + ";" + radius;
    }

}

```

Klasse: Punkt

```

package _01_30_selbst;

```

```

public class Punkt{
    private double x;
    private double y;
    public Punkt(double x, double y) {

```



```

        super();
        this.x = x;
        this.y = y;
    }
    @Override
    public String toString() {
        return "Punkt [x=" + x + ", y=" + y + "]";
    }
    public double getX() {
        return x;
    }
    public void setX(double x) {
        this.x = x;
    }
    public double getY() {
        return y;
    }
    public void setY(double y) {
        this.y = y;
    }
}

```

```

}package _03_11;

```

```

public class Punkt {
    private double x;
    private double y;
    public Punkt(double x, double y) {
        this.x = x;
        this.y = y;
    }
    @Override
    public String toString() {
        return "Punkt [x=" + x + ", y=" + y + "]";
    }

    // Getters und setters wenn erforderlich
    // und weitere nützliche Methoden

    public String toCSV() {
        return x + ";" + y;
    }
}

```

Klasse: Quadrat

```

package _01_30_selbst;

```

```

public class Quadrat extends Rechteck {

    public Quadrat(Punkt mittelpunkt, double laenge) {
        super(mittelpunkt, laenge, laenge);
    }

}

```

Klasse: Rechteck

```

package _01_30_selbst;

```

```

public class Rechteck extends Form{

    private double laenge;
    private double breite;

    public Rechteck(Punkt mittelpunkt, double laenge, double breite) {
        super(mittelpunkt);
        this.laenge = laenge;
        this.breite = breite;
    }
    public double getLaenge() {
        return laenge;
    }
    public void setLaenge(double laenge) {
        this.laenge = laenge;
    }
    public double getBreite() {
        return breite;
    }
    public void setBreite(double breite) {
        this.breite = breite;
    }
    @Override
    public String toString() {
        return "Rechteck [mittelpunkt=" + mittelpunkt + ", laenge=" +
laenge + ", breite=" + breite + "];"
    }

}

```

Klasse: Test

```
package _01_30_selbst;

public class Test {

    public static void main(String[] args) {

        Punkt p = new Punkt(0.33, 0.68);
        Punkt p2 = new Punkt(0.25,0.5);
        Punkt p3 = new Punkt(0,0);
        Form f = new Form(new Punkt(1.5,3.1));
        Rechteck r = new Rechteck(p2,3,4);
        Kreis k = new Kreis(p3,5.0);

        System.out.println(f.getMittelpunkt());
        f.setMittelpunkt(p3);

        System.out.println(k.getMittelpunkt());
        k.setMittelpunkt(p2);

    }

}
```

Klasse: Vererbung

```
package _01_30_selbst;

/**
 * Motivation: Inheritance(Vererbung)
 * - Redundanz vermeiden
 * - einfachere Wartung
 * - Klassen werden schmäler => Lesbarkeit
 * - Abbildung der realen Welt
 * - Beziehung zwischen Klassen (Is-a-relationship)
 */

public class Vererbung {

}
```

Package: _02_03_2026

Klasse: App

```
package _02_03_2026;
```

```
public class App {
```

```
    public static void main(String[] args) {
```

```
        // Device d = new Server("HHH", "IP-XXX", 250, null, 4);
        Device[] devices = new Device[5];
```

```
        devices[0] = new Server("Server_01", "192.168.172.30", 2500,
null, 12); // Polymorphie
```

```
        // Ein Server ist ein Device: Upcast
        devices[1] = new Router("Router_01", "192.168.172.10", 50,
null, 130);
```

```
        devices[2] = new Workstation("Workstation_01", "192.168.172.20",
1800, null, "RTX 4060");
```

```
        devices[3] = new Server("Server_02", "192.168.172.40", 2800, new
MaintenanceContract("ACM", 29.5), 16);
```

```
        devices[4] = new Workstation("Workstation_02", "192.168.172.21",
1800, new MaintenanceContract("Telecom", 39.50), "RTX 4060");
```

```
        for (Device device : devices) {
            System.out.println(device.getInfo());
            System.out.println(device.getPowerUsage() + " Watt");
        }
```

```
        Device d = new Router("", "192.168.172.11", 150, null, 165);
```

```
        double pu = d.getPowerConsumption();
        System.out.println(d.getInfo());
```

```
        devices[1].setContract(new MaintenanceContract("ACM", 19.90));
```

```
        System.out.println("=====Geräte mit
Wartungsvertrag===== ");
```

```
        System.out.println("Anzahl der Geräte mit Wartungsvertrag:
"+ Utilities.countDevicesWithContract(devices));
```

```
        for (Device device : devices) {
            if (device.hasMaintenanceContract()) {
                System.out.println(device.getInfo() + "\n" +
device.getContract().toString());
            }
        }
```

```

        System.out.println("=====Total Power
Usage=====");
        System.out.println(Utilities.calculateTotalPower(devices)+
"Watt");
        System.out.println("Highest monthly cost:
"+Utilities.maxMonthlyCost(devices)+"€");

```

```

        if (Device.isError()){
            //....
        }
    }
}

```

```

}

```

Klasse: Device

```

package _02_03_2026;

```

```

public abstract class Device {

```

```

    protected final String hostname;
    protected String ipAddress;
    protected double powerConsumption;
    protected MaintenanceContract contract; //Optional
    public static int counter=1;

```

```

    private static boolean error = false;

```

```

    protected Device(String hostname, String ipAddress, double
powerConsumption, MaintenanceContract contract) {
        if(hostname.isBlank() || hostname==null ) {
            throw new IllegalArgumentException("fehlerhafte Eingabe:
"+hostname);

```

```

//            System.out.println("Invalid hostname");
//            this.hostname="Default-Host"+counter;
        }else
            this.hostname = hostname;
        if (ipAddress.isBlank() || ipAddress == null) {
            System.out.println("Invalid IP-Address");
            error =true;
            this.ipAddress= "169.254.1.1";
        }else
            this.ipAddress = ipAddress;

```

```

        if (powerConsumption<0) {
            System.out.println("Invalid power consumption");
            error = true;
        }
        else
            this.powerConsumption = powerConsumption;
        this.contract = contract;
        counter++;
    }

    public void setContract(MaintenanceContract contract) {
        this.contract = contract;
    }

    public String getHostname() {
        return hostname;
    }

    public String getIpAddress() {
        return ipAddress;
    }

    public double getPowerConsumption() {
        return powerConsumption;
    }

    public MaintenanceContract getContract() {
        return contract;
    }

    public static void setError(boolean error) {
        Device.error = error;
    }

    public static boolean isError(){
        return error;
    }

    public boolean hasMaintaenanceContract() {
        return contract!= null;
    }
    public String getInfo() {
        return "=".repeat(45)+ "\nHostname: "+hostname +";\nDevice
Typ: "+ getDeviceType()+";\nIP-Adresse: "+ipAddress+"\n"
+"="."repeat(45);
    }

    public abstract double getPowerUsage();
    public abstract String getDeviceType();

```

```
}
```

Klasse: Ladeger

```
package _02_03_2026;
```

```
public class Ladegerät {

    private Zustand zustand;
    private int ladestand;

    public Ladegerät(int ladestand) {
        this.zustand = NichtLadend.getNichtLadend();
        this.ladestand = ladestand;
    }

    public void setLadestand(int ladestand) {
        this.ladestand = ladestand;
    }

    public int getLadestand() {
        return ladestand;
    }

    public void setZustand(Zustand zustand) {
        this.zustand = zustand;
    }

    public void ausloesen() {
        zustand.bearbeiten(this);
    }

}
```

Klasse: Ladestation

```
package _02_03_2026;
```

```
//public class Ladestation {
//
//    public static void main(String[] args) {
//
//        Zustand[] zustaende = new Zustand[3];
//        zustaende[0]= NichtLadend.getNichtLadend();
//        zustaende[1]= NormalLadend.getNormalLadend();
//        zustaende[2]= SchnellLadend.getSchnellLadend();
//        //wegen Konstruktoren, die private sind, erzeugen wir über die
//        //Getter eine Referenz zu möglichen Objekten
//    }
//}
```

```

//
//
// }
//
// /**
//  * Die Klasse Ladegerät hält die polymorphe Referenz Zustand auf
//  die abstrakte Klasse Zustand . Dieser Referenz werden zur Laufzeit
//  konkrete Objekte der Klassen NichtLadend, NormalLadend bzw.
//  SchnellLadend zugewiesen . Das ist möglich, da die konkreten
//  Zustandsklassen von der abstrakten Klasse Zustand abgeleitet sind .
//  Je nachdem, welchen konkreten Zustand Zustand gerade referenziert,
//  wird mit Zustand.bearbeiten(this) immer das entsprechende
//  zustandsspezifische Verhalten ausgeführt (spätes/dynamisches Binden)
//.
//  */
//
//
//}

```

```

public class Ladestation {

    public static void main(String[] args) {
        // Ladegerät startet im Zustand NichtLadend mit 10%
        Ladegerät ladegerät = new Ladegerät(10);

        System.out.println("Start-Ladestand: " +
ladegerät.getLadestand() + "%");
        System.out.println("--- Zyklus 1: ausloesen() ---");
        ladegerät.ausloesen(); // NichtLadend.bearbeiten

        // Wir erhöhen den Ladestand von außen, z.B. simuliert
        ladegerät.setLadestand(30);
        System.out.println("\nLadestand manuell gesetzt auf: " +
ladegerät.getLadestand() + "%");
        System.out.println("--- Zyklus 2: ausloesen() ---");
        ladegerät.ausloesen(); // NormalLadend.bearbeiten oder
weiterhin NichtLadend, je nach Logik

        System.out.println("\n--- Zyklus 3: ausloesen() ---");
        ladegerät.ausloesen(); // nächster Schritt im aktuellen
Zustand

        System.out.println("\nEnd-Ladestand: " +
ladegerät.getLadestand() + "%");
    }
}

```

Klasse: MaintenanceContract

package _02_03_2026;


```

public class MaintenanceContract {

    private String provider;
    private double monthlyCost;
    public MaintenanceContract(String provider, double monthlyCost) {
        this.provider = provider;
        this.monthlyCost = monthlyCost;
    }
    public String getProvider() {
        return provider;
    }
    public double getMonthlyCost() {
        return monthlyCost;
    }
    @Override
    public String toString() {
        return provider + "; " + monthlyCost + "€/Month";
    }
}

} package HaushaltskassenApp_zwei;

import javafx.geometry.Insets;
import javafx.geometry.Pos;
import javafx.scene.control.*;
import javafx.scene.layout.*;
import javafx.scene.text.Font;
import javafx.scene.text.FontWeight;

/**
 * Controller für das Hauptfenster der Anwendung
 *
 * Diese Klasse verwaltet die Tab-Navigation und koordiniert die
 verschiedenen
 * Bereiche der Anwendung (Einträge, Feste Werte, Übersichten, etc.)
 */
public class MainWindowController {

    private DataModel dataModel;
    private BorderPane root;
    private TabPane tabPane;
    private Label statusLabel;

    // Tab-Controller
    private EintraegeTabController eintraegeTab;

```

```

private FesteWerteTabController festeWerteTab;
private UebersichtTabController uebersichtTab;
private StatistikTabController statistikTab;
private BudgetTabController budgetTab;
private SparzielTabController sparzielTab;
private SonstigesTabController sonstigesTab;

public MainWindowController(DataModel dataModel) {
    this.dataModel = dataModel;

    // Status-Callback setzen
    dataModel.setStatusCallback(message -> {
        if (statusLabel != null) {
            javafx.application.Platform.runLater(() -> {
                statusLabel.setText(message);
            });
        }
    });

    createUI();
}

private void createUI() {
    root = new BorderPane();

    // Header
    VBox header = createHeader();
    root.setTop(header);

    // Tab-Bereich
    tabPane = new TabPane();

    tabPane.setTabClosingPolicy(TabPane.TabClosingPolicy.UNAVAILABLE);

    // Tabs erstellen
    eintraegeTab = new EintraegeTabController(dataModel);
    festeWerteTab = new FesteWerteTabController(dataModel);
    uebersichtTab = new UebersichtTabController(dataModel);
    statistikTab = new StatistikTabController(dataModel);
    budgetTab = new BudgetTabController(dataModel);
    sparzielTab = new SparzielTabController(dataModel);
    sonstigesTab = new SonstigesTabController(dataModel);

    tabPane.getTabs().addAll(
        new Tab("Einträge", eintraegeTab.getContent()),
        new Tab("Feste Werte", festeWerteTab.getContent()),
        new Tab("Übersicht", uebersichtTab.getContent()),
        new Tab("Statistik", statistikTab.getContent()),
        new Tab("Budget", budgetTab.getContent()),
        new Tab("Sparziele", sparzielTab.getContent()),

```

```

        new Tab("Sonstiges", sonstigesTab.getContent())
    );

    root.setCenter(tabPane);

    // Statuszeile
    statusLabel = new Label("Bereit");
    statusLabel.setPadding(new Insets(5, 10, 5, 10));
    statusLabel.setStyle("-fx-background-color: #e0e0e0;");
    root.setBottom(statusLabel);
}

private VBox createHeader() {
    VBox header = new VBox(10);
    header.setPadding(new Insets(15));
    header.setStyle("-fx-background-color: #2c3e50;");

    Label title = new Label("Haushaltskassen-App");
    title.setFont(Font.font("Arial", FontWeight.BOLD, 24));
    title.setStyle("-fx-text-fill: white;");

    HBox buttonBar = new HBox(10);
    buttonBar.setAlignment(Pos.CENTER_RIGHT);

    Button speichernBtn = new Button("Speichern");
    speichernBtn.setOnAction(e -> {
        dataModel.speichereAlleDaten();
        showInfo("Daten gespeichert");
    });

    Button aktualisierenBtn = new Button("Aktualisieren");
    aktualisierenBtn.setOnAction(e -> {
        dataModel.ladeAlleDaten();
        refreshAllTabs();
        showInfo("Daten aktualisiert");
    });

    buttonBar.getChildren().addAll(speichernBtn,
aktualisierenBtn);

    header.getChildren().addAll(title, buttonBar);
    return header;
}

private void refreshAllTabs() {
    eintraegeTab.refresh();
    festeWerteTab.refresh();
    uebersichtTab.refresh();
    statistikTab.refresh();
    budgetTab.refresh();
}

```

```

        sparzielTab.refresh();
        sonstigesTab.refresh();
    }

    private void showInfo(String message) {
        Alert alert = new Alert(Alert.AlertType.INFORMATION);
        alert.setTitle("Information");
        alert.setHeaderText(null);
        alert.setContentText(message);
        alert.showAndWait();
    }

    public BorderPane getRoot() {
        return root;
    }
}

Klasse: NichtLadend
package _02_03_2026;

public class NichtLadend extends Zustand {

    private static NichtLadend nichtLadend;

    private NichtLadend() {
        super();
    }

    // @Override
    // public void bearbeiten(Ladegerät ladegerät) {
    //     if (ladegerät.getLadestand() >= 20 &&
    ladegerät.getLadestand() < 100) {
    //         ladegerät.setZustand(NormalLadend.getNormalLadend());
    //     }
    // }

    public static NichtLadend getNichtLadend() {
        if (nichtLadend == null) { // **hier wird erzeugt**
            nichtLadend = new NichtLadend();
        }
        return nichtLadend;
    }

    @Override
    public void bearbeiten(Ladegerät ladegerät) {
        System.out.println("Zustand: NichtLadend");
        if (ladegerät.getLadestand() >= 20 && ladegerät.getLadestand()
< 100) {
            ladegerät.setZustand(NormalLadend.getNormalLadend());

```

```

        System.out.println("-> Wechsel zu NormalLadend");
    }
}

Klasse: NormalLadend
package _02_03_2026;

public class NormalLadend extends Zustand {

    private NormalLadend() {
        super();
    }

    public static void setNormalLadend(NormalLadend normalLadend) {
        NormalLadend.normalLadend = normalLadend;
    }

    private static NormalLadend normalLadend;

    // @Override
    // public void bearbeiten(Ladegerät ladegerät) {
    //     ladegerät.setLadestand(ladegerät.getLadestand()+10);
    // }

    public static NormalLadend getNormalLadend() {
        if (normalLadend == null) {
            normalLadend = new NormalLadend();
        }
        return normalLadend;
    }

    @Override
    public void bearbeiten(Ladegerät ladegerät) {
        System.out.println("Zustand: NormalLadend");
        ladegerät.setLadestand(ladegerät.getLadestand() + 10);
        System.out.println("Ladestand nach Normal-Laden: " +
ladegerät.getLadestand() + "%");

        // Beispiel: ab 80% auf SchnellLadend wechseln
        if (ladegerät.getLadestand() >= 80 && ladegerät.getLadestand()
< 100) {
            ladegerät.setZustand(SchnellLadend.getSchnellLadend());
            System.out.println("-> Wechsel zu SchnellLadend");
        }
    }
}

```

```
}
```

Klasse: Router

```
package _02_03_2026;
```

```
public class Router extends Device{

    private int throughputMbps;

    public Router(String hostname, String ipAddress, double
powerConsumption, MaintenanceContract contract,
        int throughputMbps) {
        super(hostname, ipAddress, powerConsumption, contract);
        this.throughputMbps = throughputMbps;
    }

    @Override
    public double getPowerUsage() {

        return 1.1* powerConsumption;
    }

    @Override
    public String getDeviceType() {

        return "Router";
    }

}
```

Klasse: SchnellLadend

```
package _02_03_2026;
```

```
public class SchnellLadend extends Zustand {

    private static SchnellLadend schnellLadend;

    private SchnellLadend() {
        super();
    }

    public static SchnellLadend getSchnellLadend() {
        if (schnellLadend == null) {
            schnellLadend = new SchnellLadend();
        }
        return schnellLadend;
    }

}
```

```

    }

    // @Override
    // public void bearbeiten(Ladegerät ladegerät) {
    //     ladegerät.setLadestand(ladegerät.getLadestand()+20);
    // }
    //
    @Override
    public void bearbeiten(Ladegerät ladegerät) {
        System.out.println("Zustand: SchnellLadend");
        ladegerät.setLadestand(ladegerät.getLadestand() + 20);
        System.out.println("Ladestand nach Schnell-Laden: " +
        ladegerät.getLadestand() + "%");

        // Beispiel: ab 100% wieder NichtLadend
        if (ladegerät.getLadestand() >= 100) {
            ladegerät.setZustand(NichtLadend.getNichtLadend());
            System.out.println("-> Wechsel zu NichtLadend (voll
            geladen)");
        }
    }
}

```

Klasse: Server

```
package _02_03_2026;
```

```

public class Server extends Device {

    private final int cpuCores;

    public Server(String hostname, String ipAddress, double
    powerConsumption, MaintenanceContract contract,
        int cpuCores) {
        super(hostname, ipAddress, powerConsumption, contract);
        this.cpuCores = cpuCores;
    }

    @Override
    public double getPowerUsage() {

        return 1.2*powerConsumption;
    }

    @Override
    public String getDeviceType() {

        return "Server";
    }
}

```

```
}
```

Klasse: Utilities

```
package _02_03_2026;
```

```
import _02_03_2026.Device;
```

```
public class Utilities {  
    public static double calculateTotalPower(Device[] devices) {  
        double totalPower=0.0;  
        for (Device device: devices) {  
            totalPower +=device.getPowerUsage();  
        }  
        return totalPower;  
    }  
  
    public static int countDevicesWithContract(Device[] devices) {  
        int counter=0;  
        for (Device device: devices) {  
            if (device.hasMaintaenanceContract()) {  
                counter++;  
            }  
        }  
        return counter;  
    }  
  
    public static double maxMonthlyCost(Device[] dev) {  
        double max=0.0;  
        for (Device d:dev) {  
            if (d.hasMaintaenanceContract())  
                max=max>d.getContract().getMonthlyCost()?  
max:d.getContract().getMonthlyCost();  
        }  
        return max;  
    }  
}
```

Klasse: Workstation

```
package _02_03_2026;
```

```
public class Workstation extends Device{  
    private final String gpuModel ;
```



```

        public Workstation(String hostname, String ipAddress, double
powerConsumption, MaintenanceContract contract,
        String gpuModel) {
            super(hostname, ipAddress, powerConsumption, contract);
            this.gpuModel = gpuModel;
        }

        @Override
        public double getPowerUsage() {

            return 0.9*powerConsumption;
        }

        @Override
        public String getDeviceType() {

            return "Workstation";
        }

    }

```

Klasse: Zustand

```

package _02_03_2026;

public abstract class Zustand {

    public abstract void  bearbeiten(Ladegerät ladegerät);

}

```

Package: _03_03

Klasse: Ladegeraet

```

package _03_03;
/**
 * Polymorphie:
 *
 *          this.zustand = NichtLadend.getNichtLadend();
 *
 * Das Attribut zustand ist deklariert allgemein als Zustand (die
Basisklasse), referenziert aber auf Objekte der Kindklassen
 * wie zum Beispiel auf NichtLadend-, Normalladend- oder
SchnellLadend-Objekt.
 */

```

```

class Ladegeraet{
    private Zustand zustand;
    private int ladestand;

    public Ladegeraet() {
        // Initialisiere das Attribut zustand mit einem NichtLadend-
Objekt

        //this.zustand = new NichtLadend(); ---- Error: Konstruktor
ist private

        this.zustand = NichtLadend.getNichtLadend();// --- Ok
        //setZustand(new NichtLadend()); ---- error: Konstruktor ist
private

        //setZustand(NichtLadend.getNichtLadend()); // --- Ok
    }

    public void setZustand(Zustand zustand) {
        this.zustand = zustand;
    }

    public int getLadestand() {
        return ladestand;
    }

    public void ausloesen() {
        zustand.bearbeiten(this);
    }
}package _03_03_2026;

```

```

public class Ladegerät {

    private Zustand zustand;
    private int ladestand;

    public Ladegerät(int ladestand) {
        this.zustand = NichtLadend.getNichtLadend();//oder
setZustand(NichtLadend.getNichtLadend)
        this.ladestand = ladestand;
    }

    public void setLadestand(int ladestand) {
        this.ladestand = ladestand;
    }

    public int getLadestand() {
        return ladestand;
    }
}

```

```

        public void setZustand(Zustand zustand) {
            this.zustand = zustand;
        }

        public void ausloesen() {
            zustand.bearbeiten(this);
        }
    }

Klasse: NichtLadend
package _03_03;

public class NichtLadend extends Zustand{
    private static NichtLadend nichtLadend;

    private NichtLadend() {
        //...
    }

    @Override
    public void bearbeiten(Ladegeraet ladegeraet) {

        int x = ladegeraet.getLadestand();

        if(x >= 20 && x < 100) {
            NormalLadend z = NormalLadend.getNormalLadend();

            ladegeraet.setZustand(z);

        }

    }

    public static NichtLadend getNichtLadend() {
        //return nichtLadend;
        return new NichtLadend();
    }
}

package _03_03_2026;

public class NichtLadend extends Zustand {

    private static NichtLadend nichtLadend;

    private NichtLadend() {
        super();
    }
}

```

```

    }

    // @Override
    // public void bearbeiten(Ladegerät ladegerät) {
    //     if (ladegerät.getLadestand() >= 20 &&
    ladegerät.getLadestand() < 100) {
    //         ladegerät.setZustand(NormalLadend.getNormalLadend());
    //     }
    // }

    public static NichtLadend getNichtLadend() {
        if (nichtLadend == null) { // **hier wird erzeugt**
            nichtLadend = new NichtLadend();
        }
        return nichtLadend;
    }

    @Override
    public void bearbeiten(Ladegerät ladegerät) {
        System.out.println("Zustand: NichtLadend");
        if (ladegerät.getLadestand() >= 20 && ladegerät.getLadestand()
    < 100) {
            ladegerät.setZustand(NormalLadend.getNormalLadend());
            System.out.println("-> Wechsel zu NormalLadend");
        }
    }
}

```

Klasse: NormalLadend

```
package _03_03;
```

```

public class NormalLadend extends Zustand{

    private static NormalLadend normalLadend;

    private NormalLadend() {
        //...
    }

    @Override
    public void bearbeiten(Ladegeraet ladegeraet) {
        // TODO Auto-generated method stub
    }
}

```

```

        public static NormalLadend getNormalLadend() {
            //return normalLadend;

            return new NormalLadend();
        }
    }
}

```

Klasse: Person

```
package _03_03;
```

```

public class Person {
    private String name;
    private int geburtsjahr;

    private Person mutter;

    public Person(String name, int geburtsjahr, Person mutter) {

        this.name = name;
        this.geburtsjahr = geburtsjahr;
        this.mutter = mutter;
    }

}

class Test{
    public static void main(String[] args) {
        Person p = new Person("Bob", 1945, new Person("Maria", 1927,
new Person("Sarah", 1905, null)));
    }
}package _03_03_2026;

import java.time.LocalDate;

public class Person {
    private String name;
    private Person mutter;//rekursiv
    private LocalDate geburtsdatum;
    public Person(String name, Person mutter, LocalDate geburtsdatum)
{
    super();
    this.name = name;
    this.mutter = mutter;
}
}

```

```

        this.geburtsdatum = geburtsdatum;
    }
    @Override
    public String toString() {
        return "Person [name=" + name + ", mutter=" + mutter + ",
geburtsdatum=" + geburtsdatum + "]";
    }

```

```

}

```

```

class Test{

    public static void main(String[] args) {
        Person p = new Person("Bob",new Person("Maria", new
Person("Sarah", null, LocalDate.of(1938, 1, 23)), LocalDate.of(1978,
12, 3)),LocalDate.now());

        System.out.println(p);

    }

}

```

```

}package _03_27_2026;

```

```

import java.io.Serializable;
import java.time.LocalDate;

```

```

class Konto /*implements Serializable*/{
    private String iban;
    private double kontostand;
    public Konto(String iban, double kontostand) {
        super();
        this.iban = iban;
        this.kontostand = kontostand;
    }
    @Override
    public String toString() {
        return "Konto [iban=" + iban + ", kontostand=" + kontostand +
"]";
    }
}

```

```

}

```

```

public class Person implements Serializable{

    private final String name;
    private final LocalDate geburtsdatum;
    private transient Konto account; //transient schlagwort um von der
    serialisation ausgeschlossen zu werden
    private transient String passwort;

    public Person(String name, LocalDate geburtsdatum, Konto
    account,String passwort) {
        super();
        this.name = name;
        this.geburtsdatum = geburtsdatum;
        this.account=account;
        this.passwort=passwort;
    }

    @Override
    public String toString() {
        return "Person [name=" + name + ", geburtsdatum=" +
    geburtsdatum + ", account=" + account + ", passwort="
        + passwort + "]";
    }

    public String getName() {
        return name;
    }
    public LocalDate getGeburtsdatum() {
        return geburtsdatum;
    }

}

```

Klasse: SchnellLadend

```

package _03_03;

public class SchnellLadend extends Zustand{

    private static SchnellLadend schnellLadend;

    private SchnellLadend() {
        //TODO
    }
}

```

```

        @Override
        public void bearbeiten(Ladegeraet ladegeraet) {
            // TODO Auto-generated method stub

        }

        public static SchnellLadend getSchnellLadend() {
            //TODO
            return new SchnellLadend();
        }

    }

Klasse: TeeEtickett
package _03_03;

import java.time.LocalDate;

public abstract class TeeEtickett {
    protected String name;
    protected LocalDate verfallsDatum;
    protected String[][] zutaten;

    public void berechneVerfallsDatum() {
        //TODO
    }

    public void druckeEtickett() {
        //TODO
    }

    public abstract double getPreis();
}

class KoellnerBronchilaTeeEtickett extends TeeEtickett{
    public KoellnerBronchilaTeeEtickett() {
        //TODO
    }

    //@Override
    public double getPreis() {
        // TODO Auto-generated method stub
        return 1.50;
    }
}

```



```

}

class KoellnerMagenTeeEtickett extends TeeEtickett{
    public KoellnerMagenTeeEtickett() {
        //TODO
    }

    //@Override
    public double getPreis() {
        // TODO Auto-generated method stub
        return 1.75;
    }
}

class MuenchnerMagenTeeEtickett extends TeeEtickett{
    public MuenchnerMagenTeeEtickett() {
        //TODO
    }

    //@Override
    public double getPreis() {
        // TODO Auto-generated method stub
        return 1.75;
    }
}

class MuenchnerBronchialTeeEtickett extends TeeEtickett{
    public MuenchnerBronchialTeeEtickett() {
        //TODO
    }

    //@Override
    public double getPreis() {
        // TODO Auto-generated method stub
        return 1.75;
    }
}

```

Klasse: TestEtickett

```
package _03_03;
```

```

public class TestEtickett {
    public static void main(String[] args) {
        TeeEtickett t = new KoellnerBronchilaTeeEtickett(); // new
        TeeEtickett();

        double preis = t.getPreis();

        System.out.println(preis);

        t = new KoellnerMagenTeeEtickett();
    }
}

```

```

        preis = t.getPreis();
        System.out.println(preis);
    }
}

```

Klasse: TestFactoryDesign

```

package _03_03;

import java.time.LocalDate;

// GoF (Factory Methode)

public class TestFactoryDesign {
    public static void main(String[] args) {

        LocalDate birthdayPatrik = LocalDate.of(1988, 3, 27);

        // Vergleich: NichtLadend nl = NichtLadend.getNichtLadend();

        System.out.println(birthdayPatrik);

        Uhrzeit u = Uhrzeit.create(10, 55, 45);

        System.out.println(u);
        Uhrzeit u2 = Uhrzeit.create(10, 55, 75);
        System.out.println(u2);
    }

}package NorthwindApp.GUI;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JTextField;

public class TestGUI {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        JFrame win = new JFrame();
        //Fenster initialisieren
        win.setSize(300, 200);
        win.setLocation(300, 300);
        win.setTitle("Testfenster");
        win.setResizable(false);
    }
}

```

```

        win.setLayout(null);
        win.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        //Komponenten ins Fenster
        JLabel label=new JLabel("Text");
        label.setBounds(10, 10, 80, 25);
        win.add(label);
        //Textfeld
        JTextField text =new JTextField();
        text.setBounds(100, 10, 180, 25);
        win.add(text);
        //button
        JButton button =new JButton("klick mich");
        // button.addActionListener(new ButtonClick());
        button.addActionListener(new ActionListener() {

            @Override
            public void actionPerformed(ActionEvent e) {
                // TODO Auto-generated method stub
                System.out.println(text.getText());
            }
        });
        button.setBounds(180, 40, 100, 25);
        win.add(button);

        win.setVisible(true);
    }

}

class ButtonClick implements ActionListener{
    @Override
    public void actionPerformed(ActionEvent e) {
        // TODO Auto-generated method stub
        System.out.println("ich wurde geklickt");
    }
}

```

Klasse: Uhrzeit

```

package _03_03;

/**
 *
 */
class Uhrzeit{
    private int stunden;
    private int min;
    private int sec;
}

```

```

private Uhrzeit(int stunden, int min, int sec) {

    this.stunden = stunden;
    this.min = min;
    this.sec = sec;
}

public static Uhrzeit create(int s, int m, int se) {
    if(s < 0 || s > 23)
        throw new IllegalArgumentException("invalid hour: " + s);
    if(m < 0 || m > 59)
        throw new IllegalArgumentException("invalid minute: " +
m);
    if(se < 0 || se > 59)
        throw new IllegalArgumentException("invalid second: " +
se);

    return new Uhrzeit(s, m, se);
}

@Override
public String toString() {
    return stunden + ":" + min + ":" + sec;
}

/**
 * Konstruktor
 *
 * Factory Methode
 *
 * toString
 */

}package Dienstag2;

public class Umrechnung {

    void showTime(int zeit) {
        int stunde= zeit/3600;
        int rest1 = zeit%3600;
        int minute = rest1/60;
        int sec= rest1%60;

        System.out.println(zeit+" Sekunden sind:\n"+stunde+" Stunden "
+minute +" Minuten "+sec+ " Sekunden");
    }
}

```

```

void rechneSekundenum(int zeit) {
    int stunde=0;
    int minute =0;
    int temp=zeit;

    while (temp>=3600){
        temp=temp-3600 ;
        stunde=++stunde;
    }
    while (temp>=60) {
        temp=temp-60;
        minute=++minute;
    }

    System.out.println(zeit+" Sekunden sind:\n"+stunde+ "Stunden "
+ minute+" Minuten "+temp +" Sekunden");
}

}

```

Klasse: Zustand

```

package _03_03;

public abstract class Zustand {
    public abstract void bearbeiten(Ladegeraet ladegeraet);
}package _03_03_2026;

public abstract class Zustand {

    public abstract void  bearbeiten(Ladegerät ladegerät);

}

```

Package: _03_03_2026

Klasse: DT

```

package _03_03_2026;

import java.time.DayOfWeek;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.time.LocalDateTime;

```

```

public class DT {

    public static void main(String[] args) {
        LocalDate birthdayPatrick = LocalDate.of(1988, 3, 27);
        LocalDate today = LocalDate.now();
        long daysPatrick = birthdayPatrick.until(LocalDate.now(),
ChronoUnit.DAYS);

        System.out.println(birthdayPatrick);
        System.out.println(daysPatrick+ " Tage seit
"+birthdayPatrick);

        DayOfWeek dow = LocalDateTime.now().getDayOfWeek();
        System.out.println(dow);

        int nano = LocalDateTime.now().getNano();
        System.out.println(nano);

        Uhrzeit u = Uhrzeit.getUhrzeit(10, 55, 45);
        System.out.println(u);

        Uhrzeit u2 =Uhrzeit.getUhrzeit(12, 75, 13);
        System.out.println(u2);

    }

}

```

```

class Uhrzeit{
    private int stunden;
    private int min;
    private int sec;

    private Uhrzeit(int stunden, int min, int sec) {
        super();
        this.stunden = stunden;
        this.min = min;
        this.sec = sec;
    }

    @Override
    public String toString() {
        return String.format("Uhrzeit: %02d:%02d:
%02d",stunden,min,sec);
    }
}

```



```
// /**
//  * Die Klasse Ladegerät hält die polymorphe Referenz Zustand auf
//  die abstrakte Klasse Zustand . Dieser Referenz werden zur Laufzeit
//  konkrete Objekte der Klassen NichtLadend, NormalLadend bzw.
//  SchnellLadend zugewiesen . Das ist möglich, da die konkreten
//  Zustandsklassen von der abstrakten Klasse Zustand abgeleitet sind .
//  Je nachdem, welchen konkreten Zustand Zustand gerade referenziert,
//  wird mit Zustand.bearbeiten(this) immer das entsprechende
//  zustandsspezifische Verhalten ausgeführt (spätes/dynamisches Binden)
//.
//  */
//
//
//}
```

```
public class Ladestation {

    public static void main(String[] args) {
        // Ladegerät startet im Zustand NichtLadend mit 10%
        Ladegerät ladegerät = new Ladegerät(60);

        System.out.println("Start-Ladestand: " +
ladegerät.getLadestand() + "%");
        System.out.println("--- Zyklus 1: ausloesen() ---");
        ladegerät.ausloesen(); // NichtLadend.bearbeiten

        // Wir erhöhen den Ladestand von außen, z.B. simuliert
        ladegerät.setLadestand(70);
        System.out.println("\nLadestand manuell gesetzt auf: " +
ladegerät.getLadestand() + "%");
        System.out.println("--- Zyklus 2: ausloesen() ---");
        ladegerät.ausloesen(); // NormalLadend.bearbeiten oder
weiterhin NichtLadend, je nach Logik

        System.out.println("\n--- Zyklus 3: ausloesen() ---");
        ladegerät.ausloesen(); // nächster Schritt im aktuellen
Zustand

        System.out.println("\nEnd-Ladestand: " +
ladegerät.getLadestand() + "%");
    }
}
```

Klasse: NormalLadend

```
package _03_03_2026;
```

```
public class NormalLadend extends Zustand {

    private NormalLadend() {
```



```

        super();
    }

    public static void setNormalLadend(NormalLadend normalLadend) {
        NormalLadend.normalLadend = normalLadend;
    }

    private static NormalLadend normalLadend;

    // @Override
    // public void bearbeiten(Ladegerät ladegerät) {
    //     ladegerät.setLadestand(ladegerät.getLadestand()+10);
    // }

    public static NormalLadend getNormalLadend() {
        if (normalLadend == null) {
            normalLadend = new NormalLadend();
        }
        return normalLadend;
    }

    @Override
    public void bearbeiten(Ladegerät ladegerät) {
        System.out.println("Zustand: NormalLadend");
        ladegerät.setLadestand(ladegerät.getLadestand() + 10);
        System.out.println("Ladestand nach Normal-Laden: " +
ladegerät.getLadestand() + "%");

        // Beispiel: ab 80% auf SchnellLadend wechseln
        if (ladegerät.getLadestand() >= 80 && ladegerät.getLadestand()
< 100) {
            ladegerät.setZustand(SchnellLadend.getSchnellLadend());
            System.out.println("-> Wechsel zu SchnellLadend");
        }
    }
}

```

Klasse: SchnellLadend

```
package _03_03_2026;
```

```

public class SchnellLadend extends Zustand {

    private static SchnellLadend schnellLadend;

    private SchnellLadend() {
        super();
    }
}

```

```

        public static SchnellLadend getSchnellLadend() {
            if (schnellLadend == null) {
                schnellLadend = new SchnellLadend();
            }
            return schnellLadend;
        }

// @Override
// public void bearbeiten(Ladegerät ladegerät) {
//     ladegerät.setLadestand(ladegerät.getLadestand()+20);
// }
// }
//
@Override
public void bearbeiten(Ladegerät ladegerät) {
    System.out.println("Zustand: SchnellLadend");
    ladegerät.setLadestand(ladegerät.getLadestand() + 20);
    System.out.println("Ladestand nach Schnell-Laden: " +
        ladegerät.getLadestand() + "%");

    // Beispiel: ab 100% wieder NichtLadend
    if (ladegerät.getLadestand() >= 100) {
        ladegerät.setZustand(NichtLadend.getNichtLadend());
        System.out.println("-> Wechsel zu NichtLadend (voll
geladen)");
    }
}

}

```

Package: [_03_05.ap2_2021](#)

Klasse: Artikel

```
package _03_05.ap2_2021;
```

```

public class Artikel {
    private final String name;
    private double preis;

    public Artikel(String name, double preis) {
        this.name = name;
        this.preis = preis;
    }

    public double getPreis() {
        return preis;
    }
}

```

```

        public void setPreis(double preis) {
            this.preis = preis;
        }

        public String getName() {
            return name;
        }

    }

```

Klasse: ExpressLieferung

```

package _03_05.ap2_2021;

public class ExpressLieferung implements Versand {

    @Override
    public double berechnen(Warenkorb w) {

        return 0.1*w.berechneGesamtwert();
    }

}

```

Klasse: OnlineShopApp

```

package _03_05.ap2_2021;

public class OnlineShopApp {

    public static void main(String[] args) {
        Warenkorb w = new Warenkorb();
        w.artikelHinzufuegen("Lenovo-RTX", 899.9, 5);
        w.artikelHinzufuegen("Monitor-XYZ", 299.9, 3);

        w.setVersand(new ExpressLieferung());

        System.out.println(w.berechneGesamtwert());
        System.out.println(w.getVersandkosten());
    }

}

```

Klasse: StandardLieferung

```

package _03_05.ap2_2021;

public class StandardLieferung implements Versand {

```

```

@Override
public double berechnen(Warenkorb w) {

    return 0.05*w.berechneGesamtwert();
}

}

```

Klasse: Versand

```

package _03_05.ap2_2021;

public interface Versand {
    double berechnen(Warenkorb w);
}

```

Klasse: Warenkorb

```

package _03_05.ap2_2021;

import java.util.ArrayList;

public class Warenkorb {

    private ArrayList<Artikel> artikelList = new ArrayList<Artikel>();
    // Leere Liste
    private Versand versand;

    public void artikelHinzufuegen(String name, double preis, int
stueckzahl) {
        for (int i = 0; i < stueckzahl; i++)
            artikelList.add(new Artikel(name, preis));
    }

    public double berechneGesamtwert() {
        double gesamtwert = 0.0;
        for (Artikel artikel : artikelList) {
            gesamtwert += artikel.getPreis();
        }
        return gesamtwert;
    }

    public void setVersand(Versand versand) {
        this.versand=versand;
    }

    public double getVersandkosten() {
        double versandkosten=0.0;
        versandkosten=versand.berechnen(this);
        return versandkosten;
    }
}

```

```
}
```

Package: _03_06_logistik

Klasse: App

```
package _03_06_logistik;
```

```
import java.util.ArrayList;
```

```
public class App {
    public static void main(String[] args) {

        ArrayList<Shipment>shipments = new ArrayList<Shipment>();
        shipments.add( new StandardParcel(2.0, 120));
        shipments.add(new ExpressParcel(1.2, 80, 250));
        shipments.add(new Freight(120, 450, 5000));

        double total = 0;
        for (Shipment s : shipments) {
            System.out.println(s.info());
            total += s.shippingCostEur();

            if (s instanceof Trackable t) {
                System.out.println("  Tracking: " + t.trackingCode());
            }
            if (s instanceof Insurable i) {
                System.out.println("  Insurance: " +
i.insuranceCostEur() + "€");
            }
        }

        System.out.println("TOTAL: " + total + "€");
    }
}package _03_11;
```

```
import java.io.FileNotFoundException;
import java.util.ArrayList;
import java.util.Arrays;
```

```
public class App {
    public static void main(String[] args) {
        // Service und Punkt Klasse verwenden...

        // ArrayList<Punkt> pts = new ArrayList<Punkt>(Arrays.asList(
        //         new Punkt(0.5, 0.25),
```

```

//          new Punkt(0.66, 0.89),
//          new Punkt(0.33, 0.77)
//      ));
//
//  ArrayList<Kreis> cls = new ArrayList<Kreis>(
//      Arrays.asList(
//          new Kreis(new Punkt(0, .33), 1.0),
//          new Kreis(new Punkt(0, 0.5), 1.25),
//          new Kreis(new Punkt(0.25, 0), 1.5)
//      )
//  );
//
//  Service.writePointsToCSV(pts, "points.csv");
//  Service.writeCirclesToCSV(cls, "circles.csv");
//
//  System.out.println("Programm terminated...");

```

```

    ArrayList<Punkt> pointsReadFromFile
=Service.readPointsFromCSV("Punkte_im_Csv_Format.csv");

    for( Punkt p :pointsReadFromFile) {
        System.out.println(p);
    }
}
}

```

Klasse: ExpressParcel

```
package _03_06_logistik;
```

```
public class ExpressParcel extends Shipment implements Trackable,
Insurable {
```

```
    private final double goodsValueEur;
```

```
    public ExpressParcel(double weightKg, double distanceKm, double
goodsValueEur) {
        super(weightKg, distanceKm);
        if (goodsValueEur < 0) throw new
IllegalArgumentException("goodsValueEur >= 0");
        this.goodsValueEur = goodsValueEur;
    }

```

```
@Override
```

```
public double shippingCostEur() {
    double base = 9.99;
    double weightPart = 1.10 * getWeightKg();
    double distancePart = 0.02 * getDistanceKm();
    return round2(base + weightPart + distancePart +

```

```

insuranceCostEur());
    }

    @Override
    public double insuranceCostEur() {
        double cost = goodsValueEur * 0.01;
        return round2(Math.max(2.50, cost));
    }

    @Override
    public String trackingCode() {
        return "EXP-" + Integer.toHexString(hashCode()).toUpperCase();
    }

    private static double round2(double v) {
        return Math.round(v * 100.0) / 100.0;
    }
}package _03_27;

public class ExpressPricing implements PricingStrategie{

    @Override
    public double calculatePrice(double weight, double distance) {

        return 10 + weight + distance * 0.2;
    }

}

```

Klasse: Freight

```

package _03_06_logistik;

public class Freight extends Shipment implements Insurable {

    private final double goodsValueEur;

    public Freight(double weightKg, double distanceKm, double
goodsValueEur) {
        super(weightKg, distanceKm);
        if (goodsValueEur < 0) throw new
IllegalArgumentException("goodsValueEur >= 0");
        this.goodsValueEur = goodsValueEur;
    }

    @Override
    public double shippingCostEur() {
        double base = 49.0;
        double weightPart = 0.35 * getWeightKg();
        double distancePart = 0.015 * getDistanceKm();
    }
}

```

```

        return round2(base + weightPart + distancePart +
insuranceCostEur());
    }

```

```

@Override
public double insuranceCostEur() {
    double cost = goodsValueEur * 0.005;
    return round2(Math.max(10.0, cost));
}

```

```

private static double round2(double v) {
    return Math.round(v * 100.0) / 100.0;
}

```

```

}package _01_27;

```

```

/**
 * Zugriff aus statische Elemente (Methoden und Attribute)
 *
 *      Name_der_Klasse.Name_des_Elements
 *
 * Beispiele:
 * -Integer.MAX_VALUE
 * -Person.MAX_GEWICHT
 * -LocalDate.now()
 */

```

```

public class Fuhrpark {

```

```

    public static void main(String[] args) {

```

```

        Fahrzeug f =new Fahrzeug("BMW",1977,23554.2,"Freiburg");
        System.out.println(f);
        f.fahren(12.5);
        System.out.println(f);
        f.fahren(-445.11);

```

```

        Fahrzeug f2= new Fahrzeug("Mercedes",2024,112.4,"Nürnberg");
        System.out.println(f2);
        f2.fahren(788.02);

```

```

        System.out.println("Die Anzahl der Autos im Fuhrpark sind:
"+Fahrzeug.getAutoanzahl());

```

```

        final Fahrzeug fahr= new Fahrzeug("VW-
Käfer",1974,250000.3,"Schrottplatz");
        fahr.fahren(35.7);

```

```

        // fahr=new Fahrzeug("Mercedes",2024,112.4,"Nürnberg"); geht
nicht wegen final!

```



```

        Fahrzeug f3=new Fahrzeug("Porsche
911",1998,23501.2,"Stuttgart");
        System.out.println(f3);
        f3.fahren(204.7);
    }

```

```

}

```

Klasse: Insurable

```

package _03_06_logistik;

public interface Insurable {
    double insuranceCostEur();
}

```

Klasse: Polymorphie

```

package _03_06_logistik;
/**
 *
 * Shipment s = new StandardParcel(100, 850);
 *
 * Deklarationstyp : Shipment
 *
 * Laufzeittyp: StandardParcel
 */
public class Polymorphie {

    public static void main(String[] args) {
        Shipment s = new StandardParcel(100, 850);

        //s = new Freight(150, 950, 1250);

        //String tc = ((StandardParcel) s).trackingCode();

        String tc = null;
        if(s instanceof Trackable x)
            System.out.println(x.trackingCode());

        else
            System.out.println("Shipment ist nicht Trackable");

        System.out.println( tc);
    }
}

```

Klasse: Shipment

```
package _03_06_logistik;
public abstract class Shipment {

    private final double weightKg;
    private final double distanceKm;

    protected Shipment(double weightKg, double distanceKm) /*throws
IllegalArgumentException */ {
        if (weightKg <= 0) throw new
IllegalArgumentException("weightKg > 0");
        if (distanceKm < 0) throw new
IllegalArgumentException("distanceKm >= 0");
        this.weightKg = weightKg;
        this.distanceKm = distanceKm;
    }

    public double getWeightKg() { return weightKg; }
    public double getDistanceKm() { return distanceKm; }

    // Abstrakt: jede Versandart hat eine andere Kostenformel
    public abstract double shippingCostEur();

    // Konkrete, gemeinsame Methode: nutzt Polymorphie
    public String info() {
        return getClass().getSimpleName()
            + " weight=" + weightKg
            + "kg distance=" + distanceKm
            + "km cost=" + shippingCostEur() + "€";
    }
}package util;

import interfaces.Insurable;
import model.Shipment;

public class ShipmentCostCalculator {

    public static double calculateCost(Shipment[] shipments) {

        double gesamtkosten = 0.0;
        for (Shipment s : shipments) {

            if (s instanceof Insurable insurable)
                gesamtkosten += insurable.calculateInsurance();

            gesamtkosten += s.shippingCost();

        }
    }
}
```

```

        return gesamtkosten;
    }
}

```

Klasse: StandardParcel

```

package _03_06_logistik;

public class StandardParcel extends Shipment implements Trackable {

    public StandardParcel(double weightKg, double distanceKm) {
        super(weightKg, distanceKm);
    }

    @Override
    public double shippingCostEur() {
        double base = 4.99;
        double weightPart = 0.60 * getWeightKg();
        double distancePart = Math.min(10.0, 0.01 * getDistanceKm());
        return round2(base + weightPart + distancePart);
    }

    @Override
    public String trackingCode() {
        return "STD-" + Integer.toHexString(hashCode()).toUpperCase();
    }

    private static double round2(double v) {
        return Math.round(v * 100.0) / 100.0;
    }
}package _03_27;

```

```

public class StandardPricing implements PricingStrategie{

    @Override
    public double calculatePrice(double weight, double distance) {

        return 5 + weight * 0.5 + distance * 0.1;
    }

}

```

Klasse: Trackable

```

package _03_06_logistik;

public interface Trackable {
    String trackingCode();
}package _03_27_2026;

```

```

import java.io.Serializable;
import java.time.LocalDate;

/**
 * Singleton:
 *
 * 1. Konstruktor : private
 * 2. statische Instanz generieren
 * 3. Factory Methode nach außen(public und static)
 */
public class TrackingCodeGenerator implements Serializable {
    private int counter = 1;
    //2
    private static TrackingCodeGenerator instance = new
TrackingCodeGenerator();

    //1
    private TrackingCodeGenerator() {
    }

    //3
    public static TrackingCodeGenerator getInstance() {
        return instance;
    }

    public String nextCode() {
        int year= LocalDate.now().getYear();
        return "TRK-"+year+"-" + String.format("%04d", counter++);
    }

    @Override
    public String toString() {
        return "TrackingCodeGenerator [counter=" + counter + "]";
    }
}

```

Package: _03_09

Klasse: Ausnahmbehandlung

```

package _03_09;

/**
 * Ausnahme Behandlung - Exception Handling - Exception Mechanism
 *
 *
 *
 * Ausnahme = Exception
 */

```

```

*
* Fehler = Error
*
*
* Fehlerarten:
*   - Syntax-Fehler:
*       - Compiler identifiziert
*   - Laufzeit-Fehler:
*       - Runtime identifiziert
*       - Programmabbruch
*   - Logischer Fehler
*
*       - Inhalt Fehler = Programm tut
nicht das, was beabsichtigt wurde
*       - Denkfehler
*
* Fehler-Quellen:
*   - Inputs (Werte, die von außen kommen)
*
* Konventional:
*   - if, while...
* Modern:
*   - throws, throw, try ... catch
*
*
*/

```

```

public class Ausnahmbehandlung {

    public static void main(String[] args) {

        Sparkonto sk;
        Sparkonto empf;

        try {

            sk = new Sparkonto(50);
            empf = new Sparkonto(1788);
            System.out.println(sk);
            System.out.println("Es wird Geld eingezahlt");
            sk.einzahlen(150);
            System.out.println("Einzahlung war erfolgreich");
            System.out.println(sk);

            sk.einzahlen(800);
            System.out.println(sk);
            System.out.println("Es wird Geld ausgezahlt");

```

```

        try {
            sk.auszahlen(450);
            System.out.println("Auszahlung war erfolgreich, bitte
Quittung unterschreiben");
            System.out.println(sk);
        }
        catch (IllegalOverdraftException e) {
            System.out.println("Das Auszahlen hat nicht geklappt"+
e.getMessage());
        }

        System.out.println(empf);
        System.out.println("Überweisung wird vorbereitet");

        try {
            sk.ueberweisen(empf, 300);
            System.out.println("Überweisung erfolgreich");
        }
        catch (IllegalOverdraftException e) {
            System.out.println("Das Überweisen hat nicht geklappt
"+ e.getMessage());
        }
        System.out.println(sk);
        System.out.println(empf);
        try {
            System.out.println("Überweisung wird vorbereitet");
            empf.ueberweisen(sk, 3000);
            System.out.println("Überweisung erfolgreich");
        }
        catch (IllegalOverdraftException e) {
            System.out.println("Das Überweisen hat nicht geklappt
"+ e.getMessage());
        }
        System.out.println(sk);
        System.out.println(empf);

    }
    catch (IllegalOpeningBalanceException e) {
        System.out.println("Kontoeröffnung nicht erfolgt");
        System.out.println(e.getMessage());
    }
    catch (IllegalAmountException e) {
        System.out.println("Das Einzahlen oder Auszahlen ist nicht
erfolgt");
        System.out.println(e.getMessage());
    }
    // catch (IllegalOverdraftException e) {
    //     System.out.println("Das Auszahlen ist wegen mangelnder
Deckung nicht erfolgt");

```

```
//      System.out.println(e.getMessage());
//    }

//    finally {
//        System.out.println(sk);
//        System.out.println(empf);
//    }
//

    System.out.println("Program terminated normal");
```

```
}
```

```
}
```

Klasse: `IllegalAmountException`

```
package _03_09;
```

```
public class IllegalAmountException extends Exception{
    public IllegalAmountException (String message) {
        super(message);
    }
}package _03_09;
```

```
public class IllegalOpeningBalanceException extends Exception{

    public IllegalOpeningBalanceException(String message) {
        super(message);
    }

}package _03_09;
```

```
public class IllegalOverdraftException extends Exception {

    public IllegalOverdraftException(String message) {
        super(message);
    }

}
```

```
}
```

Klasse: `Sparkonto`

```
package _03_09;
```

```
public class Sparkonto{
```

```

private double kontostand;
private static int number=1;
private final int id;
//....

/**
 *
 * @param kontostand
 * @throws IllegalOpeningBalanceException
 */

    public Sparkkonto(double kontostand) throws
IllegalOpeningBalanceException{
        super();
        this.id=number++;
        if(kontostand >= 10)
            this.kontostand = kontostand;
        else
            //throw new Exception("Kontoeröffnungsbetrag muss
mindestens 10 € sein ");
            throw new
IllegalOpeningBalanceException("Kontoeröffnungsbetrag muss mindestens
10 € sein ");
    }

    public double getKontostand() {
        return kontostand;
    }

// /**
// * 1.Ist diese Methode überhaupt erforderlich?
// * 2.Validierung?Exception
// * @param kontostand
// */
//
// public void setKontostand(double kontostand) {
//     this.kontostand = kontostand;
// }

@Override
public String toString() {
    return "Sparkkonto"+id+" [kontostand=" + kontostand + "];"
}

/**
 * Validierung:
 * 1.Betrag muss grösser als null sein, Exception
 * 2. Betrag muss kleiner oder gleich des Kontostands sein,
Exception

```



```

    *
    * @param betrag
    */
    public void auszahlen(double betrag) throws
IllegalOverdraftException,IllegalAmountException{
        if (betrag >0) {
            if (betrag<= kontostand) {
                this.kontostand -= betrag;
            }
            else {
                throw new IllegalOverdraftException("Das Konto ist
nicht genügend gedeckt!\nKontostand: "+kontostand+" €\nBetrag:
"+betrag+" €");
            }
        }
        else
        {
            throw new IllegalAmountException("Betrag darf nicht
negativ sein: " + betrag);
        }
    }

    public void einzahlen(double betrag) throws
IllegalAmountException{
        if(betrag >= 0)
        {
            this.kontostand += betrag;
        }
        else
            //throw new Exception("Betrag darf nicht negativ sein: " +
betrag);
            throw new IllegalAmountException("Betrag darf nicht
negativ sein: " + betrag);
    }

    /**
     * Validierung
     * Was soll validiert werden....Exception?
     *
     * @param empfaenger
     * @param betrag
     * @throws IllegalAmountException
     * @throws IllegalOverdraftException
     */
    public void ueberweisen(Sparkkonto empfaenger, double betrag)
throws IllegalOverdraftException, IllegalAmountException {
        this.auszahlen(betrag);
        empfaenger.einzahlen(betrag);
    }

```

```

    }
}package HaushaltskassenApp_zwei;
//
//
//import java.io.*;
//import java.time.*;
//import java.util.*;
//import java.util.Locale;
//
//public class SparzielManager {
//    private static final List<Sparziel> ziele = new ArrayList<>();
//    private static final String DATEI = "sparziel.txt";
//    private static LocalDate letzteZufuehrung = null;
//    private static final String LETZTE_ZUFUEHRUNG_KEY =
"letzteZufuehrung";
//
//    // --- SPARRATE AUS FESTEN WERTEN LESEN ---
////    public static double
getSparrateAusFesteWerte(ArrayList<FestWert> festeWerte) {
////        return festeWerte.stream()
////            .filter(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie))
////            .mapToDouble(f -> f.betrag)
////            .sum();
////    }
//    public static double
getSparrateAusFesteWerte(ArrayList<FestWert> festeWerte) {
//        return festeWerte.stream()
//            .filter(f -> !f.istEinnahme &&
//                "Sparen".equals(f.kategorie) &&
//                "Sparplan".equals(f.bezeichnung) &&
//                f.periode == 1)
//            .mapToDouble(f -> f.betrag)
//            .findFirst()
//            .orElse(0.0);
//    }
//
//    public static class Sparziel {
//        String name;
//        double zielbetrag;
//        LocalDate zielDatum;
//        double aktuell;
//        int prioritael;
//
//        public Sparziel(String name, double zielbetrag, LocalDate
zielDatum, double aktuell, int prioritael) {
//            this.name = name;
//            this.zielbetrag = zielbetrag;
//            this.zielDatum = zielDatum;

```

```

//          this.aktuell = aktuell;
//          this.prioritaet = prioritaet;
//      }
//
//      public double fehlend() { return Math.max(0, zielbetrag -
//      aktuell); }
//      public double prozent() { return zielbetrag > 0 ? (aktuell /
//      zielbetrag) * 100 : 0; }
//
//      @Override
//      public String toString() {
//          int balken = (int) (prozent() / 10);
//          return String.format("P%d: %-15s | %6.0f / %6.0f € | [%s
//      %s] %3.0f%% | %6.0f € fehlen",
//          prioritaet, name, aktuell, zielbetrag,
//          "█".repeat(balken), " ".repeat(10 - balken),
//      prozent(), fehlend());
//      }
//  }
//
//  // --- LADEN ---
//  public static void ladeSparziele() {
//      ziele.clear();
//      File file = new File(DATEI);
//      if (!file.exists()) return;
//
//      try (BufferedReader br = new BufferedReader(new
//      FileReader(file))) {
//          String line;
//          while ((line = br.readLine()) != null) {
//              line = line.trim();
//              if (line.isEmpty() || line.startsWith("#"))
//      continue;
//              String[] p = line.split("#", 5);
//              if (p.length == 5) {
//                  ziele.add(new Sparziel(
//                      p[0].trim(),
//                      Double.parseDouble(p[1].trim().replace(',', '.')),
//                      LocalDate.parse(p[2].trim()),
//                      Double.parseDouble(p[3].trim().replace(',', '.')),
//                      Integer.parseInt(p[4].trim())
//                  ));
//              }
//          }
//      } catch (Exception e) {
//          System.err.println("Fehler beim Laden: " +

```

```

e.getMessage());
/////    }
/////    }
//    public static void ladeSparziele() {
//        ziele.clear();
//        File file = new File(DATEI);
//        if (!file.exists()) return;try (BufferedReader br = new
BufferedReader(new FileReader(file))) {
//            String line;
//            while ((line = br.readLine()) != null) {
//                line = line.trim();
//                if (line.isEmpty() || line.startsWith("#")) continue;
//                if (line.startsWith(LETZTE_ZUFUEHRUNG_KEY + "#")) {
//                    String datumStr =
line.substring(LETZTE_ZUFUEHRUNG_KEY.length() +1).trim();
//                    if (!datumStr.isEmpty()) {
//                        letzteZufuehrung = LocalDate.parse(datumStr);
//                    }
//                    continue;
//                }
//                String[] p = line.split("#", 5);
//                if (p.length == 5) {
//                    ziele.add(new Sparziel(
//                        p[0].trim(),
//                        Double.parseDouble(p[1].trim().replace(',', ' ')),
//                        LocalDate.parse(p[2].trim()),
//                        Double.parseDouble(p[3].trim().replace(',', ' ')),
//                        Integer.parseInt(p[4].trim())
//                    ));
//                }
//            }
//            System.out.println("Sparziele geladen: " + ziele.size());
//        } catch (Exception e) {
//            System.err.println("Fehler beim Laden: " +
e.getMessage());
//        }
//    }
//
//
//
//    // --- SPEICHERN ---
/////    public static void speichereSparziele() {
/////        try (PrintWriter pw = new PrintWriter(new
FileWriter(DATEI))) {
/////            pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
/////            ziele.stream()
/////                .sorted(Comparator.comparingInt(s ->
s.prioritaet))
/////                .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d\n",
sz.name, sz.zielbetrag, sz.zielDatum,

```

```

sz.aktuell, sz.prioritaet));
////      } catch (IOException e) {
////      System.err.println("Fehler beim Speichern: " +
e.getMessage());
////      }
////      }
//
//      public static void speichereSparziele() {
//      try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
//      {
//          pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
//          ziele.stream()
//          .sorted(Comparator.comparingInt(s -> s.prioritaet))
//          .forEach(sz -> pw.printf("%s#%.2f#%.2f#%.2f#%d\n",
//          sz.name, sz.zielbetrag, sz.zielDatum, sz.aktuell,
//          sz.prioritaet));
//          // LETZTE ZUFÜHRUNG SPEICHERN
//          if (letzteZufuehrung != null) {
//              pw.println(LETZTE_ZUFUEHRUNG_KEY + "#" +
letzteZufuehrung);
//          }
//      } catch (IOException e) {
//          System.err.println("Fehler beim Speichern: " +
e.getMessage());
//      }
//      }
//
//      // --- NEU ---
//      public static void neuesSparziel() {
//          String name = IO.readString("Name: ");
//          double betrag = IO.readDouble("Zielbetrag: ");
//          LocalDate datum = IO.readDate("Datum: ");
//          int prio = IO.readInt("Priorität (1 = höchste): ");
//          ziele.add(new Sparziel(name, betrag, datum, 0.0, prio));
//          speichereSparziele();
//      }
//
//      // --- MONATLICHE ZUFÜHRUNG (EINZIGE METHODE) ---
//      public static void monatlicheZufuehrung(ArrayList<FestWert>
festeWerte) {
//          double sparrate = getSparrateAusFesteWerte(festeWerte);
//          if (sparrate <= 0 || ziele.isEmpty()) {
//              System.out.println("Keine Sparrate oder Sparziele
vorhanden.");
//          }
//          return;
//      }
//
//      List<Sparziel> sortiert = new ArrayList<>(ziele);
//      sortiert.sort(Comparator.comparingInt(s -> s.prioritaet));
//

```

```

//      double rest = sparrate;
//      for (Sparziel sz : sortiert) {
//          if (rest <= 0) break;
//          double fehlend = sz.fehlend();
//          if (fehlend > 0) {
//              double zuweisung = Math.min(rest, fehlend);
//              sz.aktuell += zuweisung;
//              rest -= zuweisung;
//          }
//      }
//      speichereSparziele();
//      System.out.printf("Monatliche Sparrate: %.2f € verteilt.\n",
sparrate);
//  }
//
//  public static void
automatischeZufuehrungBeimStart(ArrayList<FestWert> festeWerte){
//      LocalDate heute = LocalDate.now();
//      LocalDate monatsBeginn = heute.withDayOfMonth(1);
//      if (letzteZufuehrung == null || !
letzteZufuehrung.equals(monatsBeginn)) {
//          monatlicheZufuehrung(festeWerte);
//          letzteZufuehrung = monatsBeginn;
//          speichereSparziele(); // WICHTIG: speichern!
//      }
//  }
//
//  // --- SPARRATE ÄNDERN (EINZIGE METHODE) ---
//  public static void setzeSparrate(ArrayList<FestWert> festeWerte)
{
//      double aktuell = getSparrateAusFesteWerte(festeWerte);
//      double neu = IO.readDouble("Neue monatliche Sparrate
(aktuell: " + String.format(Locale.US, "%.2f", aktuell) + " €): ");
//      if (neu < 0) return;
//
//////      // Alte entfernen
//////      festeWerte.removeIf(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie));
//////
//////      // Neuen hinzufügen
//////      if (neu > 0) {
//////          LocalDate datum = LocalDate.now().withDayOfMonth(1);
//////          festeWerte.add(new FestWert("Sparplan", neu, false,
datum, "Sparen", 1));
//////      }
//      // Alten Sparplan entfernen
//      festeWerte.removeIf(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie) &&
//      "Sparplan".equals(f.bezeichnung));

```

```

//
//          // Neuen hinzufügen
//          if (neu > 0) {
//              LocalDate datum = LocalDate.now().withDayOfMonth(1);
//              festeWerte.add(new FestWert("Sparplan", neu, false,
datum, "Sparen", 1));
//          }
//          speichereFesteWerte(festeWerte); // <-- SOFORT SPEICHERN!
//          System.out.println("Sparrate aktualisiert und in
festeWerte.txt gespeichert.");
//      }
//
//      // --- FESTEWERTE SPEICHERN (deine Logik) ---
//      public static void speichereFesteWerte(ArrayList<FestWert>
festeWerte) {
//          try (FileWriter writer = new FileWriter("festeWerte.txt")) {
//              for (FestWert f : festeWerte) {
//                  writer.write(String.format(Locale.US, "%s#%.2f#%s#
%s#%s#%d%n",
//                      f.bezeichnung, f.betrag, f.istEinnahme, f.datum,
f.kategorie, f.periode));
//              }
//              System.out.println("Feste Werte in festeWerte.txt
gespeichert.");
//          } catch (IOException e) {
//              System.err.println("Fehler beim Speichern: " +
e.getMessage());
//          }
//      }
//
//
//      public static void aendereSparziel() {
//          zeigeSparziele();
//          if (ziele.isEmpty()) return;
//          int idx = IO.readInt("Nummer (0 = Abbruch): ") - 1;
//          if (idx < 0 || idx >= ziele.size()) return;
//
//          Sparziel sz = ziele.get(idx);
//          System.out.println("Aktuell: " + sz);
//
//          // Name
//          String n = IO.readString("Name (" + sz.name + ", Enter =
beibehalten): ");
//          if (!n.isEmpty()) sz.name = n;
//
//          // Zielbetrag
//          double b = IO.readDouble("Zielbetrag (" + sz.zielbetrag + ",
0 = beibehalten): ");
//          if (b > 0) sz.zielbetrag = b;
//

```

```

//          // Datum – NUR ÄNDERN, WENN ETWAS EINGETIPPT WURDE
//          LocalDate neuesDatum = IO.readDate("Datum (" + sz.zielDatum
+ "): ");
//          if (neuesDatum != null) {
//              sz.zielDatum = neuesDatum;
//          }
//
//          // Priorität
//          int p = IO.readInt("Priorität (" + sz.prioritaet + ", 0 =
beibehalten): ");
//          if (p > 0) sz.prioritaet = p;
//
//          speichereSparziele();
//          System.out.println("Sparziel aktualisiert.");
//      }
//
//      // --- ANZEIGEN (KORREKT NUMMERIERT) ---
//      public static void zeigeSparziele() {
//          if (ziele.isEmpty()) {
//              System.out.println("Keine Sparziele.");
//              return;
//          }
//          System.out.println("=== SPARZIELE (Prio 1 = höchste) ===");
//          List<Sparziel> sortiert = ziele.stream()
//              .sorted(Comparator.comparingInt(s -> s.prioritaet))
//              .toList();
//          for (int i = 0; i < sortiert.size(); i++) {
//              System.out.println((i + 1) + ": " + sortiert.get(i));
//          }
//      }
//
//      // --- LÖSCHEN ---
//      public static void loescheSparziel() {
//          zeigeSparziele();
//          if (ziele.isEmpty()) return;
//          int idx = IO.readInt("Löschen (0 = Abbruch): ") - 1;
//          if (idx >= 0 && idx < ziele.size()) {
//              ziele.remove(idx);
//              speichereSparziele();
//          }
//      }
//  }
//}
//

```

```

import java.io.*;
import java.time.LocalDate;
import java.util.*;

```



```

import java.util.stream.Collectors;

public class SparzielManager {
    private static final List<Sparziel> ziele = new ArrayList<>();
    private static final String DATEI = "sparziel.txt";
    private static LocalDate letzteZufuehrung = null;
    private static final String LETZTE_ZUFUEHRUNG_KEY =
"letzteZufuehrung";

    //
=====
==
    // 1. SPARRATE AUS FESTEN WERTEN LESEN (NUR EIN EINTRAG!)
    //
=====
==
    public static double getSparrateAusFesteWerte(ArrayList<FestWert>
festeWerte) {
        return festeWerte.stream()
            .filter(f -> !f.istEinnahme
                && "Sparen".equals(f.kategorie)
                && "Sparplan".equals(f.bezeichnung)
                && f.periode == 1)
            .mapToDouble(f -> f.betrag)
            .findFirst()
            .orElse(0.0);
    }

    //
=====
==
    // 2. SPARZIEL KLASSE (mit Fortschrittsbalken)
    //
=====
==
    public static class Sparziel {
        String name;
        double zielbetrag;
        LocalDate zielDatum;
        double aktuell;
        int prioritaet;

        public Sparziel(String name, double zielbetrag, LocalDate
zielDatum, double aktuell, int prioritaet) {
            this.name = name;
            this.zielbetrag = zielbetrag;
            this.zielDatum = zielDatum;
            this.aktuell = aktuell;
            this.prioritaet = prioritaet;
        }
    }
}

```

```

        public double fehlend() { return Math.max(0, zielbetrag -
aktuell); }
        public double prozent() { return zielbetrag > 0 ? (aktuell /
zielbetrag) * 100 : 0; }

        @Override
        public String toString() {
            int balken = (int) (prozent() / 10);
            return String.format("P%d: %-15s | %6.0f / %6.0f € | [%s
%s] %3.0f%% | %6.0f € fehlen",
                prioritae, name, aktuell, zielbetrag,
                "█".repeat(balken), " ".repeat(10 - balken),
prozent(), fehlend());
        }
    }

    //
=====
==
    // 3. LADEN AUS DATEI
    //
=====
==
    public static void ladeSparziele() {
        ziele.clear();
        File file = new File(DATEI);
        if (!file.exists()) return;

        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) continue;

                if (line.startsWith(LETZTE_ZUFUEHRUNG_KEY + "#")) {
                    String datumStr =
line.substring(LETZTE_ZUFUEHRUNG_KEY.length() + 1).trim();
                    if (!datumStr.isEmpty()) {
                        letzteZufuehrung = LocalDate.parse(datumStr);
                    }
                    continue;
                }

                String[] p = line.split("#", 5);
                if (p.length == 5) {
                    ziele.add(new Sparziel(
                        p[0].trim(),
                        Double.parseDouble(p[1].trim().replace(',', '.'),

```

```

        '.')),
                                LocalDate.parse(p[2].trim()),
                                Double.parseDouble(p[3].trim().replace(',', '.')),
        '.')),
                                Integer.parseInt(p[4].trim())
                                ));
    }
    }
    System.out.println("Sparziele geladen: " + ziele.size());
} catch (Exception e) {
    System.err.println("Fehler beim Laden der Sparziele: " +
e.getMessage());
}
}

//
=====
==
// 4. SPEICHERN IN DATEI
//
=====
==
public static void speichereSparziele() {
    try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
    {
        pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
        ziele.stream()
            .sorted(Comparator.comparingInt(s -> s.prioritaet))
            .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d%n",
sz.name, sz.zielbetrag, sz.zielDatum, sz.aktuell,
sz.prioritaet));

        if (letzteZufuehrung != null) {
            pw.println(LETZTE_ZUFUEHRUNG_KEY + "#" +
letzteZufuehrung);
        }
    } catch (IOException e) {
        System.err.println("Fehler beim Speichern der Sparziele: "
+ e.getMessage());
    }
}

//
=====
==
// 5. NEUES SPARZIEL
//
=====
==
public static void neuesSparziel() {

```

```

        String name = IO.readString("Name: ");
        double betrag = IO.readDouble("Zielbetrag: ");
        LocalDate datum = IO.readDate("Ziel-Datum (JJJJ-MM-TT): ");
        int prio = IO.readInt("Priorität (1 = höchste): ");
        ziele.add(new Sparziel(name, betrag, datum, 0.0, prio));
        speichereSparziele();
        System.out.println("Sparziel hinzugefügt.");
    }

    //
=====
==
    // 6. MONATLICHE ZUFÜHRUNG (PRIORISIERT!)
    //
=====
==
    public static void monatlicheZufuehrung(ArrayList<FestWert>
festeWerte) {
        double sparrate = getSparrateAusFesteWerte(festeWerte);
        if (sparrate <= 0 || ziele.isEmpty()) {
            System.out.println("Keine Sparrate oder keine
Sparziele.");
            return;
        }

        List<Sparziel> sortiert = new ArrayList<>(ziele);
        sortiert.sort(Comparator.comparingInt(s -> s.prioritaet));

        double rest = sparrate;
        for (Sparziel sz : sortiert) {
            if (rest <= 0) break;
            double fehlend = sz.fehlend();
            if (fehlend > 0) {
                double zuweisung = Math.min(rest, fehlend);
                sz.aktuell += zuweisung;
                rest -= zuweisung;
            }
        }

        speichereSparziele();
        System.out.printf("Monatliche Sparrate: %.2f € verteilt.\n",
sparrate);
    }

    //
=====
==
    // 7. AUTOMATISCHE ZUFÜHRUNG BEIM PROGRAMMSTART
    //
=====

```

```

==
    public static void
    automatischeZufuehrungBeimStart(ArrayList<FestWert> festeWerte) {
        LocalDate heute = LocalDate.now();
        LocalDate monatsBeginn = heute.withDayOfMonth(1);

        if (letzteZufuehrung == null || !
        letzteZufuehrung.equals(monatsBeginn)) {
            monatlicheZufuehrung(festeWerte);
            letzteZufuehrung = monatsBeginn;
            speichereSparziele(); // WICHTIG: speichert neue
        letzteZufuehrung
        }
    }

    //
=====
==
    // 8. SPARRATE ÄNDERN (ersetzt alten "Sparplan")
    //
=====
==
    public static void setzeSparrate(ArrayList<FestWert> festeWerte) {
        double aktuell = getSparrateAusFesteWerte(festeWerte);
        double neu = IO.readDouble("Neue monatliche Sparrate (aktuell:
" + String.format(Locale.US, "%.2f", aktuell) + " €): ");
        if (neu < 0) return;

        // Alten Sparplan entfernen
        festeWerte.removeIf(f -> !f.istEinnahme
            && "Sparen".equals(f.kategorie)
            && "Sparplan".equals(f.bezeichnung));

        // Neuen hinzufügen
        if (neu > 0) {
            LocalDate datum = LocalDate.now().withDayOfMonth(1);
            festeWerte.add(new FestWert("Sparplan", neu, false, datum,
"Sparen", 1));
        }

        // Sofort speichern!
        speichereFesteWerte(festeWerte);
        System.out.println("Sparrate aktualisiert: " +
String.format(Locale.US, "%.2f", neu) + " €/Monat");
    }

    //
=====
==
    // 9. FESTEWERTE SPEICHERN (wird von setzeSparrate() genutzt)

```

```

//
=====
==
    public static void speichereFesteWerte(ArrayList<FestWert>
festeWerte) {
        try (FileWriter writer = new FileWriter("festeWerte.txt")) {
            for (FestWert f : festeWerte) {
                writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s#%d%n",
                                f.bezeichnung, f.betrag, f.istEinnahme, f.datum,
f.kategorie, f.periode));
            }
            System.out.println("Feste Werte gespeichert (inkl.
Sparrate).");
        } catch (IOException e) {
            System.err.println("Fehler beim Speichern der festen
Werte: " + e.getMessage());
        }
    }

//
=====
==
    // 10. SPARZIEL ÄNDERN
    //
=====
==
    public static void aendereSparziel() {
        zeigeSparziele();
        if (ziele.isEmpty()) return;

        int idx = IO.readInt("Nummer ändern (0 = Abbruch): ") - 1;
        if (idx < 0 || idx >= ziele.size()) return;

        Sparziel sz = ziele.get(idx);
        System.out.println("Aktuell: " + sz);

        String n = IO.readString("Name (" + sz.name + ", Enter =
beibehalten): ");
        if (!n.isEmpty()) sz.name = n;

        double b = IO.readDouble("Zielbetrag (" + sz.zielbetrag + ", 0
= beibehalten): ");
        if (b > 0) sz.zielbetrag = b;

        LocalDate d = IO.readDate("Ziel-Datum (" + sz.zielDatum + "):
");
        if (d != null) sz.zielDatum = d;
    }

```

```

        int p = IO.readInt("Priorität (" + sz.prioritaet + ", 0 =
beibehalten): ");
        if (p > 0) sz.prioritaet = p;

        speichereSparziele();
        System.out.println("Sparziel aktualisiert.");
    }

    //
=====
// 11. SPARZIELE ANZEIGEN
//
=====
public static void zeigeSparziele() {
    if (ziele.isEmpty()) {
        System.out.println("Keine Sparziele vorhanden.");
        return;
    }
    System.out.println("=== SPARZIELE (Prio 1 = höchste) ===");
    List<Sparziel> sortiert = ziele.stream()
        .sorted(Comparator.comparingInt(s -> s.prioritaet))
        .collect(Collectors.toList());
    for (int i = 0; i < sortiert.size(); i++) {
        System.out.println((i + 1) + ": " + sortiert.get(i));
    }
}

//
=====
// 12. SPARZIEL LÖSCHEN
//
=====
public static void loescheSparziel() {
    zeigeSparziele();
    if (ziele.isEmpty()) return;

    int idx = IO.readInt("Löschen (0 = Abbruch): ") - 1;
    if (idx >= 0 && idx < ziele.size()) {
        ziele.remove(idx);
        speichereSparziele();
        System.out.println("Sparziel gelöscht.");
    }
}
}

```

Package: _03_11

Klasse: Lesen

```
package _03_11;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;

public class Lesen {
    public static void main(String[] args) {

        /**
         * Lesen:
         * 1. Datei zum Schreiben öffnen
         * 2.Etwas aus der Datei Lesen
         * 3. Datei schließen
         */

        //

        File f = new File("names.txt"); // Object auf dem Heap, es ist
        noch nicht eine Datei erzeugt worden

        //

        try {

            //Lesen mittels FileReader: Zeichenweise, End of File
            (EOF) = -1

            FileReader fr = new FileReader(f);
            FileReader fr2 = new FileReader(f);

            int c;

            while((c = fr.read()) != -1)
                System.out.println("c = " + (char)c);

            fr.close();

            //Lesen mittels BufferedReader: Zeilenweise, End of File
            (EOF) = null

            BufferedReader br = new BufferedReader(fr2);
```



```

        String line;

        while((line = br.readLine()) != null)
            System.out.println("line = " + line);
        br.close();

    } catch (IOException e) {
        System.out.println("Datei konnte nicht erzeugt werden: " +
e.getMessage());
    }

    //...
    System.out.println("Program terminates normally");
}
}

```

Klasse: Service

```

package _03_11;

import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;

public class Service {

    public static ArrayList<Punkt> readPointsFromCSV(String pfad){
        ArrayList<Punkt> result = new ArrayList<Punkt>();

        try {
            BufferedReader br = new BufferedReader(new
FileReader(pfad));

            String line;
            while ((line =br.readLine())!= null) {
                if(line.isEmpty()|| line.isBlank())
                    continue;
                String[] values = line.split(";");
                double x=Double.parseDouble(values[0]);//oder: double

```

```

x=Double.parseDouble(line.split(";")[0])
        double y=Double.parseDouble(values[1]);
        result.add(new Punkt(x,y));
    }

    } catch (FileNotFoundException e) {
        System.out.println(e.getMessage());
    } catch (NumberFormatException e) {
//      System.out.println(e.getMessage());
//    }
    }catch (IOException e) {
        System.out.println(e.getMessage());
    }
    return result;
}

```

```

    public static void writeCirclesToCSV(ArrayList<Kreis> circles,
String pathToFile) {
    /**
     *
     */
    try(PrintWriter pw = new PrintWriter(pathToFile)){

        for (Kreis k : circles) {
            pw.println(k.toCSV());
        }

        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        }
    }
    /**
     * Die Methode soll die Punkten in der Liste zeilenweise in eine
    CSV Datei schreiben, und zwar in CSV Format.
     *
     *
     * Zum Beispiel die Datei könnte folgendermaße aussehen:
     *
     * 0.3;0.5
     * 0.25;0.75
     * 0.89;0.85
     * usw
     * @throws FileNotFoundException
     */
    public static void writePointsToCSV(ArrayList<Punkt> points,
String pathToFile) {

```

```

        PrintWriter pw = null;
        try {
            pw = new PrintWriter(pathToFile);
            for (Punkt p : points)
                pw.println(p.toCSV());

            System.out.println("Das Auschreiben in die Datei
erfolgreich ausgeführt");
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Resource Freigabe");
            if(pw != null)
                pw.close();
        }

        System.out.println("Method terminated...");
    }

    // Ab Java Version 1.7 - try-with-resource-statement

    public static void
writeToCSV_v2writePointsToCSV_v2(ArrayList<Punkt> points, String
pathToFile) {

        try (PrintWriter pw = new PrintWriter(pathToFile)){

            for (Punkt p : points)
                pw.println(p.toCSV());

            System.out.println("Das Auschreiben in die Datei
erfolgreich ausgeführt");
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        }

        System.out.println("Method terminated...");
    }
}

```

Package: _03_16

Klasse: MieteComparator

package _03_16;

import java.time.LocalDate;

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

/**
 * API:
 * - Interfaces: Comparable<T>, Comparator<T>
 * - Hilfsklassen: Arrays, Collections
 * - Methoden: sort, binarySearch
 * - Sammlung von Daten: List oder Array
 */

class Mietekomparator implements Comparator<Wohnung>{

    @Override
    public int compare(Wohnung o1, Wohnung o2) {

        return
Double.valueOf(o1.getMiete()).compareTo(Double.valueOf(o2.getMiete()))
;
    }

}

public class Test {

    private static void sortAndPrintBasedOnEinzugsdatum(Wohnung[] w) {
        Arrays.sort(w);
        System.out.println("=====Wohnungen sortiert nach
Einzugsdatum=====");
        System.out.println(Arrays.toString(w));
    }

    private static void SortAndPrintBasedOnMiete(Wohnung[] w) {
        System.out.println("=====Wohnungen sortiert nach
Miete=====");
        Arrays.sort(w, new Mietekomparator());
        System.out.println(Arrays.toString(w));
    }

    private static void sortListOfWohnung(List<Wohnung> w) {
        Collections.sort(w);
        System.out.println("=====Wohnungen sortiert nach
Einzugsdatum=====");
        System.out.println(w);
    }

}

```

```

public static void main(String[] args) {

    Wohnung [] whg = {
        new Wohnung("10439", "Paul-Robeson-Str 34", 807.47, 4,
        LocalDate.of(2010, 9, 1)),
        new Wohnung("10439", "Kugler-Str. 69", 1400.69, 2,
        LocalDate.of(2023, 3, 2)),
        new Wohnung("10439", "Erich-Weinert-Str. 78", 1165.37,
        3, LocalDate.of(2005, 12, 15)),
        new Wohnung("10778", "Wielandstrasse 38", 560, 1,
        LocalDate.of(2009, 12, 15)),
        new Wohnung("10778", "Wundts str. 38", 450, 1,
        LocalDate.of(2010, 12, 15))
    };

    System.out.println(Arrays.toString(whg));
    //...
    sortAndPrintBasedOnEinzugsdatum(whg);
    SortAndPrintBasedOnMiete(whg);

    List<Wohnung> whgList = new ArrayList<Wohnung>();
    whgList.add(new Wohnung("10439", "Paul-Robeson-Str 34",
    807.47, 4, LocalDate.of(2010, 9, 1)));
    whgList.add(new Wohnung("10439", "Kugler-Str. 69", 1400.69, 2,
    LocalDate.of(2023, 3, 2)));
    whgList.add(new Wohnung("10439", "Erich-Weinert-Str. 78",
    1165.37, 3, LocalDate.of(2005, 12, 15)));
    whgList.add(new Wohnung("10778", "Wielandstrasse 38", 560, 1,
    LocalDate.of(2009, 12, 15)));
    whgList.add(new Wohnung("10778", "Wundts str. 38", 450, 1,
    LocalDate.of(2010, 12, 15)));

    //Sort based on Miete

    Collections.sort(whgList, new MieteComparator());
    System.out.println(whgList);

    //Sort Based in Einzugsdatum

    sortListOfWohnung(whgList);

}

}

```

Klasse: SortierenUndSuchen

```
package _03_16;

import java.time.LocalDate;
import java.util.Arrays;

public class SortierenUndSuchen {
    public static void main(String[] args) {
        int [] numbers = {1,3,5,7,9,2,4,6,8,6,4};

        System.out.println(Arrays.toString(numbers));
        System.out.println(Arrays.binarySearch(numbers, 5)); //-7-1 =
-8

        Arrays.sort(numbers);
        System.out.println(Arrays.toString(numbers));
        System.out.println(Arrays.binarySearch(numbers, 5)); //

        //

        LocalDate [] dates = {
            LocalDate.of(1981, 1, 19),
            LocalDate.of(1987, 12, 15),
            LocalDate.of(1984, 10, 27)
        };

        System.out.println(Arrays.binarySearch(dates,
LocalDate.of(1984, 10, 27))); // -2
    }
}
```

Klasse: Wohnung

```
package _03_16;

import java.time.LocalDate;

public class Wohnung implements Comparable<Wohnung>{

    private final String plz;
    private final String strasse;
    private double miete;
    private final int zimmerAnzahl;
    private final LocalDate einzugsdatum;

    public Wohnung(String plz, String str, double miete, int
zimmerAnzahl, LocalDate einzugsdatum) {
```

```

        this.plz = plz;
        this.strasse = str;
        this.miete = miete;
        this.zimmerAnzahl = zimmerAnzahl;
        this.einzugsdatum = einzugsdatum;
    }
    @Override
    public String toString() {
        return plz + " : " + strasse + " : " + miete + " : " +
zimmerAnzahl + " : " + einzugsdatum + "\n";
    }
    public double getMiete() {
        return miete;
    }
    public String getPlz() {
        return plz;
    }
    public String getStr() {
        return strasse;
    }
    public int getZimmerAnzahl() {
        return zimmerAnzahl;
    }
}

/**
 *
 */
@Override
public int compareTo(Wohnung o) {
    // nach Zimmeranzahl vergleichen : int -> Integer

    //return
Integer.valueOf(zimmerAnzahl).compareTo(Integer.valueOf(o.zimmerAnzahl
));

    // Nach Miete Vergelcihen: double -> Double

    //return
Double.valueOf(miete).compareTo(Double.valueOf(o.miete));

    // Nach PLZ vergleichen: String implements Comparable

    //return plz.compareTo(o.plz);

    // Nach Strasse vergleichen: String implements Comprable
    //return strasse.compareTo(o.strasse);

    // Nach Einzugsdatum vergleichen: LocalDate implements

```

Comprable

```
        return einzugsdatum.compareTo(o.einzugsdatum);  
    }  
  
}
```

Klasse: WrapperKlassen

```
package _03_16;  
/**  
 * Primitive Datentypen                Äquivalente Klassen (Wrapper  
Klassen)  
 *  
 * boolean                            Boolean  
 * byte                               Byte  
 * short                             Short  
 * char                              Character  
 * int                               Integer  
 * float                             Float  
 * long                              Long  
 * double                            Double  
 */  
public class WrapperKlassen {  
    public static void main(String[] args) {  
        //int x = 8;  
  
        Integer x = Integer.valueOf(8) ;  
  
        // Auto- Boxing/Unboxing  
        Integer y = 8;  
  
        int z = y; y.intValue();  
  
    }  
  
}
```

Package: _03_20

Klasse: Filter

```
package _03_20;  
  
import java.util.ArrayList;
```



```

import java.util.List;
import java.util.Random;
import java.util.function.Predicate;

public class Filter {

    public static List<Integer> createRondoms(int n) {
        List<Integer> result = new ArrayList<Integer>();

        Random r = new Random();
        for (int i = 0; i < n; i++) {
            result.add(r.nextInt(100));
        }
        return result;
    }

    public static List<Integer> filter(List<Integer> zahlen,
    Predicate<Integer> t) {

        List<Integer> result = new ArrayList<Integer>();

        for (Integer zahl : zahlen) {
            if(t.test(zahl))
                result.add(zahl);
        }
        return result;
    }

    /**
    public static List<Integer> filterEvens(List<Integer> zahlen)
    {
        List<Integer> evens = new ArrayList<Integer>();
        for (Integer zahl : zahlen) {
            if(zahl % 2 == 0)
                evens.add(zahl);
        }
        return evens;
    }

    public static List<Integer> filterGreaterThan50(List<Integer>
    zahlen) {
        List<Integer> greaterThan50 = new ArrayList<Integer>();
        for (Integer zahl : zahlen) {
            if(zahl > 50)
                greaterThan50.add(zahl);
        }
        return greaterThan50;
    }
    public static List<Integer>

```

```

filterMultiplesOfThree(List<Integer> zahlen) {
    List<Integer> multiplesOfThree = new ArrayList<Integer>();
    for (Integer zahl : zahlen) {
        if(zahl % 3 == 0)
            multiplesOfThree.add(zahl);
    }
    return multiplesOfThree;
}

*/
public static void main(String[] args) {

    List<Integer> numbers = createRondoms(20);

    System.out.println(numbers);

    class Odd implements IntegerTester{

        @Override
        public boolean test(Integer x) {

            return x % 2 == 1;
        }

    };

    System.out.println(filter(numbers, (x) ->x %2 == 1));

    class Even implements Predicate<Integer>{

        @Override
        public boolean test(Integer x) {

            return x % 2 == 0;
        }

    };

    System.out.println(filter(numbers, new Even()));

    class GreatherThan50 implements Predicate<Integer>{

        @Override
        public boolean test(Integer x) {

            return x > 50;
        }

    };
};

```

```

        System.out.println(filter(numbers, new GreatherThan50()));
//      System.out.println(filterMultiplesOfThree(numbers));
    }

```

```

}

```

Klasse: FilterLambda

```

package _03_20;

```

```

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

```

```

public class FilterLambda {

```

```

    public static List<Integer> createRondoms(int n) {
        List<Integer> result = new ArrayList<Integer>();

```

```

        Random r = new Random();
        for (int i = 0; i < n; i++) {
            result.add(r.nextInt(100));
        }
        return result;
    }

```

```

    public static List<Integer> filter(List<Integer> zahlen,
IntegerTester t) {

```

```

        List<Integer> result = new ArrayList<Integer>();

        for (Integer zahl : zahlen) {
            if(t.test(zahl))
                result.add(zahl);
        }
        return result;
    }

```

```

    public static void main(String[] args) {

```

```

        List<Integer> numbers = createRondoms(20);

```

```

        System.out.println(numbers);

```

```

        IntegerTester odd = x -> x % 2 == 1;
        System.out.println(filter(numbers, odd));
        //System.out.println(filter(numbers, x -> x % 2 == 1));
    }

```

```

        IntegerTester even = (x) -> x % 2 == 0;

        System.out.println(filter(numbers, even));

        IntegerTester greaterThan50 = (Integer x) -> x > 50;
        System.out.println(filter(numbers, greaterThan50));
//    System.out.println(filterMultiplesOfThree(numbers));
    }
}

```

Klasse: IntegerTester

```

package _03_20;

public interface IntegerTester {
    boolean test(Integer x);
}

```

Klasse: Test

```

package _03_20;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.function.Predicate;

public class Test {

    static List<Wohnung> createWohnungen(){
        List<Wohnung> whgList = new ArrayList<>();
        whgList.addAll(
            Arrays.asList(new Wohnung("10439", "Paul-Robeson-Str
34", 807.47, 2, LocalDate.of(2010, 9, 1)),
                new Wohnung("10439", "Kugler Str. 69", 1400.69, 2,
LocalDate.of(2023, 3, 2)),
                new Wohnung("10439", "Erich-Weinert-Str. 78",
1165.37, 3, LocalDate.of(2005, 12, 15)),
                new Wohnung("10788", "Wielandstr. 38", 560, 1,
LocalDate.of(2009, 12, 15)),
                new Wohnung("10788", "WundsStr. 38", 450, 1,
LocalDate.of(2010, 12, 15)),
            )
        );
    }
}

```

```

        new Wohnung("10439", "Schönhauser Allee 12", 945.80,
2, LocalDate.of(2018, 5, 1)),
        new Wohnung("10439", "Greifenhagener Str. 21",
1250.00, 3, LocalDate.of(2016, 11, 15)),
        new Wohnung("10439", "Stargarder Str. 5", 735.50, 1,
LocalDate.of(2012, 2, 1)),
        new Wohnung("10439", "Gleimstr. 44", 1590.90, 4,
LocalDate.of(2020, 7, 1)),
        new Wohnung("10439", "Pappelallee 97", 1020.30, 2,
LocalDate.of(2008, 9, 30)),
        new Wohnung("10788", "Nürnberger Str. 15", 880.00, 1,
LocalDate.of(2014, 4, 1)),
        new Wohnung("10788", "Kurfürstenstr. 3", 1325.75, 2,
LocalDate.of(2019, 10, 1)),
        new Wohnung("10788", "Keithstr. 22", 1749.99, 3,
LocalDate.of(2021, 1, 15)),
        new Wohnung("10788", "Lützowstr. 40", 640.00, 1,
LocalDate.of(2007, 6, 1)),
        new Wohnung("10788", "Potsdamer Str. 120", 1995.45,
4, LocalDate.of(2024, 8, 1)),
        new Wohnung("10439", "Paul-Robeson-Str 12", 890.25,
2, LocalDate.of(2011, 3, 1)),
        new Wohnung("10439", "Paul-Robeson-Str 56", 975.90,
3, LocalDate.of(2018, 10, 1)),
        new Wohnung("10439", "Kugler Str. 12", 1250.00, 2,
LocalDate.of(2020, 5, 15)),
        new Wohnung("10439", "Kugler Str. 101", 1499.99, 3,
LocalDate.of(2022, 9, 1)),
        new Wohnung("10439", "Erich-Weinert-Str. 5", 980.40,
2, LocalDate.of(2007, 1, 1)),
        new Wohnung("10439", "Erich-Weinert-Str. 120",
1355.70, 4, LocalDate.of(2019, 6, 1)),
        new Wohnung("10788", "Wielandstr. 5", 630.00, 1,
LocalDate.of(2010, 4, 1)),
        new Wohnung("10788", "Wielandstr. 77", 1420.50, 3,
LocalDate.of(2017, 2, 1)),
        new Wohnung("10788", "Wielandstr. 102", 1899.00, 4,
LocalDate.of(2021, 11, 1)),
        new Wohnung("10788", "Kugler Str. 5", 1100.00, 2,
LocalDate.of(2015, 8, 1))
    ));
    return whgList;

```

```

}
static void print(List<Wohnung> w) {
    for (Wohnung wh : w) {
        System.out.println(wh);
    }
}
public static void main(String[] args) {

```

```

        print(createWohnungen());

        //
        Predicate<Wohnung> wt = (w) -> w.getPlz().equals("10439");

        List <Wohnung> f = FilterWohnungen.filter(createWohnungen(),
wt);

System.out.println("=====
=====");
        print(f);
    }
}

```

Klasse: Wohnung

```

package _03_20;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;

public class Wohnung implements Comparable<Wohnung>{

    private final String plz;
    private final String strasse;
    private double miete;
    private final int zimmerAnzahl;
    private final LocalDate einzugsdatum;

    public Wohnung(String plz, String str, double miete, int
zimmerAnzahl, LocalDate einzugsdatum) {

        this.plz = plz;
        this.strasse = str;
        this.miete = miete;
        this.zimmerAnzahl = zimmerAnzahl;
        this.einzugsdatum = einzugsdatum;
    }
    @Override
    public String toString() {
        return plz + " : " + strasse + " : " + miete + " : " +
zimmerAnzahl + " : " + einzugsdatum + "\n";
    }
    public double getMiete() {
        return miete;
    }
    public String getPlz() {
        return plz;
    }
}

```

```

    public String getStr() {
        return strasse;
    }
    public int getZimmerAnzahl() {

        return zimmerAnzahl;
    }

    /**
     *
     */
    @Override
    public int compareTo(Wohnung o) {
        // nach Zimmeranzahl vergleichen : int -> Integer

        //return
        Integer.valueOf(zimmerAnzahl).compareTo(Integer.valueOf(o.zimmerAnzahl
    ));

        // Nach Miete Vergelcihen: double -> Double

        //return
        Double.valueOf(miete).compareTo(Double.valueOf(o.miete));

        // Nach PLZ vergleichen: String implements Comparable

        //return plz.compareTo(o.plz);

        // Nach Strasse vergleichen: String implements Comprable
        //return strasse.compareTo(o.strasse);

        // Nach Einzugsdatum vergleichen: LocalDate implements
        Comprable

        return einzugsdatum.compareTo(o.einzugsdatum);

    }

}

```

Klasse: WohnungTester
package _03_20;

```

import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;

interface WohnungTester {
    boolean test(Wohnung w);
}

public class FilterWohnungen {
    /**
     * Filter nach:
     * - PLZ
     * - Strasse
     * - Miete
     * - Zimmeranzahl
     */
    public static List<Wohnung> filter(List<Wohnung> wohnungen,
    Predicate<Wohnung> tester){

        List<Wohnung> result = new ArrayList<Wohnung>();

        for (Wohnung w : wohnungen) {
            if(tester.test(w))
                result.add(w);
        }
        return result;
    }
}

```

Package: **_03_25**

Klasse: Adresse

```

package _03_25;

public class Adresse {
    private String strasse;
    private String hnr;
    private String plz;
    private String ort;
    private String land;
    public Adresse(String strasse, String hnr, String plz, String ort,
String land) {
        this.strasse = strasse;
        this.hnr = hnr;
        this.plz = plz;
        this.ort = ort;
        this.land = land;
    }

    public Adresse(String strasse, String ort, String land) {

```



```

        this.strasse = strasse;
        this.hnr = " ";
        this.plz = " ";
        this.ort = ort;
        this.land = land;
    }

    @Override
    public String toString() {
        return "Adresse [strasse=" + strasse + ", hnr=" + hnr + ",
plz=" + plz + ", ort=" + ort + ", land=" + land
            + "]\n";
    }
}

```

Klasse: AdresseMitBuilder

```
package _03_25;
```

```

public final class AdresseMitBuilder {

    private final String strasse;
    private final String hnr;
    private final String plz;
    private final String ort;
    private final String land;

    private AdresseMitBuilder(Builder b) {
        this.strasse = b.strasse;
        this.hnr = b.hnr;
        this.plz = b.plz;
        this.ort = b.ort;
        this.land = b.land;
    }

    @Override
    public String toString() {
        return "Adresse [strasse=" + strasse + ", hnr=" + hnr + ",
plz=" + plz + ", ort=" + ort + ", land=" + land
            + "]\n";
    }
}

public static class Builder{
    private String strasse;
    private String hnr;
    private String plz;
    private String ort;
    private String land;
}

```

```

    public Builder strasse(String strasse) {
        this.strasse = strasse;
        return this;
    }
    public Builder hnr(String hnr) {
        this.hnr = hnr;
        return this;
    }

    public Builder plz(String plz) {
        this.plz = plz;
        return this;
    }

    public Builder ort(String ort) {
        this.ort = ort;
        return this;
    }
    public Builder land(String land) {
        this.land = land;
        return this;
    }
    public AdresseMitBuilder build() {

        return new AdresseMitBuilder(this);
    }
}

}

```

Klasse: Main

```

package _03_25;

public class Main {
    public static void main(String[] args) {

        // Adresse a = new Adresse("Paul-Robeson-str", "34", "10439",
        "Berlin", "Deutschland");
        //
        // //Reihenfolge spielt eine Rolle
        // Adresse b = new Adresse("Deutschalnd", "Berlin", "10439", "34",
        "Paul-Robeson-Str");
        //
        // // Wäre schöner, wenn Hausnummer und PLz Optional wären
        // Adresse c = new Adresse("Paul-Rebson Str", " ", " ", "Berlin",
        "Deutschland");
        //
        // Adresse d = new Adresse("Paul-Robeson-Str", "Berlin",

```

```

"Deutschland");
//
// // Strasse = Paul-Robeson-Str
// // Hausnummer = " "
// // PLZ = " "
// // Ort = Berlin
// // Land = Deutschland
//
// System.out.println(d);

    AdresseMitBuilder a = new AdresseMitBuilder.Builder()
        .land("Deutschland")
        .ort("Berlin")
        .hnr("34")
        .strasse("Paul-Robeson-Str")
        .plz("10439")
        .build();

}
}package _03_26;

public class Main {
    public static void main(String[] args) {
        double cost = VersandKostenRechner.berechnePreis("STANDARD", 150,
300);
        System.out.println(cost);

        cost = VersandKostenRechner.berechnePreis_v2(new
StandardPricing(), 150, 300);

        cost = VersandKostenRechner.berechnePreis_v2(new EconomyPricing(),
150, 300);

        cost = VersandKostenRechner.berechnePreis_v2((a,b) -> 3 + a * 0.3
+ b * 0.05, 150,300);
    }
}

```

Klasse: Wohnung

```

package _03_25;

public final class Wohnung {
    private final String strasse;
    private final String plz;
    private final String stadt;
    private final double miete;
    private final int zimmer;

    private Wohnung(Builder b) {
        this.strasse = b.strasse;
    }
}

```

```
        this.plz = b.plz;
        this.stadt = b.stadt;
        this.miete = b.miete;
        this.zimmer = b.zimmer;
    }
}
```

```
public static class Builder {
    private String strasse;
    private String plz;
    private String stadt;
    private double miete;
    private int zimmer;

    public Builder strasse(String s) {
        this.strasse = s;
        return this;
    }

    public Builder plz(String p) {
        this.plz = p;
        return this;
    }

    public Builder stadt(String s) {
        this.stadt = s;
        return this;
    }

    public Builder miete(double m) {
        this.miete = m;
        return this;
    }

    public Builder zimmer(int z) {
        this.zimmer = z;
        return this;
    }

    public Wohnung build() {
        return new Wohnung(this);
    }
}

}package _16_03_2026;
```

```

import java.time.LocalDate;

public class Wohnung implements Comparable<Wohnung>{

    private final String plz;
    private final String str;
    private double miete;
    private final int zimmerAnzahl;
    private final LocalDate einzugsdatum;

    public Wohnung(String plz, String str, double miete, int
zimmerAnzahl, LocalDate einzugsdatum) {
        super();
        this.plz = plz;
        this.str = str;
        this.miete = miete;
        this.zimmerAnzahl = zimmerAnzahl;
        this.einzugsdatum = einzugsdatum;
    }
    public String getPlz() {
        return plz;
    }
    public String getStr() {
        return str;
    }
    public double getMiete() {
        return miete;
    }
    public int getZimmerAnzahl() {
        return zimmerAnzahl;
    }
    @Override
    public String toString() {
        return "Wohnung ["+ plz + " : " + str + " : " + miete + " : "
+ zimmerAnzahl + " : "+einzugsdatum+"]\n";
    }
}

```

```

// @Override
// public int compareTo(Wohnung o) {
//     if (miete<o.miete)
//         return -1;
//     else if (miete==o.miete)
//         return 0;
//     else
//         return 1;
//     //besser wäre zimmerAnzahl - o. zimmerAnzahl
//     // oder miete-o.miete

```

```

// }

    /**
     * Concatenieren falls nach mehreren Grössen sortiert werden soll
     * die reihenfolge ist dabei zu beachten
     *
     */
// @Override
// public int compareTo(Wohnung o) {
//     // nach Zimmeranzahl vergleichen : int ->Integer
//     return
Integer.valueOf(zimmerAnzahl).compareTo(Integer.valueOf(o.zimmerAnzahl
));
// //autoboxing:return
Integer.valueOf(zimmerAnzahl).compareTo(o.zimmerAnzahl);
// }

// @Override
// public int compareTo(Wohnung o) {
//     //nach Miete vergleichen
//     return
Double.valueOf(miete).compareTo(Double.valueOf(o.miete));
// }

// @Override
// public int compareTo(Wohnung o) {
//     return Integer.valueOf(plz.trim() .compareTo(o.plz.trim()));
// }
//
// @Override
// public int compareTo(Wohnung o) {
//     return Integer.valueOf(str.trim().compareTo(o.str.trim()));
// }

    @Override
    public int compareTo(Wohnung o) {
        return Integer.valueOf((einzugsdatum + plz
+str) .compareTo(o.einzugsdatum+ o.plz+ o.str ));
    }

}

```

Package: _03_26

Klasse: AppConfig
package _03_26;

```

public class AppConfig {
    private String name;
    private String version;

    private static AppConfig instance = new AppConfig("XX", "1.0");

    private AppConfig(String name, String version) {
        this.name = name;
        this.version = version;
    }

    public static AppConfig getInstance() {
        return instance;
    }

    @Override
    public String toString() {
        return "AppConfig [name=" + name + ", version=" + version +
"]";
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getVersion() {
        return version;
    }
    public void setVersion(String version) {
        this.version = version;
    }
}

```

Klasse: EconomyPricing

```
package _03_26;
```

```

public class EconomyPricing implements PricingStrategie{

    @Override
    public double calculatePrice(double weight, double distance) {

```

```

        return 3 + weight * 0.3 + distance * 0.05;
    }
}

```

Klasse: ExpressPricing

```

package _03_26;

public class ExpressPricing implements PricingStrategie{

    @Override
    public double calculatePrice(double weight, double distance) {

        return 10 + weight + distance * 0.2;
    }

}

```

Klasse: Monat

```

package _03_26;

public enum Monat {

}

```

Klasse: PricingStrategie

```

package _03_26;

public interface PricingStrategie {

    double calculatePrice(double weight, double distance);
}

```

Klasse: PriorityPricing

```

package _03_26;

public class PriorityPricing implements PricingStrategie {

    @Override
    public double calculatePrice(double weight, double distance) {

        return 20 + weight * 1.5 + distance * 0.3;
    }

}

```

Klasse: StandardPricing

```

package _03_26;

```



```

public class StandardPricing implements PricingStrategie{

    @Override
    public double calculatePrice(double weight, double distance) {

        return 5 + weight * 0.5 + distance * 0.1;
    }

}

```

Klasse: VersandKostenRechner

```

package _03_26;

public class VersandKostenRechner {
    public static double berechnePreis(String typ, double gewicht,
double distanz) {
        if (typ.equals("STANDARD")) {
            return 5 + gewicht * 0.5 + distanz * 0.1;
        } else if (typ.equals("EXPRESS")) {
            return 10 + gewicht * 1.0 + distanz * 0.2;
        } else if (typ.equals("PRIORITY")) {
            return 20 + gewicht * 1.5 + distanz * 0.3;
        } else {
            throw new IllegalArgumentException("Unbekannter Typ");
        }
    }

    public static double berechnePreis_v2(PricingStrategie ps, double
gewicht, double distanz) {
        double cost = ps.calculatePrice(gewicht, distanz);
        return (gewicht <= 50)? cost : 0.9 * cost ;
    }

}

```

Package: _03_27

Klasse: Main

```

package _03_27;

import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        //
        // TrackingCodeGenerator g1 =
TrackingCodeGenerator.getInstance(); //new TrackingCodeGenerator();
        // TrackingCodeGenerator g2 =
TrackingCodeGenerator.getInstance();//new TrackingCodeGenerator();
    }
}

```

```

//
// String s11 = g1.nextCode();
// String s12 = g1.nextCode();
// String s13 = g1.nextCode();
//
// System.out.println("s11 = " + s11 + ", s12 = " + s12 + ", s13 = "
+ s13);
//
// String s21 = g2.nextCode();
// String s22 = g2.nextCode();
// String s23 = g2.nextCode();
//
// System.out.println("s21 = " + s21 + ", s22 = " + s22 + ", s23 = "
+ s23);
//
// System.out.println(g1 == g2);

List<Shipment> ships = Arrays.asList(new Shipment(50, 120, new
PriorityPricing()),
                                new Shipment(75, 125, new
EconomyPricing()),
                                new Shipment(175, 275, new
StandardPricing())
                                );

System.out.println(ships);
}
}

```

Klasse: Praefix

```

package _03_27;

public enum Praefix {
    STD, EXP
}

```

Klasse: PriorityPricing

```

package _03_27;

public class PriorityPricing implements PricingStrategie {

    @Override
    public double calculatePrice(double weight, double distance) {

        return 20 + weight * 1.5 + distance * 0.3;
    }

}

```

Klasse: Shipment

```
package _03_27;

/**
 * Singleton und Strategy Pattern
 */

public class Shipment {
    private final double weight;
    private final double distance;
    private final String trackingCode;

    private PricingStrategie strategie;

    public double getWeight() {
        return weight;
    }
    public double getDistance() {
        return distance;
    }
    public String getTrackingCode() {
        return trackingCode;
    }
    public Shipment(double weight, double distance, PricingStrategie
strategie) {

        this.weight = weight;
        this.distance = distance;
        this.trackingCode =
TrackingCodeGenerator.getInstance().nextCode();
        this.strategie = strategie;
    }

    //...

    public double calculatePrice() {
        return strategie.calculatePrice(weight, distance);
    }
    @Override
    public String toString() {
        return "\nShipment [weight=" + weight + ", distance=" +
distance + ", trackingCode=" + trackingCode
        + ", Price=" + calculatePrice() + "]\n";
    }
}
```

Klasse: TrackingCodeGenerator

```
package _03_27;
```

```

import java.time.LocalDate;

/**
 * Singleton:
 *      1. Konstruktor private deklarieren
 *      2. statische Instanz generieren
 *      3. Factory Methode nach außen (public static)
 */
public class TrackingCodeGenerator {
    private int counter = 1;
    private static TrackingCodeGenerator instance = new
TrackingCodeGenerator(); // 2

    // 1
    private TrackingCodeGenerator() {

    }

    //3

    public static TrackingCodeGenerator getInstance() {
        return instance;
    }

    public String nextCode() {
        int year = LocalDate.now().getYear();
        return "TRK-" + year + "-" + String.format("%04d", counter++);
    }

}

```

Package: [_03_27_2026](#)

Klasse: [EconomyPricing](#)

```
package _03_27_2026;
```

```
import java.io.Serializable;
```

```

public class EconomyPricing implements PricingStrategie,
Serializable{

    @Override
    public double calculatePrice(double weight, double distance) {

        return 3 + weight * 0.3 + distance * 0.05;
    }

}

```

Klasse: ExpressPricing

```
package _03_27_2026;

import java.io.Serializable;

public class ExpressPricing implements PricingStrategie,
Serializable{

    @Override
    public double calculatePrice(double weight, double distance) {

        return 10 + weight + distance * 0.2;
    }

}
```

Klasse: Main

```
package _03_27_2026;

import java.io.EOFException;
import java.io.FileInputStream;
import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {

    public static void main(String[] args) {

        //      TrackingCodeGenerator g1=
        TrackingCodeGenerator.getInstance();//new TrackingCodeGenerator()
        //      TrackingCodeGenerator g2=
        TrackingCodeGenerator.getInstance();//new TrackingCodeGenerator()
        //
        //      String s11=g1.nextCode();
        //      String s12=g1.nextCode();
        //      String s13=g1.nextCode();
        //
        //
        //      System.out.println("s11= "+s11+ " s12= "+ s12 + " s13= "+s13);
        //
        //      String s21=g2.nextCode();
        //      String s22=g2.nextCode();
        //      String s23=g2.nextCode();
    }

}
```

```
//
//
//      System.out.println("s21= "+s21+ " s22= "+ s22 + " s23= "+s23);
//
//      System.out.println(g1==g2);
```

```
        List<Shipment> ships = Arrays.asList(new Shipment(50,120,new
PriorityPricing()),
        new Shipment(75,125,new ExpressPricing()),
        new Shipment(205, 189, new EconomyPricing()),
        new Shipment(175,275,new StandardPricing()));
```

```
        System.out.println(ships);
```

```
        try {
            ObjectOutputStream oos= new ObjectOutputStream(new
FileOutputStream("shipments.ser"));
```

```
            for (Shipment s : ships) {
                oos.writeObject(s);
            }
```

```
            oos.close();
```

```
        } catch (FileNotFoundException e) {
```

```
            e.printStackTrace();
```

```
        } catch (IOException e) {
```

```
            e.printStackTrace();
```

```
        }
```

```
        try {
            ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("shipments.ser"));
            List<Shipment> shipments = new ArrayList<Shipment>();
```

```
            while(true) {    //ois.readObject()!=null geht nicht sonst
würde ich schon eine zeile auslesen....
```

```
                try {
                    Shipment ship= (Shipment) ois.readObject();
                    shipments.add(ship);
                }catch (EOFException e) {
                    break;
                }
```

```
            }
```

```
        System.out.println("shipments nach dem Auslesen: "+shipments);
```

```
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    } catch (IOException e) {  
        e.printStackTrace();  
    } catch (ClassNotFoundException e) {  
        e.printStackTrace();  
    }  
}  
}
```

Klasse: PricingStrategie

```
package _03_27_2026;
```

```
public interface PricingStrategie {  
  
    double calculatePrice(double weight, double distance);  
}
```

Klasse: PriorityPricing

```
package _03_27_2026;
```

```
import java.io.Serializable;
```

```
public class PriorityPricing implements PricingStrategie, Serializable  
{  
  
    @Override  
    public double calculatePrice(double weight, double distance) {  
        return 20 + weight * 1.5 + distance * 0.3;  
    }  
}
```

Klasse: Serialization

```
package _03_27_2026;
```

```
import java.io.FileInputStream;
```

```

import java.io.FileNotFoundException;
import java.io.FileOutputStream;
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.io.Serializable;
import java.time.LocalDate;

public class Serialization implements Serializable {
    public static void main(String[] args) {

        String s = new String("Bob");
        LocalDate d = LocalDate.now();

        Person p = new Person("Wondmu", LocalDate.of(1981, 1, 19), new
Konto("AAA", 10), "admin123");

        //Serialization
        try {
            ObjectOutputStream oos= new ObjectOutputStream(new
FileOutputStream("name.ser"));
            oos.writeObject(s);
            oos.writeObject(d);
            oos.writeObject(p);
            oos.close();

        } catch (FileNotFoundException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }

        //Deserialization

        try {
            ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("name.ser"));
            String name = (String) ois.readObject();
            System.out.println("name = " +name);

            LocalDate r =(LocalDate) ois.readObject();
            System.out.println("r = "+r);

            Person me = (Person) ois.readObject();
            System.out.println("Person = " +me);

```



```

        ois.close();
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } catch (ClassNotFoundException e) {
        e.printStackTrace();
    }
}
}

```

Klasse: Shipment

```

package _03_27_2026;

import java.io.Serializable;

/**
 * Singelton und Strategy Pattern
 */

public class Shipment implements Serializable{

    private final double weight;
    private final double distance;
    private final String trackingCode;

    private PricingStrategie strategy;//hat eine Beziehung: hier
    Komposition

    public double getWeight() {
        return weight;
    }
    public double getDistance() {
        return distance;
    }
    public String getTrackingCode() {
        return trackingCode;
    }
    public Shipment(double weight, double distance, PricingStrategie
    strategy) {
        this.weight = weight;
        this.distance = distance;
        this.trackingCode =

```

```

TrackingCodeGenerator.getInstance().nextCode();
    this.strategy=strategy;
}

//..

// public abstract double calculatePrice();
public double calculatePrice() {
    return strategy.calculatePrice(weight,distance);
}
@Override
public String toString() {
    return "Shipment [weight=" + weight + ", distance=" + distance
+ ", trackingCode=" + trackingCode
        + ", Price=" + calculatePrice() + "]\n";
}
}

```

Klasse: StandardPricing

```

package _03_27_2026;

import java.io.Serializable;

public class StandardPricing implements
PricingStrategie,Serializable{

    @Override
    public double calculatePrice(double weight, double distance) {

        return 5 + weight * 0.5 + distance * 0.1;
    }

}

```

Package: _04_03_2026

Klasse: Auto

```

package _04_03_2026;

public class Auto implements Moveable{

    private double x;
    private double y;
    public Auto(double x, double y) {
        super();
        this.x = x;
        this.y = y;
    }
}

```

```

    public void drive() {
        //...
    }

    @Override
    public void move(double dx, double dy) {
        // TODO Auto-generated method stub
        this.x +=dx;
        this.y +=dy;
    }
}

```

```

class Ball implements Moveable,Bounceable{
    private double x;
    private double y;
    public Ball(double x, double y) {
        super();
        this.x = x;
        this.y = y;
    }
    @Override
    public void bounce() {
        // TODO Auto-generated method stub

    }
    @Override
    public void move(double dx, double dy) {
        // TODO Auto-generated method stub

    }

    //....

}

```

```

class GummiEnte implements Bounceable{

    private String name;

    public GummiEnte(String name) {
        super();
        this.name = name;
    }

    @Override
    public void bounce() {

```

```

        // TODO Auto-generated method stub
    }

}

interface Moveable{
    void move(double dx,double dy);
}

interface Bounceable{
    void bounce();
}

Klasse: Interfaces
package _04_03_2026;

import _27_02_2026.Router;
import _27_02_2026.Device;

public class Interfaces {

    public static void main(String[] args) {
        I1 i = new C();// da I abstract ist können wir nicht mit new I
        ein Interface erzeugen man nimmt die unterklasse C
        i.m();

        Device d = new Router(null, null, 0, null, 0);
        Router r = new Router(null, null, 0, null, 0);

        int x=3;
    }

}

/**
 * Eine Methode in einem Interface ist standardmäßig public und
 * abstract.
 * ->Die Angabe ist optional
 */

interface I1{
    void m();
    void f();
}

interface I2{

```

```

        void m();
        void g();
    }

class C implements I1, I2{

    @Override
    public void g() {
        // TODO Auto-generated method stub

    }

    @Override
    public void m() {
        // TODO Auto-generated method stub

    }

    @Override
    public void f() {
        // TODO Auto-generated method stub

    }

    public void h() {
        //zusätzliche Methode
    }

}package _04_03_2026;

public class Interface_Fly {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

    }

}

interface Flyable{
    void hebeAb();
    void lande();
}
abstract class Animal{
    public abstract void macheGeraeusch();
}

abstract class Vehicle{
    public abstract void erhoeheGeschwindigkeit(double dx);
}

```

```

}
class Bird extends Animal implements Flyable{

    @Override
    public void macheGeraeusch() {
        System.out.println("Peeeeep!");
    }

    @Override
    public void hebeAb() {
        // TODO Auto-generated method stub
    }

    @Override
    public void lande() {
        // TODO Auto-generated method stub
    }
}

class Dog extends Animal{

    @Override
    public void macheGeraeusch() {
        System.out.println("Wuff!");
    }

}

class Plane extends Vehicle implements Flyable{

    @Override
    public void hebeAb() {
        // TODO Auto-generated method stub
    }

    @Override
    public void erhoeheGeschwindigkeit(double dx) {
        // TODO Auto-generated method stub
    }

    @Override
    public void lande() {

```

```

        // TODO Auto-generated method stub

    }

}

class Car extends Vehicle{
    private double geschwindigkeit;
    private double preis;

    public Car(double geschwindigkeit, double preis) {
        super();
        this.geschwindigkeit = geschwindigkeit;
        this.preis = preis;
    }

    @Override
    public void erhoeheGeschwindigkeit(double dx) {
        // TODO Auto-generated method stub

    }

}

package _04_03_2026;

public class Interface_Test {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Moveable [] moveables = new Moveable[2];
        moveables[0]= new Auto(0, 0);
        moveables[1]= new Ball(0.5, 0);

        int x;
        MyClass mc;
        MyEnum me;
        MyInterface mi;

    }

/**
 * Interfaces (dt.:Schnittstelle)
 * -Ist wie eine Klasse ein Dateentyp
 *     -Keywords: class, enum, interface
 * -Eine Klasse kann nicht von mehreren Klassen erben(Dimond Problem)
 *
 * Workaround: Interfaces
 * Eine Klasse kann gleichzeitin mehrere Interfaces implementieren
 * -Keywords: extends, implements

```

```

* Vererbung:
* -IST-EINE Beziehung
* Implementierung:
* -KANN-WAS Beziehung
*
* Ein Interface ist vergleichbar mit einer abstrakten Klasse:
* -Gemeinsamkeiten:
*     kann nicht instanziiert werden
*     Polymorphie
*     abstrakte Methoden
* -Unterschiede:
*     Interfaces haben KEINEN Konstruktor
*     Interfaces haben keine Instanz-Variablen
*     Methoden sind standardmäßig abstract und public
*
*/
}
class MyClass{

}
enum MyEnum{

}

interface MyInterface{

}package Logistikuebung_2;

public class InvalidShipmentDataException extends Exception{

    /**
     *
     */
    private static final long serialVersionUID = 1L;

    public InvalidShipmentDataException(String message) {
        super(message);
    }

}

```

Klasse: Mehrfachvererbung

```

package _04_03_2026;

/**
 * Maultier erbt von Pferd (Mutter) und Esel (Vater)
 * -In der realen Welt gibt es Mehrfachvererbung
 * Java verbietet Mehrfachvererbung, C++ unterstützt, aber nicht

```


empfohlen

*/

```
public class Mehrfachvererbung {
```

```
    public static void main(String[] args) {  
        //      Maultier mt = new Maultier();  
        //      mt.m();  
    }  
}
```

```
class Pferd{  
    void m() {  
  
    }  
}
```

```
class Esel{  
    void m() {  
  
    }  
}
```

```
}  
//class Maultier extends Pferd, Esel{  
//  
//}
```

Klasse: Uebung

```
package _04_03_2026;
```

```
public class Uebung {
```

```
    public static void main(String[] args) {  
        Vehicle[] vehicles =new Vehicle[5];  
        Animal [] animals = new Animal[5];  
        Flyable [] flyables = new Flyable[5];  
  
        for (int i=0; i<vehicles.length;i++) {  
            if (i%2==0)  
                vehicles[i]=new Car(0,12.000);  
            else  
                vehicles[i]=new Plane();  
        }  
  
        for (int i=0; i<animals.length;i++) {  
            if (i%2==0)  
                animals[i]=new Bird();  
            else  
                animals[i]=new Dog();  
        }  
    }  
}
```

```

    }

    for (int i=0; i<flyables.length;i++) {
        if (i%2==0)
            flyables[i]=new Bird();
        else
            flyables[i]=new Plane();
    }

}

}

```

Package: `_04_03_2026_Uebung`

Klasse: `ApplPay`

```
package _04_03_2026_Uebung;
```

```
public class ApplPay implements PaymentMethod {
    private boolean isLoggedIn;
```

```

    public ApplPay(boolean isLoggedIn) {
        super();
        this.isLoggedIn = isLoggedIn;
    }

```

```

    @Override
    public Konto fuereZahlungDurch(Konto k,double betrag) {
        betrag=Math.abs(betrag);
        if (isLoggedIn) {
            k.setKontostand(k.getKontostand()-betrag-1.5);
            System.out.println("ApplePay-Zahlung in Höhe von ***
"+String.format("%.2f€",betrag )+ " ***
durchgeführt.Bearbeitungsgebühr in Höhe von 1,50€ abgezogen.");
        }
        else
            System.out.println("Zahlung nicht möglich.Einloggen oder
Registrieren erforderlich.");
        return k;
    }
    @Override
    public Konto erhalteZahlung(Konto k, double betrag) {
        betrag=Math.abs(betrag);
        k.setKontostand(k.getKontostand()+betrag-1.5);
        System.out.println("ApplePay-Zahlung in Höhe von ***
"+String.format("%.2f€",betrag )+ " *** erhalten.Bearbeitungsgebühr in

```

```
Höhe von 1,50€ abgezogen.");
    return null;
}

}
```

Klasse: CreditCard

```
package _04_03_2026_Uebung;
```

```
public class CreditCard implements PaymentMethod {
```

```
    private double limit;
```

```
    public CreditCard(double limit) {
```

```
        this.limit = limit;
    }
```

```
    @Override
```

```
    public Konto fuereZahlungDurch(Konto k, double betrag) {
        betrag=Math.abs(betrag);
        if(betrag<=limit) {
            k.setKontostand(k.getKontostand()-Math.max(betrag+2,
1.01*betrag ));
            System.out.println("Kreditkartenzahlung in Höhe von ***
"+String.format("%.2f€",betrag )+ " ***
durchgeführt.Bearbeitungsgebühr in Höhe von "+String.format("%.2f€ (1%
%, aber mindestens 2,00€)",Math.max(2, 0.01*betrag ) )+" abgezogen.");
        }
        else
            System.out.println("Betrag liegt über dem Höchstbetrag
"+String.format("%.2f€", limit));
        return k;
    }
```

```
    @Override
```

```
    public Konto erhalteZahlung(Konto k, double betrag) {
        betrag=Math.abs(betrag);
        k.setKontostand(k.getKontostand()+Math.min(betrag*.99, betrag-
2));
        System.out.println("Kreditkartenzahlung in Höhe von ***
"+String.format("%.2f€",betrag )+ " *** erhalten.Bearbeitungsgebühr in
Höhe von "+String.format("%.2f€ (1%%, aber mindestens
```

```

2,00€)",Math.max(2, 0.01*betrag ))+" abgezogen.");
    return k;
}

```

```

}

```

Klasse: Konto

```

package _04_03_2026_Uebung;

```

```

public class Konto {
    private String name;
    private String vorname;
    private final String iban;
    private int blz;
    private String bank;
    private double kontostand;
    private double kreditrahmen;

    public Konto(String name, String vorname, String iban, int blz,
String bank, double kontostand,
        double kreditrahmen) {
        super();
        this.name = name;
        this.vorname = vorname;
        this.iban = iban;
        this.blz = blz;
        this.bank = bank;
        this.kontostand = kontostand;
        this.kreditrahmen = kreditrahmen;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getVorname() {
        return vorname;
    }
}

```

```
public void setVorname(String vorname) {
    this.vorname = vorname;
}

public String getIban() {
    return iban;
}

public int getBlz() {
    return blz;
}

public void setBlz(int blz) {
    this.blz = blz;
}

public String getBank() {
    return bank;
}

public void setBank(String bank) {
    this.bank = bank;
}

public double getKontostand() {
    return kontostand;
}

public void setKontostand(double kontostand) {
    this.kontostand = kontostand;
}

public double getKreditrahmen() {
    return kreditrahmen;
}

public void setKreditrahmen(double kreditrahmen) {
```

```

        this.kreditrahmen = kreditrahmen;
    }

    @Override
    public String toString() {
        return "Konto [name=" + name + ", vorname=" + vorname + ",
iban=" + iban + ", blz=" + blz + ", bank=" + bank
        + ", kreditrahmen=" + String.format("%.2f€",
kreditrahmen) + ", *** kontostand=" + String.format("%.2f€ ***",
kontostand) + "]";
    }
}

```

```

}

```

Klasse: PayPal

```

package _04_03_2026_Uebung;

public class PayPal implements PaymentMethod{

    private boolean isLoggedIn;

    public PayPal(boolean isLoggedIn) {
        super();
        this.isLoggedIn = isLoggedIn;
    }

    @Override
    public Konto fuereZahlungDurch(Konto k,double betrag) {
        betrag=Math.abs(betrag);
        if (isLoggedIn) {
            k.setKontostand(k.getKontostand()-betrag-2.0);
            System.out.println("Paypalzahlung in Höhe von ***
"+String.format("%.2f€",betrag) + " ***
durchgeführt.Bearbeitungsgebühr in Höhe von 2,00€ abgezogen.");
        }
        else
            System.out.println("Zahlung nicht durchgeführt.Einloggen
erforderlich.");
    }
}

```

```

        return k;
    }
    @Override
    public Konto erhalteZahlung(Konto k, double betrag) {
        betrag=Math.abs(betrag);
        k.setKontostand(k.getKontostand()+betrag-2.0);
        System.out.println("Paypalzahlung in Höhe von ***
"+String.format("%.2f€",betrag )+ " *** erhalten.Bearbeitungsgebühr in
Höhe von 2,00€ abgezogen.");
        return k;
    }
}

```

Klasse: PaymentMethod

```

package _04_03_2026_Uebung;

public interface PaymentMethod {
    Konto fuereZahlungDurch(Konto k, double betrag);
    Konto erhalteZahlung(Konto k, double betrag);
}

```

Klasse: Shop

```

package _04_03_2026_Uebung;

public class Shop {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        PaymentMethod[] methods = {
            new CreditCard(3000),
            new PayPal(true),
            new BankTransfer(),
            new ApplPay(true)
        };

        Konto konto=new Konto("Schneider", "Bob", "DE12 3456 7891 0111
2131 41", 12345678, "Millonäre-Bank", 123789.10, 200000.0);

        System.out.println(konto);
        System.out.println("=".repeat(160));
        System.out.println("Auszahlungen:");

        for (PaymentMethod m : methods) {
            System.out.println("=".repeat(160));
            m.fuereZahlungDurch(konto, 99.99);
            System.out.println("=".repeat(160));
        }
    }
}

```

```

        System.out.println(konto);
    }

    System.out.println("=".repeat(160));
    System.out.println("Einzahlungen:");
    for (PaymentMethod m : methods) {
        System.out.println("=".repeat(160));
        m.erhalteZahlung(konto, 100);
        System.out.println("=".repeat(160));
        System.out.println(konto);
    }
}

```

Package: `_05_03_2026`

Klasse: Artikel

```

package _05_03_2026;

public class Artikel {
    private final String name;
    private double preis;
    public Artikel(String name, double preis) {
        this.name = name;
        this.preis = preis;
    }
    public double getPreis() {
        return preis;
    }
    public void setPreis(double preis) {
        this.preis = preis;
    }
    public String getName() {
        return name;
    }
    @Override
    public String toString() {
        return "Artikel [name=" + name + ", preis=" + preis + "]";
    }
}

```

Klasse: Movable

```

package _05_03_2026;

public interface Movable {
    void move(double dx, double dy);
}

```



```
}
```

Klasse: Tier

```
package _05_03_2026;
```

```
public class Tier {
```

```
//...
```

```
}
```

```
interface Diet{  
    String getDietTyp();  
}
```

```
class Loewe extends Tier implements Diet{
```

```
    @Override  
    public String getDietTyp() {  
        return "Carnivore";  
    }  
}
```

```
}
```

```
class Schaf extends Tier implements Diet{
```

```
    @Override  
    public String getDietTyp() {  
        return "Herbivore";  
    }  
}
```

```
}package _03_05;
```

```
public class Tier {
```

```
//...
```

```
}
```

```
interface Diet{  
    String getDietTyp();  
}
```

```
class Loewe extends Tier implements Diet{
```

```

        @Override
        public String getDietTyp() {

            return "Carnivore";
        }
    }

class Schafe extends Tier implements Diet{

    @Override
    public String getDietTyp() {

        return "Herbivore";
    }
}

class C{
    final int i = 2;
}package _27_02_2026;

public class Tierwelt {
    public static void main(String[] args) {

        Hund h = new Hund(null, "Spitz", 3,DietTyp.CARNIVORE,
        "Schäferhund");
        Hund h2 = new Hund(new Haustier("Bobbi", "Max Mustermann"),
        "Bob", 5,DietTyp.OMNIVORE, "Dackel");

        h2.makeSound();

    }
}

class Flugzeug{

}

enum DietTyp{
    HERBIVORE, CARNIVORE, OMNIVORE
}

abstract class Tier{
    protected Haustier ht;// wenn kein Haistier , ht au null setzen!

```

```

    protected String name;
    protected int alter;
    protected DietTyp diet;

    public Tier(HausTier ht, String name, int alter, DietTyp diet) {
        super();
        this.ht = ht;
        this.name = name;
        this.alter = alter;
        this.diet=diet;
    }

    public abstract void makeSound();
}
class Hund extends Tier{

    private String rasse;

    public Hund(HausTier ht, String name, int alter,DietTyp
diet,String rasse) {
        super(ht, name, alter,diet);
        this.rasse=rasse;
    }

    @Override
    public void makeSound() {
        System.out.println("Wuff!");
    }

}
class Loewe extends Tier{

    public Loewe(String name, int alter, DietTyp diet) {
        super(null, name, alter, diet);
    }

    @Override
    public void makeSound() {
        System.out.println("Rwaaaarrrr!");
    }

}

}
class Schaf extends Tier{

```

```

        public Schaf(HausTier ht, String name, int alter, DietTyp diet) {
            super(ht, name, alter, diet);
            // TODO Auto-generated constructor stub
        }

        @Override
        public void makeSound() {
            System.out.println("Mähh!");
        }

    }
}

class Haustier{
    private String nickname;
    private String Besitzer;
    public Haustier(String nickname, String besitzer) {
        super();
        this.nickname = nickname;
        Besitzer = besitzer;
    }
}

}package Montag11082025;

public class Tower {

    public static void main(String[] args) {

        Flugzeug cessna =new Flugzeug("CessnaA7",2.5,2012,13.5,1);

        System.out.println(cessna.getDaten());

    }

}

Klasse: Versand
package _05_03_2026;

public interface Versand {

    double berechnen(Warenkorb w);

}

```

Klasse: Warenkorb

```
package _05_03_2026;

import java.util.ArrayList;

public class Warenkorb {

    private ArrayList<Artikel> artikelList= new ArrayList<Artikel>();

    public void artikelHinzufuegen(Artikel a, int stueckzahl) {
        for (int i=0;i<stueckzahl;i++) {
            artikelList.add(a);
        }
    }

    public double berechneGesamtwert() {
        double gesamt=0.0;
        for(Artikel a: artikelList) {
            gesamt += a.getPreis();
        }
        return gesamt;
    }
}
```

Package: _09_03_2026

Klasse: Ausnahmebehandlung

```
package _09_03_2026;

/**
 * Ausnahmebehandlung = Exceptionhandling = Exceptionmechanism
 *
 *
 * Ausnahme = Exception
 *
 * Fehler = Error
 *
 * Fehlerarten:
 *     -Syntaxfehler
 *           -Compiler identifiziert
 *     -Laufzeitfehler
 *           -Runtime identifiziert
 *           -Programmabbruch
 *     -Logische Fehler
 *           -inhaltlicher Fehler;das Programm macht nicht das,
was es tun sollte
 *           -Denkfehler
 *
 * Fehlerquellen:
```

```

*           -Inputs (Werte, die von aussen kommen)
*
* Konventional(klassisch):
*           -if, while.....
* Modern:
*           -throws, throw, try....catch
*
*/

```

```

public class Ausnahmebehandlung {

    public static void main(String[] args) {

        Sparkkonto sk = null;
        try {
            sk= new Sparkkonto(500);
            System.out.println(sk);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }

        // boolean success = sk.einzahlen(-150);
        // if (!success) {
        //     System.out.println("Amount should be positive!");
        // }

        try {
            //guarded region
            System.out.println("Es wird Geld eingezahlt");
            sk.einzahlen(-150);
            System.out.println("Einzahlung erfolgreich!");
            System.out.println(sk);
        } catch (Exception e) {
            // TODO Auto-generated catch block
            //Handling
            System.out.println("Bein Einzahlen ist etwas schief
gegangen!");
            System.out.println(e.getMessage());
        }

        System.out.println(sk);
        try {
            sk.setKontostand(-100);
        } catch (Exception e) {
            // TODO Auto-generated catch block
            System.out.println(e.getMessage());
        }
    }
}

```

```

        try {
            sk.einzahlen(800);
        } catch (Exception e) {
            // Handlung
            System.out.println("Beim Einzahlen ist etwas schief
gegangen!");
            System.out.println(e.getMessage());
        }
        System.out.println(sk);
        try {
            System.out.println("Es wird Geld ausgezahlt");
            sk.auszahlen(1450);
            System.out.println("Auszahlung erfolgreich!");
        } catch (Exception e) {
            System.out.println("Beim Auszahlen ist etwas schief
gelaufen");
            System.out.println(e.getMessage());
        }
        System.out.println(sk);

    }
}

```

```

class Sparkonto{
    private double kontostand;
    //IBAN.....

    public Sparkonto(double kontostand) throws Exception {
//        try {
//            setKontostand(kontostand);
//        } catch (Exception e) {
//
//            e.getMessage();
//        }
        if (kontostand<10)
            this.kontostand=kontostand;
        else {
            //throw new Exception("Kontoeröffnung muss mindestens 10 €
sein");
            throw new IllegalStateException("Kontoeröffnung muss
mindestens 10 € sein");
        }
    }
}

```

```

    }

    public double getKontostand() {
        return kontostand;
    }

    public void setKontostand(double kontostand) throws Exception {
        if (kontostand >= 0)
            this.kontostand = kontostand;
        else {
            //throw new Exception("Der Kontostand darf nicht negativ
sein: " + String.format("%.2f €", kontostand));
            throw new IllegalArgumentException("Der Kontostand darf
nicht negativ sein: " + String.format("%.2f €", kontostand));
        }
    }

    @Override
    public String toString() {
        return "Sparkonto [kontostand=" + String.format("%.2f €",
kontostand) + "]";
    }

    public void auszahlen(double betrag) throws Exception{
        if(betrag >=0) {
            if (this.kontostand >= betrag) {
                this.kontostand -= betrag;
            }
            else {
                throw new Exception("Das Konto ist für eine
Auszahlung von " + String.format("%.2f €", betrag) + " nicht genügend
gedeckt\nAktueller Kontosstand: " + String.format("%.2f €",
kontostand));
            }
        }
        else {
            throw new Exception("Betrag darf nicht negativ sein:
" + String.format("%.2f €", betrag));
        }
    }

    // public boolean einzahlen(double betrag) {
    //     if(betrag >=0) {
    //         this.kontostand += betrag;
    //         return true;
    //     }
    //     else {
    //         //System.out.println("Bertag zum Einzahlen muss positiv
sein!");
    //         return false;
    //     }
    // }

```



```

//      }
//  }

    public void einzahlen(double betrag) throws Exception {
        if(betrag >=0) {
            this.kontostand +=betrag;

        }
        else {
            throw new Exception("Betrag darf nicht negativ sein: "+
String.format("%.2f €",betrag));
        }
    }

    public void ueberweisen(Sparkkonto s, double betrag){
        if (betrag>0) {
            try {
                s.einzahlen(betrag);
            } catch (Exception e) {
                System.out.println(e.getMessage());
            };
        }else {
            try {
                s.auszahlen(betrag);
            } catch (Exception e) {
                System.out.println( e.getMessage());
            }
        }

    }

}

class IllegalStateException extends Exception{

    public IllegalStateException(String message) {
        super(message);
    }
}

class IllegalAmountException extends Exception{
    public IllegalAmountException(String message) {
        super(message);
    }
}

Klasse: DroneDelivery
package _09_03_2026;

```

```

public class DroneDelivery extends Shipment{

    public DroneDelivery(double weightKg, double distanceKm) throws
InvalidShipmentDataException {
        super(weightKg, distanceKm);
    }

    @Override
    public double shippingCost() {

        return 444;
    }
}

```

Klasse: ExpressParcel

```
package _09_03_2026;
```

```
import InOut.IO;
```

```
public class ExpressParcel extends Shipment implements Trackable,
Insurable {
```

```
    private final double warenwert;
    public static final double GRUNDPREIS=9.99;
```

```
    public ExpressParcel(double weightKg, double distanceKm, double
warenwert) throws InvalidShipmentDataException {
        super(weightKg, distanceKm);
        if (warenwert>=0) {
            this.warenwert = warenwert;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
sein");
        }
    }

```

```
    @Override
    public String toString() {
        return String.format("ExpressParcel:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
            + "Shipping-Kosten: %22.2f€\nTrackingcode: %26s\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost(),getTra
ckingCode());
    }

```

```
    @Override
    public double calculateInsurance() {
```

```

        return Math.max(warenwert*0.01,2.5);
    }

    @Override
    public String getTrackingCode() {
        return hashCode()+" ";
    }

    @Override
    public double shippingCost() {
        return GRUNDPREIS+1.1*getWeightKg()+0.02*getDistanceKm();
    }
}

```

Klasse: Insurable

```

package _09_03_2026;

public interface Insurable {

    double calculateInsurance();

}

```

Klasse: InvalidShipmentDataException

```

package _09_03_2026;

public class InvalidShipmentDataException extends Exception{
    /**
     *
     */
    private static final long serialVersionUID = 1L;

    public InvalidShipmentDataException (String message) {
        super(message);
    }
}package _10_15_lab;
/**
 * Lernziel
 * Attribute
 * Konstruktoren
 * Methoden
 * statische und nicht-statische Mitglieder einer Klasse
 */
import java.util.Scanner;
public class IO {

    /**

```

```

        * definiere folgenden Methoden
        *
        * readInt
        *   liest eine ganze zahl aus der Tastatur und gibt den
eingelesenen Wert für den Aufrufer zurück
        *   bei der eingabeaufforderung soll ein benutzerfreundlicher text
für den User angezeigt werden
        *
        * readDouble
        * wie oben
        */
// String text1 = "Dieses Programm zeigt die von Ihnen eingegebene
Zahl an,um Verwechslungen zu vermeiden.\n";
// String text2 = "Sie haben folgende Zahl eingegeben: ";
// Scanner sc = new Scanner(System.in);
// public IO(){
//
// }
//
// public void readInt() {
//     System.out.print(text1+"Geben Sie eine ganze Zahl ein: ");
//     int zahl = sc.nextInt();
//     System.out.println(text2 + zahl);
// }
//
// public void readDouble() {
//     System.out.print(text1+"Geben Sie eine Fließkommazahl ein: ");
//     double fzahl = sc.nextDouble();
//     System.out.println(text2 + fzahl);
// }

    public static int readInt(String prompt) {
        System.out.print(prompt);
        Scanner scan=new Scanner(System.in);
        return scan.nextInt();
    }

    public static double readDouble(String prompt) {
        System.out.print(prompt);
        Scanner scan=new Scanner(System.in);
        return scan.nextDouble();
    }

}

```

Klasse: Polymorphie

```
package _09_03_2026;
```

```
/**
 *
```

```

* Shipment s = new StandardParcel(100, 850);
* Deklarationstyp: Shipment
*
*
* Laufzeittyp: StandardParcel
*/

```

```

public class Polymorphie {

    public static void main(String[] args) {
        Shipment s = null;
        try {
            s = new StandardParcel(100, 850);
        } catch (InvalidShipmentDataException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }

        try {
            s = new ExpressParcel(150, 950, 1250);
        } catch (InvalidShipmentDataException e) {
            // TODO Auto-generated catch block
            e.printStackTrace();
        }
        // String tc=((StandardParcel) s).getTrackingCode();

        if (s instanceof Trackable track) {
            String tc= track.getTrackingCode();
            System.out.println("tc = "+tc);
        }
        // String tc = null;
        // if(s instanceof Trackable) {
        //     tc=((StandardParcel) s).getTrackingCode();
        // }
        // System.out.println(tc);
        if (s instanceof Trackable) {
            System.out.println("Trackingcode: " + ((Trackable)
s).getTrackingCode());
        }
    }
}

```

Klasse: Sendungen

```

package _09_03_2026;

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

```

```

public class Sendungen {

    public static void main(String[] args) {

        Shipment[] shipments=new Shipment[5];
        try {
            shipments[0]= new StandardParcel(15, 250);
            shipments[1]= new ExpressParcel(75, 500, 750);
            shipments[2]= new Freight(4500, 2300, 8000);
            shipments[3]= new Freight(400, 50, 180);
            shipments[4]= new ExpressParcel(1400, 150, 180);
        }

        catch (InvalidShipmentDataException e){
            System.out.println(e.getMessage());
        }

        File f = new File("wareneingangsliste.txt");

        try {
            PrintWriter pw = new PrintWriter(new FileWriter(f,true));

            pw.println("=".repeat(40));
            double gesamtkosten = 0.0;
            for (Shipment s : shipments) {
                pw.println(s);

                if (s instanceof Insurable insurable) {
                    gesamtkosten += insurable.calculateInsurance();
                    pw.println(String.format("Versicherung: %25.2f" ,
insurable.calculateInsurance()) + "€");
                } else {
                    pw.println("keine Versicherung verfügbar");
                }

                if (s instanceof Trackable track) {
                    pw.println(String.format("Tackingcode:
%27s",track.getTrackingCode()));
                }else {
                    pw.println("kein Trackingcode verfügbar");
                }

                pw.println(String.format("Versandkosten: %24.2f" ,
s.shippingCost()) + "€");
                gesamtkosten += s.shippingCost();
                pw.println("=".repeat(40));
            }
        }
    }
}

```

```

    }

    pw.println(String.format("Gesamtkosten: %25.2f" ,
gesamtkosten) + "€");
    pw.close();
}
catch (IOException e) {
    System.out.println("Datei konnte nicht erzeugt werden
"+e.getMessage());
}

}

/**
 * einfacher ist es, die Versicherungskosten jeweils direkt zu den
shippingkosten dazuzurechnen,
 * da man dann einfach über die shippingkosten in shipments summiert
 */
}

```

Klasse: Shipment

```

package _09_03_2026;

//import InOut.IO;

public abstract class Shipment {
    private final double weightKg;
    private final double distanceKm;

    public double getWeightKg() {
        return weightKg;
    }

    public double getDistanceKm() {
        return distanceKm;
    }

    public Shipment(double weightKg, double distanceKm) throws
InvalidShipmentDataException{
        if (weightKg >0) {
            this.weightKg=weightKg;

```

```

        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
sein");
        }

        if(distanceKm>0) {
            this.distanceKm = distanceKm;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss
positiv sein");
        }

    }

    @Override
    public String toString() {
        return String.format("Gewicht: %29.1fkg\nEntfernung: %26.1fkm\n",
weightKg,distanceKm);
    }

    public abstract double shippingCost();

}

```

Klasse: StandardParcel

```
package _09_03_2026;
```

```

public class StandardParcel extends Shipment implements Trackable{
    public static final double GRUNDPREIS=4.99;
    public StandardParcel(double weightKg, double distanceKm) throws
InvalidShipmentDataException {
        super(weightKg, distanceKm);
    }
}

```

```

    @Override
    public String toString() {
        return String.format("StandardParcel:\nShipping-Kosten:
%22.2f€\nTrackingcode:%27s\n" +super.toString(), shippingCost(),
getTrackingCode());
    }
}

```

```
@Override
```



```

        public double shippingCost() {
            return GRUNDPREIS+0.6*getWeightKg()
+Math.min(0.01*getDistanceKm(), 10);
        }

        @Override
        public String getTrackingCode() {
            return hashCode()+ "";
        }
    }

```

Klasse: Trackable

```

package _09_03_2026;

public interface Trackable {

    String getTrackingCode();

}

```

Package: _10_03_2026

Klasse: FileIO

```

package _10_03_2026;

import java.io.BufferedReader;
import java.io.BufferedWriter;

/**
 * IO Input output
 *
 *
 * Dateibasiert:
 *     -File(Datei oder Verzeichnis)
 *     -Lesen aus einer Datei
 *     -Schreiben in eine Datei
 *
 * Text-basierte Dateien
 *     -Plain text
 *     -CSV
 */

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Paths;

```

```

import java.util.ArrayList;
import java.util.List;

public class FileIO {

    public static void main(String[] args) {

        /**
         * Schreiben:
         *      -1. Datei erzeugen bzw existierende Datei zum
Schreiben öffnen
         *      -2. Etwas in die Datei schreiben
         *      -3. Datei schließen
         *
         *C:\Users\Student\OneDrive - GFN GmbH (EDU)\Desktop\Ordner 1\
LF  ZQ19A
         */
        //Datei erzeugen

        //File f = new File("C:\\Users\\Student\\OneDrive - GFN GmbH
(EDU)\\Desktop\\Ordner 1\\LF  ZQ19A\\temperaturen.txt");
        File f = new File("names.txt"); //Objekt auf dem HEAP, es ist
noch keine Datei erzeugt worden
        //C:\Users\Student\eclipse-workspace\LF_ZQ19A
        //Datei erzeugen auf dem Dateisystem

        try {
            boolean success= f.createNewFile();
            if (success)
                System.out.println("Datei wurde erzeugt");
            else
                System.out.println("Datei existiert bereits");
            System.out.println("success = "+ success);

            //Schreiben mit BufferedWriter
            BufferedWriter br= new BufferedWriter(new
FileWriter(f,true));//das true für das anhängen egal ob append oder
write

            br.write("=====Mittels BufferedWriter=====\\
n");

            br.write("Bob");
            br.newLine();//neue zeile bzw zeilenumbruch
            br.write("Alice");
            br.newLine();
            br.write("Chris");
            br.newLine();

```

```

br.flush();//jetzt wird es in die datei geschrieben
br.append("das kommt neu dazu");
br.newLine();
br.flush();
br.append("das kommt nochmal dazu");
br.newLine();
br.close();//hier ist das flush() nicht mehr nötig

```

```

//Schreiben mittels PrintWriter(beste)
PrintWriter pw = new PrintWriter(new FileWriter(f,true));

pw.write("=====Mittels PrintWriter=====\\n");
pw.println("=".repeat(30));
pw.println("Bob");
pw.println("Alice");
pw.println("Chris");
//pw.flush();
pw.close();

```

```

//Schreiben mittels FileWriter
FileWriter fw=new FileWriter(f, true);//append statt
überschreiben

```

```

fw.write("=====Mittels FileWriter=====\\n");
fw.write("Bob \\n");
fw.write("Alice\\r\\n");
fw.write("Chris\\r\\n");
fw.write("Bob \\t");
fw.write("Alice\\t");
fw.write("Chris\\n");
fw.close();

```

```

    } catch (IOException e) {
        System.out.println("Datei konnte nicht erzeugt werden
"+e.getMessage());
    }

    //Lesen
    List<String> ls = new ArrayList<String>();
    try (BufferedReader br =
Files.newBufferedReader(Paths.get("names.txt")
)) {

```

```

        String s;
        while ((s=br.readLine()) != null)
            ls.add(s);
        } catch (IOException e) {
            System.out.println("Datei konnte nicht gelesen werden
"+e.getMessage());
        }

        for(String s : ls)
            System.out.println(s);

        System.out.println("Programm terminiert normal");

    }

}

```

Klasse: Gemischt_quadratische_Gleichungen

```

package _10_03_2026;

import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Gemischt_quadratische_Gleichungen {
    public static void main(String[] args) {
        int limit = 0;
        int fileCounter = 1;
        int equationsPerFile = 10000; // Anzahl der Gleichungen pro
Datei
        int equationCount = 0;
        FileWriter writer = null;

        Scanner sc = new Scanner(System.in);
        System.out.println("Wir generieren gemischt-quadratische
Gleichungen");
        System.out.print("Geben Sie die Grenze ein: ");
        limit = sc.nextInt();

        try {
            writer = new FileWriter("C:\\Users\\Student\\OneDrive -
GFN GmbH (EDU)\\Desktop\\Ordner 1\\LF_ZQ19A\\output_" + fileCounter +
".txt");

            for (int i = -limit; i < limit + 1; i++) {
                for (int j = -limit; j < limit + 1; j++) {
                    int p = -i - j;

```

```

        int q = i * j;

        if (p < 0) {
            if (q < 0) {
                writer.write("Gleichung: \tx2" + p + "x" +
q + "=0\n");
            } else if (q > 0) {
                writer.write("Gleichung: \tx2" + p + "x+"
+ q + "=0\n");
            }
        } else if (p > 0) {
            if (q < 0) {
                writer.write("Gleichung: \tx2+" + p + "x"
+ q + "=0\n");
            } else if (q > 0) {
                writer.write("Gleichung: \tx2+" + p + "x+"
+ q + "=0\n");
            }
        }
    }

    equationCount++;

    if (equationCount >= equationsPerFile) {
        writer.close();
        fileCounter++;
        writer = new FileWriter("C:\\Users\\Student\\
OneDrive - GFN GmbH (EDU)\\Desktop\\Ordner 1\\LF_ZQ19A\\output_" +
fileCounter + ".txt");
        equationCount = 0;
    }
}

if (writer != null) {
    writer.close();
}

System.out.println("Die Gleichungen wurden in mehrere
Dateien aufgeteilt.");
} catch (IOException e) {
    System.out.println("Ein Fehler ist aufgetreten: " +
e.getMessage());
} finally {
    sc.close();
}
}
}package _26_02_26;

public class Geo {

```

```

    public static void main(String[] args) {

        Form[] forms = new Form[6]; // Ein Array Objekt der Grösse 6
        // wird erzeugt: [null,null,null,null,null,null]

        forms[0] = new Kreis(new Punkt(0,0),1.00);
        forms[1] = new Rechteck(new Punkt(0,0.7),3,6);
        forms[2] = new Quadrat(new Punkt(0,1.0),5);
        forms[3] = new Dreieck(new Punkt(0.3,0.4),3,4,90);
        forms[4] = new RechtwinkligesDreieck(new Punkt(.5,.6), 20,
21);
        forms[5] = new GleichseitigesDreieck(new Punkt(2,4), 5);

        GleichseitigesDreieck gd =(GleichseitigesDreieck)
forms[5]; //Downcasten
        gd.setEingeschlossener_winkel(61);
        gd.setZweite_seite(75);

        for(Form f : forms) {
            System.out.println(f);
        }

    }
}

```

Klasse: Uebung

```

package _10_03_2026;

import InOut.IO;

public class Uebung {

    public static void main(String[] args) {
        System.out.println("Start");
        int[] ia = { 1, 2, 3 };
        try {
            int i = IO.readInt("i: ");
            System.out.println(ia[i]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("End");
    }
}

```

Package: _10_13

Klasse: Mensch

```
package _10_13;

public class Mensch {
    //Attribute
    double gewicht; // Körpergewicht in kg
    int groesse; // Körpergröße in cm
    //Konstruktoren
    public Mensch(double w, int h) {
        gewicht = w;
        groesse = h;
    }

    //Methoden
    // BMI berechnen

    public double berechneBMI() {
        double index = gewicht / (groesse / 100.0 * groesse / 100.0);
        return index;
    }

    /**
     * Definiere hier eine Methode, die den BMI bewertet
     *
     * Kopf einer Methode:
     *
     * [Modifiers] + Rückgabewerttyp + name der Methode +
     (Parameter-list)
     */
    public String evalBMI ()
    {
        double b = berechneBMI();

        String evalText;

        if( b < 16)
            evalText = "kritisches Untergewicht";
        else if (b < 18.5)
            evalText = "Untergewicht";
        else if (b < 25)
            evalText = "Normalgewicht";
        else if (b < 30)
            evalText = "Leichtes Übergewicht";
        else
            evalText = "Übergewicht";

        return evalText;
    }
}
```

```

    }

}

Klasse: Test
package _10_13;

public class Test {
    public static void main(String[] args) {
        Mensch wondmu = new Mensch(62.5, 173);
        System.out.println(wondmu.gewicht); //0.0 Standard Wert (default
value) für double
        System.out.println(wondmu.groesse); //0 Standard Wert (dafault
value) für int

        double bmi = wondmu.berechneBMI();
    }
}

```

```

Klasse: vs
package _10_13;
/**
 * 1. Anwendung in Java
 *                               - main Methode
 * 2. Programme starten:
 *                               java Hauptklasse Bob Alice Chris
 *
 * 3. Java ist streng typisierte Sprache
 *
 *                               - Variable (Behälter) müssen vor der Verwendung
typisiert werden.
 * 4. Operatoren
 *                               - built-in Functions
 *                               - Arithemische, logische, Vergelich ...
 *                               - Anzahl von Operanden
 *                               - Precedence Table
 *
 * 5. Kontrolstrukturen:
 *                               -Verzweigung: if, if - else, swicth-case
 *                               - Loops: for, while, do while
 *
 * Einstieg in OO:
 *                               -class vs object
 */
public class Wiederholung {

    public static void main(String[] args) {

        //
        //

```



```
//    for(int i = 0; i < args.length; i++)
//        System.out.println(args[i]);

    int x;

}

}
```

Package: `_10_13_lab`

Klasse: `Mensch`

```
package _10_13_lab;
```

```
public class Mensch {
    //Attribute
    double gewicht; //Körpergewicht in kg
    int groesse; //Körpergröße in cm
    String name;
    int alter;
    String geschlecht;

    //Konstruktoren
    public Mensch(double gewicht, int groesse, String name, int
alter, String geschlecht) {
        this.gewicht = gewicht;
        this.groesse = groesse;
        this.name = name;
        this.alter = alter;
        this.geschlecht = geschlecht;
    }

    //Methoden
    // BMI berechnen
    public double berechneBMI() {
        //Berechnung
        double index = gewicht / groesse / groesse * 10000;

        //Rückgabe
        return index;
    }

    public String getName() {
        return name;
    }
}
```

```

    }

    public void setName(String name) {
        this.name = name;
    }

    public double getGewicht() {
        return gewicht;
    }

    public void setGewicht(double gewicht) {
        this.gewicht = gewicht;
    }

    public int getGroesse() {
        return groesse;
    }

    public void setGroesse(int groesse) {
        this.groesse = groesse;
    }

    /**
     * Definiere hier eine Methode, die den BMI bewertet
     *
     * Kopf einer Methode
     *
     * [Modifiers]+Rückgabewerttyp +Name der Methode+Parameterliste
     */

    public String evalBMI() {
        double b = berechneBMI()-berechneKorrektur();
        String evalText;
        if(b<16) {
            evalText="kritisches Untergewicht";
        }else if(b<18.5){
            evalText="Untergewicht";
        }else if (b<25) {
            evalText="Normalgewicht";
        }else if (b<30) {
            evalText="leichtes Übergewicht";
        }else {
            evalText="Übergewicht";
        }
        // System.out.println(b);
        return evalText;
    }

    public int getAlter() {
        return alter;
    }

```

```

    public void setAlter(int alter) {
        this.alter = alter;
    }
    public String getGeschlecht() {
        return geschlecht;
    }
    public void setGeschlecht(String geschlecht) {
        this.geschlecht = geschlecht;
    }

    public int berrechneKorrektur() {
        int korrektur = 0;
        int geschlechtskorrektur=0;
        if (alter<19) {
            korrektur=-3;}
        else if(alter<25) {
            korrektur= -1;}
        else if(alter<35) {
            korrektur= 0;}
        else if(alter<45) {
            korrektur=1 ;}
        else if(alter<55) {
            korrektur= 2;}
        else if(alter<65) {
            korrektur= 3;}
        else {
            korrektur= 4;}
        if (geschlecht.toLowerCase().equals("w")) {
            geschlechtskorrektur=-1;
        }

        return korrektur+geschlechtskorrektur;
    }
}

```

Klasse: Test

```

package _10_13_lab;

import java.util.Scanner;
public class Test {

    public static void main(String[] args) {
        double gew;
        int gro;
        int alt;
        String name;
        String gesch;
        //    Mensch wondmu = new Mensch(62.5,173,"");
        //    Mensch me = new Mensch(0.0,1,"",25,"m");
        //    System.out.println(wondmu.gewicht);
    }
}

```

```

//      System.out.println(wondmu.groesse);
//
//      double bmi=wondmu.berrechneBMI();
//      System.out.printf("Der BMI ist: %.1f\n",bmi);

        Scanner sc =new Scanner(System.in);
        System.out.print("Geben Sie Ihren Namen ein: ");
        name=sc.nextLine();
        me.setName(name);
        System.out.print("Geben Sie Ihr Geschlecht ein(m für
männlich,w für weiblich): ");
        gesch=sc.nextLine();
        me.setGeschlecht(gesch);
        do {
            System.out.print("Geben Sie Ihr Alter ein: ");
            alt=sc.nextInt();
            if(alt<0) {
                System.out.println("Fehlerhafte Eingabe!");
            }
        }while(alt<0);
        me.setAlter(alt);
        do {
            System.out.print("Geben sie Ihre Grösse in ganzen cm an: ");
            gro =sc.nextInt();
            if (gro<=0) {
                System.out.println("Fehlerhafte Eingabe!");
            }
        }while(gro<=0);
        me.setGroesse(gro);
        do {
            System.out.print("Geben Sie Ihr Gewicht in kg an: ");
            gew=sc.nextDouble();
            if (gew<=0) {
                System.out.println("Fehlerhafte Eingabe!");
            }
        }while(gew<=0);
        me.setGewicht(gew);

        System.out.printf("Der BMI von %s ist: %.1f\n
n",me.getName(),me.berrechneBMI());
        System.out.println("Dies entspricht: "+ me.evalBMI());
        sc.close();
    }

}

```

Klasse: vs

```

package _10_13_lab;
/**

```

```

* 1. Anwendung in Java
*      -main Methode
* 2. Programme starten:
*      Java Hauptklasse Bob Alice Chris
* 3. Java ist streng typisierte Sprache
*      Variable (Behälter) müssen vor der Verwendung
typisiert werden
* 4. Operatoren
*      interne Methoden(buiod in functions)
*      -Arithmetische, logische,Vergleichs....
*      -Anzahl von Operanden
*      -Precedence table(Punkt vor Strich)
* 5. Kontrollstrukturen
*      -Verzweigung: if,if-else.switch-case
*      -Loops: for,while, do while
* Einstieg in Objektorientierung(00)
*      -class vs object
*/

```

```

import java.util.Scanner;

public class Wiederholung {

    public static void main(String[] args) {

        //      System.out.println("Hallo Welt");//public void
println(String s)

//      for (int i=0;i<args.length;i++)
//          System.out.println(args[i]);
//

//      double bmi=0;
//
//      Scanner sc =new Scanner(System.in);
//      System.out.print("Geben sie Ihr Alter ein: ");
//      int alter = sc.nextInt();
//      System.out.print("Geben sie Ihre Grösse in cm an: ");
//      double groesse =sc.nextDouble();
//      System.out.print("Geben Sie Ihr Gewicht in kg an: ");
//      double gewicht=sc.nextDouble();
//
//      bmi=gewicht/groesse/groesse*10000;
//      System.out.printf("Ihr BMI ist: %.1f\n",bmi);
//      if (bmi<=18) {
//          System.out.println("Sie haben Untergewicht!Kontaktieren
Sie einen Arzt!");
//
//      } else if (bmi <=25) {

```

```

//          System.out.println("Alles in Ordnung!");
//      } else if (bmi <=30) {
//          System.out.println("Sie haben leichtes Übergewicht!");
//      } else if (bmi <=35) {
//          System.out.println("Sie haben Adipositas Stufe 1");
//      } else {
//          System.out.println("Sie haben Adipositas Stufe 2");
//          System.out.println("Kontaktieren Sie einen Arzt!");
//      }
    }
}

```

Package: _10_14

Klasse: BMIBerechnenAuswerten

```
package _10_14;
```

```
import java.util.Scanner;
```

```

/**
 * 1. Warum Methoden ?
 *
 * 2. Warum Klassen bzw OO?
 *
 * Modularisierung:
 *     - Komplexe Probleme werden in Teilprobleme zerlegt
 *     - Modularisierungseinheiten: Methoden, Klassen
 */
public class BMIBerechnenAuswerten {

    public static void main(String[] x) {
        double w = Person.readWeight();
        int h = Person.readHeight();

        Person p = new Person(w,h);

        double bmi = p.berechneBMI(); // w und h sind Argumente
        String evalText = p.evalBMI();

        double w2 = Person.readWeight();
        int h2 = Person.readHeight();

        Person p2 = new Person(w2, h2);

        double bmi2 = p2.berechneBMI();

        String evalText2 = p2.evalBMI();

        System.out.println("BMI (Person 1) = " + bmi + " - " +

```

```

evalText);
    System.out.println("BMI (Person 2) = " + bmi2+ " - " +
evalText2);

    String s = String.valueOf(true); //statische Methode

    String str = new String("Marley");
    char c = str.charAt(3); //Nicht-Statistische Methode
//    System.out.println(Math.round((Math.random() * 6 +0.5 )* 100.0
/ 100));
    }

```

```

}

```

Klasse: Klassenname

```

package _10_14;
/**
 * Modelliere eine Klasse die Punkte auf der Ebene darstellt
 *
 *
 */

/**
 * Klassenaufbau:
 *     class Klassenname
 *     {
 *     1.Attribute
 *     2.Konstruktoren
 *     3.Methoden
 *     }
 */
public class Geometrie {
    double xWert;
    double yWert;

    public static void main(String[] args) {
        Punkt p1=new Punkt(2.0,3.5);
        Punkt p2= new Punkt();
        Punkt p3= new Punkt(Punkt.readXValue(),Punkt.readYValue());

        p1.zeigePunkt();
        p1.berecheneAbstand(-3,15.5);
        p1.bestimmeGeradengleichung(4, 5.5);
        p1.bestimmeMitte(18, -13.5);
        p2.zeigePunkt(3.5,-4);
        p2.setxWert(3.5);
        p2.setyWert(-4);
        p2.zeigeZufallsPunkt();
    }
}

```

```
p2.zeigePunkt();
p2.bestimmeQuadrant(5, -0.01);
p3.zeigePunkt();
p3.berecheneAbstand(0, 0);
p3.bestimmeGeradengleichung(1, 1);
p3.bestimmeMitte(12.1, 22/7.0);
```

```
double flaeche = Punkt.berechneDreiecksflaeche(p1, p2, p3);
System.out.println("Fläche: " + flaeche);
```

```
double flaeche2 = p1.berechneDreiecksflaeche(p2, p3);
System.out.println("Fläche: " + flaeche2);
```

```
}
```

```
}
```



```

//
//public class Geometrie {
//    public static void main(String[] args) {
//        // Initialisiere Punkte
//        Punkt p1 = new Punkt(2.0, 3.5);
//        Punkt p2 = new Punkt();
//        Punkt p3 = new Punkt(Punkt.readXValue(),
//Punkt.readYValue());
//
//        // Teste bestehende Methoden
//        System.out.println("=== Punkt p1 ===");
//        p1.zeigePunkt();
//        System.out.printf("Abstand von p1 zu (-3, 15.5): %.2f\n",
//p1.berecheneAbstand(-3, 15.5));
//        System.out.println("Geradengleichung durch p1 und (4,
//5.5):");
//        p1.bestimmeGeradengleichung(4, 5.5);
//        System.out.println("Mittelpunkt zwischen p1 und (18, -
//13.5):");
//        p1.bestimmeMitte(18, -13.5);
//
//        System.out.println("\n=== Punkt p2 ===");
//        p2.zeigePunkt(3.5, -4);
//        System.out.println("Zufallspunkt:");
//        p2.zeigeZufallsPunkt();
//        p2.zeigePunkt(5, -0.01);
//        p2.bestimmeQuadrant(5, -0.01);
//
//        System.out.println("\n=== Punkt p3 ===");
//        p3.zeigePunkt();
//        System.out.printf("Abstand von p3 zu (0, 0): %.2f\n",
//p3.berecheneAbstand(0, 0));
//        System.out.println("Geradengleichung durch p3 und (1, 1):");
//        p3.bestimmeGeradengleichung(1, 1);
//        System.out.println("Mittelpunkt zwischen p3 und (12.1,
//22/7):");
//        p3.bestimmeMitte(12.1, 22/7.0);
//
//

```

```

//      // Teste Dreiecksflächen
//      System.out.println("\n=== Dreiecksflächen ===");
//      double flaeche = Punkt.berechneDreiecksflaeche(p1, p2, p3);
//      System.out.printf("Fläche des Dreiecks (p1, p2, p3)
(statisch): %.2f\n", flaeche);
//      double flaeche2 = p1.berechneDreiecksflaeche(p2, p3);
//      System.out.printf("Fläche des Dreiecks (p1, p2, p3) (nicht-
statisch): %.2f\n", flaeche2);
//
//      // Teste Transformationen
//      System.out.println("\n=== Transformationen für p1 ===");
//      p1.zeigePunkt();
//      p1.skaliere(2.0);
//      System.out.println("Nach Skalierung um Faktor 2:");
//      p1.zeigePunkt();
//      p1.verschiebe(1.5, -2.0);
//      System.out.println("Nach Verschiebung um (1.5, -2.0):");
//      p1.zeigePunkt();
//      p1.spiegeleAnXchse();
//      System.out.println("Nach Spiegelung an der x-Achse:");
//      p1.zeigePunkt();
//      p1.rotiereUmUrsprung(90.0);
//      System.out.println("Nach Rotation um Ursprung (90°):");
//      p1.zeigePunkt();
//      p1.rotiereUmPunkt(1.0, 1.0, 45.0);
//      System.out.println("Nach Rotation um (1, 1) um 45°:");
//      p1.zeigePunkt();
//
//      // Teste Abstandsberechnungen
//      System.out.println("\n=== Abstandsberechnungen ===");
//      double abstand = p1.berechneAbstandZuPunkt(p2);
//      System.out.printf("Abstand zwischen p1(%.2f/%.2f) und
p2(%.2f/%.2f): %.2f\n",
//      p1.xWert, p1.yWert, p2.xWert, p2.yWert,
abstand);
//      double abstandGerade = p1.berechneAbstandZuGerade(2.0, 1.0);
//      System.out.printf("Abstand von p1(%.2f/%.2f) zur Geraden y =
2x + 1: %.2f\n",
//      p1.xWert, p1.yWert, abstandGerade);
//
//      // Teste Geradengleichung und Mittelpunkt mit Punkt-Objekten
//      System.out.println("\n=== Geradengleichung und Mittelpunkt
===");
//      System.out.println("Geradengleichung durch p1 und p2:");
//      p1.bestimmeGeradengleichung(p2);
//      System.out.println("Mittelpunkt zwischen p1 und p3:");
//      p1.bestimmeMitte(p3);
//
//      // Teste Vektoraddition
//      System.out.println("\n=== Vektoraddition ===");

```

```

//      Punkt vektor = new Punkt(1.0, 2.0);
//      Punkt ergebnis = p1.addiereVektor(vektor);
//      System.out.printf("p1 + Vektor(%.2f/%.2f) = P(%.2f/%.2f)\n",
//      vektor.xWert, vektor.yWert,
ergebnis.xWert, ergebnis.yWert);
//
//      // Teste nächster Punkt
//      System.out.println("\n=== Nächster Punkt ===");
//      Punkt[] punkte = {new Punkt(1.0, 1.0), new Punkt(5.0, 5.0),
new Punkt(2.5, 3.0)};
//      Punkt naechster = p1.findeNaechstenPunkt(punkte);
//      System.out.printf("Nächster Punkt zu p1(%.2f/%.2f):
P(%.2f/%.2f)\n",
//      p1.xWert, p1.yWert, naechster.xWert,
naechster.yWert);
//    }
//}
/*
=== Punkt p1 ===
P(2.00/3.50)
Abstand von p1 zu (-3, 15.5): 12.50
Geradengleichung durch p1 und (4, 5.5):
die Geradengleichung lautet:  $y = 1.00 \cdot x + 1.50$ 
und schneidet die x-Achse mit  $45.00^\circ$ 
Mittelpunkt zwischen p1 und (18, -13.5):
Die Mitte ist: M(10.00/-5.00)

=== Punkt p2 ===
P(3.50/-4.00)
Zufallspunkt:
P(123.45/67.89)
P(5.00/-0.01)
Der Punkt liegt im vierten Quadranten!

=== Punkt p3 ===
P(3.00/4.00)
Abstand von p3 zu (0, 0): 5.00
Geradengleichung durch p3 und (1, 1):
die Geradengleichung lautet:  $y = -1.50 \cdot x + 8.50$ 
und schneidet die x-Achse mit  $-56.31^\circ$ 
Mittelpunkt zwischen p3 und (12.1, 3.14):
Die Mitte ist: M(7.55/3.57)

=== Dreiecksflächen ===
Fläche des Dreiecks (p1, p2, p3) (statisch): 2.50
Fläche des Dreiecks (p1, p2, p3) (nicht-statisch): 2.50

=== Transformationen für p1 ===
P(2.00/3.50)
Nach Skalierung um Faktor 2:

```

P(4.00/7.00)
Nach Verschiebung um (1.5, -2.0):
P(5.50/5.00)
Nach Spiegelung an der x-Achse:
P(5.50/-5.00)
Nach Rotation um Ursprung (90°):
P(5.00/5.50)
Nach Rotation um (1, 1) um 45°:
P(3.54/4.04)

=== Abstandsberechnungen ===

Abstand zwischen p1(3.54/4.04) und p2(0.00/0.00): 5.36
Abstand von p1(3.54/4.04) zur Geraden $y = 2x + 1$: 1.23

=== Geradengleichung und Mittelpunkt ===

Geradengleichung durch p1 und p2:
die Geradengleichung lautet: $y = 1.14 \cdot x + 0.00$
und schneidet die x-Achse mit 48.69°
Mittelpunkt zwischen p1 und p3:
Die Mitte ist: M(3.27/4.02)

=== Vektoraddition ===

p1 + Vektor(1.00/2.00) = P(4.54/6.04)

=== Nächster Punkt ===

Nächster Punkt zu p1(3.54/4.04): P(2.50/3.00)
*/package mi;

```
public class Gewinnmatrix {
    double[][] gewinne = new double[6][12];
    String[] abteilungen = {"Vertrieb", "Produktion", "IT", "HR",
"Marketing", "Finanzen"};
    String[] monate = {"Jan", "Feb", "Mrz", "Apr", "Mai", "Jun",
"Jul", "Aug", "Sep", "Okt", "Nov", "Dez"};

    // Konstruktor: Initialisiert fiktive Werte
    public Gewinnmatrix() {
        for (int i = 0; i < 6; i++) {
            for (int j = 0; j < 12; j++) {
                gewinne[i][j] = Math.round((Math.random() * 20000 -
10000) * 100.0) / 100.0; // Werte von -10.000 bis +10.000
            }
        }
    }

    // Maximum und Minimum finden
    public double[] findeMinMax() {
        double min = Double.MAX_VALUE;
        double max = Double.MIN_VALUE;
```

```

        for (double[] abt : gewinne) {
            for (double wert : abt) {
                if (wert < min) min = wert;
                if (wert > max) max = wert;
            }
        }
        return new double[]{min, max};
    }

    // Durchschnitt pro Abteilung
    public double[] durchschnittAbteilungen() {
        double[] durchschnitt = new double[6];
        for (int i = 0; i < 6; i++) {
            double summe = 0;
            for (int j = 0; j < 12; j++) {
                summe += gewinne[i][j];
            }
            durchschnitt[i] = summe / 12;
        }
        return durchschnitt;
    }

    // Ausgabe
    public void ausgabe() {
        System.out.printf("%15s", "");
        for (String monat : monate) System.out.printf("%10s", monat);
        System.out.println();
        for (int i = 0; i < 6; i++) {
            System.out.printf("%15s", abteilungen[i]);
            for (int j = 0; j < 12; j++) {
                System.out.printf("%10.2f", gewinne[i][j]);
            }
            System.out.println();
        }
    }
}

```

Klasse: Person

```

package _10_14;

import java.util.Scanner;

public class Person {
    double weight;
    int height;

    Person (double gewicht, int groesse){
        weight = gewicht;
        height = groesse;
    }
}

```

```

/**
 *
 * @param weight  Körpergewicht in kg
 * @param height  Körpergröße in cm
 * @return BMI
 */
double berechneBMI () { // weight und height sind Parameter

    return weight / (height / 100.0 * height / 100.0);
}

/**
 * Auswertung vom Body Mass Index
 */

String evalBMI () {
    double bmi = berechneBMI();

    if(bmi < 16)
        return "Kritisches Untergewicht";
    else if (bmi < 18.5)
        return "Untergewicht";
    else if (bmi < 25)
        return "Normalgewicht";
    else if (bmi < 30)
        return "Leichtes Übergewicht";

    return "Übergewicht";
}

//Übung: verwende eine alternative Schleife (for, do-while)
static double readWeight() {
    Scanner sc = new Scanner(System.in);
    System.out.print("Bitte dein Gewicht in kg:");
    double w = sc.nextDouble();
    while(w < 0 )
    {
        System.out.print("fehlerhafte Eingabe, bitte dein Gewicht
in kg:");
        w = sc.nextDouble();
    }
    return w;
}

static int readHeight() {
    Scanner sc = new Scanner(System.in);
    System.out.print("Bitte deine Größe in cm:");
    int h = sc.nextInt();
    while(h < 0 )

```

```

        {
            System.out.print("fehlerhafte Eingabe, bitte deine Größe
in cm:");
            h = sc.nextInt();
        }

        return h;
    }
}

```

Package: _10_14_lab

Klasse: BMIBerechnenUndAuswerten

```

package _10_14_lab;

import java.util.Scanner;

/**
 * 1. Warum Methoden
 *
 * 2. Warum Klassen bzw OO?
 *
 * Modularisierung:
 *     -komplexe Probleme werden in Teilprobleme(Prozeduren)
zerlegt
 *     -Modularisierungseinheiten:Methoden ,Klassen
 */
public class BMIBerechnenUndAuswerten {
    /**
     *
     * Berechnung BMI
     * berechneBMI
     */

    /**
     *
     * @param weight Körpergewicht in kg
     * @param height Körpergröße in cm
     * @return BMI
     */
    // static double berechneBMI(double weight, int height) { //weight und
height sind Parameter
    //     double h_meter=height/100.0;
    //     double index=weight/(h_meter*h_meter);
    //     return index;
    // }
    //
    // //Auswertung von Body Mass Index
    //
}

```

```

// static String evalBMI(double gewicht,int groesse) {
//     double bmi=berechneBMI(gewicht, groesse);
//     String evalText;
//     if (bmi<16)
//         evalText="kritisches Untergewicht";
//     else if (bmi<18.5)
//         evalText="Untergewicht";
//     else if (bmi<25)
//         evalText="Normalgewicht";
//     else if (bmi<30)
//         evalText="leichtes Übergewicht";
//     else
//         evalText="Übergewicht";
//     return evalText;
// }
// static String evalBMI(double bmi) {
//     String evalText;
//     if (bmi<16)
//         evalText="kritisches Untergewicht";
//     else if (bmi<18.5)
//         evalText="Untergewicht";
//     else if (bmi<25)
//         evalText="Normalgewicht";
//     else if (bmi<30)
//         evalText="leichtes Übergewicht";
//     else
//         evalText="Übergewicht";
//     return evalText;
// }
// }
//
// static double readWeight() {
//     Scanner sc = new Scanner(System.in);
//     System.out.print("Bitte dein Gewicht in kg: ");
//     double w =sc.nextDouble();
//     while(w<=0) {
//         System.out.print("fehlerhafte Eingabe, bitte dein Gewicht
in kg eingeben: ");
//         w =sc.nextDouble();
//     }
//     return w;
// }
// static double readHeight() {
//     Scanner sc = new Scanner(System.in);
//     System.out.print("Bitte deine Größe in cm: ");
//     double h =sc.nextDouble();
//     while(h<=0) {

```



```

//          System.out.print("fehlerhafte Eingabe, bitte deine Größe
in cm eingeben: ");
//          h =sc.nextDouble();
//      }
//      return h;
//  }

    public static void main(String[] args) {
        //Eingabe
        Scanner sc = new Scanner(System.in);
        double w =Person.readWeight();
        int h =Person.readHeight();
        double bmi=Person.berechneBMI(w, h);
        String evalText=Person.evalBMI(bmi);
//      //Gewicht
//      System.out.print("Bitte dein Gewicht in kg: ");
//      double w =sc.nextDouble();
//
//      //Größe
//      System.out.print("Bitte deine Größe in cm: ");
//      int h = sc.nextInt();
//
//      //Verarbeitung(Berechnung)
//      //BMI-Formel
//
//      // double bmi=w/(h/100.0*h/100.0);
//      double bmi=berechneBMI(w,h);// w und h sind Argumente
//      String evalText;
//      if (bmi<16)
//          evalText="kritisches Untergewicht";
//      else if (bmi<18.5)
//          evalText="Untergewicht";
//      else if (bmi<25)
//          evalText="Normalgewicht";
//      else if (bmi<30)
//          evalText="leichtes Übergewicht";
//      else
//          evalText="Übergewicht";
//
//      //Ausgabe
//      System.out.println("Dein BMI = "+bmi+" - "+evalText);
//
//      sc.close();
//  }
}

```

Klasse: Person

```
package _10_14_lab;

import java.util.Scanner;

public class Person {
    // double weight;
    // int height;

    static double berechneBMI(double weight, int height) { //weight und
height sind Parameter
        double h_meter=height/100.0;
        double index=weight/(h_meter*h_meter);
        return index;
    }

    //Auswertung von Body Mass Index

    static String evalBMI(double gewicht,int groesse) {
        double bmi=berechneBMI(gewicht, groesse);
        String evalText;
        if (bmi<16)
            evalText="kritisches Untergewicht";
        else if (bmi<18.5)
            evalText="Untergewicht";
        else if (bmi<25)
            evalText="Normalgewicht";
        else if (bmi<30)
            evalText="leichtes Übergewicht";
        else
            evalText="Übergewicht";
        return evalText;
    }

    static String evalBMI(double bmi) {

        String evalText;
        if (bmi<16)
            evalText="kritisches Untergewicht";
        else if (bmi<18.5)
            evalText="Untergewicht";
        else if (bmi<25)
            evalText="Normalgewicht";
        else if (bmi<30)
            evalText="leichtes Übergewicht";
        else
            evalText="Übergewicht";
        return evalText;
    }
}
```

```

    static double readWeight() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Bitte dein Gewicht in kg: ");
        double w =sc.nextDouble();
        while(w<=0) {
            System.out.print("fehlerhafte Eingabe, bitte dein Gewicht
in kg eingeben: ");
            w =sc.nextDouble();
        }
        return w;
    }
    static int readHeight() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Bitte deine Größe in cm: ");
        int h =sc.nextInt();
        while(h<=0) {
            System.out.print("fehlerhafte Eingabe, bitte deine Größe
in cm eingeben: ");
            h =sc.nextInt();
        }
        return h;
    }
}

```

Package: _10_15

Klasse: IO

```
package _10_15;
```

```
import java.util.Scanner;
```

```

/**
 * Lernziel:
 *          - Attribute
 *          - Konstruktoren
 *          - Methoden
 *          - Statische und Nicht-Statische Mitglieder einer Klasse
 */
public class IO {
    /**
     * Definiere folgende Methoden:
     *
     * 1. readInt:
     *          - liest eine ganze Zahl aus der Tastatur und gibt
den eingelesenen Wert für den Aufrufer zurück
     *          - Bei der Eingabeaufforderung soll ein benutzer-
freundlicher Text für den User angezeigt werden

```

```

    * 2. readDouble:
    *           -liest eine Fließkomma Zahl aus der Tastatur und
gibt den eingelesenen Wert für den Aufrufer zurück
    *           - Bei der Eingabeaufforderung soll ein benutzer-
freundlicher Text für den User angezeigt werden
    */

```

```

    public static int readInt(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextInt();
    }
    public static double readDouble(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextDouble();
    }
    public static String readString(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextLine();
    }
}

```

Klasse: Punkt

```
package _10_15;
```

```

public class Punkt {
    // Attribute = Unterscheidungsmerkmalen
    double x;
    double y;

    // Konstruktoren
    public Punkt(double xk, double yk) {
        x = xk;
        y = yk;
    }

    public Punkt() {
        x = Math.random();
        y = Math.random();
    }

    /**
     *
     * Definiere eine Methode, die den Punkt als Text zurück gibt in
der Format (x,
     * y)
     *

```

```

    * Beispiel:
    *
    * Wenn der Punkt aus 0.5 und 0.3 besteht, dann der Text, was
zurück gegeben
    * wird, lautet: (0., 0.3)
    */

    public String convertToString() {
        return "(" + x + " | " + y + ")";
    }

    /**
    * Definiere eine Methode, die überprüft, ob der Punkt der
Ursprungspunkt ist (Origin = 0, 0)
    */
    public boolean istUrsprung() {
        return x == 0 && y == 0;
    }

    /**
    * Definiere eine Methode, die überprüft, ob der Punkt auf dem X-
Axis liegt
    */
    public boolean istAufDemXAxis() {
        return y == 0;
    }

    /**
    * Definiere eine Methode, die den Punkt längst X-Axis um einen
gegeben Betrag verschiebt
    *
    * Definiere eine Methode, die den Punkt längst Y-Axis um einen
gegebenen Betrag verschiebt
    *
    * Definiere eine Methode, die den Punkt längst X- und Y-Axis um
zwei entsprechend gegebene Beträge
    * verschiebt
    */

    public void shiftLaengstX(double dx) {
        //x = x + dx;
        x += dx;
    }
    public void shiftLaengstY(double dy) {
        y += dy;
    }

    public void shift(double dx, double dy) {

        shiftLaengstX(dx);

```

```

        shiftLaengstY(dy);
    }

    /**
     * Definiere eine Methode, die den Abstand des Punktes zum
     Ursprungspunkt berechnet
     *
     * Es soll auch möglich sein Abstand zwischen beliebigen Punkten
     zu berechnen
     *
     * Formel für die Berechnung von Abstand zwischen Punkten - sehe
     wikipedia
     */

    public double berechneAbstandZumUrsprung() {
        //return Math.sqrt(x * x + y * y);
        return berechneAbstandZumPunkt(new Punkt(0.0, 0.0));
    }
    public double berechneAbstandZumPunkt(Punkt p) {

        return Math.sqrt(Math.pow(x - p.x, 2) + Math.pow(y - p.y, 2));
    }
    /**
     * Methoden
     */

    public void printFormatted() {
        System.out.printf("( %.2f | %.2f)",x,y);
    }

}package _10_15_lab;

import java.util.Scanner;
public class Punkt {
    //Attribute
    double x;
    double y;

    //Konstruktoren
    public Punkt(double xk,double yk) {
        x=xk;
        y=yk;
    }

    public Punkt() {
        x=Math.random();
        y=Math.random();
    }
}

```

```

//Methoden
static String fuegePunktZusammen(double xf,double yf) {
    return "("+xf+"|"+yf+")";
}
static String fuegePunktZusammen(Punkt p) {
    return "("+p.x+"|"+p.y+")";
}

public String convertToString() {
    return "("+x+"|"+y+")";
}

public boolean istUrsprung() {
    return x==0&&y==0;
}
public boolean istAufDemXAxis() {
    return y==0;
}

public Punkt verschiebeXRichtung(double a) {
    x+=a;
    return new Punkt();
}
public Punkt verschiebeYRichtung(double b) {
    y+=b;
    return new Punkt();
}

public Punkt verschiebePunkt(double a, double b) {
    x+=a;
    y+=b;
    return new Punkt();
}

public void shiftLaengsX(double dx) {
    x+=dx;    //x=x+dx;
}

public void shiftLaengsY(double dy) {
    y+=dy;    //y=y+dy;
}

public void shift(double dx, double dy) {
    x+=dx;
    y+=dy;
}
// oder:
// shiftLaengsX(dx);
// shiftLaengsY(dy);
}

```

```

    public double berechneAbstand() {
        //return Math.sqrt(x*x+y*y);
        return berechneAbstand(new Punkt(0.0,0.0));
    }

    public double berechneAbstand(double a, double b) {
        return Math.sqrt((x-a)*(x-a)+(y-b)*(y-b));
    }
    public double berechneAbstand(Punkt p) {
        return Math.sqrt((x-p.x)*(x-p.x)+(y-p.y)*(y-p.y));
        //return Math.sqrt(Math.pow(x-p.x,2)+Math.pow(y-p.y,2));
    }

}

Klasse: Test
package _10_15;

public class Test {
    public static void main(String[] args) {

        // erzeuge ein Objekt der Klasse Punkt
        Punkt pkt = new Punkt(IO.readDouble("x-koordinate: "),
        IO.readDouble("y-koordinate: ")); // rufe einen Konstruktor der Klasse
        Punkt

        Punkt p = new Punkt();
        if(p.istUrsprung())
            System.out.println("p liegt auf dem Kreuzpunkt");
        else
            System.out.println("p liegt nicht auf dem Kreuzpunkt");

        String s = pkt.convertToString();
        String s2 = p.convertToString();

        System.out.println("pkt = " + s);
        System.out.println("p = " + s2);

        //      pkt.shiftLaengstX(0.75);
        //      pkt.shiftLaengstY(0.33);

        pkt.shift(0.75, 0.33);

        System.out.println("pkt = " + pkt.convertToString()); //pkt =
(4.25 | 3.23)

        double pkt_zu_p = pkt.berechneAbstandZumPunkt(p);
        System.out.println(pkt_zu_p);
    }
}

```



```

        double pkt_zu_u = pkt.berechneAbstandZumUrsprung();
        System.out.println(pkt_zu_u);

    }
}

```

Package: `_10_15_lab`

Klasse: `Kreis`

```

package _10_15_lab;
/**
 * Kreise in der Ebene
 *
 * Hinweis:
 *     ein Kreis wird durch Radius und Mittelpunkt beschrieben
 *     Attribute identifizieren und deklarieren
 *     sinnvolle Konstruktoren definieren
 *     Methoden definieren
 * Was kann man mit Kreisobjekten machen?
 *     Fläche, Umfang berechnen
 *     Vergrössern/vergleichen
 *     Überprüfen ob ein gegebener Punkt innerhalb des Kreises
 *     Überprüfen ob Einheitskreis(mit 0 als Mittelpunkt)
 *
 */
public class Kreis {

    double r;
    Punkt m;

    public Kreis(double ra, Punkt p) {
        if (ra <= 0) {
            throw new IllegalArgumentException("Der Radius muss
positiv sein!");
        }
        r=ra;
        m=p;
    }
    public double getR() {
        return r;
    }
    public void setR(double r) {
        if(r>0)
            this.r = r;
    }
}

```

```

        else
            System.out.println("Der Radius ist positiv!!");
    }
    public Punkt getM() {
        return m;
    }
    public void setM(Punkt m) {
        this.m = m;
    }

    public double berechneFlaeche() {
        return Math.PI*Math.pow(r,2);
    }
    public double berechneUmfang() {
        return 2*r*Math.PI;
    }
    public void veraenderGroesse(double faktor) {
        if(faktor<=0)
            System.out.println("Nicht möglich!");
        else
            r *=faktor;        //r=r*faktor;
    }
    public void verschiebeKreis(double dx, double dy) {
        m.x+=dx;
        m.y+=dy;
    }
    public boolean istInnerhalb(Punkt p) {
        return m.berechneAbstand(p)<r;
    }
    public boolean istEinheitskreis() {
        return m.x==0&&m.y==0&&r==1;
    }
}

```

Klasse: Test

```
package _10_15_lab;
```

```

public class Test {

    public static void main(String[] args) {
        //erzeuge ein Objekt der Klasse Punkt
        Punkt pkt=new Punkt(IO.readDouble("x-Koordinate:
"),IO.readDouble("y-Koordinate: "));
        Punkt p =new Punkt();
        Punkt p1=new Punkt(4,3);
        Punkt p2=new Punkt(-1,-9);
        if(p.istUrsprung())
            System.out.println("p liegt im Ursprung");
        else
            System.out.println("p liegt nicht im Ursprung");
    }
}

```

```

        System.out.println("pkt("+pkt.x+"/"+pkt.y+")");
        System.out.println("p("+p.x+"/"+p.y+")");
        System.out.println("pkt"+pkt.convertToString());
//      pkt.verschiebePunkt(1.5, 3.1);
//      pkt.shiftLaengsX(0.75);
//      pkt.shiftLaengsY(0.33);

        pkt.shift(0.75, 0.33);
        System.out.println("pkt"+pkt.convertToString());

        System.out.printf("Der Abstand von %s zum Ursprung ist: %.2f\n",p1.convertToString(),p1.berechneAbstand());
        System.out.printf("Der Abstand von %s zum Punkt %s ist: %.2f\n",p1.convertToString(),p2.convertToString(),p1.berechneAbstand(p2));

        Punkt mittelpunkt=new Punkt(3.0,4.0);
        Kreis kreis1=new Kreis(5.0,mittelpunkt);
        System.out.println("Kreis erstellt mit Radius: " +
kreis1.getR() + ", Mittelpunkt: "+ kreis1.m.convertToString( ));
        System.out.printf("Fläche von Kreis 1: %.2f\n" ,
kreis1.berechneFlaeche());
        System.out.printf("Umfang von Kreis 1: %.2f\n" ,
kreis1.berechneUmfang());
        kreis1.veraenderGroesse(2.0);
        kreis1.verschiebeKreis(1.0, -2.0);
        String text2 = kreis1.istInnerhalb(new Punkt(0.0,0.0)) ? "Der
Punkt ist innerhalb": "Der Punkt ist nicht innerhalb";
        System.out.println(text2);
        Kreis einheitskreis = new Kreis(1.0, new Punkt(0.0,
0.0));System.out.println("Kreis erstellt mit Radius: " +
einheitskreis.getR() + ", Mittelpunkt: "
+einheitskreis.m.convertToString());
        String text=einheitskreis.istEinheitskreis() ? "Der Kreis ist
der Einheitskreis!": "Der Kreis ist nicht der Einheiskreis!";
        System.out.println(text);
        String text3 = einheitskreis.istInnerhalb(new Punkt(0.6,-0.8))
? "Der Punkt ist innerhalb": "Der Punkt ist nicht innerhalb";
        System.out.println(text3);

    }

}

```

Package: _10_16

Klasse: Kreis

```
package _10_16;

import _10_15.IO;
import _10_15.Punkt;

/**
 * Modelliert Kreise auf der Ebene
 *
 *
 * Hinweis:
 *     - Ein Kreis wird durch Radius und Mittelpunkt beschrieben
 *
 * TODO:
 *     - Attribute identifizieren und entsprechend deklarieren
 *     - Sinnvolle Konstruktoren definieren
 *     - Methoden definieren
 *
 * Was kann man mit Kreis Objekte machen?
 *
 *     - Flächeninhalt berechnen
 *     - Umfang berechnen
 *     - Vergrößern oder verkleinern
 *     - Kreis verschieben
 *     - Überprüfen, ob ein gegebener Punkt innerhalb des Kreises
    sich befindet.
 *     - Überprüfen, ob sich um einen Einheitskreis handelt.
 *
 * Was ist ein Einheitskreis?
 *     - Wenn der Radius = 1.0 und der
    Mittelpunkt = (0.0, 0.0)
 */

/**
 * Aufgabe eines Konstruktors:
 *
 *     - Attribute initialisieren
 *     - Korrektheit (valid)
 */
public class Kreis {
    private double radius;
    Punkt mittelpunkt;

    /**
     * Einheitskreis
     */
    public Kreis() {
        this(1.0, new Punkt(0.0, 0.0)); // Sprung: Kreis (double,
    Punkt)
    }
}
```

```

    }
    public Kreis(double r) {
//        if(r > 0)
//            radius = r;
//        else
//            {
//                System.out.println("Radius darf nicht <= 0 sein, deshalb
auf Standardwert = 1.0 gesetzt");
//                radius = 1.0;
//            }
//        mittelpunkt = new Punkt(0.0, 0.0);
//        this(r,new Punkt(0.0, 0.0));
    }

    public Kreis(double r, Punkt mp) {
        if(r > 0)
            radius = r;
        else
            {
                System.out.println("Radius darf nicht <= 0 sein, deshalb
auf Standardwert = 1.0 gesetzt");
                radius = 1.0;
            }
        mittelpunkt = mp;
    }

    public Kreis(double r, double xc, double yc) {
//        if(r > 0)
//            radius = r;
//        else
//            {
//                System.out.println("Radius darf nicht <= 0 sein, deshalb
auf Standardwert = 1.0 gesetzt");
//                radius = 1.0;
//            }
//        mittelpunkt = new Punkt(xc, yc);
//        this(r, new Punkt(xc, yc));
    }

    //Formel  $A = \pi * r^2$ 
    public double berechneFlaeche() {

        return Math.PI * Math.pow(radius, 2);
    }

    //Formel  $A = 2 * \pi * r$ 

    public double berechneUmfang() {
        return 2 * Math.PI * radius;
    }
}

```

```

    public void skaliere(double sf) {
        if(sf > 0)
            radius *= sf;
        else
            System.out.println("Kreis wurde nicht skaliert,
ungültiger Faktor: " + sf);
    }

    public void shift(double dx, double dy) {
        mittelpunkt.shift(dx, dy);
    }

    public boolean istPunktImKreis(Punkt p) {
        return mittelpunkt.berechneAbstandZumPunkt(p) < radius;
    }

    public boolean istEinheitskreis() {
        return mittelpunkt.istUrsprung() && radius == 1.0;
    }

    public String convertToString() {
        return mittelpunkt.convertToString() + ", " + radius;
    }

    public void printFormatted() {
        mittelpunkt.printFormatted();
        System.out.printf(", %.3f %n", radius);
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double r) {
        if(r > 0)
            radius = r;
        else
            System.out.println("Der Radius darf nicht mit negativem
Wert überschrieben werden: " + r);
    }
}

```

Klasse: Test

```

package _10_16;

import _10_15.IO;
import _10_15.Punkt;

```

```

public class Test {
public static void main(String[] args) {
    /**
     * Aufgabe 1.
     *
     * 1.a. Erzeuge einen Punkt mit zwei zufällige Zahlen
     * 1.b Erzeuge einen Kreis. Der Mittelpunkt ist der Punkt, der in
1.a. erzeugt wurde und der Radius
     *      soll aus der Tastatur eingelesen werden.
     *
     * 1.c Skaliere den Kreis, den du unter 1.b erzeugt hast und
verschiebe den Kreis längst x Axis um 0.5.
     *      Der Skalierungsfaktor soll aus der Tastatur eingelesen
Werden.
     *
     * 1.d. Erzeuge einen Kreis. Der Radius ist zweifache des Radiuses
vom Radius des Kreises, der in 1.b erzeugt wurde.
     *      Was ist mit dem Mittelpunkt???
     *
     * Wieviele Punkt und Kreis Objekte haben wir im Programm jetzt?
     */

    //a
    Punkt pt = new Punkt();
    //b
    Kreis k = new Kreis(IO.readDouble("Radius: "), pt);
    System.out.println(k);
    //c
    k.skaliere(IO.readDouble("Faktor: "));
    k.shift(0.5, 0);

    //d
    Kreis k2 = new Kreis(k.getRadius() * 2);
    //

    // 2 Punkt Objekte, 2 Kreis Objekte

    k.printFormatted();
    k2.printFormatted();
    pt.printFormatted();

    //k.radius = -1.5;
    k.setRadius(-1.5);
    k.printFormatted();
}
}

```

Klasse: Uebung

```
package _10_16;

public class Uebung {

}

/**
 * Lernziel:
 *          - Konstruktoren:
 *          - Werteüberprüfung, Delegation
 * (this(...)), Überladen
 *          - Getters und Setters
 *          - Validität von Objekte
 *          - Code Redundanz vermeiden
 *          - Sichtbarkeit: private, default, public
 *
 * Aufgabe:
 *          - Definiere eine Klasse, die Datum repräsentiert
 * (16.10.2025 oder 03.10.1989 ...)
 *
 * Vorgehensweise:
 *          - Denkbare Methode implementieren
 *          - Mit Objekte der Klassen in der main Methode
 * arbeiten.
 */
```

Package: _10_16_lab

Klasse: Datum

```
package _10_16_lab;

public class Datum {

    int tag;
    int monat;
    int jahr;

    public Datum(int t, int m , int j) {
        jahr=j;
        if(m>0 &&m<=12)
            monat=m;
        else {
            System.out.println("Ungültiger Monat, Monat auf 01
gesetzt!");
        }
        if(t>0 && t<=berechneTagesgrenze(m,j))
            tag=t;
        else {
            System.out.println("Ungültiger Tag,Tag auf 01 gesetzt!" );
        }
    }
}
```



```

        tag=1;
    }
}

public void printFormatted() {
    if(tag>9&&monat>9)
        System.out.printf("Das Datum ist: %2.0f.%2.0f.%4.0f\n",
(float)(tag),(float)(monat),(float)jahr);
    else if(tag<10&&monat>9)
        System.out.printf("Das Datum ist: 0%1.0f.%2.0f.%4.0f\n",
(float)(tag),(float)(monat),(float)jahr);
    else if(tag>9&&monat<10)
        System.out.printf("Das Datum ist: %2.0f.0%1.0f.%4.0f\n",
(float)(tag),(float)(monat),(float)jahr);
    else
        System.out.printf("Das Datum ist: 0%1.0f.0%1.0f.%4.0f\n",
(float)(tag),(float)(monat),(float)jahr);
}

public boolean istSchaltjahr(int j) {
    if (j%4==0) {
        if (j%100==0) {
            if (j%400==0)
                return true;
            else
                return false;
        }
        return true;
    }else
        return false;
}

public int berechneTagesgrenze(int m,int j) {
    switch(m) {
        case 1,3,5,7,8,10,12:{ return 31;}
        case 4,6,9,11: { return 30;}

        default :if (istSchaltjahr(j))
                    return 29;
                else
                    return 28;
    }
}

public static void main(String[] args) {
    Datum d1=new Datum(1,1,1901);
    d1.printFormatted();
    Datum d2=new Datum(29,2,2024);
}

```

```

        d2.printFormatted();
        Datum d3=new Datum(31,12,2026);
        d3.printFormatted();
        Datum d4=new Datum(29,2,2025);
        d4.printFormatted();
    }

```

```

}

```

Klasse: Kreis

```

package _10_16_lab;

```

```

import _10_15.Punkt;
import _10_15_lab.I0;

```

```

/**
 * Aufgabe eines Konstruktors:
 *                               -Attribute initialisieren
 *                               -Korrektheit (valid)
 */

```

```

public class Kreis {

```

```

    private double radius;
    Punkt mittelpunkt;

```

```

    public Kreis(double r, Punkt mp) {
        if (r>0)
            radius=r;
        else {
            System.out.println("Radius darf nicht <=0 sein,deshalb auf
Standardwert=1.0 gesetzt");
            radius=1.0;
        }
        mittelpunkt=mp;
    }

```

```

    /**
     * Einheitskreis
     */

```

```

    public Kreis() {
        this(1.0,new Punkt(0.0,0.0));
    }

```

```

    public Kreis(double r) {
        this(r,new Punkt(0.0,0.0));
    }

```

```

    public Kreis(double r, double xc,double yc) {
        this(r,new Punkt(xc,yc));
    }

```

```

//      if (r>0)
//          radius=r;
//      else {
//          System.out.println("Radius darf nicht <=0 sein,deshalb auf
Standardwert=1.0 gesetzt");
//          radius=1.0;
//      }
//      mittelpunkt=new Punkt(xc,yc);

}

```

```

    public static void createOblects() {
        Kreis k1= new Kreis();           //radius:1.0,mittelpunkt:
(0.0|0.0)
        Kreis k2= new Kreis(1.5);       //radius:1.5,mittelpunkt:
(0.0|0.0)

        Kreis k3= new Kreis(2.5,new Punkt(0.5,0.75));
        k3=new Kreis(IO.readDouble("Bitte Radius: "),new
Punkt(IO.readDouble("x: "),IO.readDouble("y: ")));

        Kreis k4=new Kreis(1.75,0.7,0.8);
        k4=new Kreis(IO.readDouble("Radius: "),IO.readDouble("x:
"),IO.readDouble("y: "));

        double a=IO.readDouble("Radius: ");
        double b=IO.readDouble("x: ");
        double c=IO.readDouble("y: ");

        k4=new Kreis(a,b,c);

    }
    public void bestaetigeKreis() {
        System.out.println("Kreis mit Radius: " + radius + ",
Mittelpunkt: "+ mittelpunkt.convertToString( ));
    }

    //Formel A =  $\pi * r^2$ 

    public double berechneFlaeche() {
        return Math.PI*Math.pow(radius, 2);
    }

    //Formel U =  $2\pi * r$ 

```

```

    public double berechneUmfang() {
        return 2*Math.PI*radius;
    }
    public void skaliere(double sf) {
        if (sf>0)
            radius *=sf;
        else
            System.out.println("Kreis wurde nicht skaliert!Ungültiger
Faktor: "+sf);
    }
    public void shift(double dx, double dy) {
        mittelpunkt.shift(dx, dy);
    }
    public boolean istPunktImKreis(Punkt p) {
        return mittelpunkt.berechneAbstandZumPunkt(p)<radius;
    }
    public boolean istEinheitskreis() {
        return mittelpunkt.istUrsprung() && radius==1;
    }
    public String convertToString() {
        return mittelpunkt.convertToString()+"", "+radius;
    }
    public void printFormatted() {
        mittelpunkt.printFormatted();
        System.out.printf(", r= %.3f\n",radius);
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double r) {
        if (r>0)
            radius=r;
        else {
            System.out.println("Radius muss positiv sein!");
        }
    }
}

```

Klasse: Test

```

package _10_16_lab;

import _10_15.Punkt;
import _10_15_lab.I0;

public class Test {

    public static void main(String[] args) {

```

```

//      Kreis kreis1=new Kreis(2.7,-3.0,4.0);
//      Kreis kreis2=new Kreis(5.0);
//      Kreis kreis3=new Kreis();
//      Punkt mitte=new Punkt(8.0,6.0);
//      Kreis kreis4=new Kreis(10,mitte);
//      Kreis kreis5=new Kreis(13,new Punkt(-5.0,5.0));
//
//      System.out.println("Kreis  erstellt mit Radius: " +
kreis1.radius + ", Mittelpunkt: "+ kreis1.mittelpunkt.convertToString(
));
//      System.out.println("Kreis  erstellt mit Radius: " +
kreis2.radius + ", Mittelpunkt: "+ kreis2.mittelpunkt.convertToString(
));
//      System.out.println("Kreis  erstellt mit Radius: " +
kreis3.radius + ", Mittelpunkt: "+ kreis3.mittelpunkt.convertToString(
));
//      System.out.println("Kreis  erstellt mit Radius: " +
kreis4.radius + ", Mittelpunkt: "+ kreis4.mittelpunkt.convertToString(
));
//      System.out.println("Kreis  erstellt mit Radius: " +
kreis5.radius + ", Mittelpunkt: "+ kreis5.mittelpunkt.convertToString(
));

      Punkt mitte = new Punkt();
      Kreis k1 = new Kreis(I0.readDouble("Radius: "),mitte);
//      k1.bestaetigeKreis();

      double faktor=I0.readDouble("Skalierungsfaktor: ");
      k1.skaliere(faktor);
      //k1.mittelpunkt.shiftLaengstX(0.5);
      k1.shift(0.5, 0);
//      k1.bestaetigeKreis();

      Kreis k2=new Kreis(k1.getRadius()*2.0);// /faktor
//      k2.bestaetigeKreis();
      k1.printFormatted();
      k2.printFormatted();

//      k1.radius=-1.5;
      k1.setRadius(-1.5);
      k1.printFormatted();

    }

}

```

Klasse: Uebung

```
package _10_16_lab;

public class Uebung {

    /**
     * Lernziel:
     *          -Konstruktoren
     *          -Werteüberprüfung, Delegation(this(...))
     *          -Getter/Setter
     *          -Validität von Objekten
     *          -Code Redundanz vermeiden
     *          -Sichtbarkeit: private,default, public
     * Aufgabe
     *          definiere eine Klasse ,die ein Datum
representiert(16.01.2025 oder 03.10.1989...)
     */

}
```

Package: _10_17

Klasse: Datum

```
package _10_17;

import _10_15.IO;

/**
 * TODO:
 *      - Die Klasse Datum soll mit Setters ergänzt werden.
 */

public class Datum {
    private int tag; // tag des Monats (1...28/29/30/31)
    private int monat; // jan(1) ... Dez(12)
    private int jahr; // 2025. 1981, 1970 ...

    public Datum(int tag, int monat, int jahr) {

        this.jahr = jahr;

        if (monat > 0 && monat < 13)
            this.monat = monat;
        else
        {
            System.out.println("Ungültiger Monat: " + monat);
            this.monat = Helper.readMonat();
        }
    }
}
```

```

        int n = Helper.berechneLaengeVomMonat(this.jahr, this.monat);

        if(tag > 0 && tag <= n)
            this.tag = tag;

        else
        {
            System.out.println("Ungültiger Tag: " + tag);
            this.tag = Helper.readTag(n);
        }

    }

    public int getTag() {
        return tag;
    }

    public int getMonat() {
        return monat;
    }

    public int getJahr() {
        return jahr;
    }

    public void setTag(int tag) {
        if(tag < 0 || tag > Helper.berechneLaengeVomMonat(this.jahr,
this.monat))
            System.out.println("Führt zum ungültigen Datum: " + tag +
"." + monat + "." + jahr);
        else
            this.tag = tag;
    }

    public void setMonat(int monat) {
        if(monat < 0 || monat > 12 || tag >
Helper.berechneLaengeVomMonat(this.jahr, monat))
            System.out.println("Führt zum ungültigen Datum: " + tag +
"." + monat + "." + jahr);
        else
            this.monat = monat;
    }

    public void setJahr(int jahr) {

        if(tag > Helper.berechneLaengeVomMonat(jahr, this.monat))

```

```

        System.out.println("Führt zum ungültigen Datum: " + tag +
"." + monat + "." + jahr);
    else
        this.jahr = jahr;
}

```

```

}

```

Klasse: Helper

```

package _10_17;

```

```

import _10_15.IO;

```

```

public class Helper {

```

```

    /*
    * Aufgabe:
    *         - Definiere folgende Methoden:
    *
    *
    * 1. asDE
    *         - Gibt einen Text zurück für ein gegebenes Datum und
    zwar Datum in deutscher Format formatiert.
    *         -TT.MM.JJJJ
    *
    *         Beispiel: 19.01.1981, 07.10.2025, 25.122025
    *2. asUS:
    *         -Gibt einen Text zurück für ein gegebenes Datum und
    zwar Datum in US-Format formatiert:
    *         MM/TT/JJJJ
    *3. asISO:
    *         -Gibt einen Text zurück für ein gegebenes Datum und
    zwar in ISO-Format:
    *         JJJJ-MM-TT
    *
    * Hinweis:
    *         Geht davon aus, dass Jahr immer eine vier-stellige
    Zahl ist.
    */
    public static int readMonat() {
        int monat = IO.readInt("Monat: ");
        while(monat < 1 || monat >12){
            System.out.println("nochmal versuchen: " + monat);
            monat = IO.readInt("Monat: ");
        }
        return monat;
    }
}

```



```

public static int readTag(int max) {
    int tag = IO.readInt("Tag: ");
    while (tag < 1 || tag > max)
    {
        System.out.print("noch mal versuchen... ");
        tag = IO.readInt("Tag: ");
    }
    return tag;
}

public static boolean istSchaltjahr(int jahr) {
    return (jahr % 4 == 0 && jahr % 100 != 0) || jahr % 400 == 0;
}

/**
 *
 * @param jahr
 * @param monat
 * @return 28, 29 , 30 31 oder -1 im Fehler Fall
 */
public static int berechneLaengeVomMonat(int jahr, int monat) {

    switch (monat) {
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
            return 31;
        case 4:
        case 6:
        case 9:
        case 11:
            return 30;
        case 2:
            return istSchaltjahr(jahr) ? 29 : 28; // Ternary
Conditional Operator

        default:
            return -1;
    }

}

}

```

Klasse: Test

```
package _10_17;

import java.time.LocalDate;

public class Test {
    public static void main(String[] args) {

        Datum d = new Datum(12, 3, 2024);
        System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 31.3.2024
        d.setMonat(-5);
        System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 31.5.2024
        d.setMonat(6); // -> 31.6.2024
        System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 31.5.2024

        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 29.2.2024
        //      d.setJahr(2028);
        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 29.2.2028
        //      d.setJahr(2025);
        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 29.2.2025
        //
        //      d = new Datum(19,1,1981);
        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 19.1.1981
        //      d.setJahr(1999);
        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); ;
        //      d.setJahr(2029);
        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); ;

        //      d = new Datum(6, 13, 2025);
        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr()); // 0.10. 2025
    }
}
```

Package: _10_17_1

Klasse: Datum

```
package _10_17_1;
```

```

import _10_15.IO;

/**
 * TODO:
 *      - Die Klasse Datum soll mit Setters ergänzt werden.
 */

public class Datum {
    private int tag; // tag des Monats (1...28/29/30/31)
    private int monat; // jan(1) ... Dez(12)
    private int jahr; // 2025. 1981, 1970 ...

    public Datum(int tag, int monat, int jahr) {

        this.jahr = jahr;

        if (monat > 0 && monat < 13)
            this.monat = monat;
        else
        {
            System.out.println("Ungültiger Monat: " + monat);
            this.monat = Helper.readMonat();
        }

        int n = Helper.berechneLaengeVomMonat(this.jahr, this.monat);

        if(tag > 0 && tag <= n)
            this.tag = tag;

        else
        {
            System.out.println("Ungültiger Tag: " + tag);
            this.tag = Helper.readTag(n);
        }

    }

    public int getTag() {
        return tag;
    }

    public int getMonat() {
        return monat;
    }

    public int getJahr() {
        return jahr;
    }
}

```

```

        public void setTag(int tag) {
            if (tag < 0 || tag > Helper.berechneLaengeVomMonat(this.jahr,
this.monat))
                System.out.println("Führt zum ungültigen Datum:
"+Helper.asDE(tag, this.monat, this.jahr));
            else
                this.tag=tag;
        }

        public void setMonat(int monat) {
            if (monat<0 || monat>12||
tag>Helper.berechneLaengeVomMonat(this.jahr, monat))
                System.out.println("Führt zum ungültigen Datum:
"+Helper.asDE(this.tag, monat, this.jahr));
            else
                this.monat = monat;
        }

        public void setJahr(int jahr) {
//            if(this.monat==2 && this.tag ==29 && !
Helper.istSchaltjahr(jahr))
                if (tag>Helper.berechneLaengeVomMonat(jahr, this.monat))
                    System.out.println("Führt zum ungültigen Datum:
"+Helper.asDE(this.tag, this.monat, jahr));
            else
                this.jahr=jahr;
        }
    }

```

Klasse: Helper

```
package _10_17_1;
```

```
import _10_15.IO;
```

```

public class Helper {
    /**
     * Aufgabe:
     * definiere folgenede Methoden
     * 1.asDE
     * -gibt ein Text zurück für ein gegebenes Datum in deutscher
Form formatiert TT.MM.JJJJ
     * (zb.19.01.1981, 07.10.2025, 25.12.2025)
     *
     * 2.asUS:
     * -gibt ein Text zurück für ein gegebenes Datum in amerikanischer
Form formatiert MM/TT/JJJJ
     *

```

```

*
* 3.asISO
* -gibt ein Text zurück für ein gegebenes Datum in ISO Form
formatiert JJJJ-MM-TT
*
* Hinweis:
* Geht davon aus,dass Jahr immer eine 4-stellige Zahl ist.
*
* @return
*/
public static String asDE(int tag,int monat,int jahr) {

    if (tag<10 &&monat<10)
        return "0"+tag+".0"+monat+"."+jahr;
    else if(tag<10 &&monat>9)
        return "0"+tag+"."+monat+"."+jahr;
    else if(tag>9 &&monat<10)
        return tag+".0"+monat+"."+jahr;
    else
        return tag+"."+monat+"."+jahr;
}
public static String asUS(int tag,int monat,int jahr) {

    if (tag<10 &&monat<10)
        return "0"+monat+"/0"+tag+"/"+jahr;
    else if(tag<10 &&monat>9)
        return monat+"/0"+tag+"/"+jahr;
    else if(tag>9 &&monat<10)
        return "0"+monat+"/"+tag+"/"+jahr;
    else
        return monat+"/"+tag+"/"+jahr;
}
public static String asISO(int tag,int monat,int jahr) {

    if (tag<10 &&monat<10)
        return jahr+"-0"+monat+"-0"+tag;
    else if(tag<10 &&monat>9)
        return jahr+"-"+monat+"-0"+tag;
    else if(tag>9 &&monat<10)
        return jahr+"-0"+monat+"-"+tag;
    else
        return jahr+"-"+monat+"-"+tag;
}

```

```

public static int readMonat() {
    int monat = IO.readInt("Monat: ");
    while(monat < 1 || monat >12){
        System.out.println("nochmal versuchen: " + monat);
    }
}

```

```

        monat = IO.readInt("Monat: ");
    }
    return monat;
}

public static int readTag(int max) {
    int tag = IO.readInt("Tag: ");
    while (tag < 1 || tag > max)
    {
        System.out.print("noch mal versuchen... ");
        tag = IO.readInt("Tag: ");
    }
    return tag;
}
/**
 *
 * @param jahr
 * @param monat
 * @return 28, 29 , 30, 31 oder -1 im Fehlerfall
 */
public static int berechneLaengeVomMonat(int jahr, int monat) {

    // boolean istSchaltjahr = (jahr % 4 == 0 && jahr % 100 != 0) ||
    jahr % 400 == 0;

    // int[][] lens= {{31,29,31,20,31,20,31,31,20,31,20,31},
    {31,28,31,20,31,20,31,31,20,31,20,31}};

    // return (monat<0||monat>12)?-1:(istSchaltjahr(jahr)? lens[0]
    [monat-1]:lens[1][monat-1]);

    return new int[] {31,istSchaltjahr(jahr)?
    29:28,31,20,31,20,31,31,20,31,20,31}[monat-1];

    //      switch (monat) {
    //      case 1:
    //      case 3:
    //      case 5:
    //      case 7:
    //      case 8:
    //      case 10:
    //      case 12:
    //          return 31;
    //      case 4:
    //      case 6:
    //      case 9:
    //      case 11:
    //          return 30;
    //      case 2:
    //          return Helper.istSchaltjahr(jahr) ? 29 : 28; // Ternary

```

Conditional Operator

```
//
//      default:
//      return -1;
//      }

    }
    public static boolean istSchaltjahr(int jahr) {
        return (jahr % 4 == 0 && jahr % 100 != 0) || jahr % 400 == 0;
    }
}
```

Klasse: Test

```
package _10_17_1;
```

```
import java.time.LocalDate;
```

```
public class Test {
    public static void main(String[] args) {

        Datum d = new Datum(31, 3, 2024);

        System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr());
        //      d = new Datum(6, 13, 2025);
        //      System.out.println(d.getTag() + ". " + d.getMonat() + ". " +
d.getJahr());// 0.10. 2025
        d.setJahr(2028);

        System.out.println(helper.asDE(d.getTag(),d.getMonat(),d.getJahr()));
        d.setJahr(2025);

        System.out.println(helper.asDE(d.getTag(),d.getMonat(),d.getJahr()));
        d.setMonat(5);

        System.out.println(helper.asDE(d.getTag(),d.getMonat(),d.getJahr()));
        d.setMonat(6);

        System.out.println(helper.asDE(d.getTag(),d.getMonat(),d.getJahr()));
        System.out.println(helper.asDE(1, 1 ,1930));
        System.out.println(helper.asDE(1, 10 ,1930));
        System.out.println(helper.asDE(10, 1 ,1930));
        System.out.println(helper.asDE(10, 10 ,1930));
        System.out.println(helper.asUS(1, 1 ,1930));
        System.out.println(helper.asUS(1, 10 ,1930));
        System.out.println(helper.asUS(10, 1 ,1930));
        System.out.println(helper.asUS(10, 10 ,1930));
        System.out.println(helper.asISO(1, 1 ,1930));
    }
}
```

```
System.out.println(Helper.asISO(1, 10 ,1930));
System.out.println(Helper.asISO(10, 1 ,1930));
System.out.println(Helper.asISO(10, 10 ,1930));
```

```
    }
}
```

Package: _10_17_lab

Klasse: Datum

```
package _10_17_lab;

import _10_15_lab.IO;
/**
 * TODO:
 * -Die Klasse datum soll mit Setter ausgestattet werden.
 */
public class Datum {

    private int tag;
    private int monat;
    private int jahr;

    public Datum(int tag, int monat, int jahr) {
        this.jahr = jahr;
        if(monat>0 && monat< 13)
            this.monat = monat;
        else {
            System.out.println("Ungültiger Monat: "+monat);
            this.monat=Helper.readMonat();
        }
        int n = Helper.berechneLaengeVomMonat(this.jahr, this.monat);

        if (tag>0 && tag<=n)
            this.tag=tag;
        else {
            System.out.println("Ungültiger Tag: "+tag);
            this.tag=Helper.readTag(n);
        }
    }
}
```



```

public int getTag() {
    return tag;
}

public int getMonat() {
    return monat;
}

public int getJahr() {
    return jahr;
}

public void setJahr(int jahr) {
    this.jahr=jahr;
    int n = Helper.berechneLaengeVomMonat(this.jahr, this.monat);
    if (tag<1 || tag>n) {
        System.out.println("Ungültiger Tag: "+tag);
        this.tag=Helper.readTag(n);
    }
}

public void setMonat(int monat) {
    if(monat>0 && monat< 13)
        this.monat = monat;
    else {
        System.out.println("Ungültiger Monat: "+monat);
        this.monat=Helper.readMonat();
    }
    int n = Helper.berechneLaengeVomMonat(this.jahr, this.monat);

    if (tag<1 || tag>n) {
        System.out.println("Ungültiger Tag: "+tag);
        this.tag=Helper.readTag(n);
    }
}

public void setTag(int tag) {
    int n = Helper.berechneLaengeVomMonat(this.jahr, this.monat);

    if (tag>0 && tag<=n)
        this.tag=tag;
    else {
        System.out.println("Ungültiger Tag: "+tag);
        this.tag=Helper.readTag(n);
    }
}

public void printFormatted() {
    System.out.printf("%02d.%02d.%4d\n",tag,monat,jahr);
}

```

```
}
```

Klasse: Datum2

```
package _10_17_lab;
```

```
import _10_15_lab.IO;
```

```
public class Datum2 {
```

```
    private int tag;  
    private int monat;  
    private int jahr;
```

```
    public Datum2(int tag, int monat, int jahr) {
```

```
        this.jahr = jahr;  
        if(monat>0 && monat< 13)  
            this.monat = monat;  
        else  
            this.monat=readMonat();
```

```
        switch (this.monat) {
```

```
            case 1:  
            case 3:  
            case 5:  
            case 7:  
            case 8:  
            case 10:  
            case 12:  
                if(tag>0 && tag<32)  
                    this.tag=tag;  
                else {  
                    System.out.println("Ungültiger Tag: "+tag);  
                    this.tag=readTag(31);  
                }  
                break;
```

```
            case 4:  
            case 6:  
            case 9:  
            case 11:  
                if(tag>0 && tag<31)  
                    this.tag=tag;  
                else  
                    this.tag=readTag(30);  
                break;
```

```
            default://Februar
```

```
                boolean istSchaltjahr = (jahr%4==0 && jahr %100 !=0) ||
```

```

jahr %400==0;
    if (istSchaltjahr) {
        //Schaltjahr
        if(tag>0 && tag<30)
            this.tag=tag;
        else
            this.tag=readTag(29);
    }
    else {
        //kein Schaltjahr
        if(tag>0 && tag<29)
            this.tag=tag;
        else
            this.tag=readTag(28);
    }
    break;
}
this.tag=tag;

}

```

```

public static int readMonat() {
    int monat=0;
    do {
        System.out.println("Ungültiger Monat: "+monat);
        monat=IO.readInt("Monat: ");
    } while (monat<1 || monat>12);

    return monat;
}

public static int readTag(int max) {
    int tag=IO.readInt("Tag: ");
    while (tag< 1 || tag>max){
        System.out.println("nochmal probieren.. ");
        tag=IO.readInt("Tag: ");
    }
    return tag;
}

public int getTag() {
    return tag;
}

public void setTag(int tag) {

```

```

        this.tag = tag;
    }
    public int getMonat() {
        return monat;
    }
    public void setMonat(int monat) {
        this.monat = monat;
    }
    public int getJahr() {
        return jahr;
    }
    public void setJahr(int jahr) {
        this.jahr = jahr;
    }
}

```

```

}

```

Klasse: Helper

```

package _10_17_lab;

import _10_15_lab.IO;

public class Helper {

    public static int readMonat() {
        int monat=IO.readInt("Monat: ");
        while (monat<1 || monat>12) {
            System.out.println("nochmal probieren.. "+monat);
            monat=IO.readInt("Monat: ");
        }

        return monat;
    }

    public static int readTag(int max) {
        int tag=IO.readInt("Tag: ");
        while (tag< 1 || tag>max){
            System.out.println("nochmal probieren.. ");
            tag=IO.readInt("Tag: ");
        }
        return tag;
    }

    public static int berechneLaengeVomMonat(int jahr,int monat) {
        boolean istSchaltjahr = (jahr%4==0 && jahr %100 !=0) || jahr

```

```

%400==0;

        switch (monat) {
        case 1,3,5,7,8,10,12: return 31;
        case 4,6,9,11: return 30;
        case 2:
            return istSchaltjahr ? 29 : 28;
        default:
            return -1;
        }
    }
}

```

Klasse: Test

```

package _10_17_lab;

public class Test {
    public static void main(String[] args) {
        Datum d = new Datum(29,2,2025);
        System.out.println(d.getTag()+ ". "+d.getMonat()+ ".
"+d.getJahr());

        d=new Datum(6,13,2025);
        System.out.println(d.getTag()+ ". "+d.getMonat()+ ".
"+d.getJahr());

        Datum d2 = new Datum(31,10,2025);
        System.out.println(d2.getTag()+ ". "+d2.getMonat()+ ".
"+d2.getJahr());

        d2.setJahr(2024);

        d2.setMonat(9);
        System.out.println(d2.getTag()+ ". "+d2.getMonat()+ ".
"+d2.getJahr());
        d2.printFormatted();
    }

}

```

Klasse: Uebung

```

package _10_17_lab;

public class Uebung {

}
/**

```

```

* Lernziel:
*           - Konstruktoren:
*           - Werteüberprüfung, Delegation
(this(...)), Überladen
*           - Getters und Setters
*           - Validität von Objekte
*           - Code Redundanz vermeiden
*           - Sichtbarkeit: private, default, public
*
* Aufgabe:
*           - Definiere eine Klasse, die Datum repräsentiert
(16.10.2025 oder 03.10.1989 ...)
*
* Vorgehensweise:
*           - Denkbare Methode implementieren
*           - Mit Objekte der Klassen in der main Methode
arbeiten.
*
*/

```

Package: _10_18

Klasse: Datum

```

package _10_18;

public class Datum {
    private int tag;
    private int monat;
    private int jahr; // Kann jetzt negativ sein für v. Chr.
    // Konstruktor

    public Datum(int tag, int monat, int jahr) {
        if (!istGueltigesDatum(tag, monat, jahr)) {
            throw new IllegalArgumentException("Ungültiges Datum: " +
tag + "." + monat + "." + jahr);
        }
        this.tag = tag;
        this.monat = monat;
        this.jahr = jahr;
    }

    // Setter für Tag
    public void setTag(int tag) {
        if (!istGueltigesDatum(tag, this.monat, this.jahr)) {
            throw new IllegalArgumentException("Ungültiger Tag: " +
tag + " für " + monat + "." + jahr);
        }
        this.tag = tag;
    }
}

```

```

//Setter für Monat
public void setMonat(int monat) {
    if (!istGueltigesDatum(this.tag, monat, this.jahr)) {
        throw new IllegalArgumentException("Ungültiger Monat: " +
monat + " für Tag " + tag + "." + jahr);
    }
    this.monat = monat;
}

//Setter für Jahr
public void setJahr(int jahr) {
    if (!istGueltigesDatum(this.tag, this.monat, jahr)) {
        throw new IllegalArgumentException("Ungültiges Jahr: " +
jahr + " für" + tag + "." + monat);
    }
    this.jahr = jahr;
}

//Prüft, ob ein Jahr ein Schaltjahr ist
private boolean istSchaltjahr(int jahr) {
//Schaltjahre gelten auch für v. Chr.: z. B. 4 v. Chr. ist ein
Schaltjahr
    if (jahr < 0) {
        jahr = -jahr; // Für Schaltjahrberechnung Jahr positiv
machen
    }
    return (jahr % 4 == 0 && jahr % 100 != 0) || (jahr % 400 ==
0);
}

//Prüft, ob das Datum gültig ist
private boolean istGueltigesDatum(int tag, int monat, int jahr) {
    if (tag < 1 || monat < 1 || monat > 12) {
        return false;
    }
    int[] tageInMonat = { 31, istSchaltjahr(jahr) ? 29 : 28, 31,
30, 31, 30, 31, 31, 30, 31, 30, 31 };
    return tag <= tageInMonat[monat - 1];
}

//Hilfsmethode: Gibt die Anzahl der Tage in einem Monat zurück
private int tageInMonat(int monat, int jahr) {
    int[] tageInMonat = { 31, istSchaltjahr(jahr) ? 29 : 28, 31,
30, 31, 30, 31, 31, 30, 31, 30, 31 };
    return tageInMonat[monat - 1];
}

//Methode zum Addieren von Tagen
public void addiereTage(int tage) {
    if (tage < 0) {

```

```

        throw new IllegalArgumentException("Anzahl der Tage darf
nicht negativ sein");
    }
    int neuerTag = this.tag;
    int neuerMonat = this.monat;
    int neuerJahr = this.jahr;
    neuerTag += tage;
    while (neuerTag > tageInMonat(neuerMonat, neuerJahr)) {
        neuerTag -= tageInMonat(neuerMonat, neuerJahr);
        neuerMonat++;
        if (neuerMonat > 12) {
            neuerMonat = 1;
            neuerJahr++;
            if (neuerJahr == 0)
                neuerJahr = 1; // Kein Jahr 0, direkt zu 1 n.Chr.
        }
    }
    if (!istGueltigesDatum(neuerTag, neuerMonat, neuerJahr)) {
        throw new IllegalArgumentException(
            "Resultierendes Datum ungültig: " + neuerTag + "."
+ neuerMonat + "." + neuerJahr);
    }
    this.tag = neuerTag;
    this.monat = neuerMonat;
    this.jahr = neuerJahr;
}

```

```

// Methode zum Subtrahieren von Tagen
public void subtrahiereTage(int tage) {
    if (tage < 0) {
        throw new IllegalArgumentException("Anzahl der Tage darf
nicht negativ sein");
    }
    int neuerTag = this.tag;
    int neuerMonat = this.monat;
    int neuerJahr = this.jahr;
    neuerTag -= tage;
    while (neuerTag < 1) {
        neuerMonat--;
        if (neuerMonat < 1) {
            neuerMonat = 12;
            neuerJahr--;
            if (neuerJahr == 0)
                neuerJahr = -1; // Kein Jahr 0, direkt zu 1 v.Chr.
        }
        neuerTag += tageInMonat(neuerMonat, neuerJahr);
    }
    if (!istGueltigesDatum(neuerTag, neuerMonat, neuerJahr)) {
        throw new IllegalArgumentException(
            "Resultierendes Datum ungültig: " + neuerTag + "."

```



```

+ neuerMonat + "." + neuerJahr);
    }
    this.tag = neuerTag;
    this.monat = neuerMonat;
    this.jahr = neuerJahr;
}

//Methode zur Berechnung der Differenz zwischen zwei Daten in Tagen
public int differenzInTagen(Datum anderesDatum) {
//Bestimme, welches Datum früher ist
    Datum frueheres = this.istFrueherAls(anderesDatum) ? this :
anderesDatum;
    Datum spaeteres = this.istFrueherAls(anderesDatum) ?
anderesDatum : this;
    int tage = 0;
//Kopie des früheren Datums
    Datum temp = new Datum(frueheres.tag, frueheres.monat,
frueheres.jahr);
//Tage zählen, bis das spätere Datum erreicht ist
    while (!temp.istGleich(spaeteres)) {
        temp.addiereTage(1);
        tage++;
    }
    return tage;
}

//Hilfsmethode: Prüft, ob dieses Datum früher als ein anderes ist
private boolean istFrueherAls(Datum anderes) {
    if (this.jahr < anderes.jahr)
        return true;
    if (this.jahr > anderes.jahr)
        return false;
    if (this.monat < anderes.monat)
        return true;
    if (this.monat > anderes.monat)
        return false;
    return this.tag < anderes.tag;
}

//Hilfsmethode: Prüft, ob zwei Daten gleich sind
private boolean istGleich(Datum anderes) {
    return this.tag == anderes.tag && this.monat == anderes.monat
&& this.jahr == anderes.jahr;
}

//Methode zur formatierten Ausgabe (mit v. Chr./n. Chr.)
public String getDatum() {
    if (jahr < 0) {
        return String.format("%02d.%02d.%d v. Chr.", tag, monat, -
jahr);
    }
}

```

```

        } else {
            return String.format("%02d.%02d.%d", tag, monat, jahr);
        }
    }

//Main-Methode zum Testen
    public static void main(String[] args) {
        try {
// Beispiel: Datum 23.05.2025
            Datum datum1 = new Datum(23, 5, 2025);
            System.out.println("Datum 1: " + datum1.getDatum()); //
Ausgabe:23.05.2025
// Beispiel: Datum 15.03.-44 (v. Chr.)
            Datum datum2 = new Datum(16, 9, 1991);
            System.out.println("Datum 2: " + datum2.getDatum()); //
Ausgabe:15.03.44 v. Chr.//50 Tage addieren zu
                                                                    //
15.03.-44
            datum2.addiereTage(20000);
            System.out.println("Datum 2 nach +20000 Tagen: " +
datum2.getDatum()); // Ausgabe: 04.05.44 v. Chr.//Differenz

// zwischen 23.05.2025 und 15.03.-44
            Datum datum3 = new Datum(17, 10, 2025);
            Datum datum4 = new Datum(14, 1, 1971);
            System.out.println("Differenz zwischen " +
datum3.getDatum() + " und " + datum4.getDatum() + ": "
+ datum3.differenzInTagen(datum4) + " Tage");
            ;
//Beispiel für ungültiges Datum
            Datum datum5 = new Datum(29, 2, -45); // Wirft Exception
(kein Schaltjahr)
        } catch (IllegalArgumentException e) {
            System.out.println("Fehler: " + e.getMessage());
        }
    }
}

```

Package: _10_20

Klasse: Datenstrukturen

```
package _10_20;
```

```
import java.util.ArrayList;
import java.util.Arrays;
```

```
/**
 * Array:
 *         -Datentyp (Referenz Datentyp, so wie Klassen)

```

```

*           - Sammlung von Werten vom gleichen Typ
*           - Fixed-Size (Anzahl der Elemente ist unveränderlich)
*
*           -Operationen:
*               - Anzahl der Elemente ermitteln (Attribut:
length)
*               - Index-Operator (Zugriff auf Elemente
mittels Index)
*               - foreach Loop
*
*           Syntax:
*
*               T[] v; wobei T ist ein Datentyp wie Person, Datum,
boolean, int, String...
*
*               T[] v = {e1,e2 ,e3 ,e4 , e5};      (1)
*               T[] v = new T[5];
*
*           Index:
*               - Wenn ein Array der Größe N ist, dann kann man auf die
einzelne Elemente folgendermaße zugreifen:
*
*               T x = v[i]; i = 0,1,2,3,...,N-1
*
*               N= v.length;
*
*/
public class Datenstrukturen {

    public static void main(String[] args) {

        /**
         * 1. Definiere eine Array der Größe 10.
         * 2. Fülle das Array mit Zufallszahlen
         * 3. Gebe Elemente des Arrays auf der Konsole aus.
         */
        double [] zahlen = new double [20]; //0.0 0.0 0.0 0.0 ...

//        for(int i = 0; i< zahlen.length; i++)
//            System.out.print(zahlen[i] + " ");
        System.out.println(Arrays.toString(zahlen));

        System.out.println();

//2

        for(int i = 0; i< zahlen.length; i++)
            zahlen[i] = Math.random();// 0.0 --- <1.0

//        for(int i = 0; i< zahlen.length; i++)

```

```
//          System.out.print(zahlen[i] + " ");

          System.out.println(Arrays.toString(zahlen));

          System.out.println();
      }

  }
}
```

Klasse: MehrDimensionaleArrays

```
package _10_20;

import java.util.Arrays;

public class MehrDimensionaleArrays {

    public static int[] getDiagonal(int[][][] table) {
        int[] result = new int[table.length];

        for (int i = 0; i < result.length; i++) {
            result[i] = table[i][i];
        }
        return result;
    }

    public static int[] getSumOfRow(int[][][] table) {
        int[] result = new int[table.length];

        for(int row = 0; row < table.length; row++) {
            for (int col= 0; col < table.length; col++) {
                result[row] += table[row][col];
            }
        }
        return result;
    }

    public static int[] getSumOfRow_v2(int[][][] table) {
        int[] result = new int[table.length];
        int i = 0;
        for(int[] row : table) {
            for (int x : row) {
                result[i] += x;
            }
            i++;
        }
        return result;
    }
}
```

```

/**
 *
 * Definiere eine Methode, die ein zwei dimensionaltes Array
entgegen nimmt und alle Element auf dem Diagonal
 * für den Aufrufer zurück gibt.
 *
 * Definiere eine Methode, die ein zwei dimensionaltes Array
entgegen nimmt und die Summe der Zeilen für den
 * Aufrufer zurück gibt.
 *
 *
 * Hinweis:
 *      -Das Array ist quadratisch (Anzahl Zeilen = Anzahl
Spalten)
 *      - Elemente sind int
 *
 * Beispiel:
 *      {{1,2,3,4},{2,3,4,5},{6,7,8,9},{1,3,5,7}}
 *
 * Elemente auf dem Diagonal = {1,3,8,7}
 * Summer der Zeilen = {10,14,30,16}
 */
public static void main(String[] args) {

    int [][] test = {{1,2,3,4},{2,3,4,5},{6,7,8,9},{1,3,5,7}};
    for(int [] a : test) {
        System.out.println(Arrays.toString(a));
    }
    System.out.println("=====");
    System.out.println("diagonal = " +
Arrays.toString(getDiagonal(test)));
    System.out.println("=====");
    System.out.println("Zeilensumme = " +
Arrays.toString(getSumOfRow(test)));
    System.out.println("Zeilensumme = " +
Arrays.toString(getSumOfRow_v2(test)));

    // int[][] v = {
    //      {1,4,6,8},
    //      {2,4,8,6,1,3,5,7},
    //      {4,4,4,4}
    // };
    // // new int[3][4];//
    //
    // System.out.println(v.length);//3
    //
    // for(int i = 0 ; i < v.length; i++) { // i = 0,1,2

```

```

//      for(int j = 0 ; j < v[i].length; j++) // j= 0,1,2,3
//      {
//          System.out.print(v[i][j] + " ");
//      }
//      System.out.println();
//  }
//
//  // Mittels foreach
//  System.out.println("=====");
//
//  for( int[] row: v) {
//      for(int element : row) {
//          System.out.print(element + " ");
//      }
//      System.out.println();
//  }
//  }
//  }

```

Klasse: Uebung

```
package _10_20;
```

```
import java.util.Arrays;
import java.util.Random;
```

```
public class Uebung {
```

```

    /**
     * Definiere eine Methode, die ein Array aus ganzen Zahlen
     entgegen nimmt und das kleinste Element aus dem
     * Array für den Aufrufer zurück gibt.
     *
     * Definiere eine Methode, die ein Array aus ganzen Zahlen
     entgegen nimmt und den Durchschnitt der Zahlen für den
     * Aufrufer zurück gibt.
     *
     * Definiere eine Methode, die ein Array aus N zufällige ganze
     Zahlen zwischen a und b generiert und für den
     * Aufrufer zurück gibt. N, a und b werden vom Aufrufer der
     Methode übergeben.
     *
     *
     * Lernziel:
     *          - Math.random
     *          - Random
     *          - Kontrollstrukturen: for, foreach
     *
     */

```

```

public static int getMin(int[] numbers) {
    int min = numbers[0];
    for (int i = 1; i < numbers.length; i++) {
        if(numbers[i] < min)
            min = numbers[i];
    }

    return min;
}

public static double getAvg(int[] numbers) {
    double avg;

    double sum = 0;
    for (int i = 0; i < numbers.length; i++) {
        sum += numbers[i];
    }
    avg = sum / numbers.length;
    return avg;
}
public static double getAvg_v2(int[] numbers) {
    double avg;

    double sum = 0;
    for (int number : numbers) {
        sum+= number;
    }
    avg = sum / numbers.length;
    return avg;
}
/**
 *
 * @param n
 * @param a exclusiv
 * @param b exclusive
 * @return [11, 12,13,...,19]
 */
public static int[] createArrayOf(int n, int a, int b) {
    int[] data = new int[n];

    Random r = new Random();
    for (int i = 0; i < data.length; i++) {
        data [i] = r.nextInt(b -a -1 ) + a + 1;
    }

    return data;
}
/**

```

```

*
* @param n
* @param a exklusiv
* @param b exclusive
* @return [11, 12,13,...,19]
*/
public static int[] createArrayOf_v2(int n, int a, int b) {
    int[] data = new int[n];

    for (int i = 0; i < data.length; i++) {
        data [i] = (int) (Math.random() * (b -a -1) + a + 1);
    }

    return data;
}
public static void main(String[] args) {
    int[] data = {9,3,5,1,7,6,4,-8,2,12};

    System.out.println("Das kleinste Element aus " +
Arrays.toString(data) + " = " + getMin(data));
    System.out.println("Der Durchschnitt aus " +
Arrays.toString(data) + " = " + getAvg(data));
    System.out.println(Arrays.toString(createArrayOf(10, 10,
20)));
    System.out.println(Arrays.toString(createArrayOf_v2(10, 10,
20)));
}
}

```

Klasse: Zusammenfassung

```

package _10_20;
/**
 * Arrays sind Referenz Datentypen (vs Primitive Datentypen)
 */
public class Zusammenfassung {
    public static void main(String[] args) {

        int i = 0;
        int [] ia = new int[] {1,2,3};

        int j = 0;
        int[] ja = new int[]{1,2,3};

        System.out.println(i == j); // true
        System.out.println(ia == ja); //false

    }
}

```



```
}
```

Package: _10_20_lab

Klasse: Datenstrukturen

```
package _10_20_lab;
```

```
import java.util.Arrays;
```

```
/**
 * Array:
 *   -Datentyp(Referenzdatentyp, so wie Klassen)
 *   -Sammlung von Werten vom gleichen Typ
 *   -fixed Size ( Anzahl der Elemente ist unveränderlich)
 *
 *   -Operationen:
 *       -Anzahl der Elemente ermitteln(Attribut: length)
 *       -Index-Operatoe(Zugriff auf Elemente mittels
Index)
 *       -foreach Loop
 *
 * Syntax:
 *   (Deklaration)
 *   T [] v; wobei T ein beliebiger Datentyp ist...zb Datum ,
int ,Person, boolean, String...
 *
 *   -Initialisierung:
 *   T [] v = {e1,e2,e3,.....,en};      (1)
 *   T [] v = new T [n];                (2)
 *
 *   Index
 *   -Wenn ein Arrayder Grösse N ist, dann kann man auf die
einzelnen Elemente folgendermaßen zugreifen:
 *
 *       T x = v[i]; i=0,1,2,3,.....,N-1
 *
 *       N = v.length;
 */
```

```
public class Datenstrukturen {
```

```
    public static void main(String[] args) {
        /**
         * 1.Definiere ein Array der Größe 10.
         * 2. Fülle das Array mit Zufallszahlen
         * 3. Gebe Elemente des Arrays auf der Konsole aus
         *
         */
    }
```

```

        */

        double [] zufallszahlen =new double[10];// gefüllt mit
Defaultwerten(hier 0.0)

        for (int i=0;i<zufallszahlen.length;i++) {
            zufallszahlen[i]=Math.random()*6;
        }

        for (double zufallszahl : zufallszahlen) {
            System.out.printf("%.2f\n",zufallszahl);
        }

        System.out.println(Arrays.toString(zufallszahlen));

    }

}

```

Klasse: MehrDimensionaleArrays

```

package _10_20_lab;

import java.util.Arrays;
import java.util.Iterator;

public class MehrDimensionaleArrays {

    /**
     * definiere eine Methode die ein zweidimensionales array entgegen
nimmt und alle elemente auf der Diagonalen zurück gibt.
     * definiere eine methode die ein zweidimensionales array entgegen
nimmt und die die summe der zeilen für den aufrufer zurückgibt.
     *
     * Hinweis: das array ist quadratisch
     *           elemente sind int
     *
     * @param args
     */

    public static void main(String[] args) {
//        int[][] v =new int[3][4];
        int[][] v = {
            {1,4,6,8},
            {2,4,8,6,1,3,5,7},
            {4,4,4,4},
        };
        System.out.println(v.length);
    }
}

```

```

        System.out.println();

//        for (int i =0;i<v.length;i++) {
//            System.out.println(v[i]);

//        for (int i =0;i<v.length;i++) { //i=0,1,2
//            for (int j =0;j<v[i].length;j++) { //j=0,1,2,3
//                System.out.print(v[i][j]+" ");
//            }
//            System.out.println();
//        }

        for (int[] zahlen : v) {
            for (int zahl:zahlen) {
                System.out.print(zahl+" ");
            }
            System.out.println();
        }

        System.out.println("=====");

        int[][] m= {{1,2,3,4,5},{2,7,8,4,5},{0,5,0,3,1},{7,8,-2,-9,6},
        {-7,-8,-9,-23,-99}};

        for (int[] zahlen : m) {
            for (int zahl:zahlen) {
                System.out.printf("%3d  ",zahl);
            }
            System.out.println();
        }
        System.out.println("=====");
        System.out.printf("Diagonalelemente:  %s\n",Arrays.toString(zeigeDiag(m)));
        System.out.printf("Zeilensummen:      %s\n",Arrays.toString(berechneZeilensumme(m)));

        int[][] m1=generiereZufallszahlenMatrix(15,-11,11);
        System.out.println("=====");
        for (int[] zahlen : m1) {
            for (int zahl:zahlen) {
                System.out.printf("%3d  ",zahl);
            }
            System.out.println();
        }

        System.out.println("=====");
        System.out.printf("Diagonalelemente:  %s\n",Arrays.toString(zeigeDiag(m1)));

```

```

        System.out.printf("Zeilensummen:      %s\n", Arrays.toString(berechneZeilensumme(m1)));
    }

    public static int[] zeigeDiag(int[][] matrix) {
        int [] diagonalelemente = new int [matrix.length];
        for (int i=0;i<matrix.length;i++) {
            diagonalelemente[i]=matrix[i][i];
        }
        return diagonalelemente;
    }

    public static int[] berechneZeilensumme(int[][] matrix){
        int [] zeilensumme = new int [matrix.length];
        for (int i = 0; i < matrix.length; i++) {
            for (int j=0;j<matrix[i].length;j++) {
                zeilensumme[i]+=matrix[i][j];
            }
        }
        return zeilensumme;
    }
    public static int[][] generiereZufallszahlenMatrix(int n,int a,int
b) {
        int[][] zufallszahlen= new int[n][n];
        for (int i=0;i<zufallszahlen.length;i++) {
            for (int j=0;j<zufallszahlen[i].length;j++) {
                zufallszahlen[i][j]=(int)((b-a-1)*Math.random()+a+1);
            }
        }
        return zufallszahlen;
    }
}

```

Klasse: Uebung

```

package _10_20_lab;

import java.util.Arrays;

public class Uebung {

    public static int findeMin(int[] arr) {
        int min=arr[0];
        for(int i=1;i<arr.length;i++)
            min= arr[i] <min ? arr[i]:min;
        return min;
    }
}

```

```

    public static double berechneDurchschnitt(int[] arr) {
        double sum=0.0;
        for (int zahl:arr)
            sum+=zahl;
        return (sum/arr.length);
    }

    public static int[] generiereZufallszahlen(int n,int a,int b) {
        int[] zufallszahlen= new int[n];
        for (int i=0;i<zufallszahlen.length;i++) {
            zufallszahlen[i]=(int)((b-a-1)*Math.random()+a+1);
        }
        return zufallszahlen;
    }

    public static void main(String[] args) {
        int [] zahlen=generiereZufallszahlen(100, 1, 1000);
        System.out.println(Arrays.toString(zahlen));
        System.out.println("Minimum: "+findeMin(zahlen));
        System.out.println("Durchschnitt:
"+berechneDurchschnitt(zahlen));
    }
}

```

Klasse: Zusammenfassung

```

package _10_20_lab;
/**
 * Arrays sind Referenzdatentypen(vs primitive datentypen)
 */
public class Zusammenfassung {

    public static void main(String[] args) {
        int i =0;
        int[] ia= new int[] {1,2,3};

        int j=0;
        int[] ja= new int[] {1,2,3};

        System.out.println(i==j);//true   gespeichert im Stack
        System.out.println(ia==ja); //false   gespeichert auf dem Hip
        mit einer Referenz vom Stack

        for (int k = 0; k < ja.length; k++) {

```

```

        System.out.println(ia[k]==ja[k]);
    }
}
}

```

Package: 10_21_Arbeitsblatt

Klasse: Auto

```

package _10_21_Arbeitsblatt;

public class Auto {
    String marke;
    String modell;
    int baujahr;
    int geschwindigkeit;

    public Auto(String ma,String mo,int b,int g) {
        marke=ma;
        modell=mo;
        baujahr=b;
        geschwindigkeit=g;
    }

    public Auto() {
        marke="VW";
        modell="Golf";
        baujahr=1994;
        geschwindigkeit=130;
    }

    public int beschleunigen(int dv) {
        System.out.println(modell + " beschleunigt von
"+geschwindigkeit+"km/h auf "+(geschwindigkeit+dv)+"km/h");
        geschwindigkeit +=dv;
        return geschwindigkeit;
    }

    public int bremsen(int dv) {
        int neuegeschw=geschwindigkeit<dv ? 0:geschwindigkeit-dv;
        System.out.println(modell + " bremsst von
"+geschwindigkeit+"km/h auf "+neuegeschw+"km/h");
        return neuegeschw;
    }

    public void anzeigen() {
        System.out.println("=====");
        System.out.printf("Marke: %20s\nModell: %19s\nBaujahr: %18d\
nGeschwindigkeit: %10d km/h\n",marke,modell,baujahr,geschwindigkeit);
    }
}

```

```

        System.out.println("=====");
    }

    public String getMarke() {
        return marke;
    }

    public void setMarke(String marke) {
        this.marke = marke;
    }

    public String getModel() {
        return modell;
    }

    public void setModell(String modell) {
        this.modell = modell;
    }

    public int getBaujahr() {
        return baujahr;
    }

    public void setBaujahr(int baujahr) {
        this.baujahr = baujahr;
    }

    public int getGeschwindigkeit() {
        return geschwindigkeit;
    }

    public void setGeschwindigkeit(int geschwindigkeit) {
        this.geschwindigkeit = geschwindigkeit;
    }

}

```

Klasse: Hausverwaltung

```
package _10_21_Arbeitsblatt;
```

```

public class Hausverwaltung {

    public static void main(String[] args) {
        Haus h = new Haus("Hauptstrasse 2",5,156);
        Haus h1 = new Haus("Hauptstrasse 3",2,86);
        Haus h2 = new Haus("Hauptstrasse 4",9,120);
        Haus h3 = new Haus("Hauptstrasse 5",3,206);
    }
}

```

```

        Haus h4 = new Haus("Nebenstrasse 2",17,856);
        Haus[] ha= {h,h1,h2,h3,h4};
        Wohngebiet w =new Wohngebiet(ha,"Zentrum");

        System.out.printf("Der Preis des Hauses ist: %,15.2f€\n",h.preisBerechnen(2550.49));
        System.out.printf("Der Preis des Hauses ist: %,15.2f€\n",h1.preisBerechnen(2550.49));
        System.out.printf("Der Preis des Hauses ist: %,15.2f€\n",h2.preisBerechnen(2550.49));
        System.out.printf("Der Preis des Hauses ist: %,15.2f€\n",h3.preisBerechnen(2550.49));
        System.out.printf("Der Preis des Hauses ist: %,15.2f€\n",h4.preisBerechnen(2550.49));

        System.out.printf("Die Gesamtfläche in %s ist: %d m²\n",w.name,w.berechneGesamtFlaeche());

    }

}

```

Klasse: Mitarbeiter

```

package _10_21_Arbeitsblatt;

public class Mitarbeiter {
    String name;
    String abteilung;
    double gehalt;

    public Mitarbeiter(String n,String a,double g) {
        name=n;
        abteilung=a;
        gehalt=g;
    }

    public void erhoeheGehalt(double prozent) {
        gehalt*=(1+prozent/100.0);
        System.out.println("Das Gehalt von "+name+" wurde um "+prozent+"% erhöht.");
    }

    public void zeigeInfo() {
        System.out.println("=====");
        System.out.printf("Name: %20s\nAbteilung: %15s\nGehalt: %17.2f€\n",name,abteilung,gehalt);
        System.out.println("=====");
    }
}

```



```
}
```

Klasse: Mitarbeiterverwaltung

```
package _10_21_Arbeitsblatt;

public class Mitarbeiterverwaltung {

    public static void main(String[] args) {

        Mitarbeiter[] mitarbeiter={new
Mitarbeiter("Gerd","HR",4500.0),new
Mitarbeiter("Gushti","Vorstand",12100.0),new
Mitarbeiter("Gushtine","Sekreteriat",3200.0)};

        for (Mitarbeiter mit:mitarbeiter) {
            mit.zeigeInfo();
        }

        mitarbeiter[1].erhoeheGehalt(15);

        for (Mitarbeiter mit:mitarbeiter) {
            mit.zeigeInfo();
        }

    }

}
```

Klasse: TestAuto

```
package _10_21_Arbeitsblatt;

public class TestAuto {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Auto kaefer=new Auto();
        Auto dings =new Auto("BWM","iEX-Flitzer",2020,260);

        kaefer.anzeigen();
        dings.anzeigen();
    }

}
```

```

        kaefer.geschwindigkeit=kaefer.bremsen(150);
        kaefer.anzeigen();
        kaefer.geschwindigkeit=kaefer.beschleunigen(110);
        kaefer.anzeigen();

        dings.geschwindigkeit=dings.bremsen(175);
        dings.anzeigen();
        dings.geschwindigkeit=dings.beschleunigen(100);
        dings.anzeigen();
//        kaefer.anzeigen();
//        dings.anzeigen();

    }

}

```

Klasse: Wohngebiet

```

package _10_21_Arbeitsblatt;

public class Wohngebiet {
    Haus[] haeuser;
    String name;

    public Wohngebiet(Haus[] h,String n) {
        haeuser=h;
        name=n;
    }

    public int berechneGesamtFlaeche() {
        int gesamt=0;
        for(Haus haus : haeuser) {
            gesamt += haus.flaeche;
        }
        return gesamt;
    }

}

```

Package: _10_21_lab

Klasse: CallByReference

```

package _10_21_lab;

import java.util.Arrays;

public class CallByReference {

```

```

public static void call_by_value(int x) {    //x=i=> Der Wert von i
wird kopiert und x zugewiesen!
    System.out.println("x = "+x);    //x=3;
    x=20;
    System.out.println("x = "+x);    //x=20
}

```

```

public static void call_by_reference(int[] a)
{
    //a=d=> a und d referenzieren auf ein und
dasselbe Objekt(Spitzname)
    System.out.println("a = "+Arrays.toString(a));    //a = [1,2,3]
    a[0]=5;
    System.out.println("a = "+Arrays.toString(a));    //a = [5,2,3]
}

```

```

public static void main(String[] args) {
    int i=3;
    System.out.println("i = "+i);                //i=3
    call_by_value(i);                            //call by value
    System.out.println("i = "+i);                //i=3

    int[] d= new int[]{1,2,3};                  //
    System.out.println("d = "+Arrays.toString(d));    //d = [1,2,3]
    call_by_reference(d);                        //
call by reference
    System.out.println("d = "+Arrays.toString(d));    //d = [5,2,3]
}
}

```

Klasse: Uebung

```

package _10_21_lab;

```

```

import _10_17_1.Datum;

```

```

public class Uebung {

```

```

    public static void main(String[] args) {
        Datum d = new Datum(19,1,1981); //Objekt 1
        Datum d2=d;    //Referenz Assignment-> d und d2 referenzieren
auf ein und dasselbe Objekt
        d.setJahr(2025);
        d.setMonat(10);
        d.setTag(21);

        int[] a= new int[3];    //Objekt2
        int [] b = new int[3];    //Objekt3
        int[] c=b;    //Referenz Assignment-> c und b
    }
}

```

referenzieren auf ein und dasselbe Objekt

```
c[0]=1;
b[0]=-1;
a[0]=2;
int i=0;
int j=i,k=i;          //Value Assignment: Inhalt der
Variablen i wird kopiert und an j und k zugewiesen
k=i++;
j=++i;
```

```
//Wieviele Objekte gibt es auf dem Heap an diesser Stelle?
//Ermittle Inhalt der Variablen (manuell!)
```

```
/**
 * Zeile      d      d2      a      b
 * i    j    k
c
  *8      19.01.1981
  *9      19.01.1981  19.01.1981
  *10     19.01.2025  19.01.2025
  *11     19.10.2025  19.10.2025
  *12     21.10.2025  21.10.2025
  *14     21.10.2025  21.10.2025  [0,0,0]
  *15     21.10.2025  21.10.2025  [0,0,0]
[0,0,0]
  *16     21.10.2025  21.10.2025  [0,0,0]
[0,0,0]      [0,0,0]
  *17     21.10.2025  21.10.2025  [0,0,0]
[1,0,0]      [1,0,0]
  *18     21.10.2025  21.10.2025  [0,0,0]  [-
1,0,0]      [-1,0,0]
  *19     21.10.2025  21.10.2025  [2,0,0]  [-
1,0,0]      [-1,0,0]
  *20     21.10.2025  21.10.2025  [2,0,0]  [-
1,0,0]      [-1,0,0]      0
  *21     21.10.2025  21.10.2025  [2,0,0]  [-
1,0,0]      [-1,0,0]      0  0  0
  *22     21.10.2025  21.10.2025  [2,0,0]  [-
1,0,0]      [-1,0,0]      1  0  0
  *23     21.10.2025  21.10.2025  [2,0,0]  [-
1,0,0]      [-1,0,0]      2  2  0
  *
  *
  *          */
```

```
}
```

```
}
```

Klasse: Uebung_2

```
package _10_21_lab;

import java.util.Arrays;

import _10_17_1.Datum;

/**
 * Lernziel:
 *     -Assignment by value und Call by Value
 *     -Assignment by reference und Call by Reference
 */
public class Uebung_2 {

    public static void changeMonth(Datum d) {
        d.setMonat(12);
    }

    public static void changeArray(int[] a) {
        a[0]=30;
    }

    public static void change(int x) {
        x=60;
    }

    public static void main(String[] args) {

        int i=1;
        int[] ar= new int[]{1,2,3};
        Datum today= new Datum(21,10,2025);

        change(i);
        changeArray(ar);
        changeMonth(today);

        System.out.println("i= "+i);
        System.out.println("ar = "+ Arrays.toString(ar));
        System.out.println("today = "+ today.getJahr()
+"- "+today.getMonat()+"- "+today.getTag()+" gelogen!");

    }

}
```

Klasse: Zusammenfassung

```
package _10_21_lab;

import java.util.Arrays;

/**
 * Arrays sind Referenzdatentypen(vs primitive datentypen)
 *
 * - Wie verhält sich der Operator == ?
 * - Referenz Assignment (Zuweisung):
 *   -Im Gegensatz zu Value Assignment,es findet
keine Kopierung statt!
 */
public class Zusammenfassung {

    public static void main(String[] args) {
        int i =0;
        int[] ia= new int[] {1,2,3};

        int j=0;
        int[] ja= new int[] {1,2,3};
        // == vergleicht zwei Dinge auf Identität, wenn wir zwei
Referenz Variablen vergleichen

        System.out.println(i==j);//true gespeichert im Stack
        System.out.println(ia==ja); //false gespeichert auf dem Heap
mit einer Referenz vom Stack

        for (int k = 0; k < ja.length; k++) {
            System.out.println(ia[k]==ja[k]);
        }

        System.out.println(Arrays.equals(ia, ja));//vergleicht die
Inhalte wie die for-Schleife oben

        System.out.println(Arrays.toString(ia));
        System.out.println(Arrays.toString(ja));

        int[] a=ia;//Reference Assignment (Referenz-Zuweisung)
        // a und ia referenzieren sich auf ein und dasselbe Objekt im
Speicher
        int x=i;//Value Assignment(Wert-Zuweisung)

        System.out.println("=====");
        System.out.println("a = "+Arrays.toString(a));//[1,2,3]
        System.out.println("ia = "+Arrays.toString(ia));//[1,2,3]
        System.out.println("a = "+a); //a=123459
        System.out.println("ia = "+ia); //ia=123459
    }
}
```

```

        System.out.println("ja = "+ja);          //ja=6584354
        System.out.println("a == ia ?"+(a==ia));    //true
        System.out.println("x = "+x+", i = "+i+", i == x ?"+
(i==x)); //x=0,i=0,true

        x++;
        a[0]=-1;

        System.out.println("=====");

        System.out.println("a = "+Arrays.toString(a));//[ -1,2,3]
        System.out.println("ia = "+Arrays.toString(ia));//[ -1,2,3]
        System.out.println("a = "+a);              //a=123459
        System.out.println("ia = "+ia);             //a=123459
        System.out.println("a == ia ?"+(a==ia));    //true
        System.out.println("x = "+x+", i = "+i+", i == x ?"+
(i==x)); //x=1,i=0,false

        System.out.println("=====");

    }
}

```

Package: [_10_22.arbeitsblatt](#)

Klasse: [Aufgabe_3](#)

```

package _10_22.arbeitsblatt;

public class Aufgabe_3 {
    public static void main(String[] args) {
        Haus h = new Haus("Paul-Robeson-Str. 34, 10439 Berlin", 4, 105);
        h.zeigeInfo();
    }
}package Do;
import java.util.*;
public class Ausfueren {

    public static void main(String[] args) {
        for (int i = 0; i < 10; i--) { System.out.println(i);}
    }

}

```

Klasse: [Auto](#)

```

package _10_22.arbeitsblatt;

import java.time.LocalDate;

```

```

/**
 * Datentyp Standardwert
 *
 * (byte, short, int, long) 0 (float, double) 0.0 boolean false char
 * '\0'
 *
 * Referenz Datentypen null
 */
public class Auto {
    private final String marke;
    private final String modell;
    private final int baujahr;
    private int geschwindigkeit; // 0

    public Auto() {
        this("VW", "Kaefer", LocalDate.now().getYear(), 0); //
        Constructor delegation
    }

    /**
     *
     * @param marke
     * @param modell
     * @param baujahr          <= heutiges jahr, wenn nicht wird
        automatisch auf
     *                          heutiges Jahr gesetzt
     * @param geschwindigkeit >= 0, wenn nicht wird wutomatisch auf 0
        gesetzt
     */

    public Auto(String marke, String modell, int baujahr, int
        geschwindigkeit) {
        int j = LocalDate.now().getYear(); // Jahr vom heutogen Datum
        (2025)
        this.marke = marke;
        this.modell = modell;
        if (baujahr <= j)
            this.baujahr = baujahr; // Vergangenheit
        else {
            System.out.println("Baujahr soll in der Vergangenheit
        liegen");
            this.baujahr = j;
        }
        if (geschwindigkeit >= 0)
            this.geschwindigkeit = geschwindigkeit; // >=0
        else
            System.out.println("Negative Geschwindigkeit ist
        unzulässig");
    }

```



```

    }

    public int getGeschwindigkeit() {
        return geschwindigkeit;
    }

    public void setGeschwindigkeit(int geschwindigkeit) {
        this.geschwindigkeit = geschwindigkeit;
    }

    public String getMarke() {
        return marke;
    }

    public String getModell() {
        return modell;
    }

    public int getBaujahr() {
        return baujahr;
    }

    public void beschleunigen(int wert) {
        wert = Math.abs(wert);
        this.geschwindigkeit += wert;
    }

    // - bremsen(int wert) – verringert die Geschwindigkeit (nicht
unter 0)
    public void bremsen(int wert) {
        wert = Math.abs(wert);

        this.geschwindigkeit -= wert;
        if (this.geschwindigkeit < 0) {
            this.geschwindigkeit = 0;
        }
    }

    public void anzeigen() {
        System.out.printf("Marke: %s, Modell: %s Baujahr: %d,
Geschwindigkeit: %s%n", marke, modell, baujahr,
        geschwindigkeit);
    }
}package _10_22_arbeitsblatt;

import java.time.LocalDate;

```

```

/**
 * Das Geheimnisprinzip (Information hiding)
 *
 *      Daten (Attribute) sollen nur klassenintern sichtbar sein
 */
/**
 * Datentyp          Standardwert
 * ((byte, short, int, long)    0
 * (float,double)              0.0
 * boolean                    false
 * char                        '\0'
 *
 * Referenz Datentyp          null
 */

public class Auto{

    private final String marke;
    private final String modell;
    private final int baujahr;
    private int geschwindigkeit;

    public Auto() {
        this("VW","Kaefer",LocalDate.now().getYear(),0); //Constructor
delegation
//      this.marke="VW";
//      this.modell="Kaefer";
//      this.baujahr = LocalDate.now().getYear();
//      this.geschwindigkeit =0;
    }
    /**
     *
     * @param marke
     * @param modell
     * @param baujahr<= heutiges jahr,wenn nicht wird automatisch auf
heutiges jahr gesetzt
     * @param geschwindigkeit >=0,wenn nicht wird automatisch auf 0
gesetzt
     */
    public Auto(String marke, String modell, int baujahr, int
geschwindigkeit) {
        int j =LocalDate.now().getYear();
        this.marke = marke;
        this.modell = modell;
        if (baujahr <= j)
            this.baujahr = baujahr; // Vergangenheit
        else {
            System.out.println("Baujahr soll in der Vergangenheit

```

```

liegen!");
        this.baujahr =j;
    }
    if (geschwindigkeit>=0)
        this.geschwindigkeit= geschwindigkeit;//>=0
    else {
        System.out.println("Negative Geschwindigkeiten ist
unzulässig!");
        //        this.geschwindigkeit=0;//ist der standardwert
    }
}

```

```

public int getGeschwindigkeit() {
    return geschwindigkeit;
}
public void setGeschwindigkeit(int geschwindigkeit) {
    this.geschwindigkeit = geschwindigkeit;
}
public String getMarke() {
    return marke;
}
public String getModell() {
    return modell;
}
public int getBaujahr() {
    return baujahr;
}
public void beschleunigen(int wert) {
    if (wert>0)
        this.geschwindigkeit +=wert;
    else
        System.out.println("Mit negativem Wert wird nicht
beschleunigt!");
}

```

```

public void bremsen(int wert) {
    if (wert>0)
        this.geschwindigkeit -=wert;
    else
        System.out.println("Mit negativem Wert wird nicht
gebremst!");
    if(this.geschwindigkeit<0) {
        this.geschwindigkeit=0;
    }
}

```

```

public void anzeigen() {

```

```

        System.out.println("=====");
        System.out.printf("Marke: %s\nModell: %s\nBaujahr: %d\nGeschwindigkeit: %dkm/h\n",marke,model,baujahr,geschwindigkeit);
        System.out.println("=====");
    }

```

```

}package _03_25;

```

```

public class Auto {
    private final String hersteller;
    private final int baujahr;

    private Auto(Builder b) {
        this.hersteller = b.hersteller;
        this.baujahr = b.baujahr;
    }

    public static class Builder{

        private String hersteller;
        private int baujahr;

        public Builder hersteller(String hersteller) {
            this.hersteller = hersteller;
            return this;
        }
        public Builder baujahr(int baujahr) {
            this.baujahr = baujahr;
            return this;
        }
        public Auto build() {
            return new Auto(this);
        }
    }

}

```

Klasse: Wohngebiet

```

package _10_22.arbeitsblatt;

public class Wohngebiet {
    private Haus[] haeuser;

    public Wohngebiet(int n) {
        haeuser = new Haus[n]; // null null ...
    }
    /**

```

```

    *
    * @param adresse
    * @param zimmer
    * @param flaeche
    * @return true, wenn auf dem Gebiet Platz für ein Haus gibt und
das Haus gebaut werden könnte, sonst false
    */
    public boolean baueEinHaus(String adresse, int zimmer, double
flaeche) {
        for (int i = 0; i < haeuser.length; i++) {
            if(haeuser[i] == null) {
                haeuser[i] = new Haus(adresse, zimmer, flaeche);
                return true;
            }
        }
        return false;// Es gibt keinen platz
    }
    //Füge eine Methode hinzu, die die Gesamtfläche aller Häuser
berechnet

    public double berechneGesamtFlaeche() {
        double gesamtFlaeche = 0.0;
        // for (int i = 0; i < haeuser.length; i++) {
        //     if(haeuser[i] != null)
        //         result += haeuser[i].getFlaeche();
        // }
        for (Haus haus : haeuser) {
            if(haus != null)
                gesamtFlaeche += haus.getFlaeche();
        }

        return gesamtFlaeche;
    }

    /**
    *
    * 1. Definiere eine Methode, die ermittelt Anzahl der Häuser im
Gebiet.
    *
    * 2. Definiere eine Methode, ob überhaupt im Gebiet ein Haus
existiert.
    *
    * 3. Definiere eine Methode, die überprüft ob Platz für neues
Haus gibt.
    */
    public static void main(String[] args) {
        Wohngebiet w = new Wohngebiet(3);
        System.out.println(w.baueEinHaus("Wundt Str. 18, 10778
Berlin", 3, 95));//true
        System.out.println( w.baueEinHaus("Ott Str. 28, 10758 Berlin",

```

```

4, 105)); //true
    //System.out.println(    w.baueEinHaus("Marten Str. 10, 10338
Berlin", 2, 75)); //true
    //System.out.println(w.baueEinHaus("Thorben Str. 48, 10149
Berlin", 5, 130)); //false

    System.out.println(w.berechneGesamtFlaeche());

}
}

```

Package: _10_22_arbeitsblatt

Klasse: Aufgabe_1

```

package _10_22_arbeitsblatt;

public class Aufgabe_1 {

    public static void main(String[] args) {
        Auto p = new Auto("VW","Kaefer",2024,-78);
        p.anzeigen();
        p.beschleunigen(50);
        p.anzeigen();
        p.beschleunigen(-25);
        p.anzeigen();
        p.bremsen(45);
        p.anzeigen();
    }

}package _10_22.arbeitsblatt;

public class Aufgabe_1 {
    public static void main(String[] args) {
        Auto p = new Auto();
        p.anzeigen();
        p.beschleunigen(50);
        p.anzeigen();
        p.beschleunigen(-25); //    p.beschleunigen(25);
        p.anzeigen();
        p.bremsen(65);
        p.anzeigen();

    }
}
/**
 * Das Geheimnisprinzip (Information hiding):
 *
 *         Daten (Attribute) sollen nur klassenintern sichtbar sein

```

```

*/package _10_22.arbeitsblatt;

public class Aufgabe_2 {
    public static void main(String[] args) {
        //Lege im Main-Programm ein Array aus 3 Mitarbeitern an und
        rufe erhoeheGehalt() auf

        //array-initialization-list
        Mitarbeiter[] mitarbeiterList = {
            new Mitarbeiter("Alice", "Vertrieb",
95000.0),
            new Mitarbeiter("Bob", "Entwicklung",
110000.0),
            new Mitarbeiter("Chris", "Test", 80000.0)
        };

        //
        for (Mitarbeiter mitarbeiter : mitarbeiterList) {
            mitarbeiter.zeigeInfo();
        }

        //Gehalt erhöhen

        mitarbeiterList[0].erhoeheGehalt(2);
        mitarbeiterList[1].erhoeheGehalt(3);
        mitarbeiterList[2].erhoeheGehalt(4);

    }
}

class Mitarbeiter{
    private String name;
    private String abteilung;
    private double gehalt;
    public Mitarbeiter(String name, String abteilung, double gehalt) {
        this.name = name;
        this.abteilung = abteilung;
        this.gehalt = gehalt;
    }

    //Beispiel: Marley, Test, 100.000,00
    public void zeigeInfo() {
        System.out.printf("%s, %s, %.2f%n", name, abteilung, gehalt);
    }
    public void erhoeheGehalt(double prozent) {
        this.gehalt *= 1 + prozent /100;
    }
}

}package _10_22_arbeitsblatt;

```

```

public class Aufgabe_2 {

    public static void main(String[] args) {
        Mitarbeiter[] mitarbeiter={new
Mitarbeiter("Alice","Vertrieb",95000.0),
                                new
Mitarbeiter("Bob","Entwicklung",110000.0),
                                new
Mitarbeiter("Chris","Test",80000.0)};

        for (Mitarbeiter mit:mitarbeiter) {
            mit.zeigeInfo();
        }

        mitarbeiter[1].erhoeheGehalt(15);

        for (Mitarbeiter mit:mitarbeiter) {
            mit.zeigeInfo();
        }

    }
}

class Mitarbeiter{

    private String name;
    private String abteilung;
    private double gehalt;

    public Mitarbeiter(String name, String abteilung, double gehalt) {
        this.name = name;
        this.abteilung = abteilung;
        this.gehalt = gehalt;
    }

    //Beispiel:Marley, Test,100.000,00
    public void zeigeInfo() {
        System.out.println("=====");
        System.out.printf("Name: %20s\nAbteilung: %15s\nGehalt:
%,17.2f€\n",name,abteilung,gehalt);
        System.out.println("=====");
    }

    public void erhoeheGehalt(double prozent) {
        this.gehalt *=(1+prozent/100.0);
    }
}

```



```

}package _01_27;

import java.util.Scanner;

public class Aufgabe_2 {

    static String admin = "Holger";

    public static void main(String[] args) {
        Scanner sc =new Scanner(System.in);
        System.out.print("Geben Sie den Namen an: ");
        String user_name =sc.nextLine();

        if(user_name.length() >= 3)
            System.out.println("Name ist ok");
        else
            System.out.println("Name ist zu kurz!");

        System.out.println("Anzahl von Zeichen in "+user_name + "=" +
user_name.length());
        System.out.println(user_name.toUpperCase());

        if(user_name.equals("Holger"))//if(user_name== admin) geht
nicht
            System.out.println("Du bist der Admin");
        else
            System.out.println("Du bist normaler User");

        System.out.println("user_name = "+user_name);

        sc.close();
    }
}

```

Klasse: Aufgabe_3

```

package _10_22_arbeitsblatt;

public class Aufgabe_3 {

```

```

        public static void main(String[] args) {
            Haus h = new Haus("Paul-Robenson-Str.34,10439 Berlin",4,105);
            h.zeigeInfo();

        }
    }
}

```

Klasse: Wohngebiet

```

package _10_22_arbeitsblatt;

import java.util.Iterator;

public class Wohngebiet {

    private Haus[] haeuser;
    public Wohngebiet(int n) {
        haeuser = new Haus[n];    //[null,null,...]
    }
    /**
     *
     * @param adresse
     * @param zimmer
     * @param flaeche
     * @return true, wenn auf dem Gebiet ein Haus gebaut werden
    könnte, sonst false
     */
    public boolean baueEinHaus(String adresse, int zimmer, double
    flaeche){
        for (int i = 0; i < haeuser.length; i++) {
            if(haeuser[i]==null) {
                haeuser[i]= new Haus(adresse,zimmer,flaeche);
                return true;
            }

        }

        return false;//es gibt keinen Platz
    }
    public double berechneGesamtFlaeche() {
        double result=0.0;
        //    for (int i = 0; i < haeuser.length; i++) {
        //        if(haeuser[i]!=null) {
        //            result += haeuser[i].getFlaeche();
        //        }
        //    }
        //    for(Haus haus : haeuser) {

```

```

        if(haus != null)
            result +=haus.getFlaeche();
    }

    return result;
}

/**
 * 1. Definiere eine Methode :anzahl häuser
 * 2.ob ein Haus existiert
 * 3.ob platz frei
 * @param args
 */
public int ermittleAnzahlHaeuser() {
    int count=0;
    for(Haus haus : haeuser) {
        if(haus != null)
            count++;
    }
    return count;
}

public boolean istHausDa() {
    if(ermittleAnzahlHaeuser(>0)
        return true;
    return false;
}

public boolean istPlatzFrei() {
    if (ermittleAnzahlHaeuser(< haeuser.length)
        return true;
    return false;
}

public static void main(String[] args) {
    Wohngebiet w =new Wohngebiet(3);

    System.out.println(w.baueEinHaus("Wund Str 18,10778
Berlin" ,3,95));
    System.out.println(w.baueEinHaus("Ott Str 28,10758
Berlin" ,4,105));
    System.out.println(w.baueEinHaus("Marten Str 10,10338
Berlin" ,2,75));
    //System.out.println(w.baueEinHaus("Thorben Str 48,10149
Berlin" ,5,130));
    System.out.println(w.berechneGesamtFlaeche());
    System.out.println(w.ermittleAnzahlHaeuser());
    System.out.println(w.istHausDa());
}

```

```

        System.out.println(w.istPlatzFrei());
    }

}

Package: _10_23

Klasse: Artikel
package _10_23;

import java.util.Objects;

import _10_15.I0;

public class Artikel {
    private final String name;
    private double preis;
    private int menge;

    public Artikel(String name, double preis, int menge) {
        this.name = name;
        setPreis(preis);
        setMenge(menge);
    }

    @Override
    public String toString() {
        return "Artikel [name=" + name + ", preis=" + preis + ",
menge=" + menge + "]";
    }

    public double getPreis() {
        return preis;
    }

    public void setPreis(double preis) {
        if (preis >= 0)
            this.preis = preis;
        else {
            preis = I0.readDouble("Preis vom artikel: ");
            while (preis < 0)
                preis = I0.readDouble("Preis vom artikel: ");

            this.preis = preis;
        }
    }
}

```

```

public int getMenge() {
    return menge;
}

public void setMenge(int menge) {
    if (menge > 0)
        this.menge = menge;
    else {
        menge = IO.readInt("menge vom Artikel: ");
        while (menge <= 0)
            menge = IO.readInt("menge vom Artikel: ");
        this.menge = menge;
    }
}

public String getName() {
    return name;
}

public double getGesamtPreis() {
    return preis * menge;
}

public void zeigeInfo() {
    System.out.println(this);
}

```

```

@Override
public int hashCode() {
    return Objects.hash(menge, name, preis);
}

```

```

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Artikel other = (Artikel) obj;
    return menge == other.menge && Objects.equals(name,
other.name)
        && Double.doubleToLongBits(preis) ==
Double.doubleToLongBits(other.preis);
}

```

```

    }

    public static void main(String[] args) {
        Artikel a = new Artikel("Collage Block", 1.99, 17);
        a.zeigeInfo();
        System.out.println(a.getGesamtPreis());
    }
}

```

Klasse: Datenstruktur

```

package _10_23;

import java.util.ArrayList;

/*
 * Datenstrukturen:
 * - Array
 * - List: ArrayList, Vector, LinkedList...
 *
 * Nachteil von Array:
 * - Anzahl der Elemente muss man im voraus
wissen
 * - Index Operator
 * - statisch
 * - arm an Operationen (Methoden)
 */
public class Datenstruktur {

    public static void main(String[] args) {
        // String[] names = new String[3];
        // names[0] = "Alice";
        // names[1] = "Bob";
        // names[2] = "Chris";
        //
        // System.out.println(names);

        ArrayList<String> names = new ArrayList<>(); //Vergleiche mit
Zeile 19!!!
        System.out.println(names.size());
        names.add("Alice");
        names.add("Bob");
        names.add("Chris");
        System.out.println(names.size());
        System.out.println(names);
        names.remove("Bob");
        System.out.println(names.size());
        System.out.println(names);
    }
}

```

```
    }  
}
```

Klasse: Warenkorb

```
package _10_23;  
  
import _10_15.IO;  
  
public class Warenkorb {  
    private Artikel[] artikelList;  
  
    public Warenkorb(int n) {  
        if(n > 0)  
            artikelList = new Artikel[n];  
        else  
        {  
            n = IO.readInt("Kapazität vom Warenkorb: ");  
            while(n <= 0)  
                n = IO.readInt("Kapazität vom Warenkorb: ");  
            artikelList = new Artikel[n];  
        }  
    }  
    /*  
    * - Methoden:  
    *   - artikelHinzufuegen(Artikel a)  
    *   - gesamtwertBerechnen()  
    *   - anzeigen()  
    */  
  
    public void artikelHinzufuegen(Artikel a) {  
        for (int i = 0; i < artikelList.length; i++) {  
            if(artikelList[i] == null) {  
                artikelList[i] = a;  
                System.out.println("Wrtikel wurde erfolgreich  
hinzugefügt");  
                return;  
            }  
        }  
        System.out.println("Warenkorb ist voll");  
    }  
  
    public double gesamtwertBerechnen() {  
        double gesamtwert = 0.0;  
        for (Artikel artikel : artikelList) {  
            if(artikel != null)  
                gesamtwert += artikel.getGesamtPreis();  
        }  
        return gesamtwert;  
    }  
}
```

```

    }

    public void printArtikels() {
        int nummer = 1;
        for (Artikel artikel : artikelList) {
            if(artikel != null)
                System.out.println(nummer++ + ". " + artikel);
        }
    }

    public static void main(String[] args) {
        Warenkorb w = new Warenkorb(5);
        w.artikelHinzufuegen(new Artikel("Collage Block", 2.0, 20));
        w.artikelHinzufuegen(new Artikel("Kugleschreiber", 1.50, 20));
        w.printArtikels();
        System.out.println(w.gesamtwertBerechnen());
    }
}

Klasse: Wohngebiet
package _10_23;

import java.util.Iterator;

import _10_15.IO;
import _10_22.arbeitsblatt.Haus;

public class Wohngebiet {
    private Haus[] haeuser;

    public Wohngebiet(int n) {
        if(n >= 0)
            haeuser = new Haus[n]; // null null ...
        else
        {
            System.out.println("Anzahl darf nicht negativ sein");
            n = IO.readInt("Anzahl Häuser auf dem Gebiet: ");
            while(n < 0)
                n = IO.readInt("Anzahl Häuser auf dem Gebiet: ");

            haeuser = new Haus[n];
        }
    }
    /**
     *
     * @param adresse
     * @param zimmer
     * @param flaeche
     * @return true, wenn auf dem Gebiet Platz für ein Haus gibt und
     das Haus gebaut werden könnte, sonst false

```



```

        */
        public boolean baueEinHaus(String adresse, int zimmer, double
flaeche) {
            for (int i = 0; i < haeuser.length; i++) {
                if(haeuser[i] == null) {
                    haeuser[i] = new Haus(adresse, zimmer, flaeche);
                    return true;
                }
            }
            return false;// Es gibt keinen platz
        }
    }

```

//Füge eine Methode hinzu, die die Gesamtfläche aller Häuser berechnet

```

        public double berechneGesamtFlaeche() {
            double gesamtFlaeche = 0.0;
            // for (int i = 0; i < haeuser.length; i++) {
            //     if(haeuser[i] != null)
            //         result += haeuser[i].getFlaeche();
            // }
            for (Haus haus : haeuser) {
                if(haus != null)
                    gesamtFlaeche += haus.getFlaeche();
            }
        }
    }

```

```

        return gesamtFlaeche;
    }

```

//1

```

        public int ermittleAnzahlHaeuser() {
            int anzahlHaeuser = 0;
            int i;
            for(i = 0; i < haeuser.length; i++) {
                if(haeuser[i] != null) // Es gibt ein Haus
                    anzahlHaeuser++;
            }

            return anzahlHaeuser;
        }
    }

```

```

        public int ermittleAnzahlHaeuser_v2() {
            int anzahlHaeuser = 0;
            int i = 0;
            while(i < haeuser.length) {
                if(haeuser[i] != null) // Es gibt ein Haus
                    anzahlHaeuser++;
            }
        }
    }

```

```

        i++;
    }

    return anzahlHaeuser;
}
public int ermittleAnzahlHaeuser_v3() {
    int anzahlHaeuser = 0;
    if(haeuser.length == 0)
        return 0;
    int i = 0;
    do{
        if(haeuser[i] != null) // Es gibt ein Haus
            anzahlHaeuser++;
        i++;
    }while(i < haeuser.length) ;

    return anzahlHaeuser;
}
public int ermittleAnzahlHaeuser_v4() {
    int anzahlHaeuser = 0;

    for (Haus haus : haeuser) {
        if(haus != null)
            anzahlHaeuser++;
    }

    return anzahlHaeuser;
}

public boolean existiertEinHaus() {
    return ermittleAnzahlHaeuser() > 0;
}

public boolean existiertEinHaus_v2() {
    for (Haus haus : haeuser) {
        if(haus != null)
            return true;
    }
    return false;
}

public boolean existierPlatzFuerNeuesHaus() {
    return ermittleAnzahlHaeuser() < haeuser.length;
}
public boolean existierPlatzFuerNeuesHaus_v2() {
    for (Haus haus : haeuser) {
        if(haus == null)
            return true;
    }
}

```

```

        }
        return false;
    }
}
/**
 *
 * 1. Definiere eine Methode, die ermittelt Anzahl der Häuser im
Gebiet.
 *
 * 2. Definiere eine Methode, die überprüft ob überhaupt im Gebiet
ein Haus existiert.
 *
 * 3. Definiere eine Methode, die überprüft ob Platz für neues
Haus gibt.
 * 4. Definiere eine Methode, die ein Haus und ein Index entgegen
nimmt und das Haus an der angegebenen Stelle im Wohngebiet einträgt.
 *
 * Hinweis:
 *           Betrachte alle Sonderfälle!!!
 */

public void insert(Haus h, int index) {
    if(index < 0 || index >= haeuser.length)
    {
        System.out.println("Ungültiger Index: " + index);
        return;
    }
    if (haeuser[index] == null) {
        haeuser[index] = h;
        System.out.println("Haus wurde erfolgreich eingetragen");
    }
    else
        System.out.println("Wohngebiet an der Stelle " + index + "
ist belegt.");
}
/**
 *
 * Definiere eine Methode, die textuell zeilenweise die Häuser auf
dem Gebiet auf der Konsole ausgibt, und zwar nummeriert.
 *
 * Beispiel:
 *
 * 1. Paul-Robeson-Str. 34, 10439 Berlin 2 69.0
 * 2. Kugler Str. 69, 10439 Berlin 2 51.0
 * 3. Wundt Str. 18, 10439 Berlin 1 39.0
 * usw
 *
 */
public void printHaeuser() {

```

```

System.out.println("=====");
    //System.out.println("Adresse \t Zimmer \t Fläche");
    int nummer = 1;
    for (Haus haus : haeuser) {
        if(haus != null)
            System.out.printf("%d. %s%n",nummer++, haus);
    }
}
public static void main(String[] args) {
    Wohngebiet w = new Wohngebiet(10);
    System.out.println(w.baueEinHaus("Wundt Str. 18, 10778
Berlin", 3, 95));//true
    System.out.println( w.baueEinHaus("Ott Str. 28, 10758 Berlin",
4, 105));//true
    System.out.println( w.baueEinHaus("Marten Str. 10, 10338
Berlin", 2, 75));//true
    System.out.println(w.baueEinHaus("Thorben Str. 48, 10149
Berlin", 5, 130));//false

    w.printHaeuser();

    System.out.println(w.berechneGesamtFlaeche());

    Wohngebiet w2 = new Wohngebiet(0);
    System.out.println(w2.ermittleAnzahlHaeuser_v2()); //0

}
}

```

Package: _10_23_lab

Klasse: Artikel

```
package _10_23_lab;
```

```
import _10_15.IO;
```

```

public class Artikel {
    private final String name;
    private double preis;
    private int menge;

    public Artikel(String name, double preis, int menge) {
        this.name = name;
        setPreis(preis);
        setMenge(menge);
    }
}

```

```
// public boolean equals (Object o) {
//
// if(o instanceof Artikel) {
//     Artikel temp=(Artikel) o;
//     return temp.name.equals(name) && temp.preis == preis
// &&temp.menge == menge;
// }
// return false;
//}
```

```
@Override
public String toString() {
    return "Artikel [name=" + name + ", preis=" + preis + ",
menge=" + menge + "]\n";
}
public double getPreis() {
    return preis;
}
public void setPreis(double preis) {
    if (preis>=0)
        this.preis = preis;
    else {
        preis=IO.readDouble("Preis von Artikel: ");
        while (preis<0) {
            preis=IO.readDouble("Preis von Artikel: ");
        }
        this.preis=preis;
    }
}
public int getMenge() {
    return menge;
}
public void setMenge(int menge) {
    if (menge>0)
        this.menge = menge;
    else {
        menge=IO.readInt("Menge des Artikels: ");
        while (menge<=0)
            menge=IO.readInt("Menge des Artikels: ");
        this.menge=menge;
    }
}

}
public String getName() {
    return name;
}

}

public double getGesamtPreis() {
    return preis*menge;
}
```

```

    }

    public void zeigeInfo() {
        System.out.println(this);
    }

    public static void main(String[] args) {
        Artikel a= new Artikel("College Block",1.99,17);
        a.zeigeInfo();
        System.out.println(a.getGesamtPreis());
    }
}

```

Klasse: Datenstruktur

```
package _10_23_lab;
```

```
import java.util.ArrayList;
```

```

/**
 * Datenstrukturen:
 *             -Array
 *             -List: ArrayList, Vector, LinkedList....
 *
 * Nachteil von Array:
 *             -Anzahl der Elemente muss man im vorraus wissen
 *             -Index Operator
 *             -statisch
 *             -arm an Operationen(Methoden)
 */

```

```
public class Datenstruktur {
```

```

    public static void main(String[] args) {
//        String[] names = new String[3];
//        names[0]="Alice";
//        names[1]="Bob";
//        names[2]="Chris";
//
//        System.out.println(names);
//

```

```

        ArrayList<String> names = new ArrayList<>(); //Vergleiche mit
Zeile 19!

```

```

        System.out.println(names.size());
        names.add("Alice");
        names.add("Bob");
        names.add("Chris");
        names.add("Gushti");

```

```

        names.add("Guschtine");
        System.out.println(names.size());
        System.out.println(names);

        for (String name : names) {
            System.out.println(name);
        }

        names.remove("Bob");

        System.out.println(names.size());
        System.out.println(names);

    }

}

Klasse: Warenkorb
package _10_23_lab;

import _10_15.IO;

public class Warenkorb {

    private Artikel[] artikelList;
    private int anzahlArtikel;

    public Warenkorb(int kapazitaet) {
        if (kapazitaet>0)
            artikelList = new Artikel[kapazitaet];
        else {
            kapazitaet=IO.readInt("Kapazität vom Warenkorb: ");
            while (kapazitaet>=0) {
                kapazitaet=IO.readInt("Kapazität vom Warenkorb: ");
            }
            artikelList = new Artikel[kapazitaet];
        }
        anzahlArtikel = 0;
    }

    public void artikelHinzufuegen(Artikel a) {
        if (anzahlArtikel < artikelList.length) {
            artikelList[anzahlArtikel] = a;
            anzahlArtikel++;
            System.out.println("Artikel '" + a.getName() + "'
hinzugefügt.");
        } else {
            System.out.println("Warenkorb voll!");
        }
    }
}

```

```

        }
    }

    public double gesamtwertBerechnen() {
        double gesamtwert=0.0;
        for (Artikel artikel : artikelList) {
            if(artikel !=null)
                gesamtwert +=artikel.getGesamtPreis();
        }

        return gesamtwert;
    }

    public void anzeigen() {
        int i=1;
        for (Artikel artikel : artikelList) {
            if (artikel != null)
                System.out.println((i++)+" ". +artikel);
        }
    }

    public static void main(String[] args) {
        Warenkorb korb = new Warenkorb(10);

        Artikel laptop = new Artikel("Gaming Laptop", 1299.99, 1);
        Artikel maus = new Artikel("Gaming Maus", 59.99, 2);
        Artikel tastatur = new Artikel("Mechanische Tastatur", 149.99,
1);
        Artikel monitor = new Artikel("27'' Monitor", 299.99, 2);

        korb.artikelHinzufuegen(laptop);
        korb.artikelHinzufuegen(maus);
        korb.artikelHinzufuegen(tastatur);
        korb.artikelHinzufuegen(monitor);

        korb.anzeigen();
        System.out.println("Gesamtwert: "+korb.gesamtwertBerechnen()
+"€");
    }
}

```

Klasse: Wohngebiet

```
package _10_23_lab;
```

```
import _10_15.I0;
import _10_22_arbeitsblatt.Haus;
```



```

public class Wohngebiet {
    private Haus[] haeuser;

    public Wohngebiet(int n) {
        if (n>=0)
            haeuser = new Haus[n]; // null null ...
        else {
            System.out.println("Anzahl darf nicht negativ sein!");
            while (n<0)
                haeuser= new Haus[IO.readInt("Anzahl der Häuser auf
dem Gebiet: ")]];
            haeuser = new Haus[n];
        }
    }
    /**
     *
     * @param adresse
     * @param zimmer
     * @param flaeche
     * @return true, wenn auf dem Gebiet Platz für ein Haus gibt und
das Haus gebaut werden könnte, sonst false
     */
    public boolean baueEinHaus(String adresse, int zimmer, double
flaeche) {
        for (int i = 0; i < haeuser.length; i++) {
            if(haeuser[i] == null) {
                haeuser[i] = new Haus(adresse, zimmer, flaeche);
                return true;
            }
        }
        return false;// Es gibt keinen platz
    }
    //Füge eine Methode hinzu, die die Gesamtfläche aller Häuser
berechnet

    public double berechneGesamtFlaeche() {
        double gesamtFlaeche = 0.0;
        // for (int i = 0; i < haeuser.length; i++) {
        //     if(haeuser[i] != null)
        //         result += haeuser[i].getFlaeche();
        // }
        for (Haus haus : haeuser) {
            if(haus != null)
                gesamtFlaeche += haus.getFlaeche();
        }

        return gesamtFlaeche;
    }
}

```

```

//1
public int ermittleAnzahlHaeuser_V2() {
    int anzahlHaeuser=0;
    int i=0;
    while(i<haeuser.length) {
        if (haeuser[i] != null)
            anzahlHaeuser++;
        i++;
    }
    return anzahlHaeuser;
}

public int ermittleAnzahlHaeuser_V3() {
    int anzahlHaeuser=0;
    if (haeuser.length==0)
        return 0;
    int i=0;
    do {

        if (haeuser[i] != null)
            anzahlHaeuser++;
        i++;
    }while(i<haeuser.length) ;
    return anzahlHaeuser;
}

    public int ermittleAnzahlHaeuser() {
        int anzahlHaeuser=0;

//        for (int i = 0; i < haeuser.length; i++) {
//            if( haeuser[i] != null)
//                anzahlHaeuser++;
//        }
//        int i=0;
//        while(i<haeuser.length) {
//            if (haeuser[i] != null)
//                anzahlHaeuser++;
//            i++;
//        }

        for(Haus haus : haeuser) {
            if(haus != null)
                anzahlHaeuser++;
        }
        return anzahlHaeuser;
    }

    public boolean existiertEinHaus() {
        return ermittleAnzahlHaeuser(>0;
    }

```

```

public boolean existiertEinHaus_v2() {
    for (Haus haus: haeuser) {
        if(haus != null)
            return true;
    }
    return false;
}

public boolean existiertPlatzFuerNeuesHaus() {
    return ermittleAnzahlHaeuser() < haeuser.length;
}

public boolean existiertPlatzFuerNeuesHaus_v2() {
    for (Haus haus: haeuser) {
        if(haus == null)
            return true;
    }
    return false;
}

public boolean fuegeHausAnStelleEin(Haus h, int i) {
    if (i<0 || i>=haeuser.length) {
        System.out.println("Ungültiger Index: "+i);
        return false;
    }
    if(!existiertPlatzFuerNeuesHaus())
        return false;
    if(haeuser[i]==null) {
        haeuser[i]=h;
        System.out.println("Haus ohne Probleme gebaut");
        return true;
    }else {
        for (int j = 0; j < haeuser.length; j++) {
            if(haeuser[j] == null) {
                haeuser[j] = haeuser[i];
                haeuser[i] =h;
                System.out.println("Haus wurde umgesiedelt!");
            }
        }
    }
    return true;
}

/**
 * Definiere eine Methode die textuell zeilenweise die Häuser auf
dem Gebiet aus der Konsole ausgibt und zwar nummeriert.
 *
 * beispiel:
 * 1. paul roberson str. 34, 10439 berlin 2 69.0
 * 2.Kugler Str. 69 10439 Berlin 2 51.0
 * ....

```

```

    *
    *
    */

/**
 *
 * 1. Definiere eine Methode, die ermittelt Anzahl der Häuser im
Gebiet.
 *
 * 2. Definiere eine Methode, ob überhaupt im Gebiet ein Haus
existiert.
 *
 * 3. Definiere eine Methode, die überprüft ob Platz für neues
Haus gibt.
 * 4. Definiere eine Methode, die ein Haus entgegennimmt und ein
Index und das Haus an der angegebenen Stelle im Wohngebiet einträgt.
 *
 * Hinweis: Betrachte alle Sonderfälle!!!
 */
public static void main(String[] args) {
    Wohngebiet w = new Wohngebiet(3);
    System.out.println(w.baueEinHaus("Wundt Str. 18, 10778
Berlin", 3, 95)); //true
    System.out.println( w.baueEinHaus("Ott Str. 28, 10758 Berlin",
4, 105)); //true
    //System.out.println( w.baueEinHaus("Marten Str. 10, 10338
Berlin", 2, 75)); //true
    //System.out.println(w.baueEinHaus("Thorben Str. 48, 10149
Berlin", 5, 130)); //false

    System.out.println(w.berechneGesamtFlaeche());

}
}

```

Package: _10_24

Klasse: Warenkorb

```

package _10_24;

import java.time.LocalDate;
import java.util.ArrayList;

import _10_15.IO;
import _10_23.Artikel;

public class Warenkorb {
    private ArrayList<Artikel>artikelList;

```

```

private int n ;

public Warenkorb(int n) {
    if(n > 0)
    {
        this.n = n;
    }

    else
    {
        n = IO.readInt("Kapazität vom Warenkorb: ");
        while(n <= 0)
            n = IO.readInt("Kapazität vom Warenkorb: ");

        this.n = n;
    }
    artikelList = new ArrayList<Artikel>(n);
}

public void artikelHinzufuegen(Artikel a) {
    if(a == null) {
        System.out.println("Null ist kein Artikel");
        return;
    }
    if(artikelList.size() < n)
        artikelList.add(a);
    else
        System.out.println("Warenkorb ist voll");
}

public double gesamtwertBerechnen() {
    double gesamtwert = 0.0;
    for (Artikel artikel : artikelList) {
        gesamtwert += artikel.getGesamtPreis();
    }
    return gesamtwert;
}

public void printArtikels() {
    int nummer = 1;
    for (Artikel artikel : artikelList) {
        System.out.println(nummer++ + ". " + artikel);
    }
}

/**
 *
 * Definiere eine Methode, die überprüft, ob ein gegebenes Artikel

```

im Warenkorb ist.

```
    */
    public boolean istArtikelImWarenkorb(Artikel a) {
        return artikelList.contains(a);
    }

    public static void main(String[] args) {
        Warenkorb w = new Warenkorb(5);
        w.artikelHinzufuegen(new Artikel("Collage Block", 2.0, 20));
        w.artikelHinzufuegen(new Artikel("Kugleschreiber", 1.50, 20));
        w.printArtikels();
        System.out.println(w.gesamtwertBerechnen());

        Artikel b = new Artikel("Collage Block", 2.0, 20);
        Artikel b2 = new Artikel("Collage Block", 2.0, 20);

        System.out.println(b.equals(b2)); // false

        System.out.println(w.istArtikelImWarenkorb(b));

        LocalDate d1 = LocalDate.of(2025, 10, 24);
        LocalDate d2 = LocalDate.of(2025, 10, 24);
        System.out.println(d1 == d2);
        System.out.println(d1.equals(d2)); //true
    }
}
```

Package: [_10_24_lab](#)

Klasse: Warenkorb

```
package _10_24_lab;

import java.time.LocalDate;
import java.util.ArrayList;

import _10_15.IO;
import _10_23.Artikel;

public class Warenkorb {
    // private Artikel[] artikelList;
    private ArrayList<Artikel> artikelList;
    private int kapazitaet;

    public Warenkorb(int kapazitaet) {
        if(kapazitaet > 0) {
            // artikelList = new Artikel[n];
            artikelList = new ArrayList<>(kapazitaet);
            this.kapazitaet=kapazitaet;
        }
    }
}
```

```

        else
        {
            kapazitaet = IO.readInt("Kapazität vom Warenkorb: ");
            while(kapazitaet <= 0)
                kapazitaet = IO.readInt("Kapazität vom Warenkorb: ");
//        artikelList = new Artikel[n];
        artikelList =new ArrayList<>(kapazitaet);
        this.kapazitaet=kapazitaet;
    }

}
/*
 * - Methoden:
 *   - artikelHinzufuegen(Artikel a)
 *   - gesamtwertBerechnen()
 *   - anzeigen()

 */

    public void artikelHinzufuegen(Artikel a) {
//        for (int i = 0; i < artikelList.length; i++) {
//            if(artikelList[i] == null) {
//                artikelList[i] = a;
//            }
//            if (a==null) {
//                System.out.println("Null ist kein Artikel!");
//                return;
//            }
//            if (artikelList.size() < kapazitaet) {
//                artikelList.add(a);
//                System.out.println("Artikel wurde erfolgreich
hinzugefügt");
//            }
//            else
//                System.out.println("Warenkorb ist voll");
//        }

//    }
//    System.out.println("Warenkorb ist voll");
//    }

    public double gesamtwertBerechnen() {
        double gesamtwert = 0.0;
        for (Artikel artikel : artikelList) {
//            if(artikel != null)
                gesamtwert += artikel.getGesamtPreis();
        }
        return gesamtwert;
    }
}

```

```

public void printArtikels() {
    int nummer = 1;
    for (Artikel artikel : artikelList) {
//        if(artikel != null)
            System.out.println(nummer++ + ". " + artikel);
    }
}

/**
 * 1.Definiere eine Methode,die überprüft ob ein gegebener Artikel
im Warenkorb ist
 *
 *
 */
// public boolean istImWarenkorb() {
//     return artikelList.contains(IO.readString("Nach welchem Artikel
wollen Sie im Warenkorb suchen: "));
//
// } //muss man in der ArtikelKlasse machen

public boolean istArtikelImWarenkorb_2(Artikel a) {
    for (int i = 0; i < artikelList.size(); i++) {
        Artikel temp=artikelList.get(i);
//        if (artikelList.get(i)== a)
//        if (temp.getName().equals(a.getName()) && temp.getPreis()
== a. getPreis() &&temp.getMenge()==a.getMenge())
            if(temp.equals(a))
                return true;

    }
    return false;
}

public boolean istArtikelImWarenkorb_3(Artikel a) {
    for (Artikel artikel : artikelList) {
        if (artikel.equals(a))
            return true;
    }
    return false;
}

public boolean istArtikelImWarenkorb(Artikel a) {
    return artikelList.contains(a);
}

public static void main(String[] args) {
    Warenkorb w = new Warenkorb(5);
    w.artikelHinzufuegen(new Artikel("Collage Block", 2.0, 20));
}

```



```

        w.artikelHinzufuegen(new Artikel("Kugleschreiber", 1.50, 20));
        w.printArtikels();
        System.out.println("Gesamtwert: "+w.gesamtwertBerechnen()+"€");
// System.out.println(w.istImWarenkorb());

        Artikel b = new Artikel ("Bleistift",1.25,12);
        Artikel c = new Artikel("Collage Block", 2.0, 20);

        System.out.println(w.istArtikelImWarenkorb(b));
        System.out.println(w.istArtikelImWarenkorb(c));

        LocalDate d1 =LocalDate.of(2025, 10, 24);
        LocalDate d2 =LocalDate.of(2025, 10, 24);

        System.out.println(d1==d2);//false
        System.out.println(d1.equals(d2));//true
        System.out.println(d2.equals(d1));//true

        System.out.println(w.istArtikelImWarenkorb_3(c));
    }
}

```

Klasse: Wohngebiet

```

package _10_24_lab;

import java.util.ArrayList;

import _10_15.IO;

public class Wohngebiet {
    private ArrayList<Haus> haeuser= new ArrayList<Haus>(); ;
    private int n;

    public Wohngebiet(int n) {
        if (n>0) {
            this.n=n;
        }
        else{
            n=IO.readInt("max Anzahl der Häuser:_");
            while (n<=0)
            {
                n=IO.readInt("max Anzahl der Häuser:_");
            }
        }
    }
    /**
     *
     * @param adresse
     * @param zimmer

```

```

    * @param flaeche
    * @return true, wenn auf dem Gebiet Platz für ein Haus gibt und
das Haus gebaut werden könnte, sonst false
    */
    public boolean baueEinHaus(String adresse, int zimmer, double
flaeche) {
        if(! enthaeltHaus( new Haus(adresse,zimmer,flaeche))) {
            if(haeuser.size()<n)
                return haeuser.add(new Haus(adresse, zimmer, flaeche));
            else
                return false;
        }
        return false;
    }
    //Füge eine Methode hinzu, die die Gesamtfläche aller Häuser
berechnet

    public double berechneGesamtFlaeche() {
        double gesamtFlaeche = 0.0;
//        for (int i = 0; i < haeuser.length; i++) {
//            if(haeuser[i] != null)
//                result += haeuser[i].getFlaeche();
//        }
        for (Haus haus : haeuser) {
//            if(haus != null)
                gesamtFlaeche += haus.getFlaeche();
        }

        return gesamtFlaeche;
    }

    /**
    *
    * 1. Definiere eine Methode, die ermittelt Anzahl der Häuser im
Gebiet.
    *
    * 2. Definiere eine Methode, ob überhaupt im Gebiet ein Haus
existiert.
    *
    * 3. Definiere eine Methode, die überprüft ob Platz für neues
Haus gibt.
    */
    public int ermittleAnzahlHaeuser() {
        return haeuser.size();
    }
    public boolean existiertEinHaus() {
        return haeuser.size()>0;
    }
    public boolean existiertPLatzFuerHaus() {
        return haeuser.size()<n;
    }

```

```

    }

    public void print() {
        //System.out.println(haeuser); ArrayList.toString()
        int nummer=1;
        for (Haus haus : haeuser) {
            System.out.println(nummer++ +". "+haus);
        }
    }

    public boolean baueEinHaus(Haus haus) {
        if(! enthaeltHaus( haus)) {
            if(haeuser.size()<n)
                return haeuser.add(haus);
            //return haeuser.add(new
Haus(haus.getAdresse(),haus.getZimmer(),haus.getFlaeche())); //
Bildkopie
            else
                return false;
        }
        return false;
    }

    public boolean enthaeltHaus(Haus h) {
        return haeuser.contains(h);
    }

    public static void main(String[] args) {
        Wohngebiet w = new Wohngebiet(10);
        System.out.println(w.baueEinHaus("Wundt Str. 18, 10778
Berlin", 3, 95));//true
        System.out.println(w.baueEinHaus("Ott Str. 28, 10758 Berlin",
4, 105));//true
        //System.out.println(w.baueEinHaus("Marten Str. 10, 10338
Berlin", 2, 75));//true
        //System.out.println(w.baueEinHaus("Thorben Str. 48, 10149
Berlin", 5, 130));//false
        System.out.println(w.baueEinHaus("Wielandstr.38, 10778
Berlin", 4 ,130));
        w.print();
        System.out.println(w.berechneGesamtFlaeche());
        Haus meinHaus = new Haus("Wielandstr.38, 10778 Berlin",
4 ,130);
        System.out.println(w.enthaeltHaus(meinHaus));
        System.out.println(w.baueEinHaus("Wielandstr.38, 10778
Berlin", 4 ,130));
        w.print();
    }
}

```

Package: [_11_03_2026](#)

Klasse: App

```
package _11_03_2026;
```

```
import java.util.ArrayList;
```

```
import _03_09.IllegalAmountException;
```

```
public class App {
```

```
    public static void main(String[] args) {  
        // TODO Auto-generated method stubServiceKlasse verwenden
```

```
        String path= "Punkte_im_Csv_Format.csv";
```

```
        ArrayList<Punkt> punkte = new ArrayList<Punkt>();
```

```
        for (int i=0;i<10;i++) {  
            punkte.add(new  
Punkt(Math.round(Math.random()*100)/100.0,Math.round(Math.random()*100  
) /100.0));  
        }
```

```
        Service.writeToCsv(punkte, path);
```

```
        String kreispath="Kreise_im_Csv_Format.csv";
```

```
        ArrayList<Kreis> kreise = new ArrayList<Kreis>();
```

```
        for (int i=0;i<10;i++) {  
            try {  
                kreise.add(new Kreis(new Punkt(1,2),34.1));  
                kreise.add(new Kreis(new  
Punkt(Math.round(Math.random()*100)/100.0,Math.round(Math.random()*100  
) /100.0),Math.round((Math.random()+0.01)*100)/100.0));
```

```
            } catch (IllegalAmountException e) {  
                // TODO Auto-generated catch block  
                e.printStackTrace();  
            }
```

```
            kreise.add(new Kreis(new  
Punkt(Math.round(Math.random()*100)/100.0,Math.round(Math.random()*100  
) /100.0)),Math.round(Math.random()*100)/100.0);  
        }
```

```
        Service.writeCirclesToCvs(kreise, kreispath);
```

```
    }
```

```
}
```

Klasse: Kreis

```
package _11_03_2026;
```

```
import _03_09.IllegalAmountException;
```

```
public class Kreis {
```

```
    private Punkt mittelpunkt;  
    private double radius;
```

```
    public Kreis(Punkt mittelpunkt, double radius) throws  
IllegalAmountException {  
        this.mittelpunkt=mittelpunkt;  
        if (radius>0)  
            this.radius = radius;  
        else  
            throw new IllegalAmountException("Radius muss positiv  
sein");  
    }
```

```
    public double getRadius() {  
        return radius;  
    }
```

```
    @Override  
    public String toString() {  
        return "Kreis [mittelpunkt=" + mittelpunkt + ", radius=" +  
radius + "];"  
    }  
    public String toCSV() {  
        return "P("+mittelpunkt.getX()+"|"+mittelpunkt.getY()  
+");r="+radius+";Fläche="+berechneFlaeche()  
+";Umfang="+berechneUmfang();  
    }
```

```
    public double berechneFlaeche() {  
        return Math.round(Math.PI*radius*radius*100)/100.0;  
    }
```

```
    public double berechneUmfang() {  
        return Math.round(Math.PI*radius*200)/100.0;
```

```

    }
}

```

Klasse: Lesen

```
package _11_03_2026;
```

```
import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
```

```
public class Lesen {
    public static void main(String[] args) {
```

```

        /**
         * Lesen:
         *      -1. Datei öffnen
         *      -2. Etwas aus der Datei lesen
         *      -3. Datei schließen
         *
         *C:\Users\Student\OneDrive - GFN GmbH (EDU)\Desktop\
Ordner 1\LF  ZQ19A
         *
         */
        //Datei erzeugen

        //File f = new File("C:\\Users\\Student\\OneDrive -
GFN GmbH (EDU)\\Desktop\\Ordner 1\\LF  ZQ19A\\temperaturen.txt");
        File f = new File("names.txt"); //Objekt auf dem HEAP,
es ist noch keine Datei erzeugt worden
        //C:\Users\Student\eclipse-workspace\LF_ZQ19A
        //Datei erzeugen auf dem Dateisystem

```

```
        try {
```

```

                                //Lesen mittels FileReader: Zeichenweise; End of
File EOF=-1

```

```

// überschreiben
//
//
//
//
FileReader fr=new FileReader(f);//append statt
int c;

while( (c= fr.read()) != -1)
    System.out.println("c= "+(char)c);
fr.close();

//Lesen mittels BufferedReader: Zeilenweise; End
of File (EOF)=null
BufferedReader br= new BufferedReader(fr);

String line;

while((line=br.readLine()) != null)
    System.out.println(line);
br.close();


} catch (IOException e) {
    System.out.println("Datei konnte nicht erzeugt
werden "+e.getMessage());
}

//Lesen
List<String> ls = new ArrayList<String>();
try (BufferedReader br =
Files.newBufferedReader(Paths.get("names.txt")
)) {
    String s;
    while ((s=br.readLine()) != null)
        ls.add(s);
} catch (IOException e) {
    System.out.println("Datei konnte nicht gelesen
werden "+e.getMessage());
}

```

```

//          for(String s : ls)
//          System.out.println(s);
//
//          System.out.println("Programm terminiert normal");
//
//      }
//  }

```

```

    }
//}

```

Klasse: Punkt

```

package _11_03_2026;

public class Punkt {

    private double x;
    private double y;
    public Punkt(double x, double y) {

        this.x = x;
        this.y = y;
    }
    public double getX() {
        return x;
    }
    public void setX(double x) {
        this.x = x;
    }
    public double getY() {
        return y;
    }
    public void setY(double y) {
        this.y = y;
    }
    @Override
    public String toString() {
        return x + ";" + y + " ,";
    }

    public String toCSV() {
        return x + ";" + y;
    }
}

```



```
}
```

Klasse: Service

```
package _11_03_2026;
```

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.ArrayList;
```

```
/**
 * Die Methode soll die Punkte in der Liste zeilenweise in eine CSV
 * datei schreiben und zwar im CSV Format
 *
 * z.B.könnte es so aussehen:
 *
 * 0.3;0.5
 * 0.25;0.65
 * 0.89;0.85
 * usw
 */
public class Service {

    public static void writeToCsv(ArrayList<Punkt> punkte,String
pathToFile) {

        File f = new File(pathToFile);

        PrintWriter pw =null;
        try {
            boolean success= f.createNewFile();
            if (success)
                System.out.println("Datei wurde erzeugt");
            else
                System.out.println("Datei existiert bereits");

            pw = new PrintWriter(new FileWriter(f,true));

            for (Punkt p: punkte) {
                pw.println(p.toCSV());
            }
        }
        // pw.close();
        System.out.println("Das Schreiben in die Datei endete
```

```

normal.");

        }catch (IOException e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Resource Freigabe");
            if(pw !=null)
                pw.close();
        }

    }

    public static void writeCirclesToCsvs(ArrayList<Kreis> circles,
String pathToFile) {
        PrintWriter pw = null;
        try {
            pw = new PrintWriter(pathToFile);
            for (Kreis k : circles)
                pw.println(k.toCSV());

            System.out.println("Das Auschreiben in die Datei
erfolgreich ausgeführt");
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        }
        finally {
            System.out.println("Resource Freigabe");
            if(pw != null)
                pw.close();
        }

        System.out.println("Method terminated...");
    }
}

```

```

}

```

Package: 16_03_2026

Klasse: MieteComparator
package 16_03_2026;

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

/**
 * API:
 * -Interfaces Comparable<T>, Copmarator<T>
 * -Hilfsklassen: Arrays, Collections
 * -Methoden: sort, binarySearch
 * -Sammlung von Daten: List oder Array
 */

class Mietekomparator implements Comparator<Wohnung>{

    @Override
    public int compare(Wohnung o1, Wohnung o2) {

        return
Double.valueOf(o1.getMiete()).compareTo(Double.valueOf(o2.getMiete()))
;
    }

}

public class Test {

    private static void sortAndPrintBasedOnEinzugsdatum(Wohnung[] w) {
        Arrays.sort(w);
        System.out.println("====Wohnungen sortiert nach
Einzugsdatum====");
        System.out.println(Arrays.toString(w));
    }

    private static void sortAndPrintBasedOnMiete(Wohnung[] w) {
        Comparator<Wohnung> c= new Mietekomparator();
        Arrays.sort(w, c);
        System.out.println("====Wohnungen sortiert nach
Miete====");
        System.out.println(Arrays.toString(w));
    }

    private static void sortListOfWohnungen(List<Wohnung> w) {
        Collections.sort(w);
        System.out.println("====Sortiert nach Datum in der
Liste====");
        System.out.println(w);
    }

```

```
}
```

```
public static void main(String[] args) {
```

```
    Wohnung w1 =new Wohnung("10439", "Paul-Robeson-Str 36",
807.47, 2,LocalDate.of(2010, 9, 1));
    Wohnung w2 =new Wohnung("10439", "Paul-Robeson-Str 35",
807.47, 3,LocalDate.of(2010, 9, 1));

    int i=w1.compareTo(w2);
    System.out.println(i);

    Wohnung[] whg= {
        new Wohnung("10439", "Paul-Robeson-Str 34",
807.47, 2,LocalDate.of(2010, 9, 1)),
        new Wohnung("10439","Kugler Str.
69",1400.69,2,LocalDate.of(2023, 3, 2)) ,
        new Wohnung("10439","Erich-Weinert-Str.
78",1165.37,3,LocalDate.of(2005, 12, 15)),
        new Wohnung("10788","Wielandstr.
38",560,1,LocalDate.of(2009, 12, 15)),
        new Wohnung("10788","WundsStr.
38",450,1,LocalDate.of(2010, 12, 15))
    };
};
```

```
List<Wohnung> whgList=new ArrayList<Wohnung>();
```

```
    whgList.add(new Wohnung("10439", "Paul-Robeson-Str 34",
807.47, 2,LocalDate.of(2010, 9, 1)));
    whgList.add(new Wohnung("10439","Kugler Str.
69",1400.69,2,LocalDate.of(2023, 3, 2)));
    whgList.add(new Wohnung("10439","Erich-Weinert-Str.
78",1165.37,3,LocalDate.of(2005, 12, 15)));
    whgList.add(new Wohnung("10788","Wielandstr.
38",560,1,LocalDate.of(2009, 12, 15)));
    whgList.add(new Wohnung("10788","WundsStr.
38",450,1,LocalDate.of(2010, 12, 15)));
```

```
    System.out.println(Arrays.toString(whg));
    sortAndPrintBasedOnEinzugsdatum(whg);
    sortAndPrintBasedOnMiete(whg);
```

```
    sortListOfWohnungen(whgList);
```

```
//      System.out.println(Arrays.toString(whg)); //bleibt sortiert

      //System.out.println(Arrays.binarySearch(whg, new
Wohnung("10439","Erich-Weinert-Str.78",1165.37,3)));
    }

}
```

Klasse: SortierenUndSuchen

```
package _16_03_2026;

import java.time.LocalDate;
import java.util.Arrays;

public class SortierenUndSuchen {

    public static void main(String[] args) {
        int [] numbers = {1,3,5,7,9,2,4,6,8,6,4};
// {1,2,3,4,5,6,7,8,9};

        System.out.println(Arrays.binarySearch((numbers), 5));//(-
(insertion point) - 1) wenn es nicht da ist, sonst den index

        Arrays.sort(numbers);
        System.out.println(Arrays.toString(numbers));
        System.out.println(Arrays.binarySearch((numbers), 5));

        //

        LocalDate[] dates= {LocalDate.of(1981, 1, 19),
                             LocalDate.of(1987,12,15),
                             LocalDate.of(1984,10,27)
                             };

        System.out.println(dates.toString());
        System.out.println(Arrays.binarySearch(dates,
LocalDate.of(1984,10,27)));

        Arrays.sort(dates);
        System.out.println(dates.toString());
        System.out.println(Arrays.binarySearch(dates,
LocalDate.of(1984,10,27)));

    }

}
```

Klasse: WrapperKlassen

```
package _16_03_2026;
```

```
/**
 * Primitive Datentypen                Äquivalente Klassen (Wrapper-
Klassen)
 *
 * boolean                            Boolean
 * byte                               Byte
 * short                             Short
 * char                              Character
 * int                               Integer
 * float                             Float
 * long                              Long
 * double                            Double
 *
 */
```

```
public class WrapperKlassen {

    public static void main(String[] args) {
        int x=8;//primitiv
        Integer y=8;//eine Klasse....für die man Methoden aufrufen
kann
        Integer z= Integer.valueOf(7); //Fabrikmethode

        System.out.println(x);
        System.out.println(y.MAX_VALUE);
        System.out.println(z);

    }

}
```

Klasse: Zimmer

```
package _16_03_2026;
```

```
public class Zimmer implements Comparable<Zimmer> {

    @Override
    public int compareTo(Zimmer o) {
        // TODO Auto-generated method stub
        return 0;
    }

}
```

Package: _17_03_2026

Klasse: Tescht

```
package _17_03_2026;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class Tescht {

    public static void main(String[] args) {
        List<Integer> a = new ArrayList<Integer>(Arrays.asList(1, 3,
5, 7, 9, 2, 4, 6, 8));

        List<Integer> b = new ArrayList<Integer>(a);
        // b.addAll(a);

        Collections.sort(b);
        System.out.println("a=" + a);
        System.out.println("b=" + b);

    }

}
```

Package: _18_03_26

Klasse: AnonymInnereKlassen

```
package _18_03_26;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import _16_03_2026.Wohnung;

public class AnonymeInnereKlassen {

    public static void main(String[] args) {
        List<Wohnung> whgList=new ArrayList<Wohnung>();

        whgList.add(new Wohnung("10439", "Paul-Robeson-Str 34",
807.47, 2,LocalDate.of(2010, 9, 1)));
        whgList.add(new Wohnung("10439","Kugler Str.
69",1400.69,2,LocalDate.of(2023, 3, 2)));

    }

}
```

```

        whgList.add(new Wohnung("10439","Erich-Weinert-Str.
78",1165.37,3,LocalDate.of(2005, 12, 15)));
        whgList.add(new Wohnung("10788","Wielandstr.
38",560,1,LocalDate.of(2009, 12, 15)));
        whgList.add(new Wohnung("10788","WundsStr.
38",450,1,LocalDate.of(2010, 12, 15)));

//      Comparator c=new Comparator<Wohnung>() {
//
//          @Override
//          public int compare(Wohnung o1, Wohnung o2) {
//              return
Integer.compare(o1.getZimmerAnzahl(),o2.getZimmerAnzahl());
//          }
//
//      };

//Lambda expression (erst seit Java Version 1.8)

// (parameterliste) -> expression
Comparator<Wohnung> c= ( o1, o2) ->
Integer.compare(o1.getZimmerAnzahl(),o2.getZimmerAnzahl());

Collections.sort(whgList,c);

//noch schneller:      Collections.sort(whgList,( o1, o2) ->
Integer.compare(o1.getZimmerAnzahl(),o2.getZimmerAnzahl()));

    }

}

class C{
    //Attribute

    //Konstruktoren

    //Methoden

    class D{

    }
}

```



```

}package _25_03_2026;

import java.util.Locale;

public class API {

    public static void main(String[] args) {
        Locale loic =new Locale("de", "DE");

        Locale loc2 =Locale.GERMANY;

        Locale loc3 = new Locale.Builder()
                        .setLanguage("de")
                        .setRegion("DE")
                        .build();
        Locale et= new Locale.Builder()
                        .setLanguage("et")
                        .setRegion("ET")
                        .build();

    }

}

```

Klasse: TeschtleLambda
package _18_03_26;

```

public class TeschtleLambda {

    public static void main(String[] args) {
        Vehicle v= new Vehicle() {

            @Override
            public void move() {
                // TODO Auto-generated method stub

            }

            @Override
            public boolean canFly() {
                // TODO Auto-generated method stub
                return false;
            }
        };

        //      System.out.println(Math.round(Math.random()));

        v.move();

        //      Flyable f= new Flyable() {

```

```

//
//      @Override
//      public void fly() {
//          System.out.println("OK,ich fliege...");
//      }
//  };

// mit Lambda
Flyable f= ()-> System.out.println("OK,ich fliege...");

    f.fly();

}

}

abstract class Vehicle{
    public abstract void move();
    public abstract boolean canFly();
}

interface Flyable{
    void fly();
}

//class Car extends Vehicle{
//
//      @Override
//      public void move() {
//          // TODO Auto-generated method stub
//      }
//
//      @Override
//      public boolean canFly() {
//          // TODO Auto-generated method stub
//          return false;
//      }
//
//}

```

Package: **_19_03_2026**

Klasse: **Filter**

```
package _19_03_2026;

import java.util.Collection;
import java.util.List;
import java.util.function.Predicate;
import java.util.stream.Collectors;

public class Filter {

    // Besser keinen Typnamen wie "Collection" als Variablennamen
    // verwenden
    private static final int[] NUMBERS = { 1, 2, 3, 4, 5, 6, 7, 8,
    9 };

    public static void main(String[] args) {
        // Array in eine Collection (z.B. List) umwandeln
        List<Integer> baseCollection =
        java.util.Arrays.stream(NUMBERS).boxed().collect(Collectors.toList());

        // Methode aufrufen
        Collection<Integer> evenNumbers =
        findEvenNumbers(baseCollection);
        System.out.println(evenNumbers);
    }

    public static Collection<Integer>
    findEvenNumbers(Collection<Integer> baseCollection) {
        Predicate<Integer> streamsPredicate = item -> item % 2 == 0;

        return
        baseCollection.stream().filter(streamsPredicate).collect(Collectors.to
        List());
    }
}package _20_03_2026;

import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class Filter {

    public static List<Integer> createRandoms(int n) {
        List<Integer> result = new ArrayList<Integer>();
        Random r = new Random();
        for (int i =0;i<n;i++) {
```

```

        result.add(r.nextInt(100));
    }
    return result;
}

public static List<Integer> filterOdds(List<Integer> zahlen) {
    List<Integer> odds = new ArrayList<Integer>();
    for ( Integer zahl: zahlen) {
        if (zahl%2 !=0 )
            odds.add(zahl);
    }
    return odds;
}

public static List<Integer> filter(List<Integer>
zahlen,IntegerTester t) {
    List<Integer> result = new ArrayList<Integer>();
    for ( Integer zahl: zahlen) {
        if (t.test(zahl))
            result.add(zahl);
    }
    return result;
}

public static List<Integer> filterEven(List<Integer> zahlen) {
    List<Integer> evens = new ArrayList<Integer>();
    for ( Integer zahl: zahlen) {
        if (zahl%2 ==0 )
            evens.add(zahl);
    }
    return evens;
}

public static List<Integer> filterBigger50(List<Integer> zahlen) {
    List<Integer> biggerFifty = new ArrayList<Integer>();
    for ( Integer zahl: zahlen) {
        if (zahl>50 )
            biggerFifty.add(zahl);
    }
    return biggerFifty;
}

public static List<Integer> filterDivisibleThree(List<Integer>
zahlen) {
    List<Integer> divisibleThree = new ArrayList<Integer>();
    for ( Integer zahl: zahlen) {

```

```

        if (zahl%3 ==0 )
            divisibleThree.add(zahl);
    }
    return divisibleThree;
}

public static List<Integer> filterIncludeEight(List<Integer>
zahlen) {
    List<Integer> includeEight = new ArrayList<Integer>();
    for ( Integer zahl: zahlen) {
        if (zahl%10 ==8 || zahl/10==8)
            includeEight.add(zahl);
    }
    return includeEight;
}

public static List<Integer> filterIncludeEightV2(List<Integer>
zahlen) {
    List<Integer> includeEight = new ArrayList<Integer>();
    for ( Integer zahl: zahlen) {
        List<Integer> ziffern= new ArrayList<Integer>();
        Integer temp=zahl;
        while (temp !=0) {
            ziffern.add(temp%10);
            temp /=10;
        }
        for (Integer ziffer : ziffern) {
            if(ziffer==8) {
                includeEight.add(zahl);
                break;
            }
        }
    }
    return includeEight;
}

/**
 * implementiere:
 * die geraden Zahlen filtern
 * die Zahlen grösser als 50
 * Die Zahlen die durch 3 teilbar sind
 *
 *
 *
 *
 * @param args
 */

```

```

    public static void main(String[] args) {
        List<Integer> numbers = createRandoms(20);
//      numbers.add(88);
        System.out.println((numbers));
        System.out.println(filterOdds(numbers));
        System.out.println(filterEven(numbers));
        System.out.println(filterBigger50(numbers));
        System.out.println(filterDivisibleThree(numbers));
        System.out.println(filterIncludeEight(numbers));
        System.out.println(filterIncludeEightV2(numbers));

        class Odd implements IntegerTester{
            @Override
            public boolean test(Integer i) {
                return i%2==1;
            }
        }
        System.out.println(filter(numbers,new Odd()));
        //schneller:
        System.out.println(filter(numbers,(i)-> i%2==1));
        System.out.println(filter(numbers,(i)-> i%2==0));
        System.out.println(filter(numbers,(i)-> i>50));
        System.out.println(filter(numbers,(i)-> i%3==0));
        System.out.println(filter(numbers,(i)-> i%10==8 || i/10==8));

    }

}

```

Klasse: RentOutOfLimitException

```
package _19_03_2026;
```

```
import java.time.LocalDate;
```

```
class RentOutOfLimitException extends Exception{
```

```
}
```

```
public class Wohnung implements Comparable<Wohnung>{
```

```

    private final String plz;
    private final String str;
    private double miete;
    private final int zimmerAnzahl;

```

```

private final LocalDate einzugsdatum;

public final static double mieteObereGrenz =2500;

public Wohnung(String plz, String str, double miete, int
zimmerAnzahl, LocalDate einzugsdatum)
    throws RentOutOfLimitException
//throws NullPointerException,IllegalArgumentException <- brauche
ich nicht da ein runtime fehler(unchecked)
{
    if (plz!=null)
        this.plz = plz;
    else
        throw new NullPointerException("Plz ist null");
    if (str!= null)
        this.str = str;
    else
        throw new NullPointerException("Strasse ist null");

    if(miete>0)
    {
        if (miete <= mieteObereGrenz)
            this.miete = miete;
        else {
            //this.miete=mieteObereGrenz; so würde man das
behandeln
            throw new RentOutOfLimitException();
        }
    }else
        throw new IllegalArgumentException("Miete muss positiv
sein!");
    if(zimmerAnzahl>0)
        this.zimmerAnzahl = zimmerAnzahl;
    else
        throw new IllegalArgumentException("Zimmeranzahl muss
positiv sein");

    this.einzugsdatum = einzugsdatum;
}
public String getPlz() {
    return plz;
}
public String getStr() {
    return str;
}
public double getMiete() {
    return miete;
}
public int getZimmerAnzahl() {
    return zimmerAnzahl;
}

```

```

    }
    @Override
    public String toString() {
        return "Wohnung [" + plz + " : " + str + " : " + miete + " : "
+ zimmerAnzahl + " : "+einzugsdatum+"]\n";
    }

```

```

// @Override
// public int compareTo(Wohnung o) {
//     if (miete<o.miete)
//         return -1;
//     else if (miete==o.miete)
//         return 0;
//     else
//         return 1;
//     //besser wäre zimmerAnzahl - o. zimmerAnzahl
//     // oder miete-o.miete
// }

```

```

/**
 * Concatenieren falls nach mehreren Größen sortiert werden soll
 * die reihenfolge ist dabei zu beachten
 */
// @Override
// public int compareTo(Wohnung o) {
//     // nach Zimmeranzahl vergleichen : int ->Integer
//     return
Integer.valueOf(zimmerAnzahl).compareTo(Integer.valueOf(o.zimmerAnzahl
));
// //autoboxing:return
Integer.valueOf(zimmerAnzahl).compareTo(o.zimmerAnzahl);
// }

```

```

// @Override
// public int compareTo(Wohnung o) {
//     //nach Miete vergleichen
//     return
Double.valueOf(miete).compareTo(Double.valueOf(o.miete));
// }

```

```

// @Override
// public int compareTo(Wohnung o) {
//     return Integer.valueOf(plz.trim() .compareTo(o.plz.trim()));
// }

```



```
//
// @Override
// public int compareTo(Wohnung o) {
//     return Integer.valueOf(str.trim().compareTo(o.str.trim()));
// }
```

```
    @Override
    public int compareTo(Wohnung o) {
        return Integer.valueOf((einzugsdatum + plz
+str) .compareTo(o.einzugsdatum+ o.plz+ o.str ));
    }
```

```
}
```

Klasse: Test

```
package _19_03_2026;
```

```
import java.time.LocalDate;
```

```
public class Test {
    public static void main(String[] args) {

        Wohnung w;
        try {
            w = new Wohnung("10439", "Kuglerstr. 69", 10000, 2,
LocalDate.of(2025, 12, 15));
            System.out.println(w);
        } catch (RentOutOfLimitException e) {
            System.out.println("Die Miete darf nicht die Obergrenze
überschreiten,deshalb auf obere Grenze gesetzt !");
            try {
                w = new Wohnung("10439", "Kuglerstr. 69",
Wohnung.mieteObereGrenz, 2, LocalDate.of(2025, 12, 15));
                System.out.println(w);
            } catch (RentOutOfLimitException e1) {
                e1.printStackTrace();
            }
        }

    }

}
```

Package: _20_03_2026

Klasse: DoubleTester

```
package _20_03_2026;
```

```
public interface DoubleTester {  
  
    boolean test(Double d);  
}
```

Klasse: FilterLambda

```
package _20_03_2026;
```

```
import java.util.ArrayList;  
import java.util.List;  
import java.util.Random;
```

```
public class FilterLambda {  
  
    public static List<Integer> createRandoms(int n) {  
        List<Integer> result = new ArrayList<Integer>();  
        Random r = new Random();  
        for (int i =0;i<n;i++) {  
            result.add(r.nextInt(100));  
        }  
        return result;  
    }  
  
    public static List<Integer> filter(List<Integer>  
zahlen,IntegerTester t) {  
        List<Integer> result = new ArrayList<Integer>();  
        for ( Integer zahl: zahlen) {  
            if (t.test(zahl))  
                result.add(zahl);  
        }  
        return result;  
    }  
  
    public static void main(String[] args) {  
        List<Integer> numbers = createRandoms(20);  
  
        System.out.println(numbers);  
        System.out.println(filter(numbers,(i) -> i%2==1));  
        System.out.println(filter(numbers,(i) -> i%2==0));  
    }  
}
```

```

        IntegerTester greaterThan50 = (i) -> i>50;
        System.out.println(filter(numbers,greaterThan50));

        System.out.println(filter(numbers,(i) -> i>50));
        System.out.println(filter(numbers,(i) -> i%3==0));
        System.out.println(filter(numbers,(i) -> i%10==8 || i/10==8));
    }

}

```

Klasse: FilterTescht

```

package _20_03_2026;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import _16_03_2026.Wohnung;

public class FilterTescht {

    static List<Wohnung > createWohnungen(){
        List<Wohnung> whgList = new ArrayList<>();

        Collections.addAll(whgList,
            new Wohnung("10439", "Paul-Robeson-Str 34", 807.47, 2,
LocalDate.of(2010, 9, 1)),
            new Wohnung("10439", "Kugler Str. 69", 1400.69, 2,
LocalDate.of(2023, 3, 2)),
            new Wohnung("10439", "Erich-Weinert-Str. 78", 1165.37, 3,
LocalDate.of(2005, 12, 15)),
            new Wohnung("10788", "Wielandstr. 38", 560, 1,
LocalDate.of(2009, 12, 15)),
            new Wohnung("10788", "WundsStr. 38", 450, 1,
LocalDate.of(2010, 12, 15)),
            new Wohnung("10439", "Schönhauser Allee 12", 945.80, 2,
LocalDate.of(2018, 5, 1)),
            new Wohnung("10439", "Greifenhagener Str. 21", 1250.00, 3,
LocalDate.of(2016, 11, 15)),
            new Wohnung("10439", "Stargarder Str. 5", 735.50, 1,
LocalDate.of(2012, 2, 1)),
            new Wohnung("10439", "Gleimstr. 44", 1590.90, 4,
LocalDate.of(2020, 7, 1)),
            new Wohnung("10439", "Pappelallee 97", 1020.30, 2,
LocalDate.of(2008, 9, 30)),
            new Wohnung("10788", "Nürnberger Str. 15", 880.00, 1,
LocalDate.of(2014, 4, 1)),
            new Wohnung("10788", "Kurfürstenstr. 3", 1325.75, 2,
LocalDate.of(2019, 10, 1)),

```

```

        new Wohnung("10788", "Keithstr. 22", 1749.99, 3,
LocalDate.of(2021, 1, 15)),
        new Wohnung("10788", "Lützowstr. 40", 640.00, 1,
LocalDate.of(2007, 6, 1)),
        new Wohnung("10788", "Potsdamer Str. 120", 1995.45, 4,
LocalDate.of(2024, 8, 1)),
        new Wohnung("10439", "Paul-Robeson-Str 12", 890.25, 2,
LocalDate.of(2011, 3, 1)),
        new Wohnung("10439", "Paul-Robeson-Str 56", 975.90, 3,
LocalDate.of(2018, 10, 1)),
        new Wohnung("10439", "Kugler Str. 12", 1250.00, 2,
LocalDate.of(2020, 5, 15)),
        new Wohnung("10439", "Kugler Str. 101", 1499.99, 3,
LocalDate.of(2022, 9, 1)),
        new Wohnung("10439", "Erich-Weinert-Str. 5", 980.40, 2,
LocalDate.of(2007, 1, 1)),
        new Wohnung("10439", "Erich-Weinert-Str. 120", 1355.70, 4,
LocalDate.of(2019, 6, 1)),
        new Wohnung("10788", "Wielandstr. 5", 630.00, 1,
LocalDate.of(2010, 4, 1)),
        new Wohnung("10788", "Wielandstr. 77", 1420.50, 3,
LocalDate.of(2017, 2, 1)),
        new Wohnung("10788", "Wielandstr. 102", 1899.00, 4,
LocalDate.of(2021, 11, 1)),
        new Wohnung("10788", "Kugler Str. 5", 1100.00, 2,
LocalDate.of(2015, 8, 1))
    );

```

```

        return whgList;
    }

```

```

    static void print(List<Wohnung> w) {
        for (Wohnung wh : w) {
            System.out.println(wh);
        }
    }

```

```

    public static void main(String[] args) {
        print(createWohnungen());
        // WohnungTester wt = (Wohnung w) ->{
        //     if(w.getPlz().equals("10439"))
        //         return true;
        //     else
        //         return false;
        // };
        Predicate<Wohnung> wt = ( w) -> w.getPlz().equals("10439");
    }

```

```

    List<Wohnung> f= FilterWohnungen.filtern(createWohnungen(),

```

```
wt);  
    print(f);
```

```
}
```

```
}
```

Klasse: Filter_selbst

```
package _20_03_2026;
```

```
import java.util.Collection;  
import java.util.List;  
import java.util.function.Predicate;  
import java.util.stream.Collectors;
```

```
public class Filter_selbst {
```

```
    // Besser keinen Typnamen wie "Collection" als Variablennamen  
    verwenden
```

```
    private static final int[] NUMBERS = { 1, 2, 3, 4, 5, 6, 7, 8,  
    9 };
```

```
    public static void main(String[] args) {  
        // Array in eine Collection (z.B. List) umwandeln  
        List<Integer> baseCollection =  
        java.util.Arrays.stream(NUMBERS).boxed().collect(Collectors.toList());  
  
        // Methode aufrufen  
        Collection<Integer> evenNumbers =  
        findEvenNumbers(baseCollection);  
        System.out.println(evenNumbers);  
    }
```

```
    public static Collection<Integer>  
    findEvenNumbers(Collection<Integer> baseCollection) {  
        Predicate<Integer> streamsPredicate = item -> item % 2 == 0;  
  
        return  
        baseCollection.stream().filter(streamsPredicate).collect(Collectors.to  
        List());  
    }  
}package _10_22_lab;
```

```
import java.time.LocalDate;
```

```
/**  
 * keyword final:
```

```

*           -unveränderlich
*           -final   deklarierte Variablen müssen initialisiert werden,
aber Schreibzugriff ist verboten
*           -Jeder Konstruktor muss nicht initialisierte final
deklarierte Attribute initialisieren
*/

```

```

public class FinalAttribute {

    public static void main(String[] args) {
        Person p1=new Person();
        Person p2=new Person("Slamonelle","Luigi",LocalDate.of(1966,
6,30),"Palermo","Napoli",120.0,172);
        p1.zeigeAttribute();
        p2.zeigeAttribute();
    }

}

```

```

class Person{
    /**
     * name
     * vorname
     * geburtsdatum
     * geburtsort
     * wohnort
     * gewicht
     * groesse
     */

    private String name;
    private String vorname;
    private final LocalDate geburtsdatum;
    private final String geburtsort;
    private String wohnort;
    private double gewicht;
    private int groesse;

    public Person() {
//      this.vorname="Linus";
//      this.name="Torwaldes";
//      this.geburtsdatum = LocalDate.of(1970,1,1);//erforderlich
//      this.geburtsort = "Copenhagen";//erforderlich
//      this.wohnort="Freiburg";
//      this.gewicht=.2;
//      this.groesse=20;

this("Torwaldes","Linus",LocalDate.of(1970,1,1),"Copenhagen","Freiburg",3.5,51);//muss die erste anweisung sein
    }
}

```

```

//Ein zweiter Konstruktor

    public Person(String name, String vorname, LocalDate geburtsdatum,
String geburtsort, String wohnort,
        double gewicht, int groesse) {
        this.name = name;
        this.vorname = vorname;
        this.geburtsdatum = geburtsdatum;
        this.geburtsort = geburtsort;
        this.wohnort = wohnort;
        this.gewicht = gewicht;
        this.groesse = groesse;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getVorname() {
        return vorname;
    }

    public void setVorname(String vorname) {
        this.vorname = vorname;
    }

    public String getWohnort() {
        return wohnort;
    }

    public void setWohnort(String wohnort) {
        this.wohnort = wohnort;
    }

    public double getGewicht() {
        return gewicht;
    }

    public void setGewicht(double gewicht) {
        this.gewicht = gewicht;
    }

    public int getGroesse() {
        return groesse;
    }

```

```

    }

    public void setGroesse(int groesse) {
        this.groesse = groesse;
    }

    public LocalDate getGeburtsdatum() {
        return geburtsdatum;
    }

    public String getGeburtsort() {
        return geburtsort;
    }

    public void zeigeAttribute() {
        System.out.println("=====");
        System.out.printf("Name: %21s\nVorname: %18s\nGeburtsdatum:
%13s\nGeburtsort: %15s\nWohnort: %18s\nGewicht: %16.2fkg\nGröße:
%18dcm
%n", name, vorname, geburtsdatum, geburtsort, wohnort, gewicht, groesse);
        System.out.println("=====");
    }

}package _10_22;

import java.time.LocalDate;

/**
 * key word final:
 *
 *      - Unveränderlich
 *      - final deklarierte Variable müssen initialisiert
werden, aber Schreibzugriff ist verboten
 *      - Jeder Konstruktor muss nicht initialisierte final
deklarierte Attribute initialisieren
 */
public class FinalAttribute {
    public static void main(String[] args) {
        final int i = 0;

        //i = 23;

        Person p = new Person("Desat", "Olga", LocalDate.of(2005, 12, 26),
"Bochum", "Köln", 3, 51);
        p.anzeigen();
    }
}

```



```

        //...
    }
}
/**
 * Unterscheidung zwischen static, final, ...
 */
class Person{
    /**
     * name
     * vorname
     *      geburtsdatum
     *      geburtsort
     * wohnort
     * gewicht
     * groesse

    */
    private String name;
    private String vorname;
    private final LocalDate geburtsdatum;
    private final String geburtsort ;
    private String wohnort;
    private double gewicht;
    private int groesse;

    public Person() {
        this("Torwaldes", "Linus", LocalDate.of(1969, 12, 28),
"Helsinki", "Helsinki", 3.5, 51);
    }
    //Einen zweiten Konstruktor
    public Person(String name, String vorname, LocalDate geburtsdatum,
String geburtsort, String wohnort, double gewicht, int groesse) {
        this.name = name;
        this.vorname = vorname;
        this.geburtsdatum = geburtsdatum;//
        this.geburtsort = geburtsort;
        this.wohnort = wohnort;
        this.gewicht = gewicht;//
        this.groesse = groesse;//
    }
    public void anzeigen() {
        System.out.printf("%s; %s; %s; %s; %s; %.2f; %d",name,
vorname, geburtsdatum.toString(),geburtsort, wohnort, gewicht, groesse
);
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {

```

```

        this.name = name;
    }
    public String getVorname() {
        return vorname;
    }
    public void setVorname(String vorname) {
        this.vorname = vorname;
    }
    public String getWohnort() {
        return wohnort;
    }
    public void setWohnort(String wohnort) {
        this.wohnort = wohnort;
    }
    public double getGewicht() {
        return gewicht;
    }
    public void setGewicht(double gewicht) {
        this.gewicht = gewicht;
    }
    public int getGroesse() {
        return groesse;
    }
    public void setGroesse(int groesse) {
        this.groesse = groesse;
    }
    public LocalDate getGeburtsdatum() {
        return geburtsdatum;
    }
    public String getGeburtsort() {
        return geburtsort;
    }
}

//anzeige: formatierte Ausgabe
//Setters und Getters
}package simple;

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JButton;
import java.awt.BorderLayout;
import javax.swing.JLabel;
import javax.swing.JTextField;

public class FirstGUI {

    private JFrame frame;
    private JTextField textField;

```

```

/**
 * Launch the application.
 */
public static void main(String[] args) {
    EventQueue.invokeLater(new Runnable() {
        public void run() {
            try {
                FirstGUI window = new FirstGUI();
                window.frame.setVisible(true);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });
}

/**
 * Create the application.
 */
public FirstGUI() {
    initialize();
}

/**
 * Initialize the contents of the frame.
 */
private void initialize() {
    frame = new JFrame();
    frame.setBounds(100, 100, 450, 300);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    JButton btnNewButton = new JButton("Klick mich!");
    frame.getContentPane().add(btnNewButton, BorderLayout.SOUTH);

    JLabel lblNewLabel = new JLabel("Eingabe");
    frame.getContentPane().add(lblNewLabel, BorderLayout.WEST);

    textField = new JTextField();
    frame.getContentPane().add(textField, BorderLayout.CENTER);
    textField.setColumns(10);
}

}

Klasse: IntegerTester
package _20_03_2026;

public interface IntegerTester {

```

```

        boolean test(Integer i);
    }

Klasse: PredicateStringExample
package _20_03_2026;

import java.util.Arrays;
import java.util.List;
import java.util.function.Predicate;
import java.util.stream.Collectors;

public class PredicateStringExample {

    public static void main(String[] args) {
        String[] WORDS =
            { "Baum", // B..., zu kurz, endet nicht mit n → raus
              "Bahn", // zu kurz
              "Besen", // B...n, 5 Buchstaben → rein
              "Balkon", // B...n, 6 Buchstaben → rein
              "Bienen", // B...n, 6 Buchstaben → rein
              "Boot", // zu kurz, endet nicht mit n
              "Bogen", // B...n, 5 Buchstaben → rein
              "Haus", // fängt nicht mit B an, zu kurz
            };

        // Array -> List
        List<String> baseCollection = Arrays.asList(WORDS);

        // Predicates definieren
        Predicate<String> startsWithB = s -> s.startsWith("B");
        Predicate<String> endsWithN = s -> s.endsWith("n");
        Predicate<String> lengthAtLeast5 = s -> s.length() >= 5;

        // Kombiniertes Predicate: B am Anfang, n am Ende, Länge >= 5
        Predicate<String> complexFilter =
            startsWithB.and(endsWithN).and(lengthAtLeast5);

        // Filtern und Ergebnis sammeln (veränderbare Liste)
        List<String> filtered =
            baseCollection.stream().filter(complexFilter).collect(Collectors.toList());

        System.out.println(filtered);
    }
}package _19_03_2026;

import java.util.Arrays;
import java.util.List;

```

```

import java.util.function.Predicate;
import java.util.stream.Collectors;

public class PredicateStringExample {

    public static void main(String[] args) {
        String[] WORDS =
            { "Baum", // B..., zu kurz, endet nicht mit n → raus
              "Bahn", // zu kurz
              "Besen", // B...n, 5 Buchstaben → rein
              "Balkon", // B...n, 6 Buchstaben → rein
              "Bienen", // B...n, 6 Buchstaben → rein
              "Boot", // zu kurz, endet nicht mit n
              "Bogen", // B...n, 5 Buchstaben → rein
              "Haus", // fängt nicht mit B an, zu kurz
            };

        // Array -> List
        List<String> baseCollection = Arrays.asList(WORDS);

        // Predicates definieren
        Predicate<String> startsWithB = s -> s.startsWith("B");
        Predicate<String> endsWithN = s -> s.endsWith("n");
        Predicate<String> lengthAtLeast5 = s -> s.length() >= 5;

        // Kombiniertes Predicate: B am Anfang, n am Ende, Länge >= 5
        Predicate<String> complexFilter =
            startsWithB.and(endsWithN).and(lengthAtLeast5);

        // Filtern und Ergebnis sammeln (veränderbare Liste)
        List<String> filtered =
            baseCollection.stream().filter(complexFilter).collect(Collectors.toList());

        System.out.println(filtered);
    }
}

package _03_27;

public interface PricingStrategie {
    public abstract double calculatePrice(double weight, double
distance);
}

```

Klasse: RentOutOfLimitException

```
package _20_03_2026;
```

```
import java.time.LocalDate;
```

```
class RentOutOfLimitException extends Exception{
```

```
}
```

```
public class Wohnung implements Comparable<Wohnung>{

    private final String plz;
    private final String str;
    private double miete;
    private final int zimmerAnzahl;
    private final LocalDate einzugsdatum;

    public final static double mieteObereGrenz =2500;

    public Wohnung(String plz, String str, double miete, int
zimmerAnzahl, LocalDate einzugsdatum)
        throws RentOutOfLimitException
    //throws NullPointerException,IllegalArgumentException <- brauche
ich nicht da ein runtime fehler(unchecked)
    {
        if (plz!=null)
            this.plz = plz;
        else
            throw new NullPointerException("Plz ist null");
        if (str!= null)
            this.str = str;
        else
            throw new NullPointerException("Strasse ist null");

        if(miete>0)
        {
            if (miete <= mieteObereGrenz)
                this.miete = miete;
            else {
                //this.miete=mieteObereGrenz; so würde man das
behandeln
                throw new RentOutOfLimitException();
            }
        }else
            throw new IllegalArgumentException("Miete muss positiv
sein!");
        if(zimmerAnzahl>0)
            this.zimmerAnzahl = zimmerAnzahl;
        else
            throw new IllegalArgumentException("Zimmeranzahl muss
positiv sein");

        this.einzugsdatum = einzugsdatum;
    }
    public String getPlz() {
        return plz;
    }
}
```

```

    }
    public String getStr() {
        return str;
    }
    public double getMiete() {
        return miete;
    }
    public int getZimmerAnzahl() {
        return zimmerAnzahl;
    }
    @Override
    public String toString() {
        return "Wohnung [" + plz + " : " + str + " : " + miete + " : "
+ zimmerAnzahl + " : "+einzugsdatum+"]\n";
    }

```

```

// @Override
// public int compareTo(Wohnung o) {
//     if (miete<o.miete)
//         return -1;
//     else if (miete==o.miete)
//         return 0;
//     else
//         return 1;
//     //besser wäre zimmerAnzahl - o. zimmerAnzahl
//     // oder miete-o.miete
// }

```

```

/**
 * Concatenieren falls nach mehreren Größen sortiert werden soll
 * die reihenfolge ist dabei zu beachten
 */
// @Override
// public int compareTo(Wohnung o) {
//     // nach Zimmeranzahl vergleichen : int ->Integer
//     return
Integer.valueOf(zimmerAnzahl).compareTo(Integer.valueOf(o.zimmerAnzahl
));
// //autoboxing:return
Integer.valueOf(zimmerAnzahl).compareTo(o.zimmerAnzahl);
// }

```

```

// @Override
// public int compareTo(Wohnung o) {

```

```

//      //nach Miete vergleichen
//      return
Double.valueOf(miete).compareTo(Double.valueOf(o.miete));
//  }

//  @Override
//  public int compareTo(Wohnung o) {
//      return Integer.valueOf(plz.trim() .compareTo(o.plz.trim()));
//  }
//
//  @Override
//  public int compareTo(Wohnung o) {
//      return Integer.valueOf(str.trim().compareTo(o.str.trim()));
//  }

    @Override
    public int compareTo(Wohnung o) {
        return Integer.valueOf((einzugsdatum + plz
+str) .compareTo(o.einzugsdatum+ o.plz+ o.str ));
    }

}

```

Klasse: StringTester

```

package _20_03_2026;

public interface StringTester {

    boolean test(String s);

}

```

Klasse: Test

```

package _20_03_2026;

import java.time.LocalDate;

public class Test {
    public static void main(String[] args) {

        Wohnung w;
        try {
            w = new Wohnung("10439", "Kuglerstr. 69", 10000, 2,
LocalDate.of(2025, 12, 15));
            System.out.println(w);
        } catch (RentOutOfLimitException e) {
            System.out.println("Die Miete darf nicht die Obergrenze
überschreiten,deshalb auf obere Grenze gesetzt !");
        }
    }
}

```



```

        w = new Wohnung("10439", "Kuglerstr. 69",
Wohnung.mieteObereGrenz, 2, LocalDate.of(2025, 12, 15));
        System.out.println(w);
    } catch (RentOutOfLimitException e1) {
        e1.printStackTrace();
    }

}

}

}

```

Klasse: WohnungTester

```

package _20_03_2026;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

import _16_03_2026.Wohnung;

interface WohnungTester{
    boolean test(Wohnung w);
}

interface Predicate<T>{
    boolean test(T t);
}

class FilterUtils {
    public static <T> List<T> filter(List<T> input, Predicate<? super
T> predicate) {
        List<T> result = new ArrayList<>();
        for (T element : input) {
            if (predicate.test(element)) {
                result.add(element);
            }
        }
        return result;
    }
}

public class FilterWohnungen {

    /**

```

```
* Filter nach: -Plz -Strasse -Miete -Zimmeranzahl
*
*
*/
```

```
public static List<Wohnung> filterStringPlz(List<Wohnung> whg,
StringTester t) {
    List<Wohnung> result = new ArrayList<Wohnung>();
    for (Wohnung w : whg) {
        if (t.test(w.getPlz())) {
            result.add(w);
        }
    }
    return result;
}
```

```
public static List<Wohnung> filterStringStrasse(List<Wohnung> whg,
StringTester t) {
    List<Wohnung> result = new ArrayList<Wohnung>();
    for (Wohnung w : whg) {
        if (t.test(w.getStr())) {
            result.add(w);
        }
    }
    return result;
}
```

```
public static List<Wohnung> filterDoubleMiete(List<Wohnung> whg,
DoubleTester t) {
    List<Wohnung> result = new ArrayList<Wohnung>();
    for (Wohnung w : whg) {
        if (t.test(w.getMiete())) {
            result.add(w);
        }
    }
    return result;
}
```

```
public static List<Wohnung>
filterIntegerZimmerAnzahl(List<Wohnung> whg, IntegerTester t) {
    List<Wohnung> result = new ArrayList<Wohnung>();
    for (Wohnung w : whg) {
        if (t.test(w.getZimmerAnzahl())) {
            result.add(w);
        }
    }
    return result;
}
```

```

//jetzt vereinfachen:
public static<T> List<T> filter(List<T> whg, Predicate<? super T>
t) {
    List<T> result = new ArrayList<T>();
    for (T w : whg) {
        if (t.test(w)) {
            result.add(w);
        }
    }
    return result;
}

```

```

public static List<Wohnung> filtern(List<Wohnung>
wohnungen,Predicate<Wohnung> tester){
    List <Wohnung> result = new ArrayList<Wohnung>();

    for (Wohnung w: wohnungen) {
        if (tester.test(w))
            result.add(w);
    }
    return result;
}

```

```

public static void main(String[] args) {

    List<Wohnung> whgList = new ArrayList<>();

    Collections.addAll(whgList, new Wohnung("10439", "Paul-
Robeson-Str 34", 807.47, 2, LocalDate.of(2010, 9, 1)),
        new Wohnung("10439", "Kugler Str. 69", 1400.69, 2,
LocalDate.of(2023, 3, 2)),
        new Wohnung("10439", "Erich-Weinert-Str. 78", 1165.37,
3, LocalDate.of(2005, 12, 15)),
        new Wohnung("10788", "Wielandstr. 38", 560, 1,
LocalDate.of(2009, 12, 15)),
        new Wohnung("10788", "WundsStr. 38", 450, 1,
LocalDate.of(2010, 12, 15)),
        new Wohnung("10439", "Schönhauser Allee 12", 945.80,
2, LocalDate.of(2018, 5, 1)),
        new Wohnung("10439", "Greifenhagener Str. 21",
1250.00, 3, LocalDate.of(2016, 11, 15)),

```

```

        new Wohnung("10439", "Stargarder Str. 5", 735.50, 1,
LocalDate.of(2012, 2, 1)),
        new Wohnung("10439", "Gleimstr. 44", 1590.90, 4,
LocalDate.of(2020, 7, 1)),
        new Wohnung("10439", "Pappelallee 97", 1020.30, 2,
LocalDate.of(2008, 9, 30)),
        new Wohnung("10788", "Nürnberger Str. 15", 880.00, 1,
LocalDate.of(2014, 4, 1)),
        new Wohnung("10788", "Kurfürstenstr. 3", 1325.75, 2,
LocalDate.of(2019, 10, 1)),
        new Wohnung("10788", "Keithstr. 22", 1749.99, 3,
LocalDate.of(2021, 1, 15)),
        new Wohnung("10788", "Lützowstr. 40", 640.00, 1,
LocalDate.of(2007, 6, 1)),
        new Wohnung("10788", "Potsdamer Str. 120", 1995.45, 4,
LocalDate.of(2024, 8, 1)),
        new Wohnung("10439", "Paul-Robeson-Str 12", 890.25, 2,
LocalDate.of(2011, 3, 1)),
        new Wohnung("10439", "Paul-Robeson-Str 56", 975.90, 3,
LocalDate.of(2018, 10, 1)),
        new Wohnung("10439", "Kugler Str. 12", 1250.00, 2,
LocalDate.of(2020, 5, 15)),
        new Wohnung("10439", "Kugler Str. 101", 1499.99, 3,
LocalDate.of(2022, 9, 1)),
        new Wohnung("10439", "Erich-Weinert-Str. 5", 980.40,
2, LocalDate.of(2007, 1, 1)),
        new Wohnung("10439", "Erich-Weinert-Str. 120",
1355.70, 4, LocalDate.of(2019, 6, 1)),
        new Wohnung("10788", "Wielandstr. 5", 630.00, 1,
LocalDate.of(2010, 4, 1)),
        new Wohnung("10788", "Wielandstr. 77", 1420.50, 3,
LocalDate.of(2017, 2, 1)),
        new Wohnung("10788", "Wielandstr. 102", 1899.00, 4,
LocalDate.of(2021, 11, 1)),
        new Wohnung("10788", "Kugler Str. 5", 1100.00, 2,
LocalDate.of(2015, 8, 1)));

```

```

        System.out.println(whgList);
        System.out.println(filterStringPlz(whgList, (s) ->
s.equals("10439")));
        System.out.println(filterStringStrasse(whgList, (s) ->
s.equals("WundsStr. 38")));
        System.out.println(filterStringStrasse(whgList, (s) ->
s.contains("Kugler Str")));
        System.out.println(filterDoubleMiete(whgList, (d) -> d >
800));
        System.out.println(filterIntegerZimmerAnzahl(whgList, (i) -> i
< 3));

```

```

        List<Wohnung> guenstig = FilterUtils.filter(
            whgList, w -> w.getMiete() < 600.0);

        List<Wohnung> dreiZimmerOderMehr = FilterUtils.filter(
            whgList, w -> w.getZimmerAnzahl() >= 3);
        List<Wohnung> inErichDingsbumsstrasse = FilterUtils.filter(
            whgList, w -> w.getStr().contains("Erich"));

        System.out.println(guenstig);
        System.out.println(dreiZimmerOderMehr);
        System.out.println(inErichDingsbumsstrasse);

        System.out.println(filter(whgList, (s) ->
s.getStr().contains("5")));
        // System.out.println(filter(whgList,(date) -> date.));

    }
}

```

Package: [_23_02_2026](#)

Klasse: Pseudocode

```

package _23_02_2026;

public class Pseudocode {

}

```

Package: [_23_03_2026](#)

Klasse: AVGFilter

```

package _23_03_2026;

public class AVGFilter extends Filter {

    public AVGFilter(SampleProvider sP) {
        super(sP);
    }

    @Override
    public String getType() {
        // TODO Auto-generated method stub
        return null;
    }
}

```

```

@Override
public void fetchSample(double[] mess) {
    sP.fetchSample(mess);

    double sum=0.0;
    for( double d: mess) {
        sum +=d;
    }

    mess[0] = sum/mess.length;
}

@Override
public int sampleSize() {
    // TODO Auto-generated method stub
    return 0;
}
}

```

Klasse: DruckSensor

```

package _23_03_2026;

public class DruckSensor implements SampleProvider {

    @Override
    public String getType() {

        return "Druck";
    }

    @Override
    public void fetchSample(double[] mess) {
        // TODO Auto-generated method stub

    }

    @Override
    public int sampleSize() {
        // TODO Auto-generated method stub
        return 0;
    }
}

```

Klasse: Filter

```

package _23_03_2026;

```

```

public abstract class Filter implements SampleProvider {

    protected SampleProvider sP;    //Aggregation

    public Filter(SampleProvider sP) {
        this.sP = sP;
    }

    //die methoden des Interface werden in der abstrakten nicht
    überschrieben und müssen deshalb nicht hier hingeschrieben werden

}

```

Klasse: MaxFilter

```

package _23_03_2026;

```

```

public class MaxFilter extends Filter {

    public MaxFilter(SampleProvider sP) {
        super(sP);
    }

    @Override
    public String getType() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public void fetchSample(double[] mess) {
        // TODO Auto-generated method stub
    }

    @Override
    public int sampleSize() {
        // TODO Auto-generated method stub
        return 0;
    }

}

```

Klasse: SampleProvider

```
package _23_03_2026;

public interface SampleProvider {

    String getType();
    void fetchSample(double[] mess);
    int sampleSize();

}
```

Klasse: TemperaturSensor

```
package _23_03_2026;

public class TemperaturSensor implements SampleProvider {

    @Override
    public String getType() {

        return "Temperatur";
    }

    @Override
    public void fetchSample(double[] mess) {
        // TODO Auto-generated method stub
    }

    @Override
    public int sampleSize() {
        // TODO Auto-generated method stub
        return 0;
    }

}
```

Klasse: Teschtle

```
package _23_03_2026;

public class Teschtle {

    public static void main(String[] args) {
        DruckSensor d = new DruckSensor();
        AVGFilter avg = new AVGFilter(d);

        double[] md = new double[100];
        avg.fetchSample(md);//Call by reference
    }

}
```



```
    }  
}
```

Package: [_24_03_2026](#)

Klasse: [Immutable](#)

```
package _24_03_2026;
```

```
import java.time.LocalDate;  
import java.util.ArrayList;  
import java.util.Collections;  
import java.util.List;
```

```
/**  
 * Immutable vs Mutable  
 *  
 *  
 * -> Design Pattern  
 *  
 * -Immutable entworfene Klassen :  
 *     -Die Objekte können nicht verändert werden  
 *  
 */
```

```
public class Immutable {  
  
    public static void main(String[] args) {  
        String s = new String("Bob");  
        String s2=s.toUpperCase();//Bob->BOB  
        System.out.println(s);//Bob ,da String so ist wie am Anfang  
        System.out.println(s2);//toUpperCase hat ein neues Objekt  
erstellt  
        s="Alice";//Reassignment  
        System.out.println(s);  
  
        LocalDate x= LocalDate.of(2001, 9, 11);  
        LocalDate y= LocalDate.of(1990, 10, 3);  
  
        LocalDate t=x.withMonth(12);//Sowas wie ein setter  
        System.out.println(x);//LocalDate ist Immutable ,d.h. x bleibt  
beim alten Objekt  
        System.out.println(t);  
  
        Punkt p= new Punkt(1,2);  
        System.out.println(p);  
    }  
}
```

```

    p.setY(3.14);
    System.out.println(p); //->mutable

    PunktImmutable pi = new PunktImmutable(2.71, 0.37);
    System.out.println(pi);
    PunktImmutable pi2= pi.withY(150);
    System.out.println(pi);
    System.out.println(pi2);

    List<String> sk=new ArrayList<String>();
    Collections.addAll(sk, "bügeln","stricken","häkeln");
    List<String> sk2=new ArrayList<String>();
    Collections.addAll(sk2,
    "Infinitesimalrechnung","Algebra","Stochastik");
    List<String> sk3=new ArrayList<String>();
    Collections.addAll(sk3, "keine","Ahnung");

    StudentImmutable si= new StudentImmutable("Salmonelle",
    "Luigi", LocalDate.of(1985, 12, 1), sk);
    System.out.println(si);
    StudentImmutable si2=si.withSkills(sk2);
    System.out.println(si);
    System.out.println(si2);

    StudentImmutable si3= si.withSkills(sk3);
    // si3.getSkills().add("Meinung");
    System.out.println(si3);
    // List<String> temp = si3.getSkills();
    // temp.remove(0);
    // System.out.println(si3);
    }

}

```

Klasse: Punkt

```

package _24_03_2026;
/**
 * ist die Klasse Punkt mutable definiert?
 * Wenn ja was kann man machen um die Klasse immutable zu definieren?
 */
public class Punkt {

    private double x;
    private double y;
    public Punkt(double x, double y) {
        this.x = x;
        this.y = y;
    }
}

```

```

    }
    public double getX() {
        return x;
    }
    public void setX(double x) {
        this.x = x;
    }
    public double getY() {
        return y;
    }
    public void setY(double y) {
        this.y = y;
    }
    @Override
    public String toString() {
        return "(" + x + "|" + y + ")";
    }
}

```

```

}

```

Klasse: Student

```

package _24_03_2026;

```

```

import java.time.LocalDate;
import java.util.List;

```

```

public class Student {

    private String name;
    private String vorname;
    private LocalDate geburtsdatum;

    private List<String> skills;

    public Student(String name, String vorname, LocalDate
geburtsdatum, List<String> skills) {
        this.name = name;
        this.vorname = vorname;
        this.geburtsdatum = geburtsdatum;
        this.skills = skills;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {

```

```

        this.name = name;
    }

    public String getVorname() {
        return vorname;
    }

    public void setVorname(String vorname) {
        this.vorname = vorname;
    }

    public LocalDate getGeburtsdatum() {
        return geburtsdatum;
    }

    public void setGeburtsdatum(LocalDate geburtsdatum) {
        this.geburtsdatum = geburtsdatum;
    }

    public List<String> getSkills() {
        return skills;
    }

    public void setSkills(List<String> skills) {
        this.skills = skills;
    }

    @Override
    public String toString() {
        return name + ", "+ vorname + ": " + geburtsdatum + ", \
nskills="
        + skills + "];"
    }

}

```

Klasse: StudentImmutable

```

package _24_03_2026;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public final class StudentImmutable {

    private final String name;
    private final String vorname;

```

```

    private final LocalDate geburtsdatum;

    private final List<String> skills;

    public StudentImmutable(String name, String vorname, LocalDate
geburtsdatum, List<String> skills) {
        this.name = name;
        this.vorname = vorname;
        this.geburtsdatum = geburtsdatum;
        this.skills =new ArrayList<String>(skills);
//List.copyOf(skills);
    }

    public String getName() {
        return name;
    }

    public StudentImmutable withName(String name) {
        return new StudentImmutable(name, this.vorname,
this.geburtsdatum, this.skills);
    }

    public String getVorname() {
        return vorname;
    }

    public StudentImmutable withVorname(String vorname) {
        return new StudentImmutable(this.name, vorname,
this.geburtsdatum, this.skills);
    }

    public LocalDate getGeburtsdatum() {
        return geburtsdatum;
    }

    public StudentImmutable withGeburtsdatum(LocalDate geburtsdatum) {
        return new StudentImmutable(this.name, this.vorname,
geburtsdatum, this.skills);
    }

    public List<String> getSkills() {
        //return skills;
        return new ArrayList<String>(skills);
    }

    public StudentImmutable withSkills(List<String> skills) {
        return new StudentImmutable(this.name, this.vorname,
this.geburtsdatum,skills);
    }

```

```

        @Override
        public String toString() {
            return name + "," + vorname + ": " + geburtsdatum + ", \
nskills="
                + skills + "\n";
        }

    }

```

Package: _25_03_2026

Klasse: Auto

```
package _25_03_2026;
```

```
public class Auto {
```

```
}
```

Klasse: CarData

```
package _25_03_2026;
```

```

public class CarData {
    private int seats;
    private Engine engine;
    private boolean tripComputer;
    private boolean gps;

    public CarData() {
    }

    public int getSeats() {
        return seats;
    }

    public void setSeats(int seats) {
        this.seats = seats;
    }

    public Engine getEngine() {
        return engine;
    }

    public void setEngine(Engine engine) {
        this.engine = engine;
    }
}

```

```

    public boolean isTripComputer() {
        return tripComputer;
    }

    public void setTripComputer(boolean tripComputer) {
        this.tripComputer = tripComputer;
    }

    public boolean isGps() {
        return gps;
    }

    public void setGps(boolean gps) {
        this.gps = gps;
    }
}package _03_03;

public class Client {
    public static void main(String[] args) {
        TeeEtickett meinTeeEtickett = new
EtickettenFabrikKoeln().erstelleEtickett("Magen");
    }
}

```

Package: [_25_03_2026_arbeitsblatt](#)

Klasse: Buchung

```

package _25_03_2026_arbeitsblatt;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public final class Buchung {

    private final String kunde;
    private final LocalDate datum;
    private final double preis;
    private final String bemerkung;
    private final List<String> zusatzleistung;

    @Override
    public String toString() {
        return "Buchung [kunde=" + kunde + ", datum=" + datum + ",
preis=" + preis + ", bemerkung=" + bemerkung
        + ", zusatzleistung=" + zusatzleistung + "];";
    }
}

```

```

    }

    private Buchung(Builder b) {
        this.kunde = b.kunde;
        this.datum = b.datum;
        this.preis = b.preis;
        this.bemerkung = b.bemerkung;
        this.zusatzleistung = b.zusatzleistung;
    }

    public String getKunde() {
        return kunde;
    }

    public LocalDate getDatum() {
        return datum;
    }

    public double getPreis() {
        return preis;
    }

    public String getBemerkung() {
        return bemerkung;
    }

    public List<String> getZusatzleistung() {
        return new ArrayList<String>(this.zusatzleistung);
//Defensive Copy
    }

    public Buchung withPreis(double p) {
        return new Buchung.Builder()
            .preis(p)
            .kunde(this.kunde)
            .datum(this.datum)
            .bemerkung(this.bemerkung)
            .zusatzleistung(this.zusatzleistung)

            .build();
    }

    public static class Builder {
        private String kunde;

```



```

private LocalDate datum;
private double preis;
private String bemerkung;
private List<String> zusatzleistung = new ArrayList<String>();

public Builder kunde(String k) {
    if (k.isEmpty() || k.isBlank() )
        throw new IllegalArgumentException();
    this.kunde = k;
    return this;
}

public Builder datum(LocalDate d) {
    if (d != null) {
        this.datum = d;
        return this;
    }
    throw new NullPointerException();
}

public Builder preis(double p) {
    if (preis >=0) {
        this.preis = p;
        return this;
    }
    throw new IllegalArgumentException();
}

public Builder bemerkung(String b) {
    this.bemerkung = b;
    return this;
}

public Builder zusatzleistung(List<String> lst ) {
    if (lst == null)
        this.zusatzleistung = new ArrayList<String>();
    else
        this.zusatzleistung = new
ArrayList<String>(lst);    //Defensive Copy
    return this;
}

    public Builder zusatzleistungHinzufuegen(String leistung) {
        if (leistung !=null && !leistung.isEmpty() && !
leistung.isBlank())
            zusatzleistung.add(leistung);
        return this;
    }

public Buchung build() {

```

```

        return new Buchung(this);
    }

}

```

Klasse: Verwendung

```

package _25_03_2026_arbeitsblatt;

import java.time.LocalDate;

public class Verwendung {

    public static void main(String[] args) {
        Buchung b = new Buchung.Builder()
            .kunde("Max")
            .datum(LocalDate.now())
            .preis(199.99)
            .bemerkung("zu teuer!")
            .build();
        System.out.println(b);

        Buchung b1= b.withPreis(210.99);
        System.out.println("b="+b);
        System.out.println("b1="+b1);
        b.getZusatzleistung().add("Garantie");
        System.out.println("b="+b);

    }

}

```

Package: _26_02_26

Klasse: Dreieck

```

package _26_02_26;

public class Dreieck extends Form {

    protected double erste_seite;
    protected double zweite_seite;
    protected double eingeschlossener_winkel;
    public Dreieck(Punkt mittelpunkt, double erste_seite, double
    zweite_seite, double eingeschlossener_winkel) {
        super(mittelpunkt);
    }
}

```

```

        setErste_seite(erste_seite);
        setZweite_seite(zweite_seite);
        if (eingeschlossener_winkel > 0 && eingeschlossener_winkel <
180) {
            this.eingeschlossener_winkel =
eingeschlossener_winkel*Math.PI/180.0;
        }
        else {
            System.out.println("Falsche Eingabe! Winkel auf 90°
gesetzt");
            this.eingeschlossener_winkel = Math.PI/2;
        }
    }
    public double getErste_seite() {
        return erste_seite;
    }
    public void setErste_seite(double erste_seite) {
        if (erste_seite > 0) {
            this.erste_seite = erste_seite;
        }
        else {
            this.erste_seite = 0;
        }
    }
    public double getZweite_seite() {
        return zweite_seite;
    }
    public void setZweite_seite(double zweite_seite) {
        if (zweite_seite > 0) {
            this.zweite_seite = zweite_seite;
        }
        else {
            this.zweite_seite = 0;
        }
    }
    public double getEingeschlossener_winkel() {
        return eingeschlossener_winkel;
    }
    public void setEingeschlossener_winkel(double
eingeschlossener_winkel) {
        if (eingeschlossener_winkel > 0 && eingeschlossener_winkel <
180) {
            this.eingeschlossener_winkel =
eingeschlossener_winkel*Math.PI/180.0;
        }
        else {
            System.out.println("Falsche Eingabe! Winkel auf 90°
gesetzt");
            this.eingeschlossener_winkel = Math.PI/2;
        }
    }

```

```

    }
}

    public String toString() {
        return "Dreieck [mittelpunkt= " + mittelpunkt + ", erste_seite="
+ erste_seite + ", zweite_seite=" + zweite_seite + ",
eingeschlossener_winkel="
        +
Math.round(eingeschlossener_winkel*180.0/Math.PI*100)/100.0 + "°,
erster_winkel="+Math.round(berechneWinkel()[0]*100)/100.0+"°,
zweiter_winkel="+ Math.round(berechneWinkel()[1]*100)/100.0+"°]";
    }
    @Override
    public double berechneFlaeche() {

        return
erste_seite*zweite_seite/2*Math.sin(eingeschlossener_winkel);
    }
    @Override
    public double berechneUmfang() {

        return
erste_seite+zweite_seite+Math.sqrt(erste_seite*erste_seite+zweite_seit
e*zweite_seite-
2*erste_seite*zweite_seite*Math.cos(eingeschlossener_winkel) );
    }

    public double berechneDritte_seite() {
        return
Math.sqrt(erste_seite*erste_seite+zweite_seite*zweite_seite-
2*erste_seite*zweite_seite*Math.cos(eingeschlossener_winkel));
    }

    public double[] berechneWinkel() {
        double[] winkel = {0.0,0.0};

        if (erste_seite< zweite_seite) {

winkel[0]=Math.asin(erste_seite/berechneDritte_seite()*Math.sin(einges
chlossener_winkel))/Math.PI*180;

winkel[1]=180-winkel[0]-eingeschlossener_winkel/Math.PI*180;
        }
        else {

winkel[1]=Math.asin(zweite_seite/berechneDritte_seite()*Math.sin(einge
schlossener_winkel))/Math.PI*180;

winkel[0]=180-winkel[1]-eingeschlossener_winkel/Math.PI*180;
        }
    }
}

```

```

        return winkel;
    }

```

```

}

```

Klasse: Form

```

package _26_02_26;

```

```

public abstract class Form{
    protected Punkt mittelpunkt;

    public Form(Punkt mittelpunkt) {
        super();
        this.mittelpunkt = mittelpunkt;
    }
    public Punkt getMittelpunkt() {
        return mittelpunkt;
    }
    public void setMittelpunkt(Punkt mittelpunkt) {
        this.mittelpunkt = mittelpunkt;
    }
    @Override
    public String toString() {
        return "Form [mittelpunkt=" + mittelpunkt + "]";
    }

    public abstract double berechneFlaeche() ;//ohne Körper->Methode
    abstrakt->Klasse ist abstrakt
    public abstract double berechneUmfang();
}package _01_30;

```

```

public abstract class Form{
    protected Punkt mittelpunkt;

    public Form(Punkt mittelpunkt) {
        super();//erbt von der Oberklasse Object
        this.mittelpunkt = mittelpunkt;
    }
    public Punkt getMittelpunkt() {
        return mittelpunkt;
    }
    public void setMittelpunkt(Punkt mittelpunkt) {

```

```

        this.mittelpunkt = mittelpunkt;
    }
    @Override
    public String toString() {
        return "Form [mittelpunkt=" + mittelpunkt + "]";
    }

    public abstract double berechneFlaeche() ;//ohne Körper->Methode
    abstrakt->Klasse ist abstrakt
    public abstract double berechneUmfang();

}

// Abstrakte Klassen haben keine Objekte, können nicht initialisiert
werdenpackage _09_03_2026;

import InOut.IO;

public class Freight extends Shipment implements Insurable {
    private double warenwert;
    public static final double GRUNDPREIS=49;
    public Freight(double weightKg, double dinstanceKm, double
warenwert) throws InvalidShipmentDataException {
        super(weightKg, dinstanceKm);
        if (warenwert>=0) {
            this.warenwert = warenwert;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
sein");
        }
    }

    @Override
    public String toString() {
        return String.format("Freight:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
            + "Shipping-Kosten: %22.2f€\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost());
    }

    @Override
    public double calculateInsurance() {
        return Math.max(warenwert*0.005, 10);
    }

    @Override

```

```

        public double shippingCost() {
            return GRUNDPREIS+0.35*getWeightKg()+0.015*getDistanceKm();
        }
    }

```

Klasse: Garten

```

package _26_02_26;

import java.util.ArrayList;

public class Garten {

    static ArrayList<Form> figures = new ArrayList<>();

    public static double berechneGesamtflaeche() {
        double total=0.0;
        for(Form f : figures) {
            total += f.berechneFlaeche();
        }
        return total;
    }

    public static void main(String[] args) {
        figures.add(new Rechteck(new Punkt(0,0), 3, 4));
        figures.add(new Kreis(new Punkt(1,2), 5));
        figures.add(new Quadrat(new Punkt(6,9), 8));

        System.out.println(figures);
        System.out.println(berechneGesamtflaeche());
    }
}

```

Klasse: GleichseitigesDreieck

```

package _26_02_26;

public class GleichseitigesDreieck extends Dreieck{

    public GleichseitigesDreieck(Punkt mittelpunkt, double
erste_seite) {
        super(mittelpunkt, erste_seite, erste_seite, 60);
    }

    public String toString() {
        return "gleichseitiges Dreieck [mittelpunkt= "+ mittelpunkt

```

```

+", seite=" + erste_seite + ", alle_winkel="
        + Math.round(eingeschlossener_winkel*180.0/Math.PI) +
"°]";
    }

    @Override
    public void setEingeschlossener_winkel(double angle) {
        System.out.println("Beim gleichseitigen Dreieck ist der
eingeschlossene Winkel immer 60°!");
    }

    public void setErste_seite(double seite) {
        super.setErste_seite(seite);
        super.setZweite_seite(seite);
    }

    public void setZweite_seite(double seite) {
        super.setErste_seite(seite);
        super.setZweite_seite(seite);
    }

}

```

Klasse: Kreis

```
package _26_02_26;
```

```

public class Kreis extends Form{

    private double radius;

    public Kreis(Punkt mittelpunkt, double radius) {
        super(mittelpunkt);
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    @Override
    public String toString() {
        return "Kreis [mittelpunkt=" + mittelpunkt + ", radius=" +
radius + "]";
    }

    public double berechneFlaeche() {

```



```

        return Math.round(Math.PI*radius*radius*100)/100.0;
    }
    public double berechneUmfang() {
        return Math.round(Math.PI*radius*200)/100.0;
    }
}package _10_15;
/**
 * Modelliert Kreise auf der Ebene
 *
 *
 * Hinweis:
 *     - Ein Kreis wird durch Radius und Mittelpunkt beschrieben
 *
 * TODO:
 *     - Attribute identifizieren und entsprechend deklarieren
 *     - Sinnvolle Konstruktoren definieren
 *     - Methoden definieren
 *
 * Was kann man mit Kreis Objekte machen?
 *
 *     - Flächeninhalt berechnen
 *     - Umfang berechnen
 *     - Vergrößern oder verkleinern
 *     - Kreis verschieben
 *     - Überprüfen, ob ein gegebener Punkt innerhalb des Kreises
    sich befindet.
 *     - Überprüfen, ob sich um einen Einheitskreis sich handelt.
 *
 * Was ist ein Einheitskreis?
 *     - Wenn der Radius = 1.0 und der
    Mitellpunkt = (0.0, 0.0)
 */
public class Kreis {

}

```

Klasse: Punkt

```

package _26_02_26;

public class Punkt{
    private double x;
    private double y;

    public Punkt(double x, double y) {

        this.x = x;
        this.y = y;
    }
}

```

```

    public double getX() {
        return x;
    }
    public void setX(double x) {
        this.x = x;
    }
    public double getY() {
        return y;
    }
    public void setY(double y) {
        this.y = y;
    }
    @Override
    public String toString() {
        return "Punkt [x=" + x + ", y=" + y + "]";
    }
}

```

```

}package _10_14;

```

```

import java.util.Scanner;

```

```

public class Punkt {

```

```

    /**
     * Attribute = Unterscheidungsmerkmalen
     * double x;
     * double y;
     *
     * Konstruktoren
     *
     *     * public Punkt(double xk, double yk){
     *     * x=xk;
     *     * y=yk;
     *     * }
     *
     * Methoden
     *
     * Abstände
     * Strecken
     * Mittelpunkt
     *
     */

```

```

    double xWert;
    double yWert;

```

```

    public Punkt(double xWert, double yWert) {

```

```

        this.xWert = xWert;
        this.yWert = yWert;
    }
    public Punkt() {

        this.xWert = 0.0;
        this.yWert = 0.0;
    }

    double generiereXwert(){
        return (Math.round((Math.random() * 400 ) * 100.0 / 100));
    }

    double generiereYwert() {
        return (Math.round((Math.random() * 150 ) * 100.0 / 100));
    }

    void zeigeZufallsPunkt() {
        System.out.printf("P(%.2f/%.2f)\n",generiereXwert(),generiereYwert());
    }
    void zeigePunkt(double xWert, double yWert) {
        System.out.printf("P(%.2f/%.2f)\n",xWert,yWert);
    }
    void zeigePunkt() {
        System.out.printf("P(%.2f/%.2f)\n",xWert,yWert);
    }
    double berecheneAbstand(double x, double y) {
        return Math.sqrt((x-xWert)*(x-xWert)+(y-yWert)*(y-yWert));
    }

    void bestimmeGeradengleichung(double x, double y) {
        if (x==xWert)
            System.out.println("Die Geradengleichung lautet x="+x);
        else {
            double m=(y-yWert)/(x-xWert);
            double c=-m*x+y;
            double alpha =Math.atan(m)*180.0/Math.PI;
            if (m!=0)
                System.out.printf("die Geradengleichung lautet: y=
%.2f*x+%.2f\nund schneidet die x-Achse mit %.2f°\n",m,c,alpha);
            else
                System.out.printf("die Geradengleichung lautet: y=
%.2f*x+%.2f\nund ist parrallel zur x-Achse\n",m,c);
        }
    }

    void bestimmeMitte(double x, double y) {
        System.out.printf("Die Mitte ist:

```

```

M(%.2f/%.2f)\n", (x+xWert)/2.0, (y+yWert)/2.0);
    }

    static double readXValue() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Gib den x-Wert ein: ");
        double xValue = sc.nextDouble();
        return xValue;
    }

    static double readYValue() {
        Scanner sc = new Scanner(System.in);
        System.out.print("Gib den y-Wert ein: ");
        double yValue = sc.nextDouble();
        return yValue;
    }

    void bestimmeQuadrant(double x, double y) {
        if (x>0 && y>0)
            System.out.println("Der Punkt liegt im ersten
Quadranten!");
        else if (x<0 && y>0)
            System.out.println("Der Punkt liegt im zweiten
Quadranten!");
        else if (x<0 && y<0)
            System.out.println("Der Punkt liegt im dritten
Quadranten!");
        else if (x>0 && y<0)
            System.out.println("Der Punkt liegt im vierten
Quadranten!");
        else if (x==0 && y!=0)
            System.out.println("Der Punkt liegt auf der x-Achse!");
        else if (x!=0 && y==0)
            System.out.println("Der Punkt liegt auf der y-Achse!");
        else
            System.out.println("Der Punkt ist der Ursprung(0/0)!");
    }

    void skaliere(double faktor) {
        xWert *= faktor;
        yWert *= faktor;
    }

    void verschiebe(double vx, double vy) {
        xWert += vx;
        yWert += vy;
    }

    void spiegeleAnXchse() {

```

```

        yWert = -yWert;
    }

    void spiegeleAnYAchse() {
        xWert = -xWert;
    }

    void spiegeleAmUrsprung() {
        xWert = -xWert;
        yWert = -yWert;
    }

    void rotiereUmUrsprung(double alpha) {
        double rad = alpha/180.0*Math.PI;
        double xDreh = xWert * Math.cos(rad) - yWert * Math.sin(rad);
        double yDreh = xWert * Math.sin(rad) + yWert * Math.cos(rad);
        xWert = xDreh;
        yWert = yDreh;
    }

    void rotiereUmPunkt(double x, double y, double alpha) {
        double rad = alpha / 180.0 * Math.PI;
        double xDreh = x + (xWert - x) * Math.cos(rad) - (yWert - y) *
Math.sin(rad);
        double yDreh = y + (xWert - x) * Math.sin(rad) + (yWert - y) *
Math.cos(rad);
        xWert = xDreh;
        yWert = yDreh;
    }

    double berechneAbstandZuGerade(double m, double c) {
        double A = m;
        double B = -1;
        double C = c;
        return Math.abs(A * xWert + B * yWert + C) / Math.sqrt(A * A +
B * B);
    }

    Punkt addiereVektor(Punkt vektor) {
        return new Punkt(xWert + vektor.xWert, yWert + vektor.yWert);
    }

    double berechneDreiecksflaeche(Punkt p2, Punkt p3) {
        return 0.5 * Math.abs(
            xWert * (p2.yWert - p3.yWert) +
            p2.xWert * (p3.yWert - yWert) +
            p3.xWert * (yWert - p2.yWert)
        );
    }
}

```

```

    static double berechneDreiecksflaeche(Punkt p1, Punkt p2, Punkt
p3) {
        return 0.5 * Math.abs(
            p1.xWert * (p2.yWert - p3.yWert) +
            p2.xWert * (p3.yWert - p1.yWert) +
            p3.xWert * (p1.yWert - p2.yWert)
        );
    }

    double berechneAbstandZuPunkt(Punkt andererPunkt) {
        return Math.sqrt((andererPunkt.xWert - xWert) *
(andererPunkt.xWert - xWert) +
            (andererPunkt.yWert - yWert) *
(andererPunkt.yWert - yWert));
    }

    void bestimmeGeradengleichung(Punkt andererPunkt) {
        if (andererPunkt.xWert == xWert) {
            System.out.println("Die Geradengleichung lautet x=" +
xWert);
        } else {
            double m = (andererPunkt.yWert - yWert) /
(andererPunkt.xWert - xWert);
            double c = yWert - m * xWert;
            double alpha = Math.atan(m) * 180.0 / Math.PI;
            if (m != 0) {
                System.out.printf("Die Geradengleichung lautet: y=
%.2f*x+%.2f\nund schneidet die x-Achse mit %.2f°\n", m, c, alpha);
            } else {
                System.out.printf("Die Geradengleichung lautet: y=
%.2f*x+%.2f\nund ist parallel zur x-Achse\n", m, c);
            }
        }
    }

    public double getXWert() {
        return xWert;
    }
    public void setXWert(double xWert) {
        this.xWert = xWert;
    }
    public double getYWert() {
        return yWert;
    }
    public void setYWert(double yWert) {
        this.yWert = yWert;
    }
    void bestimmeMitte(Punkt andererPunkt) {
        double mitteX = (xWert + andererPunkt.xWert) / 2.0;
        double mitteY = (yWert + andererPunkt.yWert) / 2.0;
    }

```

```

        System.out.printf("Die Mitte ist: M(%.2f/%.2f)\n", mitteX,
mitteY);
    }

```

```

    Punkt findeNaechstenPunkt(Punkt[] punkte) {
        if (punkte.length == 0) return null;
        Punkt naechster = punkte[0];
        double minAbstand = berechneAbstandZuPunkt(naechster);
        for (Punkt p : punkte) {
            double abstand = berechneAbstandZuPunkt(p);
            if (abstand < minAbstand) {
                minAbstand = abstand;
                naechster = p;
            }
        }
        return naechster;
    }

    public Punkt spiegelAnGeradeMC(double m, double c) {
        double d = m * m + 1;
        double x0 = (xWert + m * (yWert - c)) / d;
        double y0 = (m * xWert + m * m * (yWert - c) + c) / d;
        double gespiegeltX = 2 * x0 - xWert;
        double gespiegeltY = 2 * y0 - yWert;
        return new Punkt(gespiegeltX, gespiegeltY);
    }

```

```

}

```

Klasse: Quadrat

```

package _26_02_26;
/**
 * Ein Quadrat ist ein Rechteck mit Länge = Breite
 *
 * TODO:
 *      - Definieren entsprechend die Klasse Quadrat
 */
public class Quadrat extends Rechteck{

    public Quadrat(Punkt mittelpunkt, double a) {
        super(mittelpunkt,a,a);
    }

    @Override
    public void setLaenge(double laenge) {
        super.setLaenge(laenge);
        super.setBreite(laenge);
    }
    @Override

```

```

        public void setBreite(double laenge) {
            super.setLaenge(laenge);
            super.setBreite(laenge);
        }
        public String toString() {
            return "Quadrat [Mittelpunkt=" + mittelpunkt + ",
Seitenlaenge=" + laenge + "];"
        }
    }
}

```

Klasse: Rechteck

```
package _26_02_26;
```

```

public class Rechteck extends Form{

    protected double laenge;
    protected double breite;
    public Rechteck(Punkt mittelpunkt, double laenge, double breite) {

        super(mittelpunkt);
        this.laenge = laenge;
        this.breite = breite;
    }

    public double getLaenge() {
        return laenge;
    }
    public void setLaenge(double laenge) {
        this.laenge = laenge;
    }
    public double getBreite() {
        return breite;
    }
    public void setBreite(double breite) {
        this.breite = breite;
    }

    public String toString() {
        return "Rechteck [mittelpunkt=" + mittelpunkt + ", laenge=" +
laenge + ", breite=" + breite + "];"
    }

    public double berechneFlaeche() {
        return breite*laenge;
    }

    public double berechneUmfang() {
        return (breite*2+laenge*2);
    }
}

```



```

}package _01_30;

public class Rechteck extends Form{

    protected double laenge;
    protected double breite;
    public Rechteck(Punkt mittelpunkt, double laenge, double breite) {

        super(mittelpunkt);
        this.laenge = laenge;
        this.breite = breite;
    }

    public double getLaenge() {
        return laenge;
    }
    public void setLaenge(double laenge) {
        this.laenge = laenge;
    }
    public double getBreite() {
        return breite;
    }
    public void setBreite(double breite) {
        this.breite = breite;
    }

    public String toString() {
        return "Rechteck [mittelpunkt=" + mittelpunkt + ", laenge=" +
laenge + ", breite=" + breite + "]";
    }

    public double berechneFlaeche() {
        return breite*laenge;
    }

    public double berechneUmfang() {
        return (breite*2+laenge*2);
    }
}

```

```

}package _26_02_26;

public class RechtwinkligesDreieck extends Dreieck {

    public RechtwinkligesDreieck(Punkt mittelpunkt, double
erste_seite, double zweite_seite) {
        super(mittelpunkt, erste_seite, zweite_seite, 90);
    }
}

```

```

    }

    @Override
    public void setEingeschlossener_winkel(double angle) {
        System.out.println("Beim rechtwinkligen Dreieck ist der
eingeschlossene Winkel immer 90°!");
    }

    public String toString() {
        return "rechtwinkliges Dreieck [mittelpunkt= " + mittelpunkt
+", erste_seite=" + erste_seite + ", zweite_seite=" + zweite_seite +
", eingeschlossener_winkel="
+
Math.round(eingeschlossener_winkel*180.0/Math.PI*100)/100.0 + "°,
erster_winkel="+Math.round(berechneWinkel()[0]*100)/100.0+"°,
zweiter_winkel="+ Math.round(berechneWinkel()[1]*100)/100.0+"°]";
    }

}

```

Klasse: Task

```

package _26_02_26;
/**
 * es soll möglich sein
 *
 * Berechnung von:
 *     flächeninhalt und umfang für
 *     kreis, rechteck, quadrat
 */

```

```

public class Task {

}

```

Klasse: Test

```

package _26_02_26;

public class Test {
    public static void main(String[] args) {
        Punkt p = new Punkt(0.33, 0.68);
        Punkt p2 = new Punkt(0.25, 0.5);
        Punkt p3 = new Punkt(0.2, 0.3);

        // Form f = new Form(p);
        Rechteck r = new Rechteck(p2, 3.0, 4.0);
        Kreis k = new Kreis(p3, 1.0);

        // System.out.println(f.getMittelpunkt());
    }
}

```

```

// f.setMittelpunkt(p3);

//
System.out.println(k.getMittelpunkt());
k.setMittelpunkt(p);

// System.out.println(k);

    Quadrat q = new Quadrat(new Punkt(1.25,0.75),2.5);
    q.setBreite(4.5);
// System.out.println(q);
// System.out.println(k.berechneFlaeche());
// System.out.println(k.berechneUmfang());
// System.out.println(r.berechneFlaeche());
// System.out.println(r.berechneUmfang());
// System.out.println(q.berechneFlaeche());
// System.out.println(q.berechneUmfang());


    Dreieck d= new Dreieck(p3, 3,4,30);
    System.out.println("A = " +d.berechneFlaeche());
    System.out.println("U = "+d.berechneUmfang());

    System.out.println("=".repeat(30));

    RechtwinkligesDreieck rd = new RechtwinkligesDreieck(p,21,20);
    System.out.println("A = " +rd.berechneFlaeche());
    System.out.println("U = "+rd.berechneUmfang());

    System.out.println("=".repeat(30));

    d.setEingeschlossener_winkel(60);
    System.out.println("A = " +d.berechneFlaeche());
    System.out.println("U = "+d.berechneUmfang());

    System.out.println("=".repeat(30));

    rd.setEingeschlossener_winkel(60);

    System.out.println("=".repeat(30));

    d.setEingeschlossener_winkel(200);
    System.out.println("A = " +d.berechneFlaeche());
    System.out.println("U = "+d.berechneUmfang());

}
}

```

Klasse: Vererbung

```
package _26_02_26;
/**
 * Motivation:
 *           - Redundanz vermeiden
 *           - Klassen werden schmaler => Lesbarkeit
 *           - Abbildung der realen Welt
 *           - Beziehung zwischen Klassen (IS-A-Relationship)
 */
public class Vererbung {

}
```

Package: _26_03_2026

Klasse: EconomyPricing

```
package _26_03_2026;

public class EconomyPricing implements IPricingStrategy {

    @Override
    public double calculatePrice(double gewicht, double distanz) {
        double preis=3 + gewicht * 0.3 + distanz * 0.05;
        preis=gewicht > 50 ? preis*0.9 : preis;
        return preis;
    }

}
```

Klasse: ExpressPricing

```
package _26_03_2026;

public class ExpressPricing implements IPricingStrategy {

    @Override
    public double calculatePrice(double gewicht, double distanz) {
        double preis=10 + gewicht * 1.0 + distanz * 0.2;
        preis=gewicht > 50 ? preis*0.9 : preis;
        return preis;
    }

}
```

Klasse: IPricingStrategy

```
package _26_03_2026;

public interface IPricingStrategy {
```

```

        double calculatePrice(double gewicht, double distanz);
    }

Klasse: Logistikdienstleister
package _26_03_2026;

public class Logistikdienstleister {

    public static void main(String[] args) {
        double p1 = berechnePreis("STANDARD",10,100 );
        System.out.println(p1);
    }

    public static double berechnePreis(String typ, double gewicht,
double distanz) {
        if (typ.equalsIgnoreCase("STANDARD")) {
            return 5 + gewicht * 0.5 + distanz * 0.1;
        } else if (typ.equalsIgnoreCase("EXPRESS")) {
            return 10 + gewicht * 1.0 + distanz * 0.2;
        } else if (typ.equalsIgnoreCase("PRIORITY")) {
            return 20 + gewicht * 1.5 + distanz * 0.3;
        } else {
            throw new IllegalArgumentException("Unbekannter Typ");
        }
    }
}

```

```

Klasse: PriorityPricing
package _26_03_2026;

public class PriorityPricing implements IPricingStrategy {

    @Override
    public double calculatePrice(double gewicht, double distanz) {
        double preis=20 + gewicht * 1.5 + distanz * 0.3;
        preis=gewicht > 50 ? preis*0.9 : preis;
        return preis;
    }
}

```

```

Klasse: Sendung
package _26_03_2026;

public class Sendung {

```

```

private double gewicht;
private double distanz;
private String typ;
public Sendung(double gewicht, double distanz, String typ) {

    this.gewicht = gewicht;
    this.distanz = distanz;
    this.typ = typ;
}
public double getGewicht() {
    return gewicht;
}
public void setGewicht(double gewicht) {
    this.gewicht = gewicht;
}
public double getDistanz() {
    return distanz;
}
public void setDistanz(double distanz) {
    this.distanz = distanz;
}
public String getTyp() {
    return typ;
}
public void setTyp(String typ) {
    this.typ = typ;
}
@Override
public String toString() {
    return "Sendung [gewicht=" + gewicht + ", distanz=" + distanz
+ ", typ=" + typ + "]";
}

}

```

Klasse: StandardPricing

```
package _26_03_2026;
```

```

public class StandardPricing implements IPricingStrategy {

    @Override
    public double calculatePrice(double gewicht, double distanz) {

        double preis=5 + gewicht * 0.5 + distanz * 0.1;
        preis=gewicht > 50 ? preis*0.9 : preis;
        return preis;
    }
}

```

```
}
```

Klasse: Strategy

```
package _26_03_2026;
```

```
public class Strategy {
```

```
    private IPricingStrategy pricing;
```

```
    public IPricingStrategy getPricing() {  
        return pricing;  
    }
```

```
    public void setPricing(IPricingStrategy pricing) {  
        this.pricing = pricing;  
    }
```

```
    public double berechne(double gew,double dist){  
        return pricing.calculatePrice(gew, dist);  
    }
```

```
}
```

Klasse: VersandkostenRechner

```
package _26_03_2026;
```

```
import java.util.Arrays;
```

```
public class VersandkostenRechner {
```

```
    public static void main(String[] args) {
```

```
        Sendung[] sendungen = { new Sendung(10, 100, "Standard"),  
                                new Sendung(60, 200, "Standard"),  
                                new Sendung(10, 100, "Express"),  
                                new Sendung(60, 200, "Express"),  
                                new Sendung(10, 100, "Priority"),  
                                new Sendung(60, 200, "Priority"),  
                                new Sendung(10, 100, "Economy"),  
                                new Sendung(60, 200, "Economy"),  
                                };
```

```
        StandardPricing sP = new StandardPricing();  
        ExpressPricing eP = new ExpressPricing();  
        PriorityPricing pP = new PriorityPricing();  
        EconomyPricing ecP = new EconomyPricing();
```

```

        double[] preise = new double[8];

        for (int i = 0; i < 2; i++) {
            preise[i] = sP.calculatePrice(sendungen[i].getGewicht(),
sendungen[i].getDistanz());
        }
        for (int i = 2; i < 4; i++) {
            preise[i] = eP.calculatePrice(sendungen[i].getGewicht(),
sendungen[i].getDistanz());
        }
        for (int i = 4; i < 6; i++) {
            preise[i] = pP.calculatePrice(sendungen[i].getGewicht(),
sendungen[i].getDistanz());
        }
        for (int i = 6; i < 8; i++) {
            preise[i] = ecP.calculatePrice(sendungen[i].getGewicht(),
sendungen[i].getDistanz());
        }

        for (double preis : preise) {
            System.out.printf("Preis: %8.2f€\n", preis);
        }

        System.out.println("Vergleich der Ergebnisse:");
        Arrays.sort(preise);
        for (int i=0;i<preise.length-1;i++) {
            System.out.printf("Preis: %8.2f€\n", preise[i]);
            System.out.println("ist weniger als:");
        }
        System.out.printf("Preis: %8.2f€\n", preise[preise.length-1]);

    }
}

```

Package: [_26_03_StrategyPattern](#)

Klasse: [EconomyPricing](#)

```

package _26_03_StrategyPattern;

public class EconomyPricing implements PricingStrategy {

    @Override
    public double calculatePrice(double gewicht, double distanz) {

        return 3 + gewicht * 0.3 + distanz * 0.05;
    }

}

```


Klasse: ExpressPricing

```
package _26_03_StrategyPattern;

public class ExpressPricing implements PricingStrategy {

    @Override
    public double calculatePrice(double gewicht, double distanz) {
        return 10 + gewicht * 1.0 + distanz * 0.2;
    }

}
```

Klasse: Main

```
package _26_03_StrategyPattern;

public class Main {

    public static void main(String[] args) {
        double cost= VersangkostenRechner.berechnePreis("STANDARD",
150, 300);
        System.out.println(cost);

        cost = VersangkostenRechner.berechnePreisV2(new
StandardPricing(), 150, 300);
        System.out.println(cost);

        cost= VersangkostenRechner.berechnePreisV2(new
EconomyPricing(), 150, 300);
        System.out.println(cost);

        cost=VersangkostenRechner.berechnePreisV2((a, b) ->
3+a*0.3+b*0.05, 150, 300);
        System.out.println(cost);
    }

}
```

Klasse: PricingStrategy

```
package _26_03_StrategyPattern;

public interface PricingStrategy {

    double calculatePrice(double gewicht, double distanz);
}
```

Klasse: PriorityPricing

```
package _26_03_StrategyPattern;

public class PriorityPricing implements PricingStrategy {
```

```

        @Override
        public double calculatePrice(double gewicht, double distanz) {

            return 20 + gewicht * 1.5 + distanz * 0.3;
        }
    }
}

```

Klasse: StandardPricing

```

package _26_03_StrategyPattern;

public class StandardPricing implements PricingStrategy {

    @Override
    public double calculatePrice(double gewicht, double distanz) {
        return 5 + gewicht*0.5+distanz*0.1;
    }

}

```

Klasse: VersangkostenRechner

```

package _26_03_StrategyPattern;

public class VersangkostenRechner {

    public static double berechnePreis(String typ, double gewicht,
double distanz) {
        if (typ.equals("STANDARD")) {
            return 5 + gewicht * 0.5 + distanz * 0.1;
        } else if (typ.equals("EXPRESS")) {
            return 10 + gewicht * 1.0 + distanz * 0.2;
        } else if (typ.equals("PRIORITY")) {
            return 20 + gewicht * 1.5 + distanz * 0.3;
        } else {
            throw new IllegalArgumentException("Unbekannter Typ");
        }
    }

    public static double berechnePreisV2(PricingStrategy ps , double
gewicht, double distanz) {
        double cost=ps.calculatePrice(gewicht, distanz);
        return (gewicht<=50) ? cost : 0.9*cost;
    }

}

```

Package: 27_02_2026

Klasse: Device

package 27_02_2026;

```
public abstract class Device {
    protected String hostname;
    protected String ipAddress;
    protected double powerConsumption;
    protected MaintenanceContract contract;
    public Device(String hostname, String ipAddress, double
powerConsumption, MaintenanceContract contract) {
        super();
        this.hostname = hostname;
        this.ipAddress = ipAddress;
        this.powerConsumption = powerConsumption;
        this.contract = contract;
    }
    public String getHostname() {
        return hostname;
    }
    public void setHostname(String hostname) {
        this.hostname = hostname;
    }
    public String getIpAddress() {
        return ipAddress;
    }
    public void setIpAddress(String ipAddress) {
        this.ipAddress = ipAddress;
    }
    public double getPowerConsumption() {
        return powerConsumption;
    }
    public void setPowerConsumption(double powerConsumption) {
        this.powerConsumption = powerConsumption;
    }
    public MaintenanceContract getContract() {
        return contract;
    }
    public void setContract(MaintenanceContract contract) {
        this.contract = contract;
    }
    @Override
    public String toString() {
        return "Device [hostname=" + hostname + ", ipAddress=" +
ipAddress + ", powerConsumption=" + powerConsumption
        + ", contract=" + contract + "]\n";
    }
}
```

```

        public abstract double getPowerUsage();
        public abstract String getDeviceType();

        public boolean hasMaintenanceContract() {
            if (contract == null)
                return false;
            return true;
        }
        public String getInfo() {
            return "Device [hostname=" + hostname + ", ipAddress=" +
ipAddress + ", powerConsumption=" + powerConsumption
                + "Device-Typ="+getDeviceType()+", contract=" +
contract + "]\n";
        }
    }
}

```

Klasse: MaintenanceContract

```
package _27_02_2026;
```

```

public class MaintenanceContract {

    private String provider;
    private double monthlyCost;
    public MaintenanceContract(String provider, double monthlyCost) {
        super();
        this.provider = provider;
        this.monthlyCost = monthlyCost;
    }
    public String getProvider() {
        return provider;
    }
    public void setProvider(String provider) {
        this.provider = provider;
    }
    public double getMonthlyCost() {
        return monthlyCost;
    }
    public void setMonthlyCost(double monthlyCost) {
        this.monthlyCost = monthlyCost;
    }
}

```

```

        @Override
        public String toString() {
            return "MaintenanceContract [provider=" + provider + ",
monthlyCost=" + monthlyCost + "]\n";
        }

    }
}

```

Klasse: Router

```
package _27_02_2026;
```

```

public class Router extends Device{

    private int throughputMbps;

    public Router(String hostname, String ipAddress, double
powerConsumption, MaintenanceContract contract,
        int throughputMbps) {
        super(hostname, ipAddress, powerConsumption, contract);
        this.throughputMbps = throughputMbps;
    }

    public int getThroughputMbps() {
        return throughputMbps;
    }

    public void setThroughputMbps(int throughputMbps) {
        this.throughputMbps = throughputMbps;
    }

    @Override
    public double getPowerUsage() {

        return powerConsumption*1.1;
    }

    @Override
    public String getDeviceType() {

        return "Router";
    }

    @Override
    public String getInfo() {
        return "Router [throughputMbps=" + throughputMbps + ",
hostname=" + hostname + ", ipAddress=" + ipAddress

```

```

        + ", powerConsumption=" + powerConsumption + ",
contract=" + contract + ", getThroughputMbps()"
        + getThroughputMbps() + ", getPowerUsage()" +
getPowerUsage() + ", getDeviceType()" + getDeviceType()
        + ", getHostname()" + getHostname() + ",
getIpAddress()" + getIpAddress() + ", getPowerConsumption()"
        + getPowerConsumption() + ", getContract()" +
getContract() + ", toString()" + super.toString()
        + ", hasMaintenanceContract()" +
hasMaintenanceContract()
        + ", getClass()" + getClass() + ", hashCode()" +
hashCode() + "];
    }

```

```

}

```

Klasse: Server

```

package _27_02_2026;

```

```

public class Server extends Device {

    private int cpuCores;

    public Server(String hostname, String ipAddress, double
powerConsumption, MaintenanceContract contract,
        int cpuCores) {
        super(hostname, ipAddress, powerConsumption, contract);
        this.cpuCores = cpuCores;
    }

    @Override
    public double getPowerUsage() {

        return powerConsumption*1.2;
    }

    public int getCpuCores() {
        return cpuCores;
    }

    public void setCpuCores(int cpuCores) {
        this.cpuCores = cpuCores;
    }

    @Override
    public String getDeviceType() {

```

```

        return "Server";
    }

    @Override
    public String getInfo() {
        return "Server [cpuCores=" + cpuCores + ", hostname=" +
hostname + ", ipAddress=" + ipAddress
        + ", powerConsumption=" + powerConsumption + ",
contract=" + contract + ", getPowerUsage()="
        + getPowerUsage() + ", getCpuCores()=" + getCpuCores()
+ ", getDeviceType()=" + getDeviceType()
        + ", getHostname()=" + getHostname() + ",
getIpAddress()=" + getIpAddress() + ", getPowerConsumption()="
        + getPowerConsumption() + ", getContract()=" +
getContract() + ", toString()=" + super.toString()
        + ", hasMaintenanceContract()=" +
hasMaintenanceContract()
        + ", getClass()=" + getClass() + ", hashCode()=" +
hashCode() + "];"
    }

}

```

Klasse: Test

```

package _27_02_2026;

public class Test {

    public static void main(String[] args) {

        Device[] dev = new Device[3];
        dev[0]=new Server("Mail-Server", "192.168.0.200", 700, null,
16);
        dev[1]= new Router("Fritzbox 1780", "192.168.0.1", 200, null,
4);
        dev[2]= new Workstation("HP-Power-Tower", "192.168.0.10", 650,
new MaintenanceContract("Telekom", 24.99), "NvidaE300");

        System.out.println(dev[0].getInfo());
        System.out.println(dev[0].getPowerUsage());
        System.out.println("=".repeat(40));
        System.out.println(dev[1].getInfo());
        System.out.println(dev[1].getPowerUsage());
        System.out.println("=".repeat(40));
        System.out.println(dev[2].getInfo());
        System.out.println(dev[2].getPowerUsage());
        System.out.println("=".repeat(45));
    }
}

```

```

        for (Device d:dev) {
            if (d.hasMaintenanceContract()) {
                System.out.println("hat Vertrag: ");
                System.out.println("=".repeat(45));
                System.out.println(d.getInfo());
                System.out.println("=".repeat(45));
            }
        }
        System.out.println("Gesamtleistung aller Geräte:
"+calculateTotalPower(dev)+" Watt");
        System.out.println("Anzahl der Geräte mit Vertrag: "+
countDevicesWithContract(dev));
        System.out.println("Maximum der Monatskosten:
"+maxMonthlyCost(dev)+"€");
    }

    public static double calculateTotalPower(Device[] dev) {
        double summe=0.0;
        for (Device d:dev) {
            summe+=d.getPowerUsage();
        }
        return summe;
    }

    public static int countDevicesWithContract(Device[] devices) {
        int counter=0;
        for (Device d:devices) {
            if (d.hasMaintenanceContract())
                counter++;
        }
        return counter;
    }

    public static double maxMonthlyCost(Device[] dev) {
        double max=0.0;
        for (Device d:dev) {
            if (d.hasMaintenanceContract())
                max=max>d.getContract().getMonthlyCost()?
max:d.getContract().getMonthlyCost();
        }
        return max;
    }

}

```

Klasse: Workstation

package _27_02_2026;


```

public class Workstation extends Device{

    private String gpuModel;

    public Workstation(String hostname, String ipAddress, double
powerConsumtion, MaintenanceContract contract,
        String gpuModel) {
        super(hostname, ipAddress, powerConsumtion, contract);
        this.gpuModel = gpuModel;
    }

    public String getGpuModel() {
        return gpuModel;
    }

    public void setGpuModel(String gpuModel) {
        this.gpuModel = gpuModel;
    }

    @Override
    public double getPowerUsage() {
        // TODO Auto-generated method stub
        return powerConsumtion*0.9;
    }

    @Override
    public String getDeviceType() {

        return "Workstation";
    }

    @Override
    public String getInfo() {
        return "Workstation [gpuModel=" + gpuModel + ", hostname=" +
hostname + ", ipAddress=" + ipAddress
            + ", powerConsumtion=" + powerConsumtion + ",
contract=" + contract + ", getGpuModel()=" + getGpuModel()
            + ", getPowerUsage()=" + getPowerUsage() + ",
getDeviceType()=" + getDeviceType()
            + ", getHostname()=" + getHostname() + ",
getIpAddress()=" + getIpAddress()
            + ", getPowerConsumtion()=" + getPowerConsumtion() +
", getContract()=" + getContract()
            + ", toString()=" + super.toString() + ",
hasMaintenanceContract()=" + hasMaintenanceContract()
            + ", getClass()=" + getClass() + ", hashCode()=" +
hashCode() + " ]";
    }
}

```

```
}
```

Package: _27_03_2026

Klasse: ShipmentKlassen

```
package _27_03_2026;
```

```
public enum ShipmentKlassen {
```

```
    STD,EXP,FRT,DDY;
```

```
}
```

Klasse: Testklasse

```
package _27_03_2026;
```

```
public class Testklasse {
```

```
    public static void main(String[] args) {
```

```
//        TrackingCodeGenerator t1= new TrackingCodeGenerator();
```

```
//        TrackingCodeGenerator t2= new TrackingCodeGenerator();
```

```
//
```

```
//        System.out.println(t1.nextCode());
```

```
//        System.out.println(t2.nextCode());
```

```
        TrackingCodeGenerator t1= TrackingCodeGenerator.getInstance();
```

```
        TrackingCodeGenerator t2= TrackingCodeGenerator.getInstance();
```

```
        System.out.println(t1.nextCode(ShipmentKlassen.EXP));
```

```
        System.out.println(t1.nextCode(ShipmentKlassen.STD));
```

```
        System.out.println(t1.nextCode(ShipmentKlassen.EXP));
```

```
        System.out.println(t2.nextCode(ShipmentKlassen.EXP));
```

```
    }
```

```
}
```

Klasse: TrackingCodeGenerator

```
package _27_03_2026;
```

```
import java.time.LocalDate;
```

```
public class TrackingCodeGenerator {
```

```
    private int counter = 1;
```

```

private LocalDate datum=LocalDate.of(2026, 3, 27);
private static TrackingCodeGenerator instance;
//private ShipmentKlassen sk;

private TrackingCodeGenerator() {
}

public static TrackingCodeGenerator getInstance() {
    if (instance == null)
        instance= new TrackingCodeGenerator();
    return instance;
}

public String nextCode(ShipmentKlassen sk) {
    if(LocalDate.now().getYear() != datum.getYear())
        counter=1;
    datum=LocalDate.now();
    return sk.name()+"-" + LocalDate.now().getYear() + "-" +
String.format("%04d", counter++);
}
}

```

Package: 28_04_2026

Klasse: Superklasse

```
package _28_04_2026;
```

```

public abstract class Superklasse {

    public static void main(String[] args) {
        Kind k=new Kind("Nobbie",12);
        System.out.println(k);

        Eltern e=new Kind("Nonnie",21);
        System.out.println(e);

    }

}

```

```

abstract class Eltern{
    public String name;
    public abstract void printName();
}

```

```

/* @Override
    public String toString() {
        return "Eltern [name=" + name + "]";
    }

    */
    public Eltern(String name) {
        this.name = name;
    }
}

class Kind extends Eltern{

    public Kind(String name,int alter) {
        super(name);
        this.alter=alter;
    }

    int alter;
    public void printAlter() {
        System.out.println(alter);
    }
    public void printName() {

        System.out.println(name);
    }

    /* @Override
    public String toString() {
        return "Kind [alter=" + alter + " Name="+name+"]";
    }
    */
}

```

Package: app

Klasse: Sendungen

```

package app;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

import comparator.SortByDistance;
import exceptions.InvalidShipmentDataException;

```

```

import exceptions.UnknownShipmentTypeException;
import interfaces.Insurable;
import io.ShipmentCsvReader2;
import io.ShipmentReportWriter;
import model.ExpressParcel;
import model.Freight;
import model.Shipment;
import model.StandardParcel;
import util.ShipmentFilter;
import util.SortSelection;
import util.Statistic;
import util.shipmentOverviewCreator;

public class Sendungen {

    static Shipment[] shipments;
    static {
        try {
            shipments = new Shipment[] { new StandardParcel(15, 250),
new ExpressParcel(75, 500, 750),
                new Freight(4500, 2300, 8000), new Freight(400,
50, 180), new ExpressParcel(1400, 150, 380),
                new StandardParcel(50, 100), new
StandardParcel(105, 800), new StandardParcel(1.5, 40) };
        } catch (InvalidShipmentDataException e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] args) {

        util.ShipmentCostCalculator.calculateCost(shipments);

        System.out.println(shipmentOverviewCreator.createOverview(shipments));

        List<Shipment> shipments_test = new
        ArrayList<Shipment>(Arrays.asList(shipments));

        try {
            ShipmentReportWriter.writeReport(shipments_test,
"neue_warensendung.csv");
        } catch (UnknownShipmentTypeException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    List<Shipment> shipments_read = new ArrayList<Shipment>();
    try {
        shipments_read =

```

```

ShipmentCsvReader2.readShipmentsFromCSV("neue_warensendung.csv");
    } catch (UnknownShipmentTypeException e) {
        e.printStackTrace();
    } catch (InvalidShipmentDataException e) {
        e.printStackTrace();
    } // warensendungen.csv
    System.out.println(shipments_read);
    Collections.sort(shipments_test);
    SortByDistance.sortByDistance(shipments);
    System.out.println("=====SORTIERT NACH
ENTFERNUNG=====");

    System.out.println(util.shipmentOverviewCreator.createOverview(shipmen
ts));
    try {

        System.out.println(Collections.binarySearch(shipments_test,new
StandardParcel(50, 100)));
        } catch (InvalidShipmentDataException e) {
            System.out.println(e.getMessage());
        }

        SortSelection.selectSortAlgorithm(shipments);

    System.out.println(shipmentOverviewCreator.createOverview(shipments));

        System.out.println("teuerste Sendung:");
        System.out.println(Statistic.findMostExpensive(shipments));

        System.out.println("schwerste Sendung:");
        System.out.println(Statistic.findHeviest(shipments));

        System.out.println("durchschnittliche Versandkosten:");

    System.out.println(Math.round(Statistic.calaulateAvgShippingCost(shipm
ents)*100)/100.0+" €");

        System.out.println("=".repeat(40));

    System.out.println("=====FILTER=====");
        System.out.println("Sendungen mit Mindestgewicht 75 kg:");

    System.out.println(shipmentOverviewCreator.createOverviewFromList(
ShipmentFilter.filter(
shipments_test, (s) -> s.getWeightKg()>=75)
)
);

```

```

        System.out.println("=".repeat(40));
        System.out.println("Sendungen, die StandardParcel sind:");

System.out.println(shipmentOverviewCreator.createOverviewFromList(
    ShipmentFilter.filter(
        shipments_test, (s) ->
        s.getClass().getSimpleName().contains("Standard"))
    )
        );
        System.out.println("=".repeat(40));
        System.out.println("Sendungen mit mindestens 100 € Versand:");

System.out.println(shipmentOverviewCreator.createOverviewFromList(
    ShipmentFilter.filter(
        shipments_test, (s) -> s.shippingCost()>=100)
    )
        );
        System.out.println("=".repeat(40));
        System.out.println("Sendungen mit mindestens 50 € Versand\n\t
und 10 € Versicherung:");

System.out.println(shipmentOverviewCreator.createOverviewFromList(
    ShipmentFilter.filter(
        shipments_test, (s) -> s.shippingCost()>=50 && (s instanceof
        Insurable ins) && ins.calculateInsurance() >=10 )
    )
        );
        // besser pipeline und filter auf filter anwenden
    }
}

```

Klasse: Sendungen

```

package app;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

```

```

import comparator.SortByDistance;
import exceptions.InvalidShipmentDataException;
import exceptions.UnknownShipmentTypeException;
import interfaces.Insurable;
import io.ShipmentCsvReader2;
import io.ShipmentReportWriter;
import model.ExpressParcel;
import model.Freight;
import model.Shipment;
import model.StandardParcel;
import util.ShipmentFilter;
import util.SortSelection;
import util.Statistic;
import util.shipmentOverviewCreator;

public class Sendungen {

    static Shipment[] shipments;
    static {
        try {
            shipments = new Shipment[] { new StandardParcel(15, 250),
new ExpressParcel(75, 500, 750),
new Freight(4500, 2300, 8000), new Freight(400,
50, 180), new ExpressParcel(1400, 150, 380),
new StandardParcel(50, 100), new
StandardParcel(105, 800), new StandardParcel(1.5, 40) };
        } catch (InvalidShipmentDataException e) {
            System.out.println(e.getMessage());
        }
    }

    public static void main(String[] args) {

        util.ShipmentCostCalculator.calculateCost(shipments);

        System.out.println(shipmentOverviewCreator.createOverview(shipments));

        List<Shipment> shipments_test = new
        ArrayList<Shipment>(Arrays.asList(shipments));

        try {
            ShipmentReportWriter.writeReport(shipments_test,
"neue_warensendung.csv");
        } catch (UnknownShipmentTypeException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }

        List<Shipment> shipments_read = new ArrayList<Shipment>();
    }
}

```



```

        try {
            shipments_read =
ShipmentCsvReader2.readShipmentsFromCSV("neue_warensendung.csv");
        } catch (UnknownShipmentTypeException e) {
            e.printStackTrace();
        } catch (InvalidShipmentDataException e) {
            e.printStackTrace();
        } // warensendungen.csv
        System.out.println(shipments_read);
        Collections.sort(shipments_test);
        SortByDistance.sortByDistance(shipments);
        System.out.println("=====SORTIERT NACH
ENTFERNUNG=====");

        System.out.println(util.shipmentOverviewCreator.createOverview(shipmen
ts));
        try {

            System.out.println(Collections.binarySearch(shipments_test,new
StandardParcel(50, 100)));
        } catch (InvalidShipmentDataException e) {
            System.out.println(e.getMessage());
        }

        SortSelection.selectSortAlgorithm(shipments);

        System.out.println(shipmentOverviewCreator.createOverview(shipments));

        System.out.println("teuerste Sendung:");
        System.out.println(Statistic.findMostExpensive(shipments));

        System.out.println("schwerste Sendung:");
        System.out.println(Statistic.findHeviest(shipments));

        System.out.println("durchschnittliche Versandkosten:");

        System.out.println(Math.round(Statistic.calaulateAvgShippingCost(shipm
ents)*100)/100.0+" €");

        System.out.println("=".repeat(40));

        System.out.println("=====FILTER=====");
        System.out.println("Sendungen mit Mindestgewicht 75 kg:");

        System.out.println(shipmentOverviewCreator.createOverviewFromList(
ShipmentFilter.filter(
shipments_test, (s) -> s.getWeightKg()>=75)

```

```

    )
        );
        System.out.println("=".repeat(40));
        System.out.println("Sendungen, die StandardParcel sind:");

System.out.println(shipmentOverviewCreator.createOverviewFromList(
shipmentFilter.filter(
shipments_test, (s) ->
s.getClass().getSimpleName().contains("Standard"))
)
        );
        System.out.println("=".repeat(40));
        System.out.println("Sendungen mit mindestens 100 € Versand:");

System.out.println(shipmentOverviewCreator.createOverviewFromList(
shipmentFilter.filter(
shipments_test, (s) -> s.shippingCost()>=100)
)
        );
        );
        System.out.println("=".repeat(40));
        System.out.println("Sendungen mit mindestens 50 € Versand\n\t
und 10 € Versicherung:");

System.out.println(shipmentOverviewCreator.createOverviewFromList(
shipmentFilter.filter(
shipments_test, (s) -> s.shippingCost()>=50 && (s instanceof
Insurable ins) && ins.calculateInsurance() >=10 )
)
        );
        );
        // besser pipeline und filter auf filter anwenden
    }
}

```

Klasse: Test

```

package app;

import java.text.NumberFormat;
import java.util.Locale;

public class Test {

```

```

        public static void main(String[] args) {
            double preis=429.35;

System.out.println(NumberFormat.getCurrencyInstance(Locale.GERMAN).format(preis));

System.out.println(NumberFormat.getCurrencyInstance(Locale.CANADA).format(preis));

System.out.println(NumberFormat.getCurrencyInstance(Locale.ENGLISH).format(preis));

System.out.println(NumberFormat.getCurrencyInstance(Locale.JAPAN).format(preis));

        }
    }

```

Klasse: Test

```

package app;

import java.text.NumberFormat;
import java.util.Locale;

public class Test {

    public static void main(String[] args) {
        double preis=429.35;

System.out.println(NumberFormat.getCurrencyInstance(Locale.GERMAN).format(preis));

System.out.println(NumberFormat.getCurrencyInstance(Locale.CANADA).format(preis));

System.out.println(NumberFormat.getCurrencyInstance(Locale.ENGLISH).format(preis));

System.out.println(NumberFormat.getCurrencyInstance(Locale.JAPAN).format(preis));

    }
}

```

Package: bd_test

Klasse: TestConnect

```
package bd_test;
```

```
import java.sql.DriverManager;  
import java.sql.SQLException;  
import java.sql.ResultSet;  
import java.sql.*;
```

```
import java.sql.Connection;
```

```
public class TestConnect {
```

```
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/Northwind";  
        //port3306
```

```
        String user = "root";  
        String password = "";
```

```
        Connection connect = null;  
        Statement stmt = null;  
        ResultSet result = null;
```

```
        try {  
            connect = DriverManager.getConnection(url, user,  
password);  
            stmt=connect.createStatement();  
            String sql ="select EmployeeID, FirstName, LastName from  
Employees";  
            result = stmt.executeQuery(sql);  
            while(result.next()) {  
                int ID =result.getInt("EmployeeID");  
                String fname=result.getString("FirstName");  
                String lname=result.getString("LastName");  
                System.out.printf("ID: %4s, Vorname: %10s, Nachname:  
%12s\n",ID,fname,lname);  
            }  
        }
```

```
    } catch (SQLException e) {  
        // TODO Auto-generated catch block  
        e.printStackTrace();  
    } finally {  
        try {  
            if(result != null) {
```

```

        result.close();}
    if(stmt != null) {
        stmt.close();
    }
    if(connect !=null) {
        connect.close();
    }
} catch (SQLException e) {
    // TODO Auto-generated catch block
    e.printStackTrace();
}

}

}

}

```

Package: comparator

Klasse: CustomSort

```

package comparator;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import model.Shipment;

public class CustomSort {

    public static List<Shipment> sort(List<Shipment> liste,
Comparator<Shipment> c) {
        List<Shipment> copy = new ArrayList<Shipment>(liste);
        Collections.sort(copy, c);
        return copy;
    }

    public static Shipment[] sort(Shipment[] liste,
Comparator<Shipment> c) {
        return (Shipment[]) sort(Arrays.asList(liste), c).toArray();
    }
}

```

```
}
```

Klasse: CustomSort

```
package comparator;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import model.Shipment;

public class CustomSort {

    public static List<Shipment> sort(List<Shipment> liste,
Comparator<Shipment> c) {
        List<Shipment> copy = new ArrayList<Shipment>(liste);
        Collections.sort(copy, c);
        return copy;
    }

    public static Shipment[] sort(Shipment[] liste,
Comparator<Shipment> c) {
        return (Shipment[]) sort(Arrays.asList(liste), c).toArray();
    }

}
```

Klasse: DistanceComparatorAsc

```
package comparator;

import java.util.Comparator;

import model.Shipment;

public class DistanceComparatorAsc implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.getDistanceKm()).compareTo(Double.valueOf(o2.getDistanceKm()));
    }

}
```

```
}
```

Klasse: DistanceComparatorAsc

```
package comparator;
```

```
import java.util.Comparator;
```

```
import model.Shipment;
```

```
public class DistanceComparatorAsc implements Comparator<Shipment> {
```

```
    @Override
```

```
    public int compare(Shipment o1, Shipment o2) {
```

```
        return
```

```
Double.valueOf(o1.getDistanceKm()).compareTo(Double.valueOf(o2.getDistanceKm()));
```

```
    }
```

```
}
```

Klasse: DistanceComparatorDesc

```
package comparator;
```

```
import java.util.Comparator;
```

```
import model.Shipment;
```

```
public class DistanceComparatorDesc implements Comparator<Shipment> {
```

```
    @Override
```

```
    public int compare(Shipment o1, Shipment o2) {
```

```
        return
```

```
Double.valueOf(o2.getDistanceKm()).compareTo(Double.valueOf(o1.getDistanceKm()));
```

```
    }
```

```
}
```

Klasse: DistanceComparatorDesc

```
package comparator;
```

```
import java.util.Comparator;
```

```
import model.Shipment;
```

```
public class DistanceComparatorDesc implements Comparator<Shipment> {
```

```
    @Override
```

```

        public int compare(Shipment o1, Shipment o2) {
            return
Double.valueOf(o2.getDistanceKm()).compareTo(Double.valueOf(o1.getDistanceKm()));
        }
    }
}

```

Klasse: ShippingCostComparatorAsc

```

package comparator;

import java.util.Comparator;

import model.Shipment;

public class ShippingCostComparatorAsc implements Comparator<Shipment>
{
    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.shippingCost()).compareTo(Double.valueOf(o2.shippingCost()));
    }
}

```

Klasse: ShippingCostComparatorAsc

```

package comparator;

import java.util.Comparator;

import model.Shipment;

public class ShippingCostComparatorAsc implements Comparator<Shipment>
{
    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.shippingCost()).compareTo(Double.valueOf(o2.shippingCost()));
    }
}

```

Klasse: ShippingCostComparatorDesc

```

package comparator;

```



```

import java.util.Comparator;
import model.Shipment;

public class ShippingCostComparatorDesc implements
Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o2.shippingCost()).compareTo(Double.valueOf(o1.shipping
Cost()));
    }

}

```

Klasse: ShippingCostComparatorDesc

```

package comparator;

import java.util.Comparator;
import model.Shipment;

public class ShippingCostComparatorDesc implements
Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o2.shippingCost()).compareTo(Double.valueOf(o1.shipping
Cost()));
    }

}

```

Klasse: SortByDistance

```

package comparator;

import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import model.Shipment;

public class SortByDistance implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.getDistanceKm()).compareTo(Double.valueOf(o2.getDist

```

```

        anceKm()));
    }

    public static Shipment[] sortByDistance(Shipment[] sh ) {
        Comparator<Shipment> c =new SortByDistance();
        Arrays.sort(sh,c);
        return sh;
    }

    public static List<Shipment> sortByDistanceList(List<Shipment>
sh ) {
        Comparator<Shipment> c =new SortByDistance();
        Collections.sort(sh,c);
        return sh;
    }

}

```

Klasse: SortByDistance

```

package comparator;

import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import model.Shipment;

public class SortByDistance implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.getDistanceKm()).compareTo(Double.valueOf(o2.getDist
anceKm()));
    }

    public static Shipment[] sortByDistance(Shipment[] sh ) {
        Comparator<Shipment> c =new SortByDistance();
        Arrays.sort(sh,c);
        return sh;
    }

    public static List<Shipment> sortByDistanceList(List<Shipment>
sh ) {
        Comparator<Shipment> c =new SortByDistance();

```

```

        Collections.sort(sh,c);
        return sh;
    }

}

Klasse: SortByShippingCost
package comparator;

import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import model.Shipment;

public class SortByShippingCost implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.shippingCost()).compareTo(Double.valueOf(o2.shipping
Cost()));
    }

    public static Shipment[] sortByShippingCost(Shipment[] sh ) {
        Comparator<Shipment> c =new SortByShippingCost();
        Arrays.sort(sh,c);
        return sh;
    }

    public static List<Shipment> sortByShippingCostList(List<Shipment>
sh ) {
        Comparator<Shipment> c =new SortByShippingCost();
        Collections.sort(sh,c);
        return sh;
    }

}

```

Klasse: SortByShippingCost
package comparator;

```

import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

```

```

import model.Shipment;

public class SortByShippingCost implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.shippingCost()).compareTo(Double.valueOf(o2.shipping
Cost()));
    }

    public static Shipment[] sortByShippingCost(Shipment[] sh ) {
        Comparator<Shipment> c =new SortByShippingCost();
        Arrays.sort(sh,c);
        return sh;
    }

    public static List<Shipment> sortByShippingCostList(List<Shipment>
sh ) {
        Comparator<Shipment> c =new SortByShippingCost();
        Collections.sort(sh,c);
        return sh;
    }
}

```

Klasse: SortByWeight

```

package comparator;

import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import model.Shipment;

public class SortByWeight implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.getWeightKg()).compareTo(Double.valueOf(o2.getWeight
Kg()));
    }

    public static Shipment[] sortByWeight(Shipment[] sh ) {

```

```

        Comparator<Shipment> c =new SortByWeight();
        Arrays.sort(sh,c);
        return sh;
    }

    public static List<Shipment> sortByWeightList(List<Shipment> sh )
    {
        Comparator<Shipment> c =new SortByWeight();
        Collections.sort(sh,c);
        return sh;
    }
}

Klasse: SortByWeight
package comparator;

import java.util.Arrays;
import java.util.Collections;
import java.util.Comparator;
import java.util.List;

import model.Shipment;

public class SortByWeight implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.getWeightKg()).compareTo(Double.valueOf(o2.getWeight
Kg()));
    }

    public static Shipment[] sortByWeight(Shipment[] sh ) {
        Comparator<Shipment> c =new SortByWeight();
        Arrays.sort(sh,c);
        return sh;
    }

    public static List<Shipment> sortByWeightList(List<Shipment> sh )
    {
        Comparator<Shipment> c =new SortByWeight();
        Collections.sort(sh,c);
        return sh;
    }
}

```

```
}
```

Klasse: WeightComparatorAsc

```
package comparator;

import java.util.Comparator;

import model.Shipment;

public class WeightComparatorAsc implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.getWeightKg()).compareTo(Double.valueOf(o2.getWeight
Kg()));
    }

}
```

Klasse: WeightComparatorAsc

```
package comparator;

import java.util.Comparator;

import model.Shipment;

public class WeightComparatorAsc implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o1.getWeightKg()).compareTo(Double.valueOf(o2.getWeight
Kg()));
    }

}
```

Klasse: WeightComparatorDesc

```
package comparator;

import java.util.Comparator;
import model.Shipment;

public class WeightComparatorDesc implements Comparator<Shipment> {

    @Override
    public int compare(Shipment o1, Shipment o2) {
```

```

        return
Double.valueOf(o2.getWeightKg()).compareTo(Double.valueOf(o1.getWeight
Kg()));
    }

```

```

}

```

Klasse: WeightComparatorDesc

```

package comparator;

```

```

import java.util.Comparator;
import model.Shipment;

```

```

public class WeightComparatorDesc implements Comparator<Shipment> {

```

```

    @Override
    public int compare(Shipment o1, Shipment o2) {
        return
Double.valueOf(o2.getWeightKg()).compareTo(Double.valueOf(o1.getWeight
Kg()));
    }

```

```

}

```

Package: controller

Klasse: Controller

```

package controller;

```

```

import model.Kunde;
import view.View;

```

```

public class Controller {
    /**
     * fragt Eingaben von View ab
     * lädt Modelklasse aus der DB
     * übergibt Model-Klasse an View zum anzeigen
     */
    private View v;
    private Datenbank db;

    public Controller() {
        v=new View();
        db= new Datenbank();
        this.mainloop();
    }

```

```

    public void mainloop() {
        System.out.println("Eingabe von 0 beendet das Programm");
    }

```

```

        while (true) {
            int id= v.leseKundennummer();
            if (id>0) {
                Kunde k =db.sucheKunde(id);
                v.zeigeKunde(k);
            } else if (id==0) {
                System.out.println("Beendet!");
                break;
            }
        }
    }
}

```

```

}

```

Klasse: Datenbank

```

package controller;

```

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

```

```

import model.Kunde;

```

```

public class Datenbank {

```

```

    private String
url="jdbc:mysql://localhost:3306/kundeninformationssystem";
    //3306 Standardport
    //jdbc ist der Datenbanktreiber, den ich brauche(Java DataBase
Connector)
    //mysql ist die Art von DB, die ich ansprechen will
    //kundeninformationssystem ist der Datenbankname

```

```

    private String user="root";
    //root user hat alle Rechte, Vorsicht!
    private String password="";
    //Kein Passwort, BITTE NICHT NACHMACHEN!!!

```

```

    //Weil alle Methode so ein Statement brauchen:
    private Statement st;
    public Datenbank() {
        //Konstruktor initialisiert die DB-Verbindung
        try {

```



```

        Connection c = DriverManager.getConnection(url, user,
password);
        st = c.createStatement();
        //eine Methode von Connection, die mir eine Statement-
Objekt erzeugt
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

/**
 * macht hinterher unsere Kundenanfrage
 * TODO Methode implementieren
 */
public Kunde sucheKunde(int id) {
    Kunde k=null;
        //new Kunde(id,"Bert","Bertsen");
    String sql ="Select * from kunden where kunde_id= "+ id;
    System.out.println(sql);
    try {
        ResultSet rs =st.executeQuery(sql);
        //IN dem ResultSet sind meine Ergebnisse
        while (rs.next()) {
            //rs.next() gibt true zurück wenn es weitere
Ergebnisse gibt

            String vorname =rs.getString("vorname");
            String nachname = rs.getString("name");
            id= rs.getInt("kunde_id");

            k=new Kunde(id,vorname,nachname);
        }
        //st.executeQuery(sql);
    } catch (SQLException e) {
        e.printStackTrace();
    }

    //TODO hier Logik die die DB abfragt
    return k;
}
}

```

Package: dienstag

Klasse: Kreditkarte

package dienstag;

```

public class Kreditkarte {

    void berechneKosten(int onlineZeit, int anzahlSeiten) {
        double kosten=0.0;
        if (onlineZeit %15 ==0) {
            kosten= onlineZeit/15 * 0.5;
        }
        else {
            kosten= ( onlineZeit/15 +1)*0.5;
        }

        if (anzahlSeiten<=19) {
            kosten=kosten+anzahlSeiten*0.12;
        }
        else if (anzahlSeiten<=49) {
            kosten=kosten+anzahlSeiten*0.1;
        }
        else if (anzahlSeiten<=100) {
            kosten=kosten+anzahlSeiten*0.08;
        }
        else {
            kosten=kosten+anzahlSeiten*0.05;
        }

        System.out.printf("Die Kosten betragen: %.2f € ",kosten);
    }
}

```

Klasse: Theke

```

package diensttag;

import java.util.Scanner;
public class Theke {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Kreditkarte k1= new Kreditkarte();

        System.out.print("Verbrauchte Onlinezeit in Minuten: ");
        int onlineZeit=sc.nextInt();
        System.out.print("Ausgedruckte Seiten: ");
        int anzahlSeiten=sc.nextInt();
        sc.close();

        k1.berechneKosten(onlineZeit, anzahlSeiten);
    }
}

```

```

        /* k1.berrechneKosten(30,0);
           k1.berrechneKosten(31,0);
           k1.berrechneKosten(31,100); */
    }

}

```

Package: exceptions

Klasse: InvalidShipmentDataException

```
package exceptions;
```

```

public class InvalidShipmentDataException extends Exception{

    /**
     *
     */
    private static final long serialVersionUID = 1L;

    public InvalidShipmentDataException(String message) {
        super(message);
    }

}

```

Klasse: InvalidShipmentDataException

```
package exceptions;
```

```

public class InvalidShipmentDataException extends Exception{

    /**
     *
     */
    private static final long serialVersionUID = 1L;

    public InvalidShipmentDataException(String message) {
        super(message);
    }

}

```

Klasse: UnknownShipmentTypeException

```
package exceptions;
```

```

public class UnknownShipmentTypeException extends Exception {

    /**
     *
     */
    private static final long serialVersionUID = 1L;

    public UnknownShipmentTypeException(String message) {
        super(message);
    }

}

```

Klasse: UnknownShipmentTypeException

package exceptions;

```

public class UnknownShipmentTypeException extends Exception {

    /**
     *
     */
    private static final long serialVersionUID = 1L;

    public UnknownShipmentTypeException(String message) {
        super(message);
    }

}

```

Package: gui

Klasse: Eintrag

package gui;

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Objects;

public class Eintrag {

    public String bezeichnung;
    public double betrag;
    public boolean istEinnahme;
    public LocalDate datum;
    public String kategorie;

    public Eintrag(String bezeichnung, double betrag, boolean

```

```
    istEinnahme, LocalDate datum, String kategorie) {
        super();
        this.bezeichnung = bezeichnung;
        this.betrag = betrag;
        this.istEinnahme = istEinnahme;
        this.datum = datum;
        this.kategorie = kategorie;
    }

    public String getBezeichnung() {
        return bezeichnung;
    }

    public void setBezeichnung(String bezeichnung) {
        this.bezeichnung = bezeichnung;
    }

    public double getBetrag() {
        return betrag;
    }

    public void setBetrag(double betrag) {
        this.betrag = betrag;
    }

    public boolean isEinnahme() {
        return istEinnahme;
    }

    public void setIstEinnahme(boolean istEinnahme) {
        this.istEinnahme = istEinnahme;
    }

    public LocalDate getDatum() {
        return datum;
    }

    public void setDatum(LocalDate datum) {
        this.datum = datum;
    }

    public String getKategorie() {
        return kategorie;
    }

    public void setKategorie(String kategorie) {
        this.kategorie = kategorie;
    }
}
```

```

@Override
public int hashCode() {
    return Objects.hash(betrag, bezeichnung, datum, istEinnahme,
kategorie);
}

```

```

@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Eintrag other = (Eintrag) obj;
    return Double.doubleToLongBits(betrag) ==
Double.doubleToLongBits(other.betrag)
        && Objects.equals(bezeichnung, other.bezeichnung) &&
Objects.equals(datum, other.datum)
        && istEinnahme == other.istEinnahme &&
Objects.equals(kategorie, other.kategorie);
}

```

```

@Override
public String toString() {
    return "Eintrag [bezeichnung=" + bezeichnung + ", betrag=" +
betrag + ", istEinnahme=" + istEinnahme
        + ", datum=" + datum + ", kategorie=" + kategorie +
"]";
}

```

```

public static void loescheEintrag(ArrayList<Eintrag> ae,int index)
{
    if (index >= 0 && index < ae.size()) {
        ae.remove(index);
    } else {
        System.out.println("Ungültiger Index!");
    }
}

```

```

public static Eintrag aendereEintrag() {
    return new Eintrag(
        IO.readString("Bezeichnung: "),
        IO.readDouble("Betrag: "),
        IO.readBoolean("Einnahme?(j/n): "),
        IO.readDate("Datum: "),
        IO.readString("Kategorie: ")
    );
}

```

```

    }
    public static void fuegeEintragHinzu(ArrayList<Eintrag>
ae ,Eintrag e) {
        ae.add(e);
    }
}

```

Klasse: FestWert

```

package gui;

import java.time.LocalDate;

public class FestWert extends Eintrag{
    public int periode;

    public FestWert(String bezeichnung, double betrag, boolean
istEinnahme, LocalDate datum, String kategorie,
        int periode) {
        super(bezeichnung, betrag, istEinnahme, datum, kategorie);
        this.periode = periode;
    }

    public int getPeriode() {
        return periode;
    }

    public void setPeriode(int periode) {
        this.periode = periode;
    }

    @Override
    public String toString() {
        return "FestWert [periode=" + periode + ", bezeichnung=" +
bezeichnung + ", betrag=" + betrag + ", istEinnahme="
        + istEinnahme + ", datum=" + datum + ", kategorie=" +
kategorie + "]\n";
    }
}

```

Klasse: HauptfensterController

```

package gui;

```

```

import javafx.beans.property.SimpleDoubleProperty;
import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.scene.control.cell.PropertyValueFactory;
import javafx.beans.property.SimpleDoubleProperty;
import javafx.beans.property.SimpleIntegerProperty;
import javafx.beans.property.SimpleObjectProperty;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.fxml.FXML;
import javafx.fxml.Initializable;
import javafx.scene.chart.BarChart;
import javafx.scene.chart.CategoryAxis;
import javafx.scene.chart.NumberAxis;
import javafx.scene.chart.XYChart;
import javafx.scene.control.*;
import javafx.scene.control.cell.PropertyValueFactory;

import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.net.URL;
import java.time.LocalDate;
import java.util.*;

public class HauptfensterController implements Initializable {

    @FXML private TableView<Eintrag> eintraegeTable;
    @FXML private TableColumn<Eintrag, String> colBezeichnung;
    @FXML private TableColumn<Eintrag, Double> colBetrag;
    @FXML private TableColumn<Eintrag, String> colKategorie;
    @FXML private TableColumn<Eintrag, LocalDate> colDatum;

    @FXML private TableView<FestWert> festeTable;
    @FXML private TableView<SparzielManager.Sparziel> sparzielTable;

    @FXML private BarChart<String, Number> monatsChart;
    @FXML private CategoryAxis xAxis;
    @FXML private NumberAxis yAxis;

    @FXML private Label statusLabel;

    private ArrayList<Eintrag> eintraege = new ArrayList<>();
    private ArrayList<FestWert> festeWerte = new ArrayList<>();

    @Override
    public void initialize(URL location, ResourceBundle resources) {
        setupTabellen();
        ladeDaten();
    }

```



```

        SparzielManager.automatischeZufuehrungBeimStart(festeWerte);
        updateAlleAnzeigen();
        statusLabel.setText("Daten geladen – " + eintraege.size() + "
Einträge, " + festeWerte.size() + " Feste Werte");
    }

```

```

//    private void setupTabellen() {
//        // Einträge
//        colBezeichnung.setCellValueFactory(new
PropertyValueFactory<>("bezeichnung"));
//        colBetrag.setCellValueFactory(cell -> new
SimpleDoubleProperty(cell.getValue().betrag).asObject());
//        colKategorie.setCellValueFactory(new
PropertyValueFactory<>("kategorie"));
//        colDatum.setCellValueFactory(new
PropertyValueFactory<>("datum"));
//
//        // Feste Werte (vereinfacht)
//        festeTable.getColumns().addAll(
//            new TableColumn<>("Bezeichnung")
//            {{ setCellValueFactory(new PropertyValueFactory<>("bezeichnung")); }},
//            new TableColumn<>("Betrag") {{ setCellValueFactory(cell
-> new SimpleDoubleProperty(cell.getValue().betrag).asObject()); }},
//            new TableColumn<>("Kategorie")
//            {{ setCellValueFactory(new PropertyValueFactory<>("kategorie")); }},
//            new TableColumn<>("Periode") {{ setCellValueFactory(cell
-> new SimpleIntegerProperty(cell.getValue().periode).asObject()); }},
//        );
//
//        // Sparziele
//        sparzielTable.getColumns().addAll(
//            new TableColumn<>("Name") {{ setCellValueFactory(cell ->
new SimpleStringProperty(cell.getValue().name)); }},
//            new TableColumn<>("Aktuell") {{ setCellValueFactory(cell
-> new SimpleDoubleProperty(cell.getValue().aktuell).asObject()); }},
//            new TableColumn<>("Ziel") {{ setCellValueFactory(cell ->
new SimpleDoubleProperty(cell.getValue().zielbetrag).asObject()); }},
//            new TableColumn<>("Fortschritt")
//            {{ setCellValueFactory(cell -> new
SimpleDoubleProperty(cell.getValue().prozent()).asObject()); }},
//            new TableColumn<>("Priorität")
//            {{ setCellValueFactory(cell -> new
SimpleIntegerProperty(cell.getValue().prioritaet).asObject()); }},
//        );
//    }

```

```

    private void setupTabellen() {
        colBetrag.setCellFactory(tc -> new TableCell<Eintrag,
Double>() {
            @Override

```

```

        protected void updateItem(Double item, boolean empty) {
            super.updateItem(item, empty);
            setText(empty || item == null ? "" :
String.format("%.2f €", item));
        }
    });

```

```

        // === EINTRÄGE TABELLE ===
        colBezeichnung.setCellValueFactory(new
PropertyValueFactory<>("bezeichnung"));
        colBetrag.setCellValueFactory(new
PropertyValueFactory<>("betrag"));
        colKategorie.setCellValueFactory(new
PropertyValueFactory<>("kategorie"));
        colDatum.setCellValueFactory(new
PropertyValueFactory<>("datum"));

        // === FESTE WERTE TABELLE ===
        TableColumn<FestWert, String> colFestBez = new
TableColumn<>("Bezeichnung");
        colFestBez.setCellValueFactory(new
PropertyValueFactory<>("bezeichnung"));

        TableColumn<FestWert, Double> colFestBetrag = new
TableColumn<>("Betrag");
        colFestBetrag.setCellValueFactory(new
PropertyValueFactory<>("betrag"));

        TableColumn<FestWert, String> colFestKat = new
TableColumn<>("Kategorie");
        colFestKat.setCellValueFactory(new
PropertyValueFactory<>("kategorie"));

        TableColumn<FestWert, Integer> colFestPeriode = new
TableColumn<>("Periode");
        colFestPeriode.setCellValueFactory(new
PropertyValueFactory<>("periode"));

        festeTable.getColumns().setAll(colFestBez, colFestBetrag,
colFestKat, colFestPeriode);

```

```

        // === SPARZIELE TABELLE ===
        TableColumn<SparzielManager.Sparziel, String> colSparName =
new TableColumn<>("Name");
        colSparName.setCellValueFactory(cellData -> new
SimpleStringProperty(cellData.getValue().name));

```

```

        TableColumn<SparzielManager.Sparziel, Double> colSparAktuell =
new TableColumn<>("Aktuell");
        colSparAktuell.setCellValueFactory(cellData -> new
SimpleDoubleProperty(cellData.getValue().aktuell).asObject());

```

```

        TableColumn<SparzielManager.Sparziel, Double> colSparZiel =
new TableColumn<>("Ziel");
        colSparZiel.setCellValueFactory(cellData -> new
SimpleDoubleProperty(cellData.getValue().zielbetrag).asObject());

```

```

        TableColumn<SparzielManager.Sparziel, Double> colSparProzent =
new TableColumn<>("Fortschritt %");
        colSparProzent.setCellValueFactory(cellData -> new
SimpleDoubleProperty(cellData.getValue().prozent()).asObject());

```

```

        TableColumn<SparzielManager.Sparziel, Integer> colSparPrio =
new TableColumn<>("Priorität");
        colSparPrio.setCellValueFactory(cellData -> new
SimpleIntegerProperty(cellData.getValue().prioritaet).asObject());

```

```

        sparzielTable.setColumns().setAll(colSparName, colSparAktuell,
colSparZiel, colSparProzent, colSparPrio);
    }

```

```

    private void ladeDaten() {
        eintraege.clear();
        festeWerte.clear();

        // Einträge laden (dein alter Code)
        try (Scanner sc = new Scanner(new
FileReader("eintraege.txt"))) {
            while (sc.hasNextLine()) {
                String[] parts = sc.nextLine().split("#");
                if (parts.length == 5) {
                    Eintrag e = new Eintrag(parts[0].trim(),
Double.parseDouble(parts[1].trim()),
Boolean.parseBoolean(parts[2].trim()),
LocalDate.parse(parts[3].trim()), parts[4].trim());
                    eintraege.add(e);
                }
            }
        } catch (IOException e) {
            // Datei existiert nicht – leer starten
        }

        // Feste Werte laden
        try (Scanner sc = new Scanner(new
FileReader("festeWerte.txt"))) {

```

```

        while (sc.hasNextLine()) {
            String[] parts = sc.nextLine().split("#");
            if (parts.length == 6) {
                FestWert f = new FestWert(parts[0].trim(),
Double.parseDouble(parts[1].trim()),
                Boolean.parseBoolean(parts[2].trim()),
LocalDate.parse(parts[3].trim()), parts[4].trim(),
                Integer.parseInt(parts[5].trim()));
                festeWerte.add(f);
            }
        }
    } catch (IOException e) {
        // Datei existiert nicht – leer starten
    }

    BudgetManager.ladeBudgets();
    SparzielManager.ladeSparziele();
}

private void updateAlleAnzeigen() {

    eintraegeTable.setItems(FXCollections.observableArrayList(eintraege));

    festeTable.setItems(FXCollections.observableArrayList(festeWerte));

    sparzielTable.setItems(FXCollections.observableArrayList(SparzielManager.ziele));
    updateMonatsChart();
}

@FXML
private void zeigeEintragDialog() {
    // Einfacher Dialog (kann erweitert werden)
    TextInputDialog dialog = new TextInputDialog();
    dialog.setTitle("Neuer Eintrag");
    dialog.setHeaderText("Bezeichnung");
    Optional<String> bez = dialog.showAndWait();
    if (bez.isPresent()) {
        // Hier mehr Dialoge für betrag, etc. – vorerst
vereinfacht
        double betrag = 100.0; // TODO: Richtigen Dialog
        boolean einnahme = false;
        String kat = "Sonstiges";
        Eintrag e = new Eintrag(bez.get(), betrag, einnahme,
LocalDate.now(), kat);
        Eintrag.fuegeEintragHinzu(eintraege, e);
        speichereEintraege();
        updateAlleAnzeigen();
        statusLabel.setText("Eintrag hinzugefügt: " + bez.get());
    }
}

```

```

    }

    @FXML
    private void loescheEintrag() {
        Eintrag selected =
    eintraegeTable.getSelectionModel().getSelectedItem();
        if (selected != null) {
            eintraege.remove(selected);
            speichereEintraege();
            updateAlleAnzeigen();
        }
    }

    @FXML
    private void aendereEintrag() {
        // Ähnlich wie hinzufügen – TODO
        statusLabel.setText("Ändern: Implementiere Dialog!");
    }

    @FXML
    private void zeigeFestWertDialog() {
        // TODO: Dialog für FestWert
        statusLabel.setText("Fester Wert: Implementiere Dialog!");
    }

    @FXML
    private void loescheFestWert() {
        FestWert selected =
    festeTable.getSelectionModel().getSelectedItem();
        if (selected != null) {
            festeWerte.remove(selected);
            speichereFesteWerte();
            updateAlleAnzeigen();
        }
    }

    @FXML
    private void neuesSparziel() {
        SparzielManager.neuesSparziel(); // Deine Methode!
        updateAlleAnzeigen();
    }

    @FXML
    private void aendereSparziel() {
        SparzielManager.aendereSparziel();
        updateAlleAnzeigen();
    }

    @FXML

```

```

private void loescheSparziel() {
    SparzielManager.loescheSparziel();
    updateAlleAnzeigen();
}

@FXML
private void setzeSparrate() {
    SparzielManager.setzeSparrate(festeWerte);
    speichereFesteWerte();
    updateAlleAnzeigen();
}

@FXML
private void updateMonatsChart() {
    monatsChart.getData().clear();
    XYChart.Series<String, Number> series = new
XYChart.Series<>();
    series.setName("Ausgaben");

    // Beispiel-Daten (deine Uebersicht.zeigeMonatsBalken()-Logik
hier einfügen)
    Map<String, Double> katSum = new HashMap<>();
    katSum.put("Lebensmittel", 450.0);
    katSum.put("Miete", 1200.0);
    katSum.put("Transport", 150.0);

    for (Map.Entry<String, Double> entry : katSum.entrySet()) {
        series.getData().add(new XYChart.Data<>(entry.getKey(),
entry.getValue()));
    }
    monatsChart.getData().add(series);
}

private void speichereEintraege() {
    try (FileWriter writer = new FileWriter("eintraege.txt")) {
        for (Eintrag e : eintraege) {
            writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%n",
e.datum, e.kategorie));
            e.bezeichnung, e.betrag, e.istEinnahme,
e.datum, e.kategorie));
        }
    } catch (IOException e) {
        statusLabel.setText("Fehler beim Speichern!");
    }
}

private void speichereFesteWerte() {
    try (FileWriter writer = new FileWriter("festeWerte.txt")) {
        for (FestWert f : festeWerte) {
            writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%n",
f.datum, f.betrag, f.kategorie, f.bezeichnung, f.istEinnahme));
        }
    }
}

```

```

%s#%d%n",
                                f.bezeichnung, f.betrag, f.istEinnahme,
f.datum, f.kategorie, f.periode));
    }
    } catch (IOException e) {
        statusLabel.setText("Fehler beim Speichern!");
    }
}
}package _10_22.arbeitsblatt;

import _10_15.IO;

public class Haus{
    private String adresse;
    private int zimmer ; // Zimmeranzahl
    private double flaeche;
    public Haus(String adresse, int zimmer, double flaeche) {

        this.adresse = adresse;
        this.zimmer = zimmer;
        this.flaeche = flaeche;
    }

    public double preisBerechnen(double preisProQm) {
        return preisProQm * flaeche;
    }

    public void zeigeInfo() {
        System.out.printf("%s, %d, %.2f,%.2f € %n",adresse, zimmer,
flaeche, preisBerechnen(IO.readDouble("Preis pro Qm: ")));
    }

    public String getAdresse() {
        return adresse;
    }

    public void setAdresse(String adresse) {
        this.adresse = adresse;
    }

    public int getZimmer() {
        return zimmer;
    }

    public void setZimmer(int zimmer) {
        this.zimmer = zimmer;
    }

    public double getFlaeche() {

```

```

        return flaeche;
    }

    public void setFlaeche(double flaeche) {
        this.flaeche = flaeche;
    }

    @Override
    public String toString() {
        // TODO Auto-generated method stub
        return adresse + "\t"+zimmer +"\t"+flaeche+"qm";
    }

}package _10_22_arbeitsblatt;

import _10_15.IO;

public class Haus{

    private String adresse;
    private int zimmer;
    private double flaeche;

    public Haus(String adresse, int zimmer, double flaeche) {
        this.adresse = adresse;
        this.zimmer = zimmer;
        this.flaeche = flaeche;
    }

    public String getAdresse() {
        return adresse;
    }

    public void setAdresse(String adresse) {
        this.adresse = adresse;
    }

    public int getZimmer() {
        return zimmer;
    }

    public void setZimmer(int zimmer) {
        this.zimmer = zimmer;
    }
}

```



```

    public double getFlaeche() {
        return flaeche;
    }

    public void setFlaeche(double flaeche) {
        this.flaeche = flaeche;
    }

    public double preisBerechnen(double preisProQm) {
        return preisProQm*flaeche;
    }

    public void zeigeInfo() {
        System.out.printf("%s, %d, %.2fm², %, .2f€ \
n",adresse,zimmer,flaeche,preisBerechnen(IO.readDouble("Preis pro m²:
"))));
    }

}package _10_24_lab;

import java.util.Objects;

import _10_15.IO;

public class Haus{
    private String adresse;
    private int zimmer ; // Zimmeranzahl
    private double flaeche;
    public Haus(String adresse, int zimmer, double flaeche) {

        this.adresse = adresse;
        this.zimmer = zimmer;
        this.flaeche = flaeche;
    }

    public double preisBerechnen(double preisProQm) {
        return preisProQm * flaeche;
    }

    public void zeigeInfo() {
        System.out.printf("%s, %d, %.2f,%.2f € %n",adresse, zimmer,
flaeche, preisBerechnen(IO.readDouble("Preis pro Qm: ")));
    }
}

```

```

    public String getAdresse() {
        return adresse;
    }

    public void setAdresse(String adresse) {
        this.adresse = adresse;
    }

    public int getZimmer() {
        return zimmer;
    }

    public void setZimmer(int zimmer) {
        this.zimmer = zimmer;
    }

    public double getFlaeche() {
        return flaeche;
    }

    public void setFlaeche(double flaeche) {
        this.flaeche = flaeche;
    }

    @Override
    public String toString() {
        // TODO Auto-generated method stub
        return adresse + "\t"+zimmer +"\t"+flaeche+"qm";
    }

    @Override
    public int hashCode() {
        return Objects.hash(adresse, flaeche, zimmer);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Haus other = (Haus) obj;
        return Objects.equals(adresse, other.adresse)
            && Double.doubleToLongBits(flaeche) ==
            Double.doubleToLongBits(other.flaeche) && zimmer == other.zimmer;
    }

```

```

}package ArrayListAufgaben;

import java.util.Objects;

import _10_15.IO;

public class Haus {
    private String adresse;
    private int zimmer; // Zimmeranzahl
    private double flaeche;

    public Haus(String adresse, int zimmer, double flaeche) {
        this.adresse = adresse;
        this.zimmer = zimmer;
        this.flaeche = flaeche;
    }

    public double preisBerechnen(double preisProQm) {
        return preisProQm * flaeche;
    }

    public void zeigeInfo() {
        System.out.printf("%s, %d, %.2f,%.2f € %n", adresse, zimmer,
flaeche, preisBerechnen(IO.readDouble("Preis pro Qm: ")));
    }

    public String getAdresse() {
        return adresse;
    }

    public void setAdresse(String adresse) {
        this.adresse = adresse;
    }

    public int getZimmer() {
        return zimmer;
    }

    public void setZimmer(int zimmer) {
        this.zimmer = zimmer;
    }

    public double getFlaeche() {
        return flaeche;
    }

    public void setFlaeche(double flaeche) {

```

```

        this.flaeche = flaeche;
    }

    @Override
    public String toString() {
        return adresse + "\t" + zimmer + "\t" + flaeche + "qm";
    }

    @Override
    public int hashCode() {
        return Objects.hash(adresse, flaeche, zimmer);
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null)
            return false;
        if (getClass() != obj.getClass())
            return false;
        Haus other = (Haus) obj;
        return Objects.equals(adresse, other.adresse)
            && Double.doubleToLongBits(flaeche) ==
Double.doubleToLongBits(other.flaeche) && zimmer == other.zimmer;
    }
}package _10_21_Arbeitsblatt;

public class Haus {

    String adresse;
    int zimmer;
    double flaeche;

    public Haus(String a,int z,double f){
        adresse=a;
        zimmer=z;
        flaeche=f;
    }

    public double preisBerechnen(double preisProQm) {
        return preisProQm*flaeche;
    }

}

```

Klasse: HaushaltsApp

```
package gui;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

public class HaushaltsApp extends Application {

    @Override
    public void start(Stage stage) throws Exception {
        Parent root =
FXMLLoader.load(getClass().getResource("/gui/Hauptfenster.fxml"));
        Scene scene = new Scene(root, 1100, 750);
        stage.setTitle("Haushaltskasse – GUI");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}package HaushaltskassenAppGUI;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.io.*;
import java.util.Locale;
import java.util.Scanner;

public class HaushaltsAppGUI extends JFrame {
    private ArrayList<Eintrag> eintraege;
    private ArrayList<FestWert> festeWerte;

    private JTable eintragTable;
    private JTable festWertTable;
    private DefaultTableModel eintragModel;
    private DefaultTableModel festWertModel;

    private JLabel lblBilanz;

    public HaushaltsAppGUI() {
        eintraege = new ArrayList<>();
        festeWerte = new ArrayList<>();
    }
}
```

```

        ladeDaten();

        setTitle("Haushaltskassen-App");
        setSize(1000, 700);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        initUI();
        updateTables();
        updateBilanz();
    }

    private void initUI() {
        JTabbedPane tabbedPane = new JTabbedPane();

        // === Tab 1: Einträge ===
        JPanel panelEintraege = new JPanel(new BorderLayout());
        eintragModel = new DefaultTableModel(new String[]{
            "Index", "Bezeichnung", "Betrag", "Einnahme", "Datum",
            "Kategorie"
        }, 0);
        eintragTable = new JTable(eintragModel);
        JScrollPane scroll1 = new JScrollPane(eintragTable);
        panelEintraege.add(scroll1, BorderLayout.CENTER);

        JPanel btnPanel1 = new JPanel();
        JButton btnAdd = new JButton("Hinzufügen");
        JButton btnEdit = new JButton("Ändern");
        JButton btnDelete = new JButton("Löschen");
        btnPanel1.add(btnAdd); btnPanel1.add(btnEdit);
        btnPanel1.add(btnDelete);
        panelEintraege.add(btnPanel1, BorderLayout.SOUTH);

        btnAdd.addActionListener(e -> showEintragDialog(null));
        btnEdit.addActionListener(e -> editSelectedEintrag());
        btnDelete.addActionListener(e -> deleteSelectedEintrag());

        tabbedPane.addTab("Einträge", panelEintraege);

        // === Tab 2: Feste Werte ===
        JPanel panelFeste = new JPanel(new BorderLayout());
        festWertModel = new DefaultTableModel(new String[]{
            "Index", "Bezeichnung", "Betrag", "Einnahme", "Datum",
            "Kategorie", "Periode"
        }, 0);
        festWertTable = new JTable(festWertModel);
        JScrollPane scroll2 = new JScrollPane(festWertTable);
        panelFeste.add(scroll2, BorderLayout.CENTER);
    }

```

```

JPanel btnPanel2 = new JPanel();
JButton btnAddF = new JButton("Hinzufügen");
JButton btnEditF = new JButton("Ändern");
JButton btnDeleteF = new JButton("Löschen");
btnPanel2.add(btnAddF); btnPanel2.add(btnEditF);
btnPanel2.add(btnDeleteF);
panelFeste.add(btnPanel2, BorderLayout.SOUTH);

btnAddF.addActionListener(e -> showFestWertDialog(null));
btnEditF.addActionListener(e -> editSelectedFestWert());
btnDeleteF.addActionListener(e -> deleteSelectedFestWert());

tabbedPane.addTab("Feste Werte", panelFeste);

// === Tab 3: Übersicht ===
JPanel panelUebersicht = new JPanel(new BorderLayout());
JTextArea txtUebersicht = new JTextArea();
txtUebersicht.setEditable(false);
JScrollPane scrollUebersicht = new JScrollPane(txtUebersicht);
panelUebersicht.add(scrollUebersicht, BorderLayout.CENTER);

JPanel btnPanel3 = new JPanel();
JButton btnRefresh = new JButton("Aktualisieren");
btnPanel3.add(btnRefresh);
panelUebersicht.add(btnPanel3, BorderLayout.SOUTH);

btnRefresh.addActionListener(e -> {
txtUebersicht.setText(Uebersicht.getUebersichtText(eintraege,
festeWerte));
    updateBilanz();
});

tabbedPane.addTab("Übersicht", panelUebersicht);

// === Bilanz Anzeige ===
lblBilanz = new JLabel("Bilanz: 0,00 €",
SwingConstants.CENTER);
lblBilanz.setFont(new Font("Arial", Font.BOLD, 16));
add(lblBilanz, BorderLayout.NORTH);
add(tabbedPane, BorderLayout.CENTER);

// Beim Schließen speichern
addWindowListener(new java.awt.event.WindowAdapter() {
    @Override
    public void windowClosing(java.awt.event.WindowEvent
windowEvent) {
        speichereDaten();
    }
}

```

```

    });
}

// === Eintrag Dialog ===
private void showEintragDialog(Eintrag e) {
    EintragDialog dialog = new EintragDialog(this, e);
    dialog.setVisible(true);
    if (dialog.isSaved()) {
        if (e == null) {
            eintraege.add(dialog.getEintrag());
        } else {
            int index = eintraege.indexOf(e);
            eintraege.set(index, dialog.getEintrag());
        }
        updateTables();
        updateBilanz();
    }
}

private void editSelectedEintrag() {
    int row = eintragTable.getSelectedRow();
    if (row >= 0) {
        int index = (int) eintragModel.getValueAt(row, 0);
        showEintragDialog(eintraege.get(index));
    } else {
        JOptionPane.showMessageDialog(this, "Bitte einen Eintrag
auswählen!");
    }
}

private void deleteSelectedEintrag() {
    int row = eintragTable.getSelectedRow();
    if (row >= 0) {
        int index = (int) eintragModel.getValueAt(row, 0);
        eintraege.remove(index);
        updateTables();
        updateBilanz();
    }
}

// === FestWert Dialog ===
private void showFestWertDialog(FestWert f) {
    FestWertDialog dialog = new FestWertDialog(this, f);
    dialog.setVisible(true);
    if (dialog.isSaved()) {
        if (f == null) {
            festeWerte.add(dialog.getFestWert());
        } else {
            int index = festeWerte.indexOf(f);
            festeWerte.set(index, dialog.getFestWert());
        }
    }
}

```



```

        }
        updateTables();
        updateBilanz();
    }
}

private void editSelectedFestWert() {
    int row = festWertTable.getSelectedRow();
    if (row >= 0) {
        int index = (int) festWertModel.getValueAt(row, 0);
        showFestWertDialog(festeWerte.get(index));
    } else {
        JOptionPane.showMessageDialog(this, "Bitte einen festen
Wert auswählen!");
    }
}

private void deleteSelectedFestWert() {
    int row = festWertTable.getSelectedRow();
    if (row >= 0) {
        int index = (int) festWertModel.getValueAt(row, 0);
        festeWerte.remove(index);
        updateTables();
        updateBilanz();
    }
}

private void updateTables() {
    eintragModel.setRowCount(0);
    for (int i = 0; i < eintraege.size(); i++) {
        Eintrag e = eintraege.get(i);
        eintragModel.addRow(new Object[]{
            i, e.bezeichnung, e.betrag, e.istEinnahme ? "Ja" :
"Nein",
            e.datum, e.kategorie
        });
    }

    festWertModel.setRowCount(0);
    for (int i = 0; i < festeWerte.size(); i++) {
        FestWert f = festeWerte.get(i);
        festWertModel.addRow(new Object[]{
            i, f.bezeichnung, f.betrag, f.istEinnahme ? "Ja" :
"Nein",
            f.datum, f.kategorie, f.periode
        });
    }
}

private void updateBilanz() {

```

```

        double bilanz = 0.0;
        for (Eintrag e : eintraege) bilanz += e.istEinnahme ? e.betrag
: -e.betrag;
        for (FestWert f : festeWerte) bilanz += f.istEinnahme ?
f.betrag : -f.betrag;
        lblBilanz.setText(String.format("Bilanz: %.2f €", bilanz));
        lblBilanz.setForeground(bilanz >= 0 ? Color.GREEN.darker() :
Color.RED);
    }

```

```

    private void ladeDaten() {
        eintraege = new ArrayList<>();
        festeWerte = new ArrayList<>();

        // === Einträge laden ===
        File eintraegeFile = new File("eintraege.txt");
        if (eintraegeFile.exists()) {
            try (Scanner fileScanner = new Scanner(new
FileReader(eintraegeFile))) {
                while (fileScanner.hasNextLine()) {
                    String line = fileScanner.nextLine();
                    String[] parts = line.split("#", -1); // -1 behält
leere Strings

                    if (parts.length == 5) {
                        Eintrag e = new Eintrag(
                            parts[0].trim(),
                            Double.parseDouble(parts[1].trim()),
                            Boolean.parseBoolean(parts[2].trim()),
                            LocalDate.parse(parts[3].trim()),
                            parts[4].trim()
                        );
                        eintraege.add(e);
                    }
                }
            } catch (FileNotFoundException e) {
                // Datei existiert, aber kann nicht gelesen werden?
Unwahrscheinlich.
            } catch (Exception e) {
                JOptionPane.showMessageDialog(this,
                    "Fehler beim Laden der Einträge: " +
e.getMessage(),
                    "Ladefehler", JOptionPane.ERROR_MESSAGE);
            }
        }
    }

```

```

        // === Feste Werte laden ===
        File festeWerteFile = new File("festeWerte.txt");
        if (festeWerteFile.exists()) {
            try (Scanner fileScanner = new Scanner(new
FileReader(festeWerteFile))) {

```

```

        while (fileScanner.hasNextLine()) {
            String line = fileScanner.nextLine();
            String[] parts = line.split("#", -1);
            if (parts.length == 6) {
                FestWert f = new FestWert(
                    parts[0].trim(),
                    Double.parseDouble(parts[1].trim()),
                    Boolean.parseBoolean(parts[2].trim()),
                    LocalDate.parse(parts[3].trim()),
                    parts[4].trim(),
                    Integer.parseInt(parts[5].trim())
                );
                festeWerte.add(f);
            }
        }
    } catch (FileNotFoundException e) {
        // Sollte nicht passieren, da exist() true war
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this,
            "Fehler beim Laden der festen Werte: " +
e.getMessage(),
            "Ladefehler", JOptionPane.ERROR_MESSAGE);
    }
}

private void speichereDaten() {
    // === Einträge speichern ===
    try (FileWriter writer = new FileWriter("eintraege.txt")) {
        for (Eintrag e : eintraege) {
            writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%n",
                e.bezeichnung,
                e.betrag,
                e.istEinnahme,
                e.datum,
                e.kategorie
            ));
        }
    } catch (IOException e) {
        JOptionPane.showMessageDialog(this,
            "Fehler beim Speichern der Einträge: " +
e.getMessage(),
            "Speicherfehler", JOptionPane.ERROR_MESSAGE);
    }

    // === Feste Werte speichern ===
    try (FileWriter writer = new FileWriter("festeWerte.txt")) {
        for (FestWert f : festeWerte) {
            writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#%s#%n",

```

```

%s#%d%n",
        f.bezeichnung,
        f.betrag,
        f.istEinnahme,
        f.datum,
        f.kategorie,
        f.periode
    ));
    }
} catch (IOException e) {
    JOptionPane.showMessageDialog(this,
        "Fehler beim Speichern der festen Werte: " +
e.getMessage(),
        "Speicherfehler", JOptionPane.ERROR_MESSAGE);
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        try {
            UIManager.setLookAndFeel(UIManager.getLookAndFeel());
        } catch (Exception e) {}
        new HaushaltsAppGUI().setVisible(true);
    });
}
}package HaushaltskassenApp_zwei;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.stage.Stage;

/**
 * Hauptklasse für die JavaFX-GUI der Haushaltskassen-App
 *
 * Diese Klasse startet die grafische Benutzeroberfläche.
 * Um die GUI zu starten, rufen Sie diese Klasse auf (anstatt
Verwaltung.main).
 */
public class HaushaltskassenApp extends Application {

    private static DataModel dataModel;

    @Override
    public void start(Stage primaryStage) {
        // Datenmodell initialisieren
        dataModel = new DataModel();
        dataModel.ladeAlleDaten();

        // Hauptfenster erstellen
        MainWindowController mainWindow = new

```

```

MainWindowController(dataModel);
    Scene scene = new Scene(mainWindow.getRoot(), 1200, 800);

    // Stylesheet für modernes Aussehen (optional - kann später
    hinzugefügt werden)
    //
    scene.getStylesheets().add(getClass().getResource("/styles.css").toExternalForm());

    primaryStage.setTitle("Haushaltskassen-App");
    primaryStage.setScene(scene);
    primaryStage.setMinWidth(1000);
    primaryStage.setMinHeight(700);

    // Beim Schließen speichern
    primaryStage.setOnCloseRequest(e -> {
        dataModel.speichereAlleDaten();
        Schlussspruch.schreibeSchluss();
    });

    primaryStage.show();
}

/**
 * Einstiegspunkt für die GUI-Version
 */
public static void main(String[] args) {
    launch(args);
}

public static DataModel getDataModel() {
    return dataModel;
}
}

```

Klasse: IO

```

package gui;

import java.time.LocalDate;
import java.util.Scanner;

public class IO {

    public static int readInt(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextInt();
    }
}

```

```

    }
    public static double readDouble(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextDouble();
    }
    public static String readString(String prompt) {
        System.out.print(prompt);
        Scanner scan = new Scanner(System.in);
        return scan.nextLine();
    }
    public static boolean readBoolean(String prompt) {
        Scanner scan = new Scanner(System.in);
        while (true) {
            System.out.print(prompt);
            String input = scan.nextLine().trim().toLowerCase();
            if (input.equals("j") || input.equals("ja") ||
input.equals("y") || input.equals("yes")) {
                return true;
            } else if (input.equals("n") || input.equals("nein") ||
input.equals("no")) {
                return false;
            } else {
                System.out.println("Ungültige Eingabe! Bitte 'j',
'ja', 'n' oder 'nein' eingeben.");
            }
        }
    }
    public static LocalDate readDate(String prompt) {
        System.out.println(prompt);
        try {
            return LocalDate.of(
                IO.readInt("Jahr: "),
                IO.readInt("Monat: "),
                IO.readInt("Tag: ")
            );
        } catch (Exception e) {
            System.out.println("Ungültiges Datum! Heutiges Datum wird
verwendet!");
            return LocalDate.now();
        }
    }
}

```

Klasse: Jframe_first

package gui;

import java.awt.EventQueue;

```

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.border.EmptyBorder;
import javax.swing.JToolBar;
import javax.swing.JButton;
import java.awt.Color;
import javax.swing.SwingConstants;
import javax.swing.JCheckBox;

public class JFrame_first extends JFrame {

    private static final long serialVersionUID = 1L;
    private JPanel contentPane;
    /**
     * @wbp.nonvisual location=204,8
     */
    private final JToolBar toolBar = new JToolBar();

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    JFrame_first frame = new JFrame_first();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    /**
     * Create the frame.
     */
    public JFrame_first() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 300);
        contentPane = new JPanel();
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);

        JCheckBox chckbxNewCheckBox = new JCheckBox("Auswahl");
        contentPane.add(chckbxNewCheckBox);

        JButton btnNewButton = new JButton("drück Mich");
        btnNewButton.setVerticalAlignment(SwingConstants.TOP);
        btnNewButton.setForeground(Color.ORANGE);
    }
}

```

```

        btnNewButton.setBackground(Color.RED);
        contentPane.add(btnNewButton);

    }

}

```

Klasse: **Schlusspruch**

```
package gui;
```

```
public class Schlusspruch {
```

```

    static String [] sprueche= {"Rechnen ist schön, wenn am Ende mehr
    rauskommt als reinging.",
    "Wer das Geld hat, hat die Taschen voll –
    wer keins hat, die Hände.",
    "Geld ist nichts. Aber viel Geld, das ist
    etwas anderes.",
    "Geld verdirbt den Charakter – vor allem,
    wenn man keines hat.",
    "Ich spare, egal was es kostet.",
    "Geld ist wie ein sechster Sinn – ohne ihn
    kann man die anderen fünf nicht richtig nutzen.",
    "Geld spricht. Meistens sagt es: Leb
    wohl!",
    "Man sagt, Geld allein macht nicht
    glücklich. Es muss schon einem gehören.",
    "Geld ist wie ein Bumerang – es kommt nur
    selten zurück.",
    "Geld allein macht nicht glücklich – es
    gehören auch noch ein bisschen Glück und Fantasie dazu.",
    "Geld ist besser als Armut – wenn auch nur
    aus finanziellen Gründen." ,
    "Mein Kontostand ist wie eine Zwiebel:
    Wenn ich ihn ansehe, muss ich weinen.",
    "Geld ist wie Dünger. Es nützt nur, wenn
    man es verteilt." ,
    "Ich habe keine Bank, ich habe kein Geld –
    ich habe Kinder!",
    "Sparen ist gut, besonders wenn die Eltern
    es tun." ,
    "Früher war mehr Lametta – und mehr Geld
    im Portemonnaie.",
    "Geld schläft nicht, es liegt nur manchmal
    auf anderen Konten.",
    "Wer den Cent nicht ehrt, hat schon den
    Euro verloren.",
    "Am Ende des Geldes ist noch so viel Monat
    übrig.",
    "Geld allein macht nicht glücklich. Es

```



```

gehören auch noch Aktien, Gold und Grundstücke dazu.",
                                "Reich ist nicht, wer viel hat, sondern
wer wenig braucht.",
                                "Ich besitze nichts, aber alles, was ich
brauche, habe ich immer bei mir.",
                                "Die Götter haben alles Überflüssige teuer
gemacht und das Notwendige billig.",
                                "Wer zufrieden ist, ist reich.",
                                "Je weniger Bedürfnisse, desto weniger
Sorgen.",
                                "Nicht der Mangel, sondern die Gier macht
arm.",
                                "Weniger ist oft mehr."
};
static double zufallszahl=Math.random();
static int zufallszahlGanz=(int) (zufallszahl*sprueche.length);
static String schlusstext=sprueche[zufallszahlGanz];

public static void schreibeSchluss() {
    System.out.println("=".repeat(schlusstext.length()));
    try {
        System.out.println(schlusstext);
    } catch (Exception e) {

        e.printStackTrace();
    }
    System.out.println("=".repeat(schlusstext.length()));
}

}

```

Klasse: SparzielManager

```

package gui;
//
//
//import java.io.*;
//import java.time.*;
//import java.util.*;
//import java.util.Locale;
//
//public class SparzielManager {
//    private static final List<Sparziel> ziele = new ArrayList<>();
//    private static final String DATEI = "sparziel.txt";
//    private static LocalDate letzteZufuehrung = null;
//    private static final String LETZTE_ZUFUEHRUNG_KEY =
"letzteZufuehrung";
//
//    // --- SPARRATE AUS FESTEN WERTEN LESEN ---
////    public static double

```

```

getSparrateAusFesteWerte(ArrayList<FestWert> festeWerte) {
    ////      return festeWerte.stream()
    ////      .filter(f -> !f.istEinnahme &&
    "Sparen".equals(f.kategorie))
    ////      .mapToDouble(f -> f.betrag)
    ////      .sum();
    ////  }
    //  public static double
getSparrateAusFesteWerte(ArrayList<FestWert> festeWerte) {
    //      return festeWerte.stream()
    //      .filter(f -> !f.istEinnahme &&
    //      "Sparen".equals(f.kategorie) &&
    //      "Sparplan".equals(f.bezeichnung) &&
    //      f.periode == 1)
    //      .mapToDouble(f -> f.betrag)
    //      .findFirst()
    //      .orElse(0.0);
    //  }
    //
    //  public static class Sparziel {
    //      String name;
    //      double zielbetrag;
    //      LocalDate zielDatum;
    //      double aktuell;
    //      int prioritaaet;
    //
    //      public Sparziel(String name, double zielbetrag, LocalDate
    zielDatum, double aktuell, int prioritaaet) {
    //          this.name = name;
    //          this.zielbetrag = zielbetrag;
    //          this.zielDatum = zielDatum;
    //          this.aktuell = aktuell;
    //          this.prioritaaet = prioritaaet;
    //      }
    //
    //      public double fehlend() { return Math.max(0, zielbetrag -
    aktuell); }
    //      public double prozent() { return zielbetrag > 0 ? (aktuell /
    zielbetrag) * 100 : 0; }
    //
    //      @Override
    //      public String toString() {
    //          int balken = (int) (prozent() / 10);
    //          return String.format("P%d: %-15s | %6.0f / %6.0f € | [%s
    %s] %3.0f%% | %6.0f € fehlen",
    //          prioritaaet, name, aktuell, zielbetrag,
    //          "█".repeat(balken), " ".repeat(10 - balken),
    prozent(), fehlend());
    //      }
    //  }
}

```

```

//
//    // --- LADEN ---
//    public static void ladeSparziele() {
//        ziele.clear();
//        File file = new File(DATEI);
//        if (!file.exists()) return;
//
//        try (BufferedReader br = new BufferedReader(new
//        FileReader(file))) {
//            String line;
//            while ((line = br.readLine()) != null) {
//                line = line.trim();
//                if (line.isEmpty() || line.startsWith("#"))
//                continue;
//                String[] p = line.split("#", 5);
//                if (p.length == 5) {
//                    ziele.add(new Sparziel(
//                        p[0].trim(),
//                        Double.parseDouble(p[1].trim().replace(',', ' ')),
//                        LocalDate.parse(p[2].trim()),
//                        Double.parseDouble(p[3].trim().replace(',', ' ')),
//                        Integer.parseInt(p[4].trim())
//                    ));
//                }
//            }
//            System.out.println("Sparziele geladen: " +
//            ziele.size());
//        } catch (Exception e) {
//            System.err.println("Fehler beim Laden: " +
//            e.getMessage());
//        }
//    }
//    public static void ladeSparziele() {
//        ziele.clear();
//        File file = new File(DATEI);
//        if (!file.exists()) return;try (BufferedReader br = new
//        BufferedReader(new FileReader(file))) {
//            String line;
//            while ((line = br.readLine()) != null) {
//                line = line.trim();
//                if (line.isEmpty() || line.startsWith("#")) continue;
//                if (line.startsWith(LETZTE_ZUFUEHRUNG_KEY + "#")) {
//                    String datumStr =
//                    line.substring(LETZTE_ZUFUEHRUNG_KEY.length() +1).trim();
//                    if (!datumStr.isEmpty()) {
//                        letzteZufuehrung = LocalDate.parse(datumStr);
//                    }
//                }
//                continue;
//            }
//        }
//    }

```

```

//      }
//      String[] p = line.split("#", 5);
//      if (p.length == 5) {
//          ziele.add(new Sparziel(
//              p[0].trim(),
//              Double.parseDouble(p[1].trim().replace(',', ' ')),
//              LocalDate.parse(p[2].trim()),
//              Double.parseDouble(p[3].trim().replace(',', ' ')),
//              Integer.parseInt(p[4].trim())
//          ));
//      }
//  }
//      System.out.println("Sparziele geladen: " + ziele.size());
//  } catch (Exception e) {
//      System.err.println("Fehler beim Laden: " +
e.getMessage());
//  }
//  }
//
//
//
//      // --- SPEICHERN ---
////      public static void speichereSparziele() {
////          try (PrintWriter pw = new PrintWriter(new
FileWriter(DATEI))) {
////              pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
////              ziele.stream()
////                  .sorted(Comparator.comparingInt(s ->
s.prioritaet))
////                  .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d\n",
////                      sz.name, sz.zielbetrag, sz.zielDatum,
sz.aktuell, sz.prioritaet));
////          } catch (IOException e) {
////              System.err.println("Fehler beim Speichern: " +
e.getMessage());
////          }
////      }
//
//      public static void speichereSparziele() {
//          try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
//          {
//              pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
//              ziele.stream()
//                  .sorted(Comparator.comparingInt(s -> s.prioritaet))
//                  .forEach(sz -> pw.printf("%s#%.2f#%s#%.2f#%d\n",
//                      sz.name, sz.zielbetrag, sz.zielDatum, sz.aktuell,
sz.prioritaet));
//              // LETZTE ZUFÜHRUNG SPEICHERN
//              if (letzteZufuehrung != null) {
//                  pw.println(LETZTE_ZUFUEHRUNG_KEY + "#" +

```

```

    letzteZufuehrung);
    //      }
    //    } catch (IOException e) {
    //        System.err.println("Fehler beim Speichern: " +
    e.getMessage());
    //    }
    //  }
    //
    //  // --- NEU ---
    //  public static void neuesSparziel() {
    //      String name = IO.readString("Name: ");
    //      double betrag = IO.readDouble("Zielbetrag: ");
    //      LocalDate datum = IO.readDate("Datum: ");
    //      int prio = IO.readInt("Priorität (1 = höchste): ");
    //      ziele.add(new Sparziel(name, betrag, datum, 0.0, prio));
    //      speichereSparziele();
    //  }
    //
    //  // --- MONATLICHE ZUFÜHRUNG (EINZIGE METHODE) ---
    //  public static void monatlicheZufuehrung(ArrayList<FestWert>
    festeWerte) {
    //      double sparrate = getSparrateAusFesteWerte(festeWerte);
    //      if (sparrate <= 0 || ziele.isEmpty()) {
    //          System.out.println("Keine Sparrate oder Sparziele
    vorhanden.");
    //          return;
    //      }
    //
    //      List<Sparziel> sortiert = new ArrayList<>(ziele);
    //      sortiert.sort(Comparator.comparingInt(s -> s.prioritaet));
    //
    //      double rest = sparrate;
    //      for (Sparziel sz : sortiert) {
    //          if (rest <= 0) break;
    //          double fehlend = sz.fehlend();
    //          if (fehlend > 0) {
    //              double zuweisung = Math.min(rest, fehlend);
    //              sz.aktuell += zuweisung;
    //              rest -= zuweisung;
    //          }
    //      }
    //      speichereSparziele();
    //      System.out.printf("Monatliche Sparrate: %.2f € verteilt.\n",
    sparrate);
    //  }
    //
    //  public static void
    automatischeZufuehrungBeimStart(ArrayList<FestWert> festeWerte){
    //      LocalDate heute = LocalDate.now();
    //      LocalDate monatsBeginn = heute.withDayOfMonth(1);

```

```

//      if (letzteZufuehrung == null || !
letzteZufuehrung.equals(monatsBeginn)) {
//          monatlicheZufuehrung(festeWerte);
//          letzteZufuehrung = monatsBeginn;
//          speichereSparziele(); // WICHTIG: speichern!
//      }
//  }
//
//  // --- SPARRATE ÄNDERN (EINZIGE METHODE) ---
//  public static void setzeSparrate(ArrayList<FestWert> festeWerte)
//  {
//      double aktuell = getSparrateAusFesteWerte(festeWerte);
//      double neu = IO.readDouble("Neue monatliche Sparrate
(aktuell: " + String.format(Locale.US, "%.2f", aktuell) + " €): ");
//      if (neu < 0) return;
//
//      // Alte entfernen
//      festeWerte.removeIf(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie));
//
//      // Neuen hinzufügen
//      if (neu > 0) {
//          LocalDate datum = LocalDate.now().withDayOfMonth(1);
//          festeWerte.add(new FestWert("Sparplan", neu, false,
datum, "Sparen", 1));
//      }
//      // Alten Sparplan entfernen
//      festeWerte.removeIf(f -> !f.istEinnahme &&
"Sparen".equals(f.kategorie) &&
"Sparplan".equals(f.bezeichnung));
//
//      // Neuen hinzufügen
//      if (neu > 0) {
//          LocalDate datum = LocalDate.now().withDayOfMonth(1);
//          festeWerte.add(new FestWert("Sparplan", neu, false,
datum, "Sparen", 1));
//      }
//      speichereFesteWerte(festeWerte); // <-- SOFORT SPEICHERN!
//      System.out.println("Sparrate aktualisiert und in
festeWerte.txt gespeichert.");
//  }
//
//  // --- FESTEWERTE SPEICHERN (deine Logik) ---
//  public static void speichereFesteWerte(ArrayList<FestWert>
festeWerte) {
//      try (FileWriter writer = new FileWriter("festeWerte.txt")) {
//          for (FestWert f : festeWerte) {
//              writer.write(String.format(Locale.US, "%s#%.2f#%s#
%s#%s#%d%n",

```

```

//          f.bezeichnung, f.betrag, f.istEinnahme, f.datum,
f.kategorie, f.periode));
//      }
//      System.out.println("Feste Werte in festeWerte.txt
gespeichert.");
//      } catch (IOException e) {
//          System.err.println("Fehler beim Speichern: " +
e.getMessage());
//      }
//  }
//
//
//
//  public static void aendereSparziel() {
//      zeigeSparziele();
//      if (ziele.isEmpty()) return;
//      int idx = IO.readInt("Nummer (0 = Abbruch): ") - 1;
//      if (idx < 0 || idx >= ziele.size()) return;
//
//      Sparziel sz = ziele.get(idx);
//      System.out.println("Aktuell: " + sz);
//
//      // Name
//      String n = IO.readString("Name (" + sz.name + ", Enter =
beibehalten): ");
//      if (!n.isEmpty()) sz.name = n;
//
//      // Zielbetrag
//      double b = IO.readDouble("Zielbetrag (" + sz.zielbetrag + ",
0 = beibehalten): ");
//      if (b > 0) sz.zielbetrag = b;
//
//      // Datum – NUR ÄNDERN, WENN ETWAS EINGETIPPT WURDE
//      LocalDate neuesDatum = IO.readDate("Datum (" + sz.zielDatum
+ "): ");
//      if (neuesDatum != null) {
//          sz.zielDatum = neuesDatum;
//      }
//
//      // Priorität
//      int p = IO.readInt("Priorität (" + sz.prioritaet + ", 0 =
beibehalten): ");
//      if (p > 0) sz.prioritaet = p;
//
//      speichereSparziele();
//      System.out.println("Sparziel aktualisiert.");
//  }
//
//  // --- ANZEIGEN (KORREKT NUMMERIERT) ---
//  public static void zeigeSparziele() {
//      if (ziele.isEmpty()) {

```

```

//          System.out.println("Keine Sparziele.");
//          return;
//      }
//      System.out.println("=== SPARZIELE (Prio 1 = höchste) ===");
//      List<Sparziel> sortiert = ziele.stream()
//          .sorted(Comparator.comparingInt(s -> s.prioritaet))
//          .toList();
//      for (int i = 0; i < sortiert.size(); i++) {
//          System.out.println((i + 1) + ": " + sortiert.get(i));
//      }
//  }
//
//  // --- LÖSCHEN ---
//  public static void loescheSparziel() {
//      zeigeSparziele();
//      if (ziele.isEmpty()) return;
//      int idx = IO.readInt("Löschen (0 = Abbruch): ") - 1;
//      if (idx >= 0 && idx < ziele.size()) {
//          ziele.remove(idx);
//          speichereSparziele();
//      }
//  }
//}
//

```

```

import java.io.*;
import java.time.LocalDate;
import java.util.*;
import java.util.stream.Collectors;

```

```

public class SparzielManager {
    static final List<Sparziel> ziele = new ArrayList<>();
    private static final String DATEI = "sparziel.txt";
    private static LocalDate letzteZufuehrung = null;
    private static final String LETZTE_ZUFUEHRUNG_KEY =
"letzteZufuehrung";

    //
=====
==
    // 1. SPARRATE AUS FESTEN WERTEN LESEN (NUR EIN EINTRAG!)
    //
=====
==
    public static double getSparrateAusFesteWerte(ArrayList<FestWert>
festeWerte) {
        return festeWerte.stream()

```



```

        .filter(f -> !f.istEinnahme
                    && "Sparen".equals(f.kategorie)
                    && "Sparplan".equals(f.bezeichnung)
                    && f.periode == 1)
        .mapToDouble(f -> f.betrag)
        .findFirst()
        .orElse(0.0);
    }

    //
=====
==
    // 2. SPARZIEL KLASSE (mit Fortschrittsbalken)
    //
=====
==
    public static class Sparziel {
        String name;
        double zielbetrag;
        LocalDate zielDatum;
        double aktuell;
        int prioritaaet;

        public Sparziel(String name, double zielbetrag, LocalDate
zielDatum, double aktuell, int prioritaaet) {
            this.name = name;
            this.zielbetrag = zielbetrag;
            this.zielDatum = zielDatum;
            this.aktuell = aktuell;
            this.prioritaaet = prioritaaet;
        }

        public double fehlend() { return Math.max(0, zielbetrag -
aktuell); }
        public double prozent() { return zielbetrag > 0 ? (aktuell /
zielbetrag) * 100 : 0; }

        @Override
        public String toString() {
            int balken = (int) (prozent() / 10);
            return String.format("P%d: %-15s | %6.0f / %6.0f € | [%s
%s] %3.0f%% | %6.0f € fehlen",
                                prioritaaet, name, aktuell, zielbetrag,
                                "█".repeat(balken), " ".repeat(10 - balken),
                                prozent(), fehlend());
        }
    }

    //
=====

```

```

==
    // 3. LADEN AUS DATEI
    //
=====
==
    public static void ladeSparziele() {
        ziele.clear();
        File file = new File(DATEI);
        if (!file.exists()) return;

        try (BufferedReader br = new BufferedReader(new
FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty() || line.startsWith("#")) continue;

                if (line.startsWith(LETZTE_ZUFUEHRUNG_KEY + "#")) {
                    String datumStr =
line.substring(LETZTE_ZUFUEHRUNG_KEY.length() + 1).trim();
                    if (!datumStr.isEmpty()) {
                        letzteZufuehrung = LocalDate.parse(datumStr);
                    }
                    continue;
                }

                String[] p = line.split("#", 5);
                if (p.length == 5) {
                    ziele.add(new Sparziel(
                        p[0].trim(),
                        Double.parseDouble(p[1].trim().replace(',', '
'.'))),
                        LocalDate.parse(p[2].trim()),
                        Double.parseDouble(p[3].trim().replace(',', '
'.'))),
                        Integer.parseInt(p[4].trim())
                    ));
                }
            }
            System.out.println("Sparziele geladen: " + ziele.size());
        } catch (Exception e) {
            System.err.println("Fehler beim Laden der Sparziele: " +
e.getMessage());
        }
    }

    //
=====
==
    // 4. SPEICHERN IN DATEI

```

```

//
=====
==
    public static void speichereSparziele() {
        try (PrintWriter pw = new PrintWriter(new FileWriter(DATEI)))
        {
            pw.println("# Name#Ziel#Datum#Aktuell#Priorität");
            ziele.stream()
                .sorted(Comparator.comparingInt(s -> s.prioritaet))
                .forEach(sz -> pw.printf("%s#%.2f#%.2f#%.2f#%d\n",
                    sz.name, sz.zielbetrag, sz.zielDatum, sz.aktuell,
sz.prioritaet));

            if (letzteZufuehrung != null) {
                pw.println(LETZTE_ZUFUEHRUNG_KEY + "#" +
letzteZufuehrung);
            }
        } catch (IOException e) {
            System.err.println("Fehler beim Speichern der Sparziele: "
+ e.getMessage());
        }
    }

//
=====
==
    // 5. NEUES SPARZIEL
    //
=====
==
    public static void neuesSparziel() {
        String name = IO.readString("Name: ");
        double betrag = IO.readDouble("Zielbetrag: ");
        LocalDate datum = IO.readDate("Ziel-Datum (JJJJ-MM-TT): ");
        int prio = IO.readInt("Priorität (1 = höchste): ");
        ziele.add(new Sparziel(name, betrag, datum, 0.0, prio));
        speichereSparziele();
        System.out.println("Sparziel hinzugefügt.");
    }

//
=====
==
    // 6. MONATLICHE ZUFÜHRUNG (PRIORISIERT!)
    //
=====
==
    public static void monatlicheZufuehrung(ArrayList<FestWert>
festeWerte) {
        double sparrate = getSparrateAusFesteWerte(festeWerte);

```

```

        if (sparrate <= 0 || ziele.isEmpty()) {
            System.out.println("Keine Sparrate oder keine
Sparziele.");
            return;
        }

        List<Sparziel> sortiert = new ArrayList<>(ziele);
        sortiert.sort(Comparator.comparingInt(s -> s.prioritaet));

        double rest = sparrate;
        for (Sparziel sz : sortiert) {
            if (rest <= 0) break;
            double fehlend = sz.fehlend();
            if (fehlend > 0) {
                double zuweisung = Math.min(rest, fehlend);
                sz.aktuell += zuweisung;
                rest -= zuweisung;
            }
        }

        speichereSparziele();
        System.out.printf("Monatliche Sparrate: %.2f € verteilt.\n",
sparrate);
    }

```

```
//
```

```
=====
==
```

```
// 7. AUTOMATISCHE ZUFÜHRUNG BEIM PROGRAMMSTART
```

```
//
```

```
=====
==
```

```

        public static void
        automatischeZufuehrungBeimStart(ArrayList<FestWert> festeWerte) {
            LocalDate heute = LocalDate.now();
            LocalDate monatsBeginn = heute.withDayOfMonth(1);

            if (letzteZufuehrung == null || !
letzteZufuehrung.equals(monatsBeginn)) {
                monatlicheZufuehrung(festeWerte);
                letzteZufuehrung = monatsBeginn;
                speichereSparziele(); // WICHTIG: speichert neue
letzteZufuehrung
            }
        }
    }

```

```
//
```

```
=====
==
```

```
// 8. SPARRATE ÄNDERN (ersetzt alten "Sparplan")
```

```

//
=====
==
    public static void setzeSparrate(ArrayList<FestWert> festeWerte) {
        double aktuell = getSparrateAusFesteWerte(festeWerte);
        double neu = IO.readDouble("Neue monatliche Sparrate (aktuell:
" + String.format(Locale.US, "%.2f", aktuell) + " €): ");
        if (neu < 0) return;

        // Alten Sparplan entfernen
        festeWerte.removeIf(f -> !f.istEinnahme
                            && "Sparen".equals(f.kategorie)
                            && "Sparplan".equals(f.bezeichnung));

        // Neuen hinzufügen
        if (neu > 0) {
            LocalDate datum = LocalDate.now().withDayOfMonth(1);
            festeWerte.add(new FestWert("Sparplan", neu, false, datum,
"Sparen", 1));
        }

        // Sofort speichern!
        speichereFesteWerte(festeWerte);
        System.out.println("Sparrate aktualisiert: " +
String.format(Locale.US, "%.2f", neu) + " €/Monat");
    }

//
=====
==
    // 9. FESTEWERTE SPEICHERN (wird von setzeSparrate() genutzt)
    //
=====
==
    public static void speichereFesteWerte(ArrayList<FestWert>
festeWerte) {
        try (FileWriter writer = new FileWriter("festeWerte.txt")) {
            for (FestWert f : festeWerte) {
                writer.write(String.format(Locale.US, "%s#%.2f#%s#%s#
%s#%d%n",
                                f.bezeichnung, f.betrag, f.istEinnahme, f.datum,
f.kategorie, f.periode));
            }
            System.out.println("Feste Werte gespeichert (inkl.
Sparrate).");
        } catch (IOException e) {
            System.err.println("Fehler beim Speichern der festen
Werte: " + e.getMessage());
        }
    }

```

```

//
=====
==
// 10. SPARZIEL ÄNDERN
//
=====
==
    public static void aendereSparziel() {
        zeigeSparziele();
        if (ziele.isEmpty()) return;

        int idx = IO.readInt("Nummer ändern (0 = Abbruch): ") - 1;
        if (idx < 0 || idx >= ziele.size()) return;

        Sparziel sz = ziele.get(idx);
        System.out.println("Aktuell: " + sz);

        String n = IO.readString("Name (" + sz.name + ", Enter =
beibehalten): ");
        if (!n.isEmpty()) sz.name = n;

        double b = IO.readDouble("Zielbetrag (" + sz.zielbetrag + ", 0
= beibehalten): ");
        if (b > 0) sz.zielbetrag = b;

        LocalDate d = IO.readDate("Ziel-Datum (" + sz.zielDatum + "):
");
        if (d != null) sz.zielDatum = d;

        int p = IO.readInt("Priorität (" + sz.prioritaet + ", 0 =
beibehalten): ");
        if (p > 0) sz.prioritaet = p;

        speichereSparziele();
        System.out.println("Sparziel aktualisiert.");
    }

//
=====
==
// 11. SPARZIELE ANZEIGEN
//
=====
==
    public static void zeigeSparziele() {
        if (ziele.isEmpty()) {
            System.out.println("Keine Sparziele vorhanden.");
            return;
        }
    }

```

```

    }
    System.out.println("=== SPARZIELE (Prio 1 = höchste) ===");
    List<Sparziel> sortiert = ziele.stream()
        .sorted(Comparator.comparingInt(s -> s.prioritaet))
        .collect(Collectors.toList());
    for (int i = 0; i < sortiert.size(); i++) {
        System.out.println((i + 1) + ": " + sortiert.get(i));
    }
}

//
=====
==
// 12. SPARZIEL LÖSCHEN
//
=====
==
    public static void loescheSparziel() {
        zeigeSparziele();
        if (ziele.isEmpty()) return;

        int idx = IO.readInt("Löschen (0 = Abbruch): ") - 1;
        if (idx >= 0 && idx < ziele.size()) {
            ziele.remove(idx);
            speichereSparziele();
            System.out.println("Sparziel gelöscht.");
        }
    }
}

```

Klasse: Statistik

```
package gui;
```

```
/**
 * TODO: boolean array mit verschiedenen bedingungen und methoden
 vereinheitlichen!
 */
```

```
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
```

```
public class Statistik {
```

```
// public static boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme
```

```

&& e.datum.plusMonths(1).isAfter(LocalDate.now()))};

// public static double berechneMittelwert(ArrayList<Eintrag>
eintraege ) {
//     double mittelwert=0.0;
//     double summe=0.0;
//     int count=0;
//     for (Eintrag e :eintraege) {
//         if (!e.istEinnahme) {
//             summe+=e.getBetrag();
//             count++;
//         }
//     }
//     mittelwert=summe/count;
//     return mittelwert;
// }
//
// public static void printMittelwert(ArrayList<Eintrag> eintraege) {
//     double a=berechneMittelwert(eintraege);
//     System.out.println("=====");
//     System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);
//     System.out.println("=====");
// }
//
// public static void berechneStandardabweichung(ArrayList<Eintrag>
eintraege ) {
//     double standardabweichung=0.0;
//     double temp=0.0;
//     double mittelwert=berechneMittelwert(eintraege);
//     int count=0;
//     for (Eintrag e:eintraege) {
//         if (!e.istEinnahme) {
//             temp+=Math.pow(e.getBetrag()-mittelwert,2 );
//             count++;
//         }
//     }
//     if (count>1) {
//         standardabweichung=Math.sqrt(temp/(count-1));
//     }
//     System.out.println("=====");
//     System.out.printf("Standardabweichung der Ausgaben: %13.2f€\n",standardabweichung);
//     System.out.println("=====");
// }

```



```

//      else {
//          System.out.println("Eine Standardabweichung macht keinen
Sinn bei der Menge an Daten!");
//      }
//  }
//
//
//  public static void berechneMitKategorie(ArrayList<Eintrag>
eintraege){
//
//
//      Set<String> kategorienSet = new HashSet<>();
//      for (Eintrag e : eintraege) {
//          if (!e.istEinnahme) {
//              kategorienSet.add(e.getKategorie());
//          }
//      }
//
//      List<String> kategorienListe = new ArrayList<>(kategorienSet);
//      Collections.sort(kategorienListe);
//
//      System.out.println("Wähle die gewünschten Kategorien aus:");
//      for (int i = 0; i < kategorienListe.size(); i++) {
//          System.out.println((i + 1) + ": " +
kategorienListe.get(i));
//      }
//      System.out.println("0: Auswahl beenden");
//
//      Set<String> ausgewaehlteKategorien = new HashSet<>();
//      while (true) {
//          int auswahl = IO.readInt("Nummer eingeben: ");
//          if (auswahl == 0) break;
//          if (auswahl > 0 && auswahl <= kategorienListe.size()) {
//              ausgewaehlteKategorien.add(kategorienListe.get(auswahl -
1));
//          } else {
//              System.out.println("Ungültige Auswahl!");
//          }
//      }
//      System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);
//
//
//
//
//      ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
//      for (Eintrag e:eintraege) {
//          if(!e.istEinnahme) {
//              if (ausgewaehlteKategorien.contains(e.kategorie)) {
//                  auswahlbeträge.add(e.betrag);

```

```

//          }
//      }
//  }
//
//      double mit=0.0;
//      double sum=0.0;
//      double var=0.0;
//      double stdabweichung=0.0;
//      for (double a:auswahlbeträge) {
//          sum+=a;
//      }
//      mit=sum/auswahlbeträge.size();
//      if (auswahlbeträge.size()>1) {
//          for (double a:auswahlbeträge) {
//              var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
//          }
//      }
//      else {
//          var=0.0;
//      }
//      stdabweichung=Math.sqrt(var);
//
System.out.println("=====");
;
//      System.out.println("In den von Ihnen ausgewählten
Kategorien sind:");
//      System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
//      System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);
//
System.out.println("=====");
;
//
//
//
//  }
//  public static double berechneMittelwertMonat(ArrayList<Eintrag>
einträge ) {
//      double mittelwert=0.0;
//      double summe=0.0;
//      int count=0;
//      for (Eintrag e :einträge) {
//          if (!e.istEinnahme) {
//schonmal was von && gehört???
//              if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//                  summe+=e.getBetrag();
//                  count++;
//              }
//          }
//      }

```

```

//      }
//    }
//    mittelwert=summe/count;
//
//    return mittelwert;
//
//  }
//
//  public static void printMittelwertMonat(ArrayList<Eintrag>
eintraege) {
//    double a=berechneMittelwert(eintraege);
//
System.out.println("=====");
//    System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);
//
System.out.println("=====");
//  }
//
//  public static void
berechneStandardabweichungMonat(ArrayList<Eintrag> eintraege ) {
//    double standardabweichung=0.0;
//    double temp=0.0;
//    double mittelwert=berechneMittelwert(eintraege);
//    int count=0;
//    for (Eintrag e:eintraege) {
//      if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//schonmal was von && gehört???
//        if (!e.istEinnahme) {
//          temp+=Math.pow(e.getBetrag()-mittelwert,2 );
//          count++;
//        }
//      }
//    }
//    if (count>1) {
//      standardabweichung=Math.sqrt(temp/(count-1));
//
System.out.println("=====");
//    System.out.printf("Standardabweichung der Ausgaben: %13.2f€\n",standardabweichung);
//
System.out.println("=====");
//
//    }
//    else {
//      System.out.println("Eine Standardabweichung macht keinen
Sinn bei der Menge an Daten!");
//    }
//  }
//
//

```

```

// public static void berechneMitKategorieMonat(ArrayList<Eintrag>
eintraege){
//
//
//      Set<String> kategorienSet = new HashSet<>();
//      for (Eintrag e : eintraege) {
//          if (!e.istEinnahme) {
//schonmal was von && gehört???
//              if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//                  kategorienSet.add(e.getKategorie());
//              }
//          }
//      }
//
//      List<String> kategorienListe = new ArrayList<>(kategorienSet);
//      Collections.sort(kategorienListe);
//
//      System.out.println("Wähle die gewünschten Kategorien aus:");
//      for (int i = 0; i < kategorienListe.size(); i++) {
//          System.out.println((i + 1) + ": " +
kategorienListe.get(i));
//      }
//      System.out.println("0: Auswahl beenden");
//
//      Set<String> ausgewaehlteKategorien = new HashSet<>();
//      while (true) {
//          int auswahl = IO.readInt("Nummer eingeben: ");
//          if (auswahl == 0) break;
//          if (auswahl > 0 && auswahl <= kategorienListe.size()) {
//              ausgewaehlteKategorien.add(kategorienListe.get(auswahl -
1));
//          } else {
//              System.out.println("Ungültige Auswahl!");
//          }
//      }
//      System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);
//
//
//
//
//      ArrayList<Double> auswahlbeträge=new ArrayList<Double>();
//      for (Eintrag e: eintraege) {
//          if(e.datum.plusMonths(1).isAfter(LocalDate.now())) {
//              if(!e.istEinnahme) {
//                  if (ausgewaehlteKategorien.contains(e.kategorie))
//
//                      auswahlbeträge.add(e.betrag);
//              }
//          }
//      }

```

```

//      }
//    }
//
//    double mit=0.0;
//    double sum=0.0;
//    double var=0.0;
//    double stdabweichung=0.0;
//    for (double a:auswahlbeträge) {
//        sum+=a;
//    }
//    mit=sum/auswahlbeträge.size();
//    if (auswahlbeträge.size()>1) {
//        for (double a:auswahlbeträge) {
//            var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
//        }
//    }
//    else {
//        var=0.0;
//    }
//    stdabweichung=Math.sqrt(var);
//
System.out.println("=====");
;
//    System.out.println("In den von Ihnen ausgewählten
Kategorien sind:");
//    System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
//    System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);
//
System.out.println("=====");
;
//
//
//
// }
//}
//

```

```

//public static boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme
&& e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};

```

```

public static double berechneMittelwert(ArrayList<Eintrag>
eintraege ,int b) {
    double mittelwert=0.0;
    double summe=0.0;
    int count=0;

```

```

        for (Eintrag e :eintraege) {
            boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
            if (bedingung[b]) {
                summe+=e.getBetrag();
                count++;
            }
        }
        mittelwert=summe/count;

        return mittelwert;
    }

    public static void printMittelwert(ArrayList<Eintrag> eintraege, int
b) {
        double a=berechneMittelwert(eintraege,b);

        System.out.println("=====");
        System.out.printf("Mittelwert der Ausgaben: %21.2f€\n",a);

        System.out.println("=====");
    }

    public static void berechneStandardabweichung(ArrayList<Eintrag>
eintraege, int b ) {
        double standardabweichung=0.0;
        double temp=0.0;
        double mittelwert=berechneMittelwert(eintraege,b);
        int count=0;
        for (Eintrag e:eintraege) {
            boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
            if (bedingung[b]) {
                temp+=Math.pow(e.getBetrag()-mittelwert,2 );
                count++;
            }
        }
        if (count>1) {
            standardabweichung=Math.sqrt(temp/(count-1));

            System.out.println("=====");
            System.out.printf("Standardabweichung der Ausgaben: %13.2f€\
n",standardabweichung);

            System.out.println("=====");

```

```

    }
    else {
        System.out.println("Eine Standardabweichung macht keinen Sinn
bei der Menge an Daten!");
    }
}

```

```

public static void berechneMitKategorie(ArrayList<Eintrag> eintraege,
int b){

```

```

    Set<String> kategorienSet = new HashSet<>();
    for (Eintrag e : eintraege) {
        boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
        if (bedingung[b]) {
            kategorienSet.add(e.getKategorie());
        }
    }

    List<String> kategorienListe = new ArrayList<>(kategorienSet);
    Collections.sort(kategorienListe);

    System.out.println("Wähle die gewünschten Kategorien aus:");
    for (int i = 0; i < kategorienListe.size(); i++) {
        System.out.println((i + 1) + ": " + kategorienListe.get(i));
    }
    System.out.println("0: Auswahl beenden");

    Set<String> ausgewaehlteKategorien = new HashSet<>();
    while (true) {
        int auswahl = IO.readInt("Nummer eingeben: ");
        if (auswahl == 0) break;
        if (auswahl > 0 && auswahl <= kategorienListe.size()) {
            ausgewaehlteKategorien.add(kategorienListe.get(auswahl - 1));
        } else {
            System.out.println("Ungültige Auswahl!");
        }
    }
    System.out.println("Gewählte Kategorien: " +
ausgewaehlteKategorien);

```

```

    ArrayList<Double> auswahlbeträge=new ArrayList<Double>();

```

```

        for (Eintrag e:eintraege) {
            boolean[] bedingung= {!e.istEinnahme ,!e.istEinnahme &&
e.datum.plusMonths(1).isAfter(LocalDate.now()),!e.istEinnahme &&
e.datum.plusMonths(2).isAfter(LocalDate.now()) && !
e.datum.plusMonths(1).isAfter(LocalDate.now())};
            if(bedingung[b]) {
                if (ausgewaehlteKategorien.contains(e.kategorie)) {
                    auswahlbeträge.add(e.betrag);
                }
            }
        }

        double mit=0.0;
        double sum=0.0;
        double var=0.0;
        double stdabweichung=0.0;
        for (double a:auswahlbeträge) {
            sum+=a;
        }
        mit=sum/auswahlbeträge.size();
        if (auswahlbeträge.size()>1) {
            for (double a:auswahlbeträge) {
                var+=Math.pow(a-mit,2)/(auswahlbeträge.size()-1);
            }
        }
        else {
            var=0.0;
        }
        stdabweichung=Math.sqrt(var);

System.out.println("=====");
;
        System.out.println("In den von Ihnen ausgewählten Kategorien
sind:");
        System.out.printf("Mittelwert der Ausgaben: %22.2f€\n",mit);
        System.out.printf("Standardabweichung der Ausgaben: %14.2f€\n",stdabweichung);

System.out.println("=====");
;

    }

}

```


Klasse: window_builder

```
package gui;  
  
public class window_builder {  
  
}
```

Package: interfaces

Klasse: Insurable

```
package interfaces;  
  
public interface Insurable {  
  
    double calculateInsurance();  
  
}
```

Klasse: Insurable

```
package interfaces;  
  
public interface Insurable {  
  
    double calculateInsurance();  
  
}
```

Klasse: PricingStrategy

```
package interfaces;  
  
public interface PricingStrategy {  
  
    double calculatePrice();  
  
}
```

Klasse: PricingStrategy

```
package interfaces;  
  
public interface PricingStrategy {  
  
    double calculatePrice();  
  
}
```

Klasse: Trackable

```
package interfaces;

public interface Trackable {

    String getTrackingCode();

}
```

Klasse: Trackable

```
package interfaces;

public interface Trackable {

    String getTrackingCode();

}
```

Package: io

Klasse: ShipmentCsvReader2

```
package io;

import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import exceptions.InvalidShipmentDataException;
import exceptions.UnknownShipmentTypeException;
import model.Shipment;
import util.ShipmentFactory;

public class ShipmentCsvReader2 {

    public static List<Shipment> readShipmentsFromCSV(String pfad)
        throws UnknownShipmentTypeException,
        InvalidShipmentDataException {
        List<Shipment> result = new ArrayList<Shipment>();
        BufferedReader br = null;
        try {
            br = new BufferedReader(new FileReader(pfad));

            String line;
            String type = null;
            double weight = 0;
            double distance = 0;
            double warenwert = 0;
```

```

        int i = 0;
        while (br.readLine() != null && ++i < 3)
            ;

        while ((line = br.readLine()) != null) {
            if (line.isEmpty() || line.isBlank())
                continue;

            String[] values = line.trim().split(",");
            type = values[0].trim();

            weight = convertToDouble(values[1]);
            distance = convertToDouble(values[2]);
            warenwert = convertToDouble(values[3]);

            result.add(ShipmentFactory.createShipment(type,
weight, distance, warenwert));

        }

        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (IOException e) {
            System.out.println(e.getMessage());
        } finally {
            try {
                br.close();
            } catch (IOException e) {
                System.out.println(e.getMessage());
            }
        }
    }

    return result;
}

private static double convertToDouble(String s) {

    if (s != null && !s.isEmpty()) {
        try {
            return Double.parseDouble(s.trim());
        } catch (NumberFormatException e) {
            throw e;
        }
    }

    return 0;
}
}

```

Klasse: ShipmentCsvReader2

```
package io;

import java.io.BufferedReader;
import java.io.FileNotFoundException;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import exceptions.InvalidShipmentDataException;
import exceptions.UnknownShipmentTypeException;
import model.Shipment;
import util.ShipmentFactory;

public class ShipmentCsvReader2 {

    public static List<Shipment> readShipmentsFromCSV(String pfad)
        throws UnknownShipmentTypeException,
        InvalidShipmentDataException {
        List<Shipment> result = new ArrayList<Shipment>();
        BufferedReader br = null;
        try {
            br = new BufferedReader(new FileReader(pfad));

            String line;
            String type = null;
            double weight = 0;
            double distance = 0;
            double warenwert = 0;

            int i = 0;
            while (br.readLine() != null && ++i < 3)
                ;

            while ((line = br.readLine()) != null) {
                if (line.isEmpty() || line.isBlank())
                    continue;

                String[] values = line.trim().split(",");
                type = values[0].trim();

                weight = convertToDouble(values[1]);
                distance = convertToDouble(values[2]);
                warenwert = convertToDouble(values[3]);

                result.add(ShipmentFactory.createShipment(type,
weight, distance, warenwert));
            }
        }
    }
}
```

```

        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (IOException e) {
            System.out.println(e.getMessage());
        } finally {
            try {
                br.close();
            } catch (IOException e) {
                System.out.println(e.getMessage());
            }
        }

        return result;
    }

    private static double convertToDouble(String s) {

        if (s != null && !s.isEmpty()) {
            try {
                return Double.parseDouble(s.trim());
            } catch (NumberFormatException e) {
                throw e;
            }
        }

        return 0;
    }
}

```

Klasse: ShipmentReportWriter

```

package io;

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import exceptions.UnknownShipmentTypeException;
import model.ExpressParcel;
import model.Freight;
import model.Shipment;

public class ShipmentReportWriter {

    public static void writeReport(List<Shipment> shipments, String
fileName)

```

```

        throws UnknownShipmentTypeException, IOException {

        LocalDateTime timestamp = LocalDateTime.now();
        File f = new File(fileName);
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-
MM-yyyy HH:mm:ss");
        String timestampToString = timestamp.format(formatter);
        BufferedWriter br = null;
        try {
            boolean success = f.createNewFile();
            if (success)
                System.out.println("Datei wurde erzeugt");
            else
                System.out.println("Datei existiert bereits");
            br = new BufferedWriter(new FileWriter(f, false));

            br.write("=".repeat(50));
            br.newLine();
            br.write(timestampToString);
            br.newLine();
            br.write("type,weight,distance,goodsValue");

            for (Shipment shipment : shipments) {
                br.newLine();

                if
(shipment.getClass().getSimpleName().toLowerCase().equals("standardpar
cel"))
                    br.write("standard" + "," + shipment.getWeightKg()
+ "," + shipment.getDistanceKm() + "," + "0");
                else if
(shipment.getClass().getSimpleName().toLowerCase().equals("expressparc
el"))
                    br.write("express" + "," + shipment.getWeightKg()
+ "," + shipment.getDistanceKm() + "," +
                    + ((ExpressParcel)
shipment).getWarenwert());
                else if
(shipment.getClass().getSimpleName().toLowerCase().equals("freight"))
                    br.write("freight" + "," + shipment.getWeightKg()
+ "," + shipment.getDistanceKm() + "," +
                    + ((Freight) shipment).getWarenwert());
                else
                    throw new UnknownShipmentTypeException("kein
gültiges Shipment");
            }

        } catch (IOException e) {
            String output = e.getMessage();
            System.out.println(output);
        }
    }
}

```

```

        } finally {
            br.close();
        }
    }
}

```

Klasse: ShipmentReportWriter

```

package io;

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.List;
import exceptions.UnknownShipmentTypeException;
import model.ExpressParcel;
import model.Freight;
import model.Shipment;

public class ShipmentReportWriter {

    public static void writeReport(List<Shipment> shipments, String
fileName)
        throws UnknownShipmentTypeException, IOException {

        LocalDateTime timestamp = LocalDateTime.now();
        File f = new File(fileName);
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-
MM-yyyy HH:mm:ss");
        String timestampToString = timestamp.format(formatter);
        BufferedWriter br = null;
        try {
            boolean success = f.createNewFile();
            if (success)
                System.out.println("Datei wurde erzeugt");
            else
                System.out.println("Datei existiert bereits");
            br = new BufferedWriter(new FileWriter(f, false));

            br.write("=".repeat(50));
            br.newLine();
            br.write(timestampToString);
            br.newLine();
            br.write("type,weight,distance,goodsValue");

```

```

        for (Shipment shipment : shipments) {
            br.newLine();

            if
            (shipment.getClass().getSimpleName().toLowerCase().equals("standardpar
            cel"))
                br.write("standard" + "," + shipment.getWeightKg()
            + "," + shipment.getDistanceKm() + "," + "0");
            else if
            (shipment.getClass().getSimpleName().toLowerCase().equals("expressparc
            el"))
                br.write("express" + "," + shipment.getWeightKg()
            + "," + shipment.getDistanceKm() + "," +
                + ((ExpressParcel)
            shipment).getWarenwert());
            else if
            (shipment.getClass().getSimpleName().toLowerCase().equals("freight"))
                br.write("freight" + "," + shipment.getWeightKg()
            + "," + shipment.getDistanceKm() + "," +
                + ((Freight) shipment).getWarenwert());
            else
                throw new UnknownShipmentTypeException("kein
            gültiges Shipment");
        }

        } catch (IOException e) {
            String output = e.getMessage();
            System.out.println(output);
        } finally {
            br.close();
        }
    }
}

```

Package: main

Klasse: Main

```
package main;
```

```
import controller.Controller;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        /* TODO Auto-generated method stub
```

Diese Methode wird hinterher ausgeführt und erzeugt eine Controller-Instanz


```

*/
        new Controller();
    }
}

```

Package: mi

Klasse: teschtle

```

package mi;

public class teschtle {

    public static void main(String[] args) {
        Gewinnmatrix matrix = new Gewinnmatrix();
        matrix.ausgabe();
        double[] minMax = matrix.findeMinMax();
        System.out.printf("\n\tMinimum: %8.2f €\n\tMaximum: %8.2f €\n",
minMax[0], minMax[1]);
        double[] durchschnitt = matrix.durchschnittAbteilungen();
        System.out.printf(" Durchschnitt:\n");
        for (int i = 0; i < durchschnitt.length; i++) {
            System.out.printf("%15s: %8.2f €\n", matrix.abteilungen[i],
durchschnitt[i]);
        }
    }

}

```

Package: model

Klasse: ExpressParcel

```

package model;

import exceptions.InvalidShipmentDataException;
import interfaces.Insurable;
import interfaces.Trackable;

public class ExpressParcel extends Shipment implements Trackable,
Insurable {

    private final double warenwert;
    public static final double GRUNDPREIS=9.99;

    public ExpressParcel(double weightKg, double distanceKm, double
warenwert) throws InvalidShipmentDataException {
        super(weightKg, distanceKm);
        if (warenwert>=0) {

```

```

        this.warenwert = warenwert;
    }
    else {
        throw new InvalidShipmentDataException("Wert muss positiv
sein");
    }
}

@Override
public String toString() {
    return String.format("ExpressParcel:\nWarenwert: %28.2f€\
nVersicherungskosten: %18.2f€\n"
        + "Shipping-Kosten: %22.2f€\nTrackingcode: %26s\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost(),getTra
ckingCode());
}

@Override
public double calculateInsurance() {
    return Math.max(warenwert*0.01,2.5);
}

@Override
public String getTrackingCode() {
    return hashCode()+ "";
}

@Override
public double shippingCost() {
    return GRUNDPREIS+1.1*getWeightKg()+0.02*getDistanceKm();
}

public double getWarenwert() {
    return warenwert;
}

}

```

Klasse: ExpressParcel

```
package model;
```

```
import exceptions.InvalidShipmentDataException;
import interfaces.Insurable;
import interfaces.Trackable;
```

```
public class ExpressParcel extends Shipment implements Trackable,
Insurable {
```

```

        private final double warenwert;
        public static final double GRUNDPREIS=9.99;

        public ExpressParcel(double weightKg, double distanceKm, double
warenwert) throws InvalidShipmentDataException {
            super(weightKg, distanceKm);
            if (warenwert>=0) {
                this.warenwert = warenwert;
            }
            else {
                throw new InvalidShipmentDataException("Wert muss positiv
sein");
            }
        }

        @Override
        public String toString() {
            return String.format("ExpressParcel:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
                + "Shipping-Kosten: %22.2f€\nTrackingcode: %26s\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost(),getTra
ckingCode());
        }

        @Override
        public double calculateInsurance() {
            return Math.max(warenwert*0.01,2.5);
        }

        @Override
        public String getTrackingCode() {
            return hashCode()+" ";
        }

        @Override
        public double shippingCost() {
            return GRUNDPREIS+1.1*getWeightKg()+0.02*getDistanceKm();
        }

        public double getWarenwert() {
            return warenwert;
        }

    }

```

Klasse: Freight

package model;

```

import exceptions.InvalidShipmentDataException;
import interfaces.Insurable;

public class Freight extends Shipment implements Insurable {
    private double warenwert;
    public static final double GRUNDPREIS=49;
    public Freight(double weightKg, double dinstanceKm, double
warenwert) throws InvalidShipmentDataException {
        super(weightKg, dinstanceKm);
        if (warenwert>=0) {
            this.warenwert = warenwert;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
sein");
        }
    }

    @Override
    public String toString() {
        return String.format("Freight:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
            + "Shipping-Kosten: %22.2f€\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost());
    }

    public double getWarenwert() {
        return warenwert;
    }

    public void setWarenwert(double warenwert) {
        this.warenwert = warenwert;
    }

    @Override
    public double calculateInsurance() {
        return Math.max(warenwert*0.005, 10);
    }

    @Override
    public double shippingCost() {
        return GRUNDPREIS+0.35*getWeightKg()+0.015*getDistanceKm();
    }
}

```

Klasse: Freight

```
package model;

import exceptions.InvalidShipmentDataException;
import interfaces.Insurable;

public class Freight extends Shipment implements Insurable {
    private double warenwert;
    public static final double GRUNDPREIS=49;
    public Freight(double weightKg, double dinstanceKm, double
warenwert) throws InvalidShipmentDataException {
        super(weightKg, dinstanceKm);
        if (warenwert>=0) {
            this.warenwert = warenwert;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
sein");
        }
    }

    @Override
    public String toString() {
        return String.format("Freight:\nWarenwert: %28.2f€\n
nVersicherungskosten: %18.2f€\n"
            + "Shipping-Kosten: %22.2f€\n"
+super.toString(),warenwert,calculateInsurance(),shippingCost());
    }

    public double getWarenwert() {
        return warenwert;
    }

    public void setWarenwert(double warenwert) {
        this.warenwert = warenwert;
    }

    @Override
    public double calculateInsurance() {
        return Math.max(warenwert*0.005, 10);
    }

    @Override
    public double shippingCost() {
        return GRUNDPREIS+0.35*getWeightKg()+0.015*getDistanceKm();
    }
}
```

```
}
```

Klasse: Kunde

```
package model;
```

```
import java.util.ArrayList;
```

```
public class Kunde {
```

```
    private int id;  
    private String vorname;  
    private String nachname;  
    private String adresse;  
    private String kundenart;
```

```
    private ArrayList<Bestellung> bestellungen;
```

```
    public Kunde(int id, String vorname, String nachname) {  
        this.id=id;  
        this.vorname=vorname;  
        this.nachname=nachname;  
    }
```

```
    public int getId() {  
        return id;  
    }
```

```
    public void setId(int id) {  
        this.id = id;  
    }
```

```
    public String getVorname() {  
        return vorname;  
    }
```

```
    public void setVorname(String vorname) {  
        this.vorname = vorname;  
    }
```

```
    public String getNachname() {  
        return nachname;  
    }
```

```
    public void setNachname(String nachname) {  
        this.nachname = nachname;  
    }
```

```

    public String getAdresse() {
        return adresse;
    }

    public void setAdresse(String adresse) {
        this.adresse = adresse;
    }

    public String getKundenart() {
        return kundenart;
    }

    public void setKundenart(String kundenart) {
        this.kundenart = kundenart;
    }

    public ArrayList<Bestellung> getBestellungen() {
        return bestellungen;
    }

    public void setBestellungen(ArrayList<Bestellung> bestellungen) {
        this.bestellungen = bestellungen;
    }

    @Override
    public String toString() {
        return "Kunde [id=" + id + ", vorname=" + vorname + ",
nachname=" + nachname + ", adresse=" + adresse
        + ", kundenart=" + kundenart + ", bestellungen=" +
bestellungen + "]";
    }

}

```

Klasse: Shipment

```

package model;

import exceptions.InvalidShipmentDataException;

//import InOut.IO;

public abstract class Shipment implements Comparable<Shipment> {
    private final double weightKg;
    private final double distanceKm;

    public double getWeightKg() {

```

```

        return weightKg;
    }

    public double getDistanceKm() {
        return distanceKm;
    }

    public Shipment(double weightKg, double distanceKm) throws
InvalidShipmentDataException{
        if (weightKg >0) {
            this.weightKg=weightKg;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss positiv
sein");
        }

        if(distanceKm>0) {
            this.distanceKm = distanceKm;
        }
        else {
            throw new InvalidShipmentDataException("Wert muss
positiv sein");
        }

    }

    @Override
    public String toString() {
        return String.format("Gewicht: %29.1fkg\nEntfernung: %26.1fkm\n",weightKg,distanceKm);
    }

    public abstract double shippingCost();

    @Override
    public int compareTo(Shipment other) {
        return Double.compare(this.weightKg, other.weightKg);
    }

```



```
}
```

Klasse: Shipment

```
package model;
```

```
import exceptions.InvalidShipmentDataException;
```

```
//import InOut.IO;
```

```
public abstract class Shipment implements Comparable<Shipment> {  
    private final double weightKg;  
    private final double distanceKm;
```

```
  
    public double getWeightKg() {  
        return weightKg;  
    }
```

```
  
    public double getDistanceKm() {  
        return distanceKm;  
    }
```

```
  
    public Shipment(double weightKg, double distanceKm) throws  
InvalidShipmentDataException{  
        if (weightKg >0) {  
            this.weightKg=weightKg;  
        }  
        else {  
            throw new InvalidShipmentDataException("Wert muss positiv  
sein");  
        }  
    }
```

```
  
        if(distanceKm>0) {  
            this.distanceKm = distanceKm;  
        }  
        else {  
            throw new InvalidShipmentDataException("Wert muss  
positiv sein");  
        }  
    }
```

```
}
```

```
@Override
```

```
public String toString() {  
    return String.format("Gewicht: %29.1fkg\nEntfernung: %26.1fkm\
```

```

n",weightKg,distanceKm);
    }

    public abstract double shippingCost();

    @Override
    public int compareTo(Shipment other) {
        return Double.compare(this.weightKg, other.weightKg);
    }
}

```

```

}

```

Klasse: StandardParcel

```

package model;

import exceptions.InvalidShipmentDataException;
import interfaces.Trackable;

public class StandardParcel extends Shipment implements Trackable{
    public static final double GRUNDPREIS=4.99;
    public StandardParcel(double weightKg, double distanceKm) throws
InvalidShipmentDataException {
        super(weightKg, distanceKm);
    }

    @Override
    public String toString() {
        return String.format("StandardParcel:\nShipping-Kosten:
%22.2f€\nTrackingcode:%27s\n" +super.toString(), shippingCost(),
getTrackingCode());
    }

    @Override
    public double shippingCost() {
        return GRUNDPREIS+0.6*getWeightKg()
+Math.min(0.01*getDistanceKm(), 10);
    }

    @Override

```

```

        public String getTrackingCode() {
            return hashCode()+ "";
        }
    }
}

```

Klasse: StandardParcel

```

package model;

import exceptions.InvalidShipmentDataException;
import interfaces.Trackable;

public class StandardParcel extends Shipment implements Trackable{
    public static final double GRUNDPREIS=4.99;
    public StandardParcel(double weightKg, double distanceKm) throws
InvalidShipmentDataException {
        super(weightKg, distanceKm);
    }

    @Override
    public String toString() {
        return String.format("StandardParcel:\nShipping-Kosten:
%22.2f€\nTrackingcode:%27s\n" +super.toString(), shippingCost(),
getTrackingCode());
    }

    @Override
    public double shippingCost() {
        return GRUNDPREIS+0.6*getWeightKg()
+Math.min(0.01*getDistanceKm(), 10);
    }

    @Override
    public String getTrackingCode() {
        return hashCode()+ "";
    }
}

```

Package: montag04082025

Klasse: Konto

```

package montag04082025;

```

```

public class Konto {

    //Attribute
    private String kontonummer;
    private char kontoart;
    private double kontostand;
    private double kreditlimit;
    private boolean karte;

    //Methoden
    public void setKontonummer(String kontonummer){
        this.kontonummer=kontonummer;
    }
    public String getKontonummer() {
        return this.kontonummer;
    }
    public char getKontoart() {
        return kontoart;
    }
    public void setKontoart(char kontoart) {
        this.kontoart = kontoart;
    }
    public double getKontostand() {
        return kontostand;
    }
    public void setKontostand(double kontostand) {
        this.kontostand = kontostand;
    }
    public double getKreditlimit() {
        return kreditlimit;
    }
    public void setKreditlimit(double kreditlimit) {
        this.kreditlimit = kreditlimit;
    }
    public boolean isKarte() {
        return karte;
    }
    public void setKarte(boolean karte) {
        this.karte = karte;
    }

    public void einzahlenBetrag(double betrag){
        this.kontostand+=betrag;
//this.kontostand=this.kontostand+betrag;
        System.out.println("Eingezahlter Betrag: "+betrag);
    }

    public double abhebenBetrag(double betrag){
        if (betrag<=this.kontostand-this.kreditlimit){

```

```

        this.kontostand=this.kontostand-betrag;
        System.out.println("Abgehobener Betrag: "+ betrag);
    }
    else if(this.kontostand==this.kreditlimit){
        System.out.println("Keine Abhebung möglich!");
    }
    else{
        System.out.println("Verfügungsrahmen überschritten!");
        System.out.println("der Kontostand liegt bei: " +
this.kontostand + " das Kreditlimit ist bei: "+(-1*
this.kreditlimit));
        System.out.println("Abgehobener Betrag: "+
(this.kontostand - this.kreditlimit));
        this.kontostand = this.kreditlimit;
    }
    return betrag;
}

```

//methoden müssen public sein damit zugriff aus der main möglich

}

Klasse: Konto

```
package montag04082025;
```

```
public class Konto {
```

```
    //Attribute
```

```
    String kontonummer;
```

```
    char kontoart;
```

```
    double kontostand;
```

```
    double kreditlimit;
```

```
    boolean karte;
```

```
    //Methoden
```

```
    public void setKontonummer(String kontonummer){
```

```
        this.kontonummer=kontonummer;
```

```
    }
```

```
    public String getKontonummer() {
```

```
        return this.kontonummer;
```

```
    }
```

```
    public char getKontoart() {
```

```
        return kontoart;
```

```
    }
```

```
    public void setKontoart(char kontoart) {
```

```
        this.kontoart = kontoart;
```

```
    }
```

```
    public double getKontostand() {
```

```

        return kontostand;
    }
    public void setKontostand(double kontostand) {
        this.kontostand = kontostand;
    }
    public double getKreditlimit() {
        return kreditlimit;
    }
    public void setKreditlimit(double kreditlimit) {
        this.kreditlimit = kreditlimit;
    }
    public boolean isKarte() {
        return karte;
    }
    public void setKarte(boolean karte) {
        this.karte = karte;
    }
}

    void einzahlenBetrag(double betrag){
        this.kontostand+=betrag;
//this.kontostand=this.kontostand+betrag;
        System.out.println("Eingezahlter Betrag: "+betrag);
    }

    double abhebenBetrag(double betrag){
        if (betrag<=this.kontostand-this.kreditlimit){
            this.kontostand=this.kontostand-betrag;
            System.out.println("Abgehobener Betrag: "+ betrag);
        }
        else if(this.kontostand==this.kreditlimit){
            System.out.println("Keine Abhebung möglich!");
        }
        else{
            System.out.println("Verfügungsrahmen überschritten!");
            System.out.println("der Kontostand liegt bei: " +
this.kontostand + " das Kreditlimit ist bei: "+(-1*
this.kreditlimit));
            System.out.println("Abgehobener Betrag: "+
(this.kontostand - this.kreditlimit));
            this.kontostand = this.kreditlimit;
        }
        return betrag;
    }
}

```

Klasse: Kontoverwaltung

package montag04082025;

```
public class Kontoverwaltung {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Konto k1 =new Konto();
        //einlesen
        k1.setKontonummer("DE48 1234 5678 9101 12");
        k1.setKarte(true);
        k1.setKontoart('G');
        k1.setKontostand(500);
        k1.setKreditlimit(-600);
        // ohne Kapselung k1.kontostand=400;
        //mit kapselung geht das nicht mehr k1.kontostand=100;

        Konto k2 = new Konto();
        k2.setKontonummer("DE52 1235 6544 8798 45");
        k2.setKontoart('S');
        k2.setKarte(false);
        k2.setKontostand(-213);
        k2.setKreditlimit(-15000);

        //auslesen
        System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
"+k1.getKreditlimit()+" €");
        if (k1.isKarte()) {
            System.out.println("Karte wurde ausgegeben!\n");
        }
        else {
            System.out.println("Karte wurde nicht ausgegeben\n");
        }
        System.out.println(k2.getKontonummer()+" Kontostand:
"+k2.getKontostand()+" € Kontoart: " +k2.getKontoart()+" Limit:
"+k2.getKreditlimit()+" €");
        if (k2.isKarte()) {
            System.out.println("Karte wurde ausgegeben!\n");
        }
        else {
            System.out.println("Karte wurde nicht ausgegeben!\n");
        }

        //Einzahlen
        k1.einzahlenBetrag(375.0);
        System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
```

```

"+k1.getKreditlimit()+" €\n");

        //Abheben
        k1.abhebenBetrag(600.0);
        System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
"+k1.getKreditlimit()+" €\n");
        k1.abhebenBetrag(900.0);//über dem limit!
        System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
"+k1.getKreditlimit()+" €\n");
        k1.abhebenBetrag(100);
    }

}

```

Klasse: Kontoverwaltung

```
package montag04082025;
```

```

public class Kontoverwaltung {

    public static void main(String[] args) {
        // TODO Auto-generated method stub

        Konto k1 =new Konto();
        //einlesen
        k1.setKontonummer("DE48 1234 5678 9101 12");
        k1.setKarte(true);
        k1.setKontoart('G');
        k1.setKontostand(500);
        k1.setKreditlimit(-600);
        // ohne Kapselung k1.kontostand=400;

        Konto k2 = new Konto();
        k2.setKontonummer("DE52 1235 6544 8798 45");
        k2.setKontoart('S');
        k2.setKarte(false);
        k2.setKontostand(-213);
        k2.setKreditlimit(-15000);

        //auslesen
        System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
"+k1.getKreditlimit()+" €");
        if (k1.isKarte()) {
            System.out.println("Karte wurde ausgegeben!\n");
        }
        else {

```



```

        System.out.println("Karte wurde nicht ausgegeben\n");
    }
    System.out.println(k2.getKontonummer()+" Kontostand:
"+k2.getKontostand()+" € Kontoart: " +k2.getKontoart()+" Limit:
"+k2.getKreditlimit()+" €");
    if (k2.isKarte()) {
        System.out.println("Karte wurde ausgegeben!\n");
    }
    else {
        System.out.println("Karte wurde nicht ausgegeben!\n");
    }

    //Einzahlen
    k1.einzahlenBetrag(375.0);
    System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
"+k1.getKreditlimit()+" €\n");

    //Abheben
    k1.abhebenBetrag(600.0);
    System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
"+k1.getKreditlimit()+" €\n");
    k1.abhebenBetrag(900.0);//über dem limit!
    System.out.println(k1.getKontonummer()+" Kontostand:
"+k1.getKontostand()+" € Kontoart: " +k1.getKontoart()+" Limit:
"+k1.getKreditlimit()+" €\n");
    k1.abhebenBetrag(100);
}

}

```

Package: simple

Klasse: SecondGui

```

package simple;

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JButton;
import java.awt.BorderLayout;
import javax.swing.JCheckBox;
import javax.swing.JRadioButton;
import java.awt.FlowLayout;
import javax.swing.JLabel;
import javax.swing.JTextArea;
import com.jgoodies.forms.layout.FormLayout;
import com.jgoodies.forms.layout.ColumnSpec;
import com.jgoodies.forms.layout.FormSpecs;

```

```

import com.jgoodies.forms.layout.RowSpec;
import javax.swing.JSpinner;
import javax.swing.JScrollBar;
import javax.swing.JSeparator;
import javax.swing.JList;
import javax.swing.JPasswordField;
import javax.swing.JComboBox;
import javax.swing.JSlider;
import javax.swing.DefaultComboBoxModel;

public class SecondGui {

    private JFrame frame;
    private JPasswordField passwordField;

    /**
     * Launch the application.
     */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    SecondGui window = new SecondGui();
                    window.frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    /**
     * Create the application.
     */
    public SecondGui() {
        initialize();
    }

    /**
     * Initialize the contents of the frame.
     */
    private void initialize() {
        frame = new JFrame();
        frame.setBounds(100, 100, 450, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.getContentPane().setLayout(new FormLayout(new
ColumnSpec[] {
            ColumnSpec.decode("86px"),
            ColumnSpec.decode("33px"),
            FormSpecs.LABEL_COMPONENT_GAP_COLSPEC,

```

```

        ColumnSpec.decode("55px:grow"),
        FormSpecs.LABEL_COMPONENT_GAP_COLSPEC,
        ColumnSpec.decode("5px"),
        FormSpecs.LABEL_COMPONENT_GAP_COLSPEC,
        ColumnSpec.decode("65px:grow"),
        FormSpecs.LABEL_COMPONENT_GAP_COLSPEC,
        ColumnSpec.decode("35px:grow"),
        FormSpecs.LABEL_COMPONENT_GAP_COLSPEC,
        ColumnSpec.decode("45px"),
        FormSpecs.RELATED_GAP_COLSPEC,
        FormSpecs.DEFAULT_COLSPEC,},
new RowSpec[] {
    FormSpecs.RELATED_GAP_ROWSPEC,
    RowSpec.decode("22px"),
    FormSpecs.RELATED_GAP_ROWSPEC,
    FormSpecs.DEFAULT_ROWSPEC,
    FormSpecs.RELATED_GAP_ROWSPEC,
    FormSpecs.DEFAULT_ROWSPEC,
    FormSpecs.RELATED_GAP_ROWSPEC,
    FormSpecs.DEFAULT_ROWSPEC,
    FormSpecs.RELATED_GAP_ROWSPEC,
    RowSpec.decode("default:grow"),});

JLabel lblNewLabel = new JLabel("Zuerst:");
frame.getContentPane().add(lblNewLabel, "2, 2, left, center");

JButton btnNewButton = new JButton("Start");
frame.getContentPane().add(btnNewButton, "4, 2, left, top");

JTextArea textArea = new JTextArea();
frame.getContentPane().add(textArea, "6, 2, left, top");

JCheckBox chckbxNewCheckBox = new JCheckBox("Spenden");
frame.getContentPane().add(chckbxNewCheckBox, "8, 2, left,
top");

JRadioButton rdbtnNewRadioButton = new JRadioButton("Ja");
frame.getContentPane().add(rdbtnNewRadioButton, "10, 2, left,
top");

JRadioButton rdbtnNewRadioButton_1 = new JRadioButton("Nein");
frame.getContentPane().add(rdbtnNewRadioButton_1, "12, 2,
left, top");

JScrollBar scrollBar = new JScrollBar();
frame.getContentPane().add(scrollBar, "1, 4");

JSeparator separator = new JSeparator();
frame.getContentPane().add(separator, "14, 4");

```

```

        JSpinner spinner = new JSpinner();
        frame.getContentPane().add(spinner, "4, 8");

        JLabel lblNewLabel_1 = new JLabel("Passwort:");
        frame.getContentPane().add(lblNewLabel_1, "8, 8, right,
default");

        passwordField = new JPasswordField();
        frame.getContentPane().add(passwordField, "10, 8, fill,
default");

        JScrollBar scrollBar_1 = new JScrollBar();
        frame.getContentPane().add(scrollBar_1, "1, 10");

        JList list = new JList();
        frame.getContentPane().add(list, "4, 10, fill, fill");

        JComboBox comboBox = new JComboBox();
        comboBox.setModel(new DefaultComboBoxModel(new String[]
{"Start", "Mitte", "Ende"}));
        frame.getContentPane().add(comboBox, "8, 10, fill, default");

        JSlider slider = new JSlider();
        frame.getContentPane().add(slider, "12, 10");
    }
}

```

Klasse: ThirdGUI

```

package simple;

import java.awt.EventQueue;

import javax.swing.JFrame;
import javax.swing.JLabel;
import java.awt.BorderLayout;
import javax.swing.JButton;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;

public class ThirdGUI {

    private JFrame frame;
    private JTextField textField;
    private JTextField textField_1;

    /**
     * Launch the application.
     */
}

```

```

    */
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    ThirdGUI window = new ThirdGUI();
                    window.frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }

    /**
     * Create the application.
     */
    public ThirdGUI() {
        initialize();
    }

    /**
     * Initialize the contents of the frame.
     */
    private void initialize() {
        frame = new JFrame();
        frame.setResizable(false);
        frame.setBounds(100, 100, 450, 300);
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.getContentPane().setLayout(null);

        JButton btnNewButton = new JButton("Klick mich");

        btnNewButton.setBounds(0, 242, 436, 21);
        frame.getContentPane().add(btnNewButton);

        JLabel lblNewLabel = new JLabel("Vorname");
        lblNewLabel.setBounds(38, 42, 76, 12);
        frame.getContentPane().add(lblNewLabel);

        JLabel lblNewLabel_1 = new JLabel("Nachname");
        lblNewLabel_1.setBounds(38, 94, 76, 12);
        frame.getContentPane().add(lblNewLabel_1);

        textField = new JTextField();
        textField.setBounds(163, 39, 96, 18);
        frame.getContentPane().add(textField);
        textField.setColumns(10);

        textField_1 = new JTextField();
    }

```

```

textField_1.setBounds(163, 91, 96, 18);
frame.getContentPane().add(textField_1);
textField_1.setColumns(10);

JLabel lblNewLabel_2 = new JLabel("Ausgabe");
lblNewLabel_2.setBounds(38, 160, 370, 12);
frame.getContentPane().add(lblNewLabel_2);

btnNewButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        //hier kommt der code, der beim klicken des buttons
ausgeführt wird
        System.out.println("Klick");
        String eingabel=textField.getText();
        String eingabe2=textField_1.getText();

        lblNewLabel_2.setText(eingabel+" "+eingabe2);

    }
});

}
}

```

Package: test

Klasse: ModelTest

```

package test;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Date;

import model.Bestellung;
import model.Kunde;

public class ModelTest {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        //Hier möchte uch meine Modelklassen ausprobieren

        Kunde k= new Kunde(1,"Gushti","Gerdsen");

        System.out.println("ID: "+k.getId());
        System.out.println("Nachname: "+k.getNachname());
        System.out.println("Vorname: "+k.getVorname());
        k.setVorname("Luigi");
        k.setNachname("Salmonelle");
    }
}

```

```

        System.out.println("Nachname: "+k.getNachname());
        System.out.println("Vorname: "+k.getVorname());

        Bestellung b= new Bestellung(1,k,22.56);
        ArrayList<Bestellung> al=new ArrayList<Bestellung>();
        al.add(b);
        k.setBestellungen(al);
//      b.setKunde(k.setBestellungen(null).add(b));
//      k.getBestellungen().add(b);

        System.out.println("ID: "+b.getId());
        k.setAdresse("Hinterm Tor 2,79115 Freiburg");
        k.setKundenart("A");

        System.out.println("Kunde: "+b.getKunde().toString());
        b.setDatum(LocalDate.now());
        System.out.println("Datum: "+b.getDatum());
        System.out.printf("Umsatz: %.2f€\n",b.getUmsatz());
        System.out.println(b);
        System.out.println(b.getKunde());

```

```

    }

```

```

}

```

Klasse: ViewTest

```

package test;

```

```

import model.Kunde;

```

```

import view.View;

```

```

public class ViewTest {

```

```

    public static void main(String[] args) {

```

```

        View v = new View();

```

```

        int eingabe =v.leseKundennummer();

```

```

        System.out.println("Eingabe: "+eingabe);

```

```

        Kunde k = new Kunde(2,"Helga","Brauer");

```

```

        v.zeigeKunde(k);

```

```

    }

```

```

}

```

Package: util

Klasse: ShipmentCostCalculator

```
package util;

import interfaces.Insurable;
import model.Shipment;

public class ShipmentCostCalculator {

    public static double calculateCost(Shipment[] shipments) {

        double gesamtkosten = 0.0;
        for (Shipment s : shipments) {

            if (s instanceof Insurable insurable)
                gesamtkosten += insurable.calculateInsurance();

            gesamtkosten += s.shippingCost();

        }

        return gesamtkosten;

    }

}
```

Klasse: ShipmentFactory

```
package util;

import exceptions.InvalidShipmentDataException;
import exceptions.UnknownShipmentTypeException;
import model.ExpressParcel;
import model.Freight;
import model.Shipment;
import model.StandardParcel;

public class ShipmentFactory {

    public static Shipment createShipment(String type, double weight,
double distance, double goodsValue)
        throws UnknownShipmentTypeException,
InvalidShipmentDataException {
        switch (type.toLowerCase()) {
            case "standard":
                return new StandardParcel(weight, distance);
            case "express":
                return new ExpressParcel(weight, distance, goodsValue);
        }
    }

}
```



```

        case "freight":
            return new Freight(weight, distance, goodsValue);
        default:
            throw new UnknownShipmentTypeException("Unbekannter Typ: "
+ type);
    }
}

```

Klasse: ShipmentFactory

```

package util;

import exceptions.InvalidShipmentDataException;
import exceptions.UnknownShipmentTypeException;
import model.ExpressParcel;
import model.Freight;
import model.Shipment;
import model.StandardParcel;

public class ShipmentFactory {

    public static Shipment createShipment(String type, double weight,
double distance, double goodsValue)
        throws UnknownShipmentTypeException,
InvalidShipmentDataException {
        switch (type.toLowerCase()) {
            case "standard":
                return new StandardParcel(weight, distance);
            case "express":
                return new ExpressParcel(weight, distance, goodsValue);
            case "freight":
                return new Freight(weight, distance, goodsValue);
            default:
                throw new UnknownShipmentTypeException("Unbekannter Typ: "
+ type);
        }
    }
}

```

Klasse: ShipmentFilter

```

package util;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;

import model.Shipment;

```

```

public class ShipmentFilter {

    public static List<Shipment> filter(List<Shipment> sh,
    Predicate<Shipment> t){
        List<Shipment> result = new ArrayList<Shipment>();

        for(Shipment s: sh) {
            if(t.test(s))
                result.add(s);
        }
        return result;
    }
}

```

Klasse: ShipmentFilter

```

package util;

import java.util.ArrayList;
import java.util.List;
import java.util.function.Predicate;

import model.Shipment;

public class ShipmentFilter {

    public static List<Shipment> filter(List<Shipment> sh,
    Predicate<Shipment> t){
        List<Shipment> result = new ArrayList<Shipment>();

        for(Shipment s: sh) {
            if(t.test(s))
                result.add(s);
        }
        return result;
    }
}

```

Klasse: SortSelection

```

package util;

import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

import comparator.DistanceComparatorAsc;
import comparator.DistanceComparatorDesc;

```

```

import comparator.ShippingCostComparatorAsc;
import comparator.ShippingCostComparatorDesc;
import comparator.WeightComparatorAsc;
import comparator.WeightComparatorDesc;
import model.Shipment;

public class SortSelection {
    public static String auswahltxt =
"=====\\
n=====Sortierungsauswahl=====\\n"
        + "=====\\n1.
Sortieren nach Gewicht(aufsteigend)\\n"
        + "2. Sortieren nach Versandkosten(aufsteigend)\\n3.
Sortieren nach Entfernung(aufsteigend)\\
n=====\\n"
        + "4. Sortieren nach Gewicht(absteigend)\\n5. Sortieren
nach Versandkosten(absteigend)\\n"
        + "6. Sortieren nach Entfernung(absteigend)\\
n=====\\n"
        ;

    public static void selectSortAlgorithm(Shipment[] sh) {

        switch (readAuswahl()) {
            case 1:
                Arrays.sort(sh, new WeightComparatorAsc());
                System.out.println("=====Sortieren nach
Gewicht=====");
                break;
            case 2:
                Arrays.sort(sh, new ShippingCostComparatorAsc());
                System.out.println("=====Sortieren nach
Versandkosten=====");
                break;
            case 3:
                Arrays.sort(sh, new DistanceComparatorAsc());
                System.out.println("=====Sortieren nach
Entfernung=====");
                break;
            case 4:
                Arrays.sort(sh, new WeightComparatorDesc());
                System.out.println("=====Sortieren nach
Gewicht=====");
                break;
            case 5:
                Arrays.sort(sh, new ShippingCostComparatorDesc());
                System.out.println("=====Sortieren nach
Versandkosten=====");
                break;
            case 6:

```

```

        Arrays.sort(sh, new DistanceComparatorDesc());
        System.out.println("====Sortieren nach
Entfernung====");
        break;
    default:
        System.out.println("Fehlerhafte Eingabe!");
        selectSortAlgorithm(sh);
    }
}

public static void selectSortAlgorithmList(List<Shipment> shList)
{
    switch (readAuswahl()) {
        case 1:
            Collections.sort(shList, new WeightComparatorAsc());
            System.out.println("====Sortieren nach
Gewicht====");
            break;
        case 2:
            Collections.sort(shList, new ShippingCostComparatorAsc());
            System.out.println("====Sortieren nach
Versandkosten====");
            break;
        case 3:
            Collections.sort(shList, new DistanceComparatorAsc());
            System.out.println("====Sortieren nach
Entfernung====");
            break;
        case 4:
            Collections.sort(shList, new WeightComparatorDesc());
            System.out.println("====Sortieren nach
Gewicht====");
            break;
        case 5:
            Collections.sort(shList, new
ShippingCostComparatorDesc());
            System.out.println("====Sortieren nach
Versandkosten====");
            break;
        case 6:
            Collections.sort(shList, new DistanceComparatorDesc());
            System.out.println("====Sortieren nach
Entfernung====");
            break;
        default:
            selectSortAlgorithmList(shList);
            assert (false);// defaultzweig sollte nie erreicht
            werden,wenn doch gibt es ein programmabbruch
    }
}

```

```

        //nützliche Schnellbefehle
//      Collections.max(shList);
//      Collections.max(shList, new DistanceComparatorDesc());
//      Collections.reverseOrder(new DistanceComparatorDesc()); //gibt
einen Comparator zurück

    }

    private static int readAuswahl() {
        System.out.println(auswahltxt);
        Scanner sc = new Scanner(System.in);
        try {
            return Integer.parseInt(sc.next());
        } catch (NumberFormatException e) {
            System.out.println("Falsche Eingabe! Versuche erneut!");
            return readAuswahl();
        } finally {
            sc.close();
        }
    }
}

```

Klasse: SortSelection

```

package util;

import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import java.util.Scanner;

import comparator.DistanceComparatorAsc;
import comparator.DistanceComparatorDesc;
import comparator.ShippingCostComparatorAsc;
import comparator.ShippingCostComparatorDesc;
import comparator.WeightComparatorAsc;
import comparator.WeightComparatorDesc;
import model.Shipment;

public class SortSelection {
    public static String auswahltxt =
"=====\\
n=====Sortierungsauswahl=====\\n"
        + "=====\\n1.
Sortieren nach Gewicht(aufsteigend)\\n"
        + "2. Sortieren nach Versandkosten(aufsteigend)\\n3.
Sortieren nach Entfernung(aufsteigend)\\
n=====\\n"
        + "4. Sortieren nach Gewicht(absteigend)\\n5. Sortieren

```

```
nach Versandkosten(absteigend)\n"
    + "6. Sortieren nach Entfernung(absteigend)\n"
n=====
    ;
```

```
    public static void selectSortAlgorithm(Shipment[] sh) {
        switch (readAuswahl()) {
            case 1:
                Arrays.sort(sh, new WeightComparatorAsc());
                System.out.println("=====Sortieren nach
Gewicht=====");
                break;
            case 2:
                Arrays.sort(sh, new ShippingCostComparatorAsc());
                System.out.println("=====Sortieren nach
Versandkosten=====");
                break;
            case 3:
                Arrays.sort(sh, new DistanceComparatorAsc());
                System.out.println("=====Sortieren nach
Entfernung=====");
                break;
            case 4:
                Arrays.sort(sh, new WeightComparatorDesc());
                System.out.println("=====Sortieren nach
Gewicht=====");
                break;
            case 5:
                Arrays.sort(sh, new ShippingCostComparatorDesc());
                System.out.println("=====Sortieren nach
Versandkosten=====");
                break;
            case 6:
                Arrays.sort(sh, new DistanceComparatorDesc());
                System.out.println("=====Sortieren nach
Entfernung=====");
                break;
            default:
                System.out.println("Fehlerhafte Eingabe!");
                selectSortAlgorithm(sh);
        }
    }

    public static void selectSortAlgorithmList(List<Shipment> shList)
    {
        switch (readAuswahl()) {
            case 1:
                Collections.sort(shList, new WeightComparatorAsc());
                System.out.println("=====Sortieren nach
```

```

Gewicht=====");
        break;
        case 2:
            Collections.sort(shList, new ShippingCostComparatorAsc());
            System.out.println("=====Sortieren nach
Versandkosten=====");
            break;
        case 3:
            Collections.sort(shList, new DistanceComparatorAsc());
            System.out.println("=====Sortieren nach
Entfernung=====");
            break;
        case 4:
            Collections.sort(shList, new WeightComparatorDesc());
            System.out.println("=====Sortieren nach
Gewicht=====");
            break;
        case 5:
            Collections.sort(shList, new
ShippingCostComparatorDesc());
            System.out.println("=====Sortieren nach
Versandkosten=====");
            break;
        case 6:
            Collections.sort(shList, new DistanceComparatorDesc());
            System.out.println("=====Sortieren nach
Entfernung=====");
            break;
        default:
            selectSortAlgorithmList(shList);
            assert (false);// defaultzweig sollte nie erreicht
werden,wenn doch gibt es ein programmabbruch
    }

    //nützliche Schnellbefehle
//    Collections.max(shList);
//    Collections.max(shList, new DistanceComparatorDesc());
//    Collections.reverseOrder(new DistanceComparatorDesc());//gibt
einen Comparator zurück

}

```

```

private static int readAuswahl() {
    System.out.println(auswahltxt);
    Scanner sc = new Scanner(System.in);
    try {
        return Integer.parseInt(sc.next());
    } catch (NumberFormatException e) {
        System.out.println("Falsche Eingabe! Versuche erneut!");
    }
}

```

```

        return readAuswahl();
    } finally {
        sc.close();
    }
}
}

```

Klasse: Statistic

```
package util;
```

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import comparator.ShippingCostComparatorAsc;
import comparator.WeightComparatorAsc;
import model.Shipment;

```

```
public class Statistic {
```

```

    public static Shipment findMostExpensive(Shipment [] sh) {
        Shipment[] copy = Arrays.copyOf(sh,sh.length);
        Arrays.sort(copy, new ShippingCostComparatorAsc());
        return copy[copy.length-1];
    }

```

```

    public static Shipment findMostExpensiveList(List<Shipment>
shList) {
        List<Shipment> b = new ArrayList<Shipment>(shList);
        Collections.sort(b,new ShippingCostComparatorAsc());
        return b.get(b.size()-1);
    }

```

```

    public static Shipment findHeviest(Shipment [] sh) {
        Shipment[] copy = Arrays.copyOf(sh,sh.length);
        Arrays.sort(copy, new WeightComparatorAsc());
        return copy[copy.length-1];
    }

```

```

    public static Shipment findMostHeviestList(List<Shipment> shList)
{
        List<Shipment> b = new ArrayList<Shipment>(shList);
        Collections.sort(b,new WeightComparatorAsc());
        return b.get(b.size()-1);
    }

```

```

    public static double calaulateAvgShippingCost(Shipment [] sh) {

        if(sh.length==0)

```



```

        return 0.0;
        double sum=0.0;
        for(Shipment s: sh)
            sum+=s.shippingCost();
        return sum/sh.length;
    }
    public static double calculateAvgShippingCostList(List<Shipment>
shList) {
        if (shList.size()==0)
            return 0.0;
        double sum=0.0;
        for(Shipment s: shList)
            sum+=s.shippingCost();
        return sum/shList.size();
    }
}

```

Klasse: Statistic

```
package util;
```

```

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;
import comparator.ShippingCostComparatorAsc;
import comparator.WeightComparatorAsc;
import model.Shipment;

```

```

public class Statistic {

    public static Shipment findMostExpensive(Shipment [] sh) {
        Shipment[] copy = Arrays.copyOf(sh,sh.length);
        Arrays.sort(copy, new ShippingCostComparatorAsc());
        return copy[copy.length-1];
    }
    public static Shipment findMostExpensiveList(List<Shipment>
shList) {
        List<Shipment> b = new ArrayList<Shipment>(shList);
        Collections.sort(b,new ShippingCostComparatorAsc());
        return b.get(b.size()-1);
    }

    public static Shipment findHeviest(Shipment [] sh) {

```

```

        Shipment[] copy = Arrays.copyOf(sh, sh.length);
        Arrays.sort(copy, new WeightComparatorAsc());
        return copy[copy.length-1];
    }
    public static Shipment findMostHeviestList(List<Shipment> shList)
{
    List<Shipment> b = new ArrayList<Shipment>(shList);
    Collections.sort(b, new WeightComparatorAsc());
    return b.get(b.size()-1);
}

    public static double calaulateAvgShippingCost(Shipment [] sh) {
        if(sh.length==0)
            return 0.0;
        double sum=0.0;
        for(Shipment s: sh)
            sum+=s.shippingCost();
        return sum/sh.length;
    }
    public static double calaulateAvgShippingCostList(List<Shipment>
shList) {
        if (shList.size()==0)
            return 0.0;
        double sum=0.0;
        for(Shipment s: shList)
            sum+=s.shippingCost();
        return sum/shList.size();
    }
}

```

Klasse: shipmentOverviewCreator

```
package util;
```

```
import java.util.List;
```

```
import interfaces.Insurable;
```

```
import interfaces.Trackable;
```

```
import model.Shipment;
```

```
public class shipmentOverviewCreator {
```

```
    public static String createOverview(Shipment[] shipments) {
```

```

        String overview = "=".repeat(40) + "\n";
        double gesamtkosten = 0.0;
        for (Shipment s : shipments) {
            overview += s.toString() + "\n";

            if (s instanceof Insurable insurable) {
                gesamtkosten += insurable.calculateInsurance();
                overview += String.format("Versicherung: %25.2f",
insurable.calculateInsurance()) + "€\n";
            } else {
                overview += "keine Versicherung verfügbar\n";
            }

            if (s instanceof Trackable track) {
                overview += String.format("Tackingcode: %27s\n",
track.getTrackingCode());
            } else {
                overview += "kein Trackingcode verfügbar\n";
            }

            overview += String.format("Versandkosten: %24.2f",
s.shippingCost()) + "€\n";
            gesamtkosten += s.shippingCost();
            overview += "=".repeat(40) + "\n";
        }

        overview += String.format("Gesamtkosten: %25.2f",
gesamtkosten) + "€\n";

        return overview;
    }

    public static String createOverviewFromList(List<Shipment> shList)
    {
        Shipment[] temp= new Shipment[shList.size()];
        shList.toArray(temp);
        return createOverview(temp);
    }
}

```

Klasse: shipmentOverviewCreator

```
package util;
```

```
import java.util.List;
```

```
import interfaces.Insurable;
```

```
import interfaces.Trackable;
```

```

import model.Shipment;

public class shipmentOverviewCreator {

    public static String createOverview(Shipment[] shipments) {

        String overview = "=".repeat(40) + "\n";
        double gesamtkosten = 0.0;
        for (Shipment s : shipments) {
            overview += s.toString() + "\n";

            if (s instanceof Insurable insurable) {
                gesamtkosten += insurable.calculateInsurance();
                overview += String.format("Versicherung: %25.2f",
insurable.calculateInsurance()) + "€\n";
            } else {
                overview += "keine Versicherung verfügbar\n";
            }

            if (s instanceof Trackable track) {
                overview += String.format("Tackingcode: %27s\n",
track.getTrackingCode());
            } else {
                overview += "kein Trackingcode verfügbar\n";
            }

            overview += String.format("Versandkosten: %24.2f",
s.shippingCost()) + "€\n";
            gesamtkosten += s.shippingCost();
            overview += "=".repeat(40) + "\n";
        }

        overview += String.format("Gesamtkosten: %25.2f",
gesamtkosten) + "€\n";

        return overview;
    }

    public static String createOverviewFromList(List<Shipment> shList)
{
    Shipment[] temp= new Shipment[shList.size()];
    shList.toArray(temp);
    return createOverview(temp);
}

}

```

Package: view

Klasse: View

```
package view;

import java.util.InputMismatchException;
import java.util.Scanner;

import model.Kunde;

public class View {
    /**
     * Wir wollen nur eine Konsolenein- und -ausgabe sehen
     * Auf der Konsole soll eine Konsolenummer eingelesen werden
     * ein Kunde ausgegeben werden
     */

    private Scanner sc ;

    public View() {
        sc = new Scanner(System.in);
    }

    public int leseKundennummer() {
        try {
            System.out.print("Geben Sie die Kundennummer ein: ");
            int nummer=sc.nextInt();
            return nummer;
        } catch(InputMismatchException e) {
            System.out.println("Falsche Eingabe!");
            // Um den Scannerpuffer leer zu machen
            sc.next();
        }
        return -1;
    }

    public void zeigeKunde(Kunde k) {
        System.out.println(k);
    }
}
```